Embodied AI

Comprehensive Lecture Notes

Vittorio Cava

A.Y. 2026/27

Contents

I Lecture 2 **2**

1 Representations for 3D *(slides slides 2–4)* **3**

2 Notation *(slides slide 5)* **4**

3 Manifolds *(slides slide 6)* **4**

4 Defining a Surface *(slides slide 7)* **5**

5 Meanwhile in 3D-land... *(slides slides 8–9)* **5**

6 Depth Maps *(slides slides 10–14)* **6**

6.1 Depth maps *(slides slides 10–12)* 6

6.2 Depth Maps and Normal Maps *(slides slide 13)* 6

6.3 Depth Maps — Representing the Visible *(slides slide 14)* 6

7 Point Clouds *(slides slides 15–21)* **7**

7.1 Point Clouds *(slides slides 15–18)* 7

7.2 Oriented Point Clouds *(slides slide 19)* 8

7.3 No explicit connectivity *(slides slides 20–21)* 8

8 Meshes *(slides slides 22–34)* **9**

8.1 Meshes: vertices and faces *(slides slides 22–25)* 9

8.2 Triangular Meshes *(slides slides 26–31)* 10

8.3 Triangular Meshes — Geometry vs Connectivity *(slides slides 32–34)* 12

9 Parametric Surfaces *(slides slides 35–41)* **14**

9.1 Parametric surfaces in 2D *(slides slide 35)* 14

9.2 Parametric surfaces *(slides slides 36–40)* 14

9.3 Parametric Surfaces — Pros and Cons *(slides slide 41)* 16

10 Surface Representations *(slides slide 42)* **16**

11 Volume Representations *(slides slides 43–46)* **17**
11.1 Voxelized 3D *(slides slides 44–46)* . . . 17

12 Implicit Surfaces *(slides slides 47–62)* **18**
12.1 Implicit surfaces *(slides slide 47)* . . . 18
12.2 Generate vs Test *(slides slide 48)* . . . 19
12.3 Implicit Surfaces: Examples *(slides slide 49)* . . . 19
12.4 When a Field Defines a Surface *(slides slide 50)* . . . 20
12.5 Normals for Free *(slides slide 51)* . . . 21
12.6 Boolean Operations *(slides slide 52)* . . . 21
12.7 Many Functions, One Surface *(slides slide 53)* . . . 22
12.8 Signed Distance Function *(slides slides 54–58)* . . . 22
12.9 Storing a Field on a Grid *(slides slide 59)* . . . 24
12.10 Storing a Field in a Network *(slides slide 60)* . . . 24
12.11 From Implicit Surface to Mesh *(slides slide 61)* . . . 26
12.12 Implicit Surfaces: Summary *(slides slide 62)* . . . 27

13 Boundary vs Volumetric Representations *(slides slide 63)* **27**

14 Density Fields *(slides slides 64–65)* **27**
14.1 Density fields *(slides slide 64)* . . . 27
14.2 Primitive-based Density Fields *(slides slide 65)* . . . 28

15 So? *(slides slide 66)* **28**

16 And many more... *(slides slides 67–71)* **29**
16.1 Octrees *(slides slide 68)* . . . 29
16.2 Layered Depth Images *(slides slide 69)* . . . 30
16.3 Multi-plane Images *(slides slide 70)* . . . 30
16.4 CAD Models and CSG Trees *(slides slide 71)* . . . 31

17 Summary *(slides slides 72–74)* **31**

II Lecture 3 **33**

1 From Image Formation to Measurement *(slides 1–3)* **34**

2 From Representations to Measurements *(slides 4–7)* **35**
2.1 Recovered and Ambiguous Quantities *(slides 5)* . . . 35
2.2 The Back-Projected Ray *(slides 6)* . . . 35
2.3 Scale-Depth Ambiguity *(slides 7)* . . . 36

3 Three Ways to Pay for Depth *(slides 8–12)* **38**
3.1 Two Measurement Principles *(slides 9)* . . . 38
3.2 Triangulation From Two Views *(slides 10)* . . . 38
3.3 The Three Currencies *(slides 11)* . . . 39
3.4 The Single-Image Claim *(slides 12)* . . . 39

4 Inside a LiDAR *(slides 13–17)* **40**
4.1 Pulsed Time of Flight *(slides 14)* . . . 40
4.2 Structured Sparsity *(slides 15)* . . . 40
4.3 Limitations of a LiDAR Point Cloud *(slides 16)* . . . 42
4.4 Two Error Laws *(slides 17)* . . . 43

5 From Point Cloud to Depth Image *(slides 18–19)* **43**
5.1 Projecting a Cloud Into the Image *(slides 19)* . . . 44

6 From Recovery to Prediction *(slides 20–22)* **45**
6.1 Monocular Cues in Human Vision *(slides 21)* . . . 46
6.2 Depth Prediction as a Prior *(slides 22)* . . . 46

7 Cues in Closed Form *(slides 23–26)* **47**
7.1 Vanishing Points *(slides 24)* . . . 47
7.2 Apparent Size *(slides 25)* . . . 48
7.3 Vertical Position *(slides 26)* . . . 49

8 Supervised Single-View Depth *(slides 27–35)* **49**
8.1 The Supervised Setup *(slides 28)* . . . 50
8.2 Three Datasets *(slides 29)* . . . 50
8.3 The Scale-Invariant Loss *(slides 30)* . . . 50
8.4 Output Spaces *(slides 31)* . . . 52
8.5 Affine Invariance in Disparity and in Depth *(slides 32)* . . . 52
8.6 MiDaS: Mixing the Datasets *(slides 33)* . . . 53
8.7 Depth Anything V2 *(slides 34)* . . . 54
8.8 Failure on Anamorphic Art *(slides 35)* . . . 55

9 Cues Used by the Network *(slides 36–39)* **56**
9.1 The Pasted-Car Experiment *(slides 37)* . . . 56
9.2 Pitch and Roll *(slides 38)* . . . 56
9.3 Dependence on the Training Camera *(slides 39)* . . . 58

Next: Self-Supervision and a Generative Prior *(slide 40)* **58**

Overview: Part I in One Slide *(slides 1–2)* **59**

10 Supervision without Ground Truth *(slides 3–6)* **59**

10.1 View Synthesis and the Backward Warp *(slides 4)* . . . 59
10.2 Image Reconstruction as Supervision *(slides 5)* . . . 60
10.3 Where the Loss Is Blind *(slides 6)* . . . 61

11 Diffusion, From Scratch *(slides 7–22)* **62**
11.1 Supervised Learning *(slides 8)* . . . 62
11.2 Generative Learning *(slides 9)* . . . 62
11.3 Many Outputs for One Input – Text to Image *(slides 10)* . . . 63
11.4 The Forward and Reverse Process *(slides 11)* . . . 63
11.5 Training by Noise Prediction *(slides 12)* . . . 64
11.6 The Spiral Toy Model *(slides 13)* . . . 65
11.7 Training the Toy Model *(slides 14)* . . . 65
11.8 The Ideal Denoiser *(slides 15)* . . . 66
11.9 The Smoothed Distance Function *(slides 16)* . . . 67
11.10 Sampling From a Diffusion Model – Why iterative? *(slides 17)* . . . 68
11.11 The DDIM Sampler *(slides 18)* . . . 68
11.12 Probabilistic Sampling *(slides 19)* . . . 69
11.13 Three Uses of a Prior *(slides 20)* . . . 69
11.14 The Autoencoder and Latent Diffusion *(slides 21)* . . . 69
11.15 Conditioning and Guidance *(slides 22)* . . . 70

12 Marigold *(slides 23–31)* **70**
12.1 The Marigold Method *(slides 24)* . . . 70
12.2 From the Spiral to Marigold *(slides 25)* . . . 71
12.3 The Source of Affine Invariance *(slides 26)* . . . 71
12.4 Conditioning on the Image *(slides 27)* . . . 72
12.5 Training Marigold *(slides 28)* . . . 73
12.6 Inference *(slides 29)* . . . 73
12.7 Ensembling Over Samples *(slides 30)* . . . 74
12.8 Limitations of Marigold *(slides 31)* . . . 74

13 Back to 3D *(slides 32–34)* **74**
13.1 Effect of Scale and Shift Errors *(slides 33)* . . . 74
13.2 Sources of Scale and Shift *(slides 34)* . . . 75

14 Marigold-DC *(slides 35–38)* **76**
14.1 The Depth Completion Task *(slides 36)* . . . 76
14.2 Steering the Loop *(slides 37)* . . . 77
14.3 Nothing Retrained *(slides 38)* . . . 78

Summary *(slides 39–40)* **80**

A Supplementary derivations (not on the slides) **82**

III Lecture 4 **83**

Agenda and Learning Objectives (slides 1–2) **84**

1 Two-View Geometry *(slides 3–13)* **84**
1.1 Depth Cues and Binocular Stereopsis *(slides 4)* . . . 84
1.2 Two-View Stereo Matching *(slides 5)* . . . 84
1.3 The Classical Stereo Pipeline *(slides 6)* . . . 85
1.4 Epipolar Geometry *(slides 7)* . . . 85
1.5 Image Rectification *(slides 8)* . . . 86
1.6 Rectified Epipolar Constraint *(slides 9)* . . . 87
1.7 Calculating the Rectifying Rotation Matrix *(slides 10)* . . . 87
1.8 Rectified Pair and Disparity *(slides 11)* . . . 88
1.9 Disparity to Depth *(slides 12)* . . . 88
1.10 Worked Example: Baseline and Depth Error *(slides 13)* . . . 89

2 Block Matching *(slides 14–21)* **90**
2.1 Correspondence Ambiguity *(slides 15)* . . . 90
2.2 Similarity Metrics *(slides 16)* . . . 91
2.3 Block Matching *(slides 17)* . . . 92
2.4 Block Matching: Half Occlusions *(slides 18)* . . . 93
2.5 Block Matching: Assumption Violations *(slides 19)* . . . 94
2.6 Effect of Window Size *(slides 20)* . . . 94
2.7 Left-Right Consistency Test *(slides 21)* . . . 95

3 Siamese Networks *(slides 22–28)* **96**
3.1 Siamese Networks for Stereo Matching *(slides 23)* . . . 96
3.2 Siamese Network Architecture *(slides 24)* . . . 97
3.3 Training Triplets *(slides 25)* . . . 98
3.4 Hinge Loss *(slides 26)* . . . 99
3.5 Comparisons *(slides 27)* . . . 100
3.6 Similarity Constraint: Failure Cases *(slides 28)* . . . 100

4 End-to-End Learning *(slides 29–31)* **101**
4.1 DispNet and Synthetic Data *(slides 30)* . . . 101
4.2 GC-Net *(slides 31)* . . . 102

5 Stereo Without Calibration *(slides 32–45)* **103**
5.1 The Cost of Calibration *(slides 33)* . . . 103
5.2 Limits of Rectification and Local Matching *(slides 34)* . . . 104

5.3 Pointmaps *(slides 35)* . . . 104
5.4 The DUSt3R Architecture *(slides 36)* . . . 105
5.5 MASt3R Network Output *(slides 37)* . . . 106
5.6 Training Objective and Metric Scale *(slides 38)* . . . 107
5.7 The Matching Head *(slides 39)* . . . 108
5.8 InfoNCE and the Hinge Loss *(slides 40)* . . . 108
5.9 Effect of 3D Grounding *(slides 41)* . . . 109
5.10 Reciprocal Matching *(slides 42)* . . . 109
5.11 MASt3R Training Details *(slides 43)* . . . 110
5.12 Results *(slides 44)* . . . 110
5.13 The Pipeline Revisited *(slides 45)* . . . 111

Summary **112**
Comparison of the methods covered . . . 112
Key take-aways . . . 112

IV Lecture 5 **114**

Agenda and Learning Objectives *(slides 1–2)* **115**

1 Camera Calibration *(slides 3–7)* **115**
1.1 What calibration is and how it is done *(slides 4–7)* . . . 115

2 Feature Detection and Description *(slides 8–20)* **121**
2.1 Point Features *(slides 9–11)* . . . 121
2.2 Scale Invariant Feature Transform (SIFT) *(slides 12–15)* . . . 122
2.3 Matching features *(slides 14–15)* . . . 123
2.4 SuperPoint: Learned Interest Points and Descriptors *(slides 16–20)* . . . 125

3 Two-Frame Structure-from-Motion *(slides 21–35)* **129**
3.1 Epipolar Geometry *(slides 22–23)* . . . 129
3.2 Structure of the Essential Matrix *(slides 24)* . . . 131
3.3 Estimating the Essential Matrix: the 8-point algorithm *(slides 25)* . . . 131
3.4 Decomposing the Essential Matrix *(slides 26)* . . . 132
3.5 Geometry of the Four Solutions *(slides 27)* . . . 133
3.6 Disambiguation by Cheirality *(slides 28)* . . . 134
3.7 Estimating the Epipolar Geometry with Unknown Intrinsics *(slides 29–30)* . . . 135
3.8 Triangulation *(slides 31–35)* . . . 136

4 Bundle Adjustment *(slides 36–54)* **138**
4.1 Bundle Adjustment *(slides 37–39)* . . . 138
4.2 Challenges of Bundle Adjustment *(slides 40)* . . . 139

4.3 Bundle Adjustment as Nonlinear Least Squares *(slides 41)* . . . 139
4.4 Updates in the Tangent Space *(slides 42)* . . . 140
4.5 The Gauss–Newton Algorithm *(slides 43)* . . . 141
4.6 The Schur Complement *(slides 44)* . . . 141
4.7 Incremental Structure-from-Motion *(slides 45–51)* . . . 143
4.8 Building Rome in a Day, COLMAP SfM and COLMAP MVS *(slides 52–54)* . . . 145

5 Feed-Forward Structure-from-Motion *(slides 55–68)* 147
5.1 Limits of Incremental Structure-from-Motion *(slides 56)* . . . 147
5.2 From Learned Components to Learned Reconstruction *(slides 57)* . . . 148
5.3 Visual Geometry Grounded Transformer (VGGT) *(slides 58)* . . . 148
5.4 Problem Definition *(slides 59)* . . . 149
5.5 Architecture: Alternating Attention *(slides 60)* . . . 150
5.6 Prediction Heads and Tracking *(slides 61)* . . . 150
5.7 Training Objective and Normalization *(slides 62)* . . . 151
5.8 Results: Camera Pose Estimation *(slides 63)* . . . 151
5.9 Results: Dense Geometry *(slides 64)* . . . 152
5.10 Results: Matching and Tracking *(slides 65)* . . . 153
5.11 Bundle Adjustment from VGGT Predictions *(slides 66)* . . . 154
5.12 Limitations *(slides 67)* . . . 154
5.13 Comparison of Incremental and Feed-Forward SfM *(slides 68)* . . . 155

Summary *(slide 69)* 156

V Lecture 6 158

1 The SLAM Problem *(slides 3–6)* 158
1.1 The SLAM Problem *(slide 3)* . . . 158
1.2 Cameras for SLAM *(slide 4)* . . . 159
1.3 SLAM and Structure-from-Motion *(slide 5)* . . . 160
1.4 Running Example: VBR Campus *(slide 6)* . . . 160

2 Filtering: From the Bayes Filter to EKF-SLAM *(slides 7–13)* 161
2.1 SLAM as a Graphical Model *(slide 8)* . . . 161
2.2 The Bayes Filter *(slide 9)* . . . 162
2.3 Kalman Filter and Extended Kalman Filter *(slide 10)* . . . 162
2.4 The EKF-SLAM State *(slide 11)* . . . 163
2.5 Example: MonoSLAM *(slide 12)* . . . 164
2.6 Limits of Filtering *(slide 13)* . . . 165

3 Graph-Based SLAM: Factor Graphs and Least Squares *(slides 14–22)* 166
3.1 Graph-Based SLAM *(slide 15)* . . . 166

3.2 Factor Graphs *(slide 16)* . . . 167
3.3 Visual SLAM as a Factor Graph *(slide 17)* . . . 167
3.4 The Pose Graph *(slide 18)* . . . 168
3.5 From Factors to Least Squares *(slide 19)* . . . 168
3.6 Error and Information Matrix of an Edge *(slide 20)* . . . 168
3.7 Gauss-Newton on the Pose Graph *(slide 21)* . . . 169
3.8 Sparsity of the Normal Equations *(slide 22)* . . . 169

4 The Visual SLAM Pipeline: ORB-SLAM *(slides 23–32)* **170**
4.1 Pipeline Overview *(slide 24)* . . . 170
4.2 ORB Features and Matching *(slide 25)* . . . 171
4.3 Map Initialization *(slide 26)* . . . 172
4.4 Tracking *(slide 27)* . . . 173
4.5 Keyframes *(slide 28)* . . . 174
4.6 Local Mapping *(slide 29)* . . . 175
4.7 Loop Detection with Bags of Binary Words *(slide 30)* . . . 177
4.8 Loop Correction and Scale Drift *(slide 31)* . . . 178
4.9 Relocalization *(slide 32)* . . . 179

5 Research Directions *(slides 33–40)* **180**
5.1 Continuous Time: Rolling-Shutter Cameras *(slide 34)* . . . 180
5.2 Continuous Time: Cumulative B-Splines on SE(3) *(slide 35)* . . . 181
5.3 Active SLAM *(slide 36)* . . . 182
5.4 Active SLAM with an Active Stereo Head *(slide 37)* . . . 183
5.5 Active Localization: Learning Where to Look *(slide 38)* . . . 183
5.6 Active View Selection with Radiance Fields *(slide 39)* . . . 184
5.7 Outlook: Learned Front Ends *(slide 40)* . . . 185

6 Summary *(slide 41)* **185**

VI Lecture 7 **187**

1 From One View to Many *(slides 2–3)* **188**
1.1 From One View to Many *(slides 2)* . . . 188
1.2 The Novel View Synthesis Problem *(slides 3)* . . . 188

2 Beyond Surfaces *(slides 4–11)* **189**
2.1 Up to today *(slides 4)* . . . 189
2.2 Beyond Surfaces *(slides 5–6)* . . . 189
2.3 Media Are Common *(slides 7)* . . . 189
2.4 What About Reflections? *(slides 8)* . . . 189
2.5 Dropping the Surface Assumption *(slides 9)* . . . 190

2.6 Density Fields *(slides 10–11)* . . . 190

3 Sensor Physics *(slides 12–16)* **191**

3.1 Radiance *(slides 13)* . . . 191
3.2 Surface Rendering *(slides 14)* . . . 191
3.3 A Ray Through a Medium *(slides 15–16)* . . . 191

4 Volumetric Interactions *(slides 17–24)* **191**

4.1 Absorption *(slides 18–19)* . . . 192
4.2 What else? *(slides 20)* . . . 192
4.3 Emission *(slides 21)* . . . 192
4.4 What else? *(slides 22)* . . . 193
4.5 Scattering *(slides 23)* . . . 193
4.6 Volume Rendering a Ray *(slides 24)* . . . 193

5 Dissecting Volume Rendering *(slides 25–39)* **194**

5.1 Volume Rendering a Ray *(slides 26)* . . . 194
5.2 Volume Rendering a Ray: Absorption *(slides 27–28)* . . . 194
5.3 Volume Rendering a Ray: Emission *(slides 29)* . . . 195
5.4 Emission-Absorption Model *(slides 30)* . . . 195
5.5 The Out-Scattering Term *(slides 31)* . . . 195
5.6 The In-Scattering Term *(slides 32–33)* . . . 195
5.7 The Radiative Transfer Equation *(slides 34)* . . . 196
5.8 The Emission-Absorption Model *(slides 35–36)* . . . 196
5.9 Expressivity of the EA Model *(slides 37)* . . . 196
5.10 Why Model Scattering at All *(slides 38)* . . . 196
5.11 Notation From Here On *(slides 39)* . . . 197

6 From Infinitesimal to Finite Lengths *(slides 40–54)* **197**

6.1 From infinitesimal to finite lengths *(slides 41)* . . . 197
6.2 Absorption-only Volume Rendering *(slides 42)* . . . 197
6.3 Absorption Only, Heterogeneous Medium *(slides 43)* . . . 197
6.4 Transmittance *(slides 44–45)* . . . 198
6.5 Absorption-only Volume Rendering *(slides 46–48)* . . . 198
6.6 Adding Emission *(slides 49)* . . . 199
6.7 Emission-Absorption Volume Rendering *(slides 50)* . . . 199
6.8 Special Case: Homogeneous, Non-Emitting Medium *(slides 51)* . . . 199
6.9 Special Case: Homogeneous emitting medium *(slides 52–53)* . . . 200
6.10 What Can We Render? *(slides 54)* . . . 200

7 Computational Volume Rendering *(slides 55–68)* **201**

7.1 Computational Volume Rendering *(slides 56–57)* . . . 201

7.2 Ray Marching *(slides 58–63)* . . . 201
7.3 Non-Uniform Samples *(slides 64–65)* . . . 203
7.4 Uniform vs Non-Uniform sampling *(slides 66)* . . . 203
7.5 Background Radiance *(slides 67)* . . . 204
7.6 Summary *(slides 68)* . . . 204

8 Summary **204**
8.1 Comparison of the models covered . . . 204
8.2 A qualitative comparison plot . . . 204
8.3 Key take-aways . . . 205

VII Lecture 8 **206**

1 Neural Radiance Fields *(slides 1–2)* **207**

2 Computational Volume Rendering: Ray Marching *(slides 3–8)* **207**
2.1 Ray marching: sum over segments *(slides 3–6)* . . . 207
2.2 Transmittance as a recursion *(slides 4–6)* . . . 208
2.3 Enabling background radiance *(slides 7)* . . . 209
2.4 Summary: computational volume rendering *(slides 8)* . . . 210

3 Volumes: Rendering and Representation *(slides 9–10)* **210**
3.1 Option 1: a grid — Grid Representation *(slides 11–17)* . . . 211

4 Neural Radiance Fields *(slides 18–20)* **213**

5 Learning Neural Radiance Fields *(slides 21–24)* **214**
5.1 From pixels to rays *(slides 21–22)* . . . 214
5.2 Mini-batches of rays *(slides 23)* . . . 215

6 Learning Neural Radiance Fields: An Initial Attempt *(slides 25)* **216**

7 Modeling High Frequencies: A Toy Problem *(slides 26–30)* **216**

8 Uniform to Hierarchical Sampling *(slides 31–33)* **218**

9 Depth vs Inverse-Depth Sampling *(slides 34–36)* **219**
9.1 Normalised device coordinates *(slides 36)* . . . 220

10 Novel-View Synthesis and View-Dependent Effects *(slides 37–41)* **221**
10.1 Novel-View Synthesis *(slides 37)* . . . 221
10.2 View-Dependent Effects *(slides 38–40)* . . . 221
10.3 Learning Neural Radiance Fields: a simple recipe *(slides 41)* . . . 222

11 Is ray sampling accurate? ***(slides 42)*** **222**

12 Pixels as Cones of Rays ***(slides 43–46)*** **223**

13 Volumes: Rendering and Representation — Option 2: An MLP, revisited ***(slides 47)*** **224**

14 Option 3: Voxels with view-dependent radiance — Do We Need Neural Radiance? ***(slides 48–51)*** **225**

14.1 Spherical harmonics for view-dependent colour *(slides 49)* 225

14.2 The Plenoxels pipeline *(slides 50)* 226

14.3 NeRF vs. Plenoxels *(slides 51)* 227

15 Towards Hybrid Representations ***(slides 52–54)*** **227**

16 Summary ***(slides 55)*** **228**

VIII Lecture 9 **231**

1 Neural Implicit Rendering ***(slides 1–2)*** **232**

2 Review: Volume Rendering ***(slides 3–4)*** **232**

3 (Neural) Volumes: A Very Expressive Representation ***(slide 5)*** **233**

4 Back to Surfaces ***(slides 6–8)*** **233**

5 Recap: 3D Representations ***(slide 9)*** **234**

6 Recap: Implicit Surfaces ***(slide 10)*** **235**

7 Neural Implicit Surfaces ***(slides 11–12)*** **235**

8 Rendering Neural Implicit Surfaces ***(slides 13–14)*** **236**

9 Intersections of Rays with Implicit Surfaces ***(slides 15–22)*** **236**

9.1 Problem statement *(slides 15–16)* 236

9.2 Searching for roots of a 1-d function: secant method *(slides 17–18)* 236

9.3 Intersections of rays with SDFs: sphere tracing *(slides 19–20)* 237

9.4 Options and differentiability *(slides 21–22)* 238

10 Differentiable Root Finding ***(slides 23–26)*** **238**

10.1 A toy example *(slide 23)* 238

10.2 The general rule *(slides 24–26)* 238

11 Differentiable Ray Marching ***(slides 27–29)*** **239**

12 Rendering Implicit Surfaces (or SDFs) ***(slides 30–32)*** **239**

13 Learning Implicit Geometry and Appearance ***(slide 33)*** **240**

14 Rendering Neural Implicits ***(slides 34–36)*** **240**

15 Rendering Neural Implicits: Practical Considerations ***(slides 37–38)*** **241**

16 Visibility Discontinuities ***(slide 39)*** **242**

17 Rendering Implicit Surfaces ***(slide 40)*** **242**

18 Volume Rendering for Implicit Surfaces ***(slides 41–43, 49)*** **242**

19 From SDF to Density ***(slides 44–48)*** **243**

19.1 A first choice: VolSDF *(slide 44)* . . . 243

19.2 A second heuristic: NeuS *(slides 45–46)* . . . 244

19.3 Comparison of the densities *(slides 47–48)* . . . 244

20 Volume Rendering for Implicit Surfaces: results ***(slides 50–54)*** **245**

21 Summary ***(slides 55–56)*** **246**

A Appendix: supplementary material (not on the slides) **248**

A.1 From the continuous volume rendering integral to Eq. (18) . . . 248

A.2 Ψ_β is a Laplace CDF . . . 248

IX Lecture 10 **249**

1 Differentiable Primitive Rendering ***(slides 1–7)*** **250**

1.1 Recap: Rendering Volumes *(slide 3)* . . . 250

1.2 Rendering Primitives *(slides 4–6)* . . . 250

1.3 Today's Goals *(slide 7)* . . . 251

2 Mesh Rendering via Ray-casting ***(slides 8–13)*** **251**

2.1 Mesh Rendering via Ray-casting *(slide 8)* . . . 251

2.2 Mesh Rendering: Ray-Triangle Intersection *(slide 9)* . . . 251

2.3 Mesh Rendering: Ray-Mesh Intersection *(slide 10)* . . . 252

2.4 Mesh Rendering *(slide 11)* . . . 252

2.5 Mesh Rendering: Appearance *(slide 12)* . . . 253

2.6 Mesh Rendering (differentiable?) *(slide 13)* . . . 253

3 Mesh Rendering: Differentiability ***(slide 14)*** **253**

4 Differentiable Mesh Rendering ***(slides 15–20)*** **254**

4.1 Make triangles 'soft' *(slides 15–16)* . . . 254
4.2 Aggregate from multiple triangles *(slide 17)* . . . 255
4.3 Effect of γ and σ, and an example *(slides 18–19)* . . . 255
4.4 Differentiable Mesh Rendering via Ray tracing *(slide 20)* . . . 255

5 From Ray tracing to Rasterization *(slides 21–25)* **256**
5.1 Redundant computation *(slide 21)* . . . 256
5.2 From Ray tracing to Rasterization *(slides 22–23)* . . . 257
5.3 Differentiable Primitive Rendering *(slides 24–25)* . . . 258

6 Differentiable Gaussian Rendering *(slides 26–38)* **258**
6.1 What is the representation of a 3D gaussian? *(slides 26–29)* . . . 258
6.1.1 Gaussian primitive *(slide 27)* . . . 258
6.1.2 What Each Gaussian Stores *(slide 28)* . . . 259
6.1.3 Why Not Optimize the Covariance Directly? *(slide 29)* . . . 259
6.2 How to project to 2D and rasterize? *(slides 30–34)* . . . 260
6.2.1 World to camera *(slides 30–31)* . . . 260
6.2.2 Projection to the image *(slides 32–33)* . . . 260
6.2.3 Rasterize *(slide 34)* . . . 261
6.3 Tile Rasterizer *(slides 35–37)* . . . 261
6.3.1 Tile Rasterizer 1/3: one instance per touched tile *(slide 35)* . . . 261
6.3.2 Tile Rasterizer 2/3: one radix sort *(slide 36)* . . . 262
6.3.3 Tile Rasterizer 3/3: Blend Each Tile *(slide 37)* . . . 263
6.4 How to model/aggregate appearance? *(slide 38)* . . . 263

7 3D Gaussian Splatting: The Full Pipeline *(slides 39–41)* **264**
7.1 The Full Pipeline *(slide 39)* . . . 264
7.2 Backward Pass: Gradients Through Blending *(slide 40)* . . . 264
7.3 Optimization *(slide 41)* . . . 265

8 Initialization, Densification and Ablations *(slides 42–45)* **265**
8.1 Initialization: SfM Points vs Random *(slide 42)* . . . 265
8.2 Adaptive Density Control *(slide 43)* . . . 265
8.3 Ablation: Densification *(slide 44)* . . . 266
8.4 Ablation: Anisotropy *(slide 45)* . . . 267

9 Results: Quality and Speed *(slides 46–48)* **267**

10 Limitations *(slide 49)* **268**

11 Other Primitives *(slides 50–54)* **269**
11.1 Other Primitives: Fuzzy Metaballs *(slide 50)* . . . 269
11.2 2D Gaussian Splatting: The Primitive *(slide 51)* . . . 269

11.3 2D Gaussian Splatting: Ray-Splat Intersection *(slide 52)* . . . 270
11.4 From Cameras to LiDAR: Splat-LOAM *(slide 53)* . . . 270
11.5 Beyond Gaussians: Other Primitives *(slide 54)* . . . 270

12 Summary *(slide 55)* **272**

X Lecture 11 **274**

1 Relightable 3D Representations *(slides 1–6)* **274**
1.1 Recap: Neural 3D Representations *(slide 3)* . . . 274
1.2 Radiance Fields Bake In the Lighting *(slide 4)* . . . 274
1.3 Goal: Relightable 3D *(slide 5)* . . . 274
1.4 Today's Goals *(slide 6)* . . . 275

2 What We Need: The Rendering Equation *(slides 7–9)* **275**
2.1 Material: How a Surface Reflects Light *(slide 8)* . . . 276
2.2 Light: What Arrives at Each Point *(slide 9)* . . . 276

3 Let's Learn Everything We Need *(slides 10–11)* **277**
3.1 Learn Geometry, Material and Light with Networks *(slide 10)* . . . 277
3.2 Rendering a Pixel *(slide 11)* . . . 277

4 Does It Work? Why Not? *(slides 12–17)* **278**
4.1 Does It Work? *(slide 12)* . . . 278
4.2 Why? What One Capture Observes *(slide 13)* . . . 278
4.3 Many Explanations, Same Pixels *(slides 14–16)* . . . 279
4.4 It Happens on Real Objects *(slide 17)* . . . 280

5 How Do We Resolve the Ambiguity? *(slide 18)* **280**

6 Option 1: Constrain the Model *(slides 19–34)* **280**
6.1 Constrain the Light *(slide 20)* . . . 280
6.2 Constrain the Material *(slide 21)* . . . 281
6.3 Constraints Help Even with Sparse Views *(slides 22–23)* . . . 282
6.4 PhySG: The Full Pipeline *(slide 24)* . . . 282
6.5 Spherical Gaussians Make the Integral Closed-Form *(slide 25)* . . . 282
6.6 PhySG: Results *(slide 26)* . . . 284
6.7 What PhySG Ignores: Shadows & Interreflections *(slide 27)* . . . 284
6.8 Recall: Gaussians Store Baked Color *(slide 28)* . . . 284
6.9 Relightable Gaussians (IRGS): Store Material *(slide 29)* . . . 284
6.10 Blend Attributes, Then Shade *(slide 30)* . . . 285
6.11 What About Shadows? *(slide 31)* . . . 285

6.12 Putting Shadows in the Rendering Equation *(slide 32)* . . . 286
6.13 IRGS: Results *(slides 33–34)* . . . 287

7 Option 2: Capture More Informative Data *(slides 35–41)* **287**
7.1 Capture: One Light at a Time (OLAT) *(slide 36)* . . . 287
7.2 Light Adds Up *(slides 37–38)* . . . 288
7.3 Light Transport Is Linear *(slide 39)* . . . 288
7.4 Relighting = a Weighted Sum *(slides 40–41)* . . . 288

8 Option 3: Learn Priors from Data *(slides 42–47)* **289**
8.1 MaterialFusion: Prior Inside the Optimization *(slide 43)* . . . 289
8.2 Neural Gaffer: Learn to Relight Images Directly *(slide 45)* . . . 290
8.3 How Do You Tell a Network the Light? *(slide 46)* . . . 291
8.4 Neural Gaffer: Data, 3D and Limits *(slide 47)* . . . 291

9 Summary *(slide 48)* **291**

XI Lecture 12 **294**

1 Active Perception and Active Mapping *(slides 1–4)* **295**
1.1 What Is Active Perception? *(slides 1–2)* . . . 295
1.2 Active Mapping *(slide 3)* . . . 295
1.3 Our Task: Focus on the Objects *(slide 4)* . . . 296

2 The Tool: 3D Gaussian Splatting *(slides 5–8)* **297**
2.1 3D Gaussian Splatting Representation *(slide 5)* . . . 297
2.2 How Does 3DGS Work? *(slide 6)* . . . 297
2.3 Training Views, Novel Views, and What a Planner Does with 3DGS *(slides 7–8)* . . . 298

3 MAGICIAN: Planning with Imagined Gaussians *(slides 9–11)* **299**
3.1 MAGICIAN Architecture and Imagined Gaussians *(slides 9–10)* . . . 299
3.2 From Coverage to Objects *(slide 11)* . . . 300

4 The Dual-Map System and Semantic Fusion *(slides 12–17)* **300**
4.1 The System and Full Pipeline *(slides 12–13)* . . . 300
4.2 Semantics on a Real Room: From Pixels to Gaussians *(slides 14–15)* . . . 301
4.3 The Planning State: 5 cm Voxel Grid *(slides 16–17)* . . . 302

5 Multi-Signal Planning and Beam Search *(slides 18–21)* **303**
5.1 Three Signals, Three Questions *(slides 18–19)* . . . 303
5.2 Looking Ahead: Beam Search and Candidate Rendering *(slides 20–21)* . . . 304

6 Experimental Results and Generalization *(slides 22–25)* **306**

6.1 A Real Flight: Quantitative Progression and Object Lifecycle *(slides 22–24)* 306
6.2 Same System, Another Task: Object Navigation *(slide 25)* 307

7 Summary and Key Takeaways *(slide 26)* **307**
7.1 Comparison of Paradigms and Task Configurations 307
7.2 Key Takeaways and Open Directions *(slide 26)* 307

XII EAISUM **309**

Part I

Lecture 2

Abstract

These notes follow the slide deck of Lecture 02 (75 slides) in the order in which the slides present the material. Every section header carries the range of slides it covers. Formal statements are typeset in boxes (**Definition**, **Proposition**, **Algorithm**, ...), and every result or non-obvious idea is followed by a blue *Intuition* box. All figures are original TikZ/pgfplots redrawings, so the document is self-contained. The last slide of the file (slide 75, a backup slide placed after the acknowledgments) is collected in Appendix **??**.

1 Representations for 3D *(slides slides 2–4)*

The deck opens with the same bunny (and a knot) shown as a triangle mesh, a point cloud, a smooth surface and a voxel grid: the *same shape*, four very different data structures.

Life is nice in 2D-land... *(slides slide 3)*

In 2D, every camera (an old box camera, a compact camera, a phone) produces the same object: a regular grid of pixels. That single representation is shared across *sensors*, *tools* (Photoshop) and *learning frameworks* (PyTorch, TensorFlow). Nobody asks "which image representation is best".

Meanwhile in 3D land... *(slides slide 4)*

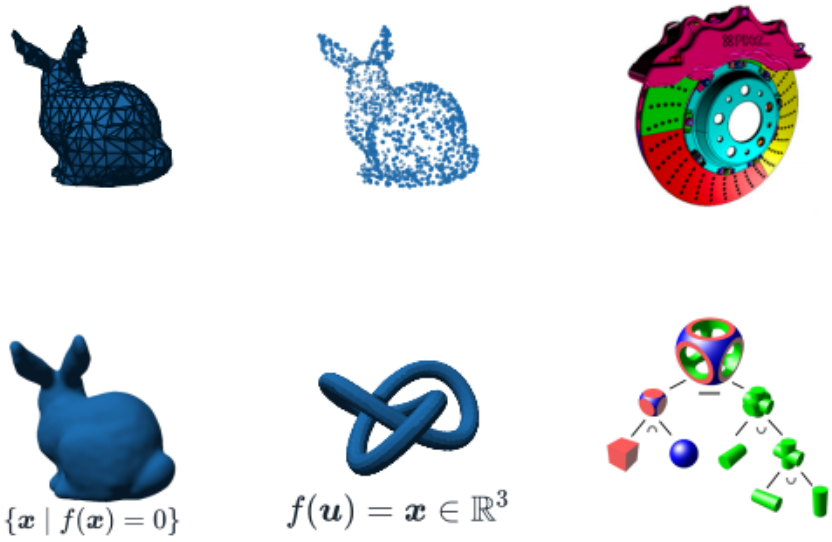


Figure 1: The same shape represented in four different ways

In 3D there is no such consensus. The slide lists: triangle meshes, point clouds, CAD models, surfaces given as a zero level set $\{\boldsymbol{x} \mid f(\boldsymbol{x}) = 0\}$, surfaces given by a parametrization $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^3$, CSG trees, "and many more".

Intuition

The 2D grid is universal because image sensors *measure* on a grid and every algorithm can assume that grid. In 3D, different tasks pull in different directions: a LiDAR returns unordered points, a designer edits smooth curves, a physics engine needs inside/outside tests, a neural network prefers fixed-size arrays. Each 3D representation is a different compromise. The rest of the lecture is a tour of these compromises.

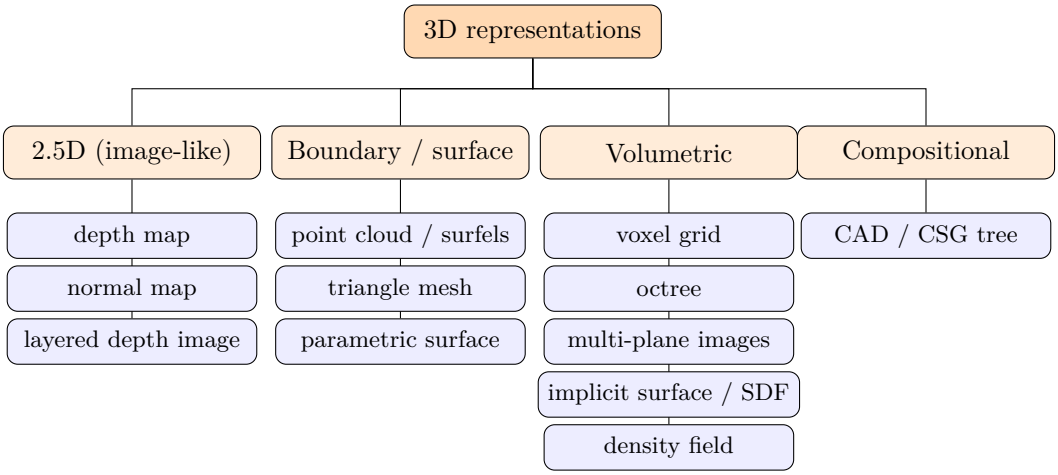


Figure 2: A map of the representations met in the lecture (our grouping, following slides 43 and 63: points, meshes and parametric surfaces are *boundary* representations; voxels and implicit fields are *volumetric*).

2 Notation *(slides slide 5)*

Definition 2.1 (Notation)**.** A point in 3D space is written in inhomogeneous coordinates as

$$\boldsymbol{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \in \mathbb{R}^3.$$

A point on the image plane carries the subscript s: $\boldsymbol{x}_s \in \mathbb{R}^2$. Discrete maps are *indexed* rather than evaluated, and brackets denote indexing:

$$D[\boldsymbol{x}_s] \in \mathbb{R}^+ \quad \text{(depth)}, \qquad N[\boldsymbol{x}_s] \in \mathbb{S}^2 \quad \text{(unit normal)}, \qquad V[i,j,k] \in [0,1] \quad \text{(occupancy)}.$$

Remark 2.2. The subscript denotes the *coordinate frame*, not the dimension of the space. Both $\boldsymbol{x}$ and $\boldsymbol{x}_s$ denote points.

Intuition

Parentheses mean "compute" ($f(\boldsymbol{x})$ runs a formula or a network); brackets mean "look up" ($D[\boldsymbol{x}_s]$ reads a pixel). The letter s is a hint that $\boldsymbol{x}_s$ lives on the *s*creen; it is a name tag, not an exponent.

3 Manifolds *(slides slide 6)*

Definition 3.1 (Manifold)**.** An n-dimensional *manifold* is a topological space that locally resembles Euclidean space: every point has a neighborhood homeomorphic to an open subset of $\mathbb{R}^n$; n is its dimension.

Lines and circles are 1-manifolds. Two-dimensional manifolds are called *surfaces*: the plane, the sphere, the torus. Each coloured piece of the shape on the slide is a *chart*: it flattens a part of the shape onto $\mathbb{R}^n$.

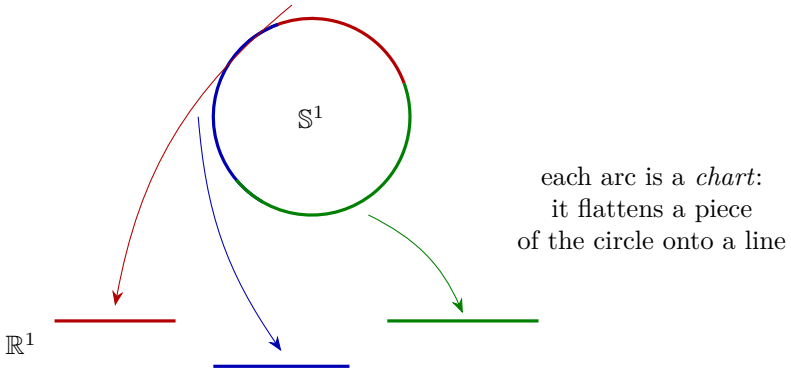


Figure 3: The circle is a 1-manifold. Three overlapping charts, each a homeomorphism from an arc to an interval of $\mathbb{R}$, cover it. (Slide 6 shows the same idea for a circle and a sphere.)

Intuition

An ant walking on a sphere sees a flat plane around itself: that local flatness is what "manifold" means. The sphere as a whole is *not* a plane, which is why one needs several overlapping charts (an atlas) to describe it, exactly as one needs several flat maps to cover the Earth without cutting it.

4 Defining a Surface *(slides slide 7)*

Definition 4.1 (Parametric and implicit forms)**.** A surface can be specified in two ways.

- **Parametric form:** a function maps a 2D domain into space, $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^3,\ \boldsymbol{u} \in \mathcal{M}$.
- **Implicit form:** the surface is the zero level set of a scalar field, $\mathcal{S} = \{\boldsymbol{x} \mid f(\boldsymbol{x}) = 0\}$.

In the parametric form the domain point $\boldsymbol{u}$ is *not* a point in space: it belongs to a 2D manifold $\mathcal{M}$, for instance the sphere $\mathbb{S}^2$.

Remark 4.2. The symbol f denotes both functions; the argument decides which is meant: $f(\boldsymbol{u})$ returns a surface point, $f(\boldsymbol{x})$ returns a scalar.

Intuition

Parametric = "a recipe that *produces* points" (feed in a domain point, get a 3D point out). Implicit = "a rule that *judges* points" (feed in a 3D point, get a number whose sign says on which side of the surface it lies). Slide 48 will turn this into the mantra *parametric generates, implicit tests.*

5 Meanwhile in 3D-land... *(slides slides 8–9)*

"But what is the *best* representation?" The slide's answer: *it depends*, on the task: acquisition vs editing vs animation vs prediction vs processing vs optimization, and more.

The goals of the lecture (slide 9):

- understand representations *computationally* (what do they look like in code?);
- understand their limitations and benefits;
- understand conversions across representations.

Intuition

There is no free lunch: a representation that makes one operation trivial tends to make another one painful. Reading each representation below, ask three questions: how do I *store* it, what is *easy* to do with it, what is *hard*? The lecture proceeds from representations tied to sensors (depth maps, point clouds), through surfaces (meshes, parametric surfaces), to volumes (voxels, implicit fields, density fields).

6 Depth Maps *(slides slides 10–14)*

6.1 Depth maps *(slides slides 10–12)*

Depth is a *common modality across sensors* (structured-light and time-of-flight cameras such as the Kinect and industrial cameras shown on slide 10); depth is treated in detail in Lectures 04–05.

Definition 6.1 (Depth map)**.** A depth map is an image $D[\boldsymbol{x}_s] \in \mathbb{R}^+$ that records how far the scene point seen by each pixel $\boldsymbol{x}_s$ is from the camera.

Because it is just an image, *standard image-processing tools can be used* on it (slide 12): filtering, resizing, convolutional networks.

6.2 Depth Maps and Normal Maps *(slides slide 13)*

The same idea extends to other surface properties, e.g. surface normals:

Definition 6.2 (Normal map)**.** $N[\boldsymbol{x}_s] \in \mathbb{S}^2$ stores, for each pixel, the unit normal of the visible surface.

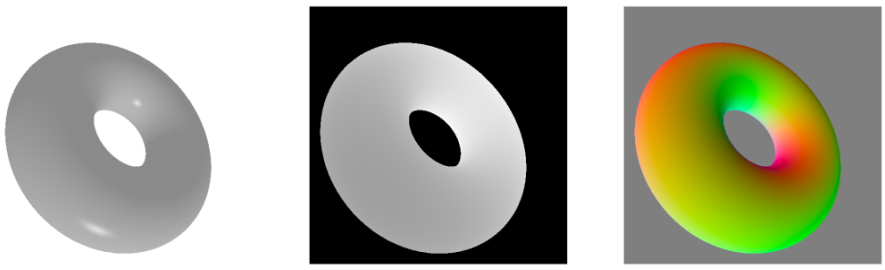


Figure 4: Depth and normal maps of a scene. The depth map is a grayscale image where brightness encodes distance; the normal map is a color image where RGB encodes the XYZ components of the unit normal.

6.3 Depth Maps — Representing the Visible *(slides slide 14)*

Remark 6.3. A depth map does not capture the *full* 3D structure: it is a *2.5D* representation, i.e. properties associated with image pixels. The slide asks: can we develop more *complete* 3D representations?

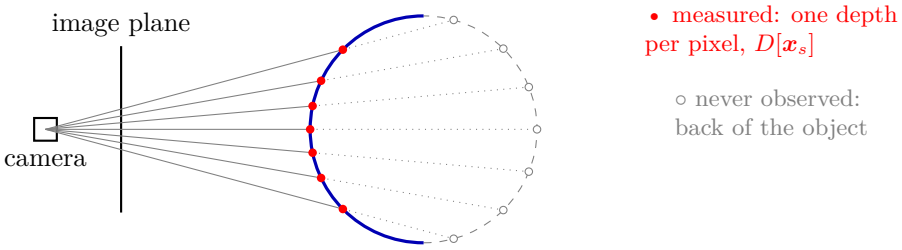


Figure 5: Side view of a depth camera. Each pixel ray returns the *first* surface hit (red); the far side of the object (dashed) and anything hidden behind the visible surface is absent from D.

Intuition

A depth map is like a relief map seen from one viewpoint: it tells the distance to whatever is *in front*, and nothing about what is behind it. Two things follow. First, cheap and fast: it is an ordinary image. Second, incomplete: to model a whole object you need several views or a representation that lives in 3D itself, which motivates point clouds next.

7 Point Clouds *(slides slides 15–21)*

7.1 Point Clouds *(slides slides 15–18)*



Figure 6: Some point clouds of scenes. Each point is a 3D position, and can be drawn as a small sphere.

Point clouds are often acquired in an image-like manner from sensors (LiDAR, depth cameras) but it can be useful to *forget* this: point clouds from multiple views can be merged into a single point cloud, which helps unify approaches independently of the sensing modality.

Definition 7.1 (Point cloud)**.** A point cloud is an *unordered set* of points $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\} \subset \mathbb{R}^3$, usually stored as an $N \times 3$ array. Reordering the rows by any permutation σ gives $\{\boldsymbol{x}_{\sigma(1)}, \ldots, \boldsymbol{x}_{\sigma(N)}\}$, the *same* point cloud.

Unlike an image, the row order carries no meaning, so processing or generation methods must be *permutation invariant*; a fully connected layer applied to the flattened array does not work.

Example 7.2 (Why a fully connected layer fails)**.** Let $X = \{\boldsymbol{x}_1 = (1,0,0), \boldsymbol{x}_2 = (0,1,0)\}$. Flattened, the two orderings give $(1,0,0,0,1,0)$ and $(0,1,0,1,0,0)$, two different vectors, so a generic linear layer returns two different outputs for one geometric object. A *symmetric* operation such as taking the per-coordinate maximum over the points returns $(1,1,0)$ in both

orders.

Intuition

Think of a bag of marbles: shaking it does not change what it contains. An operator must treat the input as a bag, not as a list. Symmetric functions (sum, mean, max over points) have this property, and network designs for point clouds are built from them.

7.2 Oriented Point Clouds *(slides slide 19)*

Definition 7.3 (Oriented point cloud, surfel)**.** An oriented point cloud stores a normal with each point: $\{(\boldsymbol{x}_1, \boldsymbol{n}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{n}_N)\}$. Points can then be treated as small flat disks rather than tiny spheres, which helps rendering under lighting. When a radius is also stored the primitives are called *surfels*.

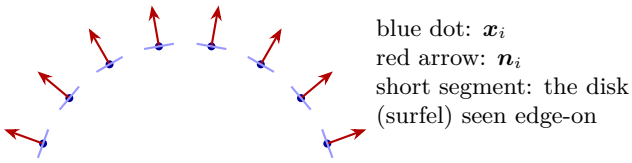


Figure 7: An oriented point cloud sampled from a curve (2D cross-section): each sample carries a position and a normal, and can be drawn as a small disk perpendicular to the normal.

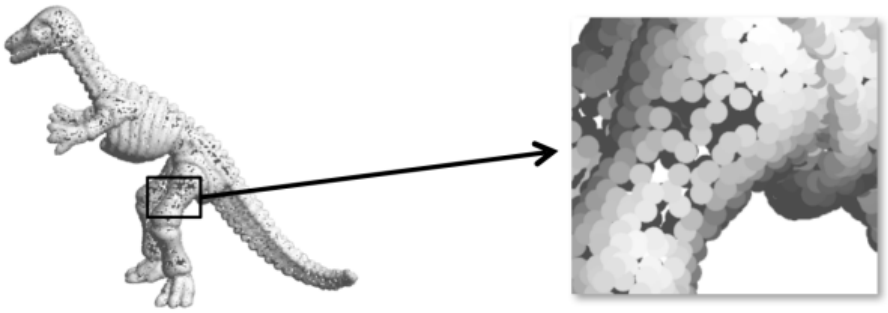


Figure 8: A surfel-based reconstruction of a dinosaur. Each surfel is a small disk with a position, a normal and a radius.

Intuition

A bare point has no facing direction, so lighting is ambiguous. Attaching a normal turns each point into a tiny mirror; adding a radius makes the mirror cover a patch of surface, so neighbours overlap and the gaps between samples close up.

7.3 No explicit connectivity *(slides slides 20–21)*

A point cloud has no explicit *connectivity* information. Two point sets that look nearly identical may come from a closed circle and from a circle with a small gap. One can always sample more points to resolve the gap, but this is super inefficient; it is more efficient to add *edges*, because connectivity does not scale like density (which grows with an area).

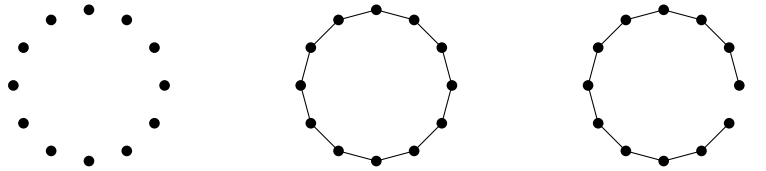


Figure 9: The same 12 points support two different shapes: only the edges (connectivity) tell them apart.

Example 7.4 (Density vs. connectivity)**.** To sample a unit sphere with spacing h one needs roughly $4\pi/h^2$ points: about 1257 for $h = 0.1$ but about 5027 for $h = 0.05$. Halving the spacing multiplies the number of points by 4, while adding one edge costs one pair of integers.

Intuition

Density is a *2D* bill (points per unit area), connectivity is a *1D* one (a few edges along features). To say "these two neighbouring points belong to the same continuous piece" it is far cheaper to draw a line than to fill the gap with hundreds of extra samples. This is exactly what meshes do.

8 Meshes *(slides slides 22–34)*

8.1 Meshes: vertices and faces *(slides slides 22–25)*

Meshes are *piecewise linear approximations* of the underlying surface; more elements allow a better approximation (slide 22 shows a torus with increasing resolution).

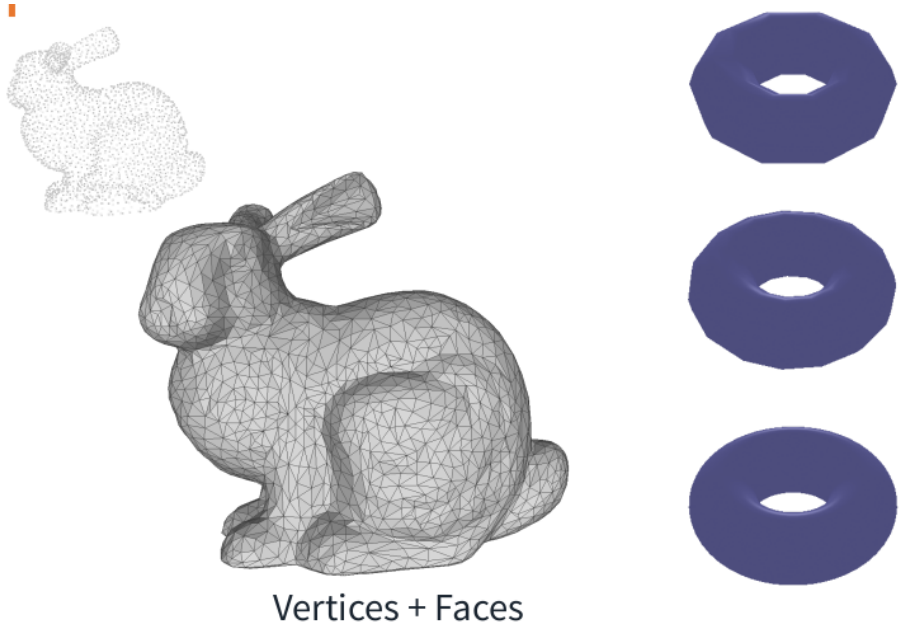
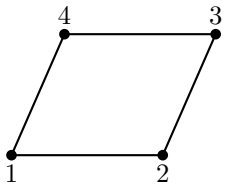
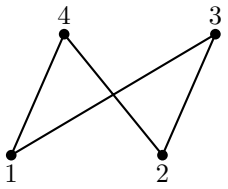


Figure 10: A torus approximated by a mesh of increasing resolution and the difference between a mesh and a pointcloud of a rabbit. The mesh is a piecewise linear approximation of the underlying surface.

Definition 8.1 (Mesh)**.** A mesh is a pair (V, F): V is an $N \times 3$ array of vertex positions (the *geometry*) and F is a list of M faces (the *topology*), each face being a list of indices of the vertices that make it up. Consecutive indices in a face are joined by an edge, so *ordering matters* in face indices.



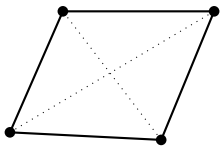
face $(1, 2, 3, 4)$: a quadrilateral



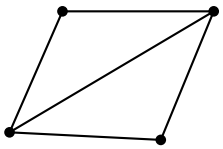
face $(1, 3, 2, 4)$: a "bow tie"

Figure 11: Same four vertices, two index orders, two different shapes: consecutive indices are connected by an edge.

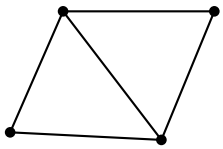
Arbitrary polygons can also be *non-planar* (four points in 3D need not lie in a plane), so one restricts to triangles: three points not on a line always define a plane.



polygon, two diagonals



triangulation A



triangulation B

Figure 12: A quad can be split into triangles in two ways. If the four points are not coplanar, A and B are two genuinely different surfaces, while each triangle on its own is always flat.

8.2 Triangular Meshes *(slides slides 26–31)*

Definition 8.2 (Triangular mesh)**.** A triangular mesh is a mesh whose faces are all triangles: $V \in \mathbb{R}^{N\times 3}$ and $F \in \{0, \ldots, N-1\}^{M\times 3}$, each row of F listing the indices of the *three* vertices of a face.

Example 8.3 (A tetrahedron in code)**.**

```
    V = np.array([[0,0,0],[1,0,0],[0,1,0],[0,0,1]], dtype=float)   # 4 x 3
F = np.array([[0,2,1],[0,1,3],[0,3,2],[1,2,3]])                  # 4 x 3, ints
```

Here $N = 4$ vertices, $M = 4$ faces and $E = 6$ edges.

A sanity check on sizes.

Proposition 8.4 (Face count of a closed genus-0 triangle mesh)**.** *Let a closed, sphere-like (genus 0) triangular mesh have V vertices, E edges and F faces. Every face has 3 edges and every edge is shared by 2 faces, so $E = 3F/2$. With Euler's formula $V - E + F = 2$ this gives*

$$F = 2V - 4.$$

Proof. Substitute $E = 3F/2$ into $V - E + F = 2$: $V - \frac{3}{2}F + F = 2$, i.e. $V - \frac{1}{2}F = 2$, hence $F = 2V - 4$. □

Intuition

You get about *twice as many triangles as vertices.* Checks: tetrahedron $V = 4 \Rightarrow F = 4$ ✓; octahedron $V = 6 \Rightarrow F = 8$ ✓; a scan with $V = 10{,}000$ vertices has $F = 19{,}996$ faces and $E = 3F/2 = 29{,}994$ edges. Useful to guess memory footprints or to spot a broken mesh.

Orientation. For a triangle, *any order gives the same triangle*, but the order determines which side is "front". The normal is $\boldsymbol{n} = (\boldsymbol{b} - \boldsymbol{a}) \times (\boldsymbol{c} - \boldsymbol{a})$.

Proposition 8.5 (Orientation and index order)**.** *A cyclic shift* $(a, b, c) \to (b, c, a)$ *leaves* $\boldsymbol{n}$ *unchanged; swapping two indices, e.g.* $(a, b, c) \to (c, b, a)$*, flips its sign.*

Proof. Expanding, $(\boldsymbol{b} - \boldsymbol{a}) \times (\boldsymbol{c} - \boldsymbol{a}) = \boldsymbol{b} \times \boldsymbol{c} - \boldsymbol{b} \times \boldsymbol{a} - \boldsymbol{a} \times \boldsymbol{c} + \boldsymbol{a} \times \boldsymbol{a} = \boldsymbol{a} \times \boldsymbol{b} + \boldsymbol{b} \times \boldsymbol{c} + \boldsymbol{c} \times \boldsymbol{a}$. This expression is unchanged by cyclic relabelling $\boldsymbol{a} \to \boldsymbol{b} \to \boldsymbol{c} \to \boldsymbol{a}$ and changes sign under the swap of any two symbols, since $\times$ is antisymmetric. □



(a, b, c): counter-clockwise, $\boldsymbol{n}$ toward the viewer (c, b, a): clockwise, $\boldsymbol{n}$ away from the viewer

Figure 13: Index order defines the winding direction and hence the side the normal points to. Swapping two indices reverses it.

Example 8.6 (Checking the tetrahedron)**.** For face $(0, 2, 1)$ of the example above, $\boldsymbol{a} = (0, 0, 0)$, $\boldsymbol{b} = (0, 1, 0)$, $\boldsymbol{c} = (1, 0, 0)$: $\boldsymbol{n} = (0, 1, 0) \times (1, 0, 0) = (0, 0, -1)$, pointing down, i.e. *outward* for the face lying in $z = 0$ (the tetrahedron sits above it). Face $(1, 2, 3)$ gives $(-1, 1, 0) \times (-1, 0, 1) = (1, 1, 1)$, also outward. All four faces are consistently oriented.

Intuition

The "classic bug": if some faces are listed clockwise and others counter-clockwise, the renderer treats some triangles as back-facing and they render black or vanish. Rule of thumb: use one winding convention for the whole mesh and check with $\boldsymbol{n}$ pointing away from the interior.

Manifold problems. Even with triangles one can still get *non-manifold* meshes. The slides identify two cases: an edge shared by more than two faces (so restrict each edge to at most two faces; one can often get away without enforcing this) and a *disconnected surface around a vertex*, which is critical for mesh-processing applications and less so for rendering.

Ray–triangle intersection and texturing.

Proposition 8.7 (Barycentric form and the ray–triangle solve)**.** *Any point of the triangle with vertices* $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$ *is* $\alpha\boldsymbol{a} + \beta\boldsymbol{b} + \gamma\boldsymbol{c}$ *with* $\alpha + \beta + \gamma = 1$ *and all three weights* ≥ 0*. Setting this equal to the ray* $\boldsymbol{o} + t\boldsymbol{d}$ *gives* 3 *linear equations in the* 3 *unknowns* (t, β, γ)*, i.e. a* 3×3 *solve per triangle:*

$$\beta(\boldsymbol{b} - \boldsymbol{a}) + \gamma(\boldsymbol{c} - \boldsymbol{a}) - t\,\boldsymbol{d} = \boldsymbol{o} - \boldsymbol{a}, \qquad \alpha = 1 - \beta - \gamma.$$



an edge seen end-on, shared by 3 faces two fans touching at a single vertex

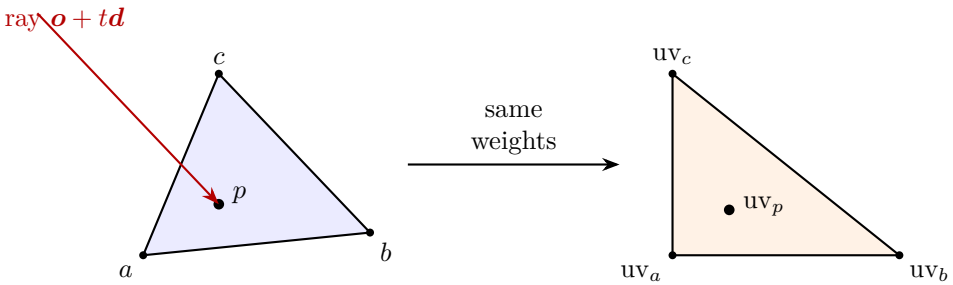
Figure 14: Two kinds of non-manifold configuration.

The ray hits the triangle iff $\beta \geq 0$, $\gamma \geq 0$, $\beta + \gamma \leq 1$ *(and* $t > 0$*).*

Proof. Substitute $\alpha = 1 - \beta - \gamma$ in $\alpha\boldsymbol{a} + \beta\boldsymbol{b} + \gamma\boldsymbol{c} = \boldsymbol{o} + t\boldsymbol{d}$ and collect terms. □

Example 8.8. Triangle $\boldsymbol{a} = (0,0,0)$, $\boldsymbol{b} = (1,0,0)$, $\boldsymbol{c} = (0,1,0)$; ray $\boldsymbol{o} = (0.25, 0.25, 1)$, $\boldsymbol{d} = (0,0,-1)$. The system reads $\beta(1,0,0) + \gamma(0,1,0) - t(0,0,-1) = (0.25, 0.25, 1)$, so $\beta = \gamma = 0.25$, $t = 1$, $\alpha = 0.5$. All weights are ≥ 0: hit at $\boldsymbol{o} + \boldsymbol{d} = (0.25, 0.25, 0)$.

Texturing is easy: each vertex also stores a coordinate into a 2D image, and inside a triangle the same weights interpolate those coordinates.



$p = \alpha a + \beta b + \gamma c$ $\mathrm{uv}_p = \alpha\,\mathrm{uv}_a + \beta\,\mathrm{uv}_b + \gamma\,\mathrm{uv}_c$

Figure 15: Left: the hit point in barycentric coordinates $(\alpha, \beta, \gamma) = (0.5, 0.25, 0.25)$. Right: the same weights interpolate the 2D texture coordinates stored at the vertices.

Intuition

Barycentric weights are "how much of each corner" a point is made of; a point near a has α close to 1. Intersecting a ray with a triangle is therefore a tiny linear-algebra problem, which is why GPUs can do billions of them, and the *same* weights carry any per-vertex quantity (texture coordinates, colours, normals) to the interior.

8.3 Triangular Meshes — Geometry vs Connectivity *(slides slides 32–34)*

The two arrays of a mesh behave very differently: V is `float`, differentiable, and gradient descent (GD) is happy; F is `int`, discrete, and GD is not happy. Modifying vertex positions can edit the shape; other changes are non-trivial (i.e. changing F).

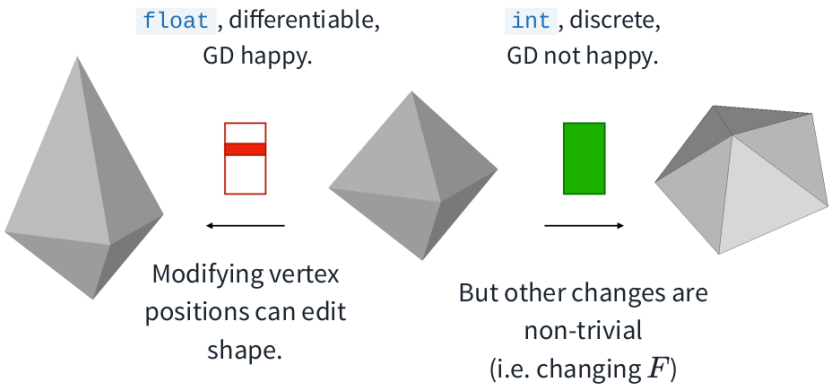


Figure 16: Editing the geometry V is easy, editing the connectivity F is hard.



edit V (`float`):
move a vertex, F untouched

edit F (`int`):
flip an edge, V untouched

Figure 17: Geometry edits are small continuous changes of real numbers; connectivity edits are combinatorial jumps.

Same counts, different meshes *(slides slide 33)*

Going from one mesh to another is non-trivial even when both have 6 vertices and 8 faces (the octahedron is one such mesh). It is even more challenging if the topology changes, e.g. torus to sphere. In general, designing methods that can predict *arbitrary* connectivity is challenging (e.g. image to mesh).

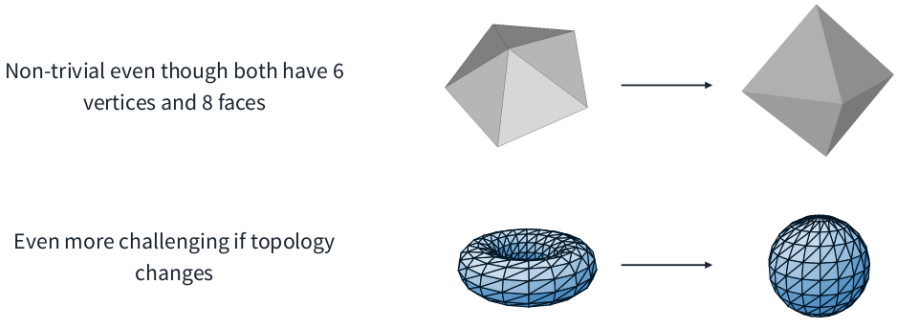


Figure 18: Two meshes with the same number of vertices and faces, but different connectivity.

Continuous deformation *(slides slide 34)*

The first change (same counts, different connectivity) is possible under continuous deformation of the shape; the torus-to-sphere change is impossible under continuous deformation (a hole cannot be closed without tearing). Yet *both* edits require similar operations for meshes: a new F and an

updated V. The slide asks: is there an alternate representation where the former (the continuous one) is easier?

Intuition

A mesh mixes a smooth part (V) with a rigid combinatorial part (F). Even a tiny, continuous change of the shape may demand a jump in F, and a network cannot easily output "an integer array of unknown length". The question hanging in the air motivates representations where shape changes are always changes of continuous numbers: parametric and, above all, implicit ones.

9 Parametric Surfaces *(slides slides 35–41)*

9.1 Parametric surfaces in 2D *(slides slide 35)*

The domain is a circle $\mathbb{S}^1$; think of it as an angle θ. A function $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^2$, $\boldsymbol{u} \in \mathbb{S}^1$ sends each angle to a point of a curve: f_1 to a pentagon, f_2 to a square.

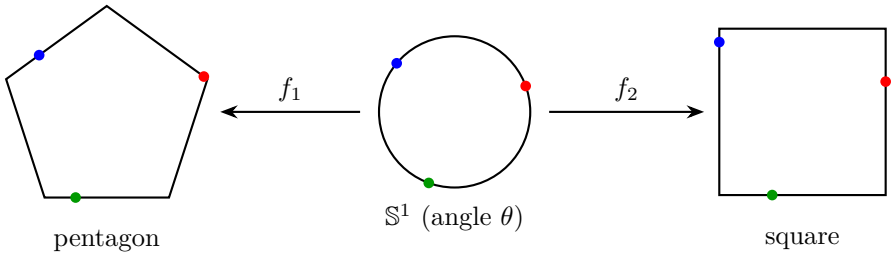


Figure 19: Two different maps f_1, f_2 from the same circle domain. Same colour = same domain point θ.

9.2 Parametric surfaces *(slides slides 36–40)*

Definition 9.1 (Parametric surface)**.** A continuous function $f : \mathcal{M} \to \mathbb{R}^3$, $f(\boldsymbol{u}) = \boldsymbol{x}$, over a 2D manifold $\mathcal{M}$ parametrizes a surface. The *connectivity/topology* is constrained to be the same as $\mathcal{M}$. This disentangles discretization (vertices/faces) from shape representation. In code:

```
def f(u):
    '''Input:  u: N-dimensional vector from a 2D manifold
       Output: x: 3D point'''
```

f can be a fixed analytic function, a parametrized family, or even a neural network $f_\theta : \mathbb{S}^2 \to \mathbb{R}^3$.

Example 9.2 (Analytic examples)**.** The saddle $f((u, v)) = (u, v, v^2 - u^2)$; and the trefoil-knot tube of slide 38:

```
def trefoil(u, v):
    # r is just a float scale factor
    x = r * np.sin(3*u) / (2 + np.cos(v))
    y = r * (np.sin(u) + 2*np.sin(2*u)) / (2 + np.cos(v + np.pi*2/3))
    z = r/2 * (np.cos(u) - 2*np.cos(2*u)) * (2 + np.cos(v)) * (2 + np.cos(v + np.pi
    *2/3)) / 4
```

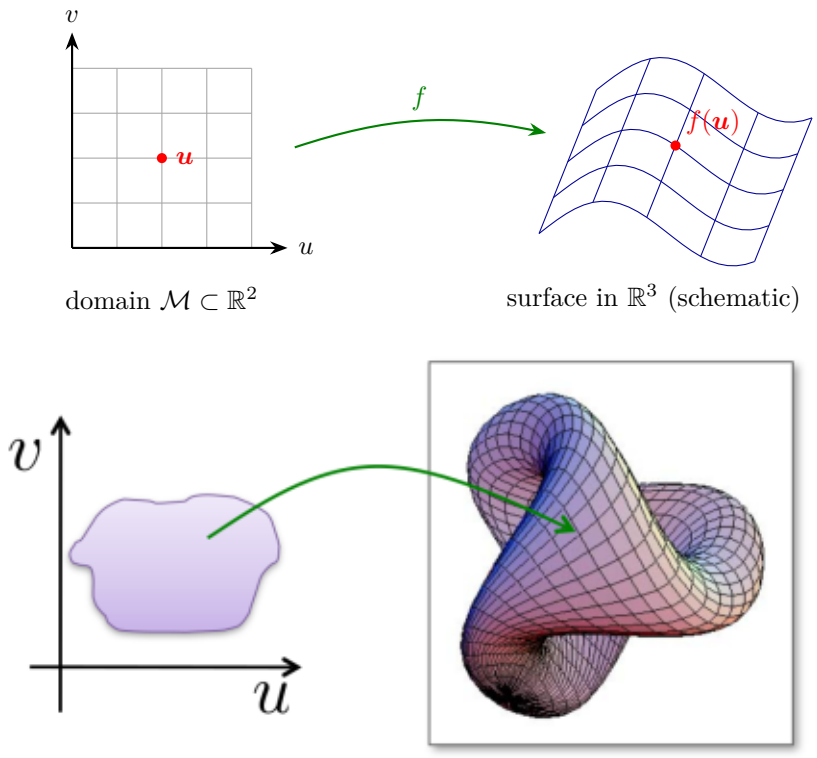


Figure 20: A parametrization warps a flat domain into a surface: grid lines of (u, v) become curves on the surface, and neighbouring parameters stay neighbouring.

Definition 9.3 (Bézier surface)**.** With $(N+1)(M+1)$ control points $\boldsymbol{k}_{i,j} \in \mathbb{R}^3$,

$$f((u,v)) = \sum_{i=0}^{N} \sum_{j=0}^{M} B_i^N(u)\, B_j^M(v)\, \boldsymbol{k}_{i,j}, \qquad B_i^N(u) = \binom{N}{i} u^i (1-u)^{N-i}.$$

Here N and M are polynomial *degrees*, not counts. The control points determine a smooth surface in 3D; Bézier patches are commonly used for designing meshes.

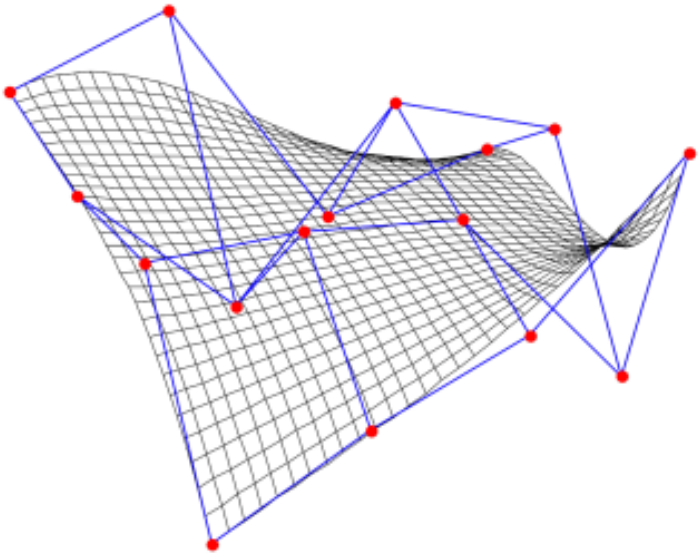


Figure 21: A Bézier surface

Example 9.4 (The bilinear patch, $N = M = 1$)**.** $B_0^1(u) = 1 - u$, $B_1^1(u) = u$, so $f = (1-u)(1-v)\boldsymbol{k}_{00} + (1-u)v\,\boldsymbol{k}_{01} + u(1-v)\boldsymbol{k}_{10} + uv\,\boldsymbol{k}_{11}$. With $\boldsymbol{k}_{00} = (0,0,0)$, $\boldsymbol{k}_{10} = (1,0,0)$, $\boldsymbol{k}_{01} = (0,1,0)$, $\boldsymbol{k}_{11} = (1,1,1)$: $f(0.5, 0.5) = \frac{1}{4}[(1,0,0) + (0,1,0) + (1,1,1)] = (0.5, 0.5, 0.25)$.

Intuition

Control points act like a coarse cage of pins: move a pin and the smooth sheet follows. The surface is a weighted average of the pins, with weights (Bernstein polynomials) that sum to one, so it stays inside the convex hull of the cage. That is why the same $(N+1)(M+1)$ numbers describe an entire smooth surface no matter how finely you later sample it.

9.3 Parametric Surfaces — Pros and Cons *(slides slide 41)*

- **Sampling is easy:** pick $\boldsymbol{u}$, evaluate f.
- **Computation in the domain:** neighbours, texture coordinates and so on are just nearby $\boldsymbol{u}$ values.
- **"Is $\boldsymbol{q}$ inside?" is hard:** you would have to search for the $\boldsymbol{u}$ whose $f(\boldsymbol{u})$ is closest to $\boldsymbol{q}$, an optimization problem.
- **Rendering is hard:** a ray hits the surface where $\boldsymbol{o} + t\boldsymbol{d} = f(\boldsymbol{u})$, a nonlinear system in $(t, \boldsymbol{u})$. In practice you convert to a mesh first.

Intuition

Generating is a *forward* computation (plug in, read off); testing or intersecting needs the *inverse* of f, which is generally not available in closed form. The price of the parametrization is also its topology: the surface has whatever topology $\mathcal{M}$ has (a sphere domain can only give sphere-like shapes).

10 Surface Representations *(slides slide 42)*

Points, meshes and parametric surfaces all describe the "skin" of an object, and none of them can turn a torus into a sphere continuously. So what can we do?

Intuition

All three fix the topology in their data (the connectivity F, the domain $\mathcal{M}$, or the absence of connectivity). To let shapes change topology smoothly, one needs a representation that assigns a value to *every point of space* rather than tracking the skin.

11 Volume Representations *(slides slides 43–46)*

Volume representations give a value to every point in space.

11.1 Voxelized 3D *(slides slides 44–46)*

Definition 11.1 (Voxel grid)**.** A voxel grid is a discretized representation of the space demarcated via cuboid endpoints $(x_{\min}, y_{\min}, z_{\min})$ and $(x_{\max}, y_{\max}, z_{\max})$: a $W \times H \times D$ array $V[i, j, k] \in [0, 1]$ storing the occupancy (or probability of occupancy) of each cell.

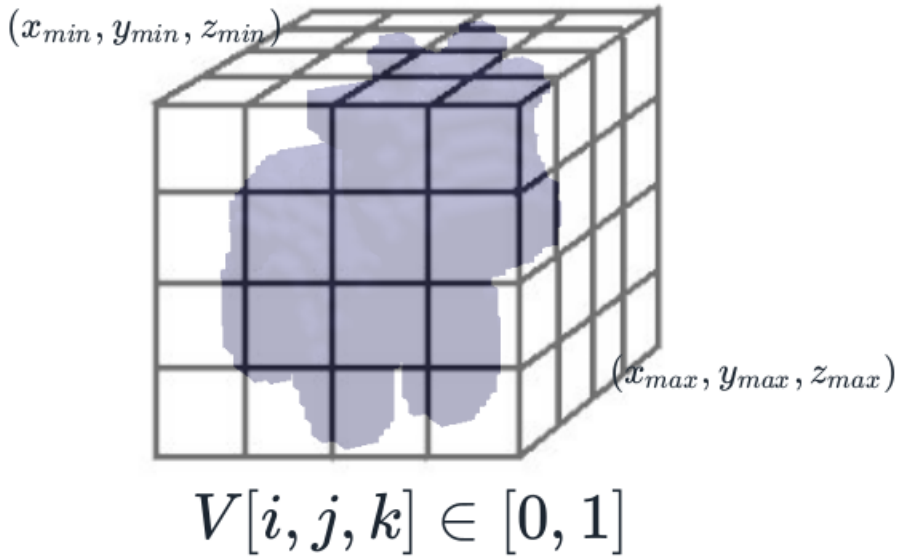


Figure 22: A $4 \times 4 \times 4$ voxel grid inside a cuboid; shaded cells are occupied.

Coding tip: disentangle *grid coordinates* from the *spatial extent in the world*. One common convention: cell size $h = (x_{\max} - x_{\min})/W$ along x, and cell i has its centre at $x_{\min} + (i + \frac{1}{2})h$; the same for y, z. For $x_{\min} = -1.5$, $x_{\max} = 1.5$, $W = 128$: $h \approx 0.0234$, and the point $x = 0.3$ falls in cell $i = \lfloor (0.3 + 1.5)/h \rfloor = 76$.

Because the representation is "image-like", *convolutions* can be used for processing and generation. It is a generic way to represent shapes *across topologies*: the sphere, the torus and the cow of slide 45 all use the same array shape, and topology is just which cells are switched on. A fixed-size output is exactly what networks like.

Resolution is expensive *(slides slide 46)*

It is computationally challenging to scale resolution ($32^3, 64^3, 128^3, 256^3$ on the slide). Worse, the surface passes through only about n^2 of the n^3 cells, so almost all the memory describes empty space or solid interior.

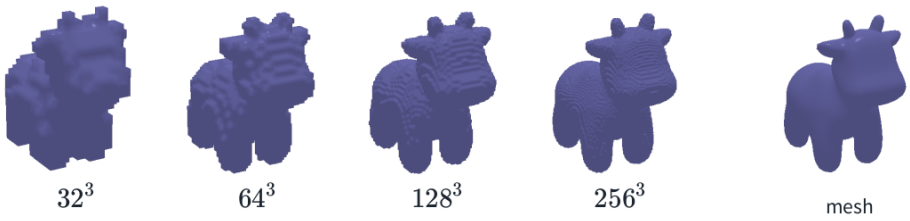


Figure 23: A rappresentation of how resolution affects surface detail. The surface passes through only about n^2 of the n^3 cells, so almost all the memory describes empty space or solid interior.

Example 11.2 (Memory of a voxel grid)**.**

n	cells n^3	surface cells $\approx n^2$	fraction $\approx 1/n$
32	32 768	1 024	3%
64	262 144	4 096	1.6%
128	2 097 152	16 384	0.8%
256	16 777 216	65 536	0.4%

At one byte per cell, 256^3 is 16 MB (64 MB with 32-bit floats) for a single object.

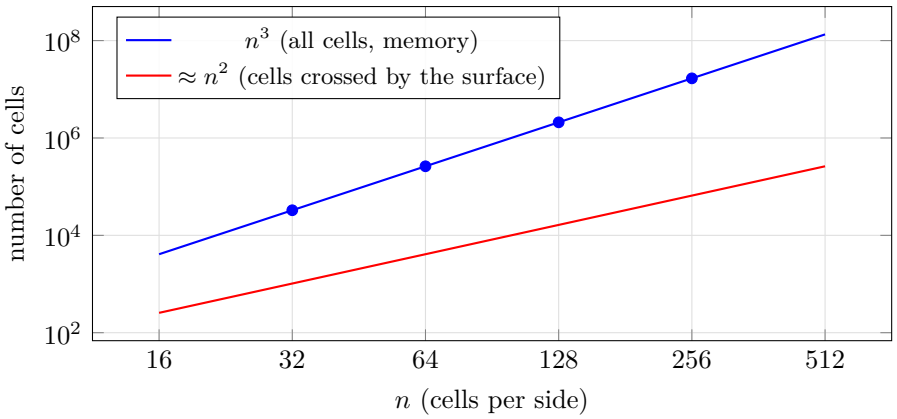


Figure 24: Doubling the resolution multiplies memory by 8 but the useful (surface) part only by 4.

Intuition

A voxel grid is a bookkeeping device: it pays the same for air as for the object. Doubling the resolution costs 8× more memory but only 4× more surface. The benefit is simplicity: any topology, fixed size, convolutions apply. The cost is that fine detail is very expensive, which motivates adaptive versions (octrees, slide 68) and continuous fields (next section).

12 Implicit Surfaces *(slides slides 47–62)*

12.1 Implicit surfaces *(slides slide 47)*

Definition 12.1 (Implicit surface)**.** A surface is the *zero level set* of a scalar field $f : \mathbb{R}^3 \to \mathbb{R}$:

$$\mathcal{S} = \{\boldsymbol{x} \in \mathbb{R}^3 \mid f(\boldsymbol{x}) = 0\}.$$

The sign tells the side: $f(\boldsymbol{x}) < 0$ inside, $f(\boldsymbol{x}) > 0$ outside (one convention, used for the rest of the lecture).

12.2 Generate vs Test *(slides slide 48)*

The unit circle, written both ways:

	Parametric	**Implicit**
Definition	$g(u) = (\cos u, \sin u)$, $u \in [0, 2\pi)$	$f(x, y) = x^2 + y^2 - 1$
Give me 100 points on it	easy: evaluate g at 100 values of u	hard: you have to search for zeros
Is $(0.5, 0.5)$ inside?	hard: no direct test	easy: $f = -0.5 < 0$, so inside

Parametric representations *generate* points; implicit representations *test* points.

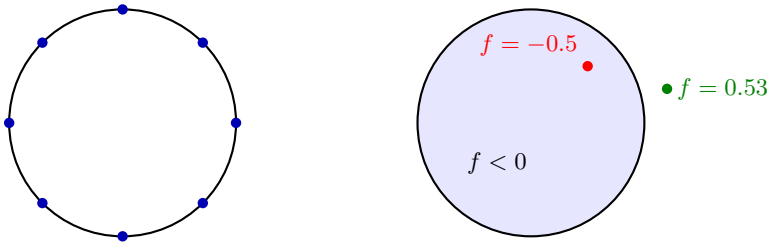


Figure 25: Left: the parametric circle produces points. Right: the implicit circle classifies any point by the sign of f.

Intuition

Parametric ≈ a machine that *prints* surface points; implicit ≈ a *bouncer* that checks tickets. Printing 100 points is trivial with the printer and awkward with the bouncer (you have to hunt for the zeros); checking a single point is trivial with the bouncer and awkward with the printer (you must invert g).

12.3 Implicit Surfaces: Examples *(slides slide 49)*

Simple functions can give concise representations of complex shapes:

$$\begin{aligned}
\text{sphere:} \quad & f(x, y, z) = x^2 + y^2 + z^2 - 1, \\
\text{torus:} \quad & f(x, y, z) = (x^2 + y^2 + z^2 + R^2 - a^2)^2 - 4R^2(x^2 + y^2), \\
\text{quartic (blob with a hole):} \quad & f(x, y, z) = x^4 + y^4 + z^4 + 2x^2y^2 + 2x^2z^2 \\
& \qquad + 2y^2z^2 + 8xyz - 10x^2 - 10y^2 - 10z^2 + 20.
\end{aligned}$$

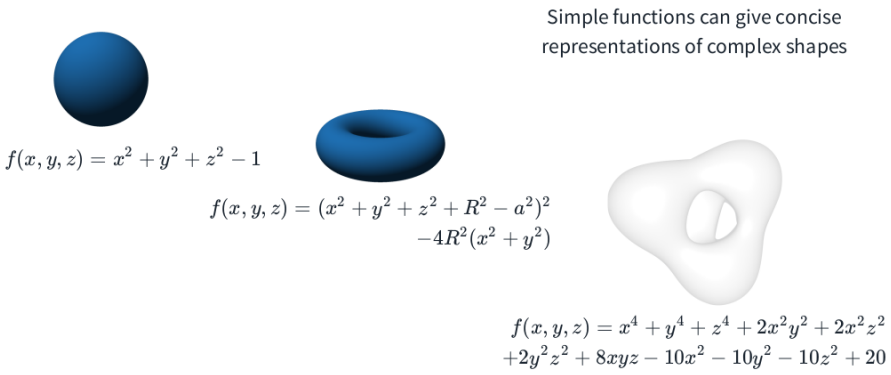


Figure 26: Implicit surfaces: sphere, torus, and a quartic with a hole. The last one is a single polynomial of degree 4, yet it has a hole in it.

12.4 When a Field Defines a Surface *(slides slide 50)*

Not every field defines a surface. Four test cases (with $r = \|\boldsymbol{x}\|$):

$f(\boldsymbol{x})$	zero set	a surface?
$\|\boldsymbol{x}\| - 1$	unit sphere	yes
$\|\boldsymbol{x}\|^2 + 1$	empty	no, there is nothing there
$\|\boldsymbol{x}\|^2$	the origin	no, a single point
$(\|\boldsymbol{x}\| - 1)^2$	unit sphere	no: $f \geq 0$ everywhere, so no inside, and $\nabla f = 0$ on it

Proposition 12.2 (When f defines a surface)**.** *f must change sign across $\mathcal{S}$, and $\nabla f \neq 0$ on $\mathcal{S}$. (If f is continuously differentiable and $\nabla f \neq 0$ at every zero of f, the implicit function theorem shows that $\mathcal{S}$ is locally the graph of a smooth function, i.e. a smooth surface.)*

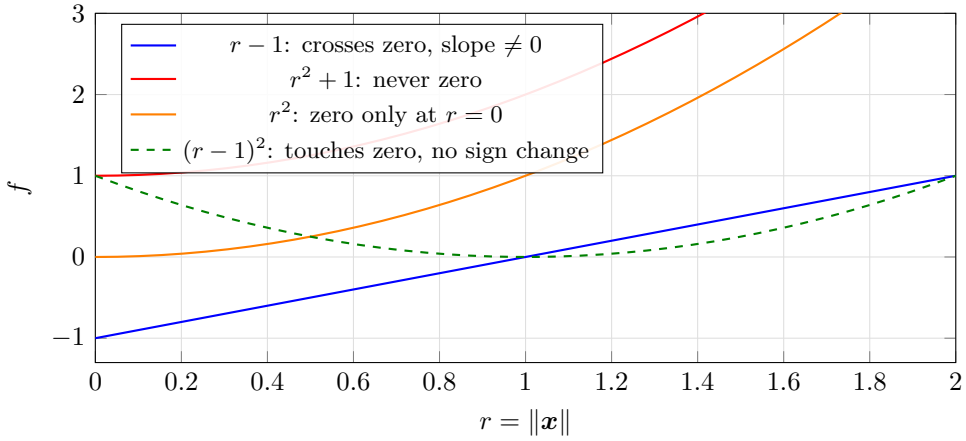


Figure 27: Radial profiles of the four candidate fields. Only the first one changes sign at the unit sphere with a non-zero slope.

Intuition

A surface is where the field *crosses* zero, like the shoreline where altitude crosses sea level. A field that only touches zero (the last row) gives a double root: no "inside", no well-defined normal. A field that never reaches zero gives nothing.

12.5 Normals for Free *(slides slide 51)*

Lemma 12.3 (Normal of an implicit surface)**.** *The gradient is perpendicular to the level sets, so on the surface*

$$\boldsymbol{n}(\boldsymbol{x}) = \frac{\nabla f(\boldsymbol{x})}{\|\nabla f(\boldsymbol{x})\|}.$$

It points toward increasing f, which by our convention is outward.

Proof. Let $\gamma(t)$ be any curve lying in $\mathcal{S}$. Then $f(\gamma(t)) = 0$ for all t; differentiating, $\nabla f(\gamma(t)) \cdot \gamma'(t) = 0$, so ∇f is orthogonal to every tangent direction of the surface. □

For the sphere, $\nabla f = 2\boldsymbol{x}$ and $\boldsymbol{n} = \boldsymbol{x}/\|\boldsymbol{x}\|$. With automatic differentiation the normals of *any* differentiable f (an MLP, say) come for free:

```
x = pts.clone().requires_grad_(True)          # (N, 3) points
v = f(x)                                      # any differentiable f, e.g. an MLP
g, = torch.autograd.grad(v.sum(), x)
n = g / g.norm(dim=-1, keepdim=True)          # (N, 3) unit normals
```

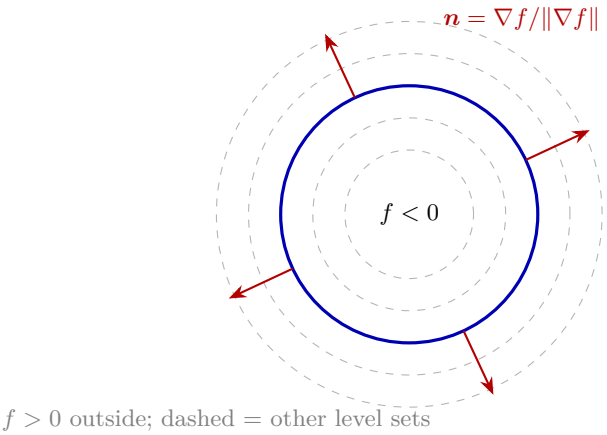


Figure 28: The gradient of f is orthogonal to the level sets and points toward larger f, i.e. out of the shape.

Intuition

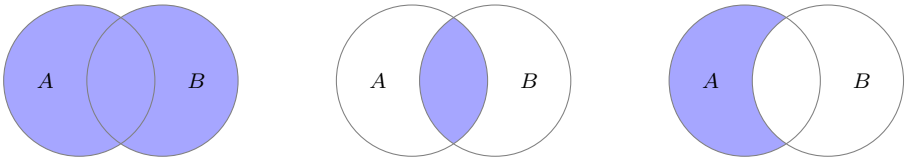
Picture f as terrain height and the surface as the zero contour line. The gradient points straight uphill, perpendicular to any contour line: that is the normal. No separate normal array has to be stored or kept consistent, unlike the per-face orientation of a mesh.

12.6 Boolean Operations *(slides slide 52)*

Proposition 12.4 (Booleans via min and max)**.** *With $f < 0$ inside,*

$$f_{A\cup B} = \min(f_A, f_B), \qquad f_{A\cap B} = \max(f_A, f_B), \qquad f_{A\setminus B} = \max(f_A, -f_B).$$

Proof. $\boldsymbol{x} \in A\cup B \iff f_A(\boldsymbol{x}) < 0$ or $f_B(\boldsymbol{x}) < 0 \iff \min(f_A, f_B) < 0$. Likewise $\boldsymbol{x} \in A\cap B \iff f_A < 0$ and $f_B < 0 \iff \max(f_A, f_B) < 0$. For $A\setminus B = A\cap B^c$, the complement of B is described (up to its boundary) by $-f_B < 0$, so $\max(f_A, -f_B) < 0$. □



$A \cup B$: $\min(f_A, f_B)$ $A \cap B$: $\max(f_A, f_B)$ $A \setminus B$: $\max(f_A, -f_B)$

Figure 29: 2D versions of the three Boolean operations (blue = result).

Intuition

"Inside the union" means "inside at least one": the most negative of the two values decides, hence min. "Inside the intersection" needs both, so the *least* negative value decides: max. Subtraction is intersection with the outside of B, and flipping the sign of f_B swaps inside and outside. The *sign* of the result is always correct; the magnitude near sharp creases is not guaranteed to remain an exact distance (see the SDF discussion next).

12.7 Many Functions, One Surface *(slides slide 53)*

The surface does not fix f: many different fields have the same zero set. So one can choose an f that is useful *away* from the surface.

12.8 Signed Distance Function *(slides slides 54–58)*

Definition 12.5 (Signed distance function (SDF))**.**

$$f(\boldsymbol{x}) = \begin{cases} -\operatorname{dist}(\boldsymbol{x}, \mathcal{S}) & \boldsymbol{x} \text{ inside} \\ +\operatorname{dist}(\boldsymbol{x}, \mathcal{S}) & \boldsymbol{x} \text{ outside} \end{cases}, \qquad \operatorname{dist}(\boldsymbol{x}, \mathcal{S}) = \min_{\boldsymbol{y} \in \mathcal{S}} \|\boldsymbol{x} - \boldsymbol{y}\|.$$

Which of the fields of slide 55 are SDFs? $x^2 + y^2 + z^2 - 1$ and the quartic of slide 49: *neither*!

Proposition 12.6 (Eikonal condition)**.** *An SDF satisfies* $\|\nabla f\| = 1$ *(wherever it is differentiable).*

Proof. Take a small step of length δ from a point. In *any* direction the distance to the surface changes by at most δ; stepping straight away from the closest surface point it changes by exactly δ. So the steepest slope of f is exactly one. □

Example 12.7. $f = \|\boldsymbol{x}\| - 1$: $\nabla f = \boldsymbol{x}/\|\boldsymbol{x}\|$, $\|\nabla f\| = 1$, an SDF. $f = \|\boldsymbol{x}\|^2 - 1$: $\nabla f = 2\boldsymbol{x}$, $\|\nabla f\| = 2\|\boldsymbol{x}\| \neq 1$, not an SDF.

Intuition

The eikonal condition says "the field grows like a ruler as you walk away from the surface". An SDF is the most *honest* field: its value literally is the distance to the surface. The other fields agree on *where* the surface is but exaggerate or shrink the distances; $r^2 - 1$ overstates by the factor $r + 1$ at radius r.

Why honesty matters: safe steps *(slides slides 57–58)*.

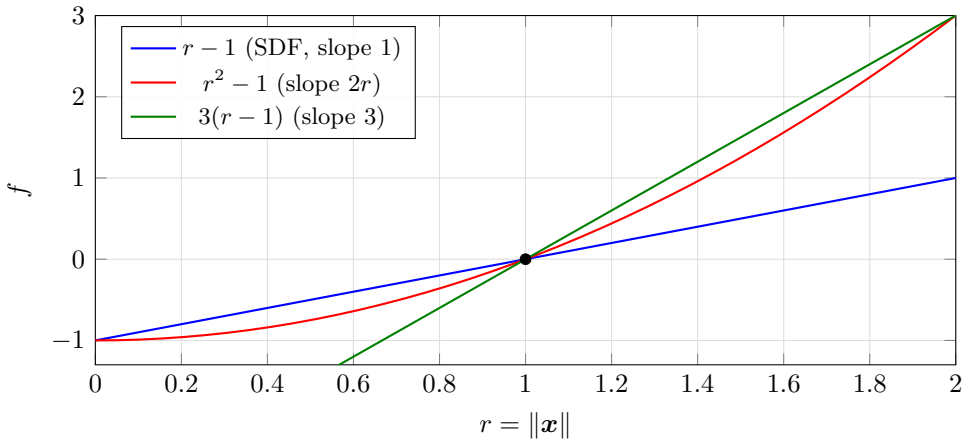


Figure 30: Three fields with the same zero set (the unit sphere, $r = 1$). Only the slope-one profile is a signed distance function.

Proposition 12.8 (Safe step)**.** *If f is an SDF, the ball of radius $|f(\boldsymbol{x})|$ around $\boldsymbol{x}$ contains no surface point in its interior. Hence a ray can safely advance by $|f(\boldsymbol{x})|$. If f overstates the distance, the ball can cross the surface and stepping by f can* jump through *it.*

Proof. By definition $|f(\boldsymbol{x})| = \operatorname{dist}(\boldsymbol{x}, \mathcal{S})$, the radius of the largest ball around $\boldsymbol{x}$ that avoids $\mathcal{S}$. For a non-SDF field the ball of radius $|f|$ can be larger than $\operatorname{dist}(\boldsymbol{x}, \mathcal{S})$ and then contains surface points. □

Example 12.9 (Slides 57–58, $\boldsymbol{x}$ at $r \approx 2.67$)**.** For $f = r^2 - 1$: $f = 6.14$, true distance 1.67, $\|\nabla f\| = 5.34$: the circle of radius $|f|$ crosses the surface. For an SDF at the same point the ball of radius $r - 1 = 1.67$ touches the surface without crossing.

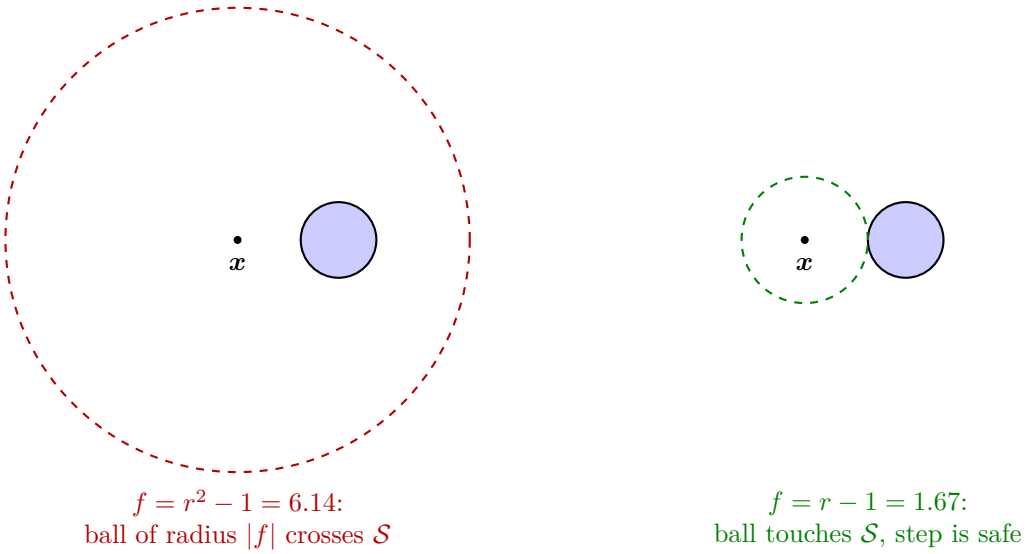


Figure 31: The same point $\boldsymbol{x}$ with two fields that define the same sphere. Left: stepping by $f = r^2 - 1$ overshoots. Right: an SDF gives the largest safe step.

Algorithm 12.1 (Sphere tracing / ray marching an SDF)**.** Given a ray $\boldsymbol{o} + t\boldsymbol{d}$ ($\|\boldsymbol{d}\| = 1$) and an SDF f: set $t \leftarrow 0$; repeat $t \leftarrow t + f(\boldsymbol{o} + t\boldsymbol{d})$ until $f < \varepsilon$ (hit) or t exceeds a maximum

(miss).

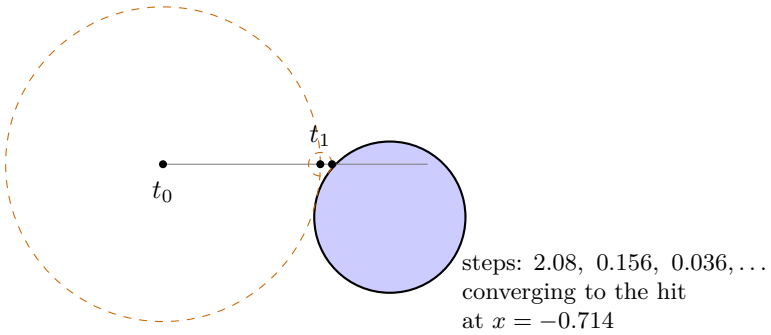


Figure 32: Sphere tracing a horizontal ray at height 0.7 against the unit sphere: each step is the SDF value, the radius of the empty ball.

Intuition

It is like walking through fog toward a wall with a distance meter: if the meter never lies (SDF) you can always walk the displayed distance without hitting the wall, and the steps shrink as you approach. A meter that overstates makes you walk into and through the wall. This is why the table on slide 66 lists "ray marching" as the way to render implicit surfaces.

12.9 Storing a Field on a Grid *(slides slide 59)*

Store samples $F[i, j, k] = f(\boldsymbol{x}_{ijk})$ at the grid points and interpolate in between: the same data structure as a voxel grid, but holding *signed distances* rather than occupancy. A *TSDF* keeps values only near the surface and clamps them to $\pm\tau$ elsewhere, $f_\tau = \text{clamp}(f, -\tau, \tau)$; it is the standard way to fuse many depth maps into one model, as in KinectFusion.

```
xs = np.linspace(-1.5, 1.5, 128)
X, Y, Z = np.meshgrid(xs, xs, xs, indexing="ij")
F = np.sqrt(X**2 + Y**2 + Z**2) - 1.0           # unit-sphere SDF on a 128^3 grid
```

Intuition

The grid now stores *how far* each cell is from the surface instead of *whether* it is occupied. Values change smoothly across the surface, so interpolation places the surface at sub-voxel accuracy, and the clamped version stores only what matters: a thin shell around the surface.

12.10 Storing a Field in a Network *(slides slide 60)*

$\{\boldsymbol{x} \mid f_\theta(\boldsymbol{x}) = 0\}$, with f_θ a small MLP (three numbers in, one out). Two requirements: $f_\theta(\boldsymbol{x}_i) \approx 0$ on the data points (the surface passes through the data), and $\|\nabla f_\theta(\boldsymbol{x})\| \approx 1$ everywhere (it behaves like an SDF). Resolution is not tied to a grid.



Figure 33: A small MLP can represent a continuous field f_θ, whose zero set is the surface.

Example 12.10 (One way to write the two requirements as a loss)**.** $\mathcal{L}(\theta) = \frac{1}{N}\sum_i |f_\theta(\boldsymbol{x}_i)| + \lambda\, \mathbb{E}_{\boldsymbol{x}}\big[(\|\nabla f_\theta(\boldsymbol{x})\| - 1)^2\big]$. The first term pulls the zero set through the samples; the second is the eikonal penalty of Proposition 7.2.

Intuition

The network is a *compressed*, resolution-free grid: query it at any point, at any density. The eikonal penalty is what stops the network from producing an arbitrary function that merely vanishes on the data.

12.11 From Implicit Surface to Mesh *(slides slide 61)*

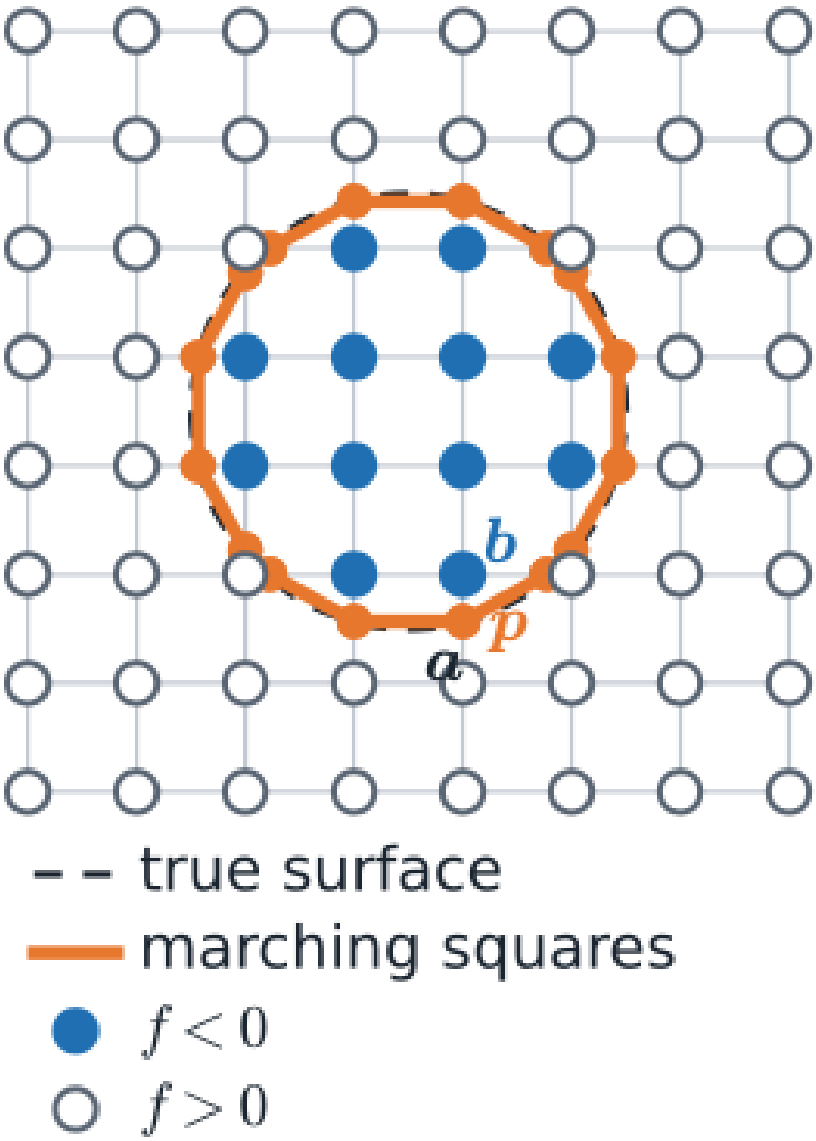


Figure 34: Marching squares (the 2D version of marching cubes) in one cell. The blue corners are inside ($f < 0$),the white corners are outside ($f > 0$). The dotted line is the true surface; the orange line is the interpolated surface from marching squares.

Algorithm 12.2 (Marching cubes)**. 1.** Evaluate f on a grid (analytic, a TSDF, or a network).
2. Find the cells whose corners differ in sign.
3. On each such edge $[\boldsymbol{a}, \boldsymbol{b}]$, place a vertex at the zero crossing $p = \boldsymbol{a} + t(\boldsymbol{b} - \boldsymbol{a})$ with $t = \dfrac{f(\boldsymbol{a})}{f(\boldsymbol{a}) - f(\boldsymbol{b})}$.
4. In 3D, join the vertices with a lookup table on the corner signs: 2^8 cases per cube.

```
from skimage.measure import marching_cubes
h = xs[1] - xs[0]                                  # voxel size, from the grid above
verts, faces, normals, _ = marching_cubes(F, level=0.0, spacing=(h, h, h))
```

```
verts += xs[0]                                    # grid -> world coordinates
```

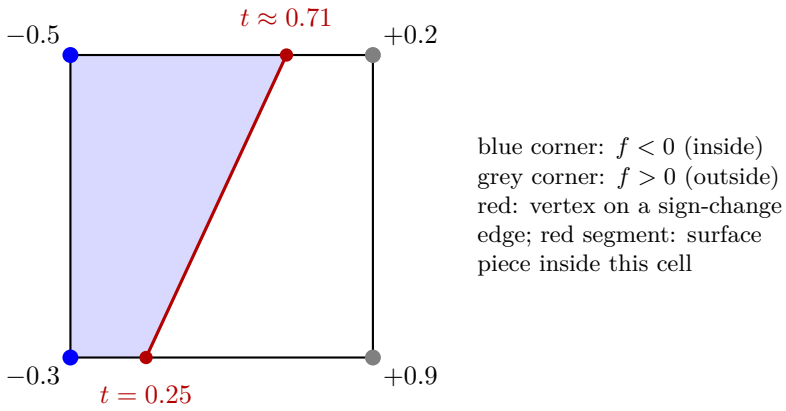


Figure 35: Marching squares (the 2D version of marching cubes) in one cell. On the bottom edge $t = \frac{-0.3}{-0.3-0.9} = 0.25$; on the top edge $t = \frac{-0.5}{-0.5-0.2} \approx 0.71$. The left and right edges have no sign change, hence no vertex.

Intuition

Marching cubes is "connect the dots along the shoreline": the corner signs say which cell edges the coastline crosses; linear interpolation of f along that edge says *where* (a corner with a small $|f|$ is close to the surface, so the vertex sits close to it). This is the bridge from implicit to mesh, which is what GPUs render.

12.12 Implicit Surfaces: Summary *(slides slide 62)*

Easy	Hard
inside or outside: the sign of f	getting points on the surface
any topology, changing continuously	no direct rasterization
booleans: a min or a max	an arbitrary f may define nothing
normals: ∇f	

The mirror image of parametric surfaces.

13 Boundary vs Volumetric Representations *(slides slide 63)*

Slide 63 summarizes the two families: *boundary* representations for shapes (point clouds, meshes, parametric surfaces) and *volumetric* representations for shapes (voxels, implicit fields).

14 Density Fields *(slides slides 64–65)*

14.1 Density fields *(slides slide 64)*

Occupancy asks "is this point inside?"; density asks "how opaque is this point?".

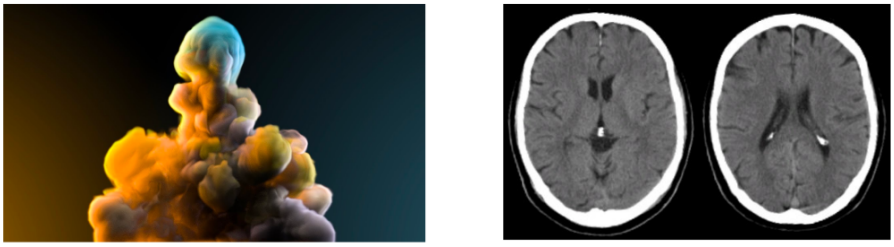


Figure 36: Two images examples

Definition 14.1 (Density field)**.** A density field assigns $\sigma(\boldsymbol{x}) \in \mathbb{R}^+$ (or $\sigma[i,j,k] \in \mathbb{R}^+$ on a grid); $\sigma(\boldsymbol{x}) \geq 0$ says how much light is absorbed per unit length at $\boldsymbol{x}$.

There is no notion of a surface: e.g. $\sigma(\boldsymbol{x}) = 0.1$ everywhere represents a semi-transparent fluid. Typical examples on the slide: smoke and CT-scan slices. The lecture will revisit this in more detail later (NeRF).

Example 14.2 (Preview: how much light survives?)**.** If a beam crosses a slab of length L of constant density σ, the fraction of light transmitted is $e^{-\sigma L}$ (absorption per unit length compounds multiplicatively). For $\sigma = 0.1$ and $L = 10$: $e^{-1} \approx 0.37$. (This is a standard consequence of the "absorbed per unit length" reading; the slides develop it later.)

14.2 Primitive-based Density Fields *(slides slide 65)*

$$\sigma(\boldsymbol{x}) = \sum_k w_k \exp\Big(-\tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_k) \Big).$$

These are *hybrid* representations for primitive-induced density fields (blobs/Gaussians with weights w_k, centres $\boldsymbol{\mu}_k$, covariances Σ_k); the lecture will revisit them in more detail (differentiable Gaussian rendering).

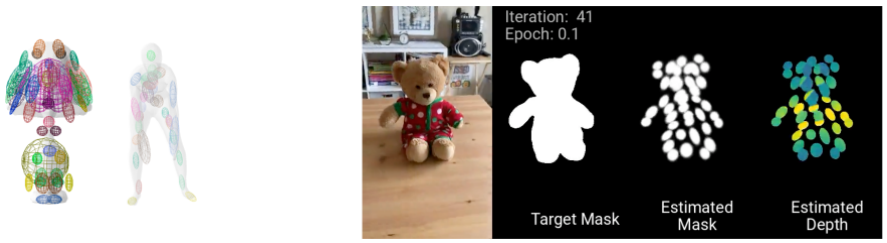


Figure 37: Images examples of primitive-based density fields

Intuition

Instead of a grid of numbers, the density is a *cloud of soft blobs*: each blob has a position, a size/orientation (via Σ_k) and a weight. The whole field can be edited by moving blobs, and only regions with blobs cost anything, so it avoids the "pay for empty space" problem of dense grids.

15 So? *(slides slide 66)*

Slide 66 compares the representations side by side (its cells are colour-coded easy / possible with effort / hard; the colours do not survive in the text of the PDF, so the qualitative ratings appear

in Section 17, based on the earlier slides):

	Points on the surface	**Inside or outside?**	**Continuous edits reach**	**Rendering**	**Typical use**
Point cloud (array $N \times 3$)	given	no surface	any point set	splats	sensors: LiDAR, RGB-D
Mesh ($V : N \times 3$, $F : M \times 3$)	sample triangles	count ray crossings	same wiring only	GPU rasterizer	graphics, games
Parametric ($f(\boldsymbol{u})$)	evaluate f	search over $\boldsymbol{u}$	same topology only	via a mesh	CAD, design
Voxels (grid $W \times H \times D$)	marching cubes	look up the cell	any topology*	blocky	3D CNNs, occupancy maps
Implicit (SDF) ($f(\boldsymbol{x})$: formula, grid, net)	marching cubes	sign of f	any topology	ray marching	reconstruction, neural fields

* with occupancy probabilities, not 0 or 1.

Intuition

No row wins every column: pick the representation for the task. Need to shoot rays fast on a GPU? Mesh. Need to change topology or run inside/outside queries? Implicit. Data straight from a LiDAR? Points. Feeding a 3D CNN? Voxels.

16 And many more... *(slides slides 67–71)*

16.1 Octrees *(slides slide 68)*

Octrees are *voxels with adaptive resolution*: surface-like efficiency for rendering and storage, volume-like efficiency for occupancy queries; but it is difficult to edit or predict the sparsity structure.

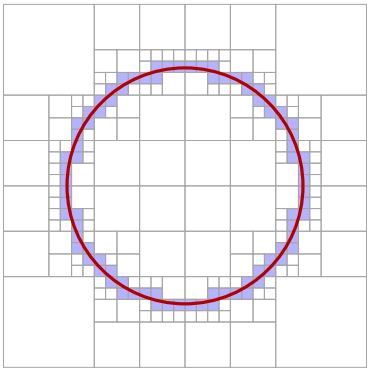


Figure 38: A quadtree (2D octree) for a disk of radius 0.65 in $[-1, 1]^2$, subdivided down to 5 levels only where the boundary passes (blue). It uses 244 leaf cells; a uniform grid of the same finest resolution (32×32) uses 1024.

Intuition

An octree spends resolution where the shape is: big cells for air and interior, tiny cells on the boundary. The trade-off is structural: the tree *shape* depends on the data, which is hard to edit or to predict with a network (the same sparse/dense tension seen with meshes' F).

16.2 Layered Depth Images *(slides slide 69)*

Definition 16.1 (Layered depth image (LDI))**.** Layer 1 is the visible content (depth and colour per pixel); layer $N{+}1$ stores the depth and colour of the *next* surface point the ray hits after layer N. This extends depth to capture "full" 3D, but the layers are often very discontinuous.

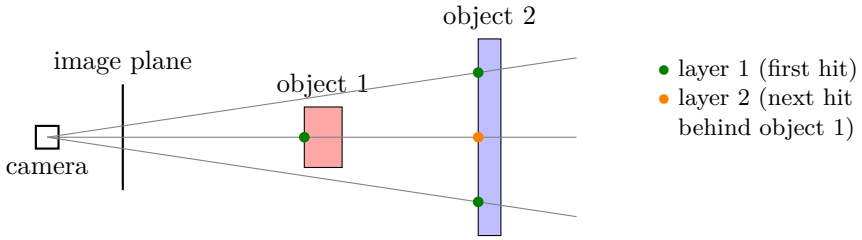


Figure 39: Three pixel rays. The upper and lower rays hit only object 2 (layer 1 only); the middle ray hits object 1 first and object 2 second, so its pixel stores two (depth, colour) entries.

Intuition

A depth map keeps only the first hit along each ray; an LDI keeps a *list* of hits. The hidden second layer (what is behind object 1) is what allows rendering from a slightly different viewpoint without holes; but the layer index jumps around from pixel to pixel, so layers are not smooth images.

16.3 Multi-plane Images *(slides slide 70)*

Multi-plane images (MPIs) are discrete volumetric representations with *non-uniform* spatial cells: a stack of fronto-parallel planes at fixed depths, each a (colour, transparency) image. They are extremely efficient to render (each layer is just a plane at fixed depth), but have limited resolution for far-away content; uniform cells may be preferable for object-centric 3D.

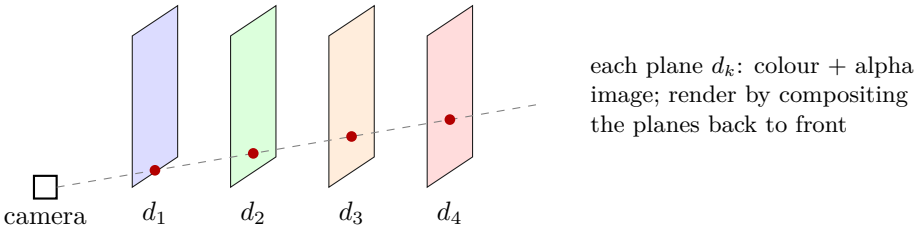


Figure 40: A multi-plane image: planes at increasing depth $d_1 < \cdots < d_4$; a pixel ray crosses all of them and the result is their composite.

Intuition

An MPI is a pile of transparent slides at fixed depths: to draw the scene from a nearby viewpoint, shift and blend the slides. Planes are spaced non-uniformly (dense near, sparse

far), which is why distant content gets little resolution.

16.4 CAD Models and CSG Trees *(slides slide 71)*

CAD models and CSG (constructive solid geometry) trees are *compositions of primitive volumes or surfaces*. They are popular across design applications but challenging for learning.

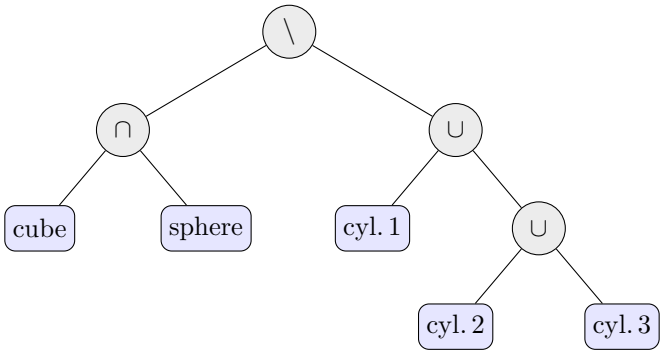


Figure 41: A CSG tree: leaves are primitives, inner nodes are Boolean operations (as on slide 71: $(\text{cube} \cap \text{sphere}) \setminus (\text{cyl.}_1 \cup (\text{cyl.}_2 \cup \text{cyl.}_3))$).

Example 16.2 (A CSG tree is a nested min / max)**.** If each primitive has an implicit function $(f_{\text{cube}}, f_{\text{sph}}, f_{c1}, f_{c2}, f_{c3})$, the tree of the figure is the field

$$f = \max\Big(\max(f_{\text{cube}}, f_{\text{sph}}),\ -\min\left(f_{c1}, \min(f_{c2}, f_{c3})\right)\Big),$$

by the Boolean rules of slide 52.

Intuition

A CSG model is a *program*: a recipe of primitives and Boolean steps. It is compact, editable and exact, but the structure (which tree, how deep) is discrete and combinatorial, so it is hard to predict with a neural network by gradient descent.

17 Summary *(slides slides 72–74)*

The summary slide (72) repeats the zoo of slide 4: mesh, point cloud, CAD model, voxels, implicit surface $\{\boldsymbol{x} \mid f(\boldsymbol{x}) = 0\}$, parametric surface $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^3$, CSG tree, and many more. Slides 73–74 are "Questions?" and Acknowledgments: the slides are adapted from *Learning for 3D Vision*, CMU 16-825, Prof. Shubham Tulsiani.

Comparison table

	Get surface points	Inside / outside	Change topology continuously	Render directly	Memory scales with
Point cloud	given	no surface	any point set	splats	points (N)
Triangle mesh	sample triangles	count ray crossings	same wiring only	rasterizer	vertices, faces
Parametric	evaluate f	search over $\boldsymbol{u}$	same topology only	via a mesh	parameters of f
Voxels	marching cubes	look up cell	with probabilities	blocky	n^3
Implicit / SDF	marching cubes	sign of f	any topology	ray marching	grid n^3 or network

easy possible with effort hard / limited. Ratings are our reading of slides 22–62 and the table of slide 66, not a table from the slides.

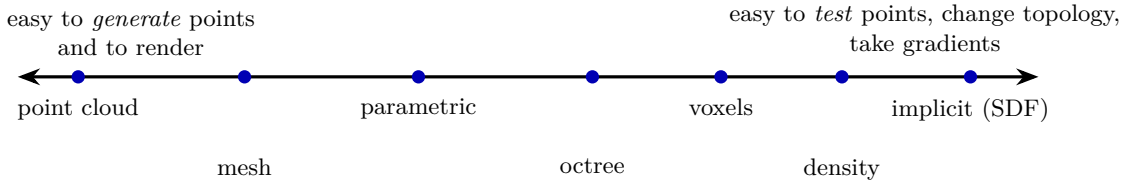


Figure 42: A qualitative map of the "generate vs. test" spectrum (our summary of slides 41, 48, 62 and 66; positions are indicative only).

Key take-aways

- **There is no universal 3D representation.** Points and depth maps come from sensors; meshes are for rendering and graphics; implicit fields are for inside/outside tests, topology changes and learning; the right choice depends on the task.
- **Parametric generates, implicit tests.** $f(\boldsymbol{u}) = \boldsymbol{x}$ makes sampling trivial and queries hard; $\{f(\boldsymbol{x}) = 0\}$ makes queries trivial (sign of f, ∇f for normals, min / max for Booleans) and sampling hard.
- **Meshes split geometry from connectivity.** V is continuous and optimizable; F is discrete, so predicting arbitrary connectivity or changing topology is hard; check winding order, and remember $F \approx 2V$ for closed genus-0 meshes.
- **Grids are simple but pay for empty space.** A n^3 voxel grid (or SDF grid) handles any topology and suits convolutions, but only $\sim n^2$ cells touch the surface; octrees and continuous fields (networks, Gaussians) fight this cost.
- **SDFs are the well-behaved implicit fields.** $\|\nabla f\| = 1$ makes $|f|$ the true distance, enabling safe ray-marching steps and TSDF fusion, and marching cubes converts any such field back into a mesh.

Part II

Lecture 3

How to read these notes. Section headers reuse the titles of the slides and are tagged with the slide range they cover. Formal statements (definitions, propositions, theorems, algorithms) are placed where the slides introduce the topic; the slides themselves state most results without proof, so the derivations are added here. Blue *Intuition* boxes explain the "why" in plain words, with worked numbers. All figures are original TikZ/pgfplots drawings; figures marked *schematic* show the qualitative trend described on the slides, not measured data.

1 From Image Formation to Measurement *(slides 1–3)*

Lecture 01 modeled how a scene becomes an image: projection sends a 3D point to a pixel and *discards the distance along the ray.* Lecture 02 covered ways to *represent* a scene: points, meshes, voxels, signed distance functions, Gaussians. Neither lecture covered how to *measure* a scene. This lecture obtains one depth value per pixel (not always – some sensors leave holes) in one of three ways: by measuring it with a laser, by stereo vision, or by *predicting* it with a learned prior.



Section	Title	Slides
3.1	From representations to measurements	4–7
3.2	Three ways to pay for depth	8–12
3.3	Inside a LiDAR	13–17
3.4	From point cloud to depth image	18–19
3.5	From recovery to prediction	20–22
3.6	Cues in closed form	23–26
3.7	Supervised single-view depth	27–35
3.8	Cues used by the network	36–39

Table 1: The agenda (slide 3) with slide ranges.

Intuition

The whole lecture is one story: an image is a *projection*, so depth is missing. Every method for getting it back must *pay* with something – time (a laser pulse), a baseline (a second viewpoint), computation (motion), or a prior (assumptions learned from data). The interesting questions are always: what did we pay, and what is therefore *guaranteed* about the answer?

2 From Representations to Measurements *(slides 4–7)*

What one image determines, and what it does not. The two preceding lectures modeled image formation and scene representation. This section separates *recovered* from *ambiguous* quantities and shows which of the two depth from one image is.

2.1 Recovered and Ambiguous Quantities *(slides 5)*

Definition 2.1 (Recovered vs. ambiguous quantity)**.** Every quantity estimated in this lecture falls into one of two cases.

- **Recovered**: the data determine it up to noise. It can be reported in meters with an error bar.
- **Ambiguous**: the data determine it only up to a *family of transformations*. Any single number reported for it comes from an added assumption.

Quantity	Free parameters	What pins it down
metric depth	0	nothing, it is already in meters
up to scale	1 (s)	one known length
up to scale and shift	2 (s, t)	two known depths
ordering only	any monotone map	a calibrated sensor

Table 2: Ambiguity ladder (slide 5). The number of free parameters says how much external information is still needed.

Intuition

Think of the free parameters as *knobs you cannot read from the data*. Metric depth has no knob. "Up to scale" has one knob: the scene may be uniformly inflated or shrunk. "Up to scale and shift" has two. "Ordering only" is the weakest: you know that A is nearer than B, but $(1, 2, 3)$ and $(1, 10, 100)$ tell exactly the same story. The slide's punchline: disagreements about depth estimation are usually disagreements about *which row of this table a quantity lives in*.

2.2 The Back-Projected Ray *(slides 6)*

Projection throws one number away per pixel; inverting it asks for that number back:

$$\boldsymbol{p}^c \;=\; z\,\boldsymbol{K}^{-1}\,\tilde{\mathbf{x}}_s. \tag{1}$$

Here $\tilde{\mathbf{x}}_s$ is the homogeneous pixel (third component 1), $\boldsymbol{K}$ the intrinsics from lecture 01, and $\boldsymbol{p}^c$ the point in the camera frame.

Proposition 2.2 (Projection is not invertible)**.** *The third component of* $\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ *is* 1*, so the free scalar* z *in* (1) *is the depth in meters along the optical axis. Given only* $\boldsymbol{x}_s$*, the set of 3D points that project to it is the whole ray*

$$\{\, z\, \boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s \;:\; z > 0 \,\}.$$

Proof. Third row of $\boldsymbol{K}$ is $(0, 0, 1)$, hence $\boldsymbol{K}\boldsymbol{p}^c = (*, *, p_z^c)^\top$ and $\boldsymbol{p}^c$ projects to $\tilde{\mathbf{x}}_s$ iff $\boldsymbol{K}\boldsymbol{p}^c = \lambda\tilde{\mathbf{x}}_s$ with $\lambda = p_z^c$ (homogeneous division). Solving, $\boldsymbol{p}^c = \lambda\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ and $\lambda = z$. Conversely every $z\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ satisfies $\boldsymbol{K}(z\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s) = z\tilde{\mathbf{x}}_s \sim \tilde{\mathbf{x}}_s$. □

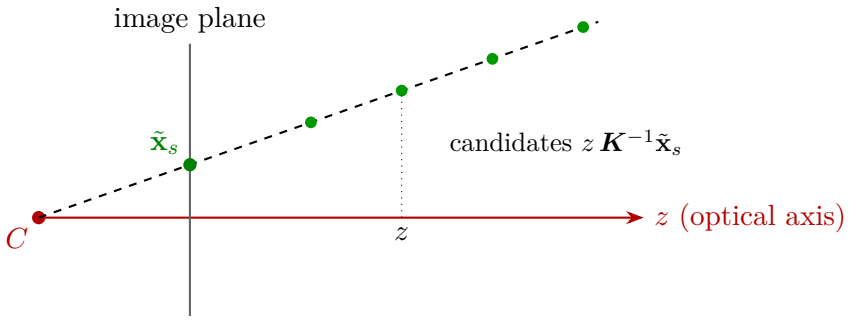


Figure 43: Back-projected ray (redrawn from slide 6; side view, y up). Every green point projects to the same pixel.

Intuition

It is like knowing the *direction* in which a laser pointer is aimed but not how far away the dot lands. The information was *destroyed at capture*, not merely hidden: no clever algorithm can un-destroy it, so every method that produces z is either measuring something extra or importing an assumption. "Producing z at every pixel is the whole of this lecture."

2.3 Scale-Depth Ambiguity *(slides 7)*

Proposition 2.3 (Global scale is unobservable from one image)**.** *Let* $\{\boldsymbol{p}_j\}$ *be a scene in the camera frame and* $s > 0$*. The scaled scene* $\{s\boldsymbol{p}_j\}$ *produces exactly the same image.*

Proof. $\pi(\boldsymbol{K}\, s\boldsymbol{p}) = \pi(s\, \boldsymbol{K}\boldsymbol{p}) = \pi(\boldsymbol{K}\boldsymbol{p})$, because the homogeneous division cancels s. □

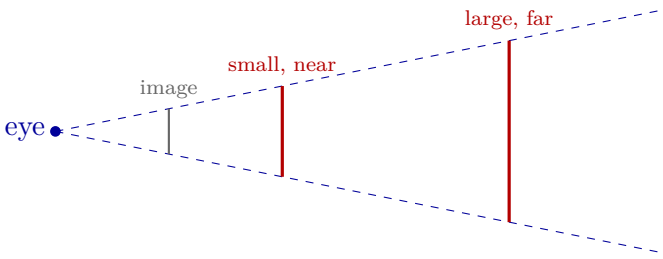


Figure 44: Scale-depth ambiguity (schematic after slide 7): a small near object and an object twice as large at twice the distance subtend the same angle.



Figure 45: two photos of a giant red bench, the second with a bicycle placed on it

Intuition

A small close object looks the same as a much larger one further away. The giant bench in the slide's photographs is a real-life example: on its own it reads as a normal bench, and only the bicycle on top reveals its true size. That bicycle is exactly a *prior about sizes* – the theme of the rest of the lecture.

3 Three Ways to Pay for Depth *(slides 8–12)*

Time, a baseline, a prior. No method returns depth for free. Each buys it with a different resource, and the resource decides what the result is good for.

3.1 Two Measurement Principles *(slides 9)*

Definition 3.1 (Time of flight and triangulation)**.** Two independent choices: the *principle* decides what is measured; the *illumination* decides only whether the sensor brings its own light (*active*) or uses ambient light (*passive*).

- **Time of flight (ToF)**: measure the delay of emitted light.
- **Triangulation**: measure the angular/pixel difference between two viewpoints separated by a known geometry.

	time of flight	triangulation
active (own light)	scanning laser, flash range camera	projected-pattern camera, active stereo
passive (ambient light)	*empty*: timing needs a known emission	stereo with a known baseline, structure from motion

Table 3: The two-by-two of depth sensing (slide 9).



Figure 46: left: speckle pattern of a projected-pattern camera with a stereo camera overlaid; right: a Hesai spinning LiDAR

Intuition

The empty cell is instructive: to time light you must *know when it left*, and ambient light has no known departure time. Also, "active" only helps with texture or timing – it does not change the physics: a projected pattern camera is still triangulation, it just paints texture onto blank walls so matching works.

3.2 Triangulation From Two Views *(slides 10)*

Two rectified cameras sit a baseline b apart. A scene point $\boldsymbol{p} = (x, y, z) \in \mathbb{R}^3$ appears at horizontal pixel offsets u_L and u_R (relative to each principal point), and similar triangles give the depth.

Theorem 3.2 (Depth from disparity)**.** *For two rectified cameras with focal length f (in pixels) and baseline b (in meters), the depth of a point with disparity $d = u_L - u_R$ is*

$$z = \frac{f\,b}{d}.$$

Proof. By the pinhole model $u_L = f\,x/z$ for the left camera, and since the right camera is displaced by b along x, $u_R = f\,(x - b)/z$. Subtracting, $d = u_L - u_R = fb/z$. (The slide draws the image plane behind the pinhole, which produces the minus signs in its two similar-triangle equations; the resulting formula is the same.) □

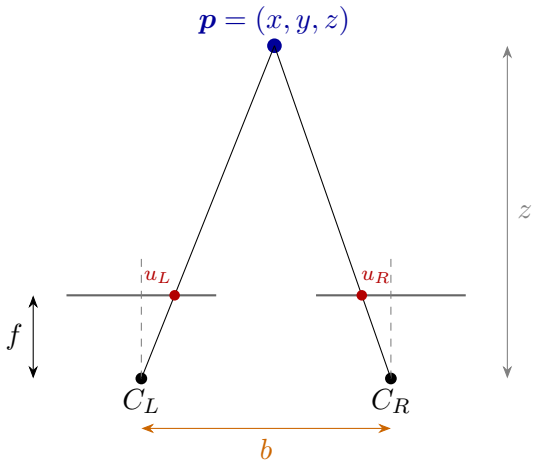


Figure 47: Rectified stereo (redrawn from slide 10, top view). With $f = 1$, $b = 3$, $z = 4$ as drawn: $u_L = 0.4$, $u_R = -0.35$, $d = 0.75 = fb/z$.

Corollary 3.3 (Unknown baseline)**.** *Disparity is inverse depth scaled by fb: $1/z = d/(fb)$. If b is unknown (a moving camera), depth is known only up to one global scale.*

Example 3.4. For $f = 700\,\text{px}$ and $b = 0.5\,\text{m}$, $fb = 350\ \text{px}\,\text{m}$ and $z = 350/d$: $d = 70\,\text{px}$ means $5\,\text{m}$, $d = 17.5\,\text{px}$ means $20\,\text{m}$, $d = 7\,\text{px}$ means $50\,\text{m}$.

Intuition

Where meters come from: b is in meters, so z is in meters – the baseline is the source of scale. Hold a finger at arm's length and blink one eye at a time: the finger jumps a lot against the background; a distant tree barely moves. That jump *is* the disparity, and it is large for near things (large $1/z$). **Matching** is the price: d requires finding the *same* point in both images, which fails on textureless or repetitive surfaces and wherever a surface is visible to one view only. Because disparity is fb times inverse depth, an unknown rig only leaves an unknown *scale* in disparity, which is why affine invariance in *disparity* suits data whose rig is unknown (used in §3.7).

3.3 The Three Currencies *(slides 11)*

3.4 The Single-Image Claim *(slides 12)*

The usual motivation for single-image depth is that it bypasses *all* these limitations. The slide's verdict: a single image does bypass the **cost**, the **sparsity** and the **hardware**, but it cannot

pay with	method	gives	costs
time	emit a pulse and time the return (LiDAR)	metric depth, directly	sparse, expensive, returns missing
a baseline	stereo, baseline known	metric depth, dense where matching works	a calibrated second camera; error growing as z^2
computation	structure from motion, baseline unknown	shape up to one global scale	a moving camera and a rigid scene
a prior	assumptions learned from data	dense, from one image, cheap	not metric, and only as good as the prior

Table 4: The three currencies (slide 11).

bypass the **physics**, because it *never measured anything*. The result is therefore an *ambiguous* quantity (same ray picture as Figure 43).

Intuition

Single-image depth is not a cheaper LiDAR; it is a different kind of object. It converts the question "how far?" into "how far, *given how scenes usually look*?" – a guess with an informed prior. Keeping this distinction in mind explains every failure mode in §3.8.

4 Inside a LiDAR *(slides 13–17)*

Paying with time. Outdoor depth labels are produced by a scanning laser, and their error sources are inherited by any network trained on them – so we need to know how a LiDAR fails.

4.1 Pulsed Time of Flight *(slides 14)*

A short pulse is emitted and a clock runs until the echo returns. The light covers the distance twice, so the interval is halved:

$$r = \frac{c\,t}{2}. \tag{2}$$

Each beam leaves in a known direction, so a return is a *direction plus a range*; the energy that comes back is reported as *intensity*. Pulse widths are a few nanoseconds (typically 5–10 ns, about 4 ns in modern full-waveform scanners). Every other part of the sensor exists to make the interval t reliable.

Example 4.1. An echo from 50 m returns after $t = 2 \cdot 50/(3 \cdot 10^8) \approx 333$ ns. A timing error of 1 ns corresponds to $c\,\delta t/2 \approx 15$ cm of range. A 5 ns pulse is $c \cdot 5\,\text{ns} \approx 1.5$ m long in space.

Intuition

It is echolocation with light: shout and time the echo. The factor $\frac{1}{2}$ is just the round trip. Because c is so large, centimeter accuracy demands sub-nanosecond timing – which is why timing electronics dominate the design.

4.2 Structured Sparsity *(slides 15)*

Azimuth resolution is set by the spin rate; elevation resolution is fixed in hardware. The two differ, so the sampling is *anisotropic*.

Definition 4.2 (Scan-line spacing)**.** For a vertical angular pitch $\Delta\theta_{\text{el}}$ (radians), the spacing between adjacent scan lines at range r is

$$s(r) \approx r\,\Delta\theta_{\text{el}}.$$

Example 4.3 (Velodyne VLP-16)**.** At 5 Hz the sensor samples every 0.1° in azimuth; at 20 Hz every 0.4° – spinning faster costs horizontal resolution. Vertically it is stuck at $\Delta\theta_{\text{el}} = 2.0° = 0.0349$ rad, so the scan lines are 0.70 m apart at 20 m and 1.75 m apart at 50 m. A pedestrian at 50 m is crossed by one scan line or by none. (For comparison, the azimuth gap at 50 m is only 0.09 m at 5 Hz and 0.35 m at 20 Hz.)

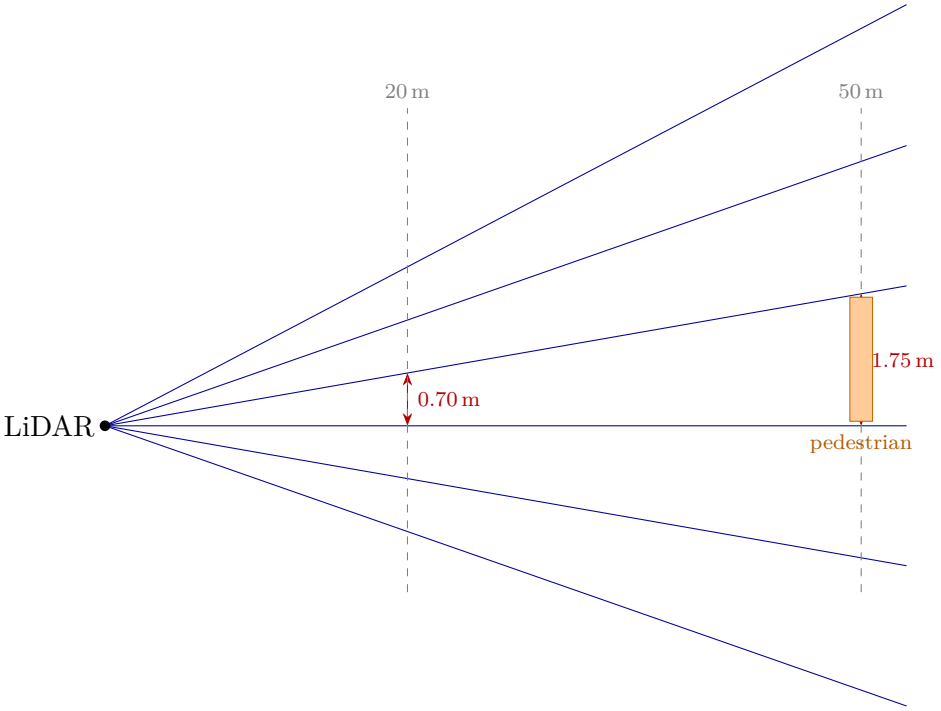


Figure 48: Original figure (not on the slides): VLP-16 elevation fan with a 2° pitch, side view. Vertical axis exaggerated: 1 cm = 1 m vertically but 1 cm = 5 m along range. At 50 m a 1.7 m pedestrian fits between two consecutive scan lines and is missed.

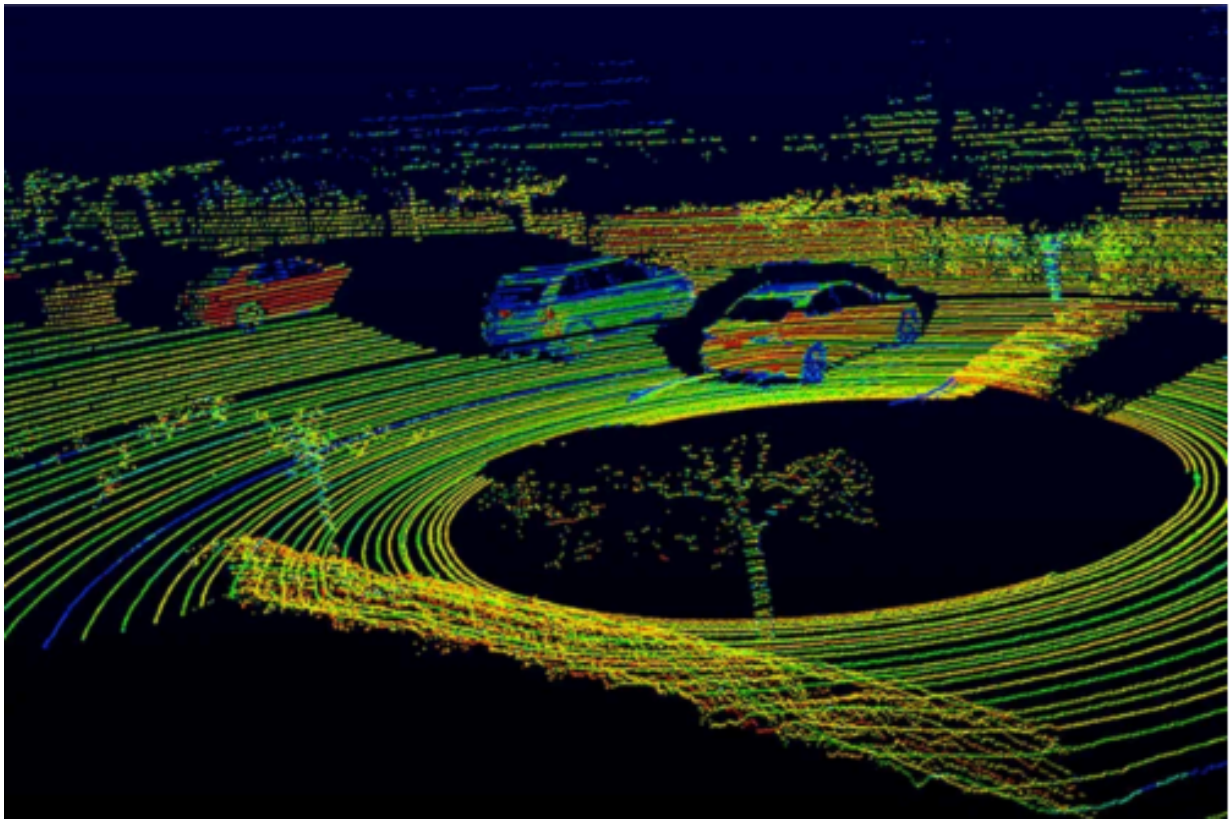


Figure 49: colored LiDAR point cloud of a parking lot: concentric ring-like scan lines with gaps between them

Intuition

Picture the grooves of a vinyl record. Samples are dense *along* a groove (azimuth) but grooves are far apart (elevation). Moving away from the sensor, the grooves spread linearly with range, while points along a groove stay comparatively tight. So a point cloud thins with distance along *one axis* much faster than the other, and the gaps form horizontal stripes.

4.3 Limitations of a LiDAR Point Cloud *(slides 16)*

- **No color and no semantics.**
- **No measurement where there was no return**, which is *distinct from a zero depth*: "I saw nothing" is not "the surface is at distance 0".
- **No sample between scan lines**, and that gap grows linearly with range.
- **Points that lie on no surface**: where one pulse straddles a silhouette, the echo mixes foreground and background into a single (wrong) range.

Intuition

The last item is the LiDAR analogue of an out-of-focus edge: half the beam hits the near object, half the far wall, and the reported range is somewhere in the empty air between them. Such "flying" points are a systematic error that a network trained on LiDAR labels will inherit at every object boundary.

4.4 Two Error Laws *(slides 17)*

The two principles fail in different ways, and the difference decides where each is worth using.

$$\text{time of flight:}\quad \delta r = \frac{c\,\delta t}{2}, \qquad \text{triangulation:}\quad \delta z \approx \frac{z^2\,\delta d}{f\,b}.$$

Proposition 4.4 (Triangulation error law)**.** *A disparity error δd produces a depth error $\delta z \approx z^2\delta d/(fb)$: quadratic in range, even for a perfect sensor.*

Proof. From Theorem 3.2, $z = fb/d$, so $\left|\frac{dz}{dd}\right| = \frac{fb}{d^2} = \frac{z^2}{fb}$. Multiply by δd. □

The timing floor $c\,\delta t/2$ is *independent of range*. What degrades with range is the *precision*: the returned power falls as $1/r^2$ or $1/r^4$ (depending on whether the target is extended or smaller than the beam – standard radar/LiDAR-equation background), and a weaker return is timed less precisely.

Example 4.5 (Numbers from the slide)**.** With $f = 700\,\text{px}$, $b = 0.5\,\text{m}$ and quarter-pixel matching ($\delta d = 0.25\,\text{px}$), $fb = 350$ and

$$\delta z(5\,\text{m}) = \tfrac{25\cdot 0.25}{350} \approx 2\,\text{cm}, \quad \delta z(20\,\text{m}) \approx 29\,\text{cm}, \quad \delta z(50\,\text{m}) \approx 1.8\,\text{m}.$$

The laser is exact where it lands, but at 50 m its scan lines are 1.75 m apart, so a pedestrian is crossed at most once.

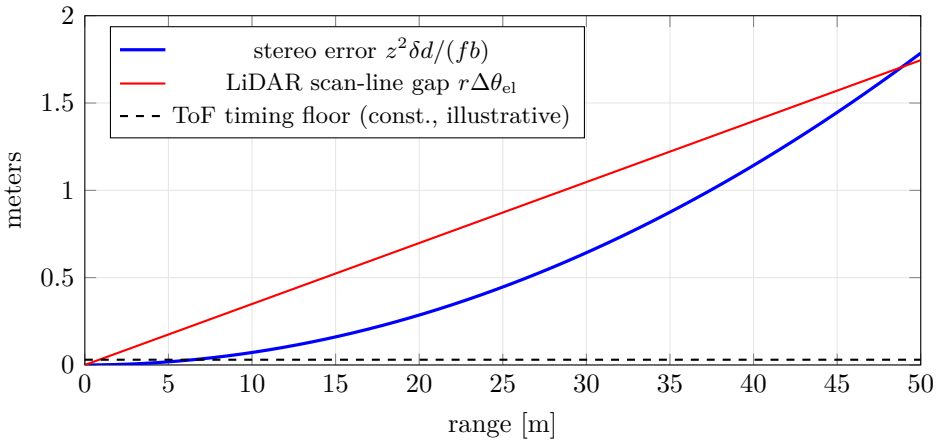


Figure 50: Original figure (not on the slides), from the slide's numbers: $f = 700\,\text{px}$, $b = 0.5\,\text{m}$, $\delta d = 0.25\,\text{px}$, $\Delta\theta_{\text{el}} = 2°$. The ToF floor level (3 cm) is only illustrative.

Intuition

The two sensors are *complementary*: LiDAR is precise where it samples but leaves holes; stereo samples everywhere but each sample degrades quadratically. This is why the remaining sections either *fuse* the two or *replace the laser with a prior*.

5 From Point Cloud to Depth Image *(slides 18–19)*

Paying with time: from cloud to depth map. A LiDAR produces a point cloud. Converting it to a depth map needs the extrinsic calibration, the intrinsics (lecture 01), and a rule for resolving points that fall in the same pixel.

5.1 Projecting a Cloud Into the Image *(slides 19)*

The extrinsics are applied, followed by the projection from lecture 01:

$$\tilde{\mathbf{x}}_s = \boldsymbol{\pi}\Big(\boldsymbol{K}\,[\,\boldsymbol{R}_{cl} \mid \boldsymbol{t}_{cl}\,]\,\tilde{\boldsymbol{p}}^{\,l}\Big)\,, \tag{3}$$

where $\boldsymbol{\pi}(\cdot)$ is the homogeneous division and $\boldsymbol{p}^l$ a point in the LiDAR frame.

Definition 5.1 (Depth map)**.** The number written into the pixel is the z component in the *camera* frame. The array of them is the depth map $\boldsymbol{D} : \Omega \to \mathbb{R}_{>0}$,

$$\boldsymbol{D}[\boldsymbol{x}_s] = \big(\boldsymbol{R}_{cl}\,\boldsymbol{p}^l + \boldsymbol{t}_{cl}\big)_z.$$

The slide stresses that this *differs from the range the LiDAR measured.*

Proposition 5.2 (Depth versus range)**.** *If θ is the angle between the ray of pixel $\boldsymbol{x}_s$ and the optical axis, then $z = r\cos\theta$, where $r = \|\boldsymbol{p}^c\|$ is the range; equivalently $r = z\,\|\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s\| \geq z$.*

Proof. By (1), $\boldsymbol{p}^c = z\,\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ with $\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s = (a, b, 1)^\top$; so $r = z\sqrt{a^2+b^2+1}$. The unit direction has z-component $1/\sqrt{a^2+b^2+1} = \cos\theta$. □

Example 5.3. A return with range $r = 20\,\text{m}$ at $30°$ off the optical axis is written as $z = 20\cos 30° \approx 17.3\,\text{m}$ in the depth map.

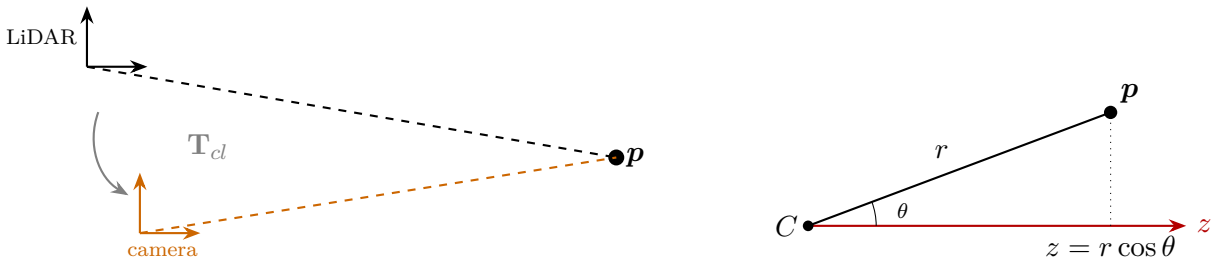


Figure 51: Left: LiDAR and camera frames related by $\mathbf{T}_{cl}$ (redrawn from slide 19). Right: range r versus depth z (original figure).

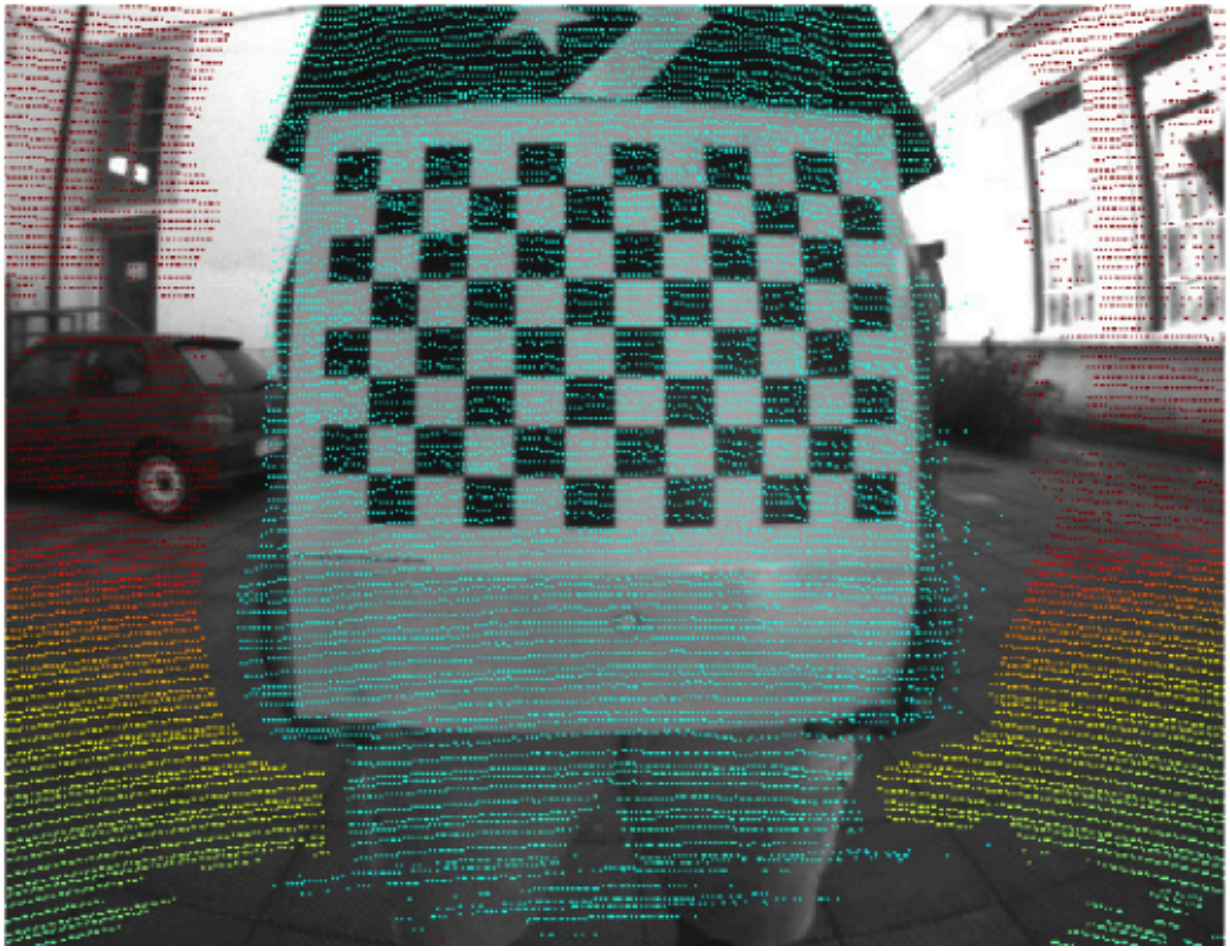


Figure 52: grayscale image of a checkerboard target with LiDAR points projected onto it, colored by depth

Algorithm 5.1 (LiDAR cloud to sparse depth map)**.** Input: points $\{\boldsymbol{p}_j^l\}$, intrinsics $\boldsymbol{K}$, extrinsics $(\boldsymbol{R}_{cl}, \boldsymbol{t}_{cl})$, image size.

1. Initialize $\boldsymbol{D} \leftarrow 0$ everywhere ("no measurement").
2. For each point: $\boldsymbol{p}^c = \boldsymbol{R}_{cl}\boldsymbol{p}^l + \boldsymbol{t}_{cl}$; skip if $p_z^c \leq 0$ (behind the camera).
3. $\tilde{\mathbf{x}}_s = \boldsymbol{\pi}(\boldsymbol{K}\boldsymbol{p}^c)$, round to a pixel; skip if outside the image.
4. If $\boldsymbol{D}[\boldsymbol{x}_s] = 0$ or $p_z^c < \boldsymbol{D}[\boldsymbol{x}_s]$, set $\boldsymbol{D}[\boldsymbol{x}_s] \leftarrow p_z^c$.

Intuition

Step 4 is one common rule for "points that fall in the same pixel" (keep the nearest, as a z-buffer does); the slide only says such a rule is needed. Valid depths are strictly positive ($\boldsymbol{D} : \Omega \to \mathbb{R}_{>0}$), so 0 can safely mark "no return" without colliding with a genuine measurement – matching the earlier warning that a missing return is not a zero depth. Also remember that the result is *sparse*: most pixels stay 0.

6 From Recovery to Prediction *(slides 20–22)*

Paying with a prior. One image does not determine depth. People judge it anyway, so they are supplying an assumption. This section turns that observation into a prior.

6.1 Monocular Cues in Human Vision *(slides 21)*

Not all 3D structures are equally likely, and human vision exploits that regularity through seven classic cues:

1. linear perspective,
2. relative size / position,
3. texture gradient,
4. occlusion (the occluded object is farther),
5. aerial perspective,
6. light and shadows,
7. blur and defocus.

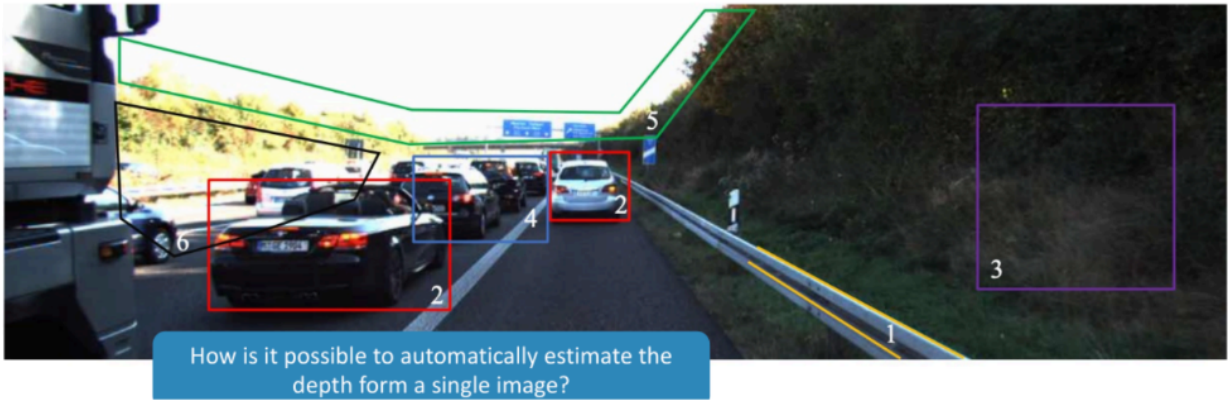


Figure 53: driving image (highway scene) with the seven cues outlined and numbered

Intuition

None of these is a measurement. Each is an *assumption about how scenes are usually arranged*: parallel lines meet, near things are big, far things are hazy. They work because the world is not a uniformly random set of 3D structures.

6.2 Depth Prediction as a Prior *(slides 22)*

Proposition 6.1 (Bayesian view of single-image depth)**.**

$$p(\boldsymbol{D} \mid \boldsymbol{I}) \;\propto\; p(\boldsymbol{I} \mid \boldsymbol{D})\, p(\boldsymbol{D}).$$

The likelihood term is nearly flat along each ray (this is the ill-posedness of §3.1), so the posterior is dominated by the prior $p(\boldsymbol{D})$. Hence single-image depth is predicted *rather than* recovered*, and a method is a claim about which scenes are likely.*

Intuition

The likelihood asks: "does this depth map explain the picture?" Along a ray, every depth explains the pixel equally well, so the picture alone cannot vote. The only thing left to break ties is what depth maps *usually look like* – and people reading depth from a single photograph is everyday evidence that such a prior exists and is learnable. Where the learned one fails is the topic of §3.8.

7 Cues in Closed Form *(slides 23–26)*

The prior, written in closed form. Vanishing points, apparent size and vertical position each follow in one line from the projection equation.

7.1 Vanishing Points *(slides 24)*

Parallel 3D lines converge in the image; the horizon is the image of the line at infinity of the ground plane. A vanishing point is the image of a *direction*, not of a location; every set of parallel lines has its own, and for a given plane they all lie on one line.

Proposition 7.1 (Vanishing point of a direction)**.** *Let $\boldsymbol{d} \in \mathbb{R}^3$ be a 3D direction. Its image $\boldsymbol{v}$ satisfies*

$$\lambda\, \boldsymbol{v} = \boldsymbol{K}\,[\boldsymbol{R} \mid \boldsymbol{t}] \begin{pmatrix} \boldsymbol{d} \\ 0 \end{pmatrix} = \boldsymbol{K}\boldsymbol{R}\,\boldsymbol{d}.$$

Proof. A direction is a point at infinity, homogeneous coordinates $(\boldsymbol{d}, 0)$. In the product $[\boldsymbol{R} \mid \boldsymbol{t}](\boldsymbol{d}, 0)^\top = \boldsymbol{R}\boldsymbol{d} + \boldsymbol{t} \cdot 0$ the translation is multiplied by zero and drops out. □

Corollary 7.2. *The horizon moves with camera **pitch and roll** only; translation does not move it.*

Example 7.3 (Horizon row for a pitched camera – extra worked case)**.** Take zero roll, image y pointing down, and pitch the camera up by θ. The level forward direction has camera-frame coordinates $\boldsymbol{R}\boldsymbol{d} = (0, \sin\theta, \cos\theta)^\top$, so $\boldsymbol{K}\boldsymbol{R}\boldsymbol{d}$ gives the horizon row $y_h = c_y + f\tan\theta$: pitching up moves the horizon *down* in the image.

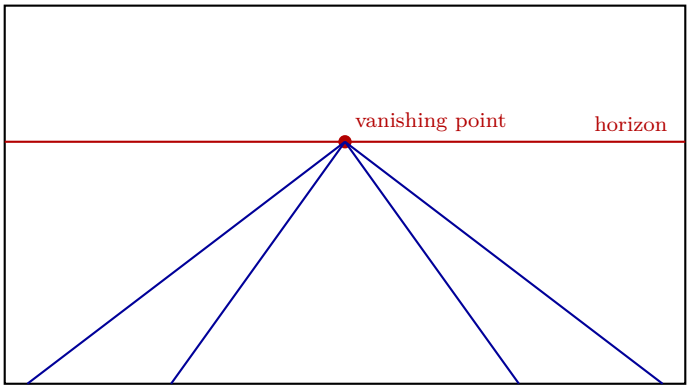


Figure 54: Original schematic (the slide shows a photograph, see below): parallel lines on the ground plane meet on the horizon.

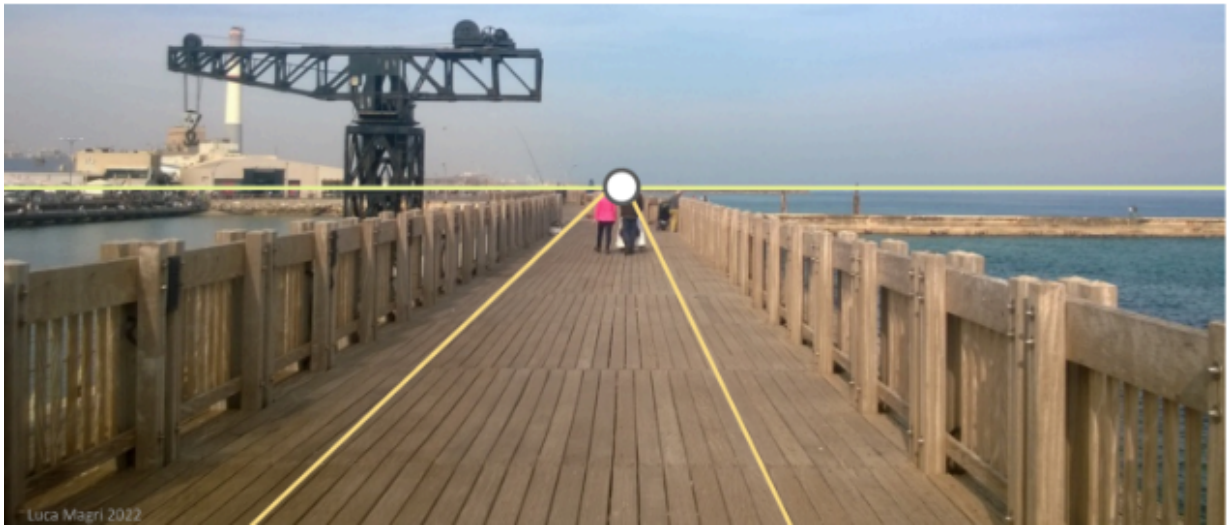


Figure 55: photo of a wooden pier with yellow lines converging to a vanishing point on the horizon

Intuition

The translation drops out because a direction has no position: walking sideways does not change where the rails converge, only tilting the camera does. The slide adds a warning to remember: *a network later on does not respect this* (§3.8).

7.2 Apparent Size *(slides 25)*

Proposition 7.4 (Depth from apparent size)**.** *If the true height H of an object is known and its image height is h (in pixels), then*

$$z = \frac{f\,H}{h}.$$

Proof. The camera center, the top and the bottom of the object form a triangle similar to the one formed by the center and the image of the object: $h/f = H/z$. □

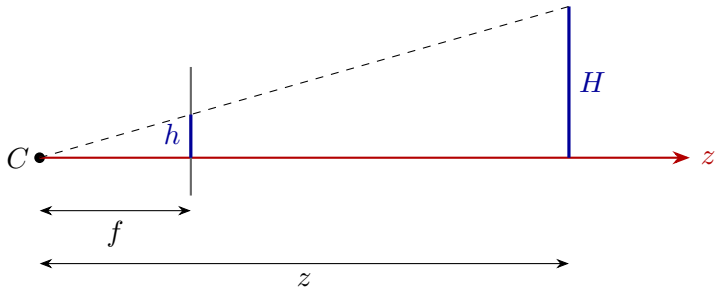


Figure 56: Apparent size (redrawn from slide 25, side view).

Example 7.5. A car $H = 1.5\,\text{m}$ tall that appears $h = 50\,\text{px}$ tall with $f = 700\,\text{px}$ is at $z = 700 \cdot 1.5/50 = 21\,\text{m}$.

Intuition

This works on driving data because almost everything in it is a car, a truck or a pedestrian, and those have known sizes: the *prior about sizes* is doing the work. It fails for objects of unknown size – the giant bench of §3.1 again.

7.3 Vertical Position *(slides 26)*

Proposition 7.6 (Depth from the ground contact point)**.** *On a flat ground plane with known camera height Y, an object touching the ground at image row y_s (horizon row y_h) has depth*

$$z = \frac{f\,Y}{y_s - y_h}.$$

Proof. For a level camera, the ground point has camera-frame vertical coordinate Y (below the camera) and depth z; projection gives $y_s - y_h = fY/z$. Solve for z. □

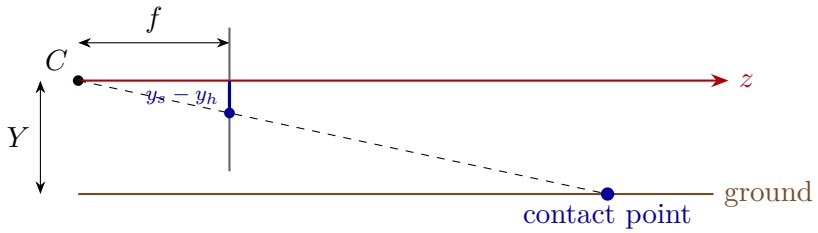


Figure 57: Vertical position (redrawn from slide 26, side view; horizon = optical axis for a level camera).

Corollary 7.7 (Sensitivity)**.** *Differentiating,* $\delta z = z^2\,\delta y/(fY)$.

Example 7.8. $f = 700\,\text{px}$, $Y = 1.65\,\text{m}$, contact point 40 px below the horizon: $z = 700 \cdot 1.65/40 \approx 28.9\,\text{m}$. A one-pixel error moves this by $z^2/(fY) \approx 0.73\,\text{m}$.

Intuition

No knowledge of the object is required, only its contact point with the ground; but *the assumptions carry the estimate*: flat ground, known camera height, known horizon. Note the resemblance to Proposition 4.4: the error also grows as z^2, with the camera height Y playing the role of a baseline (a remark of these notes, not on the slide). Cues near the horizon are extremely sensitive.

8 Supervised Single-View Depth *(slides 27–35)*

Paying with a prior, part two. Fitting the prior requires an architecture, a loss that respects the scale ambiguity, and measured depth data.

8.1 The Supervised Setup *(slides 28)*

Definition 8.1 (Supervised depth learning)**.** A training set of image / depth-map pairs, and a network from $3 \times H \times W$ to $1 \times H \times W$ trained with a per-pixel loss:

$$\text{TR} = \{(\boldsymbol{I}_i, \boldsymbol{D}_i)\}_{i=1}^{N}, \qquad \hat{\boldsymbol{D}} = f_\theta(\boldsymbol{I}).$$

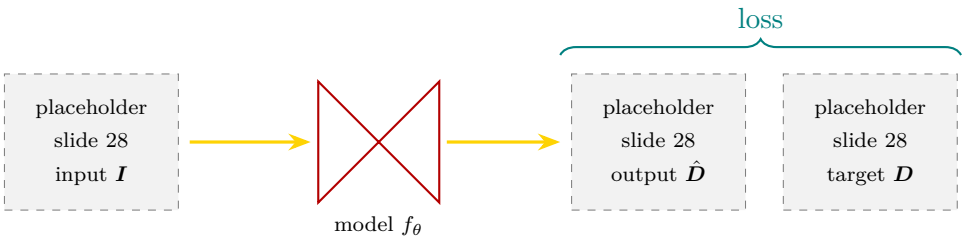


Figure 58: The supervised setup (redrawn from slide 28; the two depth maps are photographs to copy from the slide).

Intuition

Data, network and per-pixel loss are the whole recipe; the slide's summary is that *every remaining design decision is inside the loss* (*what* counts as a correct prediction when scale is unobservable).

8.2 Three Datasets *(slides 29)*

Each of the three sits in a different cell of the principles table: a projected-pattern camera, a scanning laser, and triangulation with no known baseline.

- **NYU Depth v2**: indoor, from a structured-light depth camera, with its raw depth *inpainted* to give a dense label.
- **KITTI**: driving, LiDAR projected into the camera as in §3.4.
- **MegaDepth**: internet photo collections through structure from motion and multi-view stereo. Dense, varied, and *scale-ambiguous by construction.*

These are the $\boldsymbol{D}_i$ of the previous slide. Each inherits the error sources of the sensor that produced it, and the last carries no metric scale at all – which is what the next two methods are designed around.

Intuition

Every dataset is a *sensor's opinion* of depth. NYU inherits the holes and inpainting of a structured-light camera, KITTI inherits LiDAR sparsity and flying points, MegaDepth inherits SfM's arbitrary scale. Datasets with "no scale & shift" or "ordinal" labels cannot be trained on with a plain L2 loss: the loss has to be told which ambiguity to ignore.

8.3 The Scale-Invariant Loss *(slides 30)*

The direct choice, $\mathcal{D}(\boldsymbol{D}, \hat{\boldsymbol{D}}) = \|\boldsymbol{D} - \hat{\boldsymbol{D}}\|_2^2$, penalizes the network for a global scale the image does not determine. What is required is a loss invariant to that scale:

$$\mathcal{D}(\boldsymbol{y}, \boldsymbol{y}^*) = \mathcal{D}(s\boldsymbol{y}, \boldsymbol{y}^*) \quad \forall s > 0.$$

Dataset	Annotation	Depth type	Accuracy	Diversity	# images
DIML Indoor	RGB-D	metric	medium	medium	220K
MegaDepth	SfM	no scale	medium	medium	130K
ReDWeb	stereo	no scale & shift	medium	high	3600
WSVD	stereo	no scale & shift	medium	high	1.5M
3D Movies	stereo	no scale & shift	medium	high	75K
DIW	user clicks	ordinal pair	low	high	496K
ETH3D	laser	metric	high	low	454
Sintel	synthetic	(metric)	high	medium	1064
KITTI	laser/stereo	metric	medium	low	93K
NYUDv2	RGB-D	metric	medium	low	407K
TUM-RGBD	RGB-D	metric	medium	low	80K

Table 5: Trimmed transcription of the dataset table on slide 29 (the indoor/outdoor/dynamic/video/dense check-mark columns are omitted; copy the full table from the slide if needed).

Definition 8.2 (Eigen–Puhrsch–Fergus scale-invariant loss (NIPS 2014))**.** Mean squared error in log space, plus a per-image offset:

$$\mathcal{D}(\boldsymbol{y},\boldsymbol{y}^*) = \frac{1}{n}\sum_{i=1}^{n}\big(\log y_i - \log y_i^* + \alpha\big)^2, \qquad \alpha = \frac{1}{n}\sum_{i=1}^{n}\big(\log y_i^* - \log y_i\big).$$

Lemma 8.3 (Closed form and invariance)**.** *Write* $d_i = \log y_i - \log y_i^*$*. Then* $\alpha = -\bar{d}$ *minimizes the expression over all offsets, and*

$$\mathcal{D}(\boldsymbol{y},\boldsymbol{y}^*) = \frac{1}{n}\sum_i d_i^2 - \frac{1}{n^2}\Big(\sum_i d_i\Big)^2 = \mathrm{Var}(d_i).$$

Moreover $\mathcal{D}(s\boldsymbol{y},\boldsymbol{y}^*) = \mathcal{D}(\boldsymbol{y},\boldsymbol{y}^*)$ *for every* $s > 0$*.*

Proof. Differentiating $\sum_i (d_i+\alpha)^2$ in α and setting to zero gives $\alpha = -\bar{d}$; substituting yields $\frac{1}{n}\sum (d_i - \bar{d})^2$, which is the stated variance. Replacing $\boldsymbol{y}$ by $s\boldsymbol{y}$ adds the constant $\log s$ to every d_i; a variance does not change under a constant shift. □

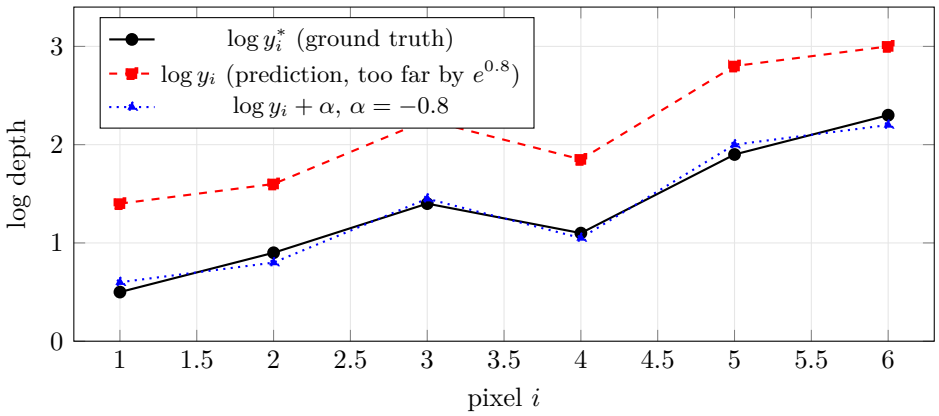


Figure 59: Original toy example (not on the slides). The prediction has the right shape but a wrong global scale. Naive log-MSE would be ≈ 0.646; the scale-invariant loss, after aligning by e^{α}, is only ≈ 0.0058.

Intuition

In log space a wrong *scale* becomes a constant *offset*, and the loss subtracts the best constant offset before looking at the errors. So only the *shape* of the depth map is judged. Two side remarks from the slide: log space stops far pixels (numerically large) from dominating, and $e^{\alpha}\boldsymbol{y}$ is the scalar multiple that best aligns the prediction to the ground truth – every scalar multiple of $\boldsymbol{y}$ scores identically.

8.4 Output Spaces *(slides 31)*

Five quantities a depth network can predict differ in how many unknowns are left free, and in *what the free parameters act on.*

	output	free param.	aligned by	trained this way to
1	metric depth	0	nothing	plan and measure
2	scale-invariant depth	1 (s)	one scale	share one scale per scene
3	affine-invariant disparity	2 (s,t)	scale and shift	mix datasets with no common zero
4	affine-invariant depth	2 (s,t)	scale and shift	mix datasets in depth (3.11)
5	ordinal	any monotone map	nothing	rank only

Table 6: Output spaces (slide 31). Scale-invariant: $\hat{y} \approx s\, y^*$; affine-invariant: $\hat{q} \approx s\, q^* + t$ with $q = 1/y$ for disparity.

These are routinely conflated: Eigen's log-space is *one* parameter, affine invariance *two*. The affine row appears twice because the same pair (s,t) can act on disparity or on depth, and the two differ in 3D (next subsection). Benchmarks align the free parameters against the ground truth first, then score with δ and AbsRel, so a model is scored on *shape, not on meters.*

Example 8.4 (Standard metrics, background)**.** AbsRel $= \frac{1}{n}\sum_i |\hat{y}_i - y_i^*|/y_i^*$, and δ_1 is the fraction of pixels with $\max(\hat{y}_i/y_i^*,\, y_i^*/\hat{y}_i) < 1.25$; both are computed after the alignment above.

8.5 Affine Invariance in Disparity and in Depth *(slides 32)*

Proposition 8.5 (Affine map in disparity = projective map in space)**.** *An affine map in inverse depth,*

$$\frac{1}{z} \longmapsto \frac{s}{z} + t \quad\Longleftrightarrow\quad z \longmapsto \frac{z}{s + t\, z},$$

is a nonlinear map in depth. Back-projecting the transformed depth gives $\hat{\boldsymbol{p}} = \boldsymbol{p}/(s + t\, p_z)$*, which is a* projective *map of space: planes remain planes.*

Proof. $s/z + t = (s + tz)/z$, so $z \mapsto z/(s + tz)$. Back-projection by (1): $\hat{\boldsymbol{p}} = \frac{z}{s+tz}\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s = \boldsymbol{p}/(s + tp_z)$. To see planes stay planes, let $\boldsymbol{a}^\top \boldsymbol{p} = c$ and $w = s + tp_z$. From $\hat{\boldsymbol{p}} = \boldsymbol{p}/w$ we get $\hat{p}_z = p_z/w$, hence $p_z = s\hat{p}_z/(1 - t\hat{p}_z)$ and $w = s/(1 - t\hat{p}_z)$, so $\boldsymbol{p} = w\hat{\boldsymbol{p}}$. Substituting, $s\, \boldsymbol{a}^\top \hat{\boldsymbol{p}} = c\,(1 - t\hat{p}_z)$, which is linear in $\hat{\boldsymbol{p}}$. □

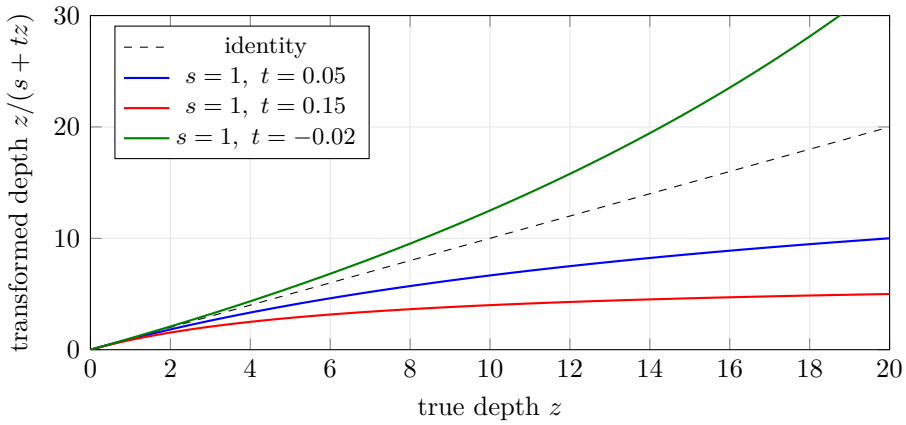


Figure 60: Original figure (not on the slides): an affine map in disparity acts on *depth* as $z \mapsto z/(s+tz)$, a compressive or expansive warp, not a rescaling.

Intuition

A ground plane is therefore *tilted, not bent*: you can decide a scale and shift wrongly, and flat things stay flat but the world may look sheared toward or away from the camera. An affine map in depth itself would instead be $z \mapsto sz + t$, a rigid stretch and translation along the ray. The two choices give the same 2-parameter count but *different 3D geometry* – the reason the table lists the affine row twice.

8.6 MiDaS: Mixing the Datasets *(slides 33)*

No single dataset covers the visual world, so MiDaS trains on several. The obstacle is that they disagree about what they measure (metric, no scale, no scale and shift, ordinal, ...). Its recipe:

- predict in **disparity space** (inverse depth up to scale and shift);
- train with a **scale-and-shift-invariant loss**, where the alignment (s, t) is a least-squares fit solved *in closed form per image*;
- add a **multi-scale gradient matching** term, which is the gradient loss again, at four resolutions.

The output space is the affine-invariant *disparity* row of the table, not the affine-invariant *depth* row under it.

Lemma 8.6 (Closed-form alignment)**.** *For predictions $\hat{q}_i$ and targets q_i^*, the pair*

$$(s, t) = \arg\min_{s,t} \sum_i \left(s\,\hat{q}_i + t - q_i^*\right)^2$$

is $s = \dfrac{\mathrm{Cov}(\hat{q}, q^*)}{\mathrm{Var}(\hat{q})}$, $t = \bar{q}^* - s\,\bar{\hat{q}}$ *(assuming* $\mathrm{Var}(\hat{q}) > 0$*).*

Proof. Setting $\partial/\partial t = 0$ gives $t = \bar{q}^* - s\bar{\hat{q}}$. Setting $\partial/\partial s = 0$: $s\sum \hat{q}_i^2 + t\sum \hat{q}_i = \sum \hat{q}_i q_i^*$. Substituting t and dividing by n: $s(\overline{\hat{q}^2} - \bar{\hat{q}}^2) = \overline{\hat{q}q^*} - \bar{\hat{q}}\bar{q}^*$. □

The loss is then evaluated on the residual *after* alignment. The gradient matching term, in its standard form, penalizes gradients of the aligned residual R^k at each of the $K = 4$ scales k:

$\frac{1}{M}\sum_{k=1}^{K}\sum_i(|\nabla_x R_i^k| + |\nabla_y R_i^k|)$ (form from the original MiDaS paper; the slide only says "the gradient loss again, at four resolutions").

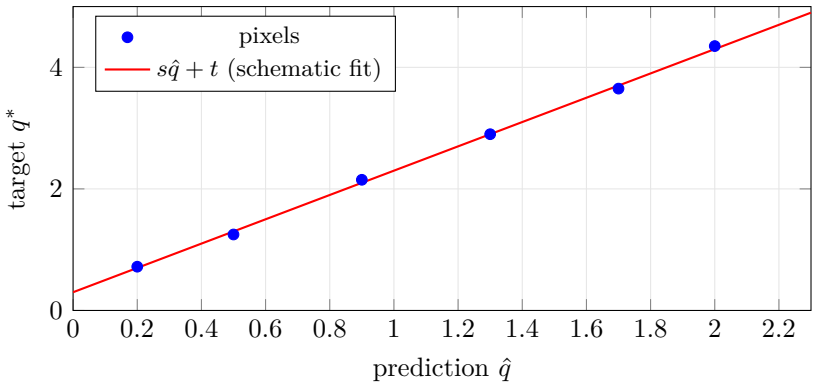


Figure 61: Original schematic (not on the slides): per-image least-squares alignment of a prediction to targets that use a different scale and zero.

Intuition

Each training image gets its *own* best-fit (s, t) before the network is judged, like a teacher who marks a student's drawing after allowing it to be enlarged and moved on the page. That is what lets one network learn from a metric laser dataset, an up-to-scale SfM dataset and a stereo-movie dataset at the same time. Working in *disparity* matters because an unknown rig only contributes an unknown scale there (Corollary 3.3).

8.7 Depth Anything V2 *(slides 34)*

Labeled depth data is the bottleneck, so the second version removes the requirement for it (the first version also trained on labeled real data).

Algorithm 8.1 (Teacher–student training)**.**
1. Train a **teacher** on precise *synthetic* images only, where depth is exact and complete.
2. Use the teacher to **pseudo-label** a very large set of unlabeled real photographs.
3. Train the **student** on those pseudo-labels.

Architecture and losses: encoder from self-supervised pretraining, decoder from dense prediction, and the *same* scale-and-shift-invariant plus gradient-matching losses as MiDaS. The output space is affine-invariant inverse depth; metric models are a separate fine-tuning step on top.

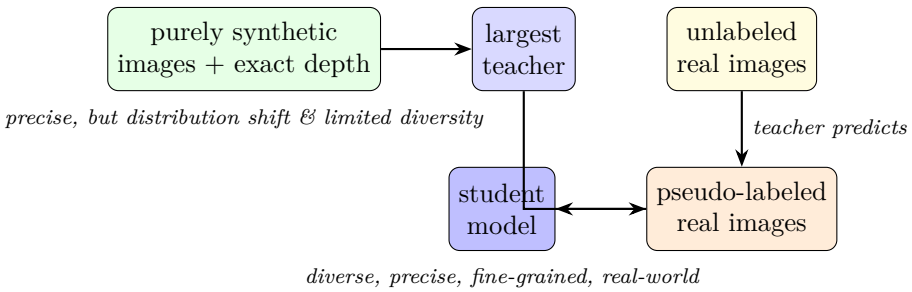


Figure 62: Depth Anything V2 pipeline (redrawn from slide 34).

Intuition

Synthetic images give *perfect* labels but look unlike the real world; real photos look right but have no labels. The teacher transfers its precision to real photos through pseudo-labels, so the student sees real-world diversity with clean, fine-grained targets. The price: the student inherits what the teacher believes – its prior – rather than anything measured.

8.8 Failure on Anamorphic Art *(slides 35)*

Anamorphic street art is a flat pavement painted to look like a hole. The predicted depth map is wrong throughout, with no indication of uncertainty. Nothing has malfunctioned: the prior behaves as trained, on an input constructed to exploit it. *The prior is both the method and the failure mode.*



Figure 63: left: photo of a street painting of a clock tower on pavement; right: predicted depth map from MegaDepth demo

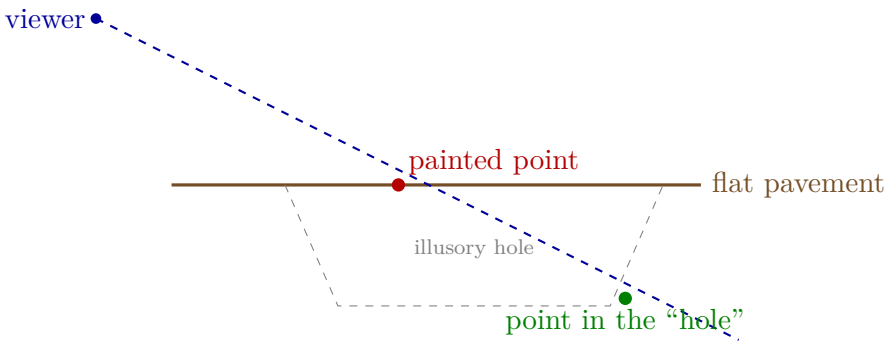


Figure 64: Original figure (not on the slides): from the painting's viewpoint the painted point on the flat ground and the point at the bottom of the imagined hole lie on the same ray (Proposition 2.2). One image cannot distinguish them, so the network chooses the "likely" one.

Intuition

The picture is a concrete instance of the back-projected ray: the artist chooses a flat surface whose appearance is *exactly* what a deep hole would look like from one viewpoint. Depth-from-one-image cannot know; it picks the answer its training data make most probable. Note also the "no indication of uncertainty": the network is confidently wrong.

9 Cues Used by the Network *(slides 36–39)*

What the prior turns out to be. A monocular depth network is a prior fitted to a dataset. Three probes examine which image evidence that prior relies on.

9.1 The Pasted-Car Experiment *(slides 37)*

A car is cropped from the image and pasted back at a chosen ground contact point and a chosen scale, placing the two cues of §3.6 in conflict: both correct (position & size), *position only*, or *size only*. The finding (paraphrased): across four networks, the vertical position in the image wins over the apparent size of known obstacles.

- Position only tracks the true distance; size only does not.
- The result holds across four networks.
- Vertical position needs no object model and generalizes to objects absent from the training set.

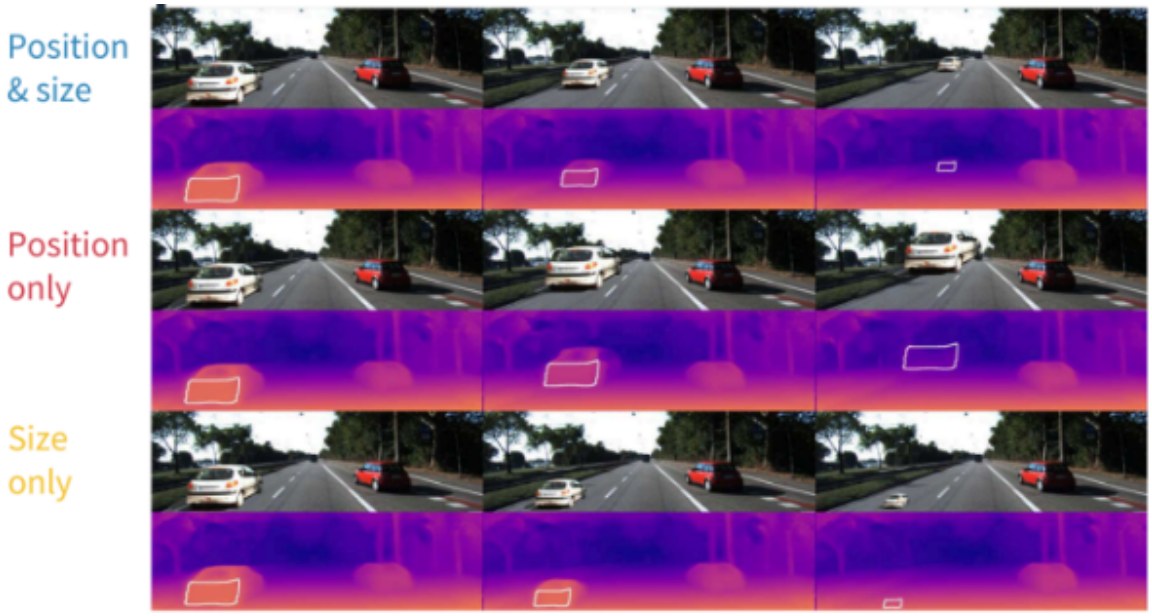


Figure 65: grid of pasted-car examples (position & size / position only / size only) with predicted depth maps

Intuition

The networks behave like the vertical-position cue of Proposition 7.6, not like Proposition 7.4. Sensible: vertical position works for *anything* touching the ground, whereas size needs a size prior per object class. But it also means a car pasted "floating" higher in the frame is judged farther, even if it appears huge.

9.2 Pitch and Roll *(slides 38)*

A pitch change is simulated with a vertical crop and a roll with a rotated crop; both are recovered from the predicted depth. Findings: the networks **only partially** track camera orientation, and small pitch changes measurably disturb the estimated distance to obstacles.

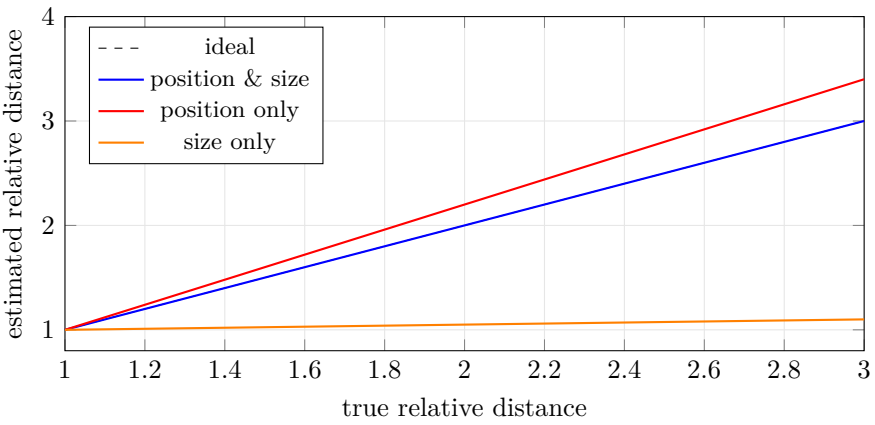


Figure 66: Qualitative schematic after the plot on slide 37, not measured data: positional cues track the ideal line (with some overestimate and larger spread), size-only stays almost flat.

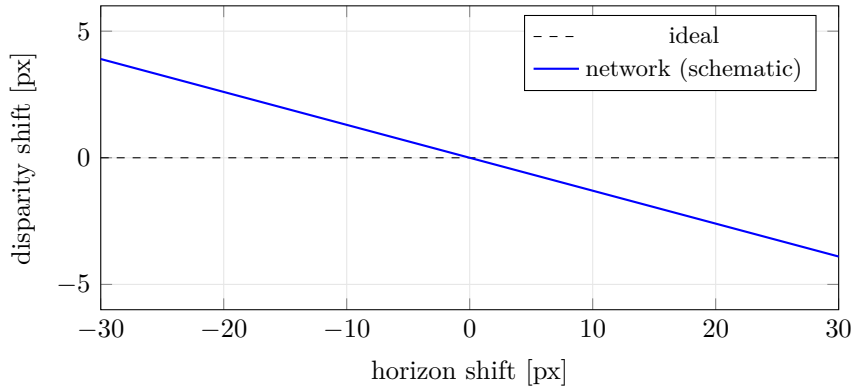


Figure 67: Qualitative schematic after the plot on slide 38, not measured data: an ideal system would ignore the crop (zero shift), the networks' output drifts roughly linearly and opposite to the horizon shift.

Intuition

A crop does *not* change an object's distance to the horizon, $y_s - y_h$, so by Proposition 7.6 the true depth cannot change. But it does change the object's *absolute* vertical position, and the predictions change with it. The network reads vertical image position, rather than distance to the horizon – exactly the shortcut warned about in §3.6, where the horizon depends on pitch and roll only.

9.3 Dependence on the Training Camera *(slides 39)*

A third probe: what happens when only the *camera* changes. A network trained at one focal length predicts the wrong depth at another, with the scene held identical. The apparent-size relation (Proposition 7.4) contains f, and the network has absorbed the training value as a constant. Supplying the intrinsics to the network as extra input channels removes most of the error. The slide's summary: the prior is fitted to the training *camera* as much as to the training *scenes*.

Proposition 9.1 (Idealized focal-length bias – worked consequence, not on the slide)**.** *Suppose a network has absorbed $z = f_0 H/h$ with the training focal length f_0. At test focal length f the image height is $h' = fH/z$, so the predicted depth is*

$$\hat{z} = \frac{f_0 H}{h'} = \frac{f_0}{f}\, z.$$

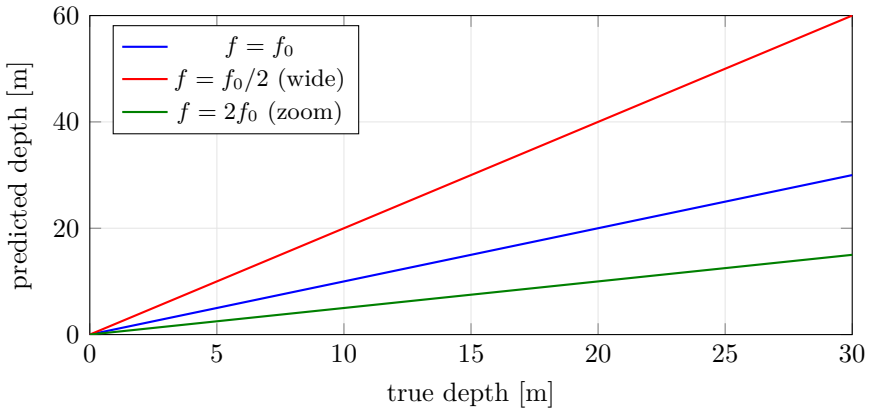


Figure 68: Original figure (not on the slides): predicted depth under the idealization of Proposition 9.1.

Intuition

Zooming in makes everything look bigger, and a network that silently assumes the training focal length reads "bigger" as "closer". This is a scale error, so it is precisely the kind of ambiguity metric prediction must resolve – giving the network the intrinsics removes the hidden constant.

Next: Self-Supervision and a Generative Prior *(slides 40)*

Part I closes owing two debts: the prior was fitted to labels that a sensor had to measure, and mixing those labels cost two free parameters. Part II removes the labels, then pays back the two numbers.

Part I in One Slide *(slides 1–2)*

Part I obtained depth in two ways and closed *owing two debts*:

- **Time.** A laser measures range directly: metric and exact where it lands, but sparse, and it produces the labels everything else trains on.
- **A prior.** A network fitted to those labels is dense and cheap, but it *predicts* rather than recovers depth. The prior was fitted to labels that a sensor had to measure.

Mixing labels of different origin moved the output down a table, at a cost of free parameters.

Definition 9.2 (Output types and their free parameters)**.** A depth-like output is defined only up to a transformation, and each transformation leaves free parameters that something else must later fix.

output	free parameters	aligned by
metric depth	0	nothing
scale-invariant depth	1	one scale s
affine-invariant disparity or depth	2	scale and shift (s, t)
ordinal	any monotone map	nothing

Intuition

Each free parameter is a *number a sensor must still supply.* If a network says "this pixel is twice as far as that one", it has told you shape but not meters. Example: the truth is $D = 2.0\,\hat{D} + 0.3$ m. A metric network outputs D itself; an affine-invariant one outputs $\hat{D}$ and you need the pair (2.0, 0.3) to recover meters. The whole lecture is about who supplies those numbers: the baseline of a stereo rig, a large generative prior, or a handful of LiDAR returns.

10 Supervision without Ground Truth *(slides 3–6)*

This is section 3.9 of the lecture. Its thesis: *a stereo pair supplies its own supervision, provided the warp connecting the two images is differentiable.*

10.1 View Synthesis and the Backward Warp *(slides 4)*

Training uses stereo pairs $\{(\boldsymbol{I}_L, \boldsymbol{I}_R)\}$ only; there is no depth anywhere in the training set. The objective is to reproduce one image of the pair from the other. Depth is the only internal quantity that makes this task solvable, so if the network solves it, depth is what it must have learned. This holds only where a pixel is **textured** and visible in **both** views; elsewhere the objective says nothing and a hand-written prior fills the gap.

Definition 10.1 (Backward warp)**.** Given a disparity map $d(u, v)$ defined on the *target* image, the backward warp synthesizes the target from the source by

$$\tilde{\boldsymbol{I}}_L(u, v) = \boldsymbol{I}_R(u - d(u, v),\, v). \tag{4}$$

Every *target* pixel is mapped to exactly one *source* location.

Proposition 10.2 (Why the warp must be backward)**.** *Forward mapping (pushing each source pixel to a target location) lets several sources land on one target pixel and leaves other*

target pixels empty: collisions and holes. Backward mapping assigns each target pixel exactly one value, which makes the warp a differentiable function *of d rather than a scatter operation.*

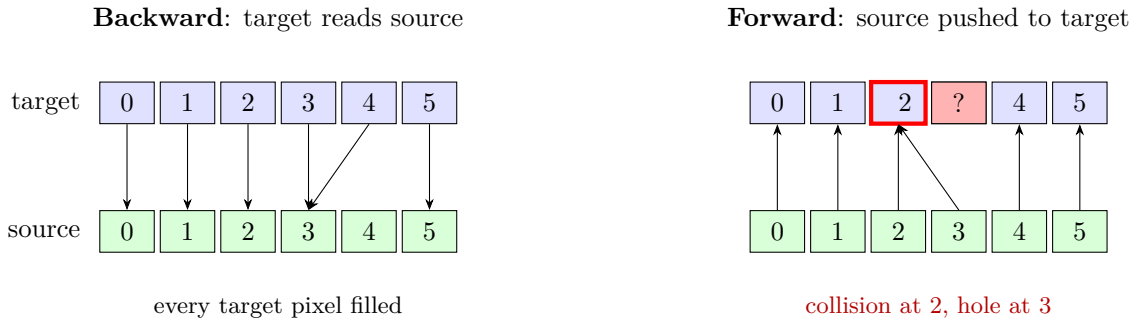


Figure 69: Original 1D illustration of backward versus forward mapping (slide 4 shows the same effect on small coloured grids).

Intuition

Think of filling in a form versus mailing letters. Backward warping is the form: each target pixel asks *"where in the other image should I look?"* and gets exactly one answer. Forward warping is the post office: every source pixel is mailed to an address; some mailboxes get two letters (collision), others none (hole), and "two letters or none" has no useful gradient. Note the sign convention: slide 4 writes $u - d$ for the left image as target, slide 5 writes $+d$ for a target/source pair set up the other way round; only the consistent use matters.

10.2 Image Reconstruction as Supervision *(slides 5)*

The pipeline of slide 5 has three steps: (1) a deep CNN predicts an inverse depth from the left image; (2) the right image is warped with it into a synthetic left image; (3) the reconstruction error against the real left image is the loss.

Definition 10.3 (Disparity and photometric loss)**.** Under a rectified stereo rig with focal length f and baseline b, the predicted inverse depth is the disparity

$$d(\boldsymbol{p}) = \frac{f\,b}{z(\boldsymbol{p})}. \tag{5}$$

The self-supervised loss compares the warped and real image,

$$\mathcal{L}_{\text{photo}} = \sum_{\boldsymbol{p}} \|\tilde{\boldsymbol{I}}(\boldsymbol{p}) - \boldsymbol{I}_L(\boldsymbol{p})\|, \qquad \tilde{\boldsymbol{I}}(\boldsymbol{p}) = \boldsymbol{I}_R(\boldsymbol{p} + d(\boldsymbol{p})). \tag{6}$$

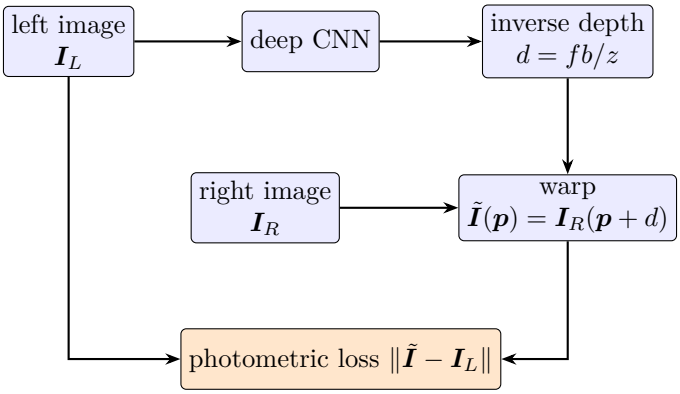


Figure 70: Self-supervised stereo training (redrawn from slide 5). No depth label appears anywhere.

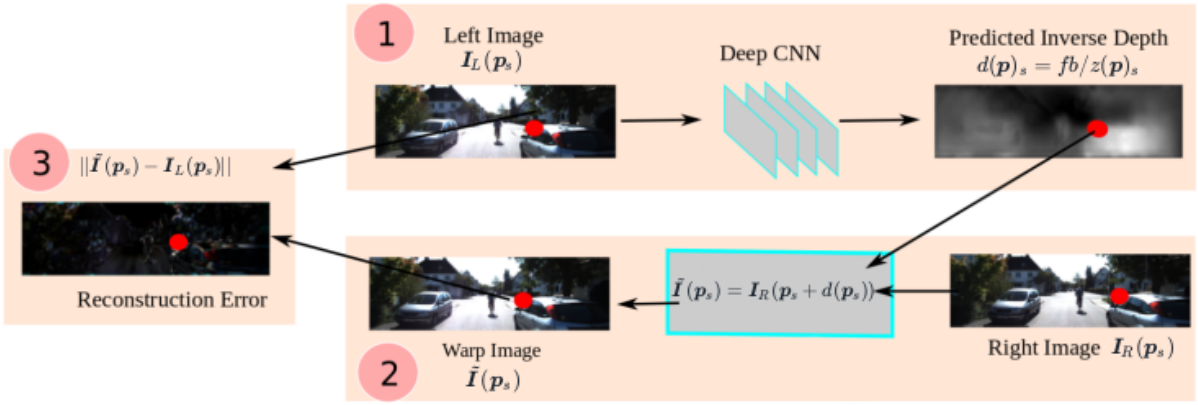


Figure 71: the KITTI street scene: left image, predicted inverse-depth map, warped image and reconstruction-error map (steps 1–3, with the red marker point)

Intuition

Depth is the *only* unknown the network can vary to make the warped image match the real one, so a network that solves the task has learned depth. Worked numbers (illustrative): with $f = 700$ px and $b = 0.5$ m, a car at $z = 10$ m has $d = 35$ px, whereas at $z = 70$ m it has only $d = 5$ px. Disparity is *inverse* depth: distant objects move the image little, so the training signal is weak at range.

10.3 Where the Loss Is Blind *(slides 6)*

Proposition 10.4 (Blind spots of the photometric loss)**.** *(a)* Textureless regions. *Writing the residual as* $r = \boldsymbol{I}_R(u - d) - \boldsymbol{I}_L(u)$ *and the squared loss as* $\frac{1}{2}r^2$*,*

$$\frac{\partial \mathcal{L}}{\partial d} = -\, r \cdot \boldsymbol{I}_R'(u - d), \tag{7}$$

which vanishes wherever $\boldsymbol{I}(u_0 + 1) \approx \boldsymbol{I}(u_0)$ *(flat image, zero derivative): such pixels receive* no training signal at all.
(b) Occlusions. *A pixel visible to one camera only has no match in the other view, so the residual there is meaningless.*

Both gaps are filled by hand-written priors: a smoothness term, left-right consistency, a minimum over the two reprojections, and masking of pixels that do not move between frames.

Proposition 10.5 (The baseline decides what the trained model is worth)**.** *Trained on a stereo pair, the model inherits the known baseline b and is metric* for that rig. *Trained on video, the baseline is unknown, so the output is shape up to one global scale and every image needs its own alignment.*

Intuition

The photometric loss is a detective that only works with clues. Flat white wall: no clue, no gradient, so the smoothness prior guesses. Something hidden from one eye: no partner to compare with. The baseline is the ruler: with stereo the ruler length b is known, so meters come free; with a monocular video the ruler length is not known, so you get a beautiful shape in unknown units (one free parameter, as in the table of Section 1).

11 Diffusion, From Scratch *(slides 7–22)*

This is section 3.10. Its thesis: *a model trained to remove noise is a prior: it points toward the data from anywhere in the space, and it can be sampled, conditioned or steered.* (Adapted on the slides from MIT 6.S183, Lecture 1, by D. Pfrommer.)

11.1 Supervised Learning *(slides 8)*

Supervised learning fits **one output per input**. At a depth edge that output is the average of the near surface and the far one: a distance that belongs to neither.

Proposition 11.1 (MSE regression returns the conditional mean)**.** *Let Y be the target and $\boldsymbol{x}$ the input. Among all functions f, the minimizer of $\mathbb{E}\|f(\boldsymbol{x}) - Y\|^2$ is $f^\star(\boldsymbol{x}) = \mathbb{E}[Y \mid \boldsymbol{x}]$.*

Proof. For fixed $\boldsymbol{x}$ with $m = \mathbb{E}[Y \mid \boldsymbol{x}]$, $\mathbb{E}[\|f - Y\|^2 \mid \boldsymbol{x}] = \|f - m\|^2 + \mathbb{E}[\|Y - m\|^2 \mid \boldsymbol{x}]$; the second term does not depend on f, and the first is minimized by $f = m$. □

Intuition

At a depth edge half the training examples say 2 m (near surface) and half say 10 m (far surface). The best single answer under squared error is 6 m, which is the depth of *nothing*. That is the blurred, flying-pixel edge of regression networks.

11.2 Generative Learning *(slides 9)*

A generative model fits a **distribution** over outputs, not one value. "The band is what a diffusion model represents."

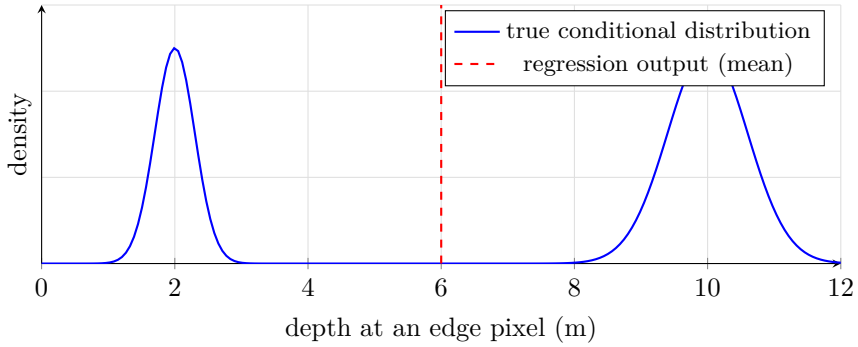


Figure 72: Original toy figure: a bimodal conditional distribution at a depth edge. The regressor answers the mean, which lies in a gap where the density is zero; a generative model can return a sample from one of the two modes.

11.3 Many Outputs for One Input – Text to Image *(slides 10)*

One conditioning input has many valid completions. A regressor averages them; a diffusion model picks one.

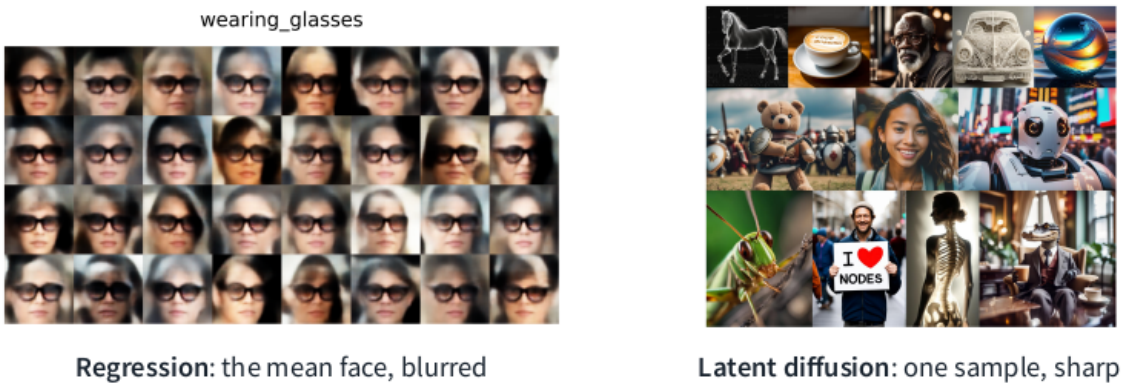


Figure 73: left: grid of "wearing_glasses" faces from regression (the mean face, blurred); right: grid of latent-diffusion samples (one sample, sharp)

Intuition

Blur is not a training failure. It is the correct answer to the wrong question: the conditional mean of a set of sharp images is not a sharp image. The same averaging is what flattens a depth edge.

11.4 The Forward and Reverse Process *(slides 11)*

The **forward process** adds noise to data one step at a time until nothing of the data is left. It is fixed and has no parameters. It is inspired by Brownian motion (many small independent displacements), and after enough steps the sample is indistinguishable from Gaussian noise. A diffusion model learns the **reverse process**, which recovers data from noise; that reverse process *is the prior.*

Definition 11.2 (Forward and reverse processes)**.** The forward chain $q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})$ maps $\boldsymbol{x}_0$ (data) to $\boldsymbol{x}_T$ (noise) and is fixed; the model $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ learns the way back.

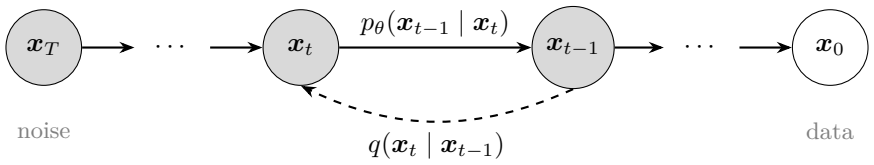


Figure 74: Forward (dashed, fixed) and reverse (solid, learned) chains, redrawn from slide 11.

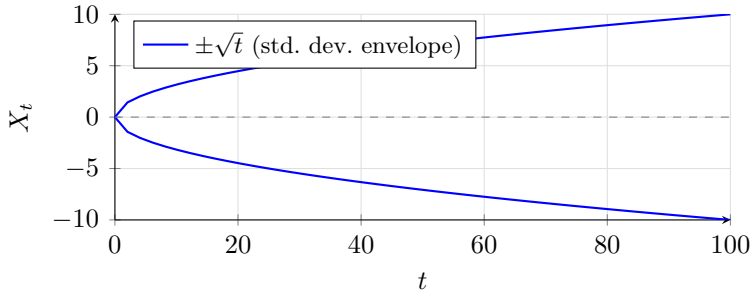


Figure 75: Original figure: the spread of a random walk grows like $\sqrt{t}$, the mechanism that buries the data in noise.

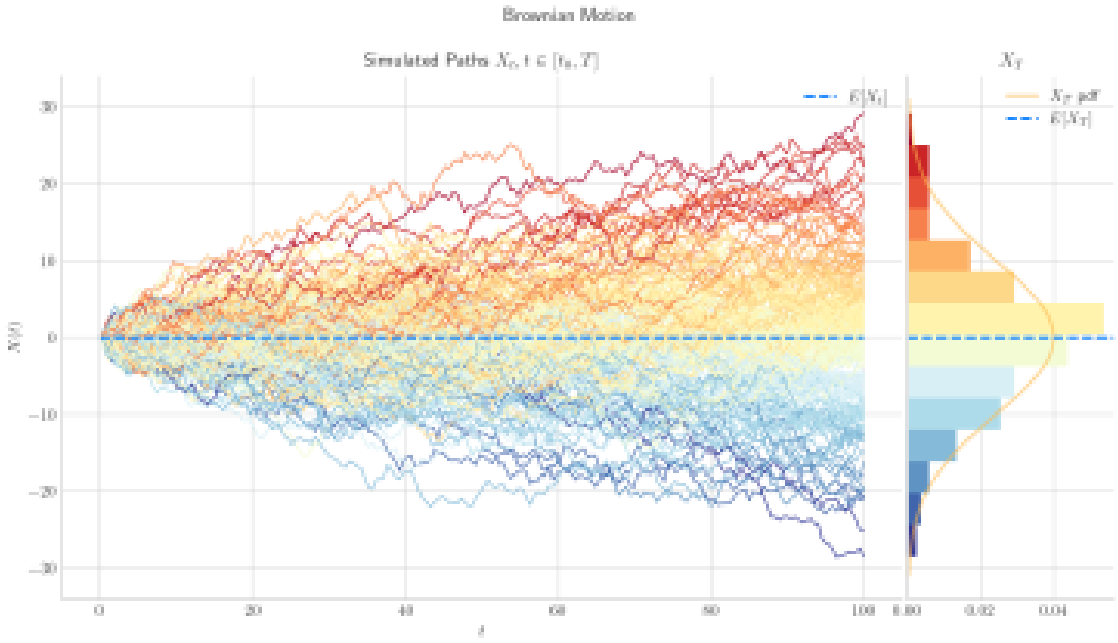


Figure 76: Brownian-motion simulated paths (coloured bundle of trajectories) with the histogram of X_T on the right, and the two small face images (noisy, clean) below the chain

11.5 Training by Noise Prediction *(slides 12)*

A denoising diffusion model estimates the noise vector $\boldsymbol{\epsilon}$ from a noise level $\sigma > 0$ and a noisy input $\boldsymbol{x}_\sigma \approx \boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}$, where $\boldsymbol{x}_0$ lies on the data manifold $\mathcal{K} \subset \mathbb{R}^n$.

Definition 11.3 (Denoiser and training loss)**.** A denoiser $\boldsymbol{\epsilon}_\theta : \mathbb{R}^n \times \mathbb{R}_+ \to \mathbb{R}^n$ is learned by

minimizing

$$L_{\text{simple}}(\theta) := \mathbb{E}_{\boldsymbol{x}_0,\sigma,\boldsymbol{\epsilon}}\Big[\big\|\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}, \sigma) - \boldsymbol{\epsilon}\big\|^2\Big], \tag{8}$$

with $\boldsymbol{x}_0$ sampled from the training data, σ from a training noise schedule, and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

Intuition

The recipe is three lines: take a clean example, add noise of a random strength you remember, and ask the network "what noise did I add?". Subtracting the guessed noise gives the guessed clean point, $\hat{\boldsymbol{x}}_0 = \boldsymbol{x}_\sigma - \sigma\boldsymbol{\epsilon}_\theta$. No labels beyond the data itself are needed.

11.6 The Spiral Toy Model *(slides 13)*

Everything that follows is shown on a 2D data set, where the prior is visible: *it is where the points are*. The data are points of a 2D spiral $\boldsymbol{x}_0$. The network is a fully connected MLP fed with $\boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}$ and a 2D noise-level embedding of σ, and it predicts the noise.

```
def get_sigma_embeds(sigma):
    sigma = sigma.unsqueeze(1)
    return torch.cat([torch.sin(torch.log(sigma)/2),
                      torch.cos(torch.log(sigma)/2)], dim=1)

class TimeInputMLP(nn.Module):
    def __init__(self, dim, hidden_dims):
        super().__init__()
        layers = []
        for in_dim, out_dim in pairwise((dim + 2,) + hidden_dims):
            layers.extend([nn.Linear(in_dim, out_dim), nn.GELU()])
        layers.append(nn.Linear(hidden_dims[-1], dim))
        self.net = nn.Sequential(*layers)
        self.input_dims = (dim,)

    def forward(self, x, sigma):
        sigma_embeds = get_sigma_embeds(sigma)
        nn_input = torch.cat([x, sigma_embeds], dim=1)
        return self.net(nn_input)

model = TimeInputMLP(dim=2, hidden_dims=(16,128,128,128,128,16))
```

Intuition

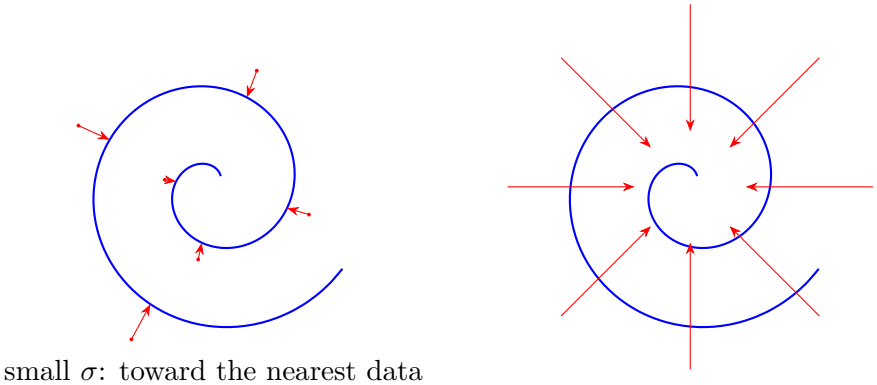
The extra two inputs $(\sin, \cos)$ of $\log\sigma/2$ tell the network *how noisy* its input is. This matters because the right answer depends on σ: at small σ the noise is a small nudge off the nearest data point, at large σ the input is almost pure noise. Note that the first layer has width $\text{dim} + 2 = 4$: two coordinates plus the two embedding numbers.

11.7 Training the Toy Model *(slides 14)*

```
def training_loop(loader, model, schedule, epochs=10000):
    optimizer = torch.optim.Adam(model.parameters())
    for _ in range(epochs):
        for x0 in loader:
            optimizer.zero_grad()
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = model(x0 + sigma * eps, sigma)
```

```
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()
```

The learned denoiser can be drawn as a **vector field** by plotting $\boldsymbol{x} - \sigma\boldsymbol{\epsilon}_\theta(\boldsymbol{x}, \sigma)$. *At small noise the arrows point at the nearest data. At large noise they point at the middle of the whole data set.* The slide's training curve falls from about 0.70 to about 0.41 over 15 000 epochs.



small σ: toward the nearest data

large σ: toward the middle

Figure 77: Original sketch of the denoiser field on the spiral (compare slide 14).

Remark 11.4 (Why the loss does not reach zero). The curve plateaus near 0.41, not 0. This is expected: even the best possible denoiser cannot recover $\boldsymbol{\epsilon}$ exactly, because many $(\boldsymbol{x}_0, \boldsymbol{\epsilon})$ pairs produce the same $\boldsymbol{x}_\sigma$; the floor equals the average posterior variance of $\boldsymbol{\epsilon}$ (Appendix A, item A.2).

11.8 The Ideal Denoiser *(slides 15)*

The **ideal denoiser** minimizes the training loss; it is a function of the data set $\mathcal{K}$ and the noise level, and nothing else:

$$\boldsymbol{\epsilon}^* := \arg\min_{\boldsymbol{\epsilon}_\theta} \mathbb{E}_{\boldsymbol{x}_0,\sigma,\boldsymbol{\epsilon}} \big\|\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}, \sigma) - \boldsymbol{\epsilon}\big\|^2. \tag{9}$$

Theorem 11.5 (Ideal denoiser for a finite data set)**.** *For a finite $\mathcal{K}$ (data uniformly distributed on $\mathcal{K}$, Gaussian noise),*

$$\boldsymbol{\epsilon}^*(\boldsymbol{x}_\sigma, \sigma) = \frac{\sum_{\boldsymbol{x}_0 \in \mathcal{K}} (\boldsymbol{x}_\sigma - \boldsymbol{x}_0)\, \exp\!\big(-\|\boldsymbol{x}_\sigma - \boldsymbol{x}_0\|^2 / 2\sigma^2\big)}{\sigma \sum_{\boldsymbol{x}_0 \in \mathcal{K}} \exp\!\big(-\|\boldsymbol{x}_\sigma - \boldsymbol{x}_0\|^2 / 2\sigma^2\big)}. \tag{10}$$

It is a weighted average of all data points: nearer points count more.

Proof. By the conditional-mean proposition, $\boldsymbol{\epsilon}^*(\boldsymbol{x}_\sigma, \sigma) = \mathbb{E}[\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma]$ with $\boldsymbol{\epsilon} = (\boldsymbol{x}_\sigma - \boldsymbol{x}_0)/\sigma$. By Bayes, $p(\boldsymbol{x}_0 \mid \boldsymbol{x}_\sigma) \propto \exp(-\|\boldsymbol{x}_\sigma - \boldsymbol{x}_0\|^2/2\sigma^2)$ on $\mathcal{K}$. Taking the expectation of $(\boldsymbol{x}_\sigma - \boldsymbol{x}_0)/\sigma$ under these weights gives the formula. □

Intuition

Worked 1D example with $\mathcal{K} = \{-1, +1\}$, $\boldsymbol{x}_\sigma = 0.2$. For $\sigma = 0.5$ the weights are $e^{-1.44/0.5} \approx 0.056$ (for -1) and $e^{-0.64/0.5} \approx 0.278$ (for $+1$), giving $\boldsymbol{\epsilon}^* \approx -0.93$ and clean estimate $\hat{\boldsymbol{x}}_0 = 0.2 - 0.5 \cdot (-0.93) \approx 0.66$: pulled toward the nearer point $+1$. For a large $\sigma = 5$ the two

weights are almost equal (0.972 vs 0.987), $\boldsymbol{\epsilon}^* \approx 0.04$ and $\hat{\boldsymbol{x}}_0 \approx 0.01$: the arrow points at the middle of the data set, exactly as in the slide's three panels ($\sigma = 0.01, 0.1, 0.5$).

11.9 The Smoothed Distance Function *(slides 16)*

The distance to a set, $\operatorname{dist}_{\mathcal{K}}(\boldsymbol{x}) := \inf\{\|\boldsymbol{x} - \boldsymbol{x}_0\| : \boldsymbol{x}_0 \in \mathcal{K}\}$, is not differentiable everywhere, so its gradient is hard to learn. Smoothing it by σ fixes that.

Definition 11.6 (Smoothed distance)**.**

$$D_\sigma(\boldsymbol{x}) := -\sigma^2 \log\Big(\sum_{\boldsymbol{x}_0 \in \mathcal{K}} \exp(-\|\boldsymbol{x}_0 - \boldsymbol{x}\|^2 / 2\sigma^2) \Big). \tag{11}$$

(The slide denotes the smoothed squared distance by $\operatorname{dist}^2_{\mathcal{K}}(\boldsymbol{x}, \sigma)$ and writes the identity below with a factor $\frac{1}{2}$; with the definition as printed, $\frac{1}{2}\operatorname{dist}^2_{\mathcal{K}}(\boldsymbol{x}, \sigma)$ corresponds to D_σ.)

Proposition 11.7 (Denoiser = gradient of the smoothed distance)**.** $\frac{1}{2}\nabla_{\boldsymbol{x}} \operatorname{dist}^2_{\mathcal{K}}(\boldsymbol{x}, \sigma) = \nabla_{\boldsymbol{x}} D_\sigma(\boldsymbol{x}) = \sigma\, \boldsymbol{\epsilon}^*(\boldsymbol{x}, \sigma)$, *and* $\min_{\boldsymbol{x}_0} \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_0\|^2 - \sigma^2 \log|\mathcal{K}| \le D_\sigma(\boldsymbol{x}) \le \min_{\boldsymbol{x}_0} \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_0\|^2$.

Proof. Differentiating, $\nabla D_\sigma = -\sigma^2 \cdot \frac{\sum e^{-\|\boldsymbol{x}_0 - \boldsymbol{x}\|^2/2\sigma^2} (-(\boldsymbol{x} - \boldsymbol{x}_0)/\sigma^2)}{\sum e^{-\|\boldsymbol{x}_0 - \boldsymbol{x}\|^2/2\sigma^2}} = \frac{\sum (\boldsymbol{x} - \boldsymbol{x}_0) e^{\cdots}}{\sum e^{\cdots}} = \sigma \boldsymbol{\epsilon}^*$ by Theorem 11.5. The bounds follow from $e^{-a_{\min}/\sigma^2} \le \sum_i e^{-a_i/\sigma^2} \le |\mathcal{K}|\, e^{-a_{\min}/\sigma^2}$ with $a_i = \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_{0,i}\|^2$. □

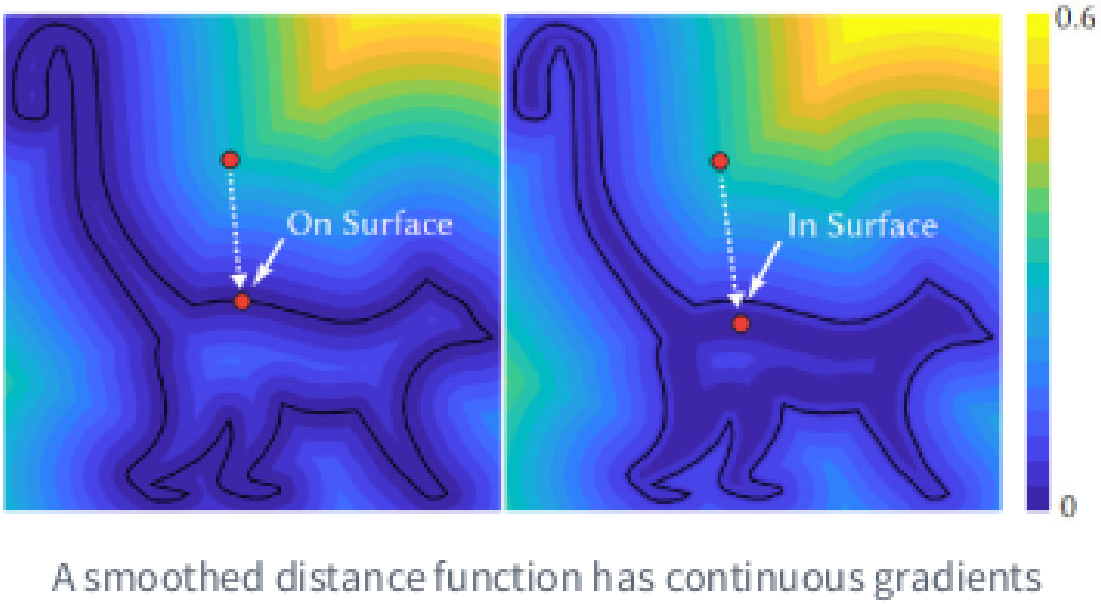


Figure 78: the two coloured distance-field images of the cat silhouette ("On Surface" and "In Surface" with the dotted arrows), with the colour bar 0–0.6

Intuition

min has kinks where two data points are equally near; a *soft*-min has none, and its slope is the vector from the weighted mean of the data to $\boldsymbol{x}$. So the denoiser is a compass that is defined *everywhere* and always points toward the data: that is what makes it a prior. Equivalently $\hat{\boldsymbol{x}}_0 = \boldsymbol{x}_\sigma - \sigma\boldsymbol{\epsilon}^*$ is simply the weighted mean of the data points.

11.10 Sampling From a Diffusion Model – Why iterative? *(slides 17)*

Given a noisy $\boldsymbol{x}_\sigma$ and its noise level, the denoiser estimates the clean point directly,

$$\hat{\boldsymbol{x}}_0(\boldsymbol{x}_\sigma, \sigma) := \boldsymbol{x}_\sigma - \sigma\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_\sigma, \sigma). \tag{12}$$

A sampler walks a decreasing sequence of noise levels, taking one such estimate at each step. **DDIM** is the update

$$\boldsymbol{x}_{t-1} = \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t). \tag{13}$$

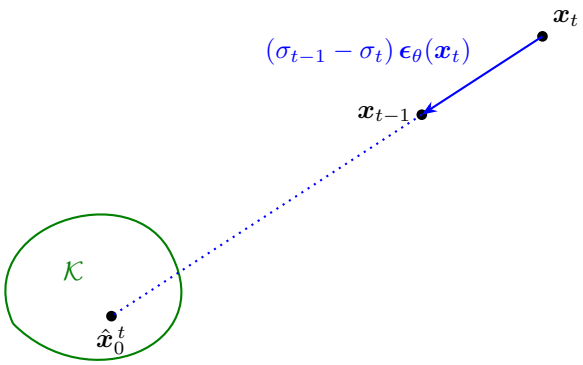


Figure 79: DDIM step (redrawn from slide 17): a straight line toward the current estimate $\hat{\boldsymbol{x}}_0^t$ of the data.

Intuition

One step goes only *part* of the way to the current best guess, because the guess is poor while the input is very noisy; after moving, you re-evaluate the compass. The step is a straight line toward the estimate. This is the equation that depth completion reaches inside and steers (Section 14). Algebraically the step re-noises the clean estimate to the lower level: $\boldsymbol{x}_{t-1} = \hat{\boldsymbol{x}}_0^t + \sigma_{t-1}\boldsymbol{\epsilon}_\theta$ (Appendix A).

11.11 The DDIM Sampler *(slides 18)*

Algorithm 11.1 (DDIM sampler). `for t = N, ..., 1:` $\boldsymbol{x}_{t-1} \leftarrow \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t)$; `return` $\boldsymbol{x}_0$.
Start from $\boldsymbol{x}_N = \sigma_N \cdot \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

```
@torch.no_grad()
def samples_ddim(model, sigmas, batchsize=1):
    xt = model.rand_input(batchsize) * sigmas[0]
    for sig, sig_prev in pairwise(sigmas):
        xt = xt - (sig - sig_prev) * model.predict_eps(xt, sig)
    return xt
```

On the toy model: *fifty steps, no retraining, and the trajectory follows the field.*

11.12 Probabilistic Sampling *(slides 19)*

DDIM is deterministic: the same starting noise always gives the same sample. Adding a noise term back makes the sampler stochastic:

$$\boldsymbol{x}_{t-1} = \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t) + \mu\,\boldsymbol{w}_t, \qquad \boldsymbol{w}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}). \tag{14}$$

(The slide writes $\sigma_{t'}$ for the next, lower level.) With $\mu = 0$ this is DDIM again; as μ grows the trajectories spread, and the same input gives different samples. The slide shows $\mu = 0.001, 0.01, 0.1$.

Intuition

μ is a "temperature": $\mu = 0$ rolls a ball down a fixed groove, $\mu > 0$ shakes the table on the way. This is the reason different seeds give different depth maps in Marigold (Section 12).

11.13 Three Uses of a Prior *(slides 20)*

A prior that exists as a **separate object** can be used in three ways, each a later section of the lecture.

use	meaning	section
Sample	draw from the distribution the model represents	3.10
Condition	train it with a side input $\boldsymbol{c}$, so it samples from $\mathrm{p}(\boldsymbol{x} \mid \boldsymbol{c})$	3.11
Guide	modify the state inside the sampling loop at test time, with no retraining	3.13

The third is the one that turns a depth prior into a metric depth map.

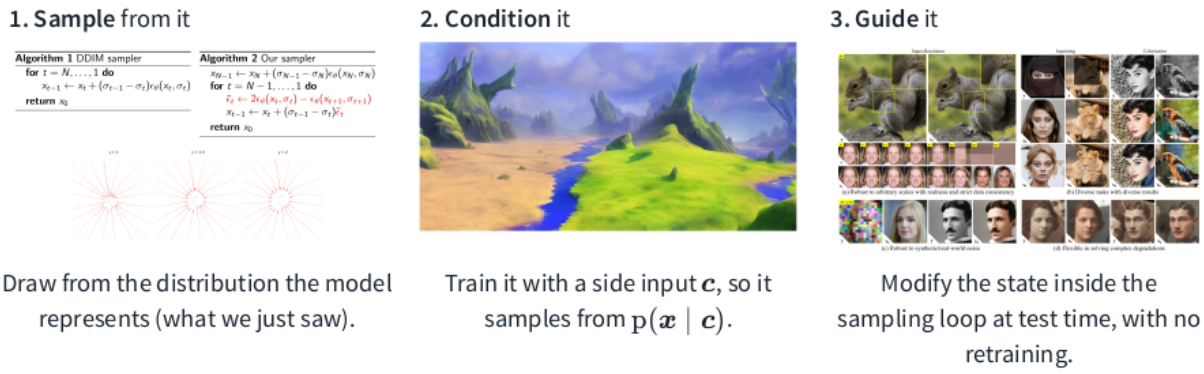


Figure 80: the three panels: sampler comparison plots (Algorithm 1 vs Algorithm 2), the generated landscape (conditioning), and the super-resolution / inpainting / colorization examples (guidance)

11.14 The Autoencoder and Latent Diffusion *(slides 21)*

Images are big, so an autoencoder compresses them first: an encoder $\mathcal{E}$ maps $\boldsymbol{x}$ to a latent $\boldsymbol{z}$, and a decoder $\mathcal{D}$ maps back, $\boldsymbol{z} = \mathcal{E}(\boldsymbol{x})$, $\tilde{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z})$. The autoencoder is trained **first**, then **frozen**; the diffusion model never sees pixels and you decode once, at the end. Diffusion runs on $\boldsymbol{z}$ with the same loss:

$$L = \mathbb{E}_{\sigma, \boldsymbol{z}_0, \boldsymbol{\epsilon}}\Big[\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{z}_0 + \sigma\boldsymbol{\epsilon}, \sigma)\big\|^2\Big]. \tag{15}$$

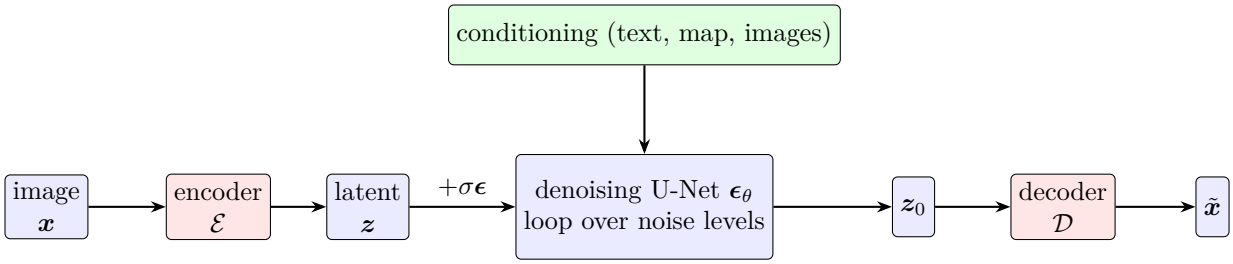


Figure 81: Latent diffusion in outline (original, inspired by slide 21). Pink boxes are frozen autoencoder parts.

Intuition

Worked numbers: $768 \times 768 \times 3 \to 96 \times 96 \times 4$, i.e. 1 769 472 numbers become 36 864: eight times smaller per side, four channels, **48 times fewer numbers**. Each of the 50 denoising steps is therefore 48 times cheaper than it would be in pixel space.

11.15 Conditioning and Guidance *(slides 22)*

Conditioning is a side input the network was *trained with*; the model responds to it because it was trained to. **Guidance** modifies the sampling trajectory *at test time*, using a criterion the network was never trained on. Sampling is a loop, and its state can be modified between steps.

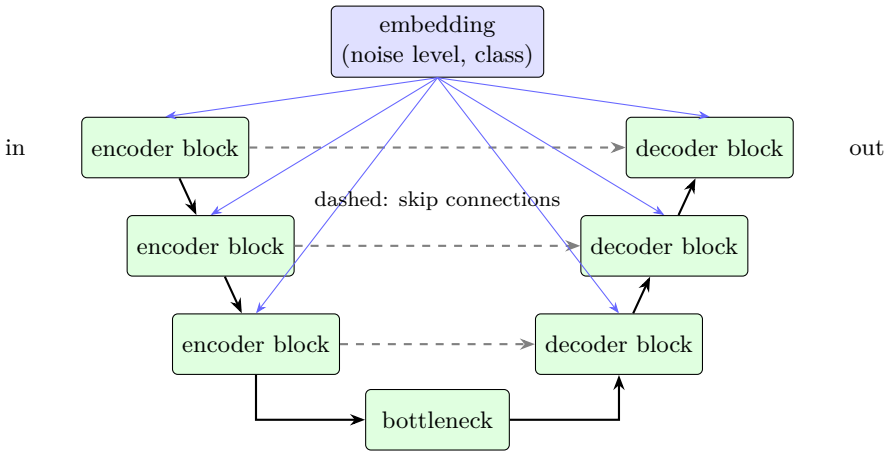


Figure 82: Simplified U-Net redrawn from slide 22. Each block is GroupNorm–SiLU–Conv3×3 with the embedding injected through a Linear layer (scale and shift); attention is optional.

12 Marigold *(slides 23–31)*

Section 3.11: a depth estimator from an image diffusion model. Stable Diffusion's autoencoder is kept, only its denoiser is fine-tuned, and the output is a depth map rather than an image.

12.1 The Marigold Method *(slides 24)*

The argument: a monocular estimator's knowledge of the visual world is **bounded by its training set**, while a text-to-image model trained on billions of captioned photographs has already seen it. Hence: if the cornerstone of monocular depth is a comprehensive representation of the visual world, one should be able to derive a depth estimator from a pretrained image diffusion model.

Definition 12.1 (Marigold)**.** Take a pretrained text-to-image latent diffusion model, **freeze the VAE**, and fine-tune only the denoising U-Net so that what it denoises is a *depth* latent rather than an image latent. Properties from the slide: affine-invariant monocular depth; trained on **synthetic data only**; about 2.5 days on a single consumer GPU.

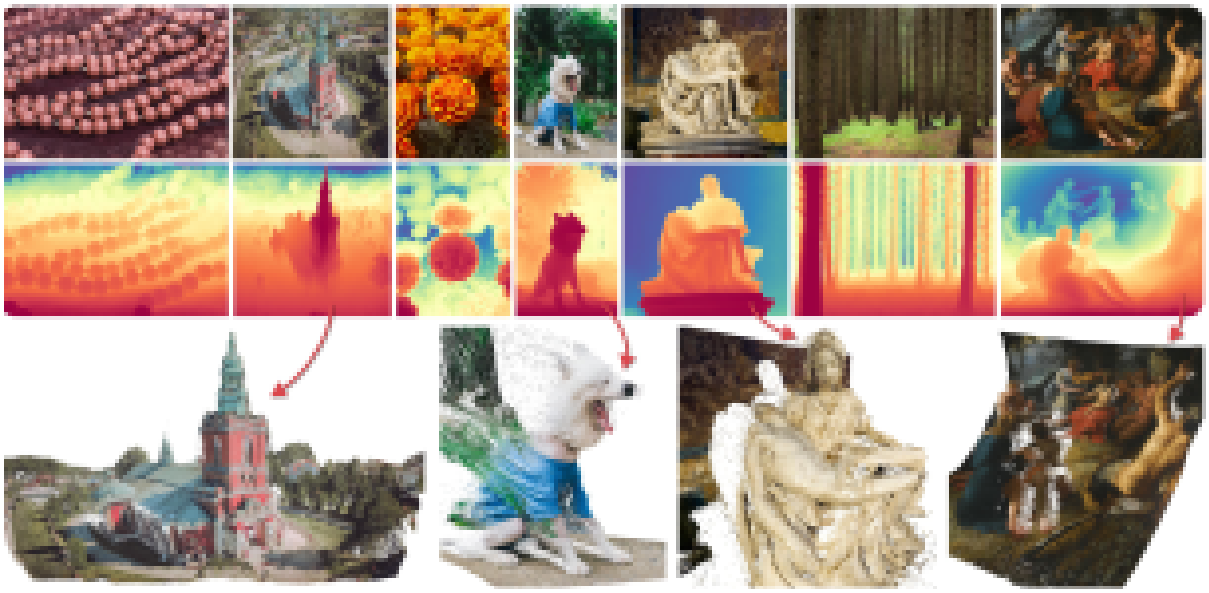


Figure 83: the strip of example results: photographs (beads, church, marigolds, dog, sculpture, forest, painting) with their depth maps and back-projected 3D views

12.2 From the Spiral to Marigold *(slides 25)*

Marigold is the spiral model with one extra input; nothing in the algorithm changes.

	toy model	Marigold
starts from	random weights	Stable Diffusion's weights
a data point	a 2D point	a depth latent
the denoiser	a small MLP	Stable Diffusion's U-Net
the condition	none	the image latent
trained on	spiral points	74K rendered image–depth pairs
the sampler	DDIM	the same DDIM
a sample is	a point on the spiral	a depth map the RGB allows

The first row is the whole bet: *the visual prior is already in those weights*, and fine-tuning moves what they denoise from images to depth. (Stable Diffusion indexes noise levels by a timestep t and scales its input; both are fixed functions of σ, so the process is the same.)

12.3 The Source of Affine Invariance *(slides 26)*

The normalization defines the output space: the affine-invariant *depth* row of the table in Part I, not the affine-invariant *disparity* row that MiDaS uses.

Definition 12.2 (Percentile normalization)**.** With $\boldsymbol{D}_2$ and $\boldsymbol{D}_{98}$ the 2nd and 98th percentiles

of the depth map $\boldsymbol{D}$, the training target is

$$\tilde{\boldsymbol{D}} = \Big(\frac{\boldsymbol{D} - \boldsymbol{D}_2}{\boldsymbol{D}_{98} - \boldsymbol{D}_2} - 0.5\Big) \times 2, \qquad \text{and metric depth is recovered as} \qquad \tilde{\boldsymbol{D}}_{\text{metric}} = s\tilde{\boldsymbol{D}} + t. \tag{16}$$

Proposition 12.3 (Normalization removes scale and shift)**.** *If* $\boldsymbol{D}' = a\boldsymbol{D} + b$ *with* $a > 0$*, then* $\tilde{\boldsymbol{D}}' = \tilde{\boldsymbol{D}}$*.*

Proof. Percentiles commute with increasing affine maps: $\boldsymbol{D}'_2 = a\boldsymbol{D}_2 + b$, $\boldsymbol{D}'_{98} = a\boldsymbol{D}_{98} + b$. Hence $(\boldsymbol{D}' - \boldsymbol{D}'_2)/(\boldsymbol{D}'_{98} - \boldsymbol{D}'_2) = a(\boldsymbol{D} - \boldsymbol{D}_2)/(a(\boldsymbol{D}_{98} - \boldsymbol{D}_2))$, the same ratio. □

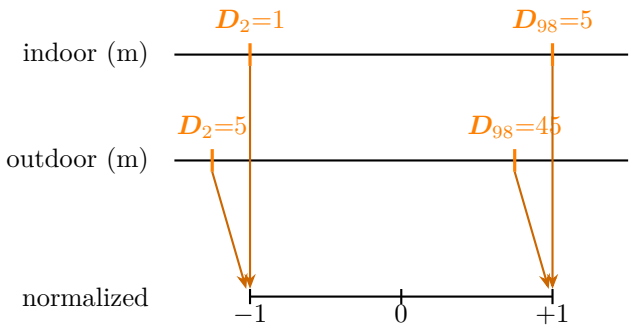


Figure 84: Original figure: an indoor map (few meters) and an outdoor map (tens of meters) land on the same $[-1, +1]$ range; each map is normalized by its *own* percentiles (slide 26). The numbers are illustrative.

Intuition

Every training map is stretched to fill $[-1, +1]$, so a room and a highway look identical to the network in terms of range. **The model never sees meters and never sees a common zero, so it cannot learn them; it is constrained to predict shape.** Example: in a room with $\boldsymbol{D}_2 = 1$ m and $\boldsymbol{D}_{98} = 5$ m, a pixel at 3 m maps to 0; on a street with $\boldsymbol{D}_2 = 5$ m and $\boldsymbol{D}_{98} = 45$ m, a pixel at 25 m also maps to 0.

12.4 Conditioning on the Image *(slides 27)*

The denoiser takes the noisy depth latent $\boldsymbol{z}_\sigma^{(\boldsymbol{D})}$ and the image latent $\boldsymbol{z}^{(\boldsymbol{I})}$:

$$\boldsymbol{\epsilon}_\theta(\boldsymbol{z}_\sigma^{(\boldsymbol{D})}, \boldsymbol{z}^{(\boldsymbol{I})}, \sigma), \qquad \boldsymbol{z}_\sigma = \text{cat}(\boldsymbol{z}_\sigma^{(\boldsymbol{D})}, \boldsymbol{z}^{(\boldsymbol{I})}). \tag{17}$$

The depth map is replicated to three channels and encoded by the **frozen RGB autoencoder**, whose reconstruction error on depth is negligible. **Concatenation** suits a condition that is spatially aligned with the output, pixel for pixel; cross-attention is for conditions that are not.

Definition 12.4 (Four changes to the network)**.** (1) Concatenate the image latent and the noisy depth latent along the channels; (2) double the U-Net's input channels; (3) duplicate the first layer's weights and **halve** them, so activation magnitudes do not inflate; (4) **disable text conditioning** (original Stable Diffusion).

Intuition

Why halve? Let the original first layer be $W\boldsymbol{z}$. The new layer sees $[\boldsymbol{z}; \boldsymbol{z}]$ and uses $[W/2,\ W/2]$, so its output is $W\boldsymbol{z}/2 + W\boldsymbol{z}/2 = W\boldsymbol{z}$: at the start of fine-tuning the pretrained network behaves as before. Without halving, the same input would produce $2W\boldsymbol{z}$, twice the activations

the pretrained weights expect. Since the latent has 4 channels, the new input has 8. Concatenation works here because the depth map and the image are aligned pixel by pixel.

12.5 Training Marigold *(slides 28)*

```
def training_loop(loader, unet, vae, schedule, epochs):
    optimizer = torch.optim.Adam(unet.parameters())
    for _ in range(epochs):
        for image, depth in loader:                    # pairs
            optimizer.zero_grad()
            # start lock
            x0 = vae.encode(normalize(depth))          # a depth latent
            cond = vae.encode(image)                   # the condition
            # end lock
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = unet(x0 + sigma*eps, cond, sigma)   # sees both
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()
```

The objective is the one from the spiral, unchanged. Only the U-Net trains; the VAE is frozen at both ends. Data: 74K rendered pairs, about 2.5 days on one consumer GPU.

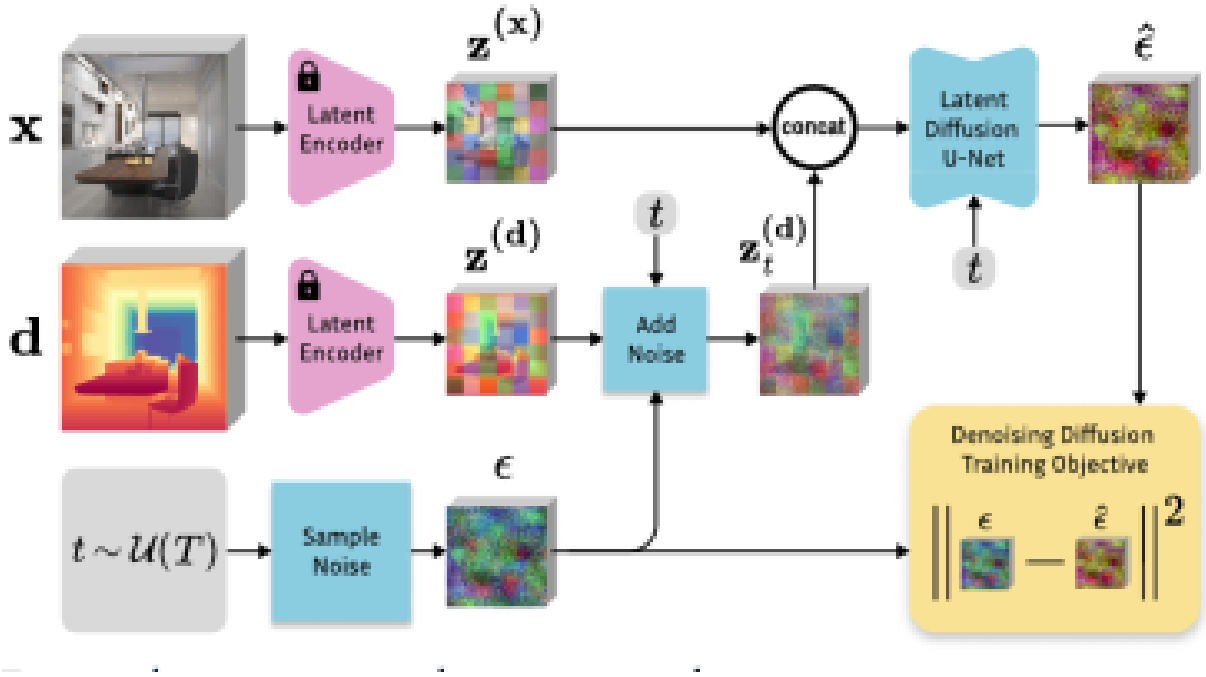


Figure 85: Marigold training (redrawn from slides 27–28). Pink: frozen. The noise ϵ is also fed to the loss.

12.6 Inference *(slides 29)*

Algorithm 12.1 (Marigold inference, one run)**.** (1) Encode the image **once**; (2) initialize the depth latent $z_T^{(D)}$ as pure Gaussian noise; (3) run the denoising loop, concatenating the image latent at **every** step; (4) decode $z_0^{(D)}$ and average the three channels. From the paper: 50 DDIM steps.

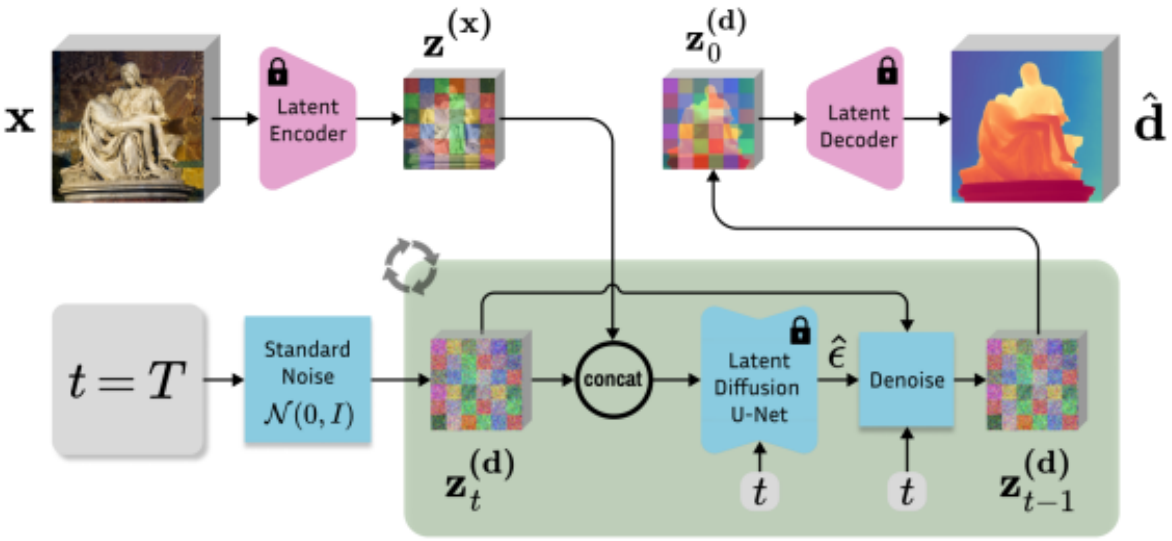


Figure 86: Marigold inference

12.7 Ensembling Over Samples *(slides 30)*

Probabilistic sampling is why different seeds give different depth maps; Marigold runs the loop N times and combines the results.

Algorithm 12.2 (Ensembling)**.** (1) Each run is affine-invariant on its own scale, so first fit a scale and a shift per run so that the maps agree with each other (minimize the pairwise distance between the aligned maps, with a penalty that keeps the range on $[0, 1]$); (2) take the pixel-wise **median**. Default $N = 10$; **no ground truth is used**.

Intuition

Ten drafts of the same drawing, each in its own unknown units. First rescale and shift each draft to line up with the others (they only agree up to (s_i, t_i)), then let the median vote at each pixel, which ignores outlier drafts.

12.8 Limitations of Marigold *(slides 31)*

- **No metric scale.**
- **No zero**: not even a common origin between two images of the same room.
- **Slow**: 50 denoising steps $\times$ 10 ensemble members $= 500$ network evaluations per image.
- **It returns a sample** (non-deterministic).

13 Back to 3D *(slides 32–34)*

Section 3.12: back-projection turns a depth map into points and shows which of the unknown parameters distort the result.

13.1 Effect of Scale and Shift Errors *(slides 33)*

A depth map is a 2.5D surface: one depth per pixel, with nothing behind what is visible.

Definition 13.1 (Back-projection)**.** A pixel with intrinsics-normalized ray $\boldsymbol{r} = \boldsymbol{K}^{-1}(u, v, 1)^\top$ and depth z back-projects to $\boldsymbol{p} = z\,\boldsymbol{r}$.

Proposition 13.2 (Affine depth error)**.** *If $\hat{z} = sz+t$, the reconstructed point is $\hat{\boldsymbol{p}} = (s+t/z)\,\boldsymbol{p}$. In particular: the **scale** s multiplies the whole cloud (a similarity, so shape is preserved); the **shift** t drags every point a constant depth along its own ray, a different displacement in every direction (the shape can distort).*

Proof. $\hat{\boldsymbol{p}} = \hat{z}\,\boldsymbol{r} = (sz + t)\boldsymbol{r} = (s + t/z)\,z\boldsymbol{r}$. □

Corollary 13.3 (Affine error in disparity)**.** *If disparity is $\hat{d} = sd + t$ then $\hat{z} = 1/\hat{d}$ and $\hat{\boldsymbol{p}} = \boldsymbol{p}/(s + tz)$: a* projective *map of space.*

Proof. $\hat{z} = \dfrac{1}{s/z + t} = \dfrac{z}{s + tz}$, so $\hat{\boldsymbol{p}} = \hat{z}\boldsymbol{r} = \boldsymbol{p}/(s + tz)$. □

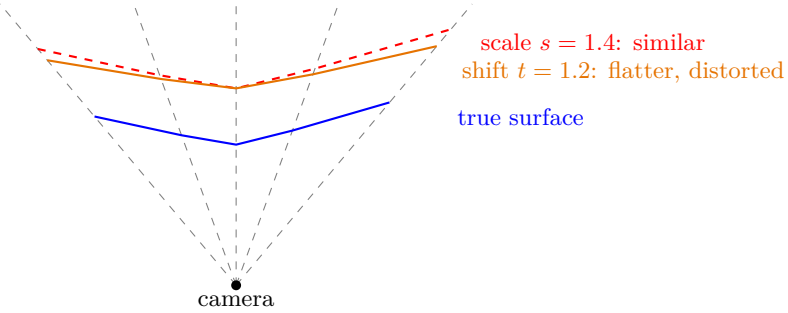


Figure 87: Original side view. A bowl-shaped surface seen along five rays. Scaling by s keeps the shape (red); adding a constant depth t along each ray (orange) keeps the depth differences but widens the lateral spread, so the bowl becomes flatter.

Intuition

Scale is a photocopier enlargement: bigger, same shape. Shift is pushing every point straight away from the camera by the same distance: because rays fan out, far-off directions move sideways more than the centre, so straight things bend. Example: $s = 1$, $t = 1$ m: a point at $z = 2$ is scaled by $1 + 1/2 = 1.5$, one at $z = 10$ by only 1.1; near points are magnified more, so a wall receding from the camera is bent.

13.2 Sources of Scale and Shift *(slides 34)*

Two numbers are required, and a single image cannot supply them. Possible sources:

- a **metric-depth model**; camera-aware models narrow the generalization gap by taking the intrinsics as input (3.8);
- a **known baseline** at training time (3.9);
- a **known object size**: the apparent-size cue, which needs recognition;
- **known camera height plus a flat-ground assumption**: the vertical-position cue, which needs flat ground;
- or a **handful of real depth measurements**.

14 Marigold-DC *(slides 35–38)*

Section 3.13: guidance from sparse depth measurements. Nothing is retrained: the denoising loop is modified at test time to move the sample toward the available measurements.

14.1 The Depth Completion Task *(slides 36)*

Definition 14.1 (Depth completion)**.** Depth completion upgrades sparse depth measurements into dense depth maps guided by a conventional image. The sparse points come from the LiDAR returns of Part I, structure-from-motion landmarks, or a phone's depth sensor.

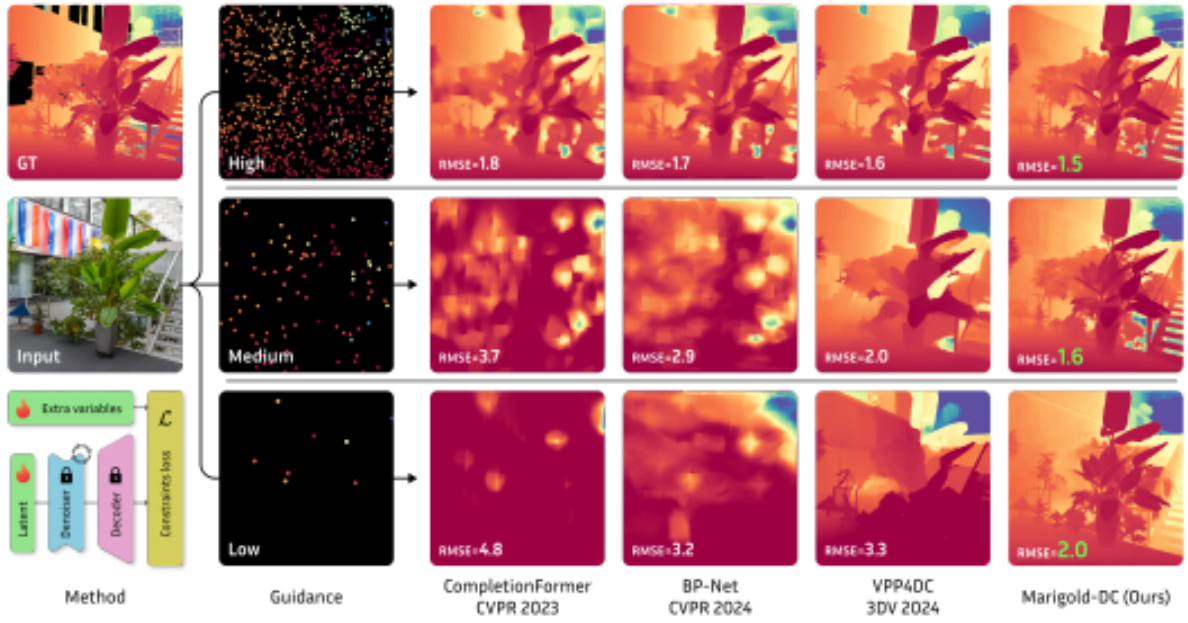


Figure 88: the comparison grid: ground truth, input image and the sparse guidance at high / medium / low density, with the outputs of CompletionFormer, BP-Net, VPP4DC and Marigold-DC and their RMSE labels

guidance density	CompletionFormer	BP-Net	VPP4DC	Marigold-DC
high	1.8	1.7	1.6	**1.5**
medium	3.7	2.9	2.0	**1.6**
low	4.8	3.2	3.3	**2.0**

RMSE values printed on slide 36 (units not stated on the slide).

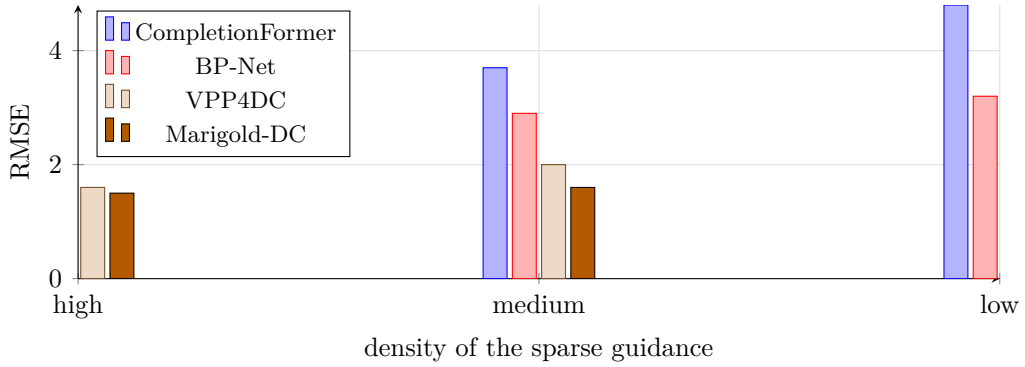


Figure 89: Plot of the RMSE numbers on slide 36. Baselines degrade quickly as the guidance gets sparser; Marigold-DC degrades slowly because the prior fills what the sensor did not sample.

14.2 Steering the Loop *(slides 37)*

Depth completion is reframed as *image-conditional depth generation guided by sparse measurements*. It is **generation rather than regression**: the output comes from a prior. The image **conditions** the network, the sparse depth **guides** the loop, and the measurements were never part of training. Every step already walks toward the model's current estimate; a second pull is added, toward the measurements.

Algorithm 14.1 (Guided step)**.** At every step of the loop: (1) read the *preview*, the current guess at the finished depth map, $\hat{\boldsymbol{z}}_0^t = \boldsymbol{z}_t - \sigma_t \boldsymbol{\epsilon}_\theta$ (Tweedie formula); (2) decode it and turn it into meters with the scale and the shift, $\hat{\boldsymbol{d}}_t^{(\mathrm{m})} = \hat{a}\,\mathcal{D}(\hat{\boldsymbol{z}}_0^t) + \hat{b}$; (3) compare it with the few pixels Ω that have a measurement $\boldsymbol{c}$, $\mathcal{L} = \left\| \hat{\boldsymbol{d}}_t^{(\mathrm{m})} - \boldsymbol{c} \right\|_\Omega$; (4) nudge the latent and the two scalars $(\hat{a}, \hat{b})$ toward the measurements by gradient descent on $\mathcal{L}$; (5) take the ordinary DDIM step.

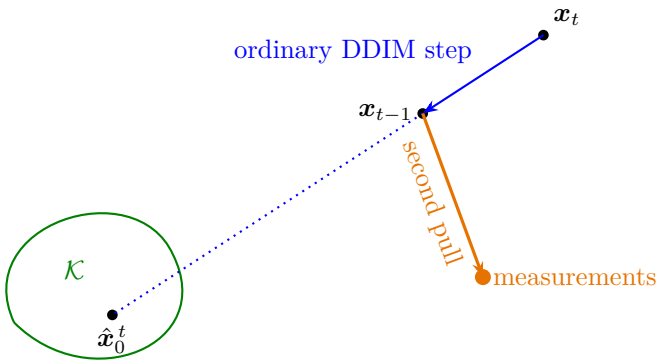


Figure 90: The guided step (redrawn from slide 37): DDIM step plus a second pull toward the sparse measurements.

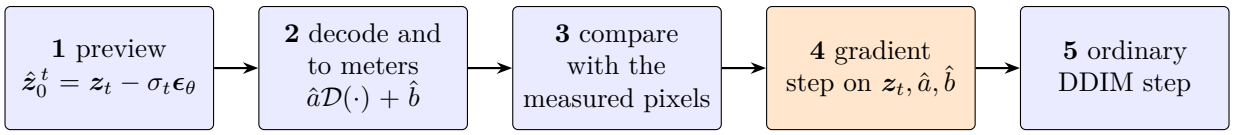


Figure 91: The five steps as a pipeline.

14.3 Nothing Retrained *(slides 38)*

Marigold gives affine-invariant depth; the sparse map gives *metric* depth at a few pixels. The unknowns are the **latent**, which decides the shape, and the **scale and shift**, which decide the meters. All three are optimized, for one image, over fifty steps, and then discarded. Gradients are taken with respect to the latent and the two scalars, so **no weights change**: the U-Net and the VAE are frozen throughout.

Theorem 14.2 (Zero-shot property, as claimed on the slide)**.** *There is no depth-completion dataset, no architecture change and no new checkpoint. The method is therefore zero-shot across indoor and outdoor scenes, where the baselines ship one checkpoint per domain.*

Intuition

The frozen network is a very good but unit-less sketch artist; the LiDAR points are a few tape-measure readings. At each step the sketch is bent slightly (through the latent) so that it does not contradict the readings, and the ruler (scale and shift) is adjusted until the sketch reads in meters. Because the readings are used only at test time, the same model handles a room and a street: the debts of Section 1 are paid at once, the prior supplying what the sensor cannot sample and the sensor the two numbers the prior cannot know.

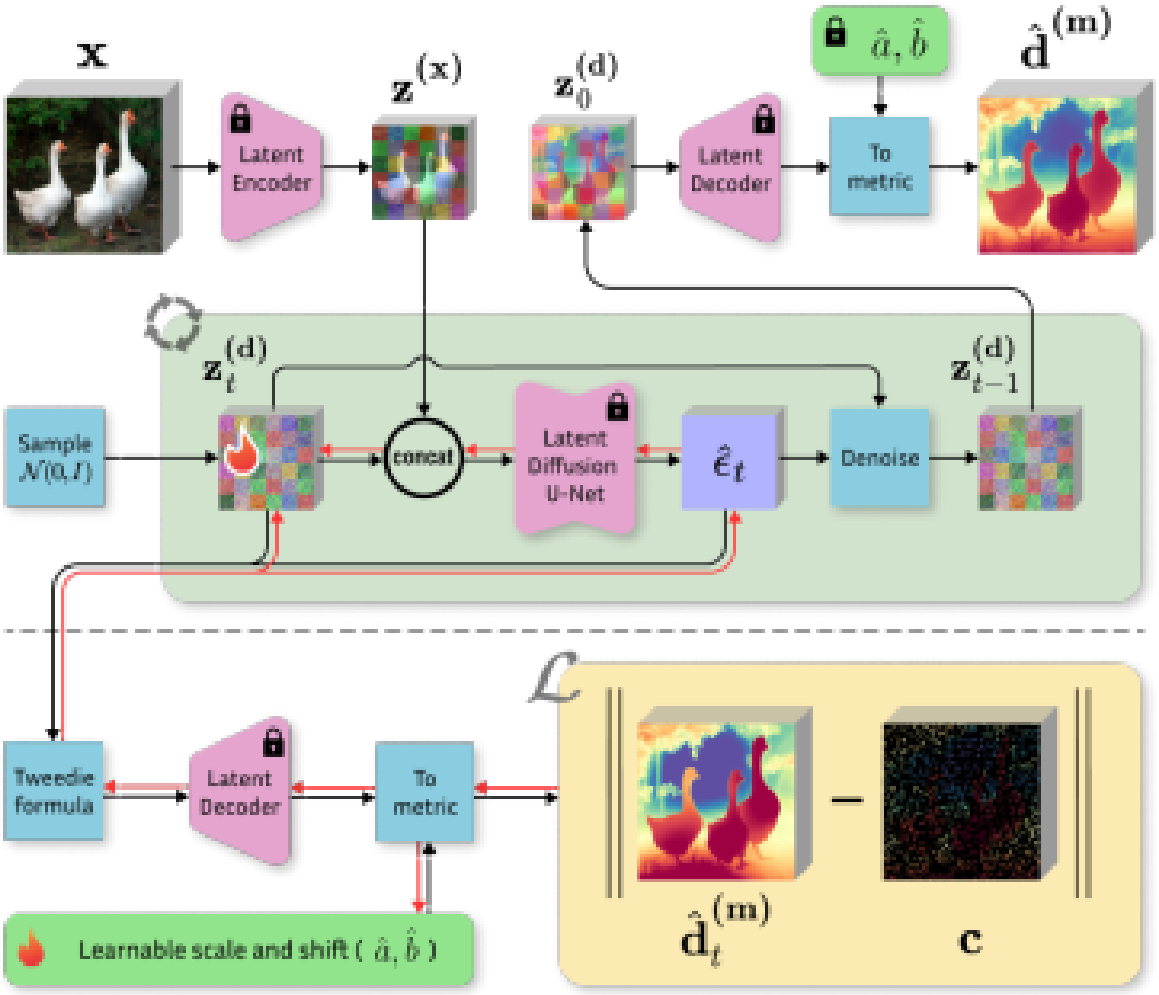


Figure 92: the Marigold-DC diagram: goose photograph, encoder/decoder, frozen U-Net loop with the red gradient arrows, sparse map **c** and the resulting metric depth (the block structure is redrawn in the figures above)

Summary *(slides 39–40)*

Comparison of the methods covered

method	pays with	output	strength	main limitation
Pulsed LiDAR	time	metric, sparse	exact where it lands; error nearly range-independent	stripes growing $\propto r$; no return $\neq 0$; mixed points at edges
Rectified stereo	baseline	metric, dense where matched	dense; metric from b	error $\propto z^2/(fb)$; fails on texture-poor, repeated, occluded regions
Structure from motion	computation	shape up to scale	no second camera needed	unknown global scale; rigid scene
Structured light (NYU)	projected pattern	metric, dense indoors	dense labels at short range	inpainted holes; indoor only
Geometric cues (size, vertical position)	assumptions	metric if assumptions hold	closed form, interpretable	need known H, flat ground, known Y and horizon
Eigen scale-invariant loss	prior	depth up to 1 scale	loss respects scale ambiguity	one scale per scene; dataset-specific sensor errors
MiDaS	prior, mixed data	affine-invariant disparity	mixes datasets with no common zero	not metric; shape only
Depth Anything V2	prior, pseudo-labels	affine-invariant inverse depth	scales with unlabeled real images	not metric without fine-tuning; inherits teacher's prior

Every method in this lecture bought depth with a different resource. The last section bought it with two at once.

method	paid with	when	free parameters left
A laser (3.3, 3.4)	time	test	0, but sparse
Stereo	a known baseline	test	0, error grows as z^2
Structure from motion	viewpoints, no baseline	test	1, one global scale
Supervised (3.7)	a prior, on sensor labels	train	0, 1 or 2, set by the labels
Self-supervision (3.9)	a baseline	train	0, for that rig
Marigold (3.11)	a generative prior	train	2
Marigold-DC (3.13)	a prior and time	both	0, dense, nothing retrained

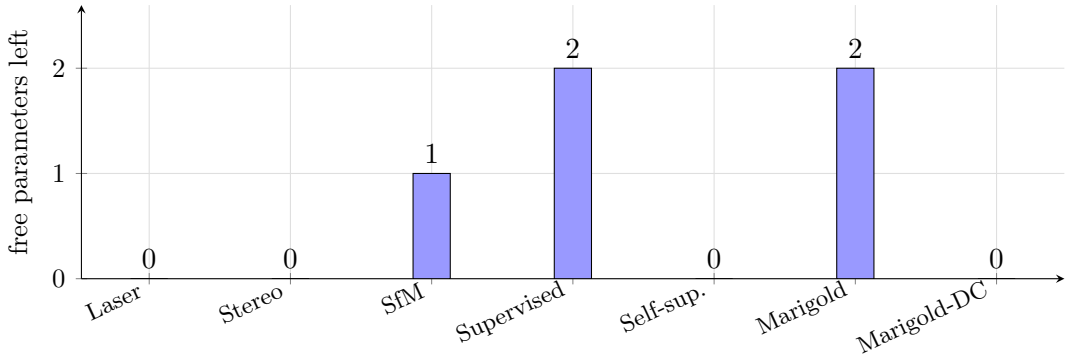


Figure 93: Free parameters left after each method (last column of the table). "Supervised" is drawn at its worst case: it ranges from 0 to 2 depending on the labels.

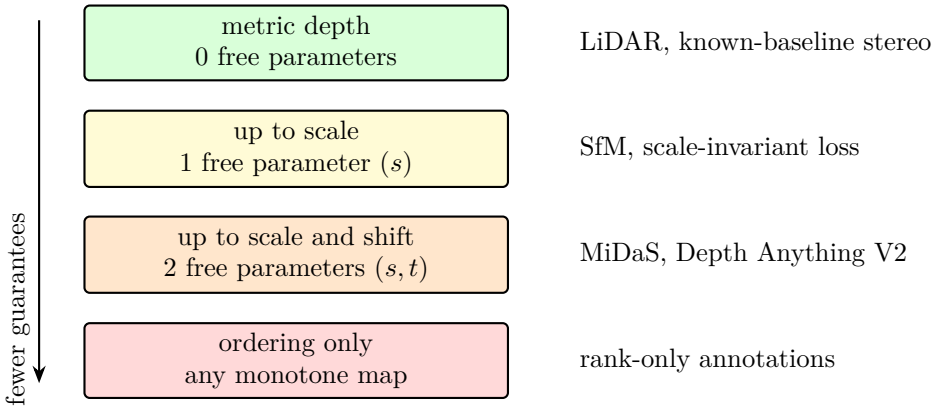


Figure 94: Output spaces ordered by how much of the depth is actually determined. Moving down the ladder, the claim weakens but the data requirement does too.

What each method lets you claim

Key take-aways

- **Depth from one pixel is undetermined.** Back-projection returns a ray; metric depth always needs a *currency*: time (LiDAR), a baseline (stereo) or a prior (learning). A single image bypasses cost, sparsity and hardware, but not the physics.
- **The two physical principles fail differently.** Time of flight has a range-independent timing floor but sparse, structured samples; triangulation is dense but its error grows as $z^2/(fb)$. LiDAR labels must be converted with the camera-frame z, a z-buffer and a validity mask.
- **A learned depth network is a prior, and its output is ambiguous.** The likelihood is flat along each ray, so scale-invariant and affine-invariant losses (in log space or in disparity) ask the network only for shape; disparity is preferred because planes stay planes under an affine map in inverse depth.
- **The prior is both the method and the failure mode.** Networks read vertical image position more than apparent size, only partially respect pitch and roll, and absorb the training focal length; anamorphic art defeats them with no sign of uncertainty.
- **Always ask "recovered or ambiguous?"** Whether a reported depth is in meters, up to scale, or up to scale and shift determines what it can be used for (planning versus shape) and is the key to reading benchmark numbers (aligned first, then scored with AbsRel and δ).
- **The prior supplies what the sensor cannot sample, and the sensor supplies the two numbers the prior cannot know.**
- Self-supervision from stereo needs no depth labels, but the photometric loss is blind on flat and occluded pixels (hand-written priors fill the gap) and is metric only for the baseline it was trained with.
- A diffusion denoiser is a prior: it learns the gradient of a smoothed distance to the data, is defined everywhere, and can be sampled, conditioned, or guided; regression by contrast returns the conditional mean, which blurs depth edges.
- Marigold reuses Stable Diffusion's weights: frozen VAE, U-Net fine-tuned with the image latent concatenated; percentile normalization makes the output affine-invariant, so it gives shape but no meters, no common zero, and costs $50 \times 10 = 500$ evaluations.

- An affine error $\hat{z} = sz + t$ maps to $\hat{\boldsymbol{p}} = (s + t/z)\boldsymbol{p}$: scale is harmless to shape, shift distorts it. Marigold-DC fixes both by optimizing the latent, scale and shift at test time against a few metric points, with no retraining.

Acknowledgments and sources *(slides 41–42)*

The diffusion section (3.10) is adapted from *Introduction to Diffusion*, MIT 6.S183, IAP 2026, Lecture 1, by D. Pfrommer; parts of both halves are adapted from *Depth Estimation* (Politecnico di Milano, 2024) by L. Magri. The reference list on slide 42 (Pfrommer, Yuan, Ke et al., Rombach et al., Yang et al., Ranftl et al., Godard et al., Ho–Jain–Abbeel, Song–Meng–Ermon, Karras et al., and others) and further reading (Yuan's smalldiffusion) are given there.

A Supplementary derivations (not on the slides)

The following results are *not* on the slides; they are added to justify remarks made in the main text.

Proposition A.1 (DDIM step = re-noising the clean estimate)**.** *Equation* (13) *is equivalent to* $\boldsymbol{x}_{t-1} = \hat{\boldsymbol{x}}_0^t + \sigma_{t-1}\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t)$.

Proof. $\hat{\boldsymbol{x}}_0^t = \boldsymbol{x}_t - \sigma_t\boldsymbol{\epsilon}_\theta$, hence $\boldsymbol{x}_t = \hat{\boldsymbol{x}}_0^t + \sigma_t\boldsymbol{\epsilon}_\theta$ and $\boldsymbol{x}_{t-1} = \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\boldsymbol{\epsilon}_\theta = \hat{\boldsymbol{x}}_0^t + \sigma_{t-1}\boldsymbol{\epsilon}_\theta$. □

Reading: keep the current clean estimate and re-attach the *same* predicted noise, only with the smaller amplitude σ_{t-1}.

Proposition A.2 (Irreducible floor of the training loss)**.** *For the ideal denoiser,* $L_{simple}(\boldsymbol{\epsilon}^*) = \mathbb{E}_{\boldsymbol{x}_\sigma,\sigma}\,\mathrm{tr}\,\mathrm{Cov}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma)$*, which is strictly positive whenever the posterior of* $\boldsymbol{x}_0$ *given* $\boldsymbol{x}_\sigma$ *is not a point mass.*

Proof. Applying the conditional-mean decomposition of Section 3.1 with $Y = \boldsymbol{\epsilon}$ and $f = \mathbb{E}[\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma]$ gives $\mathbb{E}\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}^*\|^2 = \mathbb{E}\,\mathrm{tr}\,\mathrm{Cov}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma)$; and $\boldsymbol{\epsilon} = (\boldsymbol{x}_\sigma - \boldsymbol{x}_0)/\sigma$, so $\mathrm{Cov}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma) = \mathrm{Cov}(\boldsymbol{x}_0 \mid \boldsymbol{x}_\sigma)/\sigma^2$. □

Reading: this explains why the slide-14 curve stops around 0.41 rather than 0: the remaining loss is genuine ambiguity about which data point the noisy input came from, not an optimization failure.

Part III

Lecture 4

Abstract

These notes follow the slide deck *04 Stereo Depth Estimation* (60 slides) section by section; every header carries its slide range. Formal statements (definitions, lemmas, proofs) are merged in where the slides introduce a topic, and blue *Intuition* boxes explain the "why" in plain words. **Source note:** only the slide deck was uploaded, no companion chapter. The formal content therefore comes from standard references that the slides themselves cite (Hartley–Zisserman / Szeliski for geometry) and from the papers named on the slides (Zbontar–LeCun, Mayer et al., Kendall et al., Wang et al., Leroy et al.). Where a statement goes beyond what the slides say, it is marked as such. Diagrams are redrawn in TikZ; photographs are left as dashed placeholders that name the slide to copy from.

Agenda and Learning Objectives *(slides 1–2)*

The lecture has five parts: 4.1 Two-View Geometry, 4.2 Block Matching, 4.3 Siamese Networks, 4.4 End-to-End Learning, 4.5 Stereo Without Calibration. After it you should be able to:

1. explain why calibration and rectification reduce stereo matching to a 1D search, and derive $z = fb/d$ together with its error;
2. explain block matching, its failure modes and the left-right consistency test;
3. describe how learned matching costs and end-to-end networks entered stereo;
4. explain how matching in 3D removes calibration and rectification from the matching step, and what it gives up.

1 Two-View Geometry *(slides 3–13)*

1.1 Depth Cues and Binocular Stereopsis *(slides 4)*

Recovering depth from a single image is *ill-posed* (previous lecture). Three ways around it:

- **Time-of-flight (ToF)** sensors measure the round-trip time to an obstacle ⇒ metric depth.
- **Stereo vision** triangulates using two views; because the baseline is known, depth is metric.
- **A depth prior** (object assumptions or a neural network) usually gives depth only *up to an affine* transformation.

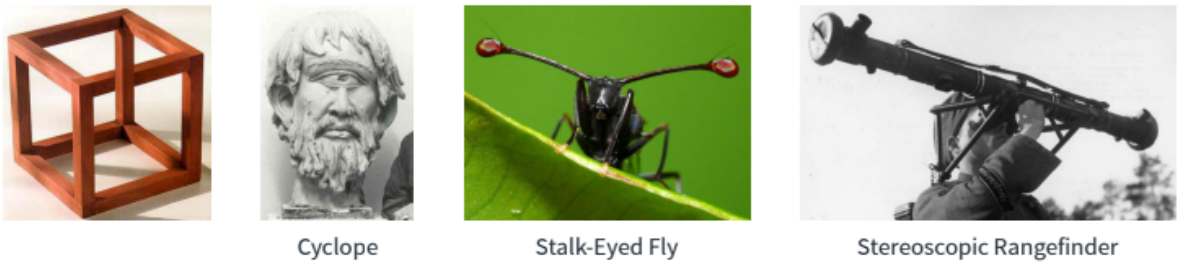


Figure 95: Historical and natural binocular systems: Cyclops (cube illusion), stalk-eyed fly, stereoscopic rangefinder

Intuition

Your two eyes are a metric ruler *because* the distance between them is fixed. A single eye (or a monocular network) can say "this is nearer than that" but not "this is 12 m away". Stereo trades a second camera for a physical scale.

1.2 Two-View Stereo Matching *(slides 5)*

Goal: recover the *disparity* (proportional to inverse depth) for every pixel of a reference image from two input images.



Figure 96: Left image, right image and the colour-coded disparity map (Geiger et al., ACCV 2010).

1.3 The Classical Stereo Pipeline *(slides 6)*

Task: build a dense 3D model of a static scene from two images. Pipeline:

1. calibrate the cameras intrinsically and extrinsically;
2. rectify the images using the calibration;
3. compute a disparity map for the reference image;
4. remove outliers with a consistency/occlusion test;
5. obtain depth from disparity using the calibration;
6. build a 3D model, e.g. by volumetric fusion and meshing.

Steps 1–2 (highlighted on the slide) are exactly what Section 5 will remove; we revisit this list at the end (*The Pipeline Revisited*).

1.4 Epipolar Geometry *(slides 7)*

Definition 1.1 (Essential matrix)**.** Let a 3D point $\boldsymbol{x}$ project to $\bar{\boldsymbol{x}}_1 \propto \mathbf{K}_1\boldsymbol{x}$ in camera 1 and to $\bar{\boldsymbol{x}}_2 \propto \mathbf{K}_2(\mathbb{R}\boldsymbol{x}+\boldsymbol{t})$ in camera 2 ($\bar{\boldsymbol{x}}$ are homogeneous pixel coordinates). The *normalized coordinates* are $\tilde{\mathbf{x}}_i = \mathbf{K}_i^{-1}\bar{\boldsymbol{x}}_i$. The *essential matrix* is $\mathbb{E} = [\boldsymbol{t}]_\times\mathbb{R}$, where $[\boldsymbol{t}]_\times$ is the skew-symmetric matrix with $[\boldsymbol{t}]_\times\boldsymbol{v} = \boldsymbol{t}\times\boldsymbol{v}$.

Proposition 1.2 (Epipolar constraint)**.** *Corresponding points satisfy* $\tilde{\mathbf{x}}_2^\top\mathbb{E}\,\tilde{\mathbf{x}}_1 = 0$. *Hence the match of* $\bar{\boldsymbol{x}}_1$ *lies on the* epipolar line $\tilde{\boldsymbol{l}}_2 = \mathbb{E}\tilde{\mathbf{x}}_1$ *in image 2, and the match of* $\bar{\boldsymbol{x}}_2$ *lies on* $\tilde{\boldsymbol{l}}_1 = \mathbb{E}^\top\tilde{\mathbf{x}}_2$ *in image 1. All epipolar lines of an image pass through its* epipole.

Proof. Up to positive scale, $\tilde{\mathbf{x}}_1 \propto \boldsymbol{x}$ and $\tilde{\mathbf{x}}_2 \propto \mathbb{R}\boldsymbol{x}+\boldsymbol{t}$. Then $(\mathbb{R}\boldsymbol{x}+\boldsymbol{t})^\top[\boldsymbol{t}]_\times\mathbb{R}\boldsymbol{x} = (\mathbb{R}\boldsymbol{x})^\top[\boldsymbol{t}]_\times(\mathbb{R}\boldsymbol{x}) + \boldsymbol{t}^\top[\boldsymbol{t}]_\times\mathbb{R}\boldsymbol{x} = 0+0$, because $\boldsymbol{v}^\top[\boldsymbol{t}]_\times\boldsymbol{v} = 0$ and $\boldsymbol{t}^\top(\boldsymbol{t}\times\boldsymbol{u}) = 0$ for every $\boldsymbol{v},\boldsymbol{u}$. Scaling does not change a zero. The statement on lines is the observation that $\tilde{\mathbf{x}}_2^\top\tilde{\boldsymbol{l}}_2 = 0$ is the equation of a line in image 2. □

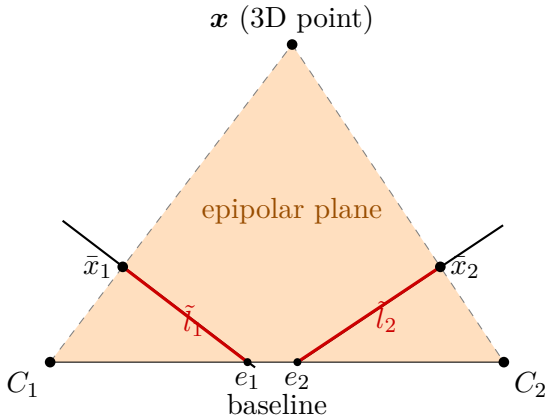


Figure 97: Epipolar geometry (redrawn from slide 7). The camera centres and the 3D point span the epipolar plane; it cuts each image plane in an epipolar line, and the baseline pierces the image planes at the epipoles.

Example 1.3 (Why the 1D search matters)**.** For a VGA image ($640 \times 480 \approx 307{,}000$ pixels), an unconstrained correspondence search tests about 3×10^5 candidates per pixel; searching along one epipolar line tests about 640. The saving is 480×, i.e. the image height.

Intuition

Given where a pixel sits in image 1, the 3D point can be anywhere along its viewing ray. Seen from camera 2, that ray is a straight line, not a blob: the matching pixel must lie on it. So we ask "which point *on this line* looks the same?" instead of "which point *in the image*?".

1.5 Image Rectification *(slides 8)*

If both cameras face exactly the same direction (image planes co-planar), the epipoles are at infinity and the epipolar lines are parallel: the correspondence of a pixel lies on the *same horizontal scanline*. Real rigs are never perfectly aligned, but we can re-warp both images by a *rotation about the camera centres*, mapping them to a common plane parallel to the baseline; this is *rectification*. Because the rotation is about the camera centre, the 3D structure need not be known.

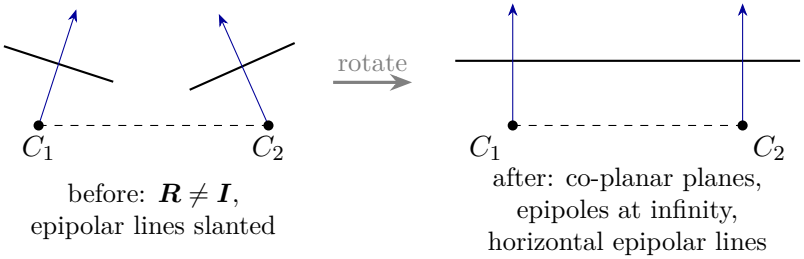


Figure 98: Rectification as a pure rotation of each camera (top view; blue arrows are optical axes). After the rotation both image planes are the same plane, parallel to the baseline.

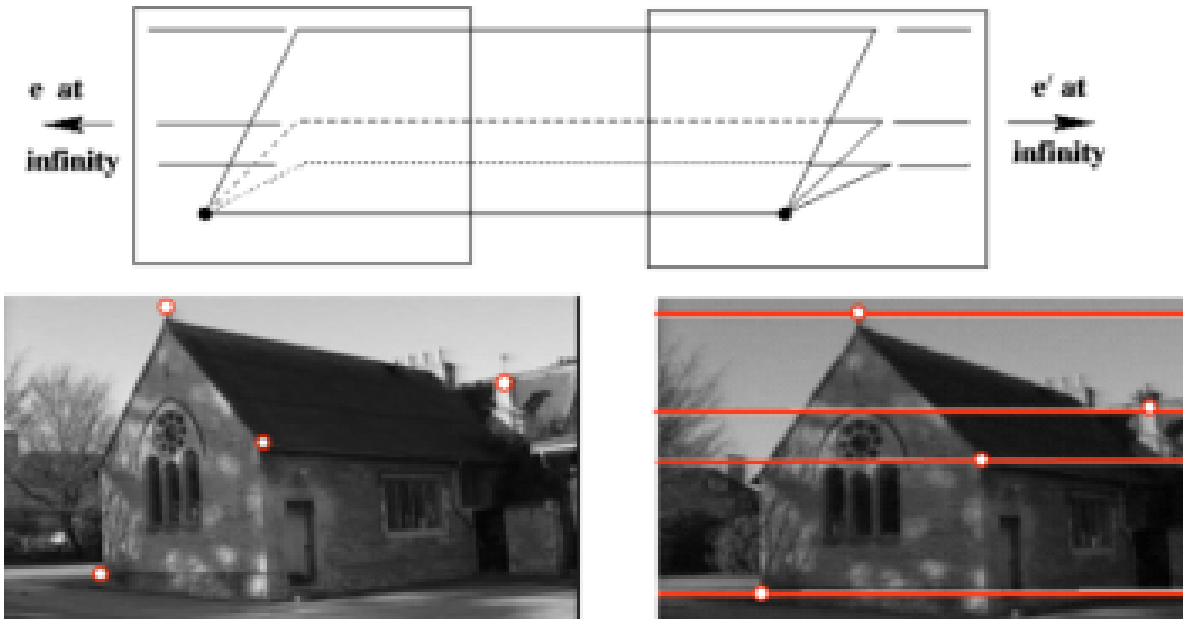


Figure 99: Example: pair of a house before and after rectification.

1.6 Rectified Epipolar Constraint *(slides 9)*

Proposition 1.4 (Rectified setup)**.** *Let* $\mathbf{K}_1 = \mathbf{K}_2 = \mathbb{R} = \boldsymbol{I}$ *and* $\boldsymbol{t} = (t, 0, 0)^\top$*. Then*

$$\mathbb{E} = [\boldsymbol{t}]_\times = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -t \\ 0 & t & 0 \end{pmatrix}, \qquad \bar{\boldsymbol{x}}_2^\top \mathbb{E}\, \bar{\boldsymbol{x}}_1 = t\, y_1 - t\, y_2 = 0 \implies y_1 = y_2.$$

Proof. With $\bar{\boldsymbol{x}}_1 = (x_1, y_1, 1)^\top$ we get $\mathbb{E}\bar{\boldsymbol{x}}_1 = (0, -t, t\, y_1)^\top$, and $\bar{\boldsymbol{x}}_2^\top (0, -t, t\, y_1)^\top = -t\, y_2 + t\, y_1$. For $t \neq 0$ this vanishes iff $y_1 = y_2$. □

Intuition

The only thing separating the two cameras is a sideways slide. A sideways slide moves every point horizontally and never vertically, so the match is on the same row. This is why stereo code can loop along scanlines.

1.7 Calculating the Rectifying Rotation Matrix *(slides 10)*

Lemma 1.5 (Rectifying rotation)**.** *Let* $\mathbb{R}, \boldsymbol{t}$ *be the relative pose* ($\boldsymbol{x}_2 = \mathbb{R}\boldsymbol{x}_1 + \boldsymbol{t}$)*. Define*

$$\boldsymbol{r}_1 = \frac{-\mathbb{R}^\top \boldsymbol{t}}{\|\mathbb{R}^\top \boldsymbol{t}\|_2}, \quad \boldsymbol{r}_2 = \frac{[(0,0,1)^\top]_\times \boldsymbol{r}_1}{\|[(0,0,1)^\top]_\times \boldsymbol{r}_1\|_2}, \quad \boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2, \quad \boldsymbol{R}_{\text{rect}} = (\boldsymbol{r}_1, \boldsymbol{r}_2, \boldsymbol{r}_3)^\top.$$

Then $\boldsymbol{R}_{\text{rect}}$ *is a rotation and* $\boldsymbol{R}_{\text{rect}}\boldsymbol{r}_1 = (1, 0, 0)^\top$*: the epipole of camera 1 becomes an ideal point in the* x*-direction.*

Proof. The centre of camera 2 in the frame of camera 1 solves $\mathbb{R}\boldsymbol{x} + \boldsymbol{t} = \boldsymbol{0}$, i.e. $\boldsymbol{x} = -\mathbb{R}^\top \boldsymbol{t}$; the epipole of image 1 is this direction, so it is $\propto \boldsymbol{r}_1$. By construction $\boldsymbol{r}_2 \perp \boldsymbol{r}_1$ (a cross product

is orthogonal to its factors), both are unit vectors, and $\boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2$ completes a right-handed orthonormal basis. Hence the rows of $\boldsymbol{R}_{\text{rect}}$ are orthonormal and $\boldsymbol{R}_{\text{rect}}\boldsymbol{r}_1 = (\boldsymbol{r}_1^\top \boldsymbol{r}_1, \boldsymbol{r}_2^\top \boldsymbol{r}_1, \boldsymbol{r}_3^\top \boldsymbol{r}_1)^\top = (1, 0, 0)^\top$. □

Algorithm 1.1 (Rectification). *Input:* two images, intrinsics $\mathbf{K}_1, \mathbf{K}_2$ and relative pose $(\mathbb{R}, \boldsymbol{t})$.

1. Estimate $\mathbb{E}$, decompose it into $\boldsymbol{t}$ and $\mathbb{R}$, and build $\boldsymbol{R}_{\text{rect}}$ as in Lemma 1.5.
2. Warp image 1: $\tilde{\mathbf{x}}_1' = \mathbf{K}\,\boldsymbol{R}_{\text{rect}}\,\mathbf{K}_1^{-1}\,\bar{\boldsymbol{x}}_1$.
3. Warp image 2: $\tilde{\mathbf{x}}_2' = \mathbf{K}\,\boldsymbol{R}_{\text{rect}}\,\mathbb{R}^\top\,\mathbf{K}_2^{-1}\,\bar{\boldsymbol{x}}_2$.

($\mathbf{K}$ is the common intrinsic matrix of the rectified pair.)

Intuition

Think of $\boldsymbol{r}_1$ as "the direction pointing from camera 1 to camera 2". Rectification spins both cameras so that this direction becomes the image x-axis. Camera 2 first needs undoing its own rotation $\mathbb{R}$ (hence $\mathbb{R}^\top$) so that both cameras are spun in the same world frame. The recipe for $\boldsymbol{r}_2$ also explains a limit stated later (slide 34): if the baseline points along the optical axis (forward motion), $\boldsymbol{r}_1 \parallel (0, 0, 1)^\top$, the cross product vanishes and $\boldsymbol{r}_2$ is undefined – planar rectification breaks down.

1.8 Rectified Pair and Disparity *(slides 11)*

In a rectified pair the correspondence of a pixel lies in the same image row; the horizontal displacement is the *disparity* $d = x_1 - x_2$ (on the slide 0–160 px, with the match to the left). Disparity is anti-proportional to depth: far points move less than near points.

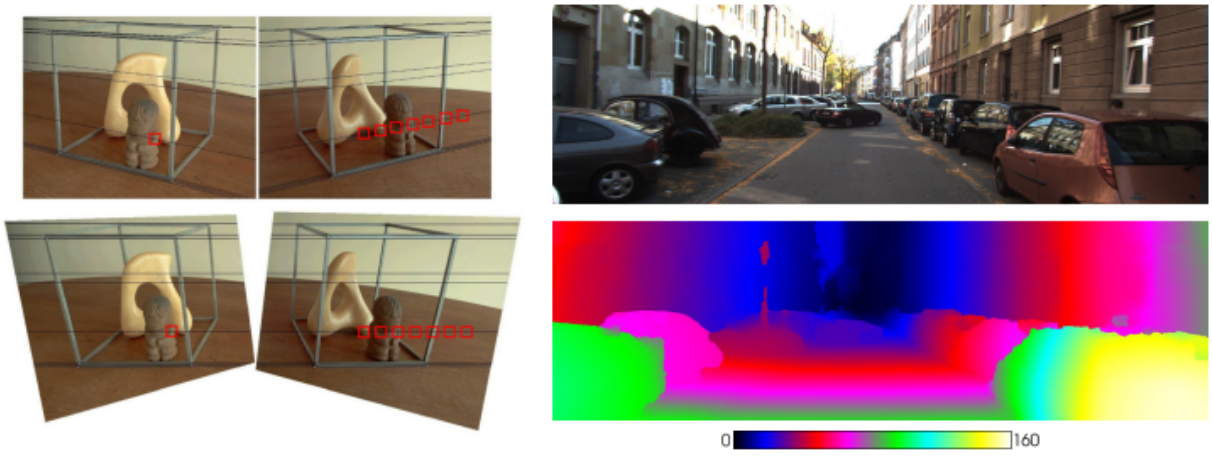


Figure 100: Rectified pair (two lab scenes and a street scene) with colour-coded disparity maps and the 0–160 px colour bar.

1.9 Disparity to Depth *(slides 12)*

Theorem 1.6 (Triangulation in a rectified pair). *Let the cameras have focal length f (in*

pixels) and baseline b, and let $d = x_1 - x_2$ be the disparity of a point at depth z. Then

$$z = \frac{f\,b}{d}.$$

Proof (slide version). The left and right rays lie in the epipolar plane and intersect at the 3D point; similar triangles give $\frac{z-f}{b-d} = \frac{z}{b}$. Hence $zb - fb = zb - zd$, so $zd = fb$. □

Proof (direct). Place camera 1 at the origin and camera 2 at $(b, 0, 0)$. A point (X, Y, z) projects to $x_1 = fX/z$ and $x_2 = f(X - b)/z$ (principal point at the origin). Subtracting, $d = x_1 - x_2 = fb/z$. □

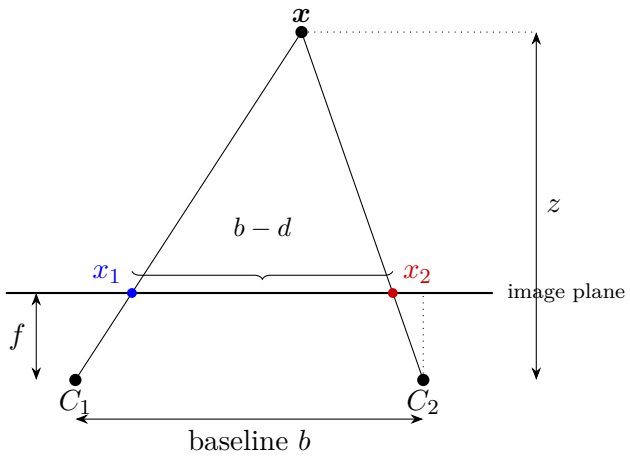


Figure 101: Triangulation (redrawn from slide 12). With $f = 1$, $b = 4$, $z = 4$ in drawing units, each camera measures its own image coordinate ($x_1 = 0.65$ from C_1, $x_2 = -0.35$ from C_2), so $d = x_1 - x_2 = 1 = fb/z$; the similar triangles $(f,\ b - d)$ and $(z,\ b)$ give the formula.

Intuition

Close one eye and point at your thumb, then switch eyes: the thumb jumps a lot. A distant tree barely moves. Disparity *is* that jump, and $z = fb/d$ converts "how much it jumped" into "how far it is". Doubling the baseline doubles every jump, so a given pixel precision resolves finer depth steps.

1.10 Worked Example: Baseline and Depth Error *(slides 13)*

Theorem 1.7 (Depth error)**.** *A disparity error δd causes, to first order, a depth error*

$$\delta z \approx \left|\frac{\partial z}{\partial d}\right| \delta d = \frac{fb}{d^2}\,\delta d = \frac{z^2}{fb}\,\delta d.$$

Proof. Differentiate $z = fb\,d^{-1}$: $\partial z/\partial d = -fb/d^2$. Substitute $d = fb/z$ to get $fb/d^2 = z^2/(fb)$. □

Corollary 1.8. *The relative error $\delta z/z = z\,\delta d/(fb)$ grows linearly with z; doubling b halves δz at every depth; if the smallest measurable disparity is $d_{\min}$, the usable range is $z_{\max} = fb/d_{\min}$.*

Example 1.9 (Car-like rig, $f \approx 721\,\text{px}$, $b \approx 0.54\,\text{m}$, $fb \approx 389\,\text{px·m}$, $\delta d = 1\,\text{px}$).

depth z	disparity d	depth error δz
10 m	38.9 px	0.26 m
20 m	19.5 px	1.03 m
40 m	9.7 px	4.11 m

(Beyond the slide: the linearisation is first order; with $d - 1$ at $z = 40\,\text{m}$ the exact value is $389/8.73 - 40 \approx 4.6\,\text{m}$.)

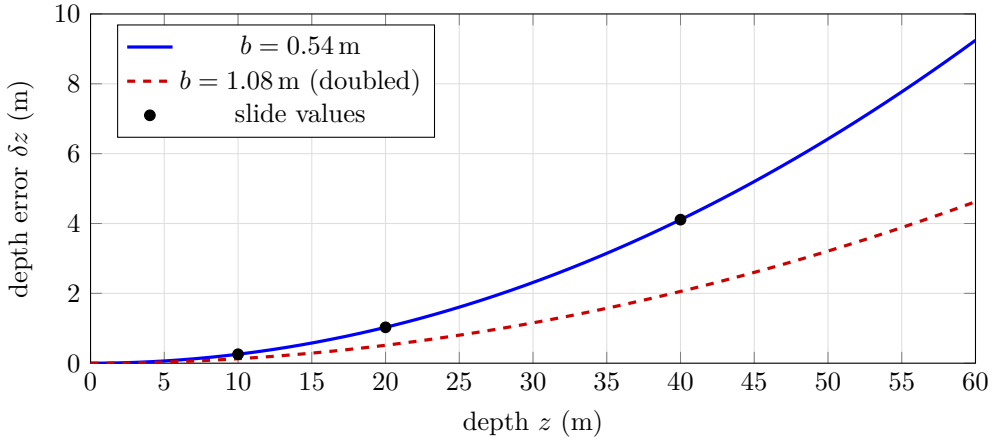


Figure 102: Depth error for a one-pixel disparity error, $\delta z = z^2 \delta d/(fb)$ with $f = 721\,\text{px}$. The error is quadratic in depth; doubling the baseline halves it everywhere (new figure, not on the slides).

Intuition

At 40 m the disparity is only 9.7 px. Being off by one pixel means being off by about 10 % of the measurement, and that 10 % turns into 4 m. Near the camera the same one-pixel slip costs a few centimetres. A wider baseline helps the arithmetic, but the two views then differ more and matching gets harder – the central design tension of stereo.

2 Block Matching *(slides 14–21)*

2.1 Correspondence Ambiguity *(slides 15)*

How do we decide that two image points correspond? Assume they look "similar". But a single pixel does not reveal local structure (ambiguity), so we compare a small patch – and even then the task is difficult (repetitive or textureless regions, changing appearance).



Figure 103: Three grayscale views of an animal with red boxes marking a tail/patch whose appearance changes

2.2 Similarity Metrics *(slides 16)*

Take two $K \times K$ windows flattened to vectors $\boldsymbol{w}_L, \boldsymbol{w}_R \in \mathbb{R}^{K^2}$.

Definition 2.1 (SSD and ZNCC)**.**

$$\mathrm{SSD}(x, y, d) = \|\boldsymbol{w}_L(x, y) - \boldsymbol{w}_R(x - d, y)\|_2^2,$$
$$\mathrm{ZNCC}(x, y, d) = \frac{(\boldsymbol{w}_L - \mu_L)^\top (\boldsymbol{w}_R - \mu_R)}{\|\boldsymbol{w}_L - \mu_L\|_2 \, \|\boldsymbol{w}_R - \mu_R\|_2},$$

with μ_L, μ_R the means of the two windows (arguments omitted on the right). SSD is a *cost* (minimise); ZNCC is a *similarity* in $[-1, 1]$ (maximise).

Lemma 2.2 (Gain/bias invariance)**.** *For $a > 0$ and $b \in \mathbb{R}$,* $\mathrm{ZNCC}(a\boldsymbol{w}_L + b\mathbf{1}, \boldsymbol{w}_R) = \mathrm{ZNCC}(\boldsymbol{w}_L, \boldsymbol{w}_R)$*. Moreover, for the normalised windows $\hat{\boldsymbol{w}} = (\boldsymbol{w} - \mu)/\|\boldsymbol{w} - \mu\|_2$ one has* $\mathrm{SSD}(\hat{\boldsymbol{w}}_L, \hat{\boldsymbol{w}}_R) = 2 - 2\,\mathrm{ZNCC}(\boldsymbol{w}_L, \boldsymbol{w}_R)$*.*

Proof. The mean of $a\boldsymbol{w}_L + b\mathbf{1}$ is $a\mu_L + b$, so the centred vector is $a(\boldsymbol{w}_L - \mu_L)$ and its norm $a\|\boldsymbol{w}_L - \mu_L\|$ (as $a > 0$); a cancels in the ratio. For unit vectors $\boldsymbol{u}, \boldsymbol{v}$: $\|\boldsymbol{u} - \boldsymbol{v}\|^2 = 2 - 2\boldsymbol{u}^\top \boldsymbol{v}$, and $\hat{\boldsymbol{w}}_L^\top \hat{\boldsymbol{w}}_R = \mathrm{ZNCC}$. □

Example 2.3. Let $\boldsymbol{w}_L = (10, 20, 30, 40)$ and $\boldsymbol{w}_R = \boldsymbol{w}_L + 100 = (110, 120, 130, 140)$ (same texture, brighter exposure). Then $\mathrm{SSD} = 4 \cdot 100^2 = 40{,}000$ – a terrible score – but $\mathrm{ZNCC} = 1$, a perfect match. For the reversed window $(40, 30, 20, 10)$, $\mathrm{ZNCC} = -1$.

Intuition

SSD asks "are the numbers equal?", ZNCC asks "do the two patches have the same *pattern of ups and downs*?". When the two cameras have different gain or exposure, only the second question is the right one. Other metrics exist (Szeliski, Ch. 12.3; Hirschmüller and Scharstein, TPAMI 2009).

2.3 Block Matching *(slides 17)*

Algorithm 2.1 (Block matching with winner-takes-all)**.** *Input:* a rectified image pair.

1. Choose a disparity range $[0, d_{\max}]$.
2. For every pixel (x, y) and every d, compute the matching cost $C(x, y, d)$ (e.g. SSD), giving a *cost volume*.
3. Winner-takes-all: $\hat{d}(x, y) = \arg\min_{d \in [0, d_{\max}]} C(x, y, d)$.
4. Repeat with the roles of the images exchanged and apply the left-right consistency check to remove outliers.

Naively this costs $O(W H D K^2)$ per image.

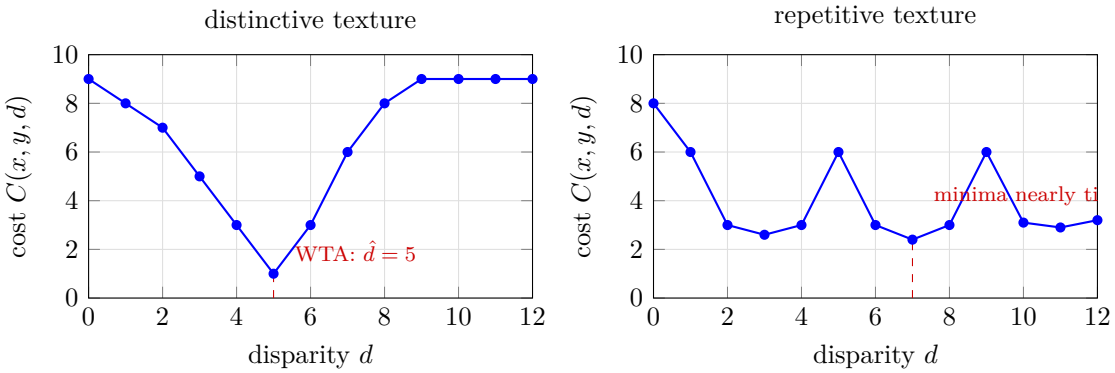


Figure 104: Illustrative cost curves along one scanline (values invented for illustration, not measured). Left: a clear unique minimum. Right: repeated texture produces several near-equal minima, so noise can make winner-takes-all pick the wrong one.

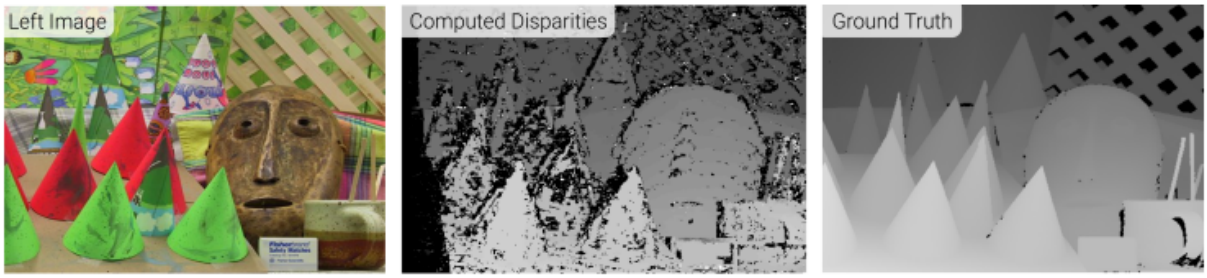


Figure 105: Left image, computed disparities (noisy, with holes) and ground truth

Intuition

Block matching is a "find-my-twin" game on one row: slide a small window of the left image along the row of the right image and keep the position with the lowest cost. Each pixel decides alone – nobody checks whether neighbours agree – which is why the output is noisy and why later sections add regularisation and learning.

2.4 Block Matching: Half Occlusions *(slides 18)*

A *half-occlusion* is a region visible in the left image but not in the right one (red region on the slide).

Proposition 2.4 (Width of the half-occluded strip)**.** *Let a fronto-parallel foreground with disparity d_f stand in front of a background with disparity $d_b < d_f$. The strip of background that is visible in the left image but hidden in the right image lies immediately left of the foreground's left edge in the left image and has width $d_f - d_b$ pixels.*

Proof. Let the foreground occupy $[a, e]$ in the left image; it appears at $[a - d_f, e - d_f]$ in the right image. A background pixel x of the left image appears at $x - d_b$ in the right image. It is hidden in the right image iff its right-image position $x - d_b$ falls inside the foreground's footprint $[a - d_f, e - d_f]$ while $x < a$ (it is not itself foreground). The condition $x - d_b \geq a - d_f$ reads $x \geq a - (d_f - d_b)$, so the hidden background pixels are exactly $x \in [a - (d_f - d_b),\, a)$, an interval of length $d_f - d_b$. □

Example 2.5. With the rig of the previous section, a car at 10 m ($d_f = 38.9$ px) in front of a wall at 40 m ($d_b = 9.7$ px) hides a strip 29.2 px wide next to it. No matching cost can find a twin there, because the twin does not exist.

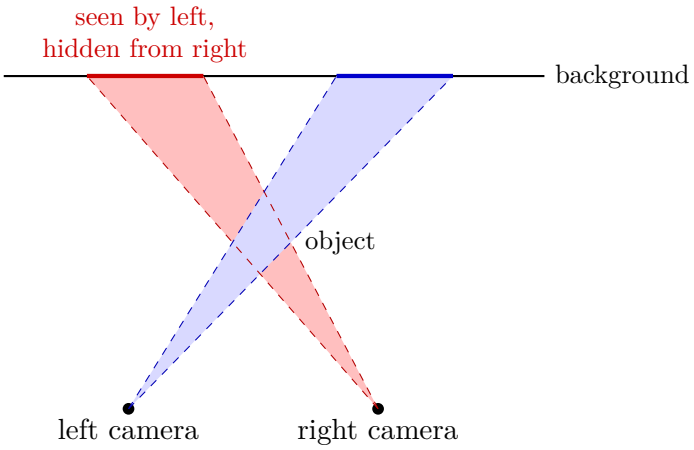


Figure 106: Half-occlusion (top view, redrawn from slide 18). Red: background strip hidden from the right camera but visible to the left one; blue: strip hidden from the left camera.

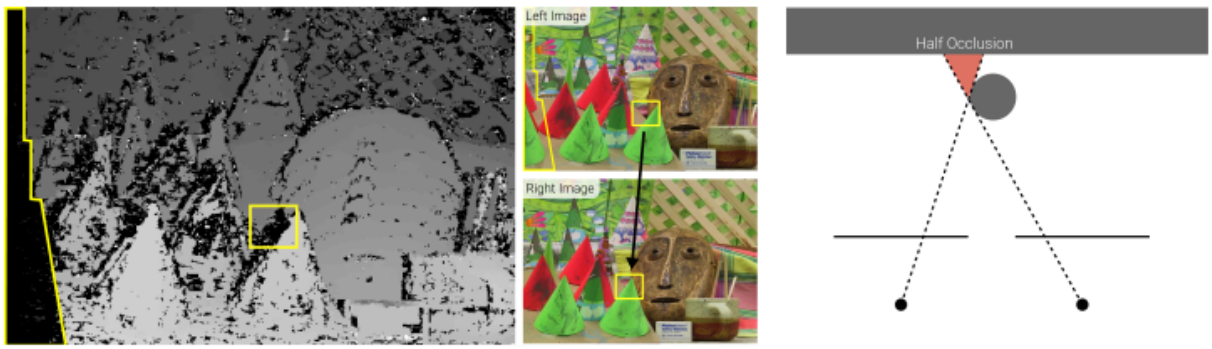


Figure 107: Left image with the half-occluded region marked in red, zoomed left/right crops, and the occlusion sketch

2.5 Block Matching: Assumption Violations *(slides 19)*

Block matching assumes that *all pixels inside the window are displaced by the same d*: the *fronto-parallel assumption*. It is often wrong: slanted surfaces deform perspectively between views, and at disparity discontinuities the window content changes differently on the two sides.

Proposition 2.6 (Disparity on a plane is affine)**.** *For a plane $\boldsymbol{n}^\top \boldsymbol{X} = h$ $(h \neq 0)$ in the rectified camera frame, $1/z$ is an affine function of the pixel (x, y); hence so is the disparity $d = fb/z$.*

Proof. A pixel $\bar{\boldsymbol{x}}$ sees $\boldsymbol{X} = z\,\mathbf{K}^{-1}\bar{\boldsymbol{x}}$. Substituting in the plane equation, $z\,\boldsymbol{n}^\top\mathbf{K}^{-1}\bar{\boldsymbol{x}} = h$, so $1/z = \boldsymbol{n}^\top\mathbf{K}^{-1}\bar{\boldsymbol{x}}/h$, which is linear in $\bar{\boldsymbol{x}} = (x, y, 1)^\top$. Multiply by fb. □

Intuition

A fronto-parallel wall has constant disparity; a floor or a slanted wall has a disparity that *ramps* across the window. If the ramp is 0.1 px per pixel and the window is 11 pixels wide, the left and right windows disagree by about 1.1 px at the edges, enough to hurt the match. Larger windows suffer more.

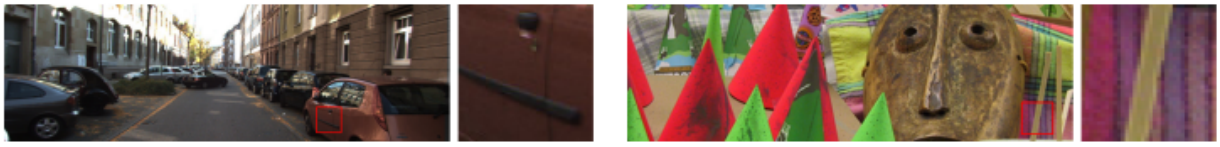


Figure 108: Street scene and Teddy-like scene crops showing perspective deformation of windows on slanted surfaces

2.6 Effect of Window Size *(slides 20)*

Trade-off. Small windows give matching ambiguities and noisy disparity maps. Larger windows give smoother results but lose detail and suffer *border bleeding*: when a window straddles a strong edge, left and right patches get the same disparity because of the dominant edge; the weakly textured background (red on the slide) lacks gradients to overcome it.

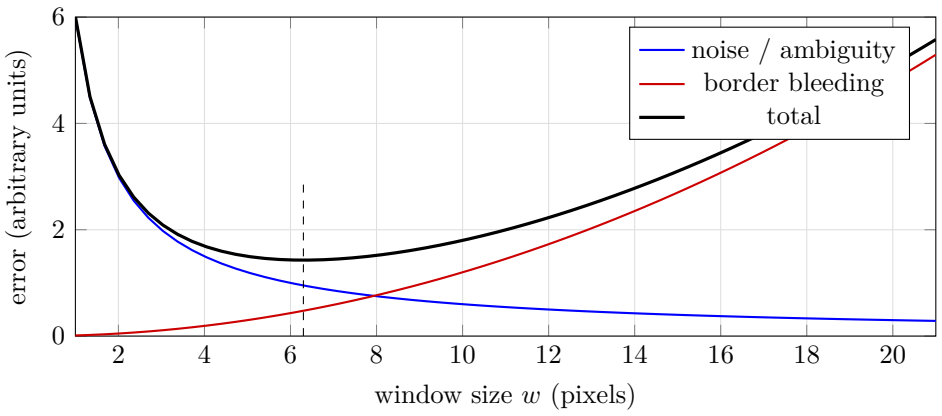


Figure 109: Conceptual bias–variance-style trade-off in window size (illustrative curves, not measured; the slides show it with 5×5 vs 11×11 results).



Figure 110: Disparity results with 5×5 and 11×11 windows, with "border bleeding" regions boxed

2.7 Left-Right Consistency Test *(slides 21)*

Definition 2.7 (Left-right consistency)**.** Let $d_L(x, y)$ be the disparity from matching with the left image as reference, and $d_R(x', y)$ the one with the right image as reference. A pixel is *consistent* (kept) if

$$\left| d_L(x, y) - d_R(x - d_L(x, y),\, y) \right| \leq \tau,$$

typically with a small threshold τ (about one pixel); otherwise it is marked invalid. In words: follow the match from left to right and back; you must land where you started (a cycle).

Example 2.8. Left disparities at $x = 2, \ldots, 6$ are $d_L = (2, 2, 2, 4, 2)$, so these pixels map to right pixels $x' = x - d_L = (0, 1, 2, 1, 4)$. Suppose $d_R(0) = d_R(1) = d_R(2) = d_R(4) = 2$. Then the differences $|d_L - d_R(x')|$ are $(0, 0, 0, 2, 0)$: with $\tau = 1$ the pixel $x = 5$ is rejected. Note that $x = 3$ and $x = 5$ both claim right pixel 1, which is impossible for a surface that is visible in both views.

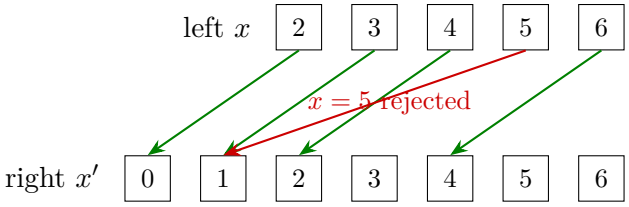


Figure 111: Toy left-to-right matches from the example (new figure). Two left pixels landing on the same right pixel flags an inconsistency.

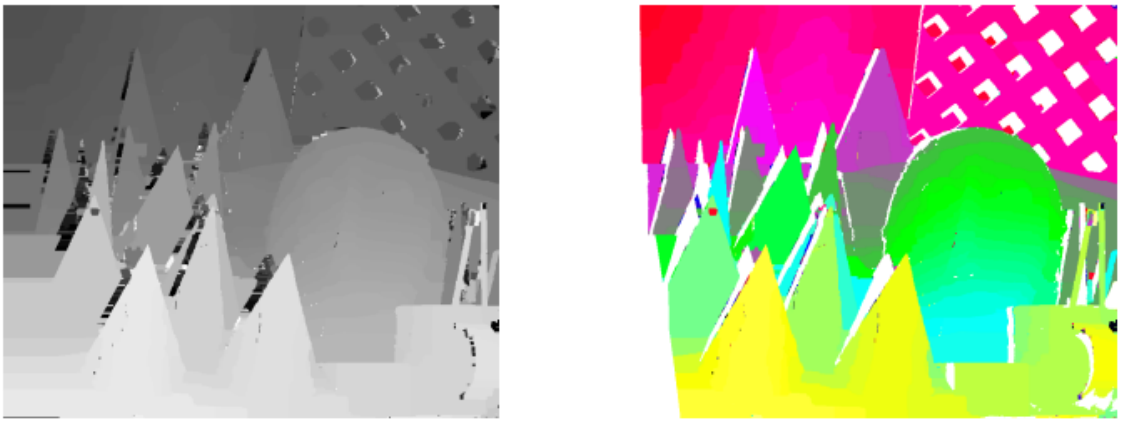


Figure 112: Left and right disparity maps used for the consistency check

Intuition

Two witnesses must tell the same story: if "left says my twin is 4 px away" but "that twin says its twin is 2 px away", at least one of them is lying. Half-occluded pixels fail this test automatically, since the supposed twin belongs to something else.

3 Siamese Networks *(slides 22–28)*

3.1 Siamese Networks for Stereo Matching *(slides 23)*

Hand-crafted features and similarity metrics do not capture all relevant geometric and radiometric invariances or occlusion patterns; the world is too complex to specify them by hand. Zbontar and LeCun (JMLR 2016) therefore treat *matching cost computation as an image classification problem*:

- train a CNN patch-wise on image pairs with ground-truth disparity (e.g. KITTI Stereo 2015);
- compute features for each of the two images with the learned model;
- correlate features between the two images (dot product);
- take the maximum per pixel (winner-takes-all) or run a global optimisation.

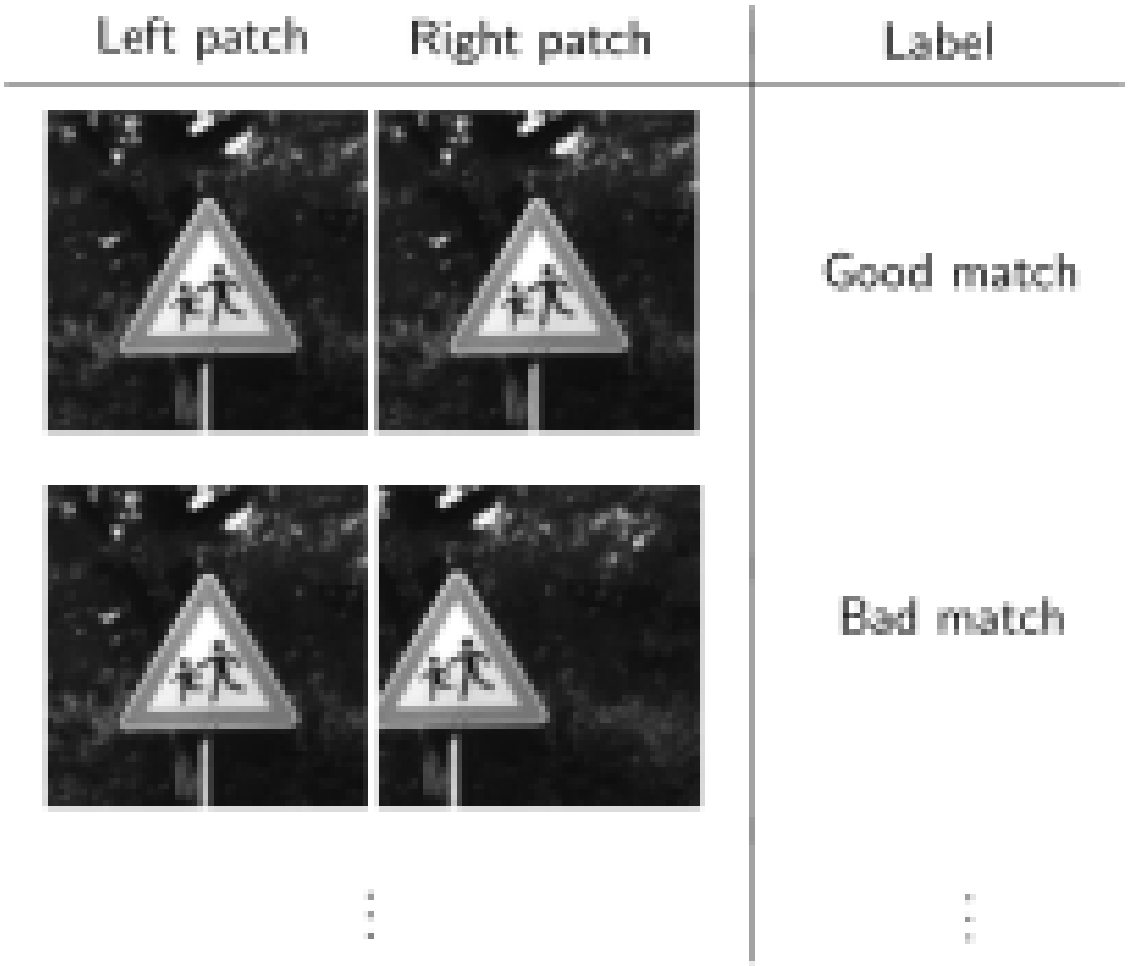


Figure 113: Small table of left patch / right patch / label (good match, bad match) with traffic-sign crops

3.2 Siamese Network Architecture *(slides 24)*

Two variants are compared on the slide.

Learned similarity	**Cosine similarity**
learn features *and* the similarity metric; potentially more expressive; slow ($W \times H \times D$ MLP evaluations)	learn features and apply a dot product; the features must do the heavy lifting; fast matching (no network evaluation)

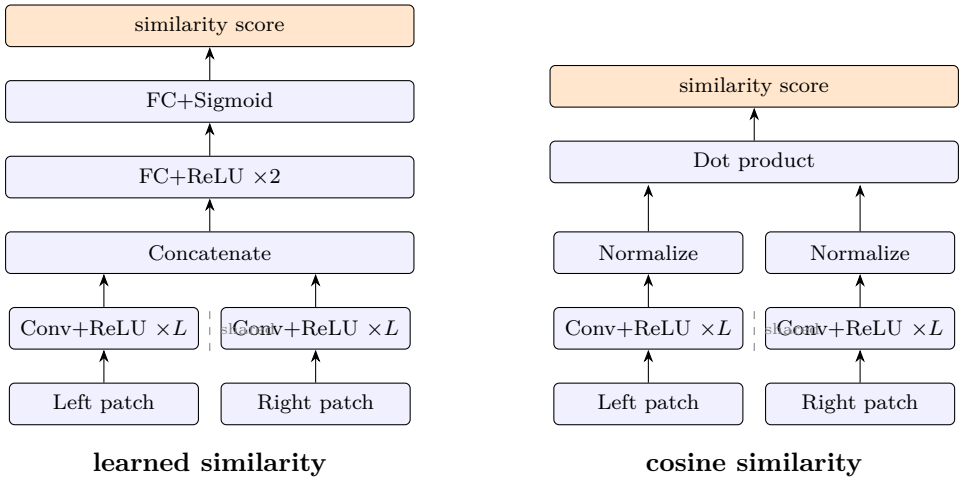


Figure 114: The two Siamese stereo architectures of Zbontar and LeCun (redrawn from slide 24; layer counts are schematic). Both towers share weights.

Example 3.1 (Why the cosine head is fast)**.** For a KITTI-sized image (1242×375) and $D = 228$ candidate disparities, the learned-similarity head needs $WHD \approx 1.06 \times 10^8$ network evaluations. The cosine head runs each tower once per image, then each cost is just a dot product of two stored feature vectors.

Intuition

Learned similarity is like hiring an expert who inspects every candidate pair. Cosine similarity is like giving every patch a fingerprint once and then comparing fingerprints: much cheaper, but the fingerprint must be good enough that a plain dot product suffices.

3.3 Training Triplets *(slides 25)*

Definition 3.2 (Training triplet)**.** A triplet is $(\boldsymbol{w}_L(\boldsymbol{x}_L^{\text{ref}}),\ \boldsymbol{w}_R(\boldsymbol{x}_R^{\text{neg}}),\ \boldsymbol{w}_R(\boldsymbol{x}_R^{\text{pos}}))$ with

$$\boldsymbol{x}_R^{\text{neg}} = (x_L^{\text{ref}} - d + o_{\text{neg}},\ y_L^{\text{ref}}), \qquad \boldsymbol{x}_R^{\text{pos}} = (x_L^{\text{ref}} - d + o_{\text{pos}},\ y_L^{\text{ref}}),$$

where d is the ground-truth disparity, $o_{\text{neg}} \sim \mathcal{U}(\{-N_{\text{hi}}, \dots, -N_{\text{lo}}, N_{\text{lo}}, \dots, N_{\text{hi}}\})$ and $o_{\text{pos}} \sim \mathcal{U}(\{-P_{\text{hi}}, \dots, P_{\text{hi}}\})$. Typical values: $P_{\text{hi}} = 1$, $N_{\text{lo}} = 3$, $N_{\text{hi}} = 6$.

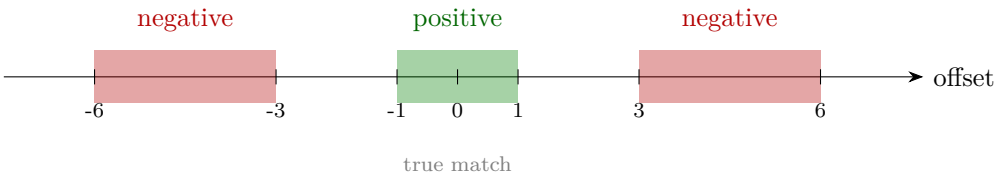


Figure 115: Where positive and negative right patches are sampled relative to the true match (new figure; ranges $P_{\text{hi}} = 1$, $N_{\text{lo}} = 3$, $N_{\text{hi}} = 6$ from the slide).

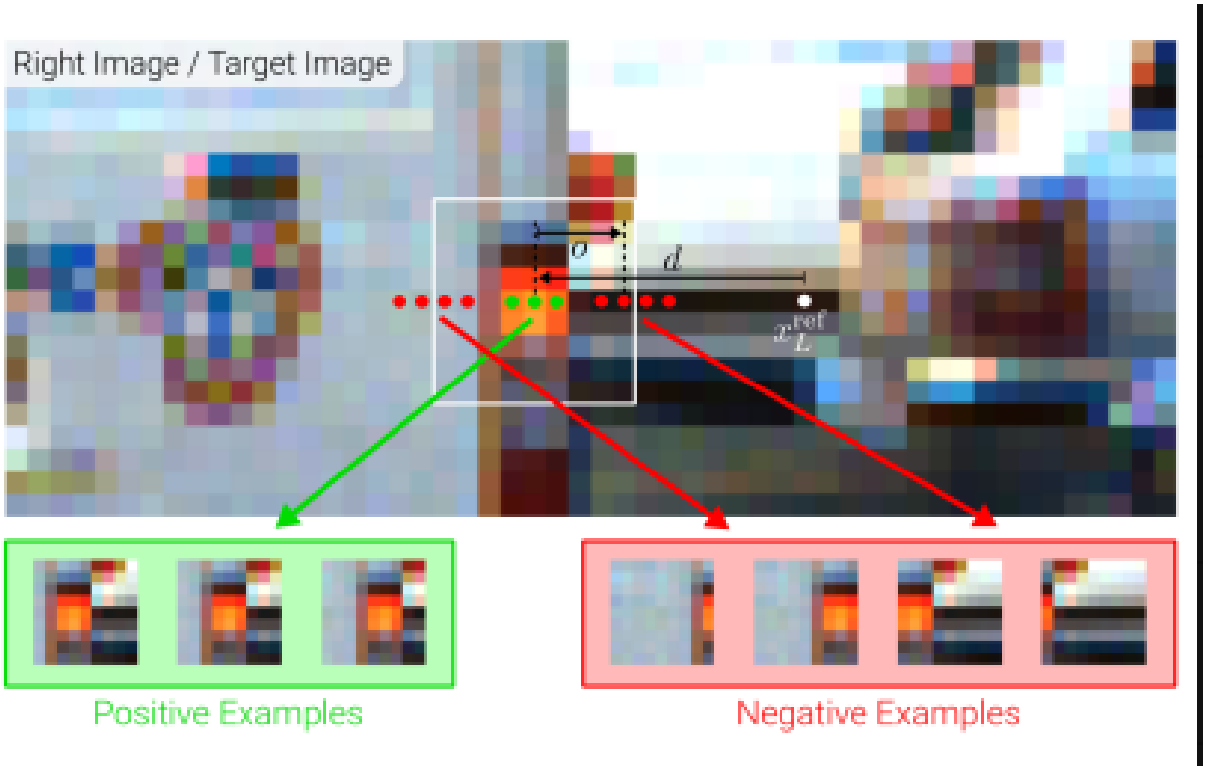


Figure 116: Zoomed right image with a window and rows of green "positive" crops and red "negative" crops

Intuition

Negatives are not random patches: they come from just a few pixels away from the truth (3 to 6 px), so they look similar and the network must learn the fine cues. Positives are allowed ± 1 px of slack, because a match is never pixel-perfect. Negatives closer than N_{lo} are excluded since they would be nearly indistinguishable from the positives. This is a mild form of *hard negative mining*. With these values there are 3 positive and 8 negative offsets to draw from.

3.4 Hinge Loss *(slides 26)*

Definition 3.3 (Hinge (margin) loss)**.** With s_+ and s_- the network scores of the positive and negative example and margin $m > 0$,

$$\mathcal{L} = \max(0,\, m + s_- - s_+).$$

Lemma 3.4. *$\mathcal{L} = 0$ iff $s_+ \geq s_- + m$. When $\mathcal{L} > 0$, $\partial\mathcal{L}/\partial s_+ = -1$ and $\partial\mathcal{L}/\partial s_- = +1$: gradient descent raises s_+ and lowers s_- at the same rate.*

Example 3.5 ($m = 0.2$, the value used in the paper)**.**

s_+	s_-	$m + s_- - s_+$	loss	meaning
0.9	0.3	-0.4	0	clearly separated, nothing to learn
0.6	0.5	0.1	0.1	right order but too close: push apart
0.4	0.7	0.5	0.5	wrong order: large correction

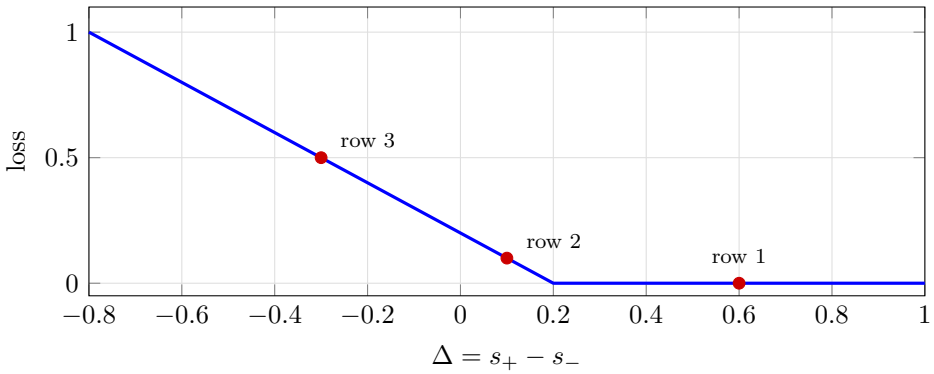


Figure 117: Hinge loss with $m = 0.2$ as a function of the score gap; red dots are the three rows of the table (new figure).

Intuition

The loss does not demand that the positive scores high in absolute terms, only that it beats the negative by a safety margin. Once it does, the example is "solved" and stops contributing – training focuses its effort on the pairs that are still confusable.

3.5 Comparisons *(slides 27)*

The slide compares, on a KITTI street scene, the left input, standard block matching, the Siamese-network matching cost, a semiglobal-matching (SGM, "optimize") variant, the left and right disparity maps, and the left-right consistency test. The qualitative message: learned costs are far less noisy than SAD/SSD-style block matching, and adding a global optimisation (SGM) on top improves them further.

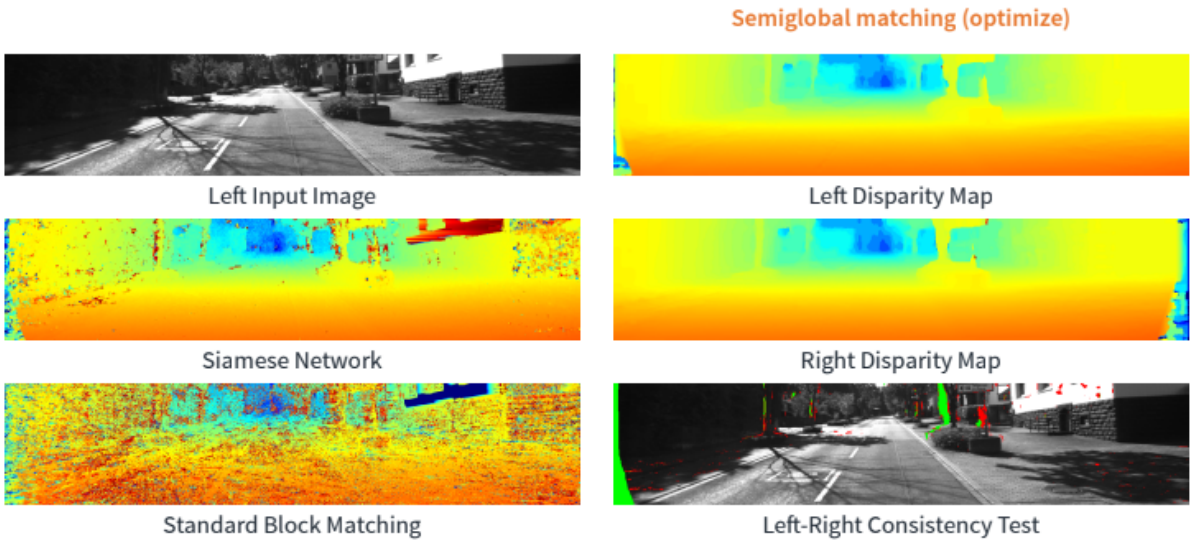


Figure 118: Grid of six images: left input, Siamese network, standard block matching, left disparity (SGM), right disparity, left-right consistency test

3.6 Similarity Constraint: Failure Cases *(slides 28)*

The underlying assumption is that *corresponding regions look similar and non-corresponding regions look different*. It fails for: **repetitions** (several patches look alike), **textureless surfaces** (everything looks alike), **occlusions** (the twin does not exist) and **non-Lambertian surfaces**

(the same point looks different from two views, e.g. reflections). The fix suggested on the slide is spatial regularisation (smoothness priors, cf. Huang, Lee and Mumford on range-image statistics).

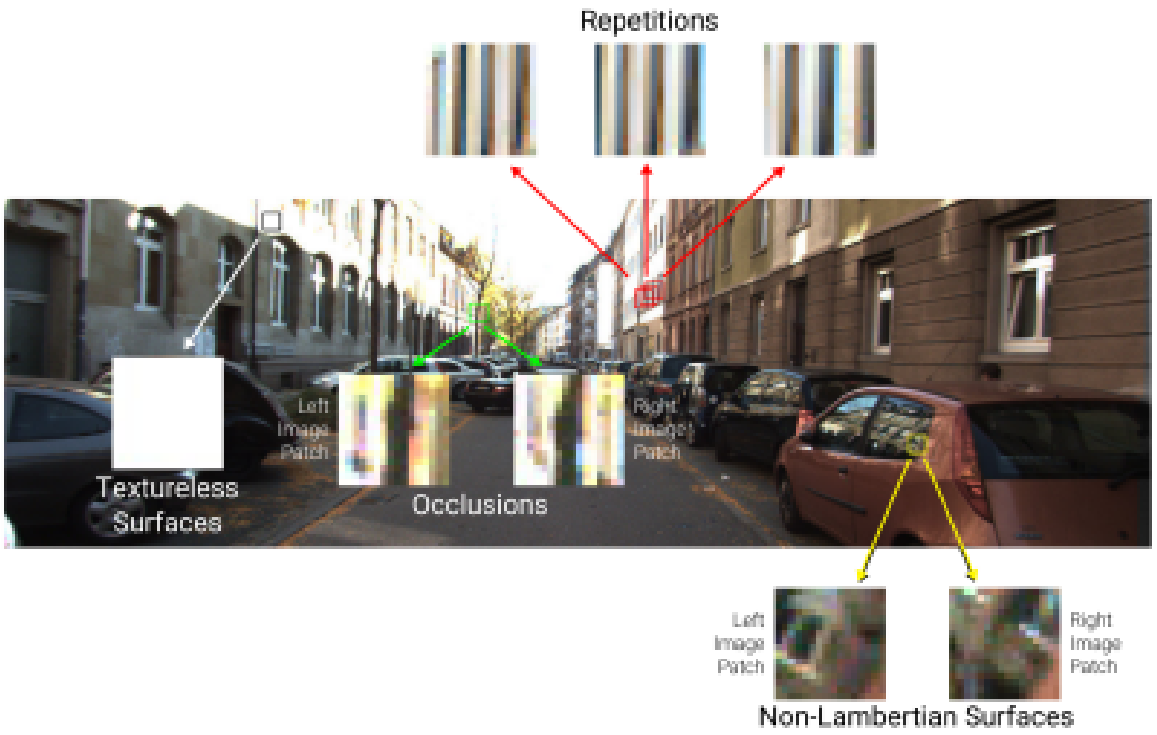


Figure 119: Street photo annotated with "Repetitions", "Textureless surfaces", "Occlusions" and "Non-Lambertian surfaces" crops

Intuition

A matching cost only looks at one pixel (or patch) at a time. When the local evidence is ambiguous or absent, it needs a prior about how the world behaves – "neighbouring pixels usually have similar depth" – to choose between candidates.

4 End-to-End Learning *(slides 29–31)*

4.1 DispNet and Synthetic Data *(slides 30)*

- DispNet (Mayer et al., CVPR 2016) was the first end-to-end trained deep network for stereo.
- It uses a U-Net-like architecture with skip connections to retain details.
- A *correlation layer* compares left and right feature maps (the slide notes: 40 px displacement at feature resolution corresponds to 160 px in the input image, i.e. a stride of 4).
- Training uses a multi-scale loss (disparity error in pixels) and *curriculum learning* (easy to hard).
- Pre-train on large *synthetic* datasets (cheap annotations: FlyingThings3D, Monkaa), then fine-tune on a little real data (expensive annotations).

Definition 4.1 (1D correlation layer *(general form, not written on the slide)*)**.** For feature maps f_L, f_R and maximum displacement D, a correlation layer outputs the volume $c(x, y, d) = \langle f_L(x, y), f_R(x - d, y) \rangle$ for $d = 0, \ldots, D$.

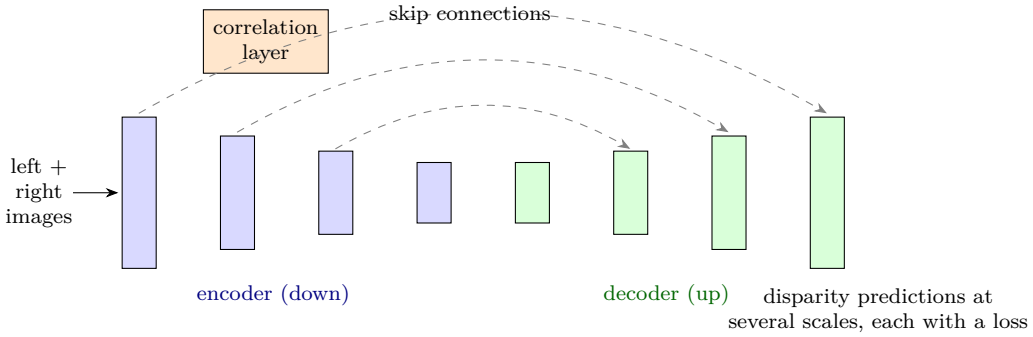


Figure 120: Schematic of a DispNet-style encoder–decoder with skip connections (new schematic, not a reproduction of the slide's network figure).

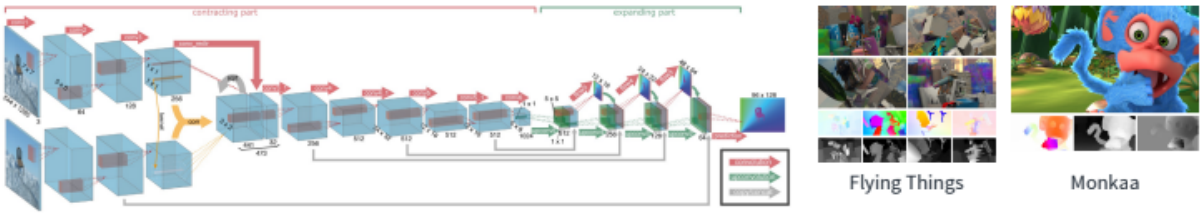


Figure 121: DispNet network diagram plus FlyingThings3D and Monkaa sample images

Intuition

Hand-designed block matching has no way to "learn" that a textureless wall continues smoothly. A network trained on millions of synthetic examples sees every ambiguity, and the U-Net structure lets coarse levels settle the global layout while fine levels sharpen the edges. The synthetic→real recipe exists because perfect ground truth is trivial for a renderer and very costly for real scenes.

4.2 GC-Net *(slides 31)*

Key idea (Kendall et al., ICCV 2017): build a *disparity cost volume* and apply *3D convolutions* to it. The learned matching cost $c_\theta(d)$ is converted to a disparity with a differentiable expectation.

Definition 4.2 (Soft argmin)**.**

$$\hat{d} = \mathbb{E}[d] = \sum_{d=0}^{d_{\max}} p(d)\, d, \qquad p(d) = \mathrm{softmax}_d(-c_\theta(d)) = \frac{e^{-c_\theta(d)}}{\sum_k e^{-c_\theta(k)}}.$$

Proposition 4.3. *(i)* $\hat{d} \in [0, d_{\max}]$ *and* $\hat{d}$ *may be non-integer (sub-pixel). (ii)* $\hat{d}$ *is differentiable in the costs with* $\dfrac{\partial \hat{d}}{\partial c(k)} = -p(k)\,(k - \hat{d})$.

Proof. (i) p is a probability vector, so $\hat{d}$ is a convex combination of the integers $0, \dots, d_{\max}$. (ii) $\partial p(d)/\partial c(k) = -p(d)(\delta_{dk} - p(k))$, hence $\partial \hat{d}/\partial c(k) = \sum_d d\,[-p(d)(\delta_{dk} - p(k))] = -p(k)k + p(k)\hat{d}$. □

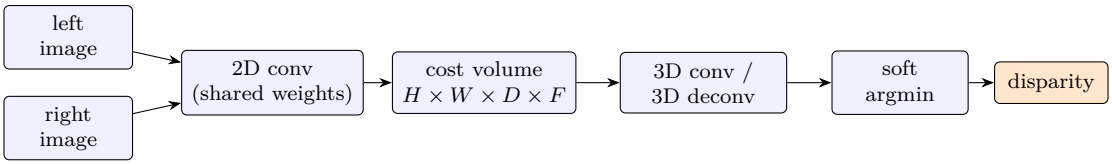


Figure 122: GC-Net pipeline (redrawn from slide 31).

Example 4.4 (Soft argmin on two cost profiles, $d = 0, \dots, 4$)**.**

	costs $c(d)$	$p(d)$	$\hat{d}$
unimodal	$(3, 1, 0, 1, 3)$	$(0.027, 0.200, 0.545, 0.200, 0.027)$	2.0
bimodal	$(0, 4, 4, 4, 0)$	$(0.487, 0.009, 0.009, 0.009, 0.487)$	2.0

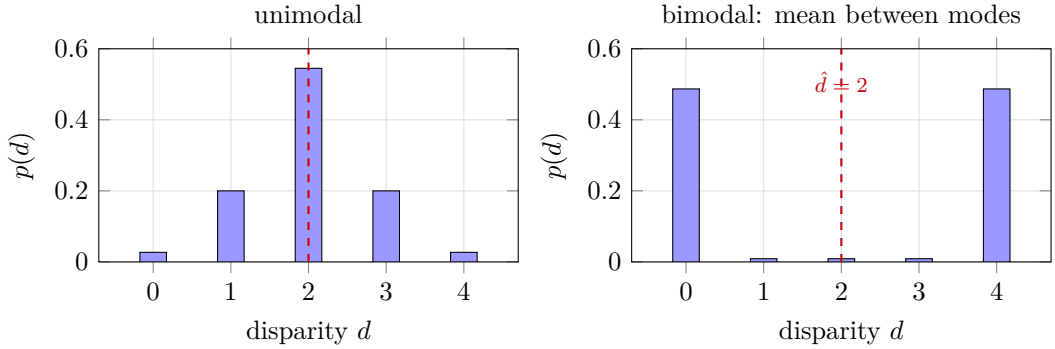


Figure 123: Soft-argmin outputs (red dashed line) for the two profiles of the example. In the bimodal case the expectation is a disparity that neither mode supports (new figure).

Example 4.5 (Memory of a 3D volume (illustrative sizes))**.** A volume with $H = 256$, $W = 512$, $D = 48$ (disparities at quarter resolution) and $F = 32$ float32 channels takes $256 \cdot 512 \cdot 48 \cdot 32 \cdot 4 \approx 0.8\,\text{GB}$, before any activations of the 3D convolutions are stored for back-propagation.

Intuition

An ordinary argmin is a hard decision with no gradient. The soft argmin "votes" with weights given by the softmax, which makes the whole network trainable end to end and allows sub-pixel answers. Its weakness is visible in the bimodal row: at an object boundary the cost has two valleys (foreground and background); the expectation averages them and invents a surface in between – the *smoothness bias* that SMD-Nets later address (backup slides). 3D convolutions add context along the disparity axis at the price of memory ("slightly better performance but large memory requirements").

5 Stereo Without Calibration *(slides 32–45)*

5.1 The Cost of Calibration *(slides 33)*

Calibration is measured *offline*, with a target, before any image pair is matched. It drifts with temperature, vibration and refocusing; a small rotation error leaves corresponding points on

different rows after rectification. Two arbitrary photographs come with no calibration at all and cannot be rectified. The learned stereo networks of Sec. 4.4 inherit all of this: rectified input and a fixed disparity range. (Calibration is the topic of the next lecture.)

5.2 Limits of Rectification and Local Matching *(slides 34)*

- Forward motion puts the epipole *inside* the image, where planar rectification breaks down (compare the remark after Lemma 1.5).
- Large rotations between the views distort the rectified images heavily.
- A wide baseline lowers the depth error (Theorem 1.7) but changes the appearance of every patch, so local matching fails.

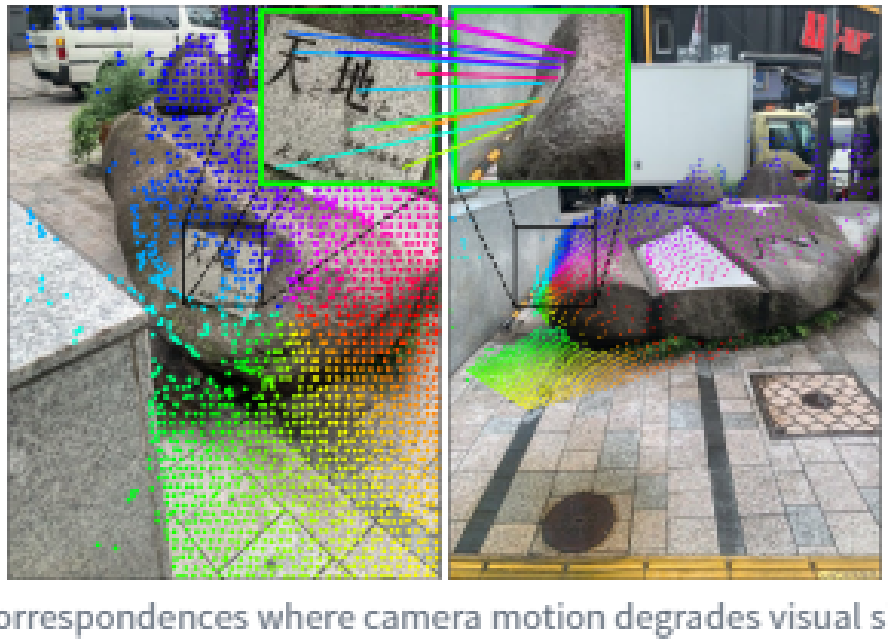


Figure 124: Dense correspondences where camera motion degrades visual similarity

5.3 Pointmaps *(slides 35)*

Definition 5.1 (Pointmap)**.** A pointmap $\boldsymbol{X}^{a,1} \in \mathbb{R}^{H\times W\times 3}$ assigns to each pixel $\boldsymbol{x}_s$ of image I_a, $a \in \{1,2\}$, its 3D point $\boldsymbol{X}^{a,1}[\boldsymbol{x}_s] \in \mathbb{R}^3$ expressed in the frame of camera 1 (the second index names the frame). For image 1 with depth z, intrinsics $\mathbf{K}_1$ and augmented pixel $\bar{\boldsymbol{x}}_s$:

$$\boldsymbol{X}^{1,1}[\boldsymbol{x}_s] = z\,\mathbf{K}_1^{-1}\bar{\boldsymbol{x}}_s.$$

DUSt3R regresses $\hat{\boldsymbol{X}}^{1,1}$ and $\hat{\boldsymbol{X}}^{2,1}$ from two images – both in the frame of camera 1 (the hat marks a prediction). Two pixels correspond when their 3D points coincide: *matching becomes a 3D problem.*

Proposition 5.2 (Disparity map as a special case)**.** *For a rectified pair with focal length f, principal point (c_x, c_y) and baseline b, a pixel (x, y) with disparity d has pointmap entry* $\boldsymbol{X}^{1,1} = \dfrac{b}{d}\,(x - c_x,\; y - c_y,\; f)^\top$.

Proof. $z = fb/d$ by Theorem 1.6, and $\mathbf{K}_1^{-1}\bar{\boldsymbol{x}} = ((x - c_x)/f, (y - c_y)/f, 1)^\top$; multiply. □

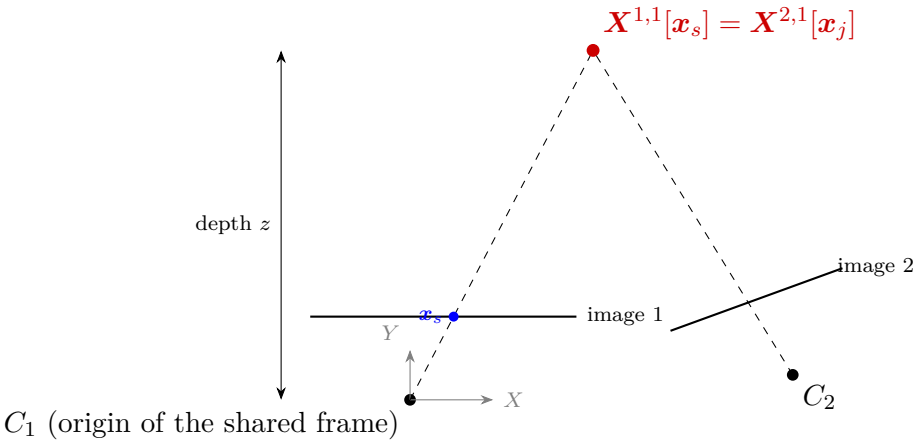


Figure 125: Pointmaps (new figure). Pixel $\boldsymbol{x}_s$ in image 1 and pixel $\boldsymbol{x}_j$ in image 2 are *the same 3D point* expressed in camera 1's frame; this is the correspondence criterion.

Intuition

A disparity map stores one number per pixel and only makes sense along a rectified row. A pointmap stores the whole 3D position in a frame shared by both images, so the notion "these two pixels see the same thing" becomes simply "their 3D points are equal" – no rows, no calibration of the pair needed to state it.

5.4 The DUSt3R Architecture *(slides 36)*

$$\boldsymbol{H}^a = \text{Encoder}(I_a), \quad \boldsymbol{H}'^1, \boldsymbol{H}'^2 = \text{Decoder}(\boldsymbol{H}^1, \boldsymbol{H}^2), \quad \hat{\boldsymbol{X}}^{a,1}, C^a = \text{Head}_{3\text{D}}([\boldsymbol{H}^a, \boldsymbol{H}'^a]).$$

- **Siamese encoder:** one ViT-Large (a vision transformer over image patches) encodes both images with *shared weights* into tokens $\boldsymbol{H}^a$.
- **Two ViT-Base decoders** exchange information through *cross-attention* and output tokens $\boldsymbol{H}'^a$.
- **Heads** regress a pointmap $\hat{\boldsymbol{X}}^{a,1}$ and a per-pixel confidence C^a from the concatenated tokens.
- **MASt3R** (blue on the slide) extends DUSt3R with local descriptors and fast reciprocal matching.

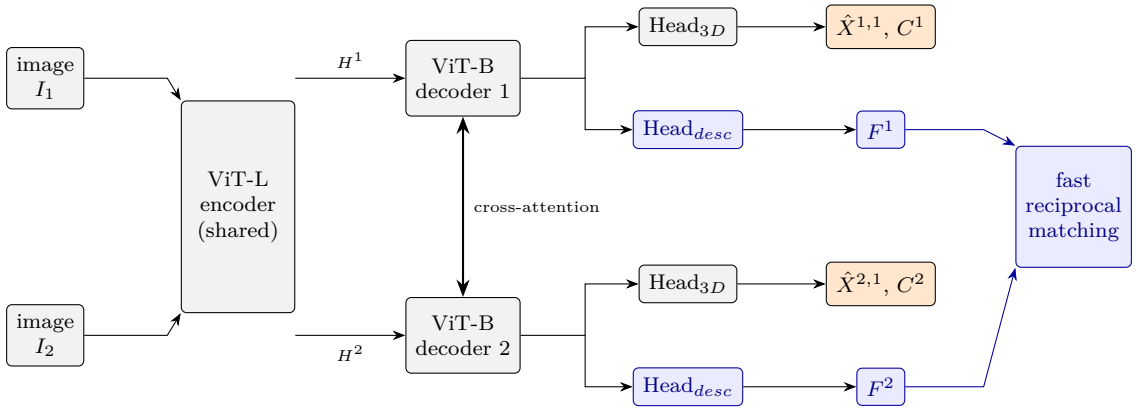


Figure 126: DUSt3R/MASt3R architecture (redrawn from slides 36 and 39; blue = MASt3R additions).

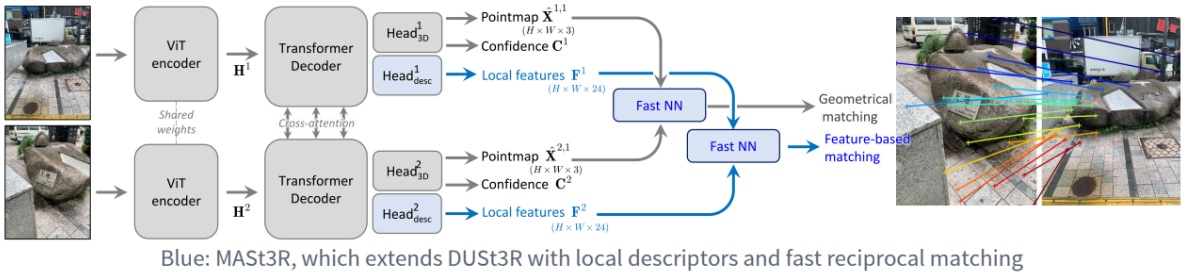


Figure 127: Photos at the left (input images) and right (resulting geometric matching and feature-based fast matching visuals) of the architecture figure

Intuition

The encoder turns each image into a bag of patch tokens independently; the decoders then let the two images "talk" through cross-attention so that each token can relate to the other view. The heads translate tokens into per-pixel 3D coordinates. Crucially, both outputs are in the *same* frame, which is what makes 3D matching possible.

5.5 MASt3R Network Output *(slides 37)*

Per image, the network outputs three dense maps: a pointmap $\hat{\boldsymbol{X}} \in \mathbb{R}^{H \times W \times 3}$ (with $\hat{\boldsymbol{X}}[i] \in \mathbb{R}^3$), a confidence $C \in \mathbb{R}^{H \times W}$ (with $C[i] \in \mathbb{R}$) and a local descriptor map $F \in \mathbb{R}^{H \times W \times 24}$ (with $F[i] \in \mathbb{R}^{24}$).

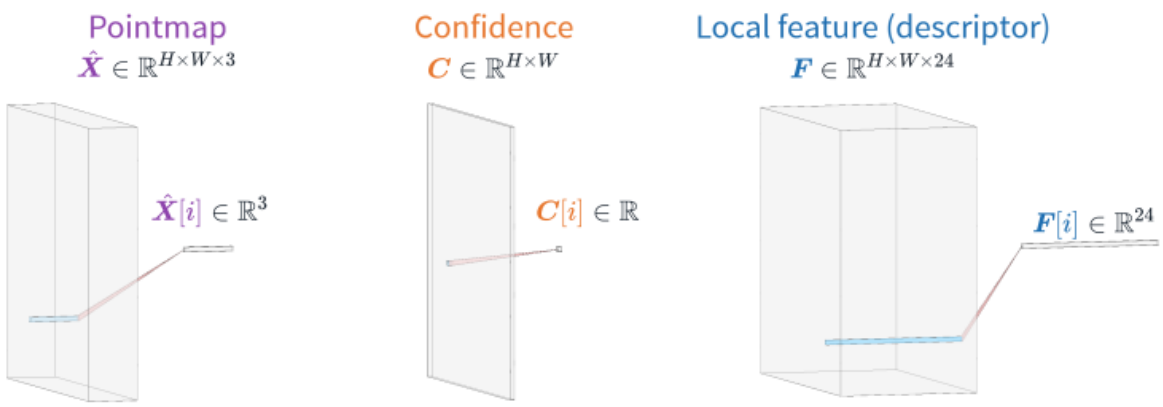


Figure 128: Three 3D-slab sketches: pointmap $\hat{X}$, confidence C and local feature F with a highlighted pixel

5.6 Training Objective and Metric Scale *(slides 38)*

Definition 5.3 (Confidence-weighted regression loss)**.** With $\rho, \hat{\rho}$ the mean distances of the valid ground-truth and predicted points to the origin,

$$\ell_{\text{regr}}(a,i) = \left\| \tfrac{1}{\hat{\rho}} \hat{\boldsymbol{X}}^{a,1}[i] - \tfrac{1}{\rho} \boldsymbol{X}^{a,1}[i] \right\|, \qquad \mathcal{L}_{\text{conf}} = \sum_{a\in\{1,2\}} \sum_{i\in\mathcal{V}^a} C^a[i]\, \ell_{\text{regr}}(a,i) - \alpha \log C^a[i],$$

$\mathcal{V}^a$ the pixels of I_a with valid ground truth, $\alpha = 0.2$. Dividing by $\rho, \hat{\rho}$ removes the unknown scale.

Proposition 5.4 (Optimal confidence)**.** *For a fixed error $\ell > 0$, the function $C \mapsto C\ell - \alpha \log C$ on $C > 0$ is minimised at $C^* = \alpha/\ell$, with minimum value $\alpha(1 + \log(\ell/\alpha))$.*

Proof. The derivative $\ell - \alpha/C$ vanishes at $C = \alpha/\ell$, and the second derivative $\alpha/C^2 > 0$ shows it is a minimum. □

Example 5.5. With $\alpha = 0.2$: a pixel with error $\ell = 0.1$ gets $C^* = 2$; one with $\ell = 1$ gets $C^* = 0.2$. (The paper's parametrisation of C may constrain its range; the proposition is the unconstrained picture.)

Intuition

Without the $-\alpha \log C$ term the network could "switch off" every hard pixel by shrinking its confidence towards zero and the loss would vanish. The log term charges a price for low confidence, so the best compromise is a confidence *inversely proportional to the expected error*: it learns to say "I am unsure here" only where it really is (sky, reflections, textureless areas).

Note (from the original paper, not from the slides): to my knowledge, for datasets with metric ground truth MASt3R does not apply this normalisation, which is how it learns metric scale; the slides only state the normalisation and later say the metric scale "comes from a learned prior".

5.7 The Matching Head *(slides 39)*

Regressed 3D points are robust to viewpoint changes but imprecise. MASt3R therefore adds a second head per image producing dense local descriptors from the same tokens:

$$F^a = \mathrm{Head}^a_{\mathrm{desc}}([\boldsymbol{H}^a, \boldsymbol{H}'^a]) \in \mathbb{R}^{H\times W\times 24}, \quad a \in \{1,2\}.$$

It is a two-layer MLP with a GELU activation; the descriptor $F^a[i] \in \mathbb{R}^{24}$ of each pixel is normalised to unit length. It is trained jointly with the 3D head, so the descriptors stay *grounded* in the pointmaps.

5.8 InfoNCE and the Hinge Loss *(slides 40)*

Definition 5.6 (Matching loss)**.** With temperature $\tau = 0.07$ and $s_\tau(i,j) = \exp(F^1[i]^\top F^2[j]/\tau)$,

$$\mathcal{L}_{\mathrm{match}} = -\sum_{(i,j)\in\mathcal{M}} \left(\log \frac{s_\tau(i,j)}{\sum_{i'\in\mathcal{P}^1} s_\tau(i',j)} + \log \frac{s_\tau(i,j)}{\sum_{j'\in\mathcal{P}^2} s_\tau(i,j')} \right).$$

$\mathcal{M}$: ground-truth correspondences (pixel pairs with the same ground-truth 3D point); $\mathcal{P}^1, \mathcal{P}^2$: their pixels in each image. The total loss is $\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{conf}} + \beta\mathcal{L}_{\mathrm{match}}$ with $\beta = 1$.

Proposition 5.7 (InfoNCE contains a smooth hinge)**.** *If only one positive and one negative candidate are present, each term equals* $\log(1 + e^{-(u_+ - u_-)/\tau})$ *with* $u_\pm$ *the descriptor dot products. This is a smooth version of the hinge loss of Sec. 4.3 (compare* $\tau\log(1+e^{-\Delta/\tau}) \to \max(0,-\Delta)$ *as* $\tau \to 0$*).*

Proof. $-\log \dfrac{e^{u_+/\tau}}{e^{u_+/\tau} + e^{u_-/\tau}} = \log(1 + e^{-(u_+ - u_-)/\tau})$. □

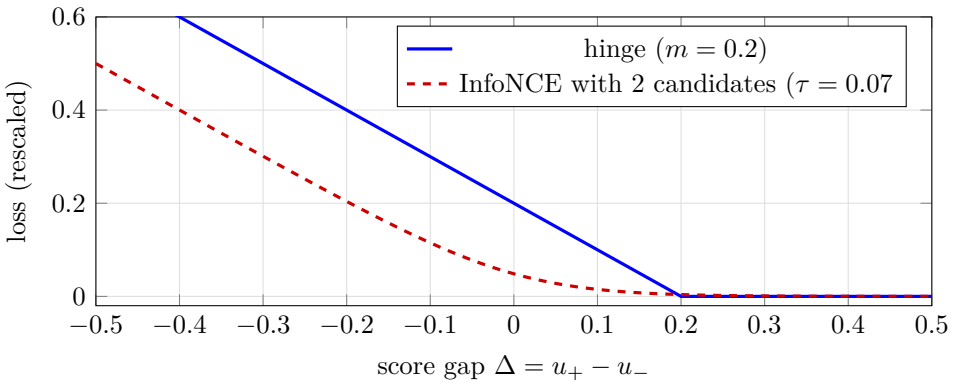


Figure 129: Hinge versus a two-candidate InfoNCE term (new figure; the second is multiplied by τ so the slopes are comparable). InfoNCE never becomes exactly zero: it keeps pushing already-separated pairs apart.

Intuition

The hinge loss compares the match with *one* negative and a fixed margin. InfoNCE is a cross-entropy over *all* candidates: every other matched pixel acts as a negative and only the exact pixel is rewarded. That gives a much richer learning signal per correspondence, and

the low temperature makes the descriptor space sharply peaked.

5.9 Effect of 3D Grounding *(slides 41)*

Ablation on the Map-free relocalization benchmark (validation set; estimate the metric pose of a query from a single reference image). All rows take depth from the same monocular depth network.

	model, matching on	VCRE AUC ↑	median error ↓	pose AUC ↑
I	DUSt3R, 3D points	0.704	1.10 m, 9.4°	0.344
II	MASt3R, 3D points	0.732	0.94 m, 3.6°	0.409
III	MASt3R, matching loss only, descriptors	0.744	1.10 m, 10.8°	0.382
IV	MASt3R, descriptors	**0.752**	**0.93 m, 3.0°**	**0.435**

VCRE AUC: area under the curve of the reprojection error of virtual 3D points under the estimated pose, up to 90 px. Pose AUC: area under the curve of pose accuracy.

- Descriptors beat matching on 3D points (II vs. IV: 0.732 → 0.752).
- Without the 3D regression loss, the median rotation error grows from 3.0° to 10.8° (III vs. IV).

Intuition

Descriptors give *precise* matches; the 3D loss gives *geometric sanity*. Row III shows that a descriptor trained alone has decent reprojection scores but poor rotation – the matches are locally sharp yet not globally consistent. Training both together gives the best of each.

5.10 Reciprocal Matching *(slides 42)*

Definition 5.8 (Reciprocal (mutual nearest-neighbour) matches)**.**

$$\hat{\mathcal{M}} = \{(i,j) \mid j = \mathrm{NN}_2(F^1[i]) \text{ and } i = \mathrm{NN}_1(F^2[j])\}, \qquad \mathrm{NN}_2(F^1[i]) = \arg\min_j \|F^2[j] - F^1[i]\|,$$

and NN_1 is the same from image 2 to image 1.

Lemma 5.9. *$\hat{\mathcal{M}}$ is a partial one-to-one matching: no pixel of either image appears in more than one pair.*

Proof. If $(i,j),(i,j') \in \hat{\mathcal{M}}$ then $j = \mathrm{NN}_2(F^1[i]) = j'$ (nearest neighbour is a function, ignoring ties); symmetrically for a pixel j appearing with two i's. □

Example 5.10 (1D toy descriptors)**.** $F^1 = (0,\ 1,\ 5)$ and $F^2 = (0.9,\ 5.2,\ 6)$. Then NN_2 sends $0 \to 0.9$, $1 \to 0.9$, $5 \to 5.2$, while NN_1 sends $0.9 \to 1$, $5.2 \to 5$, $6 \to 5$. Mutual pairs: $(1, 0.9)$ and $(5, 5.2)$. The pair $(0, 0.9)$ is rejected because 0.9 prefers 1.

- It is the left-right consistency test of Sec. 4.2, applied over the whole image instead of one row.

- Without rectification there is no epipolar line: every pixel of image 2 is a candidate, so a naive implementation compares all pixel pairs, $O(W^2H^2)$. For a 512×384 image this is $\approx 3.9 \times 10^{10}$ pairs (illustrative size).

Intuition

It is the same "two witnesses must agree" idea as the left-right test, but without a row to search along. Mutual agreement is a cheap, strong outlier filter; the price is a huge candidate set, which is why MASt3R uses a *fast* matching scheme (the slides name it but do not detail it).

5.11 MASt3R Training Details *(slides 43)*

- **Data:** 14 datasets (indoor, outdoor, synthetic, real, object-centric), 10 with metric ground truth (Habitat, ARKitScenes, BlendedMVS, MegaDepth, Static Scenes 3D, ScanNet++, CO3D-v2, Waymo, Map-free, WildRGB-D, Virtual KITTI 2, Unreal4K, TartanAir and an internal dataset).
- **Amount:** 650k pairs per epoch, equally from all datasets, 35 epochs: $650\text{k} \times 35 = 22.75$ million pairs, batch size 64.
- **Images:** 512 px on the long side, five aspect ratios, random crops and colour jitter.
- **Model:** DUSt3R network (ViT-L encoder, ViT-B decoder), initialised from the public DUSt3R checkpoint.
- **Optimisation:** AdamW, learning rate 10^{-4} with 7 warm-up epochs and cosine decay, weight decay 0.05; 4096 ground-truth correspondences per pair for the matching loss.
- Hardware and training time: not reported in the paper.

5.12 Results *(slides 44)*

Map-free relocalization, test set

method	VCRE AUC ↑
LoFTR, monocular depth	0.634
DUSt3R	0.697
MASt3R	**0.933**

DTU multi-view stereo, zero-shot (mm)

method	Acc. ↓	Comp. ↓	Overall ↓
DUSt3R	2.677	0.805	1.741
MASt3R	**0.403**	**0.344**	**0.374**

DTU: Acc. is the mean distance from the reconstruction to the ground truth (structured light), Comp. the reverse, Overall their mean.

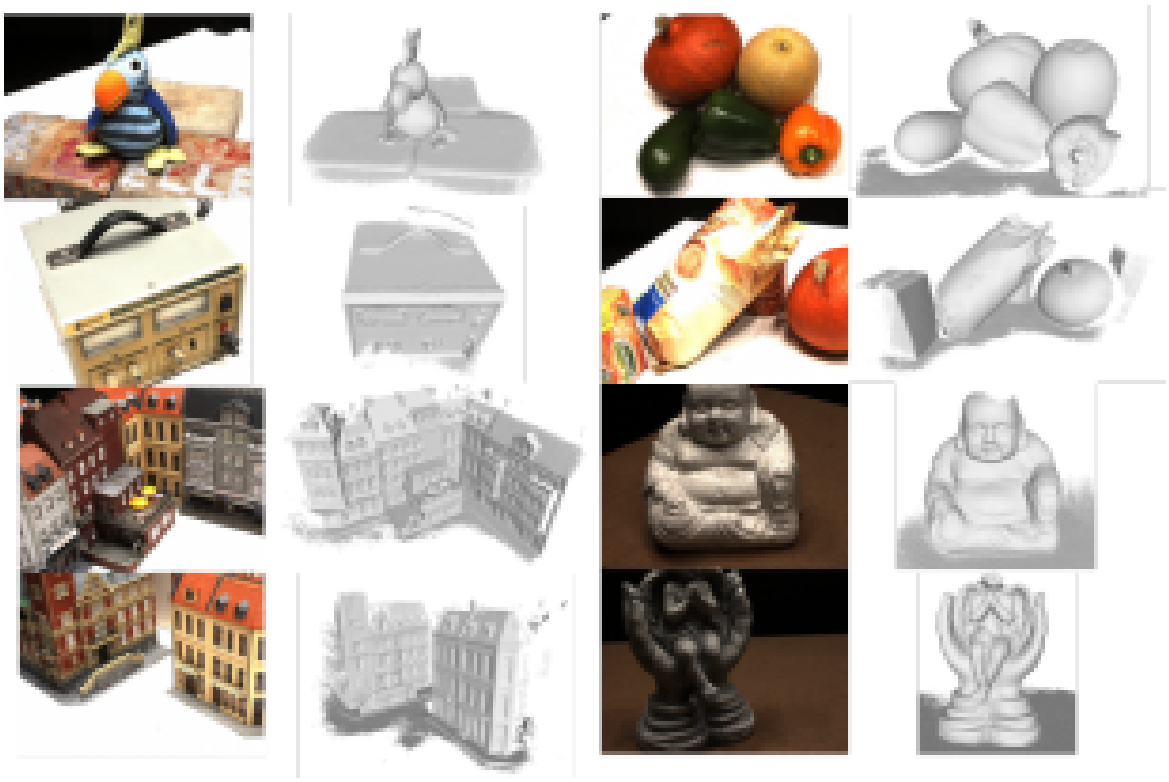


Figure 130: Grid of object photos/reconstructions from DTU

5.13 The Pipeline Revisited *(slides 45)*

Step	Calibrated stereo	MASt3R
1. Calibration	measured offline, required	not needed to match; focal estimated; known intrinsics for the best poses
2. Rectification	required	not used
3. Disparity	1D search along a row	reciprocal nearest neighbours over the whole image
4. Outliers	left-right consistency test	reciprocity, the same test in 2D
5. Depth	$z = fb/d$, metric from the measured baseline	pointmap, metric scale from a learned prior
6. 3D model	fusion of depth maps	pointmaps, or triangulation of the matches

Given up: a measured metric scale, full resolution (512-pixel windows), low compute, and more than two images per inference. A calibrated rig still wins for real-time and embedded systems and whenever metric scale must be *measured*.

Intuition

Each step of the classical recipe has a counterpart, but the machinery moves from "geometry we measure" to "geometry the network has learned". You gain freedom (arbitrary cameras, wide baselines, forward motion) and lose guarantees (scale, speed, resolution).

Summary

Comparison of the methods covered

Method	Matching signal	Needs calibration + rectification?	Search / output	Main strength / weakness
Block matching (SSD/ZNCC)	hand-crafted patch cost, WTA	yes	1D along row; disparity	simple; noisy, fronto-parallel assumption, border bleeding
Siamese CNN (Zbontar–LeCun)	learned cost (MLP) or cosine of learned features	yes	1D along row; cost volume + WTA/SGM	far less noise; slow if MLP head; ambiguity remains
DispNet	correlation layer, end-to-end regression	yes (fixed disparity range)	disparity at several scales	fast, uses synthetic pre-training; rectified input only
GC-Net	3D conv on cost volume, soft argmin	yes (fixed disparity range)	sub-pixel disparity	better accuracy; large memory (3D volume), smoothness bias
DUSt3R / MASt3R	pointmaps + 24-d descriptors, reciprocal NN	no (focal estimated)	whole image; pointmap, matches, pose	arbitrary views and wide baselines, scale from a prior; no measured scale, lower resolution, compute heavy

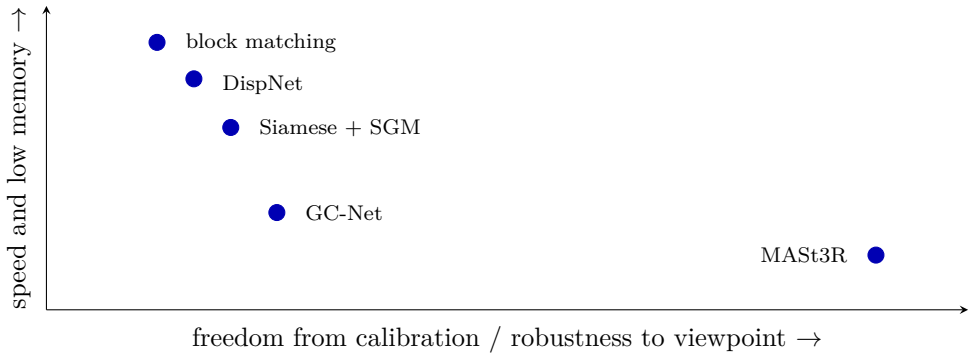


Figure 131: Qualitative placement of the methods on two axes, drawn from the discussion on the slides (calibration/rectification dependence, memory and compute); positions are *not* measured values.

Key take-aways

- **Geometry buys you a 1D search.** With calibration and rectification, matching is along a scanline (a 480× smaller search for VGA) and depth follows from $z = fb/d$.
- **Stereo error is quadratic in depth:** $\delta z \approx z^2 \delta d/(fb)$. A wider baseline helps the arithmetic but makes matching harder.
- **Block matching fails where local similarity fails** (occlusions, slanted surfaces, repetitions, textureless areas); window size trades noise against border bleeding, and the left-right consistency test removes many outliers.
- **Learning replaced the hand-made parts in two steps:** learned matching costs (Siamese nets, hinge loss on triplets), then end-to-end networks (DispNet, GC-Net with cost volumes and soft argmin) – still tied to rectified input and a fixed disparity range.
- **Matching in 3D drops calibration and rectification from the matching step** (DUSt3R/MASt3R: pointmaps, descriptors, reciprocal matching) at the price of measured

metric scale, resolution, compute and the number of views; calibrated rigs still win for real-time and embedded, metrically accurate use.

Part IV

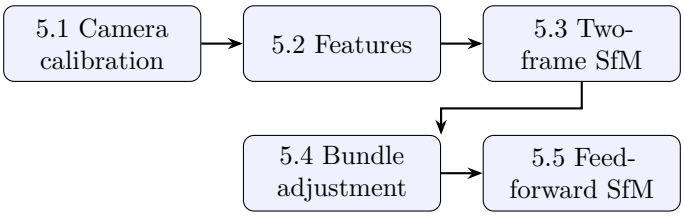
Lecture 5

Abstract

These notes follow the slide deck section by section (slide ranges are given in every heading). Formal material – definitions, lemmas, theorems and proofs – is merged in at the point where the slides introduce the topic; it is taken from the standard references cited on the slides (Zhang 2000, Lowe 1999/2004, Hartley–Zisserman, Triggs et al. 2000, Schönberger–Frahm 2016, the SuperPoint and VGGT papers) and re-derived here. Every formal result is followed by a blue *Intuition* box. Boxes titled *Beyond the slides* mark facts that are not on the slides. Dashed boxes mark photos that should be copied in from the slide deck.

Agenda and Learning Objectives *(slides 1–2)*

The lecture takes us from a single calibrated camera to a full reconstruction pipeline and finally to networks that perform the whole reconstruction in one forward pass.



Learning objectives (slide 2).

1. Derive the epipolar constraint, estimate the essential matrix from correspondences, recover the relative pose up to scale, and triangulate a point from two views.
2. Formulate bundle adjustment as least squares over poses in SE(3) and 3D points, solve it with Gauss–Newton and the Schur complement, and describe the stages of incremental SfM.
3. Explain how a feed-forward transformer predicts cameras, depth, point maps and tracks, and how bundle adjustment refines them.

Notation used throughout.

Symbol	Meaning
$\bar{\boldsymbol{x}} = (x, y, 1)^\mathsf{T}$	homogeneous pixel coordinates
$\tilde{\mathbf{x}} = \boldsymbol{K}^{-1}\bar{\boldsymbol{x}}$	local ray direction of a pixel (calibrated camera)
$\boldsymbol{R}, \boldsymbol{t}$, $\boldsymbol{T} = [\boldsymbol{R}\|\boldsymbol{t}] \in \mathrm{SE}(3)$	rotation, translation, pose (world $\to$ camera)
$[\boldsymbol{t}]_\times$	skew-symmetric matrix with $[\boldsymbol{t}]_\times \boldsymbol{v} = \boldsymbol{t} \times \boldsymbol{v}$
$\mathbb{E} = [\boldsymbol{t}]_\times \boldsymbol{R}$, $\tilde{\boldsymbol{F}} = \boldsymbol{K}_2^{-\mathsf{T}} \mathbb{E} \boldsymbol{K}_1^{-1}$	essential / fundamental matrix
$\boldsymbol{x}_p^w$, $\boldsymbol{x}_{ip}^s$	3D world point p, its observation in image i
$\propto$	equality up to a nonzero scale factor

1 Camera Calibration *(slides 3–7)*

1.1 What calibration is and how it is done *(slides 4–7)*

Camera calibration is the process of finding the intrinsic and extrinsic parameters of a camera. Most commonly a *known calibration target* (an image or a checkerboard) is used.

Definition 1.1 (Pinhole camera with intrinsics and extrinsics)**.** A world point $\boldsymbol{x}^w \in \mathbb{R}^3$ is mapped to a pixel by

$$\lambda\, \bar{\boldsymbol{x}} = \boldsymbol{K}\,[\boldsymbol{R}\,|\,\boldsymbol{t}] \begin{pmatrix} \boldsymbol{x}^w \\ 1 \end{pmatrix}, \qquad \boldsymbol{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

where $\boldsymbol{K}$ holds the *intrinsics* (focal lengths f_x, f_y in pixels, skew s, principal point (c_x, c_y)) and $[\boldsymbol{R}|\boldsymbol{t}] \in \mathrm{SE}(3)$ are the *extrinsics*, i.e. the camera pose. Lens *distortion* (e.g. radial,

$\boldsymbol{x}_d = \boldsymbol{x}\,(1 + k_1 r^2 + k_2 r^4)$ with $r = \|\boldsymbol{x}\|$ in normalized coordinates) is modelled by extra parameters applied before $\boldsymbol{K}$.

Intuition

$\boldsymbol{K}$ says how the metric ray through the pinhole is converted to pixels; $[\boldsymbol{R}|\boldsymbol{t}]$ says where the camera sits in the world. Calibration is an inverse problem: we look at something whose geometry we know perfectly (the target) and ask which camera would have produced those pixels.

The procedure on slides 4–7 has four stages (Fig. 132):

1. the known target is **captured in different poses** (slide 5);
2. **features** (e.g. corners) on the target are detected in the images (slide 6);
3. a **closed-form solution** initializes all parameters except the distortion parameters;
4. a **non-linear optimization** of all parameters minimizes the reprojection errors (slide 7), jointly over intrinsics and extrinsics (=poses).

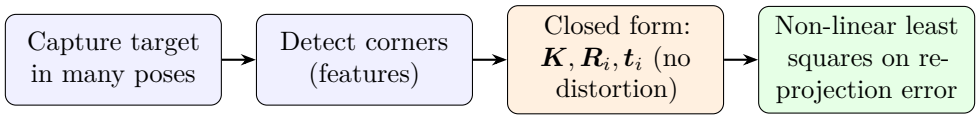


Figure 132: The calibration pipeline of Zhang (2000), redrawn from the description on slides 4–7.



Figure 133: A known calibration target held in front of several cameras (multi-camera calibration setup).

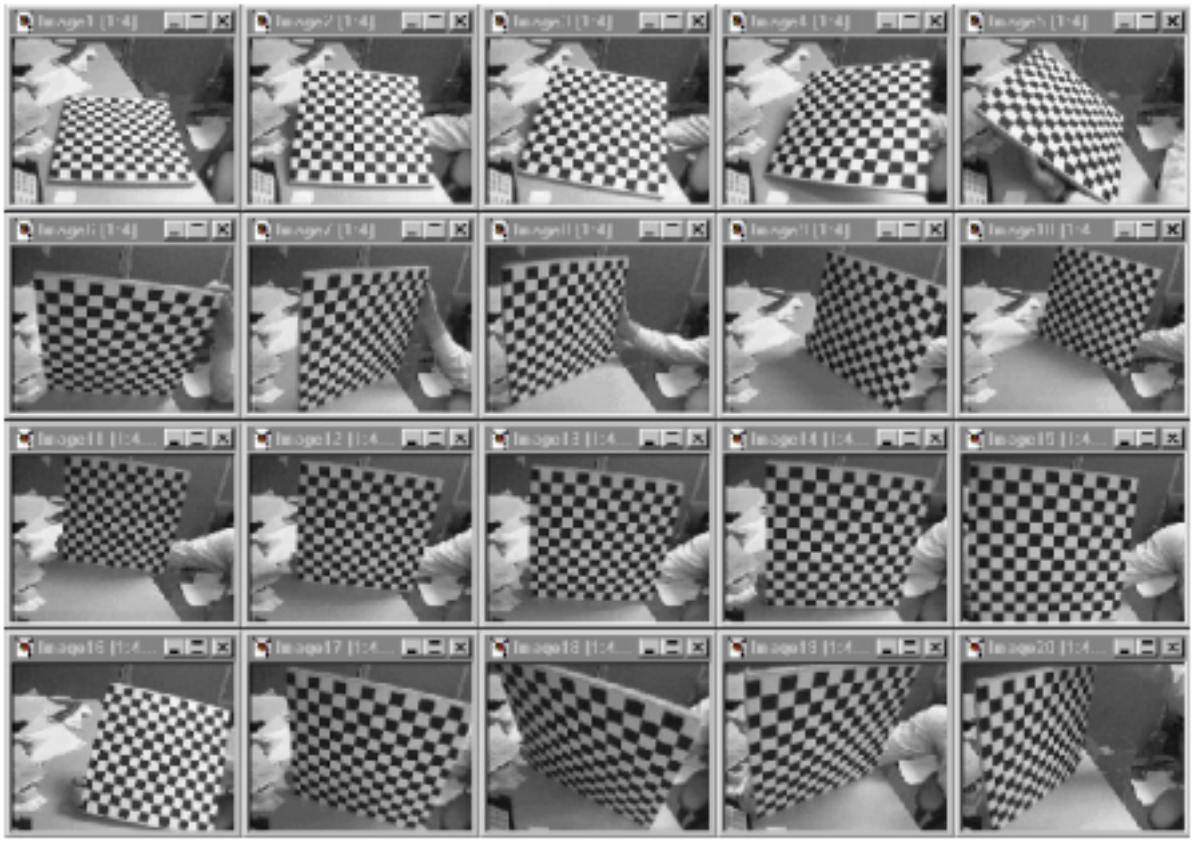


Figure 134: The calibration checkerboard captured in 20 different poses.

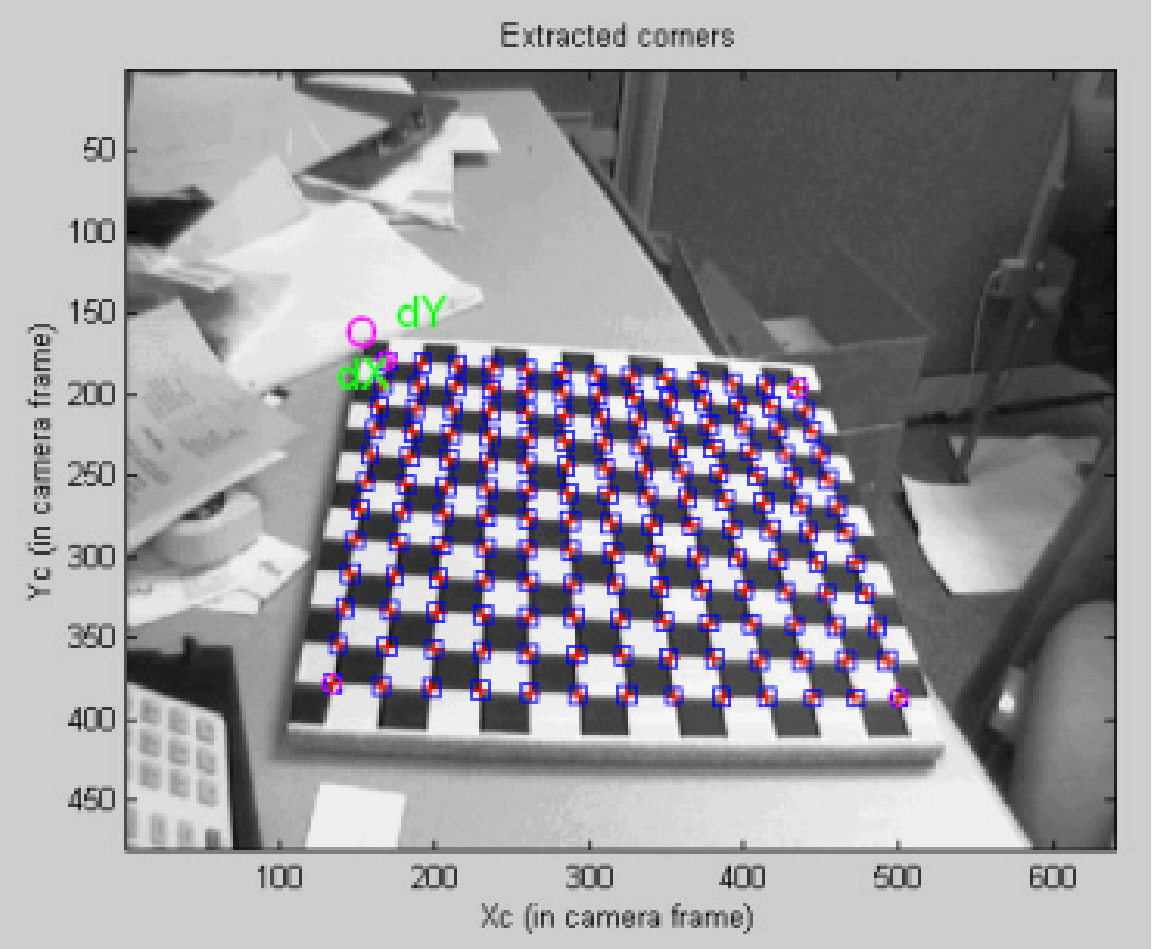


Figure 135: Corners detected on the target (red crosses, blue squares: extracted corners).

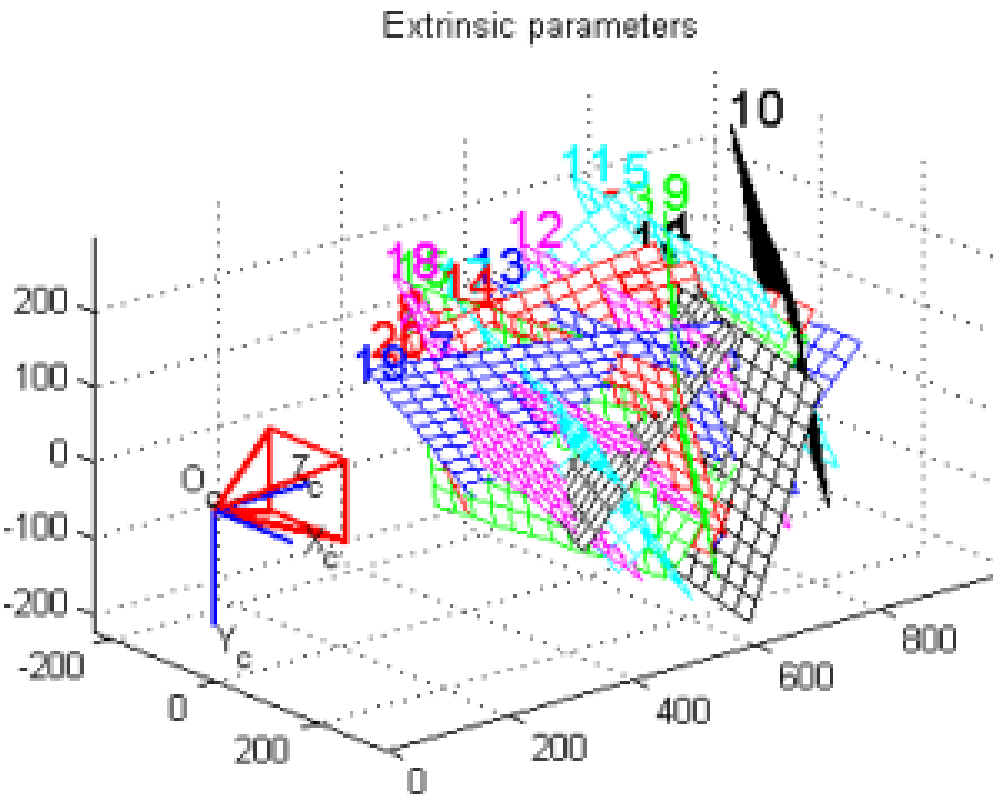


Figure 136: Recovered extrinsic parameters: the target planes in the camera frame after joint optimization (Zhang, TPAMI 2000).

Theorem 1.2 (Zhang's constraints from a plane)**.** *Let the target lie in the plane $Z = 0$ of the world, so a target point is $(X, Y, 0)$. The homography $\boldsymbol{H} = [\boldsymbol{h}_1\ \boldsymbol{h}_2\ \boldsymbol{h}_3]$ mapping target coordinates $(X, Y, 1)^\mathsf{T}$ to pixels satisfies $\boldsymbol{H} = \lambda \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}]$. With $\boldsymbol{B} = \boldsymbol{K}^{-\mathsf{T}}\boldsymbol{K}^{-1}$ (symmetric, positive definite) every view yields the two linear equations in $\boldsymbol{B}$*

$$\boldsymbol{h}_1^\mathsf{T}\boldsymbol{B}\,\boldsymbol{h}_2 = 0, \qquad \boldsymbol{h}_1^\mathsf{T}\boldsymbol{B}\,\boldsymbol{h}_1 = \boldsymbol{h}_2^\mathsf{T}\boldsymbol{B}\,\boldsymbol{h}_2.$$

Proof. For $Z = 0$ the column $\boldsymbol{r}_3$ is multiplied by zero, so $\lambda\bar{\boldsymbol{x}} = \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}](X, Y, 1)^\mathsf{T}$, which gives $\boldsymbol{H}$. From $\boldsymbol{h}_i = \lambda \boldsymbol{K}\boldsymbol{r}_i$ we get $\boldsymbol{r}_i = \lambda^{-1}\boldsymbol{K}^{-1}\boldsymbol{h}_i$ for $i = 1, 2$. Rotation columns are orthogonal and unit norm: $\boldsymbol{r}_1^\mathsf{T}\boldsymbol{r}_2 = 0$ gives $\boldsymbol{h}_1^\mathsf{T}\boldsymbol{K}^{-\mathsf{T}}\boldsymbol{K}^{-1}\boldsymbol{h}_2 = 0$ and $\boldsymbol{r}_1^\mathsf{T}\boldsymbol{r}_1 = \boldsymbol{r}_2^\mathsf{T}\boldsymbol{r}_2$ gives the second equation. □

Intuition

A rotation matrix has two "rules": its columns are perpendicular and equally long. A single view of a flat board gives a homography, and those two rules, pushed through the unknown $\boldsymbol{K}$, become two equations on $\boldsymbol{B}$ (which encodes $\boldsymbol{K}$). The board in one pose cannot tell you everything; a tilted board in a different pose tells you something new. That is why the target is captured in different poses.

Counting. $\boldsymbol{B}$ is symmetric, so it has 6 entries; it is defined up to scale, so 5 unknowns (matching the 5 intrinsics of $\boldsymbol{K}$). Each view gives 2 equations, hence ≥ 3 views for a general $\boldsymbol{K}$ (or 2 views if the skew $s = 0$ is assumed).

Algorithm 1.1 (Zhang's calibration)**.**
1. For each view j estimate the homography $\boldsymbol{H}_j$ from the detected corners (DLT, as for homography estimation in lecture 01).
2. Stack the two equations of Thm. 1.2 for all views as $\boldsymbol{V}\boldsymbol{b} = 0$ and take the right singular vector of the smallest singular value; recover $\boldsymbol{K}$ from $\boldsymbol{b}$.
3. For every view: $\lambda = 1/\|\boldsymbol{K}^{-1}\boldsymbol{h}_1\|$, $\boldsymbol{r}_1 = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_1$, $\boldsymbol{r}_2 = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_2$, $\boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2$, $\boldsymbol{t} = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_3$. (Project $[\boldsymbol{r}_1\boldsymbol{r}_2\boldsymbol{r}_3]$ to the closest rotation with an SVD.) Distortion is initialized to zero.
4. Refine all parameters jointly by minimizing the reprojection error $\sum_{j,m}\|\bar{\boldsymbol{x}}_{jm} - \hat{\pi}(\boldsymbol{K}, \boldsymbol{k}, \boldsymbol{R}_j, \boldsymbol{t}_j, \boldsymbol{X}_m)\|^2$ with a non-linear least-squares solver (Levenberg–Marquardt).

Intuition

Steps 1–3 are cheap and linear but ignore noise statistics and distortion; they only have to be *good enough to start*. Step 4 is the honest estimator. This "closed form to initialize, non-linear least squares to polish" pattern returns again in bundle adjustment (Section 5.4).

Example 1.3 (Size of a calibration problem (illustrative numbers))**.** A 9×6 checkerboard (54 inner corners) seen in 20 views gives $2 \cdot 54 \cdot 20 = 2160$ scalar residuals. The unknowns are 5 intrinsics, 2 distortion coefficients and $6 \cdot 20 = 120$ pose parameters: 127 in total. The problem is hugely over-determined, which is what makes it accurate.

2 Feature Detection and Description *(slides 8–20)*

2.1 Point Features *(slides 9–11)*

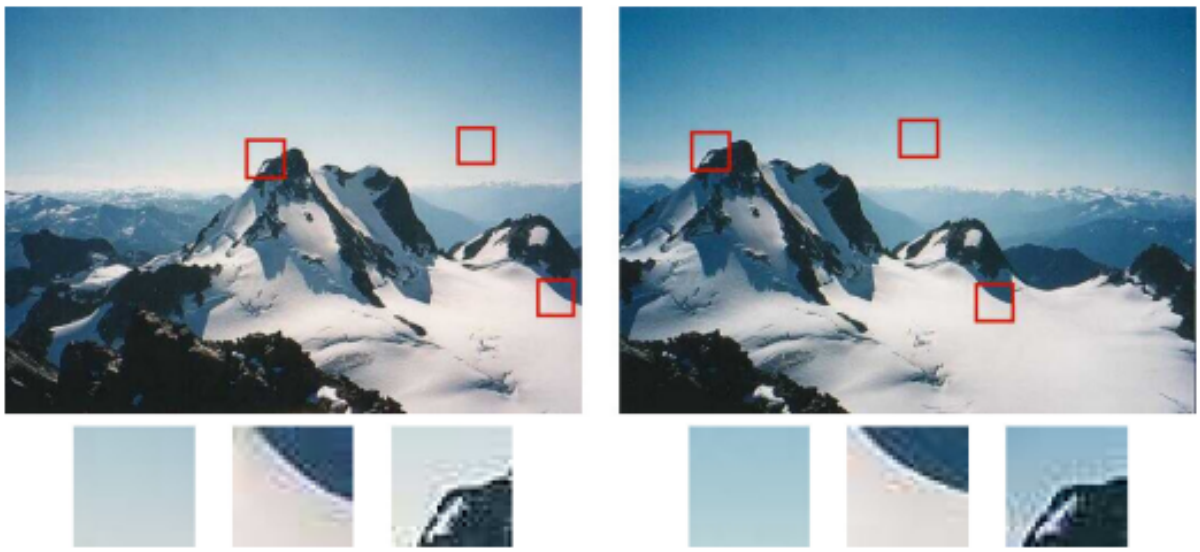


Figure 137: Two views of the same mountain scene; red boxes mark patches that are flat sky (uninformative), an edge, and a distinctive corner-like region (image credit: R. Szeliski).

Definition 2.1 (Point feature)**.** A point feature is a pair $(\boldsymbol{k}, \boldsymbol{f})$: a *keypoint* $\boldsymbol{k} = (x, y)^\mathsf{T} \in \mathbb{R}^2$, the pixel location of a salient region, and a *descriptor* $\boldsymbol{f} \in \mathbb{R}^D$ that describes the appearance of the image around $\boldsymbol{k}$. An image yields a set of M features $\{(\boldsymbol{k}_j, \boldsymbol{f}_j)\}_{j=1}^{M}$. Two features $(\boldsymbol{k}, \boldsymbol{f})$

and $(\boldsymbol{k}', \boldsymbol{f}')$ in two images *match* when their descriptors are close, i.e. $\|\boldsymbol{f} - \boldsymbol{f}'\|$ is small.

Intuition

Think of a descriptor as a "fingerprint" of a patch. A flat piece of sky has a fingerprint that matches thousands of other patches (useless), whereas the tip of a mountain has a unique one. Point features describe local salient regions, they can be used to describe and match images from different viewpoints, and they form the basis of the sparse 3D reconstruction methods of this lecture.

The requirements (slide 11): features should be **invariant** to perspective effects and illumination, and the same 3D point should have **similar vectors independent of pose/viewpoint**. Plain RGB/intensity patches do not have this property, so we need something better.



Figure 138: Object recognition with local scale-invariant features: SIFT patches on three known products are found in a cluttered scene (Lowe, ICCV 1999).

2.2 Scale Invariant Feature Transform (SIFT) *(slides 12–15)*

Detection (slide 12). SIFT builds a *scale space* by iteratively filtering the image with a Gaussian. Adjacent scales are subtracted, yielding *Difference of Gaussian* (DoG) images; interest points (=blobs) are detected as extrema in the resulting scale space (Fig. 139).

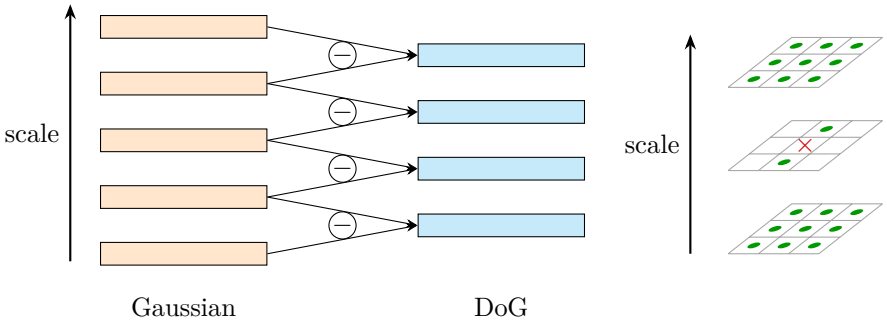


Figure 139: Left: first octave of the Gaussian scale space and the DoG images obtained by subtracting neighbouring scales. Right: a candidate (red cross) is kept only if it is larger or smaller than all its 26 neighbours in space and scale (green).

Proposition 2.2 (DoG approximates the scale-normalized Laplacian)**.** *Let* $G(\boldsymbol{x}, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\|\boldsymbol{x}\|^2/2\sigma^2}$*. Then* $\partial G/\partial\sigma = \sigma\nabla^2 G$ *and, for a scale ratio* k *close to 1,*

$$G(\boldsymbol{x}, k\sigma) - G(\boldsymbol{x}, \sigma) \;\approx\; (k-1)\,\sigma^2\nabla^2 G(\boldsymbol{x}, \sigma).$$

Proof. G solves the heat equation $\partial G/\partial(\sigma^2) = \frac{1}{2}\nabla^2 G$, i.e. $\partial G/\partial\sigma = \sigma\nabla^2 G$. A finite difference in σ gives $G(k\sigma) - G(\sigma) \approx (k\sigma - \sigma)\,\partial G/\partial\sigma = (k-1)\sigma^2\nabla^2 G$. □

Intuition

Subtracting two slightly different blurs is a cheap way to compute a blob detector (the Laplacian). Multiplying by σ^2 makes its response comparable across scales: a big blob at a coarse scale gives the same peak value as a small blob at a fine scale, so extrema over (position, scale) find blobs together with their size. That is where scale invariance comes from.

Description (slide 13). SIFT **rotates** the descriptor to **align with the dominant gradient orientation**; **gradient histograms** are computed for local sub-regions; all histograms are concatenated and normalized to form a **128D feature vector**.

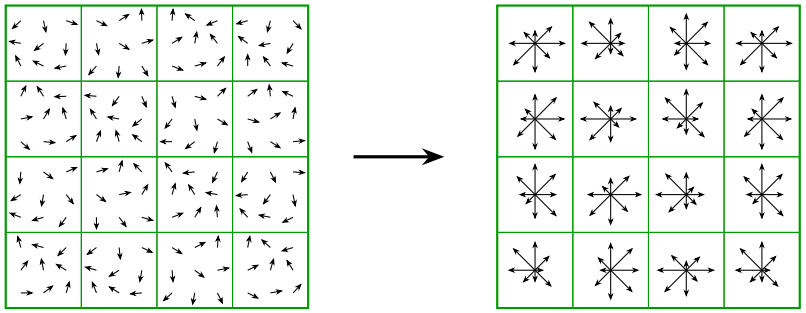


image gradients (4×4 cells) descriptor: $4 \times 4 \times 8 = 128$ numbers

Figure 140: Schematic of the SIFT descriptor: the gradients around the keypoint (after rotating to the dominant orientation) are pooled into an orientation histogram with 8 bins per cell; 4×4 cells give a 128D vector.

Intuition

Histograms of gradient *directions* throw away exact pixel positions inside each cell (robust to small shifts) but keep the coarse layout (distinctive). Rotating to the dominant orientation removes the camera's in-plane rotation; normalizing the vector removes affine illumination changes (gain).

Beyond the slides

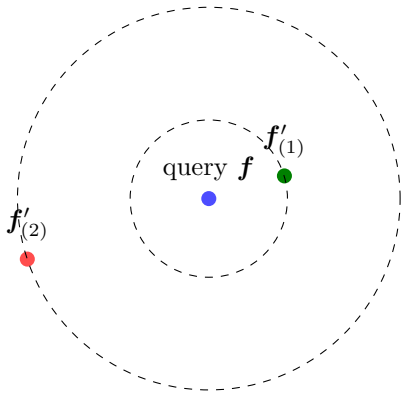
In Lowe's implementation, the vector is normalized, entries are clipped at 0.2, and it is renormalized, which reduces the influence of non-linear illumination effects. The slide shows a 2×2 grid only for drawing convenience.

2.3 Matching features *(slides 14–15)*

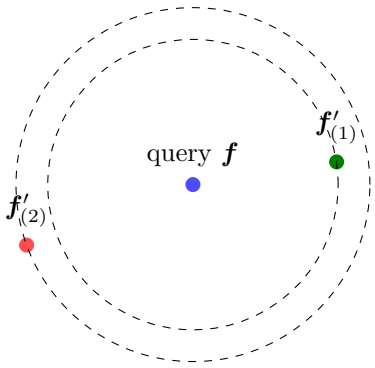
Many algorithms exist (SIFT, SURF, U-SURF, BRISK, ORB, FAST and learned ones such as SuperPoint). SIFT was a seminal work thanks to its invariance and robustness and enabled large-scale SfM; despite being 20 years old it is still used, e.g. in COLMAP. Correspondences are

retrieved by efficient nearest-neighbour search. Ambiguous matches are filtered with the **ratio test**: compute the distance to the closest neighbour divided by the distance to the second closest; a large ratio (> 0.8) indicates that the match might not be correct.

Definition 2.3 (Lowe's ratio test)**.** For a query descriptor $\boldsymbol{f}$ with nearest neighbours $\boldsymbol{f}'_{(1)}, \boldsymbol{f}'_{(2)}$ in the other image, accept the match $\boldsymbol{f} \leftrightarrow \boldsymbol{f}'_{(1)}$ iff $\dfrac{\|\boldsymbol{f} - \boldsymbol{f}'_{(1)}\|}{\|\boldsymbol{f} - \boldsymbol{f}'_{(2)}\|} < \tau$, with typically $\tau = 0.8$.



$d_1 = 1.04,\ d_2 = 2.53$, ratio 0.41: **accept**



$d_1 = 1.92,\ d_2 = 2.34$, ratio 0.82: **reject**

Figure 141: Toy example in a 2D descriptor space. If the best match is not clearly better than the runner-up, the match is ambiguous.

Intuition

If I ask "who is my best friend?" and the answer is "person A, and person B is almost as close", I should not trust the answer. The ratio test measures that gap. With ratio 0.41 the closest descriptor is more than twice as close as the next; with ratio 0.82 it is basically a coin flip between two candidates (e.g. a repeated window on a facade).

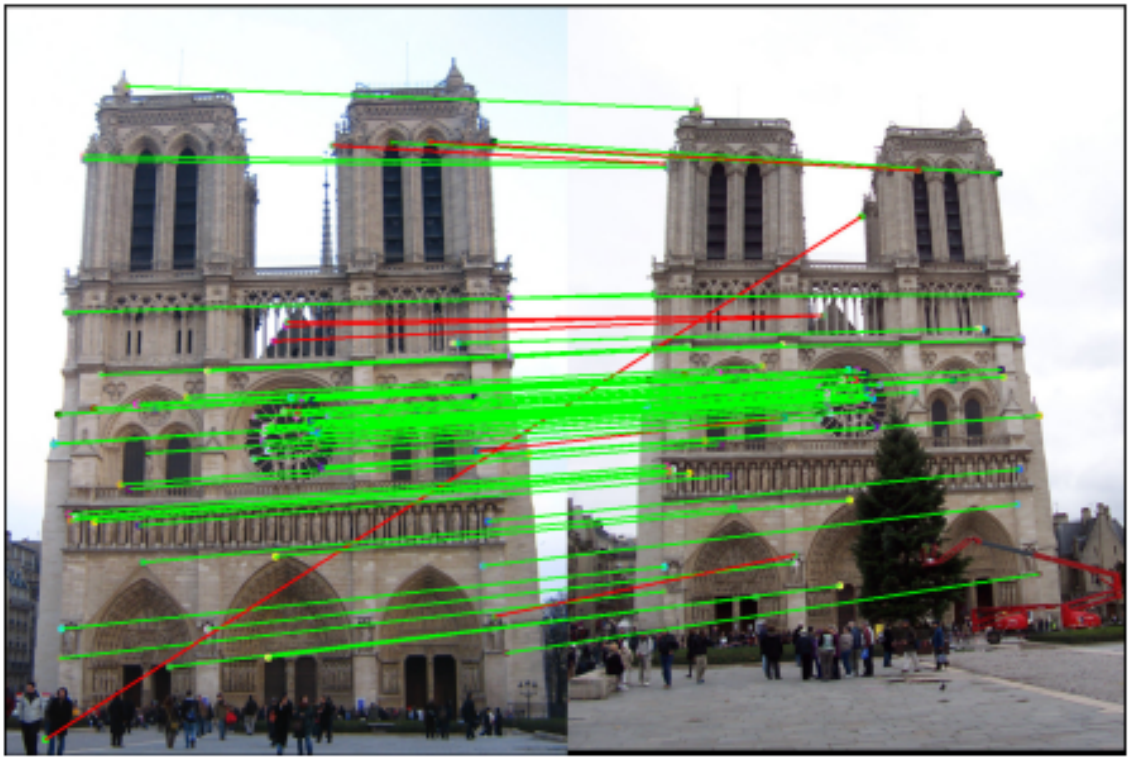


Figure 142: Two images of Notre-Dame with SIFT correspondences (green: correct, red: wrong) (image credits: J. Hays).

2.4 SuperPoint: Learned Interest Points and Descriptors *(slides 16–20)*

Idea (slide 16). A fully convolutional network computes interest points and their descriptors in one forward pass over the full image. Training is **self-supervised** in three stages (Fig. 143): (a) a base detector, MagicPoint, is trained on rendered shapes whose corners are known; (b) it labels real, unlabeled images by Homographic Adaptation; (c) detector and descriptor are trained jointly on image pairs related by a known homography. Runtime: about 13 ms for a 480×640 image on one Titan X GPU (about 70 fps).

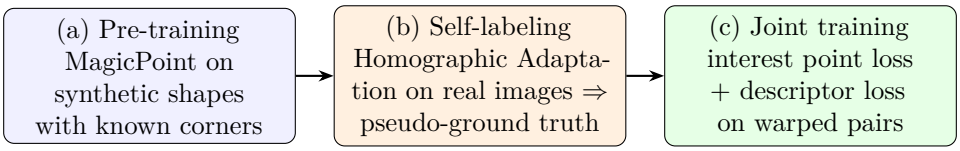


Figure 143: The three SuperPoint training stages (slide 16).

Base detector and training details (slide 17). MagicPoint is trained for 200,000 iterations on *Synthetic Shapes* (quads, triangles, cubes, stars, lines, checkerboards, quad grids), rendered on the fly so that no image is seen twice. Pseudo-ground truth: 80,000 MS-COCO images, grayscale, 240×320, Homographic Adaptation with $N_h = 100$, run twice. Joint training: one random homography per image (milder than in adaptation), cross-entropy for interest points, a hinge loss for descriptors. Optimization: batch size 32, Adam with learning rate 10^{-3}, augmentation (Gaussian noise, motion blur, brightness), descriptor length $D = 256$.

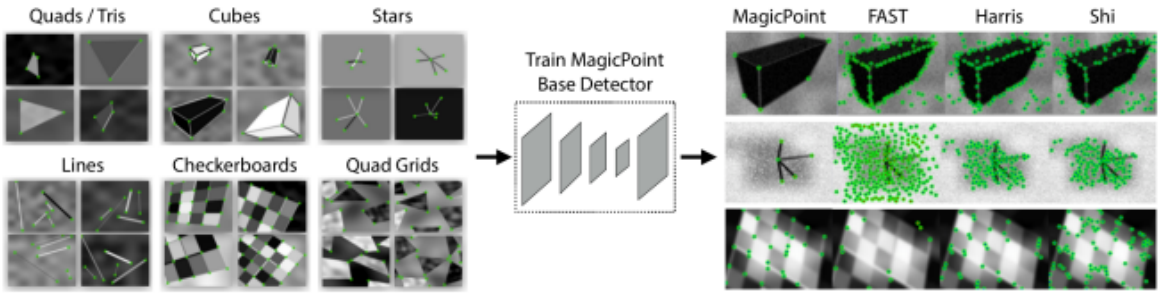


Figure 144: Left: Synthetic Shapes with known corners. Right: MagicPoint against FAST, Harris and Shi–Tomasi corners on noisy images.

Homographic Adaptation (slide 18).

Definition 2.4 (Covariant detector and Homographic Adaptation)**.** Let f_θ map an image to a heatmap of interest points and let $\tilde{\boldsymbol{H}}(\cdot)$ denote warping an image or heatmap by the homography $\tilde{\boldsymbol{H}}$. f_θ is *covariant* if $f_\theta(\tilde{\boldsymbol{H}}(\boldsymbol{I})) = \tilde{\boldsymbol{H}}(f_\theta(\boldsymbol{I}))$. Homographic Adaptation computes the label heatmap

$$\hat{F}(\boldsymbol{I}) = \frac{1}{N_h} \sum_{i=1}^{N_h} \tilde{\boldsymbol{H}}_i^{-1}(f_\theta(\tilde{\boldsymbol{H}}_i(\boldsymbol{I}))),$$

with random homographies $\tilde{\boldsymbol{H}}_i$ and $N_h = 100$ warps.

Lemma 2.5 (Averaging over warps)**.** *If f_θ is exactly covariant, then $\hat{F}(\boldsymbol{I}) = f_\theta(\boldsymbol{I})$ for every N_h.*

Proof. Each term equals $\tilde{\boldsymbol{H}}_i^{-1}(\tilde{\boldsymbol{H}}_i(f_\theta(\boldsymbol{I}))) = f_\theta(\boldsymbol{I})$, and the mean of N_h equal terms is that term. □

Intuition

A real network is only approximately covariant: it finds a corner from one viewpoint and misses it from another. Looking at the image through 100 different warps and voting (after un-warping the heatmaps) keeps the points the detector finds *consistently* and averages out accidental misses, so the labels impose covariance on the training data. The labels are generated on everyday photographs with no annotation of interest points: that is what "self-supervised" means here.

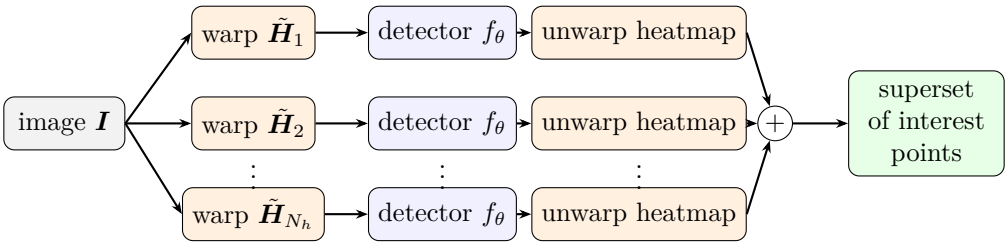


Figure 145: Homographic Adaptation (redrawn from slide 18): detect on warped copies, un-warp, average, and extract the pseudo-ground-truth points.

Architecture (slide 19). The encoder consists of eight 3×3 convolutions and three 2×2 max pools, hence one cell c per 8×8 pixels. Two heads share it (Fig. 146).

- **Keypoint head:** $\boldsymbol{S} \in \mathbb{R}^{H/8 \times W/8 \times 65}$. Each cell is classified into 65 classes: one per pixel and one for "no keypoint" (the 65th class). The loss is the cross-entropy against the pseudo-ground-truth label y_c, averaged over the $HW/64$ cells:

$$\mathcal{L}_p = -\frac{64}{HW} \sum_c \log \frac{\exp \boldsymbol{S}[c, y_c]}{\sum_{j=1}^{65} \exp \boldsymbol{S}[c, j]}.$$

- **Descriptor head:** $\boldsymbol{F} \in \mathbb{R}^{H/8 \times W/8 \times 256}$, one unit-length descriptor $\boldsymbol{f} \in \mathbb{R}^{256}$ per cell, interpolated at each keypoint $\boldsymbol{k}$ (bi-cubic interpolation, L2 normalization).

- **Descriptor loss:** over all pairs of cells of the two images, with $s = 1$ if the homography maps one onto the other,

$$s \max(0,\, 1 - \boldsymbol{f}^\top \boldsymbol{f}') + (1 - s) \max(0,\, \boldsymbol{f}^\top \boldsymbol{f}' - 0.2).$$

Self-supervision through known homography.

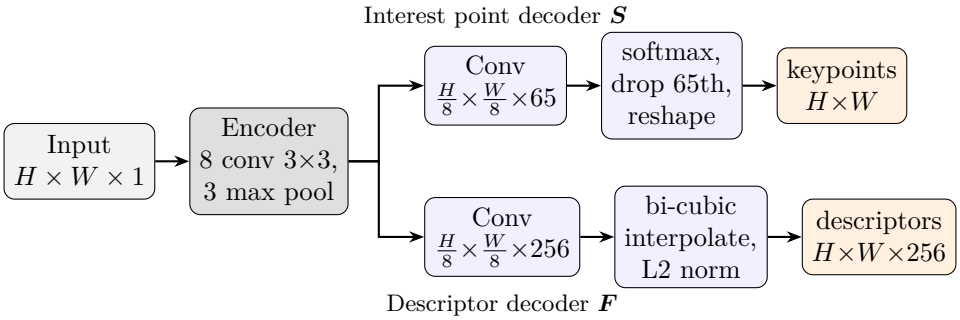


Figure 146: SuperPoint architecture (redrawn from slide 19).

Example 2.6 (Tensor sizes for a 480×640 image)**.** There are $60 \times 80 = 4800$ cells. The keypoint head outputs a $60 \times 80 \times 65$ tensor; after the softmax the dustbin channel is dropped and the remaining 64 channels are reshaped to 8×8 pixels, giving a full-resolution 480×640 heatmap. The descriptor head outputs $60 \times 80 \times 256$, i.e. a coarse semi-dense grid of descriptors that is interpolated at the detected keypoints. At 13 ms per image this is ≈ 77 frames per second (the slide quotes 70).

Intuition

Instead of asking "is this pixel a corner?" 480×640 times, the network asks each 8×8 cell "which of your 64 pixels is the corner, or is there none?". That is a 65-way classification, solved with an ordinary softmax and cross-entropy. The descriptor loss pulls descriptors of cells that the homography matches towards the same direction (dot product close to 1, margin 1) and pushes non-matching ones apart (dot product below 0.2).

Results on HPatches (slide 20). HPatches has 116 scenes, 57 with large illumination changes and 59 with large viewpoint changes; at most 1000 points per method at 480×640.

Method	ε=1	ε=3	ε=5	Rep.	MLE↓	NN mAP	M. Score
SuperPoint	0.310	**0.684**	**0.829**	0.581	1.158	**0.821**	**0.470**
LIFT	0.284	0.598	0.717	0.449	1.102	0.664	0.315
SIFT	**0.424**	0.676	0.759	0.495	**0.833**	0.694	0.313
ORB	0.150	0.395	0.538	**0.641**	1.157	0.735	0.266

Table 7: HPatches. ε=1, 3, 5: share of image pairs whose estimated homography is correct within ε pixels; Rep.: detector repeatability; MLE: mean localization error in pixels (lower is better); NN mAP: mean average precision of nearest-neighbour descriptor matching; M. Score: matching score.

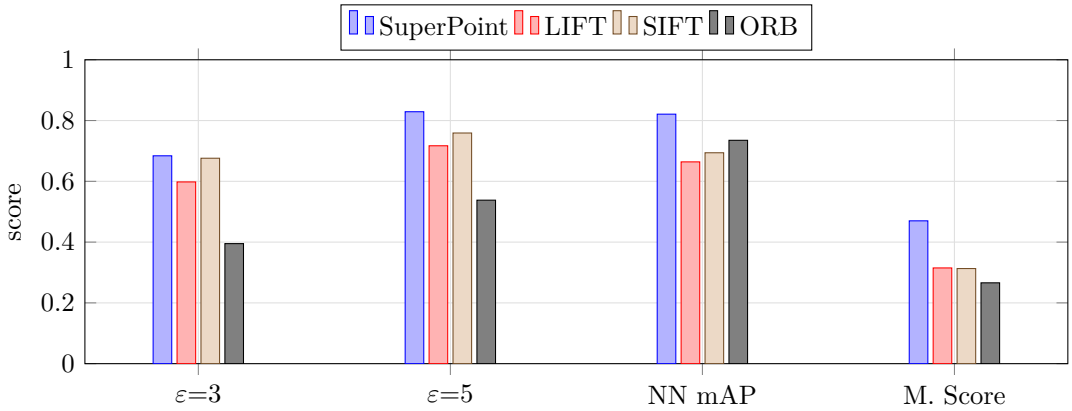


Figure 147: Selected HPatches metrics from the table (higher is better).

Intuition

SuperPoint wins on the metrics that depend on *descriptors* and on how well matches are spread across the image (homography accuracy, NN mAP, matching score), while SIFT is still best at sub-pixel *localization* (lowest MLE) and at ε=1. On slide 20 the fourth row shows a failure of SuperPoint and LIFT under an extreme in-plane rotation, a case absent from their training data: a learned method is only as invariant as its training augmentations.

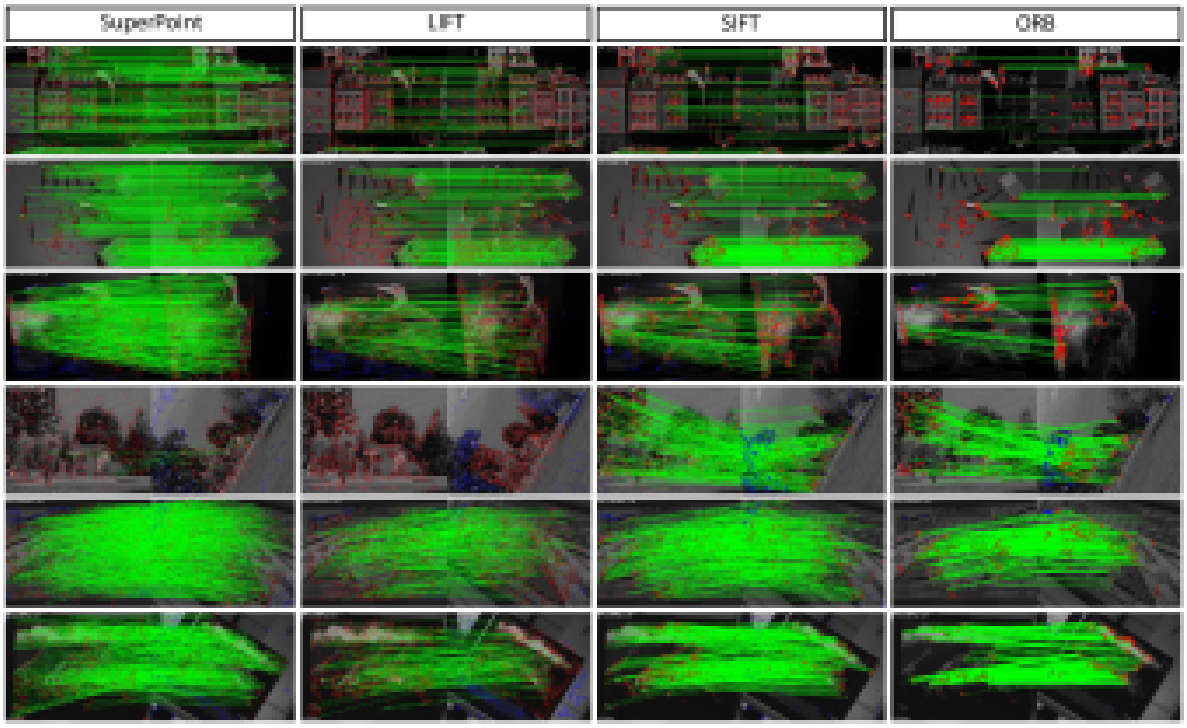


Figure 148: Qualitative matches of SuperPoint, LIFT, SIFT and ORB on six HPatches pairs (green: matches; the fourth row shows an extreme in-plane rotation).

3 Two-Frame Structure-from-Motion *(slides 21–35)*

3.1 Epipolar Geometry *(slides 22–23)*

Goal: recovery of camera pose (and 3D structure) from image correspondences. The required relationships are described by the two-view *epipolar geometry*.

- Let $\boldsymbol{R}$ and $\boldsymbol{t}$ denote the relative pose between two perspective cameras.
- A 3D point $\boldsymbol{x}$ is projected to pixel $\bar{\boldsymbol{x}}_1$ in image 1 and to pixel $\bar{\boldsymbol{x}}_2$ in image 2.
- The 3D point $\boldsymbol{x}$ and the two camera centers span the **epipolar plane**.
- The correspondence of $\bar{\boldsymbol{x}}_1$ in image 2 must lie on the **epipolar line** $\tilde{\boldsymbol{l}}_2$ in image 2.
- All epipolar lines pass through the **epipole**.

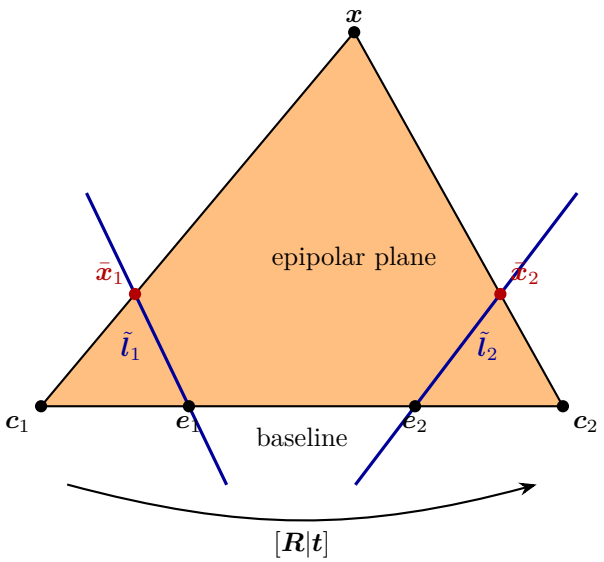


Figure 149: Epipolar geometry (redrawn from slide 22). The image planes are drawn edge-on: their intersection with the epipolar plane is the epipolar line.

Definition 3.1 (Epipolar plane, lines and epipoles)**.** For a world point $\boldsymbol{x}$ and camera centers $\boldsymbol{c}_1, \boldsymbol{c}_2$, the *epipolar plane* is the plane through $\boldsymbol{x}, \boldsymbol{c}_1, \boldsymbol{c}_2$. The *baseline* is the line $\boldsymbol{c}_1\boldsymbol{c}_2$. The *epipole* $\boldsymbol{e}_i$ in image i is the image of the other camera's center. The *epipolar line* $\tilde{\boldsymbol{l}}_2$ of a pixel $\bar{\boldsymbol{x}}_1$ is the intersection of the epipolar plane with image 2.

Intuition

If I see a point in camera 1, I know it lies somewhere on the ray through $\boldsymbol{c}_1$ and $\bar{\boldsymbol{x}}_1$, but not how far. Camera 2 sees that whole ray as a *line*. So the 2D search for a match shrinks to a 1D search along the epipolar line (slide 30 shows this on real images). Because every ray passes through $\boldsymbol{c}_1$, all the epipolar lines in image 2 pass through the projection of $\boldsymbol{c}_1$: the epipole.

Derivation of the epipolar constraint (slide 23). Let $\boldsymbol{K}_i$ be the calibration matrix of camera i, and $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$ the local ray direction of pixel $\bar{\boldsymbol{x}}_i$ in camera i.

Theorem 3.2 (Epipolar constraint)**.** *Let $\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$ relate the coordinates of a 3D point in the two camera frames, and let $\mathbb{E} = [\boldsymbol{t}]_\times \boldsymbol{R}$ be the* essential matrix*. Then*

$$\tilde{\mathbf{x}}_2^\mathsf{T}\, \mathbb{E}\, \tilde{\mathbf{x}}_1 = 0.$$

Proof. A point's camera coordinates are proportional to its ray direction: $\boldsymbol{x}_i \propto \tilde{\mathbf{x}}_i$, say $\boldsymbol{x}_1 = z_1\tilde{\mathbf{x}}_1$, $\boldsymbol{x}_2 = z_2\tilde{\mathbf{x}}_2$. Then $z_2\tilde{\mathbf{x}}_2 = z_1\boldsymbol{R}\tilde{\mathbf{x}}_1 + \boldsymbol{t}$, so $\tilde{\mathbf{x}}_2 \propto \boldsymbol{R}\tilde{\mathbf{x}}_1 + s\,\boldsymbol{t}$ for some scalar s (slide: $\boldsymbol{x}_2 \propto \boldsymbol{R}\tilde{\mathbf{x}}_1 + s\boldsymbol{t}$). Taking the cross product with $\boldsymbol{t}$ (i.e. multiplying by $[\boldsymbol{t}]_\times$, which kills $\boldsymbol{t}$) gives $[\boldsymbol{t}]_\times\tilde{\mathbf{x}}_2 \propto [\boldsymbol{t}]_\times\boldsymbol{R}\tilde{\mathbf{x}}_1$. Taking the dot product with $\tilde{\mathbf{x}}_2$: the left side is $\tilde{\mathbf{x}}_2^\mathsf{T}[\boldsymbol{t}]_\times\tilde{\mathbf{x}}_2 = 0$ because $[\boldsymbol{t}]_\times$ is skew-symmetric ($\boldsymbol{v}^\mathsf{T}[\boldsymbol{t}]_\times\boldsymbol{v} = \boldsymbol{v}\cdot(\boldsymbol{t}\times\boldsymbol{v}) = 0$ by the triple product). Hence $\tilde{\mathbf{x}}_2^\mathsf{T}[\boldsymbol{t}]_\times\boldsymbol{R}\tilde{\mathbf{x}}_1 = 0$. □

Intuition

The three vectors $\tilde{\mathbf{x}}_2$, $\boldsymbol{R}\tilde{\mathbf{x}}_1$ and $\boldsymbol{t}$ all lie in the epipolar plane, so they are linearly dependent; their scalar triple product, $\tilde{\mathbf{x}}_2\cdot(\boldsymbol{t}\times\boldsymbol{R}\tilde{\mathbf{x}}_1)$, vanishes. That is the entire constraint, written as a bilinear form $\tilde{\mathbf{x}}_2^\mathsf{T}\mathbb{E}\,\tilde{\mathbf{x}}_1 = 0$. It does not involve depth: we can test correspondences before

knowing any 3D structure.

Example 3.3 (The rectified stereo pair)**.** Take $\boldsymbol{R} = \boldsymbol{I}$ and $\boldsymbol{t} = (1,0,0)^\mathsf{T}$ (a pure sideways shift of one unit). Then $\mathbb{E} = [\boldsymbol{t}]_\times = \left(\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{smallmatrix}\right)$ and for $\tilde{\mathbf{x}}_i = (u_i, v_i, 1)^\mathsf{T}$: $\mathbb{E}\tilde{\mathbf{x}}_1 = (0,-1,v_1)^\mathsf{T}$, so $\tilde{\mathbf{x}}_2^\mathsf{T}\mathbb{E}\tilde{\mathbf{x}}_1 = v_1 - v_2 = 0$: corresponding points have the **same row**, i.e. epipolar lines are horizontal. This is the stereo setting of lecture 04. With $\boldsymbol{X} = (X,Y,Z)^\mathsf{T}$ in camera 1, $u_1 = X/Z$ and $u_2 = (X+1)/Z$, so $u_2 - u_1 = 1/Z$. If $u_1 = 0.1$ and $u_2 = 0.3$, then $Z = 5$ (in units of the baseline).

3.2 Structure of the Essential Matrix *(slides 24)*

Proposition 3.4 (Properties of $\mathbb{E}$)**.** *Let $\mathbb{E} = [\boldsymbol{t}]_\times \boldsymbol{R}$ with $\boldsymbol{R} \in \mathrm{SO}(3)$ and $\boldsymbol{t} \neq 0$. Then*

(i) $\mathbb{E}\mathbb{E}^\mathsf{T} = [\boldsymbol{t}]_\times [\boldsymbol{t}]_\times^\mathsf{T} = \|\boldsymbol{t}\|^2 \boldsymbol{I} - \boldsymbol{t}\boldsymbol{t}^\mathsf{T}$*, hence the singular values of $\mathbb{E}$ are $(\|\boldsymbol{t}\|, \|\boldsymbol{t}\|, 0)$ and $\mathbb{E}$ has rank 2;*

(ii) $\boldsymbol{t}^\mathsf{T}\mathbb{E} = 0$ *(left null vector $\boldsymbol{t} \propto \boldsymbol{e}_2$) and* $\mathbb{E}\boldsymbol{R}^\mathsf{T}\boldsymbol{t} = 0$ *(right null vector $\boldsymbol{R}^\mathsf{T}\boldsymbol{t} \propto \boldsymbol{e}_1$): each epipole is the image of the other camera's center;*

(iii) $\mathbb{E}$ has 5 degrees of freedom: 3 for $\boldsymbol{R}$ and 3 for $\boldsymbol{t}$, minus one for the scale;

(iv) (scale ambiguity) scaling the scene and $\boldsymbol{t}$ by the same α leaves every image point unchanged, since $\alpha\boldsymbol{x}_2 = \boldsymbol{R}(\alpha\boldsymbol{x}_1) + \alpha\boldsymbol{t}$. Hence $\|\boldsymbol{t}\|$ cannot be observed and $\mathbb{E}$ is defined only up to scale, sign included.

Proof. (i) $\mathbb{E}\mathbb{E}^\mathsf{T} = [\boldsymbol{t}]_\times \boldsymbol{R}\boldsymbol{R}^\mathsf{T}[\boldsymbol{t}]_\times^\mathsf{T} = [\boldsymbol{t}]_\times[\boldsymbol{t}]_\times^\mathsf{T} = \|\boldsymbol{t}\|^2\boldsymbol{I} - \boldsymbol{t}\boldsymbol{t}^\mathsf{T}$, which has eigenvalues $\|\boldsymbol{t}\|^2$ (twice, on $\boldsymbol{t}^\perp$) and 0 (along $\boldsymbol{t}$). (ii) $\boldsymbol{t}^\mathsf{T}[\boldsymbol{t}]_\times = -([\boldsymbol{t}]_\times \boldsymbol{t})^\mathsf{T} = 0$, and $\mathbb{E}\boldsymbol{R}^\mathsf{T}\boldsymbol{t} = [\boldsymbol{t}]_\times \boldsymbol{t} = 0$. The camera-1 center in camera-2 coordinates is $\boldsymbol{R}\cdot\boldsymbol{0} + \boldsymbol{t} = \boldsymbol{t}$, so $\boldsymbol{e}_2 \propto \boldsymbol{t}$; the camera-2 center in camera-1 coordinates is $-\boldsymbol{R}^\mathsf{T}\boldsymbol{t}$, so $\boldsymbol{e}_1 \propto \boldsymbol{R}^\mathsf{T}\boldsymbol{t}$. (iii) and (iv) follow from counting and from the displayed identity. □

Intuition

Rank 2 is the algebraic signature of "all epipolar lines pass through one point": the epipole is the null direction. The scale ambiguity is the same effect that makes a photo of a model train indistinguishable from a photo of a real train: a single moving camera can recover the *shape* of the scene and its *motion direction*, never the absolute size, unless something metric (a known length, an IMU, a stereo baseline) is available.

Theorem 3.5 (Characterization of essential matrices)**.** *A real 3×3 matrix is an essential matrix (up to scale) iff its SVD has the form $\boldsymbol{U}\,\mathrm{diag}(\sigma,\sigma,0)\boldsymbol{V}^\mathsf{T}$ with $\sigma > 0$.*

Proof. ($\Rightarrow$) is Prop. 3.4(i). ($\Leftarrow$) Normalize $\sigma = 1$ and choose $\boldsymbol{U}, \boldsymbol{V}$ with $\det = +1$ (flip the sign of the third column if needed: this does not change the matrix because $\sigma_3 = 0$). Theorem 3.7 below shows $\boldsymbol{U}\,\mathrm{diag}(1,1,0)\boldsymbol{V}^\mathsf{T} = [\boldsymbol{u}_3]_\times \boldsymbol{R}$ with $\boldsymbol{R} = \boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T} \in \mathrm{SO}(3)$. □

3.3 Estimating the Essential Matrix: the 8-point algorithm *(slides 25)*

Each correspondence $\tilde{\mathbf{x}}_1 = (x_1, y_1, 1)^\mathsf{T}$, $\tilde{\mathbf{x}}_2 = (x_2, y_2, 1)^\mathsf{T}$ gives one *linear* equation in the nine entries of $\mathbb{E}$. Stacking the rows of a matrix into a vector,

$$\tilde{\mathbf{x}}_2^\mathsf{T}\mathbb{E}\tilde{\mathbf{x}}_1 = \mathrm{vec}(\tilde{\mathbf{x}}_2^\mathsf{T}\mathbb{E}\tilde{\mathbf{x}}_1) = (\tilde{\mathbf{x}}_1 \otimes \tilde{\mathbf{x}}_2)^\mathsf{T}\,\mathrm{vec}(\mathbb{E}) = \boldsymbol{a}^\mathsf{T}\,\mathrm{vec}(\mathbb{E}) = 0,$$

$$\boldsymbol{a} = (x_2x_1,\ x_2y_1,\ x_2,\ y_2x_1,\ y_2y_1,\ y_2,\ x_1,\ y_1,\ 1)^\mathsf{T}, \qquad \mathrm{vec}(\mathbb{E}) = (e_{11}, e_{12}, \ldots, e_{33})^\mathsf{T}.$$

Algorithm 3.1 (Normalized 8-point algorithm)**.**

1. (Normalization) Whiten the observations to zero mean and unit variance, because products of two measurements amplify noise unevenly (Hartley 1997).
2. Stack the rows $\boldsymbol{a}_i^\mathsf{T}$ of $N \geq 8$ correspondences into $\boldsymbol{A} \in \mathbb{R}^{N\times 9}$ and solve $\min\|\boldsymbol{A}\,\mathrm{vec}(\mathbb{E})\|$ s.t. $\|\mathrm{vec}(\mathbb{E})\| = 1$: the solution is the last right singular vector of $\boldsymbol{A}$.
3. (Projection) The estimate is not an essential matrix when there is noise. Take its SVD $\boldsymbol{U}\,\mathrm{diag}(\sigma_1,\sigma_2,\sigma_3)\boldsymbol{V}^\mathsf{T}$; the closest essential matrix in the Frobenius norm has singular values $(\bar\sigma,\bar\sigma,0)$ with $\bar\sigma = (\sigma_1+\sigma_2)/2$. As the scale is free, take $\mathbb{E} = \boldsymbol{U}\,\mathrm{diag}(1,1,0)\boldsymbol{V}^\mathsf{T}$.
4. (Undo the normalization.)

Intuition

Eight equations for nine unknowns defined up to scale: that is the minimal linear count. The $\boldsymbol{A}$-matrix is built so that $\boldsymbol{A}\,\mathrm{vec}(\mathbb{E})$ lists the "epipolar errors" of all correspondences, and the last singular vector is the $\mathbb{E}$ that makes their sum of squares smallest. Step 3 then repairs the answer: the noisy $\mathbb{E}$ has three unequal singular values, so we average the top two and zero the third (which restores rank 2 and equal non-zero singular values).

Lemma 3.6 (Projection onto essential matrices)**.** *Let* $\boldsymbol{M} = \boldsymbol{U}\,\mathrm{diag}(\sigma_1,\sigma_2,\sigma_3)\boldsymbol{V}^\mathsf{T}$ *with* $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$*. Among matrices of the form* $\boldsymbol{U}'\,\mathrm{diag}(\sigma,\sigma,0)\boldsymbol{V}'^\mathsf{T}$ *the one closest to* $\boldsymbol{M}$ *in the Frobenius norm is* $\boldsymbol{U}\,\mathrm{diag}(\bar\sigma,\bar\sigma,0)\boldsymbol{V}^\mathsf{T}$ *with* $\bar\sigma = (\sigma_1+\sigma_2)/2$*.*

Sketch. The Frobenius distance between matrices with singular values $\boldsymbol{\sigma}$ and $\boldsymbol{\sigma}'$ (sorted) is minimized by sharing the singular vectors and is then $\|\boldsymbol{\sigma}-\boldsymbol{\sigma}'\|_2$ (Mirsky/von Neumann). Minimizing $(\sigma_1-s)^2+(\sigma_2-s)^2+\sigma_3^2$ over s gives $s=\bar\sigma$. □

Intuition

It is just "the average is the best single number to replace two numbers in the least-squares sense".

5-point solver. The essential matrix has 5 degrees of freedom, hence 5 correspondences suffice (Nistér 2004). This minimal solver is what runs inside RANSAC in COLMAP and OpenCV.

Beyond the slides

Minimal solvers are used inside RANSAC because the number of samples needed to hit an all-inlier set grows quickly with the sample size: with an inlier ratio $w = 0.5$, the probability that a random sample of 8 is all inliers is $w^8 \approx 0.4\%$ while for 5 it is $w^5 \approx 3.1\%$, so about $8\times$ fewer iterations are needed.

3.4 Decomposing the Essential Matrix *(slides 26)*

Write $\mathbb{E} = \boldsymbol{U}\,\mathrm{diag}(1,1,0)\boldsymbol{V}^\mathsf{T}$, where $\boldsymbol{u}_k, \boldsymbol{v}_k$ denote the k-th columns of $\boldsymbol{U},\boldsymbol{V}$ with $\det\boldsymbol{U} = \det\boldsymbol{V} = +1$. Let

$$\boldsymbol{W} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Theorem 3.7 (Four-fold ambiguity)**.** $\mathbb{E} = [\boldsymbol{t}]_\times \boldsymbol{R}$ *has exactly the four solutions (all up to*

scale)

$$\boldsymbol{R} \in \{\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T},\ \boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}\}, \qquad \boldsymbol{t} \in \{+\boldsymbol{u}_3,\ -\boldsymbol{u}_3\}, \qquad \|\boldsymbol{t}\| = 1.$$

Proof. For a unit vector $\boldsymbol{u}_3 = \boldsymbol{U}\boldsymbol{e}_3$ and $\det \boldsymbol{U} = 1$ we have $[\boldsymbol{u}_3]_\times = \boldsymbol{U}[\boldsymbol{e}_3]_\times \boldsymbol{U}^\mathsf{T}$, and $[\boldsymbol{e}_3]_\times = \operatorname{diag}(1,1,0)\boldsymbol{W}$ (direct multiplication). Hence $[\boldsymbol{u}_3]_\times = \boldsymbol{U}\operatorname{diag}(1,1,0)\boldsymbol{W}\boldsymbol{U}^\mathsf{T}$. Then

$$[\boldsymbol{u}_3]_\times (\boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}) = \boldsymbol{U}\operatorname{diag}(1,1,0)\boldsymbol{W}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T} = \mathbb{E}.$$

Moreover, $\boldsymbol{W}^2 = \operatorname{diag}(-1,-1,1)$ gives $[\boldsymbol{u}_3]_\times (\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}) = \boldsymbol{U}\operatorname{diag}(1,1,0)\boldsymbol{W}^2\boldsymbol{V}^\mathsf{T} = -\mathbb{E}$, i.e. $[-\boldsymbol{u}_3]_\times(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}) = \mathbb{E}$. So $(\boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}, +\boldsymbol{u}_3)$ and $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}, -\boldsymbol{u}_3)$ solve $\mathbb{E} = [\boldsymbol{t}]_\times \boldsymbol{R}$ exactly. Since $\mathbb{E}$ and $-\mathbb{E}$ are the same essential matrix (it is only defined up to scale, sign included), the pairs $(\boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}, -\boldsymbol{u}_3)$ and $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}, +\boldsymbol{u}_3)$ are equally valid. The null space of $[\boldsymbol{t}]_\times$ is $\operatorname{span}(\boldsymbol{t})$, which forces $\boldsymbol{t} \propto \boldsymbol{u}_3$, so there are no other solutions. Finally $\boldsymbol{R}' = \boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T} = (\boldsymbol{U}\operatorname{diag}(-1,-1,1)\boldsymbol{U}^\mathsf{T})\boldsymbol{R}$ with $\boldsymbol{R} = \boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}$: it differs from $\boldsymbol{R}$ by a half turn about $\boldsymbol{u}_3 \propto \boldsymbol{t}$. □

Intuition

Two independent ambiguities hide in $\mathbb{E}$. (1) The *sign* of $\boldsymbol{t}$: $\mathbb{E}$ and $-\mathbb{E}$ are the same constraint, so we cannot tell if the second camera is left or right of the first from the epipolar constraint alone. (2) The *twisted pair*: rotating camera 2 by 180° about the baseline maps every epipolar plane to itself, so it also satisfies the constraint. $2 \times 2 = 4$ candidates; the geometry of the next slide tells us which one is real.

3.5 Geometry of the Four Solutions *(slides 27)*

With the true pose $(\boldsymbol{R}, \boldsymbol{t})$ and the other rotation $\boldsymbol{R}'$, the four candidates place a scene point as drawn in Fig. 210.

- **Baseline reversal** ($\boldsymbol{t} \to -\boldsymbol{t}$, left to right in the slide): moves the center of camera 2, $-\boldsymbol{R}^\mathsf{T}\boldsymbol{t}$, to the other side of camera 1. Every point lands behind both cameras.
- **Twisted pair** ($\boldsymbol{R} \to \boldsymbol{R}'$, top to bottom): $\boldsymbol{R}'^\mathsf{T}\boldsymbol{R} = \boldsymbol{U}\operatorname{diag}(-1,-1,1)\boldsymbol{U}^\mathsf{T}$, a half turn about $\boldsymbol{u}_3 \propto \boldsymbol{t}$: camera 2 turns by 180° about the baseline. Each point lands in front of one camera only.

Only $(\boldsymbol{R}, \boldsymbol{t})$ puts the scene points in front of both cameras.

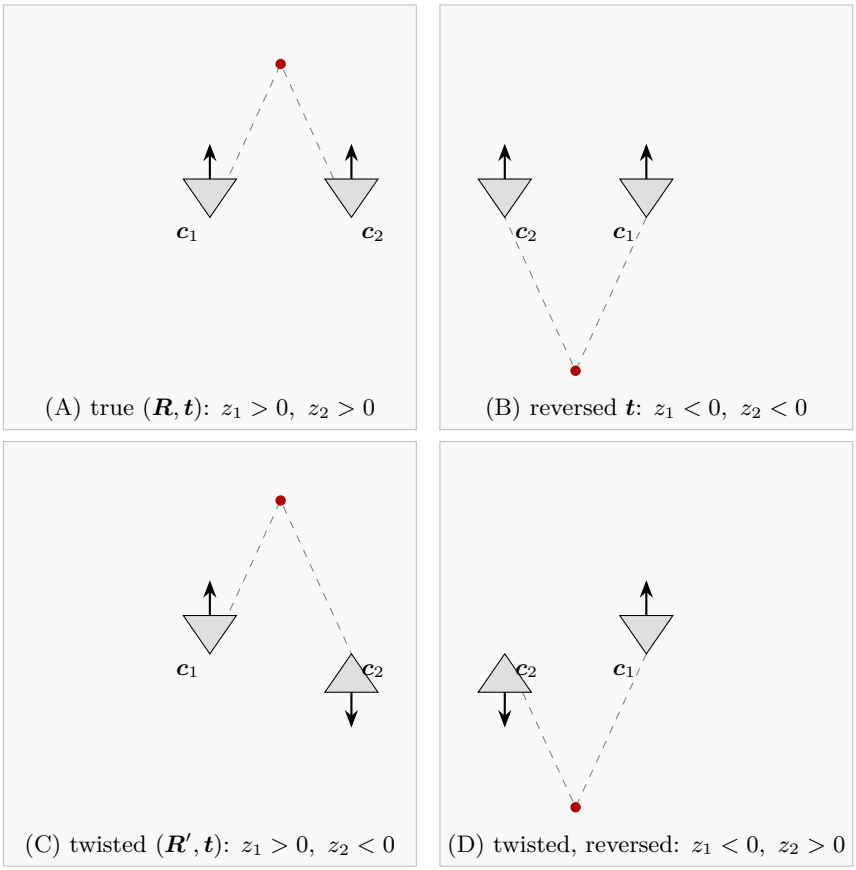


Figure 150: Schematic of the four candidate poses (after slide 27; in 2D, cameras drawn as triangles with an arrow along the viewing direction, red dot: reconstructed point, dashed: reconstructed rays). Only panel A has the point in front of both cameras.

Intuition

The point is the same in all four panels; what changes is where the two cameras "look". If I flip the baseline, I must also look in the opposite direction to see the same image, so the point appears behind me. If I twist camera 2 around the baseline, camera 2 now looks away from the scene: the point is behind it but still in front of camera 1.

3.6 Disambiguation by Cheirality *(slides 28)*

Cheirality: a reconstructed point must lie in front of both cameras. For each correspondence and each candidate $(\boldsymbol{R}_k, \boldsymbol{t}_k)$, the depths z_1, z_2 along the two rays follow from $\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$ with $\boldsymbol{x}_i = z_i \tilde{\mathbf{x}}_i$:

$$z_2 \tilde{\mathbf{x}}_2 = z_1 \boldsymbol{R}_k \tilde{\mathbf{x}}_1 + \boldsymbol{t}_k \iff \begin{bmatrix} -\boldsymbol{R}_k \tilde{\mathbf{x}}_1 & \tilde{\mathbf{x}}_2 \end{bmatrix} \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \boldsymbol{t}_k.$$

Three equations, two unknowns: solve by least squares.

Algorithm 3.2 (Pose from the essential matrix).
1. Decompose $\mathbb{E}$ into the four candidates (Thm. 3.7).
2. For each candidate and each correspondence solve the least-squares system above.
3. Keep the candidate with the most correspondences with $z_1 > 0$ and $z_2 > 0$. Count over *all* points: noise and points near infinity can flip individual signs.

Result: $[\boldsymbol{R}|\boldsymbol{t}]$, metric up to one global scale, fixed here by the convention $\|\boldsymbol{t}\| = 1$; later by the gauge of bundle adjustment (5.4) or by a normalization (5.5). In OpenCV: `findEssentialMat`

(5-point solver inside RANSAC), then `recoverPose` (decomposition and this test).

Example 3.8 (Cheirality in numbers)**.** Use the rectified example: $u_1 = 0.1$, $u_2 = 0.3$, $\boldsymbol{R} = \boldsymbol{I}$. For $\boldsymbol{t}_k = (1,0,0)^\mathsf{T}$ the system $[-\tilde{\mathbf{x}}_1 \ \tilde{\mathbf{x}}_2](z_1, z_2)^\mathsf{T} = \boldsymbol{t}_k$ with $\tilde{\mathbf{x}}_1 = (0.1, v, 1)^\mathsf{T}$, $\tilde{\mathbf{x}}_2 = (0.3, v, 1)^\mathsf{T}$ reads $-0.1z_1 + 0.3z_2 = 1$, $-vz_1 + vz_2 = 0$, $-z_1 + z_2 = 0$, so $z_1 = z_2 = z$ and $0.2z = 1$, $z = 5 > 0$: accepted. For $\boldsymbol{t}_k = (-1,0,0)^\mathsf{T}$ we get $0.2z = -1$, $z = -5 < 0$: the point would be behind both cameras: rejected.

3.7 Estimating the Epipolar Geometry with Unknown Intrinsics *(slides 29–30)*

If the calibration is unknown, we cannot use the local ray directions $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$ and only know the pixel coordinates $\bar{\boldsymbol{x}}_i$.

Definition 3.9 (Fundamental matrix)**.** Substituting $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$ in the epipolar constraint gives

$$\tilde{\mathbf{x}}_2^\mathsf{T}\mathbb{E}\tilde{\mathbf{x}}_1 = \bar{\boldsymbol{x}}_2^\mathsf{T} \underbrace{\boldsymbol{K}_2^{-\mathsf{T}}\mathbb{E}\boldsymbol{K}_1^{-1}}_{\tilde{\boldsymbol{F}}} \bar{\boldsymbol{x}}_1 = 0.$$

$\tilde{\boldsymbol{F}}$ is the *fundamental matrix*. Like $\mathbb{E}$ it is (without noise) of rank two and the epipoles can be recovered in the same way (as null vectors). The epipolar line of $\bar{\boldsymbol{x}}_1$ in image 2 is $\tilde{\boldsymbol{l}}_2 = \tilde{\boldsymbol{F}}\bar{\boldsymbol{x}}_1$.

The intrinsic parameters cannot be directly determined from $\tilde{\boldsymbol{F}}$ alone (a learned model that estimates the field of view directly from the images appears in 5.5).

Intuition

$\tilde{\boldsymbol{F}}$ is just $\mathbb{E}$ "dressed" in the unknown camera matrices: the same geometry, but expressed in pixels. It has 7 degrees of freedom (rank 2 and defined up to scale, $9 - 1 - 1$) instead of 5, because two extra parameters are not tied to a rigid motion. It is enough to draw epipolar lines and reject outliers, but you need $\boldsymbol{K}_i$ to convert it into a metric pose.

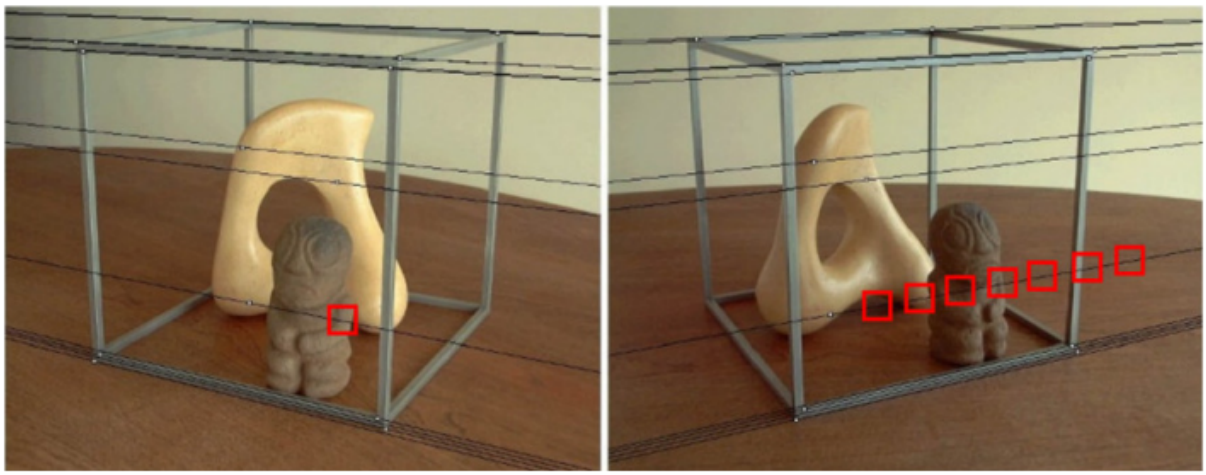


Figure 151: Epipolar lines for two real images based on the estimated epipolar geometry; corresponding points lie on the corresponding epipolar lines.

3.8 Triangulation *(slides 31–35)*

Problem (slide 32). Given the camera intrinsics and extrinsics, how can we recover 3D geometry? With noisy 2D observations $\bar{\boldsymbol{x}}_1, \bar{\boldsymbol{x}}_2$ the two rays might not intersect; we want the 3D point $\boldsymbol{x}$ that is, in some sense, closest to the two rays (Fig. 152).

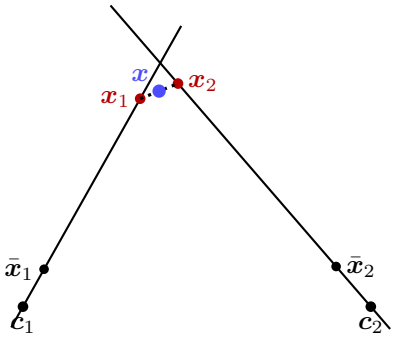


Figure 152: Two noisy rays do not intersect; $\boldsymbol{x}_1, \boldsymbol{x}_2$ are their closest points and $\boldsymbol{x}$ their midpoint (slide 32).

Proposition 3.10 (Midpoint triangulation)**.** *Let the rays be $\boldsymbol{c}_i + s_i \boldsymbol{d}_i$ ($\|\boldsymbol{d}_i\| = 1$) and $\boldsymbol{b} = \boldsymbol{c}_2 - \boldsymbol{c}_1$. The closest points have parameters solving*

$$\begin{pmatrix} 1 & -\boldsymbol{d}_1^\mathsf{T}\boldsymbol{d}_2 \\ \boldsymbol{d}_1^\mathsf{T}\boldsymbol{d}_2 & -1 \end{pmatrix} \begin{pmatrix} s_1 \\ s_2 \end{pmatrix} = \begin{pmatrix} \boldsymbol{d}_1^\mathsf{T}\boldsymbol{b} \\ \boldsymbol{d}_2^\mathsf{T}\boldsymbol{b} \end{pmatrix},$$

and the midpoint $\boldsymbol{x} = \frac{1}{2}(\boldsymbol{x}_1 + \boldsymbol{x}_2)$ minimizes the sum of squared distances to the two rays. The system degenerates exactly when $\boldsymbol{d}_1 \parallel \boldsymbol{d}_2$.

Proof. Minimize $\|\boldsymbol{c}_1 + s_1\boldsymbol{d}_1 - \boldsymbol{c}_2 - s_2\boldsymbol{d}_2\|^2$. Setting the derivatives in s_1, s_2 to zero gives $\boldsymbol{d}_1^\mathsf{T}(s_1\boldsymbol{d}_1 - s_2\boldsymbol{d}_2 - \boldsymbol{b}) = 0$ and $\boldsymbol{d}_2^\mathsf{T}(s_1\boldsymbol{d}_1 - s_2\boldsymbol{d}_2 - \boldsymbol{b}) = 0$, which is the stated system. Its determinant is $-1 + (\boldsymbol{d}_1^\mathsf{T}\boldsymbol{d}_2)^2 = -\sin^2\gamma$, where γ is the angle between the rays: zero iff the rays are parallel. □

Intuition

Two rays in 3D generically miss each other by a small gap; the segment joining their closest points is perpendicular to both. Taking its midpoint is the natural compromise. The determinant $\sin^2\gamma$ already hints at the next topic: when the rays are nearly parallel the problem becomes ill-conditioned.

Linear triangulation (DLT, slide 33). Let $\tilde{\boldsymbol{x}}_i^s = \tilde{\boldsymbol{P}}_i \tilde{\boldsymbol{x}}^w$ denote the projection of a 3D world point onto the image of the i-th camera. Both sides are homogeneous: they have the same direction but may differ in magnitude, so we consider the cross product $\bar{\boldsymbol{x}}_i^s \times \tilde{\boldsymbol{P}}_i \tilde{\boldsymbol{x}}^w = 0$. With $\tilde{\boldsymbol{p}}_{ik}^\mathsf{T}$ the k-th row of $\tilde{\boldsymbol{P}}_i$ this gives

$$\underbrace{\begin{pmatrix} x_i^s\,\tilde{\boldsymbol{p}}_{i3}^\mathsf{T} - \tilde{\boldsymbol{p}}_{i1}^\mathsf{T} \\ y_i^s\,\tilde{\boldsymbol{p}}_{i3}^\mathsf{T} - \tilde{\boldsymbol{p}}_{i2}^\mathsf{T} \end{pmatrix}}_{\boldsymbol{A}_i} \tilde{\boldsymbol{x}}^w = 0.$$

Stacking $N \geq 2$ observations yields $\boldsymbol{A}\tilde{\boldsymbol{x}}^w = 0$. As $\tilde{\boldsymbol{x}}^w$ is homogeneous this is a constrained least-squares problem; the solution is the right singular vector of $\boldsymbol{A}$ with the smallest singular value (the Direct Linear Transformation, as for homography estimation in lecture 01).

Intuition

Each view contributes two independent linear equations (the third row of the cross product is a combination of the other two), and the unknown 3D point has 3 degrees of freedom, so two views already give $4 \geq 3$ equations. A single image of a point removes two degrees of freedom (it constrains the point to a line); the second one removes the rest and adds one redundant equation to absorb noise.

Reprojection error minimization (slide 34). While DLT often works well, it is not invariant to projective transformations. The gold standard is to minimize the reprojection error numerically,

$$\bar{\boldsymbol{x}}_w^* = \arg\min_{\bar{\boldsymbol{x}}_w} \sum_{i=1}^{N} \|\bar{\boldsymbol{x}}_i^s(\bar{\boldsymbol{x}}_w) - \bar{\boldsymbol{x}}_i^o\|_2^2,$$

with observations $\bar{\boldsymbol{x}}_i^o$. This allows taking measurement noise appropriately into account. For two views the minimum can also be obtained in closed form as the solution of a sixth degree polynomial (Hartley and Zisserman, Section 12.5).

Triangulation uncertainty (slide 35). Triangulation works differently well depending on the relative camera pose: uncertainty (shaded region) increases as the rays become more parallel. There is a **trade-off**: feature matching is easier for nearby views, but triangulation is harder. (A learned prior can also reconstruct views with little or no overlap, see 5.5.)

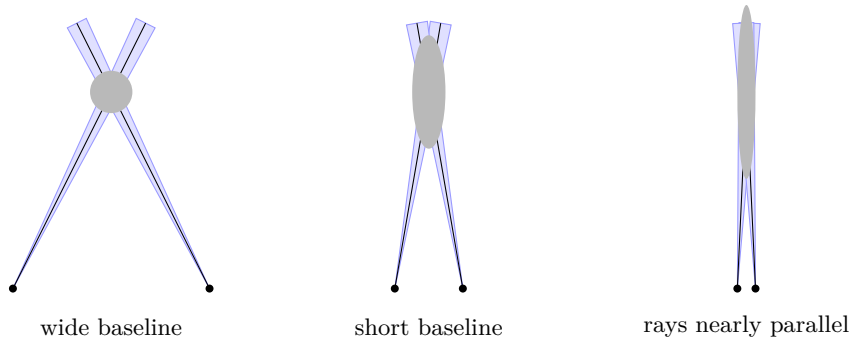


Figure 153: Schematic of triangulation uncertainty (after slide 35): the grey region is where the two noisy rays are consistent with the observations. It grows as the rays become parallel. The ellipses are drawn by hand, not computed.

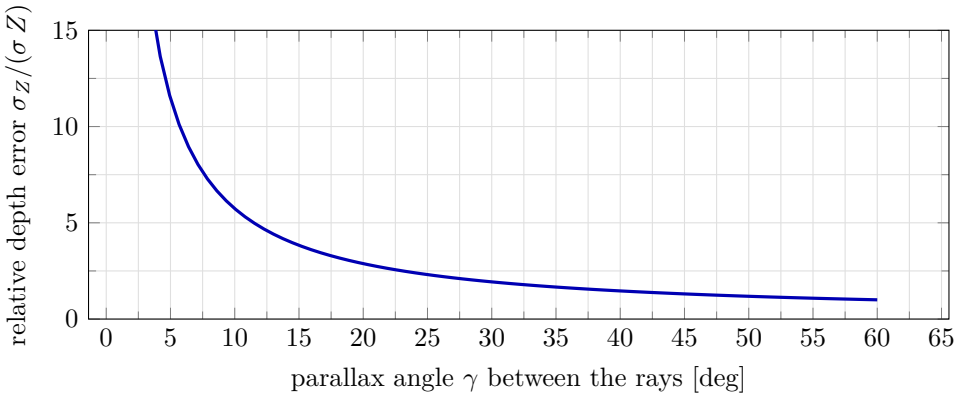


Figure 154: Original illustration of the trade-off. For a point at distance Z seen under angular noise σ per ray, the depth error is, to first order, $\sigma_Z \approx \sigma Z/(2\sin(\gamma/2)) \approx \sigma Z/\gamma$ (small γ). Below about 5° the error explodes; above about 30° it is flat, but matching becomes hard.

Intuition

Hold a finger at arm's length and look with one eye at a time: it jumps against the background. Do the same with a mountain: nothing moves. The jump is the parallax. Parallax is the signal that triangulation measures; if it is tiny, noise swamps it. Meanwhile large viewpoint changes make appearance very different, so matching fails. SfM pipelines therefore choose their image pairs carefully ('enough baseline for triangulation', slide 48).

4 Bundle Adjustment *(slides 36–54)*

4.1 Bundle Adjustment *(slides 37–39)*



Figure 155: Photo collection of Notre-Dame (left) and the resulting sparse reconstruction with camera frusta (right) (Snavely, Seitz and Szeliski, SIGGRAPH 2006).

Goal: optimize the reprojection errors (distance between an observed feature and the projected 3D point in the image plane) with respect to the camera parameters and the 3D point cloud.

Definition 4.1 (Bundle adjustment)**.** Let $\mathcal{T} = \{\boldsymbol{T}_i\}$ with $\boldsymbol{T}_i \in \mathrm{SE}(3)$ $(i = 1..N)$ denote the camera poses; the calibrations $\boldsymbol{K}_i$ are known and fixed. Let $\mathcal{X}_w = \{\boldsymbol{x}_p^w\}$, $\boldsymbol{x}_p^w \in \mathbb{R}^3$ $(p = 1..P)$ be the 3D points in world coordinates and $\mathcal{X}_s = \{\boldsymbol{x}_{ip}^s\}$, $\boldsymbol{x}_{ip}^s \in \mathbb{R}^2$ the image observations in all cameras. Bundle adjustment minimizes the reprojection error of all observations:

$$\mathcal{T}^*, \mathcal{X}_w^* = \arg\min_{\mathcal{T}, \mathcal{X}_w} \sum_{i=1}^{N} \sum_{p=1}^{P} w_{ip} \left\| \boldsymbol{x}_{ip}^s - \pi_i(\boldsymbol{x}_p^w) \right\|_2^2,$$

where w_{ip} indicates whether point p is observed in image i and $\pi_i(\boldsymbol{x}_p^w)$ is the projection of the 3D world point into camera i with its pose $\boldsymbol{T}_i = [\boldsymbol{R}_i | \boldsymbol{t}_i]$ and calibration $\boldsymbol{K}_i$:

$$\pi_i(\boldsymbol{x}_p^w) = \begin{pmatrix} \tilde{x}_{ip}^s / \tilde{w}_{ip}^s \\ \tilde{y}_{ip}^s / \tilde{w}_{ip}^s \end{pmatrix}, \qquad \tilde{\boldsymbol{x}}_{ip}^s = \boldsymbol{K}_i (\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i).$$

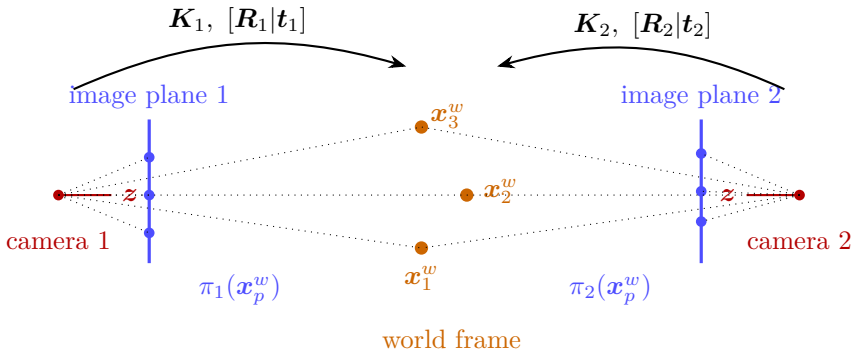


Figure 156: The bundle adjustment setup (slide 39): the 3D points are projected into both cameras by π_i. With the $\boldsymbol{K}_i$ fixed, bundle adjustment optimizes the poses $\mathcal{T}$ and the points $\mathcal{X}_w$ jointly.

Intuition

Imagine each 3D point and each camera attached to the others by elastic bands: every observation is a band pulling the *predicted* pixel towards the *measured* pixel. The name "bundle" refers to the bundles of light rays between each camera center and its points. Adjusting them all at once (poses and points) finds the configuration where the total tension (squared reprojection error) is lowest. This is the maximum-likelihood estimate under independent isotropic Gaussian pixel noise.

4.2 Challenges of Bundle Adjustment *(slides 40)*

- **Initialization.** The energy landscape is highly non-convex: a good initialization is crucial to avoid bad local minima. As initializing all 3D points and cameras jointly is difficult (occlusion, viewpoint, matching outliers), *incremental* bundle adjustment starts from a carefully selected two-view reconstruction and iteratively adds new images/cameras.
- **Optimization.** With millions of features and thousands of cameras, large-scale bundle adjustment is computationally demanding (cubic complexity in the number of unknowns). Luckily the problem is *sparse* (not every 3D point is observed in every camera), and efficient sparse implementations (e.g. g2o) can be exploited.
- Here the calibrations $\boldsymbol{K}_i$ are fixed; bundle adjustment can also estimate one $\boldsymbol{K}_i$ per camera.
- Predicting cameras and points with a network removes the incremental initialization (5.5).

4.3 Bundle Adjustment as Nonlinear Least Squares *(slides 41)*

Collect the unknowns in $\mathcal{S} = (\boldsymbol{T}_1, \dots, \boldsymbol{T}_N, \boldsymbol{x}_1^w, \dots, \boldsymbol{x}_P^w)$ with $\boldsymbol{T}_i = \begin{pmatrix} \boldsymbol{R}_i & \boldsymbol{t}_i \\ \boldsymbol{0}^\mathsf{T} & 1 \end{pmatrix} \in \mathrm{SE}(3)$ and $\boldsymbol{x}_p^w \in \mathbb{R}^3$. Each residual compares the observation with the projection of the point, moved into camera i by its pose:

$$E(\mathcal{S}) = \sum_{i,p} w_{ip} \|\boldsymbol{r}_{ip}\|_2^2, \qquad \boldsymbol{r}_{ip} = \boldsymbol{x}_{ip}^s - \pi(\boldsymbol{K}_i(\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i)),$$

with the perspective division $\pi(\tilde{x}, \tilde{y}, \tilde{w}) = (\tilde{x}/\tilde{w}, \tilde{y}/\tilde{w})$. Stacking all residuals into $\boldsymbol{r}(\mathcal{S})$ gives $E(\mathcal{S}) = \|\boldsymbol{r}(\mathcal{S})\|_2^2$.

- π_i contains a rotation and a perspective division: there is no closed form, and the minimization iterates from an initialization.
- The poses do not form a vector space, so each iteration must update them on SE(3).

- E does not change under a global rotation, translation and scale of all poses and points: these 7 degrees of freedom are fixed by holding one pose and one distance.

Proposition 4.2 (Gauge freedom)**.** *Let* $(\boldsymbol{R}_i, \boldsymbol{t}_i, \boldsymbol{x}_p^w)$ *be any solution. For any rotation* $\boldsymbol{Q}$*, translation* $\boldsymbol{a}$ *and scale* $\alpha > 0$*, define* $\boldsymbol{x}_p'^w = \alpha \boldsymbol{Q} \boldsymbol{x}_p^w + \boldsymbol{a}$*,* $\boldsymbol{R}_i' = \boldsymbol{R}_i \boldsymbol{Q}^\mathsf{T}$*,* $\boldsymbol{t}_i' = \alpha \boldsymbol{t}_i - \boldsymbol{R}_i \boldsymbol{Q}^\mathsf{T} \boldsymbol{a}$*. Then every projection, and hence* E*, is unchanged. Consequently* E *is constant along a 7-dimensional family of solutions (the Hessian has a 7-dimensional null space).*

Proof. $\boldsymbol{R}_i' \boldsymbol{x}_p'^w + \boldsymbol{t}_i' = \alpha \boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{R}_i \boldsymbol{Q}^\mathsf{T} \boldsymbol{a} + \alpha \boldsymbol{t}_i - \boldsymbol{R}_i \boldsymbol{Q}^\mathsf{T} \boldsymbol{a} = \alpha (\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i)$, so each camera-frame point is only multiplied by α. Since a pixel is a ratio of coordinates, π_i is invariant to scaling its argument, and all residuals are unchanged. The family has $3 + 3 + 1 = 7$ parameters. □

Intuition

The images do not know where the world origin is, which way is "north", or whether the scene is a toy or a real building (the scale ambiguity of Section 5.3). Fixing *one pose* (6 dof: $\boldsymbol{T}_1 = \boldsymbol{I}$) plus *one distance* (1 dof: e.g. $\|\boldsymbol{t}_2\| = 1$) removes the 7 free directions; this is precisely the "gauge" chosen in the initialization of incremental SfM (slide 48).

4.4 Updates in the Tangent Space *(slides 42)*

A pose cannot be updated by addition: $\boldsymbol{T}_i + \Delta \boldsymbol{T}$ is in general not a rigid transformation, since $\boldsymbol{R}^\mathsf{T} \boldsymbol{R} = \boldsymbol{I}$ breaks, and its 12 entries carry only 6 degrees of freedom. The update is instead a 6-vector in the Lie algebra $\mathfrak{se}(3) \cong \mathbb{R}^6$, the tangent space of SE(3), mapped onto the group by the exponential map. Points live in $\mathbb{R}^3$ and are added:

$$\boldsymbol{T}_i \leftarrow \operatorname{Exp}(\Delta \boldsymbol{\xi}_i)\, \boldsymbol{T}_i, \qquad \boldsymbol{x}_p^w \leftarrow \boldsymbol{x}_p^w + \Delta \boldsymbol{x}_p, \qquad \operatorname{Exp}(\Delta \boldsymbol{\xi}) = \exp \begin{pmatrix} [\Delta \boldsymbol{\omega}]_\times & \Delta \boldsymbol{\rho} \\ \boldsymbol{0}^\mathsf{T} & 0 \end{pmatrix}, \quad \Delta \boldsymbol{\xi} = (\Delta \boldsymbol{\rho}, \Delta \boldsymbol{\omega}),$$

where $\Delta \boldsymbol{\omega}$ is an axis-angle rotation, $\Delta \boldsymbol{\rho}$ the translational part and exp the matrix exponential. The update is *minimal and unconstrained*: 6 numbers per pose, and every update stays in SE(3).

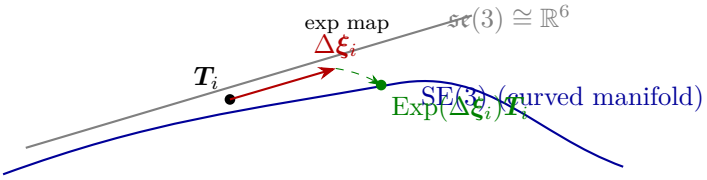


Figure 157: Local update on a curved space: take a step $\Delta \boldsymbol{\xi}_i$ in the flat tangent space, then map it back onto the group with the exponential map (schematic).

Beyond the slides

For $\theta = \|\Delta \boldsymbol{\omega}\|$ the exponential has a closed form (Rodrigues): $\exp[\Delta \boldsymbol{\omega}]_\times = \boldsymbol{I} + \frac{\sin\theta}{\theta}[\Delta \boldsymbol{\omega}]_\times + \frac{1-\cos\theta}{\theta^2}[\Delta \boldsymbol{\omega}]_\times^2$, and $\operatorname{Exp}(\Delta \boldsymbol{\xi}) = \begin{pmatrix} \exp[\Delta \boldsymbol{\omega}]_\times & \boldsymbol{V} \Delta \boldsymbol{\rho} \\ \boldsymbol{0}^\mathsf{T} & 1 \end{pmatrix}$ with $\boldsymbol{V} = \boldsymbol{I} + \frac{1-\cos\theta}{\theta^2}[\Delta \boldsymbol{\omega}]_\times + \frac{\theta - \sin\theta}{\theta^3}[\Delta \boldsymbol{\omega}]_\times^2$.

Intuition

Think of walking on the surface of the Earth: you plan a step on a flat map (the tangent plane) and then "wrap" it back onto the sphere. If you added numbers straight into the 12 matrix entries you would step *off* the surface (the matrix stops being a rotation).

4.5 The Gauss–Newton Algorithm *(slides 43)*

Each residual depends on one pose and one point. Around the current estimate it is linearized in their increments:

$$\boldsymbol{r}_{ip}(\mathrm{Exp}(\Delta\boldsymbol{\xi}_i)\boldsymbol{T}_i,\ \boldsymbol{x}_p^w + \Delta\boldsymbol{x}_p) \approx \boldsymbol{r}_{ip} + \boldsymbol{J}_{ip}^{\xi}\Delta\boldsymbol{\xi}_i + \boldsymbol{J}_{ip}^{x}\Delta\boldsymbol{x}_p.$$

Lemma 4.3 (Jacobians of the reprojection residual)**.** *With $\boldsymbol{x}_{ip}^c = \boldsymbol{R}_i\boldsymbol{x}_p^w + \boldsymbol{t}_i$ and $\boldsymbol{J}_\pi \in \mathbb{R}^{2\times 3}$ the Jacobian of the projection $\boldsymbol{x}^c \mapsto \pi(\boldsymbol{K}_i\boldsymbol{x}^c)$ (so $\boldsymbol{J}_\pi = (\partial\pi/\partial\tilde{\boldsymbol{x}})\boldsymbol{K}_i$), both blocks at zero increment are*

$$\boldsymbol{J}_{ip}^{\xi} = -\boldsymbol{J}_\pi\begin{bmatrix}\boldsymbol{I} & -[\boldsymbol{x}_{ip}^c]_\times\end{bmatrix} \in \mathbb{R}^{2\times 6}, \qquad \boldsymbol{J}_{ip}^{x} = -\boldsymbol{J}_\pi\boldsymbol{R}_i \in \mathbb{R}^{2\times 3}.$$

Proof. For a small $\Delta\boldsymbol{\xi} = (\Delta\boldsymbol{\rho}, \Delta\boldsymbol{\omega})$, $\mathrm{Exp}(\Delta\boldsymbol{\xi}) \approx \boldsymbol{I} + \begin{pmatrix}[\Delta\boldsymbol{\omega}]_\times & \Delta\boldsymbol{\rho}\\ \boldsymbol{0}^\mathsf{T} & 0\end{pmatrix}$, so the camera-frame point becomes $\boldsymbol{x}^c + \Delta\boldsymbol{\omega}\times\boldsymbol{x}^c + \Delta\boldsymbol{\rho} = \boldsymbol{x}^c + \Delta\boldsymbol{\rho} - [\boldsymbol{x}^c]_\times\Delta\boldsymbol{\omega}$. The residual is $\boldsymbol{x}^s - \pi(\boldsymbol{K}\boldsymbol{x}^c)$, so by the chain rule $\partial\boldsymbol{r}/\partial\Delta\boldsymbol{\xi} = -\boldsymbol{J}_\pi[\boldsymbol{I} \quad -[\boldsymbol{x}^c]_\times]$. A change $\Delta\boldsymbol{x}$ of the world point changes $\boldsymbol{x}^c$ by $\boldsymbol{R}_i\Delta\boldsymbol{x}$, giving $-\boldsymbol{J}_\pi\boldsymbol{R}_i$. □

Intuition

The first block says: moving the camera by $\Delta\boldsymbol{\rho}$ is the same as moving the point by $\Delta\boldsymbol{\rho}$ in the camera frame, while rotating by $\Delta\boldsymbol{\omega}$ moves the point sideways by an amount proportional to its distance ($\Delta\boldsymbol{\omega}\times\boldsymbol{x}^c$): far points are sensitive to rotation, near points to translation. The leading minus sign comes from $\boldsymbol{r}$ = observed − predicted.

With all increments stacked into $\Delta\boldsymbol{s} = (\Delta\boldsymbol{\xi}_1, \ldots, \Delta\boldsymbol{\xi}_N, \Delta\boldsymbol{x}_1, \ldots, \Delta\boldsymbol{x}_P) \in \mathbb{R}^{6N+3P}$, minimizing $\|\boldsymbol{r} + \boldsymbol{J}\Delta\boldsymbol{s}\|_2^2$ gives the normal equations; the update is repeated until convergence:

$$\boldsymbol{H}\Delta\boldsymbol{s} = -\boldsymbol{b}, \quad \boldsymbol{H} = \boldsymbol{J}^\mathsf{T}\boldsymbol{J}, \quad \boldsymbol{b} = \boldsymbol{J}^\mathsf{T}\boldsymbol{r}, \qquad \boldsymbol{T}_i \leftarrow \mathrm{Exp}(\Delta\boldsymbol{\xi}_i)\boldsymbol{T}_i, \quad \boldsymbol{x}_p^w \leftarrow \boldsymbol{x}_p^w + \Delta\boldsymbol{x}_p.$$

Algorithm 4.1 (Gauss–Newton for bundle adjustment)**.**

1. Start from an initialization $\mathcal{S}_0$ (e.g. two-view reconstruction).
2. Compute all residuals $\boldsymbol{r}_{ip}$ and Jacobians (Lemma 4.3).
3. Build $\boldsymbol{H} = \boldsymbol{J}^\mathsf{T}\boldsymbol{J}$ and $\boldsymbol{b} = \boldsymbol{J}^\mathsf{T}\boldsymbol{r}$ and solve $\boldsymbol{H}\Delta\boldsymbol{s} = -\boldsymbol{b}$ (exploiting sparsity, next subsection).
4. Update poses on SE(3) and points in $\mathbb{R}^3$; repeat steps 2–4 until $\|\Delta\boldsymbol{s}\|$ or the decrease of E is small.

Beyond the slides

In practice, Levenberg–Marquardt is used: $(\boldsymbol{H} + \lambda\boldsymbol{D})\Delta\boldsymbol{s} = -\boldsymbol{b}$ with a damping λ that interpolates between Gauss–Newton ($\lambda \to 0$, fast near the minimum) and gradient descent (λ large, safe far away). Robust loss functions (Huber, Cauchy) are commonly applied to reduce the effect of outliers. COLMAP uses the solver Ceres (slide 51).

4.6 The Schur Complement *(slides 44)*

With the pose increments $\Delta\boldsymbol{\xi} \in \mathbb{R}^{6N}$ ordered before the point increments $\Delta\boldsymbol{x} \in \mathbb{R}^{3P}$, the normal equations read

$$\begin{pmatrix}\boldsymbol{H}_{\xi\xi} & \boldsymbol{H}_{\xi x}\\ \boldsymbol{H}_{x\xi} & \boldsymbol{H}_{xx}\end{pmatrix}\begin{pmatrix}\Delta\boldsymbol{\xi}\\ \Delta\boldsymbol{x}\end{pmatrix} = -\begin{pmatrix}\boldsymbol{b}_\xi\\ \boldsymbol{b}_x\end{pmatrix}.$$

$\boldsymbol{H}_{xx}$ is block diagonal, one 3×3 block per point, so its inverse is cheap. Eliminating the points leaves a system in the $6N$ pose increments alone, and back substitution recovers the points.

Theorem 4.4 (Schur complement reduction)**.** *If $\boldsymbol{H}_{xx}$ is invertible, the pose increments solve*

$$(\boldsymbol{H}_{\xi\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{H}_{x\xi})\Delta\boldsymbol{\xi} = -(\boldsymbol{b}_\xi - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{b}_x), \qquad \Delta\boldsymbol{x} = -\boldsymbol{H}_{xx}^{-1}(\boldsymbol{b}_x + \boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi}),$$

and these give the same pose update as the full system.

Proof. The second block row reads $\boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi} + \boldsymbol{H}_{xx}\Delta\boldsymbol{x} = -\boldsymbol{b}_x$, which gives $\Delta\boldsymbol{x}$ as stated. Substituting it into the first block row, $\boldsymbol{H}_{\xi\xi}\Delta\boldsymbol{\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}(\boldsymbol{b}_x + \boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi}) = -\boldsymbol{b}_\xi$, and rearranging gives the reduced system. □

The matrix on the left, $\boldsymbol{S} = \boldsymbol{H}_{\xi\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{H}_{x\xi}$, is the Schur complement of $\boldsymbol{H}_{xx}$ in $\boldsymbol{H}$. It yields the same pose update from $6N$ unknowns instead of $6N + 3P$: thousands instead of millions. Each point is then updated independently.

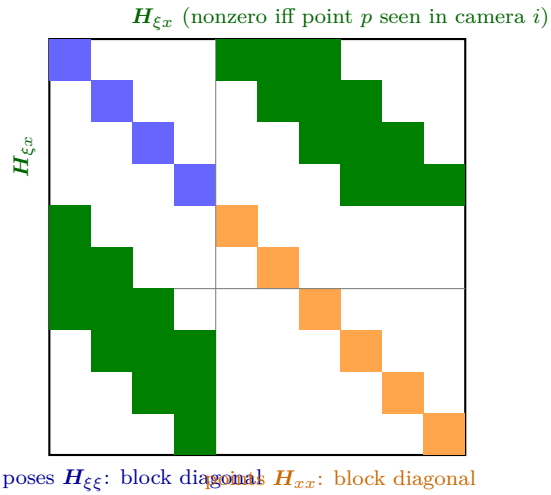


Figure 158: Arrowhead sparsity of the BA Hessian for $N = 4$ cameras and $P = 6$ points (original illustration). Cameras and points interact only if they share an observation. Eliminating the points (orange) leaves a small 4×4-block reduced camera system.

Example 4.5 (Why the Schur complement matters)**.** Take $N = 100$ cameras and $P = 100{,}000$ points (a modest dataset). The full system has $6N + 3P = 300{,}600$ unknowns, a dense solve would cost $\sim (3 \cdot 10^5)^3 \approx 2.7 \cdot 10^{16}$ operations. The reduced system has $6N = 600$ unknowns, costing $\sim 600^3 \approx 2.2 \cdot 10^8$ operations; the 10^5 small 3×3 inversions are trivial. That is a factor of about 10^8.

Intuition

The points never talk directly to each other, only through the cameras that see them. So we can "integrate out" each point (a tiny 3×3 problem), which replaces the point-to-camera coupling by an additional camera-to-camera coupling for every pair of cameras that share a point. After solving for the cameras, each point just reads the new camera positions and updates itself. (This is the standard trick behind *reduced camera systems* in Triggs et al. 2000.)

4.7 Incremental Structure-from-Motion *(slides 45–51)*

The feature matching and reconstruction pipeline (COLMAP, Schönberger and Frahm 2016) has two big stages (Fig. 159): **correspondence search** (find and match robust 2D features across images) and **incremental reconstruction** (start with two views and incrementally add cameras).

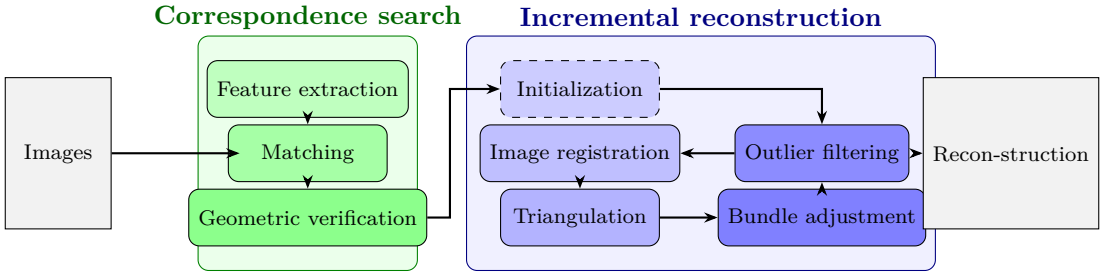


Figure 159: Incremental SfM pipeline of COLMAP (redrawn from slide 45). The registration–triangulation–filtering–bundle-adjustment loop runs once per added image.

Feature extraction (slide 46). Detect features (e.g. SIFT, SURF, BRISK) in all input images.

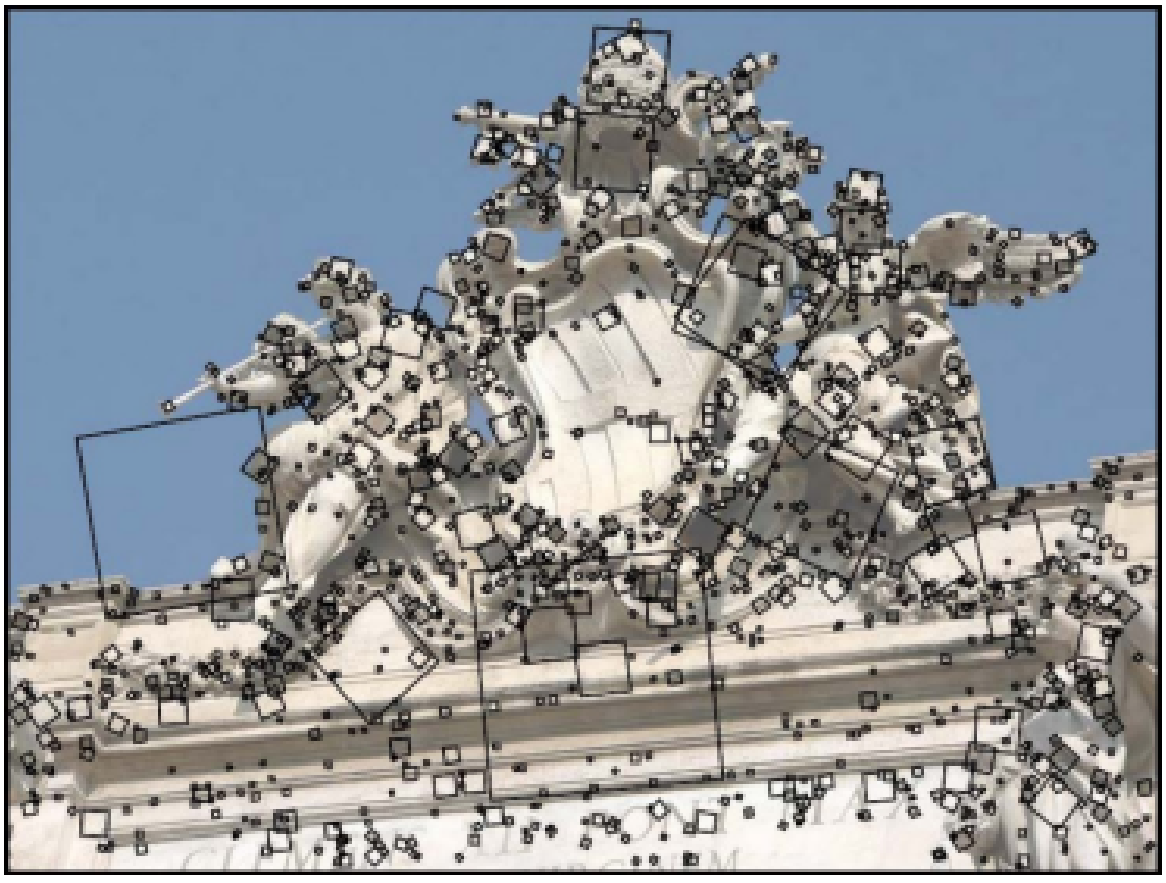


Figure 160: Features detected on a photograph of a monument: boxes show location, scale and orientation of each feature (Schönberger and Frahm, CVPR 2016).

Feature matching and geometric verification (slide 47). Find overlapping image pairs and the associated feature correspondences. Candidate matches are verified geometrically, i.e.

they are kept only if they are consistent with an epipolar geometry or homography (RANSAC).

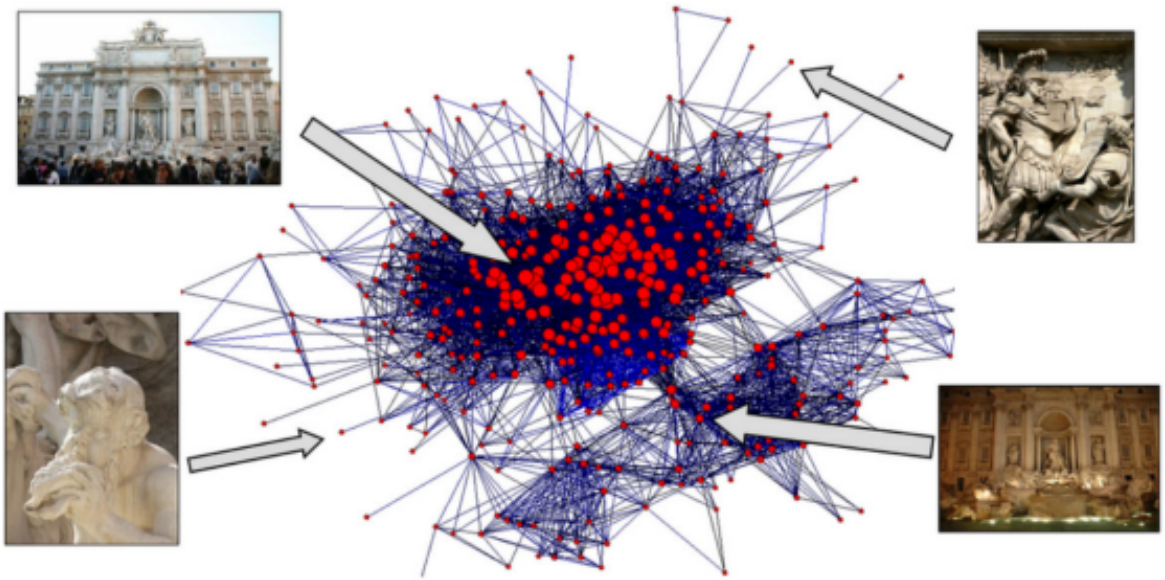


Figure 161: Graph of images (nodes) connected when they overlap; example photos of the Trevi fountain at different scales and lighting (Schönberger and Frahm, CVPR 2016).

Initialization (slide 48).

Algorithm 4.2 (SfM initialization from two views)**.**

1. Select **two views** with many correspondences $\bar{\boldsymbol{x}}_1 \leftrightarrow \bar{\boldsymbol{x}}_2$ and enough baseline for triangulation.
2. Estimate $\mathbb{E}$ from $\tilde{\mathbf{x}}_2^\mathsf{T}\mathbb{E}\tilde{\mathbf{x}}_1 = 0$, with $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$, by the 5-point solver inside RANSAC.
3. Decompose $\mathbb{E}$ into $[\boldsymbol{R}|\boldsymbol{t}]$ and keep the candidate that puts the points in front of both cameras (Alg. 3.2).
4. Fix the gauge: $\boldsymbol{T}_1 = \boldsymbol{I}$ and $\boldsymbol{T}_2 = [\boldsymbol{R}|\boldsymbol{t}]$ with $\|\boldsymbol{t}\| = 1$ (this removes the 7 gauge degrees of freedom, Prop. 3.3).
5. **Triangulate** the inliers to obtain the first points $\boldsymbol{x}_p^w$.

Image registration (slide 49). Find a new image with correspondences to the current set and estimate its camera pose. Let $\mathcal{X} = \{\bar{\boldsymbol{x}}_i^s, \bar{\boldsymbol{x}}_i^w\}_{i=1}^N$ be a set of 3D-to-2D correspondences related by $\bar{\boldsymbol{x}}_i^s = \boldsymbol{P}\bar{\boldsymbol{x}}_i^w$. As the vectors are homogeneous, they have the same direction but differ in magnitude, so the equation can be expressed as $\bar{\boldsymbol{x}}_i^s \times \boldsymbol{P}\bar{\boldsymbol{x}}_i^w = 0$. Using the Direct Linear Transform this is a linear equation in the entries of $\boldsymbol{P}$; the solution of the constrained system (fixing $\boldsymbol{P}$'s scale) is given by SVD. Given that $\boldsymbol{P} = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}]$ and $\boldsymbol{K}$ is upper-triangular, both $\boldsymbol{K}$ and $\boldsymbol{R}$ can be obtained from the 3×3 front submatrix of $\boldsymbol{P}$ by standard RQ factorization. If $\boldsymbol{K}$ is known, we can even estimate $\boldsymbol{P}$ from only three points (P3P). In practice, RANSAC is used to remove outliers.

Intuition

Registration is the "PnP" problem: given a few points whose 3D position is already known from earlier triangulation, find where the new camera stands. $\boldsymbol{P}$ has 11 degrees of freedom (a 3×4 matrix up to scale) and every 3D–2D pair provides two independent equations, so DLT needs at least 6 points ($11/2 = 5.5$, rounded up). If $\boldsymbol{K}$ is known only 6 unknowns remain, and 3 points (6 equations) suffice, at the price of up to four solutions (P3P). Triangulation (known cameras $\rightarrow$ points) and registration (known points $\rightarrow$ camera) alternate: this is how the reconstruction "crawls" outwards.

Triangulation (slide 50). Given the newly registered image, new correspondences can be triangulated. In COLMAP, a robust triangulation method handles outliers.

Bundle adjustment and outlier filtering (slide 51). Minimize the reprojection error of all observations with respect to all poses and 3D points (Def. 4.1). Since incremental SfM only affects the model locally, COLMAP performs *local* BA on the locally connected images, and *global* BA only once in a while for efficiency. For the sparse large-scale optimization it uses Ceres. After each BA, observations with large reprojection errors and cameras with abnormal fields of view or large distortion coefficients are removed.

Algorithm 4.3 (Incremental SfM (COLMAP-style))**.**

1. Extract features in all images, match overlapping pairs, verify geometrically.
2. Initialize with a two-view reconstruction (Alg. 4.1).
3. **Repeat** until no image can be added: register the best new image (DLT/P3P inside RANSAC); triangulate new points; run local BA; filter outliers (large reprojection error, abnormal cameras); occasionally run global BA.

Intuition

Because the cost is highly non-convex, you "grow a crystal": start from a tiny well-conditioned seed (two views) and add only what is consistent with it, re-optimizing after each addition so errors never accumulate far from where the optimizer can fix them. The price: images are registered one at a time, so an early mistake propagates (a limit discussed in 5.5).

4.8 Building Rome in a Day, COLMAP SfM and COLMAP MVS *(slides 52–54)*

- **Building Rome in a Day** (Agarwal et al., ICCV 2009): first large-scale SfM from unstructured images (slide 52).
- **COLMAP** significantly improves accuracy and robustness compared to prior work (slide 53).
- COLMAP features a second **multi-view stereo** stage to obtain dense geometry (slide 54; Schönberger et al., ECCV 2016). A feed-forward network predicts dense depth without known cameras (5.5), whereas MVS needs SfM first.



Figure 162: Sparse point cloud and camera frusta of the Colosseum reconstructed from internet photos (Agarwal, Snavely, Simon, Seitz and Szeliski, "Building Rome in a Day", ICCV 2009).

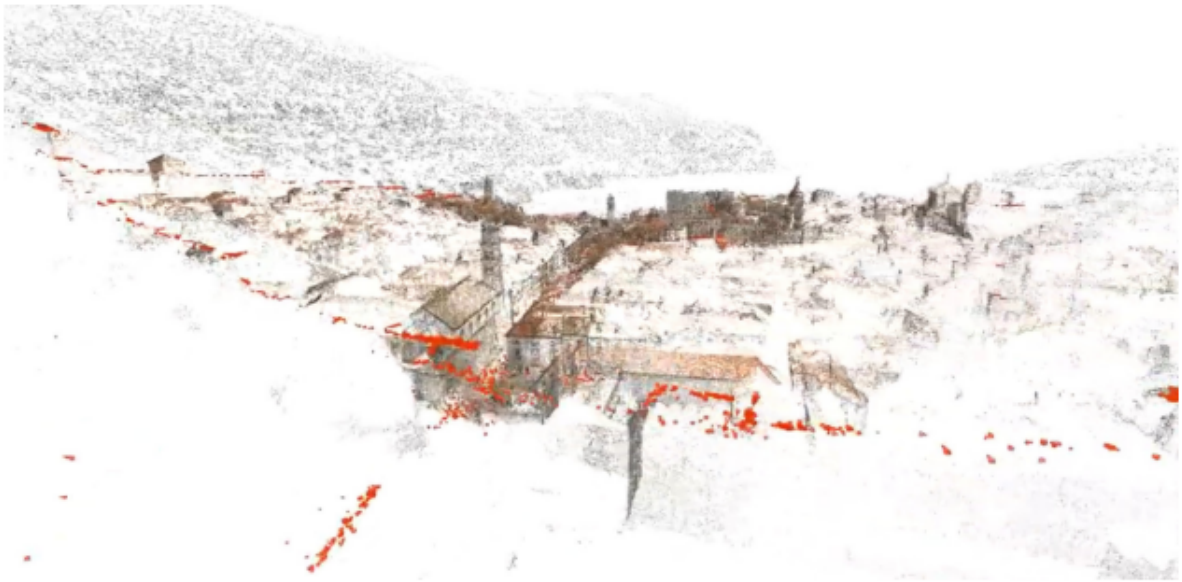


Figure 163: COLMAP sparse reconstruction of a city scene; red marks the camera positions (Schönberger and Frahm, CVPR 2016).



Figure 164: Dense COLMAP multi-view stereo reconstruction of a building (Schönberger, Zheng, Frahm and Pollefeys, ECCV 2016).

5 Feed-Forward Structure-from-Motion *(slides 55–68)*

5.1 Limits of Incremental Structure-from-Motion *(slides 56)*

Looking back at Fig. 159:

- every box is a separately engineered stage, with its own thresholds and failure cases;
- images are registered one at a time: a poor initial pair, outliers or drift propagate to every later image;
- runtime grows to hours for large photo collections; few views, low overlap or little texture leave too few correspondences;
- **Global SfM** estimates all cameras jointly without learning: rotation averaging, then camera positions, then one bundle adjustment. GLOMAP (Pan, Baráth, Pollefeys and Schönberger, ECCV 2024), a recent global pipeline, is on par with COLMAP in accuracy and orders of magnitude faster.

Intuition

Incremental SfM is like building a house brick by brick with one mason: reliable but slow, and a crooked first brick tilts the whole house. Global SfM hires many masons who work simultaneously and then reconcile. Feed-forward SfM, the topic of this section, replaces the masons by a single network that has seen millions of houses and sketches the whole building at once.

5.2 From Learned Components to Learned Reconstruction *(slides 57)*

Year	Method	What is learned
2018	SuperPoint	keypoints and their descriptors, from one network (5.2)
2020	SuperGlue	the matching of two keypoint sets, with a graph neural network
2021	LoFTR	dense matches between two images, with a transformer and no detector
2024	VGGSfM	a complete SfM pipeline, differentiable, trained end to end
2024	DUSt3R, MASt3R	pointmaps from two images (lecture 04); more views need a global alignment
2025	VGGT	cameras, depth, point maps and tracks for many views, in one forward pass

Table 8: Progressive replacement of the classical pipeline by learned components (slide 57).

Definition 5.1 (Pointmap)**.** For an $H \times W$ image $\boldsymbol{I}_a$, the pointmap $\hat{\boldsymbol{X}}^{a,1} \in \mathbb{R}^{H\times W\times 3}$ stores *one 3D point per pixel*, for both images $a \in \{1, 2\}$, expressed in the frame of camera 1.

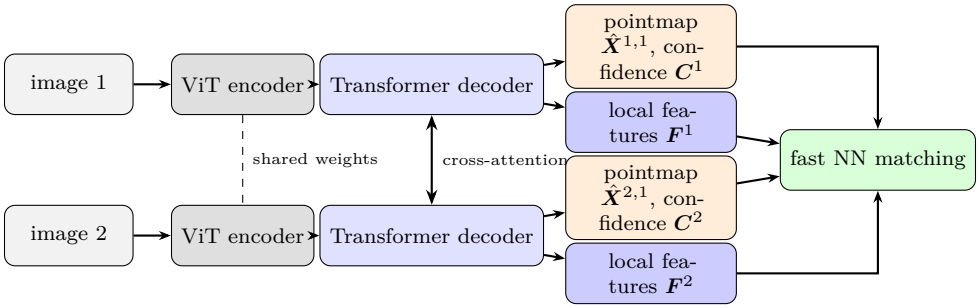


Figure 165: MASt3R-style two-view network (simplified from slide 57): tokens $\boldsymbol{H}^a$ from a ViT encoder are decoded with cross-attention into pointmaps with confidence (grey path) and local features (blue path), used for geometrical or feature-based matching.

Intuition

A pointmap is a depth map that already lives in a common world frame: once pixel (u, v) of image 2 and pixel (u', v') of image 1 carry (almost) the same 3D point, they are a correspondence *by construction*. DUSt3R and MASt3R use this to skip the detect-describe-match-verify chain, but for more than two images they still need a post-hoc global alignment; VGGT removes that last step.

5.3 Visual Geometry Grounded Transformer (VGGT) *(slides 58)*

- **Input:** one to hundreds of images of a scene; the first image defines the world frame.
- **Output,** in one forward pass of under a second: camera parameters, depth maps, point maps and point tracks for every image.
- Without post-processing, it often outperforms methods that refine their output by geometric optimization.



Figure 166: VGGT takes a set of images of a scene (left) and predicts, in one forward pass, camera poses, a dense point cloud, depth maps and point tracks (Wang et al., CVPR 2025).

5.4 Problem Definition *(slides 59)*

Definition 5.2 (VGGT as a function)**.** VGGT is one network with learned weights θ. It takes N images $\boldsymbol{I}_1, \ldots, \boldsymbol{I}_N$, each of $H \times W$ pixels, and predicts four outputs per image i (the hat marks a prediction):

$$f_\theta(\boldsymbol{I}_1, \ldots, \boldsymbol{I}_N) = (\hat{\boldsymbol{g}}_i,\ \hat{\boldsymbol{D}}_i,\ \hat{\boldsymbol{X}}_i,\ \boldsymbol{F}_i)_{i=1}^{N}.$$

- **Camera** $\hat{\boldsymbol{g}}_i \in \mathbb{R}^9$: rotation quaternion $\hat{\boldsymbol{q}}_i \in \mathbb{R}^4$, translation $\hat{\boldsymbol{t}}_i \in \mathbb{R}^3$ and field of view $\hat{\boldsymbol{\phi}}_i = (\hat{\phi}_x, \hat{\phi}_y)$. They give the pose $\boldsymbol{T}_i$ and the calibration $\boldsymbol{K}_i$ with focal lengths $f_x = W/(2\tan(\phi_x/2))$, $f_y = H/(2\tan(\phi_y/2))$ and the principal point at the image center.
- **Depth map** $\hat{\boldsymbol{D}}_i \in \mathbb{R}^{H\times W}$: the depth of every pixel.
- **Point map** $\hat{\boldsymbol{X}}_i \in \mathbb{R}^{H\times W\times 3}$: the 3D point of every pixel, in the frame of camera 1, which is the world frame.
- **Feature map** $\boldsymbol{F}_i \in \mathbb{R}^{H\times W\times C}$: C numbers per pixel, used to track points across the images.

The outputs are redundant, since the cameras follow from the point maps (PnP), yet predicting all of them improves accuracy.

Example 5.3 (From field of view to focal length)**.** For $W = 518$ pixels and a predicted horizontal field of view $\phi_x = 60°$: $f_x = 518/(2\tan 30°) = 518/1.1547 \approx 449$ pixels. With the principal point at $(259, 259)$ this fully specifies $\boldsymbol{K}$ (no calibration needed, in contrast to Section 5.1).

Intuition

Classical SfM needs $\boldsymbol{K}$ from calibration or self-calibration (and fails when it is wrong). VGGT simply *guesses* the field of view from the image content as a human would (a wide-angle shot looks different from a telephoto one). Predicting the same geometry in several forms (cameras, depth, point maps) gives the network several consistent "views" of one underlying 3D structure to learn from.

5.5 Architecture: Alternating Attention *(slides 60)*

- **Image tokens,** as $\boldsymbol{H}^a$ in DUSt3R: a vision transformer (DINO) cuts each image $\boldsymbol{I}_i$ into patches and turns each patch into a token, a feature vector.
- **Camera token:** a transformer returns one vector per input token. A 518×518 image gives $37 \times 37 = 1369$ patch tokens, hence 1369 outputs, one per patch, for the per-pixel maps. One extra learned token per image, the same for every image, yields one extra output: the camera head reads $\hat{\boldsymbol{g}}_i$ from it.
- **Alternating attention,** $L = 24$ times: *global attention* among the tokens of all images (as the cross-attention of DUSt3R but for N images), then *frame attention* among the tokens of one image.
- **Outputs:** the camera head turns each camera token into $\hat{\boldsymbol{g}}_i$; the DPT (Dense Prediction Transformer) head turns the image tokens into $\hat{\boldsymbol{D}}_i, \hat{\boldsymbol{X}}_i$ and $\boldsymbol{F}_i$.

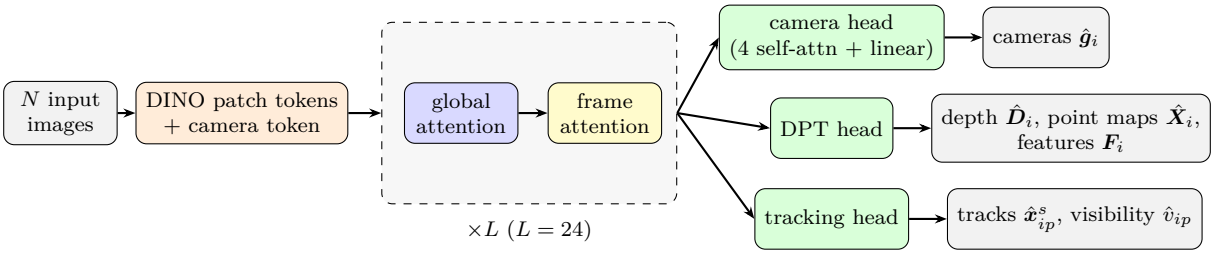


Figure 167: VGGT architecture (redrawn and simplified from slide 60).

Example 5.4 (Cost of global versus frame attention)**.** With $N = 10$ images and $T = 1369$ patch tokens per image there are $NT = 13{,}690$ tokens. Global attention scores $(NT)^2 \approx 1.87 \cdot 10^8$ token pairs per head and layer; frame attention scores $N \cdot T^2 \approx 1.87 \cdot 10^7$. Frame attention is $N = 10$ times cheaper.

Intuition

Global attention is where views "talk" to each other (it learns correspondence and relative pose implicitly); frame attention lets each image digest its own content cheaply. Alternating the two 24 times mimics what SfM does iteratively (match, then triangulate and refine), but inside one differentiable network with no hand-set thresholds. The camera token acts as a learned "summary slot" per image from which the camera is read out; no temporal order is assumed, so the images may be an unordered photo collection.

5.6 Prediction Heads and Tracking *(slides 61)*

- **Camera head:** four self-attention layers and a linear layer turn the camera token of image i into $\hat{\boldsymbol{g}}_i$.
- **DPT head,** the Dense Prediction Transformer decoder (Ranftl et al., ICCV 2021), turns tokens back into per-pixel maps: $\hat{\boldsymbol{D}}_i$, $\hat{\boldsymbol{X}}_i$, $\boldsymbol{F}_i$, and confidence maps $\boldsymbol{C}_i^D, \boldsymbol{C}_i^X \in \mathbb{R}^{H \times W}$ that say how much to trust each pixel of $\hat{\boldsymbol{D}}_i$ and $\hat{\boldsymbol{X}}_i$.
- **Tracking head,** as in CoTracker2 (a point tracker for videos): given the pixel of a point p in one image, it compares the feature of that pixel with the feature maps $\boldsymbol{F}_i$ of all images, and predicts where the point appears in each image, $\hat{\boldsymbol{x}}^s_{ip}$, and whether it is visible there, $\hat{v}_{ip} \in [0, 1]$.

5.7 Training Objective and Normalization *(slides 62)*

The four outputs are trained jointly, against ground truth written without a hat:

$$\mathcal{L} = \mathcal{L}_{\text{camera}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{pmap}} + \lambda \mathcal{L}_{\text{track}}, \qquad \lambda = 0.05,$$

$$\mathcal{L}_{\text{depth}} = \sum_{i=1}^{N} \|\boldsymbol{C}_i^D \odot (\hat{\boldsymbol{D}}_i - \boldsymbol{D}_i)\| + \|\boldsymbol{C}_i^D \odot (\nabla \hat{\boldsymbol{D}}_i - \nabla \boldsymbol{D}_i)\| - \alpha \log \boldsymbol{C}_i^D.$$

Here $\odot$ is the per-pixel product, ∇ the image gradient, and α the weight of the term $-\alpha \log \boldsymbol{C}_i^D$, which keeps the confidence from dropping to zero. $\mathcal{L}_{\text{camera}}$ is a Huber loss, squared for small errors and linear for large ones, between $\hat{\boldsymbol{g}}_i$ and $\boldsymbol{g}_i$; $\mathcal{L}_{\text{pmap}}$ is like $\mathcal{L}_{\text{depth}}$ for $\hat{\boldsymbol{X}}_i$ with $\boldsymbol{C}_i^X$; $\mathcal{L}_{\text{track}}$ is the distance of $\hat{\boldsymbol{x}}_{ip}^s$ to the true positions, plus a visibility term.

Intuition

The confidence $\boldsymbol{C}$ weights each pixel's error: unreliable pixels (sky, reflections, moving objects) can lower their confidence and so pay less. Without the $-\alpha \log \boldsymbol{C}$ term the network could cheat by predicting $\boldsymbol{C} = 0$ everywhere. The term charges a price for low confidence, so confidence drops only where errors would be large. The gradient term in the depth loss encourages sharp, correct edges.

Proposition 5.5 (Why a normalization is required)**.** *Let $(\boldsymbol{T}_i, \boldsymbol{X})$ be a ground-truth reconstruction. Any similarity transform of the whole scene, applied consistently to cameras and points, produces the same images (Prop. 3.3). A regression target that is not fixed by a convention is therefore not a function of the images.*

Proof. If two targets differ by a similarity, they map to identical inputs, so the network would be asked to output two different answers for the same input; the loss-minimizing prediction would blur between them. □

The images do not fix the global frame and scale (5.3). The ground truth is therefore expressed in the frame of camera 1 and divided by s, the mean distance of its 3D points to the origin; the network learns to predict in this normalized frame.

Intuition

It is the neural-network version of the gauge fixing of bundle adjustment: BA fixes the first pose and one distance; VGGT fixes the first pose and the average point distance ("scene scale = 1"). Every training scene is thus expressed in the same canonical units.

Training data and cost. 17 datasets; 64 A100 GPUs for nine days; 2 to 24 images per scene, at most 518 pixels per side (the slide quotes around 1.5M EUR for this compute).

5.8 Results: Camera Pose Estimation *(slides 63)*

10 random frames per scene; time on one H100 GPU. AUC@30: area under the accuracy curve of the relative rotation and translation errors, up to 30°. RealEstate10K: walkthrough videos of houses, used for training by none of the methods. CO3Dv2: videos around everyday objects. PixSfM: COLMAP with feature-metric refinement. DUSt3R and MASt3R with global alignment.

Method	RealEstate10K ↑	CO3Dv2 ↑	Time
COLMAP + SuperPoint + SuperGlue	45.2	25.3	≈ 15 s
PixSfM	49.4	30.1	> 20 s
DUSt3R	67.7	76.7	≈ 7 s
MASt3R	76.4	81.8	≈ 9 s
VGGSfM v2	78.9	83.4	≈ 10 s
VGGT, feed-forward	85.3	88.2	≈ 0.2 s
VGGT + bundle adjustment	**93.5**	**91.8**	≈ 1.8 s

Table 9: Camera pose estimation, AUC@30 (slide 63).

Intuition

Ten random frames is a sparse, wide-baseline setting, the weakest regime of classical SfM (few overlaps, little parallax per pair). That is where a learned prior helps most: COLMAP+SuperPoint+SuperGlue gets 45.2 on RealEstate10K, VGGT 85.3 in a fifth of a second. On dense captures with thousands of images, incremental SfM remains the reference (5.4). The speed difference (0.2 s versus ∼ 15 s) is a factor of about 75.

5.9 Results: Dense Geometry *(slides 64)*

DTU (lecture 04), Chamfer distance in mm (lower is better); GeoMVSNet is a learned MVS method given the true cameras.

DTU			ETH3D (laser-scanned GT)	
Method	GT cameras	Overall↓	Method	Overall↓
GeoMVSNet	yes	0.295	DUSt3R	1.005
MASt3R	yes	0.374	MASt3R	0.826
DUSt3R	no	1.741	VGGT, point head	0.709
VGGT	no	0.382	VGGT, depth + camera	**0.677**

Table 10: Dense geometry (slide 64).

- Depth unprojected with the predicted cameras beats the point head: the decomposition into depth and camera pays off.
- VGGT obtains dense depth without known cameras, whereas COLMAP's MVS needs SfM first (5.4); on DTU, VGGT (no GT cameras) is within 0.01 mm of MASt3R, which *uses* the true cameras.

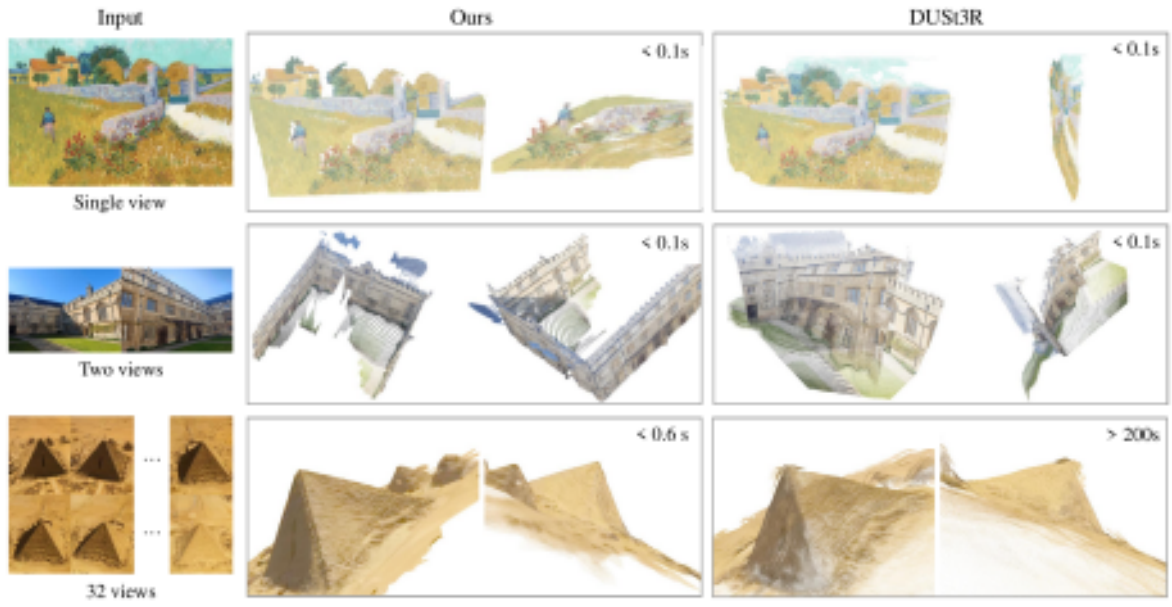


Figure 168: Input, VGGT ("Ours") and DUSt3R: one image, two images without overlap, 32 images.

5.10 Results: Matching and Tracking *(slides 65)*

ScanNet-1500: 1500 indoor image pairs; pose from the matches through the essential matrix (5.3); AUC of the pose error.

Method	AUC@5°	AUC@10°	AUC@20°
SuperGlue	16.2	33.8	51.8
LoFTR	22.1	40.8	57.6
RoMa	31.8	53.4	70.9
VGGT	**33.9**	**55.2**	**73.4**

Table 11: Two-view matching, pose from the essential matrix (slide 65).

RoMa is a dense learned matcher and the strongest baseline. The tracking head is *not trained* for two-view matching. The features also serve other tasks: a dynamic point tracker improves when given them, and novel view synthesis works without the input cameras.

Intuition

It is a surprising result: a network trained for 3D reconstruction produces features that outperform dedicated two-view matchers, because "being the same 3D point" is a stronger training signal than "looking alike". Note how the pipeline of Section 5.3 (matches $\rightarrow \mathbb{E} \rightarrow$ pose) is used here as an *evaluation protocol*, which shows the classical geometry remains the common language.



Figure 169: Point tracking: VGGT tracks (top) and a dynamic point tracker (CoTracker) given VGGT features (bottom) (Wang et al., CVPR 2025).

5.11 Bundle Adjustment from VGGT Predictions *(slides 66)*

Use the same objective as before,

$$\mathcal{T}^*, \mathcal{X}_w^* = \arg\min_{\mathcal{T}, \mathcal{X}_w} \sum_{i=1}^{N} \sum_{p=1}^{P} w_{ip} \|\boldsymbol{x}_{ip}^s - \pi_i(\boldsymbol{x}_p^w)\|_2^2,$$

with the following ingredients, all predicted:

- observations $\boldsymbol{x}_{ip}^s$ and weights w_{ip}: tracks and visibilities from the tracking head;
- initial poses $\boldsymbol{T}_i$: the camera head, $\hat{\boldsymbol{g}}_i$, which also gives the calibrations $\boldsymbol{K}_i$;
- initial points $\mathcal{X}_w$: the predicted depth maps $\hat{\boldsymbol{D}}_i$, unprojected with the predicted cameras.

Initialization, image registration and triangulation (5.4) are no longer needed; with bundle adjustment the reconstruction takes about 2 s. AUC@30 on RealEstate10K rises from 85.3 to 93.5 with bundle adjustment.

Intuition

The two worlds complement each other: the network gives a very good starting point in the right basin of attraction (solving the non-convexity problem of Section 5.4), and BA then enforces what the network cannot guarantee, namely that the output is *exactly* geometrically consistent with the observed tracks. This is the same "closed-form initialization + non-linear refinement" pattern as in Zhang's calibration.

5.12 Limitations *(slides 67)*

Stated in the paper:

- no fisheye or panoramic images;
- accuracy drops under extreme rotations between the input images;
- minor non-rigid motion is handled, substantial non-rigid deformation fails;
- runtime and memory grow with the number of frames: 200 frames take 8.75 s and 40.63 GB of GPU memory (one H100, images of 336×518 pixels).

Beyond the paper:

- training needs large 3D-labeled datasets and large compute: 17 datasets, 64 A100 GPUs for nine days;
- no geometric guarantee: the output is a prediction, not the optimum of an objective;
- bundle adjustment still improves the result: 85.3 to 93.5 AUC@30 on RealEstate10K.

5.13 Comparison of Incremental and Feed-Forward SfM *(slides 68)*

	Incremental SfM	VGGT
Camera model	perspective, calibrated or self-calibrated	perspective, principal point at the center
Views	one at a time	all jointly
Assumptions	overlap and texture, a good initial pair	scenes like the training data, mostly rigid
Solution	RANSAC and nonlinear least squares	one forward pass of a transformer
Runtime	minutes to hours	under a second for tens of frames
Scale	thousands of images	hundreds of frames, bounded by GPU memory
Output	sparse points and cameras; dense with MVS	cameras, depth, point maps, tracks
Failure modes	low overlap, little texture, drift	fisheye images, extreme rotations, non-rigid motion
Guarantees	local optimum of the reprojection error	none

Table 12: Incremental SfM (Schönberger and Frahm, CVPR 2016) versus VGGT (Wang et al., CVPR 2025), slide 68.

Intuition

The two are complementary rather than competing: use the feed-forward network when views are few, wide-baseline or the time budget is tight (robots, interactive tools), and use classical incremental or global SfM when you have thousands of images and need a certified, repeatable optimum. The hybrid VGGT + BA gives the best accuracy.

Summary *(slides 69)*

Comparison of the methods covered

Method	Input → output	Core idea	Main limitation
Zhang calibration (5.1)	plane images → $\boldsymbol{K}$, poses	homography constraints on $\boldsymbol{B}$, then non-linear LS	needs a known target
SIFT (5.2)	image → 128D features	DoG extrema in scale space, gradient histograms	hand-crafted; slow
SuperPoint (5.2)	image → keypoints + 256D descriptors	self-supervised, homographic adaptation	fails for rotations not in training data
8-point / 5-point (5.3)	≥ 8 / 5 matches → $\mathbb{E}$	linear epipolar constraint (+ rank-2 projection)	scale unobservable; 4-fold ambiguity
Triangulation (5.3)	2 views → 3D point	DLT or reprojection error	ill-conditioned for small parallax
Bundle adjustment (5.4)	observations → poses, points	Gauss–Newton on SE(3) + Schur complement	needs good initialization
Incremental SfM / COLMAP (5.4)	image set → sparse model	seed from two views, register, triangulate, BA	slow; drift; many thresholds
VGGT (5.5)	N images → cameras, depth, points, tracks	alternating attention, trained on 17 datasets	no guarantees; GPU memory
VGGT + BA (5.5)	N images → refined model	learned initialization + BA	extra ~ 1.6 s

Table 13: Summary of the methods of Lecture 05.

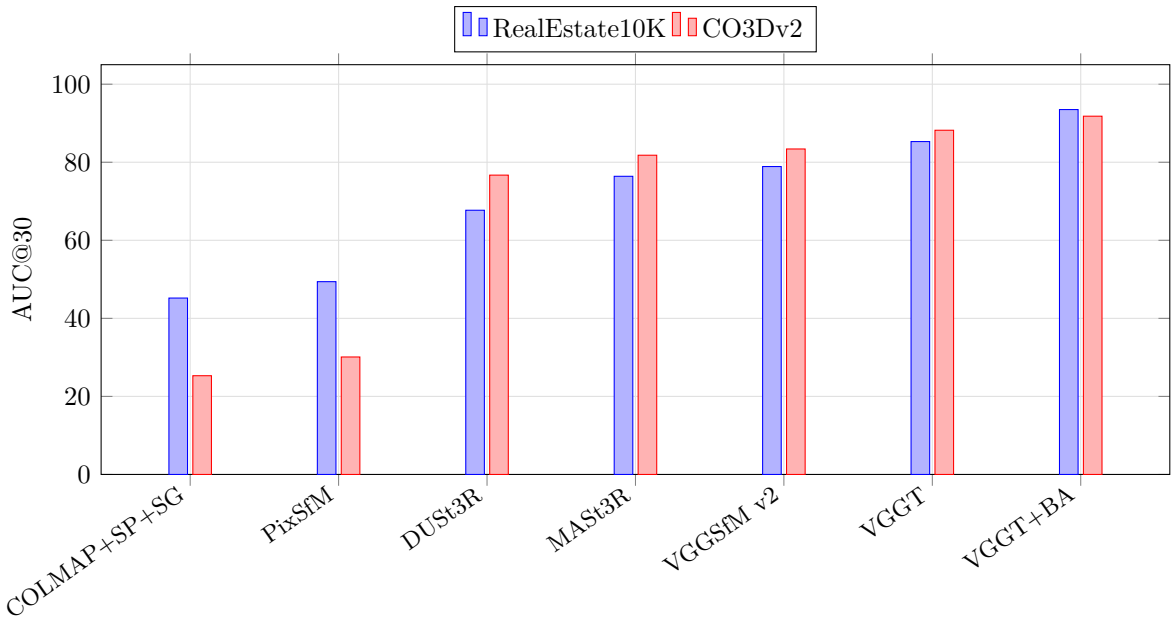


Figure 170: Camera pose accuracy of classical, learned two-view, and feed-forward methods (data of the table on slide 63; ten random frames per scene). The jump from the classical pipeline to learned methods is largest in the sparse regime, and bundle adjustment on top of VGGT adds the final gain.

Key take-aways

1. **Geometry supplies the constraints:** the epipolar constraint, triangulation and the reprojection error are the common language of every method in this lecture, classical or

learned (even VGGT is evaluated and refined with them).

2. **Classical SfM = robust estimation + non-linear least squares.** Sparsity and the Schur complement keep bundle adjustment tractable (a factor $\sim 10^8$ in our example); its hard part is the initialization, which is why reconstruction is built incrementally from a good seed.

3. **Scale and gauge are never observable from images alone:** the essential matrix is defined up to scale and the four-fold decomposition ambiguity is resolved by cheirality; BA fixes one pose and one distance; VGGT normalizes its training targets.

4. **VGGT learns the whole mapping** from images to cameras, depth, point maps and tracks in one forward pass of a transformer with alternating global and frame attention.

5. **The most accurate poses come from a learned initialization refined by bundle adjustment** (AUC@30: 85.3 $\rightarrow$ 93.5 on RealEstate10K).

Acknowledgments (slide 70): the slides are adapted from the Computer Vision course at the University of Tübingen (Prof. Andreas Geiger); Section 5.5 figures and results are from Wang et al., "VGGT: Visual Geometry Grounded Transformer", CVPR 2025.

Part V

Lecture 6

About these notes

The notes follow the slide deck in order; every `\section` and `\subsection` carries the title of the corresponding slide(s) and its slide range. Formulas on the slides are embedded as pictures, so the formal content (definitions, lemmas, theorems, proofs) was reconstructed from the references quoted in the slide footers (Thrun–Burgard–Fox, Stachniss, Dellaert–Kaess, Mur-Artal et al., Lovegrove et al. and others) and from lectures 01 and 05 (Lie groups, bundle adjustment). Dashed empty boxes mark photographs or screenshots that must be copied from the indicated slide; all other figures are redrawn in TikZ/pgfplots.

Notation. World-to-camera poses $\mathbf{T}_i \in \mathrm{SE}(3)$ (as in lecture 05); 3D points $\mathbf{X}_j$ (or $\mathbf{x}_j^w$); pixel observations $\mathbf{u}_{ij}$; $\pi(\cdot)$ is the pinhole projection including intrinsics; $\|\mathbf{v}\|_\Omega^2 = \mathbf{v}^\top \Omega\, \mathbf{v}$ is the squared Mahalanobis norm. The letter $\mathbf{H}$ is the Gauss–Newton matrix of Section 6.3, while the EKF measurement Jacobian is called $\mathbf{C}_t$.

Agenda and Learning Objectives *(slides 1–2)*

Agenda: 6.1 The SLAM problem; 6.2 Filtering; 6.3 Graph-based SLAM; 6.4 The visual SLAM pipeline; 6.5 Research directions.

After the lecture you should be able to:

1. formulate SLAM as joint estimation of trajectory and map, and derive the Kalman filter and EKF-SLAM together with their cost;
2. formulate graph-based SLAM as least squares over a factor graph, solve it with Gauss–Newton, and relate its sparsity to marginalization;
3. describe the threads of a keyframe-based visual SLAM system: tracking, local mapping, loop detection and loop correction.

1 The SLAM Problem *(slides 3–6)*

1.1 The SLAM Problem *(slide 3)*

Definition 1.1 (SLAM)**.** *Simultaneous localization and mapping* computes, at the same time, the camera poses (the trajectory) $\boldsymbol{x}_{1:t} = \{\boldsymbol{x}_1, \dots, \boldsymbol{x}_t\}$ and the map of the environment $\mathbf{m}$, given the measurements $\boldsymbol{z}_{1:t}$ (and, if available, controls $\mathbf{u}_{1:t}$), i.e. the posterior

$$p(\boldsymbol{x}_{1:t}, \mathbf{m} \mid \boldsymbol{z}_{1:t}, \mathbf{u}_{1:t}).$$

Localization estimates the poses given the map; *mapping* builds the map given the poses.

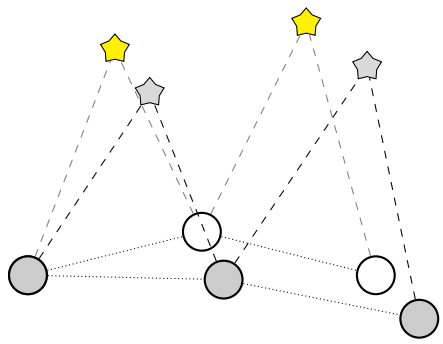
SLAM is a *chicken-or-egg problem*: a map is needed to localize, and a pose estimate is needed to build a map. Without any map, chaining relative motion estimates (*dead reckoning*) accumulates error without bound.

Proposition 1.2 (Dead-reckoning drift)**.** *If each relative motion estimate carries independent zero-mean error of variance σ^2 per axis, the position variance after t steps is $t\sigma^2$, so the standard deviation grows as $\sigma\sqrt{t}$ and is unbounded.*

Proof. The estimated position is $\sum_{k=1}^{t}(\Delta_k + \varepsilon_k)$ with independent ε_k; variances of independent terms add. □

Intuition

With $\sigma = 1\,\text{cm}$ per step, after 1000 steps the 1σ error is $\sqrt{1000} \approx 32\,\text{cm}$, even though no single step was bad. Orientation errors are worse: a small heading error bends the whole subsequent path. A map that is *re-observed* gives absolute references and stops this growth.



star: landmark (yellow true, gray estimated)

circle: pose (white true, gray estimated)

dashed: observations

Figure 171: Redrawn from slide 3: true and estimated poses/landmarks and the observations linking them. Estimated poses drift away from the truth while the landmarks keep them mutually consistent.

1.2 Cameras for SLAM *(slide 4)*

Cameras are compact, accurate, non-invasive, well understood, and today cheap and ubiquitous. Images carry *appearance*: two rooms that look identical to a 2D laser scanner can be told apart by a camera. Applications range from augmented reality and vision-based navigation to self-driving cars, mobile devices, inspection drones and VR headsets. The cost: a camera senses geometry only indirectly, through photometric effects, and a single camera measures *no depth*.

Proposition 1.3 (Scale ambiguity of a monocular camera)**.** *Let a scene* $\{\mathbf{X}_j\}$ *and camera poses* $\{(\mathbf{R}_i, \mathbf{t}_i)\}$ *produce pixels* $\mathbf{u}_{ij} = \pi(\mathbf{R}_i\mathbf{X}_j + \mathbf{t}_i)$*. For any* $s > 0$*, the scene* $\{s\mathbf{X}_j\}$ *with translations* $\{s\mathbf{t}_i\}$ *produces exactly the same pixels.*

Proof. $\mathbf{R}_i(s\mathbf{X}_j) + s\mathbf{t}_i = s(\mathbf{R}_i\mathbf{X}_j + \mathbf{t}_i)$, and the perspective projection satisfies $\pi(s\mathbf{p}) = \pi(\mathbf{p})$ because it divides by the depth. □

Intuition

A photograph of a toy house and of a real house can be identical. A single camera therefore recovers the world only *up to a similarity* (rotation, translation *and* scale: 7 degrees of freedom), which is why monocular maps also drift in scale (Section 6.4).

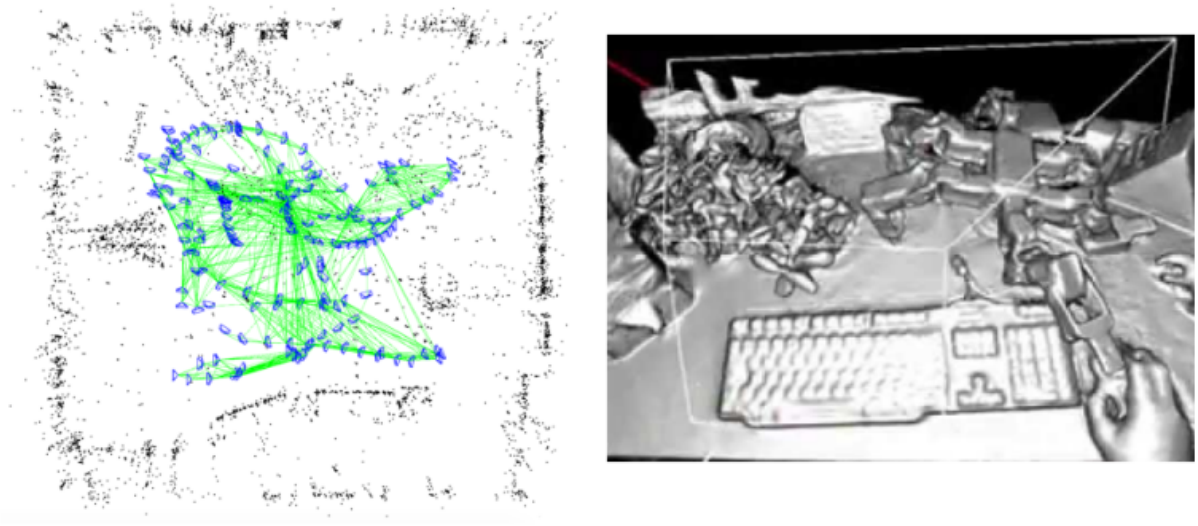


Figure 172: Maps built from one camera: sparse features of a room (left) and a dense desktop (right).

1.3 SLAM and Structure-from-Motion *(slide 5)*

The unknowns and the error are those of lecture 05: camera poses $\mathbf{T}_i$ (world to camera), 3D points $\mathbf{X}_j$, and the reprojection error of every observation. Both problems stem from the same maximum a posteriori formulation; the *setting* differs.

Structure-from-motion (lecture 05)	**SLAM**
Unordered set of images, all available from the start	Frames arrive one at a time; solved incrementally as measurements arrive
Solved offline, analyzing all images at once	Pose and map needed at every frame, in real time
No notion of returning to a place	Returning to a visited place closes a loop and corrects long-term drift

Intuition

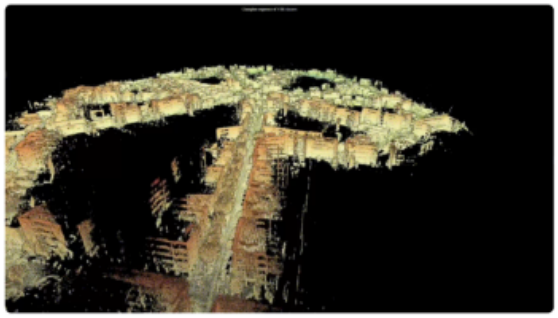
SfM is like developing all holiday photos and then assembling the 3D model at home; SLAM is like navigating a city while drawing the map as you walk. Same maths, very different time budget.

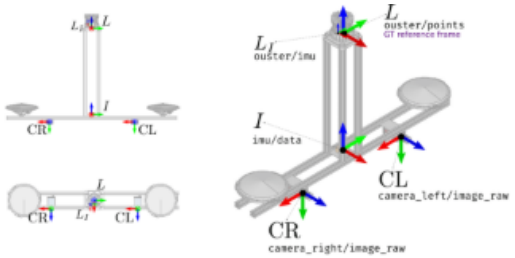
1.4 Running Example: VBR Campus *(slide 6)*

VBR (Vision Benchmark in Rome) was recorded hand-held and by car with a global-shutter stereo camera of wide baseline, a 3D LiDAR, an IMU and an RTK-GPS. Its ground truth refines the RTK-GPS with LiDAR bundle adjustment, accurate to ± 3 cm over 1.5 km; its six datasets cover over 40 km. The running example of the lecture is a sequence of around 11 km.

VBR (Vision Benchmark in Rome) was recorded hand-held and by car with a global-shutter stereo camera of wide baseline, a 3D LiDAR, an IMU and an RTK-GPS. Its ground truth refines the RTK-GPS with LiDAR bundle adjustment, accurate to ±3 cm over 1.5 km; its six datasets cover over 40 km.

- Running example - around 11 km





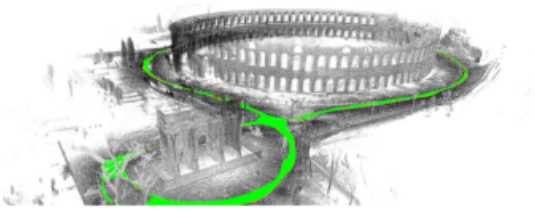


Figure 173: VBR: sensor-rig reference frames (camera left/right CL, CR, LiDAR L, IMU I) and two point-cloud/map renderings of the Rome sequences.

2 Filtering: From the Bayes Filter to EKF-SLAM *(slides 7–13)*

2.1 SLAM as a Graphical Model *(slide 8)*

The camera poses and the map are unknown; the measurements are observed. Without odometry, as with a single camera, a motion model alone links consecutive poses.

Definition 2.1 (Full and online SLAM)**.** *Full SLAM* estimates the whole trajectory and the map, $p(\boldsymbol{x}_{1:t}, \mathbf{m} \mid \boldsymbol{z}_{1:t}, \mathbf{u}_{1:t})$. *Online SLAM* keeps only the current pose,

$$p(\boldsymbol{x}_t, \mathbf{m} \mid \boldsymbol{z}_{1:t}, \mathbf{u}_{1:t}) = \int \cdots \int p(\boldsymbol{x}_{1:t}, \mathbf{m} \mid \boldsymbol{z}_{1:t}, \mathbf{u}_{1:t})\, d\boldsymbol{x}_1 \cdots d\boldsymbol{x}_{t-1},$$

i.e. it *marginalizes* the past poses, and the integrals are solved recursively one step at a time.

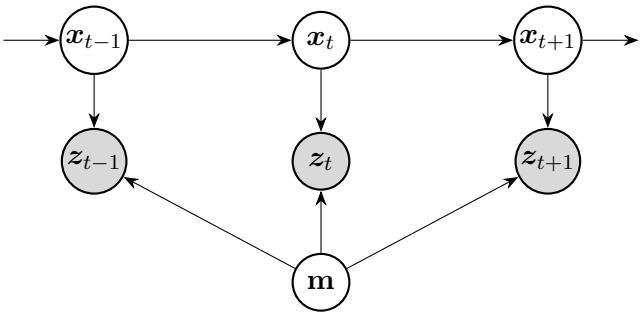


Figure 174: Graphical model of SLAM (slide 8): poses form a Markov chain, the map is static, and each (gray, observed) measurement depends on the current pose and the map. Controls are omitted, as in the monocular case.

Intuition

Full SLAM keeps the entire path (a growing problem, solved later by graphs); online SLAM forgets the past path but remembers what it taught us, packed inside a single Gaussian over the current pose and the map. Marginalizing is "summing over everything I no longer care about".

2.2 The Bayes Filter *(slide 9)*

Let $\mathbf{s}_t = (\boldsymbol{x}_t, \mathbf{m})$ be the state: camera pose and map.

Theorem 2.2 (Recursive Bayes filter)**.** *Let* $\mathrm{bel}(\mathbf{s}_t) = p(\mathbf{s}_t \mid \boldsymbol{z}_{1:t}, \mathbf{u}_{1:t})$*. Under the Markov assumptions, one frame at a time:*

$$\overline{\mathrm{bel}}(\mathbf{s}_t) = \int p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{u}_t)\, \mathrm{bel}(\mathbf{s}_{t-1})\, d\mathbf{s}_{t-1} \qquad \textit{(prediction)}$$

$$\mathrm{bel}(\mathbf{s}_t) = \eta\, p(\boldsymbol{z}_t \mid \mathbf{s}_t)\, \overline{\mathrm{bel}}(\mathbf{s}_t) \qquad \textit{(update)}$$

where $p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{u}_t)$ *is the* motion model*,* $p(\boldsymbol{z}_t \mid \mathbf{s}_t)$ *the* measurement model *and* η *a normalizer.*

Proof. By Bayes' rule, $\mathrm{bel}(\mathbf{s}_t) = \eta\, p(\boldsymbol{z}_t \mid \mathbf{s}_t, \boldsymbol{z}_{1:t-1}, \mathbf{u}_{1:t})\, p(\mathbf{s}_t \mid \boldsymbol{z}_{1:t-1}, \mathbf{u}_{1:t})$. Given the state, the current measurement is independent of the past, so the first factor is $p(\boldsymbol{z}_t \mid \mathbf{s}_t)$. For the second factor, the law of total probability gives $\int p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \boldsymbol{z}_{1:t-1}, \mathbf{u}_{1:t})\, p(\mathbf{s}_{t-1} \mid \boldsymbol{z}_{1:t-1}, \mathbf{u}_{1:t})\, d\mathbf{s}_{t-1}$; the state is Markov, so $p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \dots) = p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{u}_t)$, and $\mathbf{u}_t$ carries no information on $\mathbf{s}_{t-1}$, so the last factor is $\mathrm{bel}(\mathbf{s}_{t-1})$. □

Intuition

Two beats, forever: *predict* (move the belief with the motion model, which *widens* it, since motion adds uncertainty) and *update* (multiply by how well each hypothesis explains the new image, which *narrows* it). The one-dimensional example below follows slide 9: a camera on a line.

Example 2.3 (Camera on a line)**.** Prior $\mathcal{N}(0, 1)$. Motion model: move $+1$ with noise variance 1, so the prediction is $\mathcal{N}(1, 1+1) = \mathcal{N}(1, 2)$. Measurement $z = 2$ with noise variance 1. The posterior is the normalized product of the prediction and the measurement likelihood: $\sigma^2 = (2^{-1} + 1^{-1})^{-1} = 2/3$ and $\mu = \frac{2}{3}\left(\frac{1}{2} \cdot 1 + \frac{1}{1} \cdot 2\right) = 5/3$.

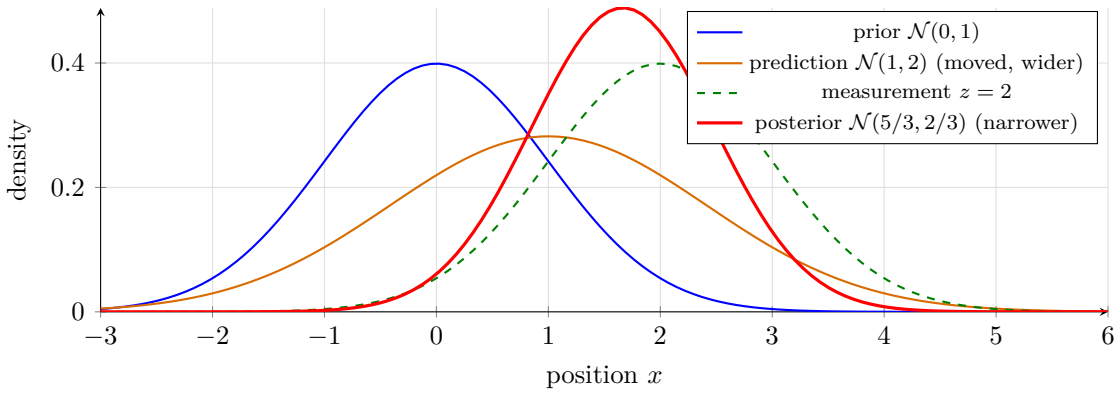


Figure 175: The Bayes filter in one dimension (slide 9): prediction shifts and widens the belief, the measurement narrows it.

2.3 Kalman Filter and Extended Kalman Filter *(slide 10)*

With Gaussian noise the belief stays Gaussian, $\mathrm{bel}(\mathbf{s}_t) = \mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)$. Let the models be $\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{u}_t) + \boldsymbol{\varepsilon}_t$, $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(0, \mathbf{R}_t)$ and $\boldsymbol{z}_t = h(\mathbf{s}_t) + \boldsymbol{\delta}_t$, $\boldsymbol{\delta}_t \sim \mathcal{N}(0, \mathbf{Q}_t)$. The *extended Kalman filter* (EKF) linearizes both around the current estimate, with Jacobians $\mathbf{G}_t = \partial g / \partial \mathbf{s}\,|_{\boldsymbol{\mu}_{t-1}}$ and $\mathbf{C}_t = \partial h / \partial \mathbf{s}\,|_{\bar{\boldsymbol{\mu}}_t}$.

Algorithm 2.1 (Extended Kalman filter step)**. Predict:** $\bar{\boldsymbol{\mu}}_t = g(\boldsymbol{\mu}_{t-1}, \mathbf{u}_t)$, $\quad \bar{\Sigma}_t = \mathbf{G}_t \Sigma_{t-1} \mathbf{G}_t^\top + \mathbf{R}_t$.
Update: $\mathbf{S}_t = \mathbf{C}_t \bar{\Sigma}_t \mathbf{C}_t^\top + \mathbf{Q}_t$ (innovation covariance), $\quad \mathbf{K}_t = \bar{\Sigma}_t \mathbf{C}_t^\top \mathbf{S}_t^{-1}$ (Kalman gain),
$\boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + \mathbf{K}_t\big(\boldsymbol{z}_t - h(\bar{\boldsymbol{\mu}}_t)\big)$, $\quad \Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t)\bar{\Sigma}_t$.

Theorem 2.4 (Kalman filter (linear case))**.** *If* $g(\mathbf{s},\mathbf{u}) = \mathbf{A}\mathbf{s} + \mathbf{B}\mathbf{u}$ *and* $h(\mathbf{s}) = \mathbf{C}\mathbf{s}$*, the Jacobians are* $\mathbf{G}_t = \mathbf{A}$ *and* $\mathbf{C}_t = \mathbf{C}$*, and the EKF equations give the* exact *posterior. This is Kalman's recursive filter for discrete-time linear systems.*

Proof. The prediction is a linear map of a Gaussian plus Gaussian noise, hence Gaussian with the stated mean and covariance. For the update, $(\mathbf{s}_t, \boldsymbol{z}_t)$ is jointly Gaussian with $\operatorname{cov}(\boldsymbol{z}_t) = \mathbf{C}\bar{\Sigma}\mathbf{C}^\top + \mathbf{Q} = \mathbf{S}$ and $\operatorname{cov}(\mathbf{s}_t, \boldsymbol{z}_t) = \bar{\Sigma}\mathbf{C}^\top$. Conditioning a joint Gaussian gives $\mathbb{E}[\mathbf{s} \mid \boldsymbol{z}] = \bar{\boldsymbol{\mu}} + \bar{\Sigma}\mathbf{C}^\top\mathbf{S}^{-1}(\boldsymbol{z} - \mathbf{C}\bar{\boldsymbol{\mu}})$ and $\operatorname{cov} = \bar{\Sigma} - \bar{\Sigma}\mathbf{C}^\top\mathbf{S}^{-1}\mathbf{C}\bar{\Sigma} = (\mathbf{I} - \mathbf{K}\mathbf{C})\bar{\Sigma}$. □

Intuition

The gain $\mathbf{K}$ is a trust dial between model and sensor. In 1D with $\bar{\sigma}^2 = 2$ and measurement noise $q = 1$: $K = \bar{\sigma}^2/(\bar{\sigma}^2 + q) = 2/3$, so the posterior mean moves two thirds of the way from the prediction (1) toward the measurement (2), giving 5/3, exactly Example 2.3; the variance shrinks to $(1-K)\bar{\sigma}^2 = 2/3$. For nonlinear g, h the EKF is only an approximation: it converges in the linear case but can *diverge* if nonlinearities are large, because Jacobians are evaluated once, at the current estimate.

2.4 The EKF-SLAM State *(slide 11)*

EKF-SLAM estimates the camera and the positions of all the points with *one* Gaussian: a single mean and a single joint covariance,

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{x}_c \\ \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_N \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} \Sigma_{cc} & \Sigma_{cm} \\ \Sigma_{mc} & \Sigma_{mm} \end{pmatrix}.$$

In MonoSLAM the camera block $\boldsymbol{x}_c$ has 13 numbers (position 3, orientation quaternion 4, linear velocity 3, angular velocity 3); each point adds 3, so the state has $13 + 3N$ entries and Σ has $(13 + 3N)^2$. The off-diagonal blocks are the *correlations* between the camera and the points and between the points. The filter cycle is: state prediction (the motion model moves the camera block only), measurement prediction, measurement, data association, update (all blocks change). The camera starts at the origin of its own frame; a new point enters the state when first observed.

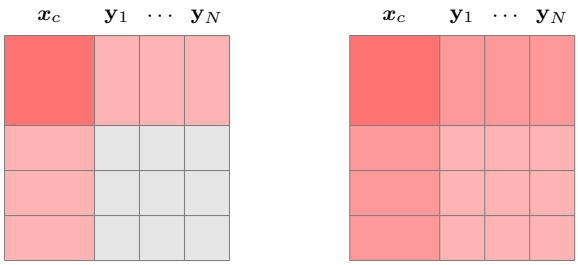


Figure 176: Which blocks of the EKF-SLAM covariance change (red). Left, prediction: only the camera block and its correlations, Σ_{mm} is untouched. Right, update: every block changes.

Proposition 2.5 (Cost of EKF-SLAM)**.** *The prediction touches only* Σ_{cc} *and* Σ_{cm} *(the Jacobian of* g *is* $\operatorname{diag}(\mathbf{G}_c, \mathbf{I})$*), but the update has a dense gain* $\mathbf{K}$*, so it modifies the* whole *covariance:* $O(N^2)$ *time per step and* $O(N^2)$ *memory for* N *points.*

Proof. Since g moves only the camera, $\mathbf{G}_t = \operatorname{diag}(\mathbf{G}_c, \mathbf{I})$, giving $\bar{\Sigma}_{cc} = \mathbf{G}_c\Sigma_{cc}\mathbf{G}_c^\top + \mathbf{R}$, $\bar{\Sigma}_{cm} = \mathbf{G}_c\Sigma_{cm}$ and $\bar{\Sigma}_{mm} = \Sigma_{mm}$. In the update, $\Sigma \leftarrow \Sigma - \mathbf{K}\mathbf{S}\mathbf{K}^\top$ with $\mathbf{K} = \bar{\Sigma}\mathbf{C}^\top\mathbf{S}^{-1}$; because all points are correlated with the camera, $\mathbf{K}$ has no zero blocks and the rank-update is dense. □

Intuition

Numbers: with $N = 100$ points the state has 313 entries and Σ about 98,000 numbers: trivial. With $N = 10{,}000$ points, $30{,}013^2 \approx 9 \times 10^8$ entries, about 7.2 GB in double precision, *and* touched at every frame. Correlations are a feature (observing one point improves the others) but they are what makes the filter expensive.

2.5 Example: MonoSLAM *(slide 12)*

MonoSLAM is EKF-SLAM with a single camera: it recovers the 3D trajectory of a camera moving rapidly through an unknown scene, at 30 Hz on a standard PC, and drives live augmented reality from a hand-held camera.

- **Motion model:** constant linear and angular velocity, with unknown accelerations as Gaussian noise; a hand-held camera has no control input. With $\boldsymbol{x}_c = (\mathbf{r}, \mathbf{q}, \mathbf{v}, \boldsymbol{\omega})$: $\mathbf{r} \leftarrow \mathbf{r} + (\mathbf{v} + \mathbf{V})\Delta t$, $\mathbf{q} \leftarrow \mathbf{q} \otimes \mathbf{q}\big((\boldsymbol{\omega} + \boldsymbol{\Omega})\Delta t\big)$, $\mathbf{v} \leftarrow \mathbf{v} + \mathbf{V}$, $\boldsymbol{\omega} \leftarrow \boldsymbol{\omega} + \boldsymbol{\Omega}$, with $\mathbf{V}, \boldsymbol{\Omega}$ Gaussian impulses.
- **Measurement model:** pinhole projection of each point, with radial distortion.
- **Active search:** the innovation covariance $\mathbf{S}$ defines an ellipse around each predicted feature, and only these ellipses are searched, for at most 10 or 12 features per frame.
- About 100 features at 30 Hz take 19 ms of the 33 ms of a frame.

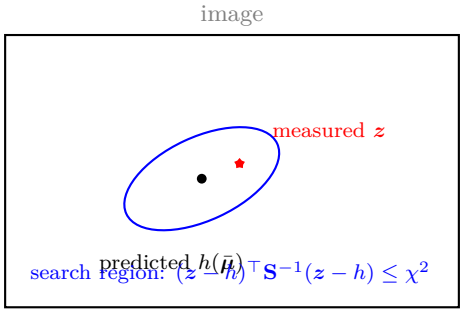


Figure 177: Active search (slide 12): the EKF predicts where a feature should appear and how uncertain that prediction is; only the ellipse is searched.

Intuition

Instead of scanning the whole image for each of 100 features, the filter says "it must be about *here*, give or take this much". Searching a few small ellipses is why a 1990s-style EKF reaches 30 Hz; and the ellipse shrinks as the map is better known.



Search ellipses in a wide-angle image (top); predicted camera paths of the motion model (bottom)

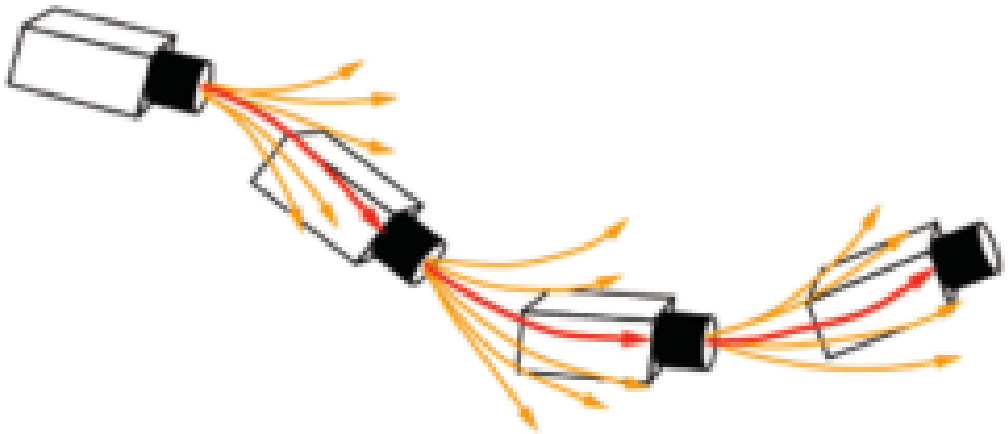


Figure 178: MonoSLAM: search ellipses in a wide-angle image (top) and predicted camera paths of the motion model (bottom).

2.6 Limits of Filtering *(slide 13)*

- **Cost:** every update touches the whole covariance, $O(N^2)$ per step and in memory. The map must stay small: about 100 features in MonoSLAM at 30 Hz.
- **Linearization:** Jacobians are evaluated once, at the current estimate, and never recomputed; accuracy can fall well below bundle adjustment, and the EKF can diverge if nonlinearities are large.
- **Past poses:** all poses but the current one are marginalized after every frame; what they contributed survives only in the Gaussian over the current pose and the map.

Proposition 2.6 (Marginalization creates fill-in)**.** *In information form $\Lambda = \Sigma^{-1}$, partition the variables as $(\mathbf{a}, \mathbf{b})$. Marginalizing out $\mathbf{b}$ gives the information matrix $\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ (Schur complement). Hence all variables that were connected to $\mathbf{b}$ become directly connected to each other.*

Proof. For a Gaussian with information matrix Λ, the marginal covariance of $\mathbf{a}$ is the corresponding block of Λ^{-1}, which by the block-inversion formula equals $(\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba})^{-1}$. The term $\Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ is non-zero in block (k, l) whenever k and l are both neighbours of $\mathbf{b}$. □

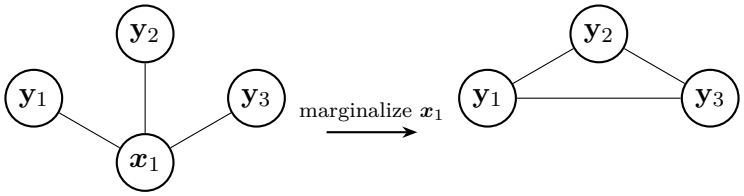


Figure 179: Marginalizing a pose that saw three points turns a sparse star into a dense clique: past poses cannot be dropped for free.

Intuition

Filtering throws away the past poses, and the price of forgetting is a dense belief over everything they saw. Graph-based SLAM (next section) keeps the past poses as explicit variables, so the system stays sparse and can be *relinearized*.

3 Graph-Based SLAM: Factor Graphs and Least Squares *(slides 14–22)*

3.1 Graph-Based SLAM *(slide 15)*

Graph-based SLAM keeps *every keyframe pose* and represents the problem as a graph. Every node is a camera pose or a map point; every edge is a spatial constraint between two nodes, given by a (uncertain) measurement. Observing a previously seen area creates constraints between non-successive poses. The **front end** builds the graph from raw data; the **back end** finds the node configuration that minimizes the error of all constraints.

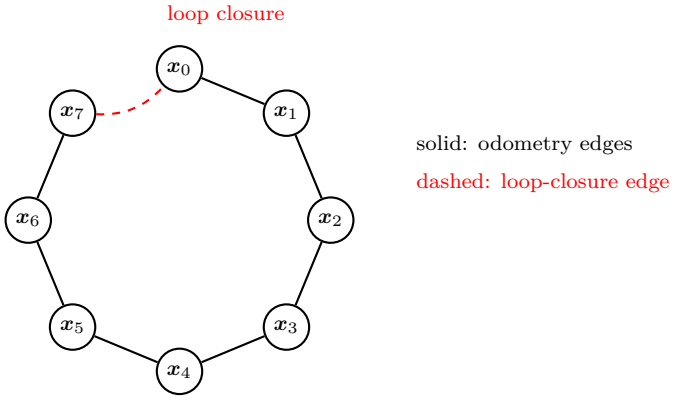


Figure 180: A pose graph (slide 15): odometry links consecutive poses; recognizing a visited place adds a constraint between non-successive poses.

3.2 Factor Graphs *(slide 16)*

Definition 3.1 (Factor graph)**.** A *factor graph* is a bipartite graph with *variable* nodes (circles: the unknown poses and points) and *factor* nodes (squares). Each factor ϕ_k is joined only to the variables Θ_k it depends on, and the global function is the product

$$\phi(\Theta) = \prod_k \phi_k(\Theta_k).$$

Given the measurements $\mathcal{Z}$, the posterior is proportional to such a product: $p(\Theta \mid \mathcal{Z}) \propto \prod_k \phi_k(\Theta_k)$.

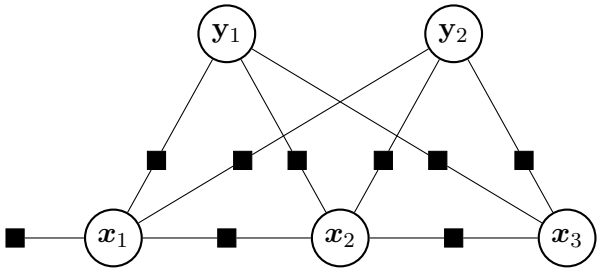


Figure 181: Toy factor graph (redrawn in the spirit of Dellaert and Kaess, Fig. 1.3): three poses, two points, nine factors, namely one prior, two motion factors, and one factor per measurement (six).

Intuition

Think of a factor as "one piece of evidence", and its edges as the unknowns it talks about. A GPS reading talks about one pose; an odometry reading about two consecutive poses; a feature observation about one pose and one point. The factor graph is just the table of contents of who-constrains-whom, which will turn out to be exactly the sparsity pattern of the linear system.

3.3 Visual SLAM as a Factor Graph *(slide 17)*

The variables are the camera poses $\mathbf{T}_i$ and the points $\mathbf{X}_j$. Each observation adds one *reprojection factor*, a Gaussian on the residual of lecture 05:

$$\phi_{ij}(\mathbf{T}_i, \mathbf{X}_j) \propto \exp\Big(-\tfrac{1}{2} \|\mathbf{r}_{ij}\|^2_{\Sigma_{ij}^{-1}} \Big), \qquad \mathbf{r}_{ij} = \pi(\mathbf{T}_i \mathbf{X}_j) - \mathbf{u}_{ij},$$

with Σ_{ij} the covariance of the pixel noise. A *prior* factor on $\mathbf{T}_1$ anchors the solution in space; with one camera the scale stays free unless one distance is fixed too (lecture 05). With these Gaussian factors, the most probable variables are the bundle adjustment solution of lecture 05.

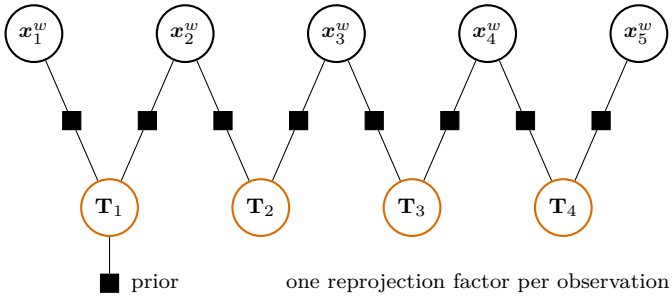


Figure 182: Visual SLAM factor graph (slide 17): each pose sees two neighbouring points.

Intuition

Bundle adjustment is not a different algorithm from SLAM's back end: it is the MAP estimate of this graph. The prior on $\mathbf{T}_1$ pins the *gauge*: the whole reconstruction could otherwise slide or rotate in space (and, for one camera, scale) with no change in error.

3.4 The Pose Graph *(slide 18)*

The pose graph keeps only the poses. Each edge is a measured *relative pose* between two keyframes i and j:

- **Odometry edge:** between consecutive keyframes, from the motion estimated between them.
- **Loop-closure edge:** between non-successive keyframes that observe the same part of the scene, a virtual measurement of keyframe j seen from keyframe i.

Each edge carries a measured relative pose $\mathbf{Z}_{ij}$ and an information matrix Ω_{ij}. Eliminating the points exactly would connect every pair of poses that saw a common point (Proposition 2.6); the pose graph is kept sparse *by construction*, by using only a selected set of relative-pose edges.

3.5 From Factors to Least Squares *(slide 19)*

Each factor is a Gaussian in its residual, $\phi_k \propto \exp(-\frac{1}{2}\mathbf{e}_k^\top \Omega_k \mathbf{e}_k)$.

Theorem 3.2 (MAP on a factor graph is nonlinear least squares)**.** *The most probable variables minimize a sum of squared Mahalanobis errors:*

$$\Theta^\star = \arg\max_\Theta \prod_k \phi_k(\Theta_k) = \arg\min_\Theta \; F(\Theta), \qquad F(\Theta) = \sum_k \mathbf{e}_k(\Theta_k)^\top \Omega_k \, \mathbf{e}_k(\Theta_k),$$

where $\mathbf{e}_k$ is the residual of factor k (predicted minus actual measurement) and Ω_k the inverse covariance of the measurement. The sum runs over prior, motion and measurement terms alike.

Proof. $\log \prod_k \phi_k = \sum_k \log \phi_k = \text{const} - \frac{1}{2}\sum_k \mathbf{e}_k^\top \Omega_k \mathbf{e}_k$, and log is monotone, so maximizing the product is minimizing the sum. □

Intuition

Logs turn products into sums, Gaussians into parabolas. Worked number: two measurements of the same 1D position, 10.0 with $\sigma = 1$ ($\Omega = 1$) and 10.6 with $\sigma = 0.5$ ($\Omega = 4$). Minimizing $1\,(x-10)^2 + 4\,(x-10.6)^2$ gives $x = \frac{1\cdot 10 + 4\cdot 10.6}{5} = 10.48$: the more certain reading pulls four times harder. This is bundle adjustment of lecture 05, *weighted*.

3.6 Error and Information Matrix of an Edge *(slide 20)*

An edge measures the pose of keyframe j seen from keyframe i, $\mathbf{Z}_{ij}$. Since each pose maps world to camera, the poses predict it as $\mathbf{T}_i\mathbf{T}_j^{-1}$, which maps camera-$j$ coordinates to camera i. The residual compares the two:

$$\mathbf{e}_{ij} = \log\!\big(\mathbf{Z}_{ij}^{-1}\,\mathbf{T}_i\mathbf{T}_j^{-1}\big)^\vee \in \mathbb{R}^6, \qquad F = \sum_{(i,j)} \mathbf{e}_{ij}^\top \Omega_{ij}\, \mathbf{e}_{ij}.$$

Here $\mathbf{Z}_{ij}^{-1}$ maps from i to j, so $\mathbf{e}_{ij} = \mathbf{0}$ exactly when the poses agree with the measurement; Ω_{ij} is the inverse covariance of $\mathbf{Z}_{ij}$ and encodes its uncertainty: the larger Ω_{ij}, the more the edge matters in the optimization.

Intuition

Think of every edge as a spring whose rest length is the measurement and whose stiffness is Ω_{ij}. Odometry springs are stiff but there are many of them (errors accumulate); a loop-closure spring is softer but connects two far-away parts of the path. Numbers: $\sigma_{\text{odom}} = 0.1\,\text{m}$ gives $\Omega = 100$, while

$\sigma_{\text{loop}} = 1\,\text{m}$ gives $\Omega = 1$: a loop edge is $100\times$ weaker per unit squared error, yet it can still bend a long chain because the chain's accumulated error is spread over hundreds of springs.

3.7 Gauss-Newton on the Pose Graph *(slide 21)*

Around the current poses, every residual is linearized in the increments of lecture 05, $\mathbf{T}_i \leftarrow \exp(\delta\boldsymbol{\xi}_i^{\wedge})\mathbf{T}_i$: $\mathbf{e}_{ij}(\mathbf{x} \boxplus \delta\mathbf{x}) \approx \mathbf{e}_{ij} + \mathbf{J}_{ij}\,\delta\mathbf{x}$.

Algorithm 3.1 (Gauss–Newton / Levenberg–Marquardt on the pose graph)**.** Repeat until convergence, starting from a good initial guess (e.g. chained odometry):

1. $\mathbf{H} = \sum_{ij} \mathbf{J}_{ij}^{\top}\Omega_{ij}\mathbf{J}_{ij}, \quad \mathbf{b} = \sum_{ij} \mathbf{J}_{ij}^{\top}\Omega_{ij}\mathbf{e}_{ij}$.
2. Solve the normal equations $\mathbf{H}\,\delta\mathbf{x} = -\mathbf{b}$ (Gauss–Newton), or $(\mathbf{H} + \lambda\mathbf{I})\,\delta\mathbf{x} = -\mathbf{b}$ (Levenberg–Marquardt).
3. Update $\mathbf{x} \leftarrow \mathbf{x} \boxplus \delta\mathbf{x}$. In LM, the damping λ is doubled when a step increases the error and halved otherwise.

Derivation of the normal equations. Substituting the linearization, $F(\mathbf{x}\boxplus\delta\mathbf{x}) \approx \sum(\mathbf{e}+\mathbf{J}\delta\mathbf{x})^{\top}\Omega(\mathbf{e}+\mathbf{J}\delta\mathbf{x}) = c + 2\,\mathbf{b}^{\top}\delta\mathbf{x} + \delta\mathbf{x}^{\top}\mathbf{H}\,\delta\mathbf{x}$. Setting the gradient $2\mathbf{b} + 2\mathbf{H}\delta\mathbf{x}$ to zero gives $\mathbf{H}\delta\mathbf{x} = -\mathbf{b}$. □

Proposition 3.3 (Gauge freedom)**.** *The error F is unchanged if all poses move together, $\mathbf{T}_i \mapsto \mathbf{T}_i\mathbf{S}$ for any rigid $\mathbf{S}$. Hence $\mathbf{H}$ is singular (null space of dimension 6), and the graph is anchored by holding $\mathbf{T}_1$ fixed, as the prior on the first pose did.*

Proof. $(\mathbf{T}_i\mathbf{S})(\mathbf{T}_j\mathbf{S})^{-1} = \mathbf{T}_i\mathbf{T}_j^{-1}$, so every $\mathbf{e}_{ij}$ is invariant. Differentiating along the 6-parameter family shows $\mathbf{J}\mathbf{v} = \mathbf{0}$ for its tangent directions $\mathbf{v}$, hence $\mathbf{H}\mathbf{v} = \sum \mathbf{J}^{\top}\Omega\mathbf{J}\mathbf{v} = \mathbf{0}$. □

Intuition

Pose graphs only know *relative* information, like a map with distances but no coordinates: you can pick the whole thing up and put it down elsewhere. Fixing one pose nails it to the table. LM damping is a safety belt: a large λ takes small, gradient-like steps; a small λ takes full Gauss–Newton steps. This handles highly non-convex problems better, and loop closures are exactly where the problem is non-convex.

3.8 Sparsity of the Normal Equations *(slide 22)*

A residual depends on $\mathbf{T}_i$ and $\mathbf{T}_j$ only, so its Jacobian has two non-zero blocks, $\mathbf{J}_{ij} = [\cdots\ \mathbf{A}_{ij}\ \cdots\ \mathbf{B}_{ij}\ \cdots]$, and each edge adds to *four blocks of* $\mathbf{H}$ *and two of* $\mathbf{b}$:

$$\mathbf{H}_{ii} \mathrel{+}= \mathbf{A}^{\top}\Omega\mathbf{A}, \quad \mathbf{H}_{ij} \mathrel{+}= \mathbf{A}^{\top}\Omega\mathbf{B}, \quad \mathbf{H}_{ji} \mathrel{+}= \mathbf{B}^{\top}\Omega\mathbf{A}, \quad \mathbf{H}_{jj} \mathrel{+}= \mathbf{B}^{\top}\Omega\mathbf{B}, \qquad \mathbf{b}_i \mathrel{+}= \mathbf{A}^{\top}\Omega\mathbf{e}, \quad \mathbf{b}_j \mathrel{+}= \mathbf{B}^{\top}\Omega\mathbf{e}.$$

Proposition 3.4 (Sparsity pattern of $\mathbf{H}$)**.** *The block $\mathbf{H}_{ij}$ is non-zero if and only if there is an edge between i and j. The block pattern of $\mathbf{H}$ is therefore the adjacency matrix of the graph plus the diagonal, and $\mathbf{H}$ is assembled by summing the contribution of each edge.*

Proof. Off-diagonal blocks only receive the terms $\mathbf{A}^{\top}\Omega\mathbf{B}$ above, one per edge joining i and j; all other products $\mathbf{J}_{kl}^{\top}\Omega\mathbf{J}_{kl}$ are zero in block (i, j). □

Intuition

Worked numbers: 1000 poses with 6 degrees of freedom give $\mathbf{H} \in \mathbb{R}^{6000\times6000}$, i.e. 36 million entries if dense. But there are only about 1000 diagonal and 2×999 off-diagonal blocks of 36 numbers, roughly 108,000 non-zeros (0.3%). Sparse solvers (sparse Cholesky, conjugate gradients) exploit this, which is why graph-based SLAM scales to the 11 km running example while a dense EKF covariance cannot.

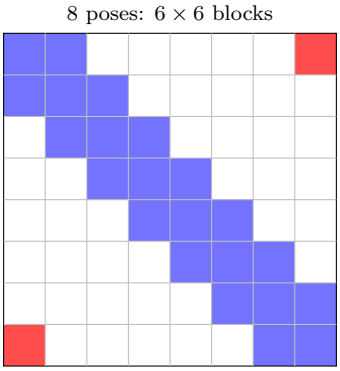


Figure 183: Block sparsity of **H** for the ring pose graph of slide 15: diagonal and first off-diagonals come from odometry (blue), the two red blocks come from the single loop closure.

4 The Visual SLAM Pipeline: ORB-SLAM *(slides 23–32)*

4.1 Pipeline Overview *(slide 24)*

ORB-SLAM, a keyframe-based monocular SLAM system, runs three threads in parallel: **tracking** localizes the camera at every frame and decides when to insert a keyframe; **local mapping** processes each new keyframe and runs bundle adjustment around it; **loop closing** searches for a loop at every keyframe and corrects the drift by pose graph optimization. The *front end* abstracts the images into features and data associations; the *back end* estimates the variables from them (Section 6.3).

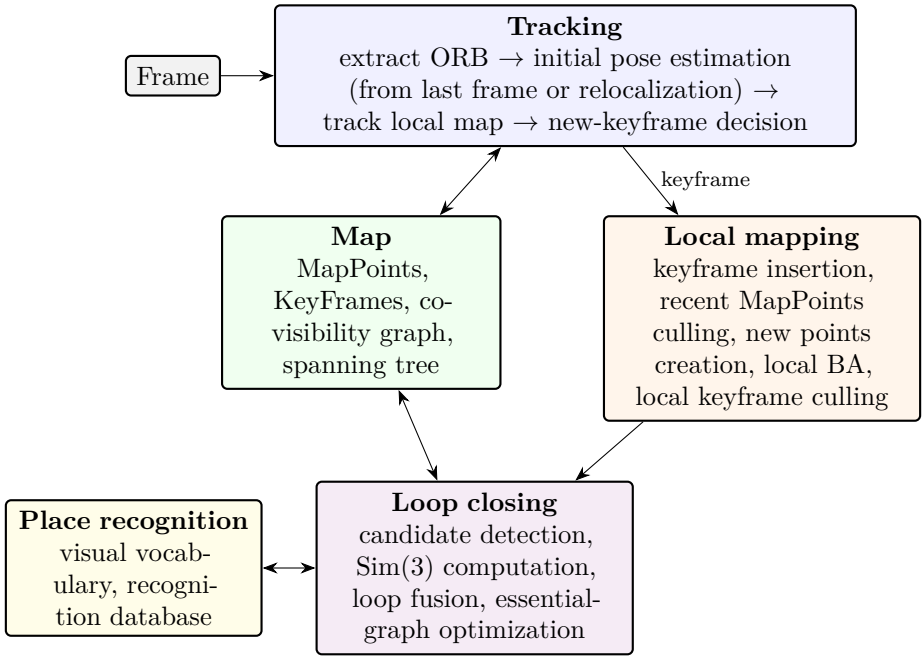


Figure 184: ORB-SLAM threads and data structures (redrawn from the slide 24 block diagram).

Intuition

The split mirrors the lecture: Section 6.3 was the back end (sparse least squares, assumed data associations), and this section is a front end that produces those associations fast enough for 30 frames per second. Three threads exist because the three jobs have different deadlines: tracking must finish within one frame, local mapping within a keyframe interval, loop closing can take longer.

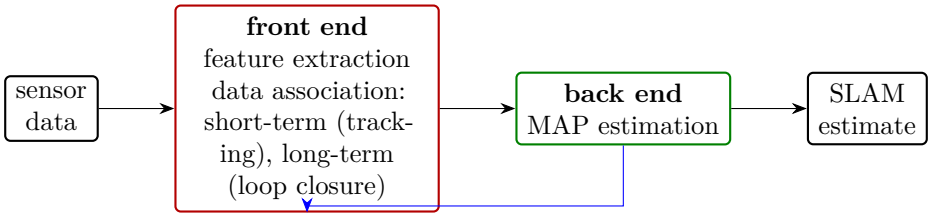


Figure 185: Front end / back end split (slide 24, right): the back end also feeds predictions back to guide data association.

4.2 ORB Features and Matching *(slide 25)*

ORB-SLAM uses the same features for tracking, mapping, relocalization and loop detection. As in lecture 05, a feature is a keypoint with a descriptor.

- Extraction must take much less than the 33 ms between frames, which excludes SIFT at about 300 ms.
- **ORB**: corners from the FAST detector, given an orientation, found at 8 scale levels a factor 1.2 apart, each with a binary descriptor; the orientation makes it rotation invariant.
- 1000 keypoints per image up to 752×480 pixels, 2000 on larger images (1241×376), spread over a grid.

Definition 4.1 (Hamming distance)**.** For binary descriptors $\mathbf{d}, \mathbf{d}' \in \{0,1\}^n$ the *Hamming distance* is the number of differing bits, $\|\mathbf{d} \oplus \mathbf{d}'\|_1 = \text{popcount}(\mathbf{d} \text{ XOR } \mathbf{d}')$.

Intuition

Matching two descriptors is one XOR and one popcount instruction: orders of magnitude cheaper than the Euclidean distance between 128 floats of SIFT. Example with 8 bits: `10110010` vs. `10011010` differ in 2 positions, so $d = 2$. The 8-level pyramid with factor 1.2 covers a total scale range of $1.2^7 \approx 3.6$, which is why ORB matches survive strong zoom changes.

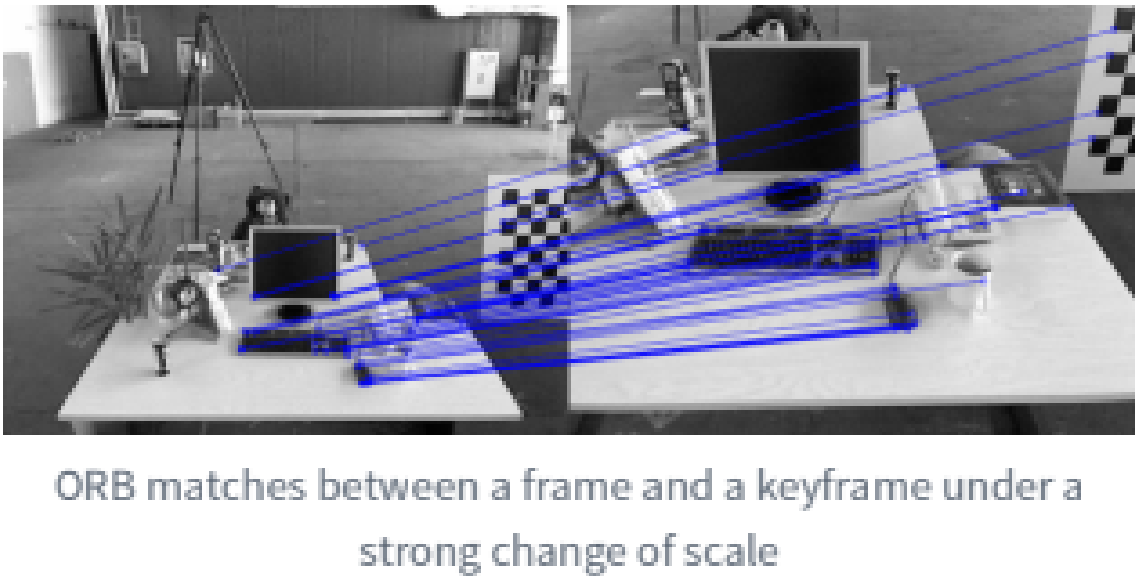


Figure 186: ORB matches between a frame and a keyframe under a strong change of scale.

4.3 Map Initialization *(slide 26)*

The first map needs the relative pose of two frames and triangulated points, for planar and general scenes alike. Between a reference frame 1 and the current frame 2, two models are fitted in parallel with RANSAC: a homography $\mathbf{x}_2 \simeq \mathbf{H}_{21}\mathbf{x}_1$ (4 points, planar scene) and a fundamental matrix $\mathbf{x}_2^\top \mathbf{F}_{21}\mathbf{x}_1 = 0$ (8 points, general scene), each scored by its symmetric transfer error

$$S_M = \sum_i \Big[\rho_M\big(d^2(\mathbf{x}_1^i, M^{-1}\mathbf{x}_2^i)\big) + \rho_M\big(d^2(\mathbf{x}_2^i, M\mathbf{x}_1^i)\big)\Big].$$

Algorithm 4.1 (ORB-SLAM map initialization)**.**

1. Fit $\mathbf{H}_{21}$ and $\mathbf{F}_{21}$ with RANSAC and compute the scores S_H, S_F.
2. Select the homography if $S_H/(S_H + S_F) > 0.45$ (planar scenes and low parallax); otherwise the fundamental matrix.
3. Recover the motion hypotheses: 8 from the essential matrix or 4 from the homography (lecture 05); triangulate with all of them.
4. If there is no clear winner, *create no map* and try with new frames.
5. Refine the first map by full bundle adjustment; its scale is arbitrary (lecture 05).

Intuition

A homography explains almost anything seen with *no parallax* (a plane, or pure rotation), but then depth is ill-defined and triangulation is unreliable. ORB-SLAM prefers to *wait for parallax* rather than start from a wrong planar map; the slide shows two other systems that started wrongly and drifted, while ORB-SLAM, which waited, did not.

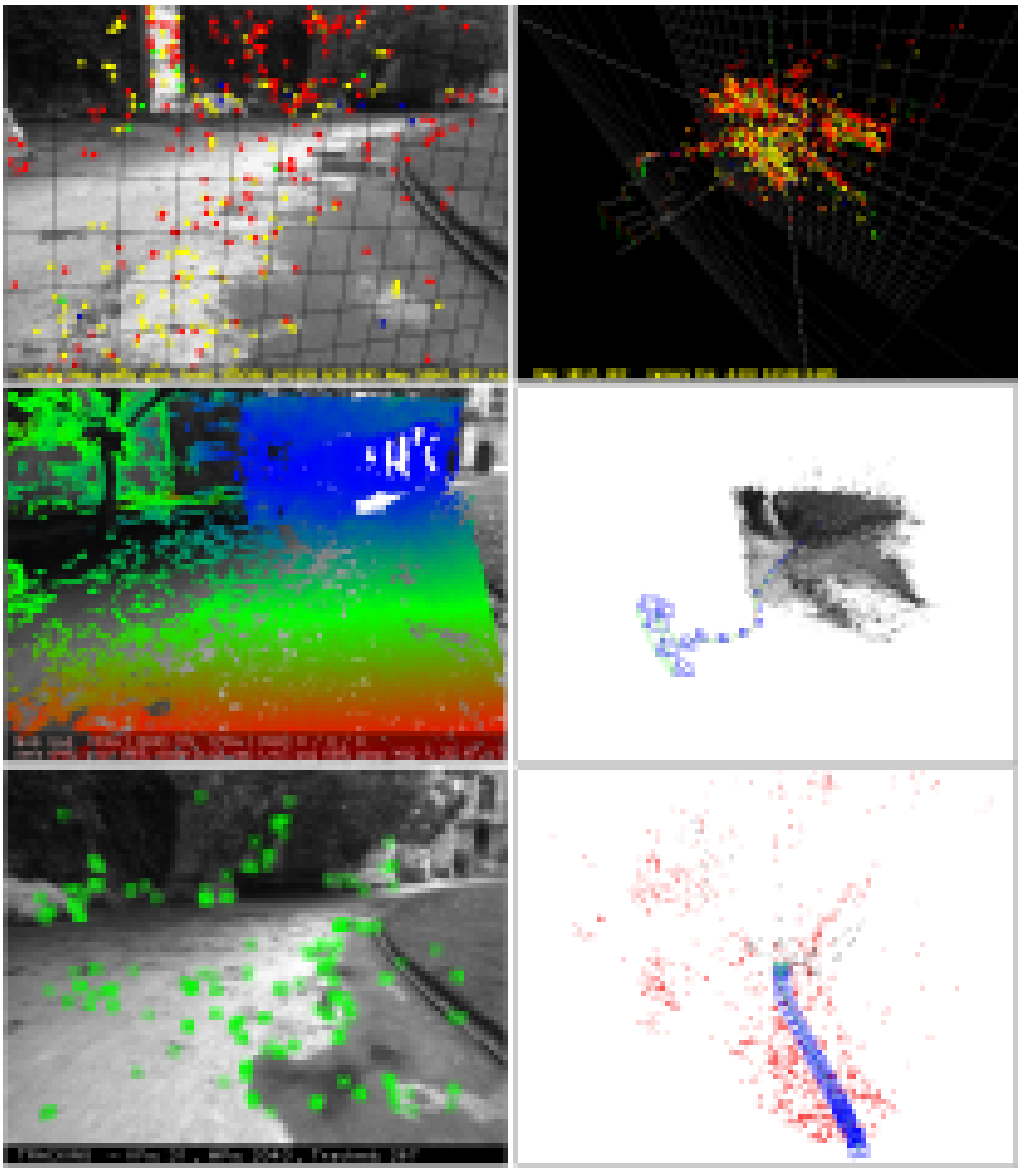


Figure 187: Some time after initialization: two other systems (top, middle) started from a wrong planar map; ORB-SLAM (bottom) waited for parallax.

4.4 Tracking *(slide 27)*

The tracking thread estimates the pose of every frame:

1. **Prediction:** a constant-velocity motion model, $\mathbf{T}_t^{\text{pred}} = (\mathbf{T}_{t-1}\mathbf{T}_{t-2}^{-1})\,\mathbf{T}_{t-1}$, predicts the pose and guides the search for the points seen in the last frame; if too few are found, a wider search follows.
2. **Local map:** the keyframes that share points with the frame, and their covisible neighbours; their points are projected into the frame and matched near the projection, at the predicted scale.
3. **Motion-only bundle adjustment:** the reprojection error of lecture 05 over the pose alone, points fixed, with a robust (Huber) cost,

$$\mathbf{T}^\star = \arg\min_{\mathbf{T}} \sum_j \rho_{\text{Huber}}\Big(\|\mathbf{u}_j - \pi(\mathbf{T}\mathbf{X}_j)\|^2_{\Sigma^{-1}}\Big).$$

Intuition

Only 6 unknowns are optimized, so this runs in a few milliseconds per frame. The Huber cost behaves quadratically for small errors and linearly for large ones, so one wrong match (an outlier with a 50-pixel error) cannot dominate 100 good ones. **PTAM**, a monocular system for small augmented-reality workspaces, predicts with a decaying velocity and tracks coarse to fine: 50 points at the coarsest image scale, then up to 1000.

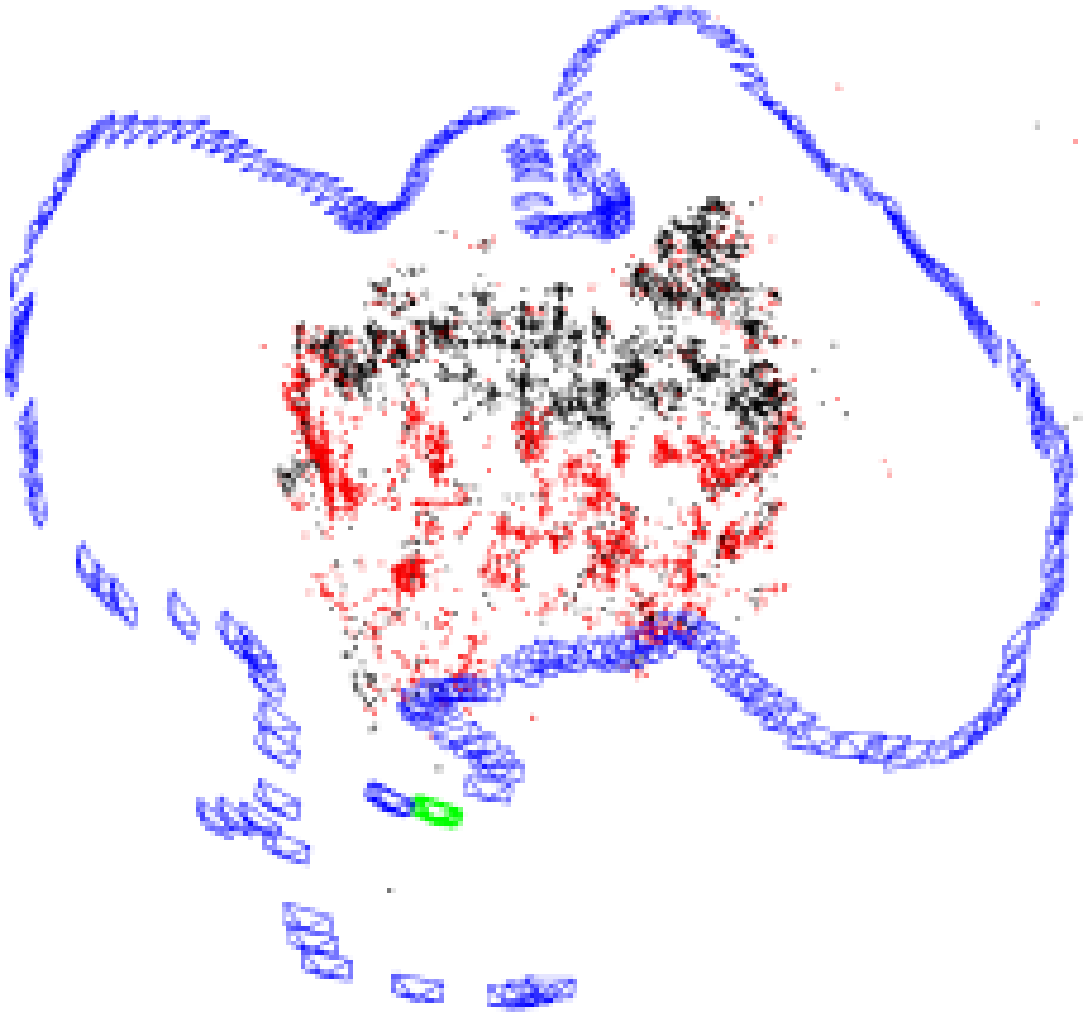


Figure 188: Keyframes (blue), current camera (green), map points (black) and local map (red).

4.5 Keyframes *(slide 28)*

PTAM splits SLAM into two threads that run in parallel: **tracking** processes every frame in real time (freed from updating the map, it can process each image more thoroughly), and **mapping** works on keyframes only: many video frames are redundant, and a smaller set of useful keyframes is refined by bundle adjustment without strict real-time limits.

- PTAM: a frame becomes a keyframe if tracking is good, more than twenty frames have passed, and the camera is a minimum distance (set by the mean scene depth) from the existing keyframes.
- ORB-SLAM instead inserts keyframes *as fast as possible*, which keeps tracking robust to rotations, and culls the redundant ones in local mapping.

Intuition

Keyframes are the "postcards" of the trip: you do not keep every frame of a 30 Hz video (11 km would be hundreds of thousands), only enough views to triangulate the scene. ORB-SLAM's strategy is generous insertion followed by pruning: it is easier to remove a redundant postcard than to recover a view you never kept.

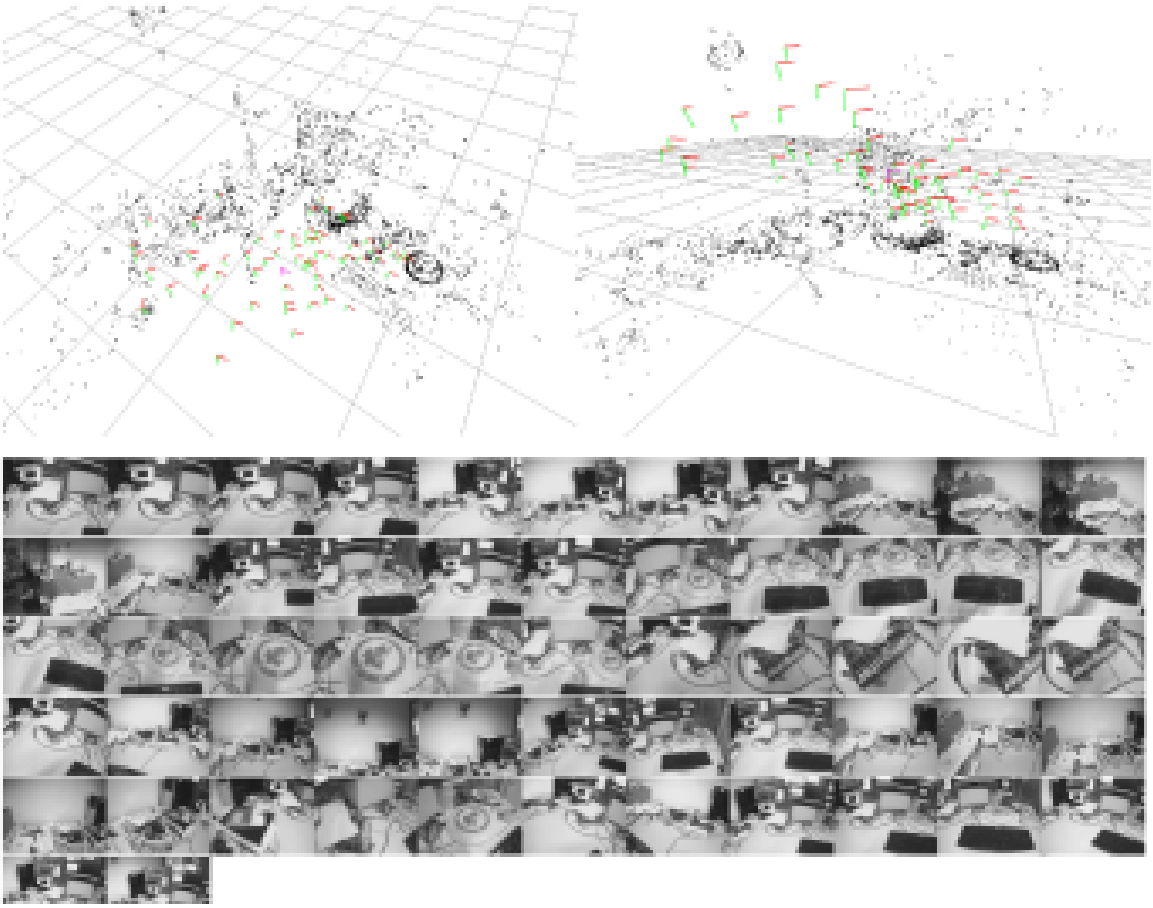


Figure 189: A desk mapped by PTAM: two views of the map with its keyframes (top), and the 57 keyframes (bottom).

4.6 Local Mapping *(slide 29)*

Definition 4.2 (Covisibility graph)**.** Nodes are keyframes; an edge joins two keyframes that share at least 15 map points, weighted by the number of shared points.

For every new keyframe the local mapping thread:

1. inserts it in the covisibility graph;
2. culls recent points: a point stays only if it is found in more than 25% of the frames predicted to see it, and from at least three keyframes;
3. triangulates new points from unmatched features in covisible keyframes, checking depth, parallax, reprojection error and scale;
4. runs **local bundle adjustment** on the keyframe, its covisible keyframes and all their points; the other keyframes that see these points stay fixed;
5. culls redundant keyframes, whose points are 90% seen by at least three other keyframes.

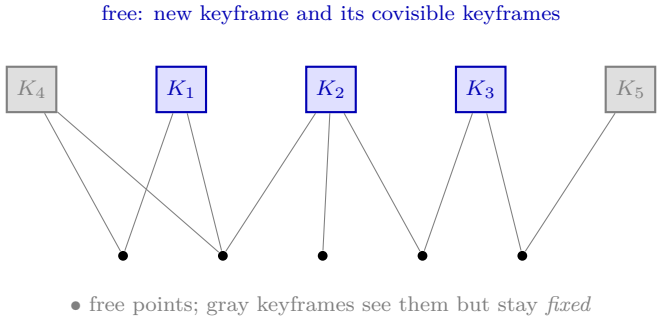


Figure 190: Local bundle adjustment (slide 29, step 4): only a small neighbourhood is optimized; fixed keyframes anchor it to the rest of the map.

Intuition

The covisibility graph answers "who looks at the same things as me?". Optimizing only that neighbourhood keeps the cost bounded while the map grows to kilometres: *local* BA is constant-time per keyframe, unlike EKF-SLAM whose cost grows with the whole map. Fixing the outer keyframes plays the role of the prior on $\mathbf{T}_1$ for the local problem.

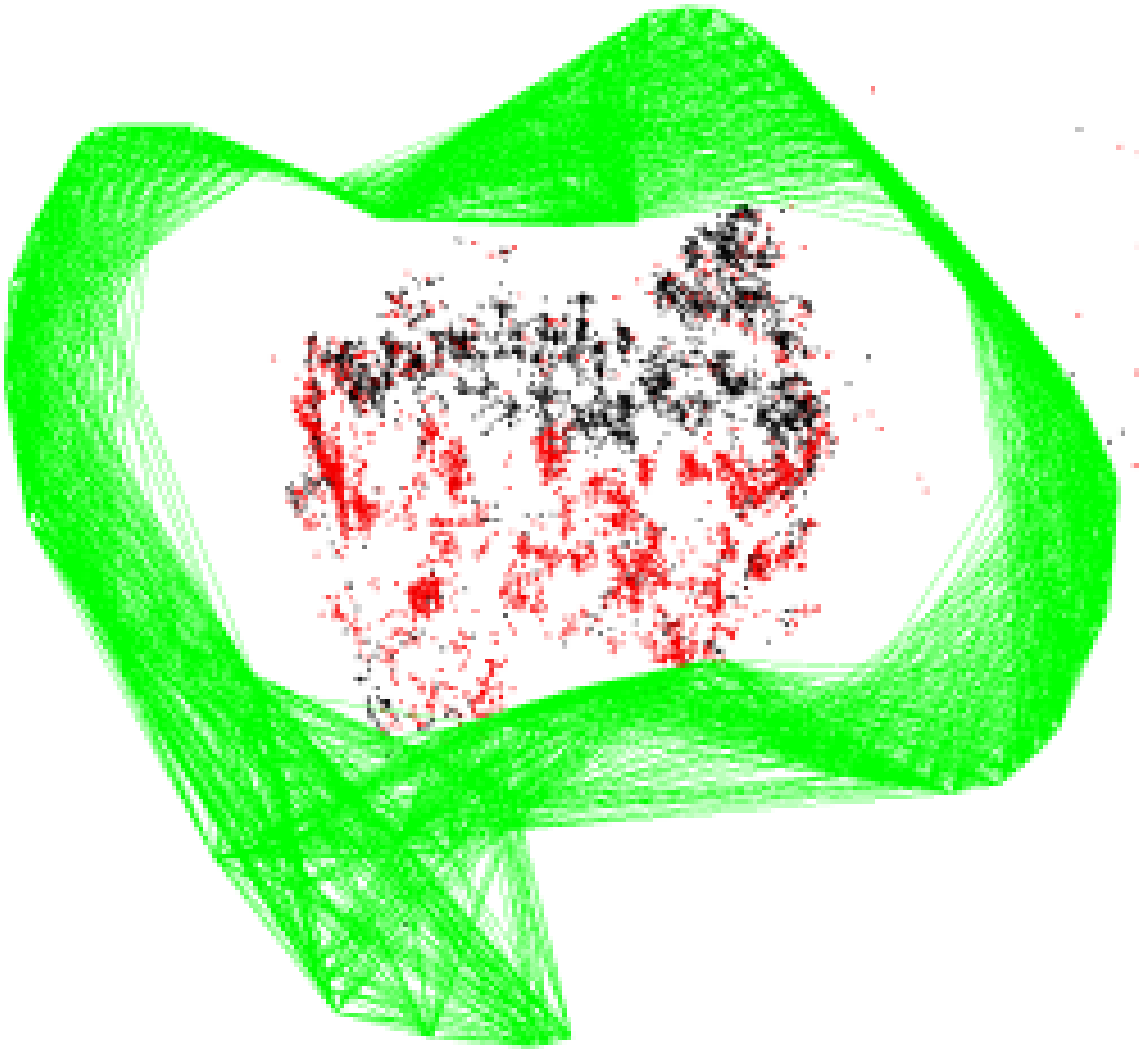


Figure 191: Covisibility graph of an indoor handheld sequence (TUM RGB-D).

4.7 Loop Detection with Bags of Binary Words *(slide 30)*

A loop is detected by *appearance*: each image is compared with all the past ones, fast enough for every frame.

- **Vocabulary:** binary descriptors clustered offline into a tree of 10 branches and 6 levels whose leaves are the visual words; an image becomes a vector of word weights.
- **Inverse index:** for each word, the images that contain it, so a query visits only images sharing words with it.
- **Checks:** the score is normalized by that of the previous image; a match must agree with previous matches and be supported by a fundamental matrix with at least 12 RANSAC inliers.
- ORB-SLAM uses ORB words, groups candidates by covisibility rather than by time, and accepts a loop after three consecutive consistent candidates. 21.6 ms per image on average, over 26292 images.

Proposition 4.3 (Why a hierarchical vocabulary is fast)**.** *A tree with branching factor k and L levels has k^L leaves (words), but quantizing a descriptor needs only kL comparisons.*

Proof. At each level the descriptor is compared with the k children of the current node and descends into the closest one; there are L levels. □

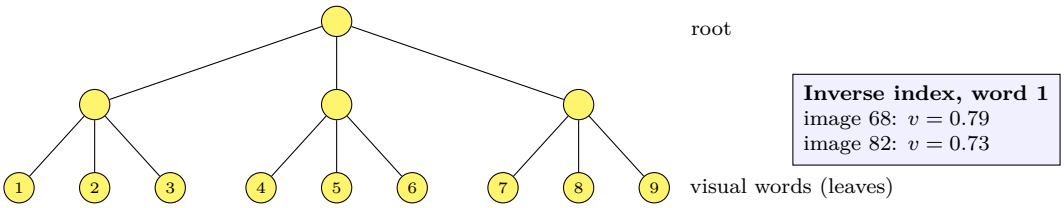


Figure 192: Vocabulary tree and inverse index (slide 30; the sketch shows $k = 3,\ L = 2$, the system uses $k = 10,\ L = 6$).

Intuition

Numbers: 10 branches and 6 levels give $10^6 = 1{,}000{,}000$ words, but a descriptor is quantized with only $10 \times 6 = 60$ comparisons. The inverse index is the book index idea: to find pages about "Cholesky" you do not read the whole book. Normalizing by the score of the previous image avoids false alarms in places that simply look similar to everything (corridors). The score between two images lies between 0 and 1.

4.8 Loop Correction and Scale Drift *(slide 31)*

One camera cannot observe scale, so the scale drifts too: a monocular map drifts in **seven** degrees of freedom (rotation, translation and scale). A pose graph over SE(3) closes the loop but *keeps the scale drift*. The poses become similarities (lecture 01).

Definition 4.4 (Similarity transformation Sim(3))**.** $\mathbf{S} = \begin{pmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}$, with $\mathbf{R} \in SO(3)$, $\mathbf{t} \in \mathbb{R}^3$, $s > 0$; it acts on points as $\mathbf{X} \mapsto s\mathbf{R}\mathbf{X} + \mathbf{t}$ and has 7 degrees of freedom. The pose-graph residual has the same form as before, $\mathbf{e}_{ij} = \log(\mathbf{Z}_{ij}^{-1}\mathbf{S}_i\mathbf{S}_j^{-1})^\vee \in \mathbb{R}^7$.

- Odometry edges keep their measured motion with scale 1; the loop edge carries the measured change of scale.
- **ORB-SLAM:** the loop similarity comes from 3D–3D point matches, by RANSAC and Horn's method; duplicated points are fused; the pose graph over Sim(3) is optimized on the *essential graph*: a spanning tree, the covisibility edges with at least 100 shared points, and the loop edges.

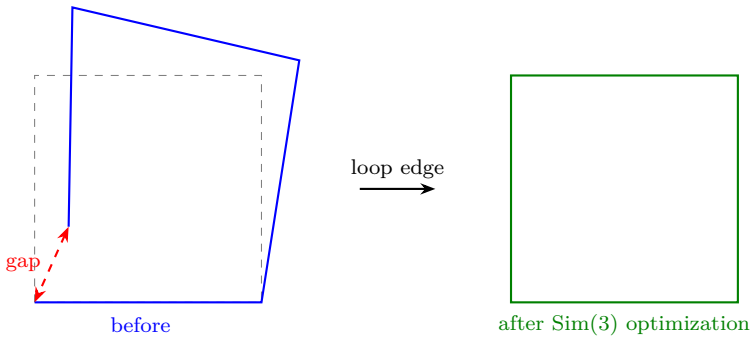


Figure 193: Loop closure (sketch of the slide 31 experiment): drift in rotation, translation and scale leaves the loop open; the loop edge and its scale are distributed along the cycle.

Intuition

Numbers: if after a loop the end of the trajectory is estimated at relative scale 1.10 with respect to the start, the loop edge says so, and the Sim(3) optimization spreads $\ln 1.10 \approx 0.0953$ over, say, 100 keyframes, about 0.00095 of log-scale each. A SE(3) optimization has no scale variable, so it can bend

the path but the scale error stays: the slide shows the 6 DoF result still distorted while the 7 DoF result matches the aerial photo.

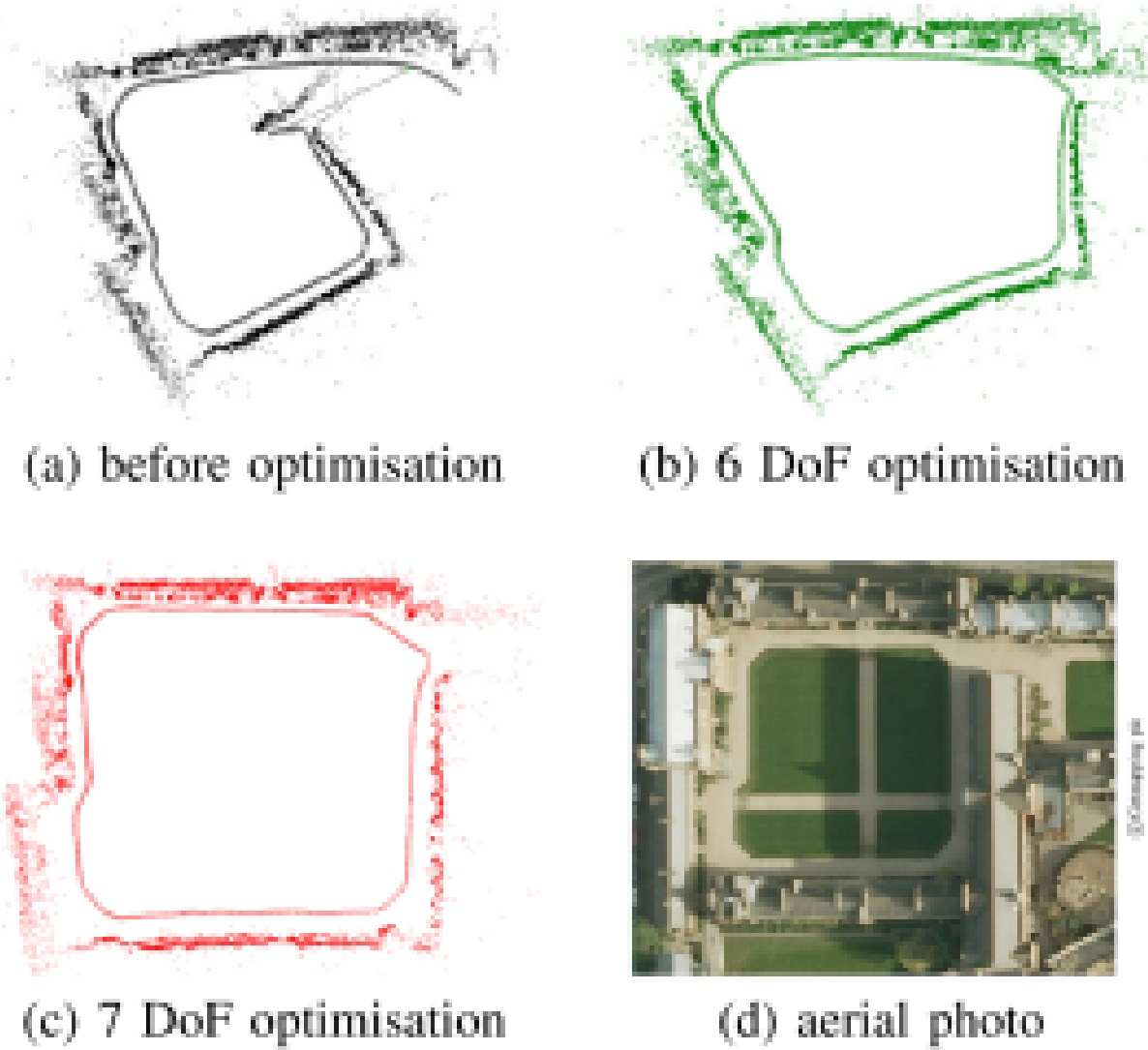


Figure 194: A square circled by a hand-held camera: (a) before optimization, (b) over SE(3) (6 DoF), (c) over Sim(3) (7 DoF), (d) aerial photo.

4.9 Relocalization *(slide 32)*

When tracking is lost, after an occlusion or an abrupt motion, the camera is localized again in the existing map:

1. the frame is converted into a bag of words, and the place recognition database returns candidate keyframes;
2. for each candidate, ORB features are matched to its map points, and RANSAC with PnP (lecture 05) estimates a camera pose;
3. a pose with enough inliers is refined and guides the search for more matches; with enough inliers, tracking resumes.

On indoor handheld sequences with ground truth (TUM RGB-D), 78.4% of 2769 frames are relocalized in a map built from the first 30 s; with people walking through the scene, 77.9% of 859 frames.

Intuition

Relocalization reuses everything already built: the same bag-of-words database that detects loops here answers "where am I?" from a single image. It is a *global* localization problem (no prior pose), solved by PnP inside RANSAC, which tolerates the moving people that spoil many of the matches.

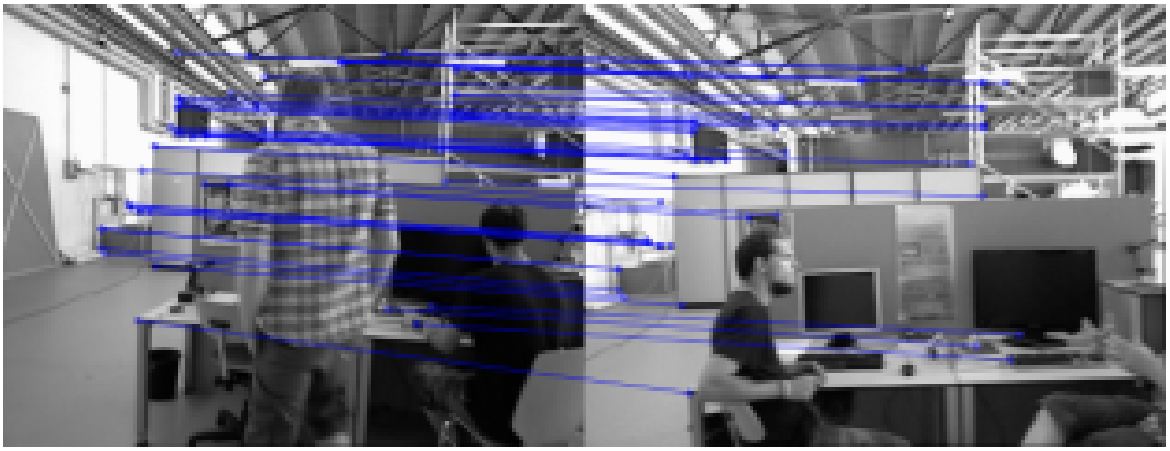


Figure 195: A relocalization among moving people: ORB matches between the frame and a keyframe.

5 Research Directions *(slides 33–40)*

5.1 Continuous Time: Rolling-Shutter Cameras *(slide 34)*

A rolling-shutter camera exposes every line of the image for a different period, each one more delayed than the last. When the camera moves during the exposure, the image appears distorted and skewed. So far every image had one pose, as with a global shutter, where all pixels are exposed over the same duration. Earlier work removed the distortion first, to recreate a global-shutter image. On a simulated monocular sequence, the loop-closure position error is 0.0472 of the distance travelled when time is modelled as for a global shutter, and 0.0057 when the rolling shutter is modelled.

Intuition

The numbers: 4.7% versus 0.57% of the path length, an 8× reduction ($0.0472/0.0057 \approx 8.3$) just from admitting that the rows of an image are taken at different times. Each scanline is a tiny camera with its own timestamp, so the *trajectory*, not a pose per image, becomes the unknown.

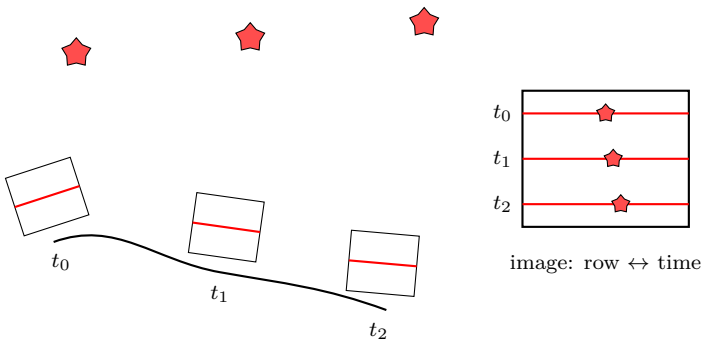


Figure 196: Each scanline is a camera with its own time along the trajectory (redrawn from slide 34); together they form the image.



Figure 197: A rolling-shutter warp (distorted and skewed building facade).

5.2 Continuous Time: Cumulative B-Splines on SE(3) *(slide 35)*

The trajectory becomes a continuous function of time, a *cumulative cubic B-spline* built with the Lie algebra (lecture 05). Control poses $\mathbf{T}_i$ sit at uniform times t_i, Δt apart. For $t \in [t_i, t_{i+1})$ let $u = (t - t_i)/\Delta t \in [0, 1)$; the four poses $\mathbf{T}_{i-1}, \dots, \mathbf{T}_{i+2}$ give

$$\mathbf{T}(t) = \exp\big(\tilde{B}_3(u)\,\boldsymbol{\delta}_3\big)\exp\big(\tilde{B}_2(u)\,\boldsymbol{\delta}_2\big)\exp\big(\tilde{B}_1(u)\,\boldsymbol{\delta}_1\big)\,\mathbf{T}_{i-1}, \qquad \boldsymbol{\delta}_j = \log\big(\mathbf{T}_{i-1+j}\mathbf{T}_{i-2+j}^{-1}\big),$$

with cumulative basis functions

$$\tilde{B}_1 = \tfrac{1}{6}(5 + 3u - 3u^2 + u^3), \quad \tilde{B}_2 = \tfrac{1}{6}(1 + 3u + 3u^2 - 2u^3), \quad \tilde{B}_3 = \tfrac{1}{6}u^3.$$

(Spline written for world-to-camera poses with left-multiplied increments.) $\boldsymbol{\delta}_j$ is the increment from control pose $j-1$ to j, and the trajectory is twice continuously differentiable. Each pixel is projected with the pose at its own time: a point on image row v is observed at $t = t_0 + \frac{v}{h}\,t_{\mathrm{ro}}$, h being the image height and t_{ro} the frame readout time. The control poses are the unknowns of the least squares, optimized over a sliding window.

Intuition

A spline is a flexible ruler pinned at control poses: you do not store a pose per row, only a handful of control poses per window, and any timestamp inside gives a smooth pose. Advantage: IMU, rolling-shutter and event-camera measurements at arbitrary timestamps can all be written as functions of the same few unknowns. Further reading: a survey of continuous-time state estimation in robotics (splines and Gaussian processes), Talbot et al., T-RO 2025.

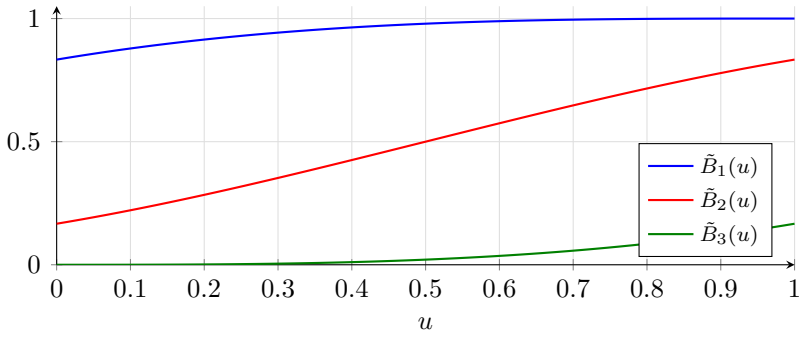


Figure 198: Cumulative cubic B-spline basis functions: at $u = 0$ the weights are $(5/6, 1/6, 0)$ and at $u = 1$ they are $(1, 5/6, 1/6)$, which equals the $u = 0$ weights of the next interval after shifting the control poses by one, so the curve is continuous.

5.3 Active SLAM *(slide 36)*

SLAM is *passive*: it does not guide the motion. **Active SLAM** plans and controls the motion of the robot to build the most accurate and complete model of the environment. It is a decision between two opposite goals, exploring new areas or revisiting those already seen (the *exploration–exploitation dilemma*). With high uncertainty, places where a loop could close carry more information than new frontiers.

Algorithm 5.1 (Active SLAM loop)**.** Repeat until a stopping condition is met: (1) identify candidate actions; (2) compute the utility of each; (3) select and execute the best.

The utility is the expected uncertainty of the pose and the map. For a Gaussian belief whose covariance $\Sigma \in \mathbb{R}^{n\times n}$ has eigenvalues $\lambda_1, \ldots, \lambda_n$, the *D-optimality* criterion measures the volume of the uncertainty ellipsoid,

$$\text{D-opt}(\Sigma) = \exp\Big(\tfrac{1}{n}\sum_{k=1}^{n} \log \lambda_k\Big) = \det(\Sigma)^{1/n},$$

to be minimized.

Intuition

Worked number: eigenvalues $(4, 1)$ give $\det^{1/2} = 2$; after an informative view that shrinks the long axis to $(1, 1)$ the criterion is 1. The geometric mean of the eigenvalues is used because it rewards shrinking *any* axis, not only the largest one.

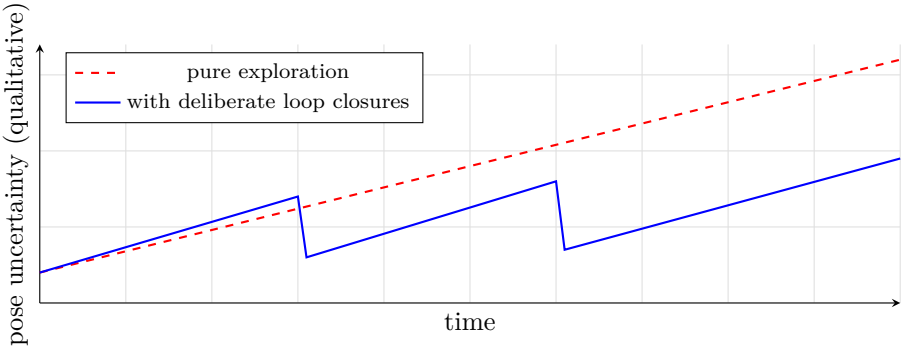


Figure 199: Exploration–exploitation (qualitative sketch): exploring grows uncertainty; revisiting a known place closes a loop and resets it.

5.4 Active SLAM with an Active Stereo Head *(slide 37)*

The work that named active SLAM: a wheeled robot carries a stereo head with four degrees of freedom. An EKF estimates the robot and a map of point features, and the head fixates one feature at a time; it decides where to look, not where to move. For each visible feature i the EKF predicts the measurement and its innovation covariance $\mathbf{S}_i$. The head fixates the feature whose position is hardest to predict, the one with the largest 3σ ellipsoid:

$$i^\star = \arg\max_i \ \mathrm{Vol}_{3\sigma}(\mathbf{S}_i) = \arg\max_i \ \det(\mathbf{S}_i)^{1/2}.$$

Intuition

It is MonoSLAM's search ellipse turned into a decision: look at the thing you are *least sure about*, because a measurement there reduces uncertainty the most. The ellipsoid volume is $\frac{4}{3}\pi\, 3^3\sqrt{\det \mathbf{S}_i}$ for a 3D innovation, so ranking by $\det \mathbf{S}_i$ is the same as ranking by volume.

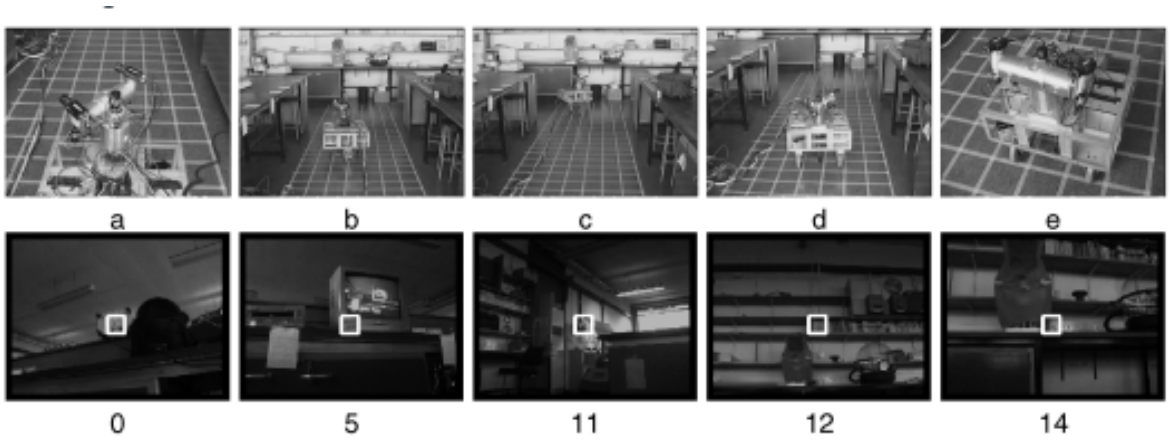


Figure 200: The robot and its active head (top); fixated views of features 0, 5, 11, 12 and 14, each marked by a white square (bottom).

5.5 Active Localization: Learning Where to Look *(slide 38)*

Active localization chooses where the camera should look so that it localizes accurately in a known map, built by structure-from-motion.

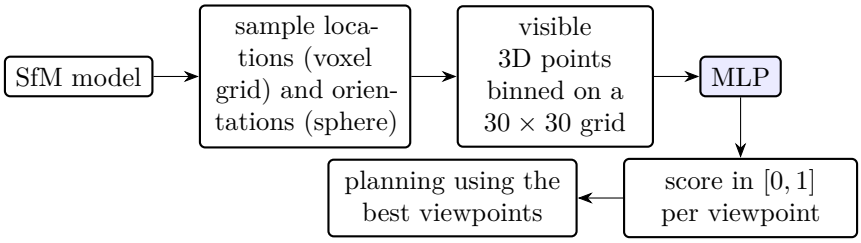


Figure 201: Active localization pipeline (redrawn from slide 38).

- **Candidate viewpoints:** positions on a voxel grid, each with orientations sampled on the sphere; for each, the visible 3D points are binned on a 30×30 grid and a small MLP scores the viewpoint between 0 and 1.
- **Self-supervised:** each training viewpoint is labelled by registering an image rendered from it against the map; one inference takes 0.02 s.
- 72.9% of 92k test viewpoints localize within 5 cm and 0.4°, against 60.7% for the best choice by Fisher information.

Intuition

That is about 67,000 good viewpoints against 56,000, roughly 11,000 more, and 0.02 s per scoring makes planning interactive. The lesson of the slide: *where the points lie in the image* matters more than the Fisher information matrix, which only counts how much information there is, not whether it is spread well enough to constrain pose.

5.6 Active View Selection with Radiance Fields *(slide 39)*

A **radiance field** is a scene model whose parameters $\boldsymbol{\theta}$ hold radiance and density at points in space, rendered into the image seen from a camera pose by volume rendering. **FisherRF** picks the next camera pose $\mathbf{x}$ by its expected information gain about $\boldsymbol{\theta}$. Let $\mathbf{I}_\mathbf{x} = \mathbf{J}_\mathbf{x}^\top \mathbf{J}_\mathbf{x}$ be the Fisher information of the view ($\mathbf{J}_\mathbf{x}$ the Jacobian of its rendered image with respect to $\boldsymbol{\theta}$; no image from that view is needed) and $\mathbf{I}_\mathcal{D} = \sum_{\mathbf{x}' \in \mathcal{D}} \mathbf{I}_{\mathbf{x}'}$ the sum over the views already captured.

Proposition 5.1 (Trace bound on the information gain)**.** *For a candidate view* $\mathbf{x}$ *with* $\mathbf{I}_\mathbf{x} \succeq 0$ *and an (invertible, regularized)* $\mathbf{I}_\mathcal{D} \succ 0$*,*

$$\mathrm{IG}(\mathbf{x}) = \tfrac{1}{2} \log\det\!\left(\mathbf{I} + \mathbf{I}_\mathbf{x} \mathbf{I}_\mathcal{D}^{-1}\right) \;\le\; \tfrac{1}{2} \operatorname{tr}\!\left(\mathbf{I}_\mathbf{x} \mathbf{I}_\mathcal{D}^{-1}\right).$$

Proof. $\mathbf{I}_\mathbf{x}\mathbf{I}_\mathcal{D}^{-1}$ is similar to the PSD matrix $\mathbf{I}_\mathcal{D}^{-1/2}\mathbf{I}_\mathbf{x}\mathbf{I}_\mathcal{D}^{-1/2}$, so it has eigenvalues $\mu_k \ge 0$. Then $\log\det(\mathbf{I}+\cdot) = \sum_k \log(1+\mu_k) \le \sum_k \mu_k = \operatorname{tr}(\cdot)$ because $\log(1+\mu) \le \mu$. □

With over 200 million parameters, both matrices are kept *diagonal*, so the bound becomes $\frac{1}{2}\sum_k (\mathbf{I}_\mathbf{x})_{kk}/(\mathbf{I}_\mathcal{D})_{kk}$. With 3D Gaussian Splatting (a scene of 3D Gaussians), views are evaluated at 70 fps.

Intuition

Toy numbers for 3 parameters: $\mathrm{diag}(\mathbf{I}_\mathbf{x}) = (1, 4, 0.1)$ and $\mathrm{diag}(\mathbf{I}_\mathcal{D}) = (10, 1, 0.1)$ give ratios $(0.1, 4, 1)$, bound $\frac{1}{2} \cdot 5.1 = 2.55$. The view is valuable mostly because it informs the second parameter, which previous views barely constrained. In words: *go where the model is least certain and the new image would speak the loudest.*

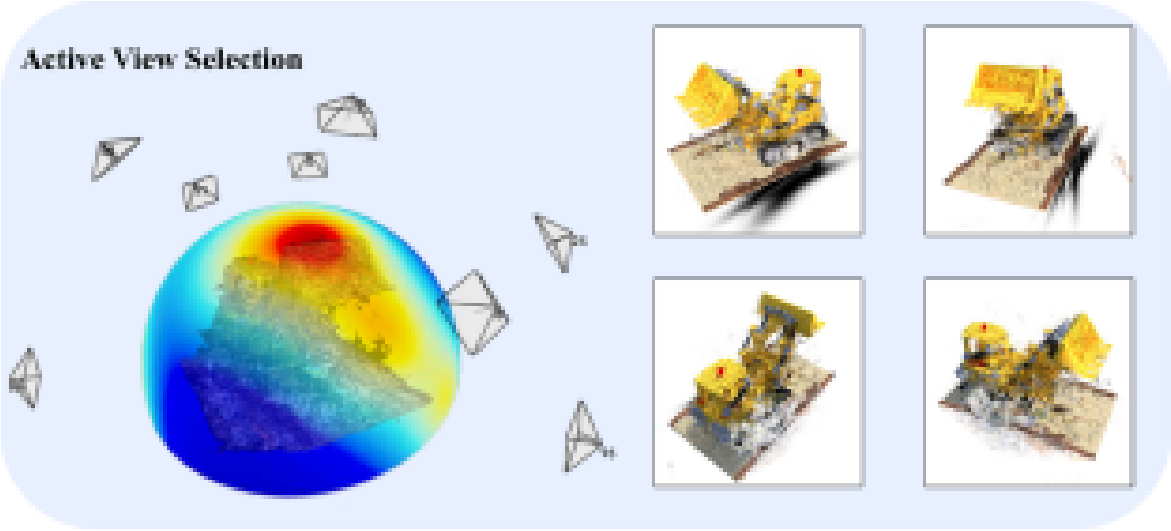


Figure 202: Information gain over the viewing sphere of a scene, with candidate views (frusta) and four of their renders.

5.7 Outlook: Learned Front Ends *(slide 40)*

Two recent systems keep the back-end machinery of this lecture, least squares over a graph of relative constraints on a Lie group, and replace the feature front end by a network from earlier lectures.

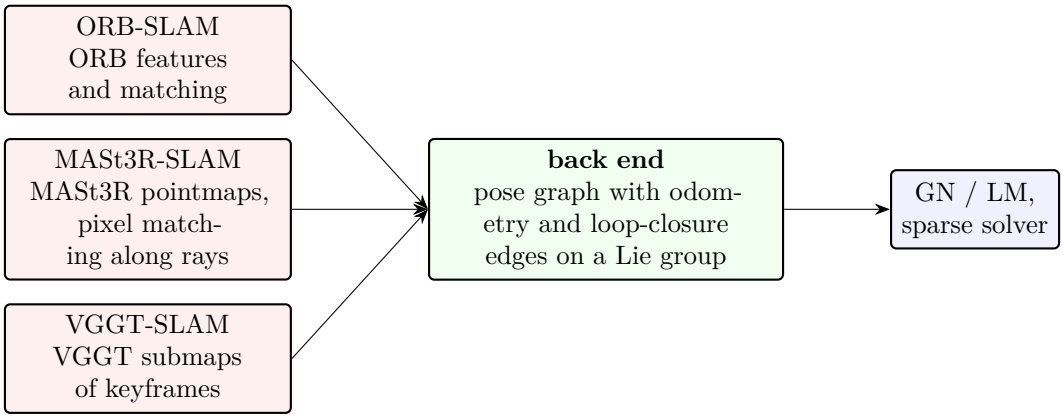


Figure 203: Same back end, different front ends (slide 40).

	MASt3R-SLAM	**VGGT-SLAM**
Front end	MASt3R (lecture 04) predicts pointmaps of each frame and its keyframe; pixels matched along rays	VGGT (lecture 05) reconstructs a submap of keyframes in one pass
Group	keyframe poses in Sim(3)	each submap gets a homography in SL(4) (15 DoF)
Solver	Gauss–Newton, sparse Cholesky, on ray errors; loops by image retrieval	pose graph with odometry and loop-closure edges, Levenberg–Marquardt
Speed / camera	about 15 FPS on one GPU; no camera model beyond a single centre	without calibration, submaps agree only up to a projective transformation

Intuition

Degrees of freedom of the group tell how much ambiguity the front end leaves: SE(3) has 6 (metric, e.g. stereo), Sim(3) has 7 (one camera: unknown scale), and SL(4) has 15 (an *uncalibrated* reconstruction, where even angles and parallelism are not trustworthy and each submap needs a full projective correction). The back end is the same sparse least squares of Section 6.3; only the group and the residual change.

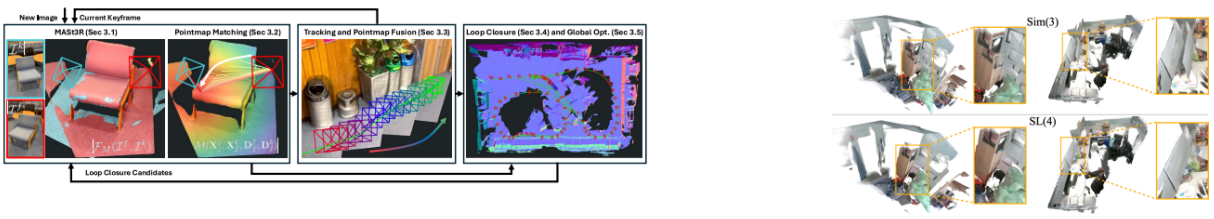


Figure 204: MASt3R-SLAM pipeline (left) and VGGT-SLAM Sim(3) vs. SL(4) comparison renderings (right).

6 Summary *(slide 41)*

Method	Estimate / variables	Cost	Loops / drift	Notes
EKF-SLAM, MonoSLAM	current pose (13) + N points, one Gaussian	$O(N^2)$ per step and memory; ≈ 100 features at 30 Hz	no explicit loop mechanism; past poses marginalized	linearized once; may diverge
Factor graph / pose graph	all keyframe poses (+ points), MAP = sparse nonlinear LS	GN/LM on sparse $\mathbf{H}$ (Cholesky, CG)	loop edge between non-successive poses	relinearizes; needs good initial guess
PTAM	tracking thread + mapping thread, BA on keyframes	BA outside the real-time loop	no large-scale loop closing	small AR workspaces
ORB-SLAM	ORB features; 3 threads; local BA; essential graph over Sim(3)	local BA bounded; 21.6 ms/image loop detection	bags of binary words; Sim(3) corrects scale drift	relocalization $\approx 78\%$ of frames
Continuous-time splines	control poses of a cumulative B-spline	sliding window of control poses	–	rolling shutter: 4.7% $\rightarrow$ 0.57% error
Active SLAM	choose actions by utility (D-opt)	utility per candidate action	revisit vs. explore	active head: largest 3σ ellipsoid
MASt3R-SLAM / VGGT-SLAM	keyframes on Sim(3) / submaps on SL(4)	GN+Cholesky / LM; ≈ 15 FPS (MASt3R)	image retrieval / pose graph loops	learned front end, same back end

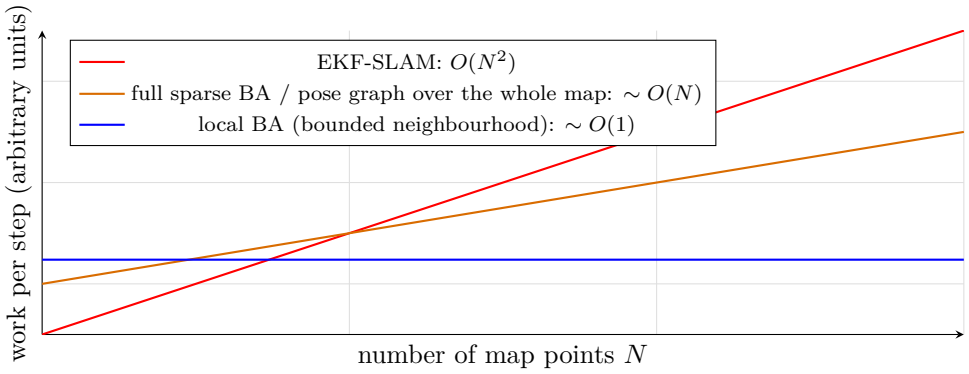


Figure 205: Qualitative scaling of the per-step cost with the map size: the filter pays quadratically for correlations, sparse least squares roughly linearly, and keyframe-based local BA stays bounded (constants are illustrative only).

Key take-aways

- **SLAM = joint MAP estimation** of trajectory and map from sequential frames; it is SfM with a real-time budget and loop closure.
- **Filtering** (Bayes filter $\rightarrow$ KF $\rightarrow$ EKF-SLAM) keeps one Gaussian over the current pose and map; marginalizing past poses makes the covariance dense, so the cost is $O(N^2)$ and the map must stay tiny (about 100 features at 30 Hz).
- **Graph-based SLAM** keeps the keyframes: MAP on a factor graph is sparse nonlinear least squares, solved by Gauss–Newton/LM; the sparsity pattern of $\mathbf{H}$ equals the graph adjacency.
- **A modern pipeline** (ORB-SLAM) runs tracking (every frame), local mapping (local BA on the covisibility graph) and loop closing (bags of binary words, Sim(3) pose graph to fix scale drift) in parallel.
- **Research directions** change the front end or the problem statement: continuous time for rolling shutter, active viewpoint selection, and learned front ends (MASt3R, VGGT) on top of the same least-squares back end.

Part VI

Lecture 7

About these notes. The structure follows the slide deck; every section title is (nearly) the slide title, with the slide range in italics. No companion book chapter was provided, so the formal material (definitions, propositions, proofs) is derived from the equations on the slides, and I flag explicitly whenever I add something that is not on the slides (for instance the normalisation of the phase function, or the partition-of-unity identity for the ray-marching weights). Two typos on the slides are corrected in remarks (slides 53 and 68). Photographs are left as blank frames with the slide number and the caption to use. All figures are original TikZ/pgfplots.

1 From One View to Many *(slides 2–3)*

1.1 From One View to Many *(slides 2)*

[PHOTO PLACEHOLDER]
Insert the image from **slide 2**:
Input images, 3D surface, light & reflectance and cameras recovered by IDR. Figure credits: Yariv et al.

5.5

The lectures so far dealt with one or two views (single-view depth, stereo). From now on we have *many* images of the same scene with known (or estimated) cameras, and the goal is stated on the slide:

Definition 1.1 (Multi-view objective)**.** Find a *learnable* 3D representation $\mathcal{S}_\theta$ that is *consistent with all observed views*: if we render $\mathcal{S}_\theta$ from the camera of image I_k, we should reproduce I_k, for every k.

Intuition

Think of the scene as an unknown object hidden in a box and the photos as "shadows" of it. Any representation (mesh, points, field, ...) is acceptable as long as, when we "shine light" through it from camera k, we get image k back. The words *learnable* and *rendering* are the two ingredients of the course from here on: the representation has parameters we can optimise, and a differentiable renderer links it to pixels. The next lectures cover different representations and optimisation methods; **today we study the renderer** for one family of representations (density fields).

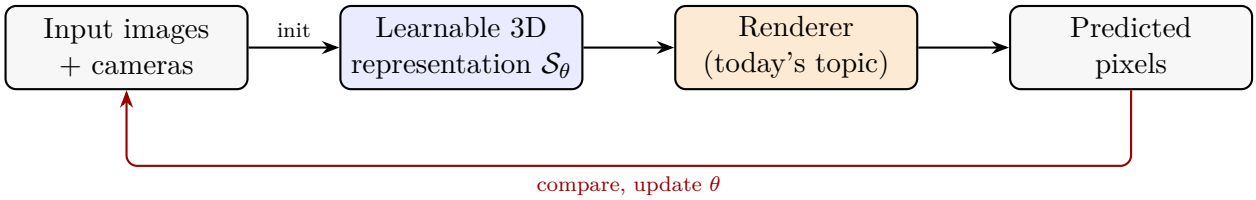


Figure 206: The analysis-by-synthesis loop behind multi-view learning.

1.2 The Novel View Synthesis Problem *(slides 3)*

Definition 1.2 (Novel view synthesis (NVS))**.** Given a set of images $\{(I_k, \text{camera}_k)\}_{k=1}^{K}$ of a scene, synthesise the image $I_\star$ that a camera at a *new* pose would have captured.

The slide shows the classical example of Chen & Williams (1993): the images in the corners are real and all others are interpolated from the nearest three source images (the centre image from the two upper corners and the lower-left corner).

[PHOTO PLACEHOLDER]
Insert the image from **slide 3**:
2D view interpolation: source images in the corners, all others interpolated from the nearest three. Figure credits: Chen & Williams 1993.

6.5

Intuition

Before neural nets, NVS meant *interpolating between photos.* The modern view: instead of blending pixels directly, recover a 3D model and re-render it. The question this lecture answers is: *what kind of 3D model, and how do we turn it into pixels?*

2 Beyond Surfaces *(slides 4–11)*

2.1 Up to today *(slides 4)*

Until now our observations were equivalent to *surfaces*: meshes, point clouds, surface fields (implicit surfaces / occupancy grids). The slide ends with a joke (the Vsauce "Or is it?" frame): is that really all there is?

2.2 Beyond Surfaces *(slides 5–6)*

The slide asks: *is the world made only of opaque surfaces?* The examples (slide 6) are mist over a hillside, sunbeams through a forest and honey.

[**PHOTO PLACEHOLDER**]
Insert the image from **slide 6**:
Beyond surfaces: fog, light shafts in a forest, honey.

4.5

2.3 Media Are Common *(slides 7)*

Participating media (fog, smoke, clouds, nebulae, translucent minerals) are everywhere.

[**PHOTO PLACEHOLDER**]
Insert the image from **slide 7**:
Media are common: atmosphere, clouds, nebulae, translucent objects. Figure credits: I. Gkioulekas and W. Jarosz.

5.5

2.4 What About Reflections? *(slides 8)*

The slide plays the NeRF video of a car (Mildenhall et al., ECCV 2020) with the question: *do we really need surfaces?* A shiny car body shows different colours from different directions; no single surface point "owns" that colour.

[**PHOTO PLACEHOLDER**]
Insert the image from **slide 8**:
Do we really *need surfaces? Video credits: Mildenhall et al., ECCV 2020 (NeRF).*

5.5

2.5 Dropping the Surface Assumption *(slides 9)*

For fog, flames, jade and reflections, no single surface point explains the pixel. The slide proposes a change of question:

Stop asking "where is the surface?" $\longrightarrow$ **ask** "how much matter is at every point in space?"

Intuition

A surface becomes a *special case*: the density jumps from zero (empty air) to "opaque" within a very thin layer. Figure 207 contrasts the two: an opaque surface is a step in density, fog is a soft bump. If our representation is the *density function*, both live in the same family, and we never have to decide in advance which one we are looking at.

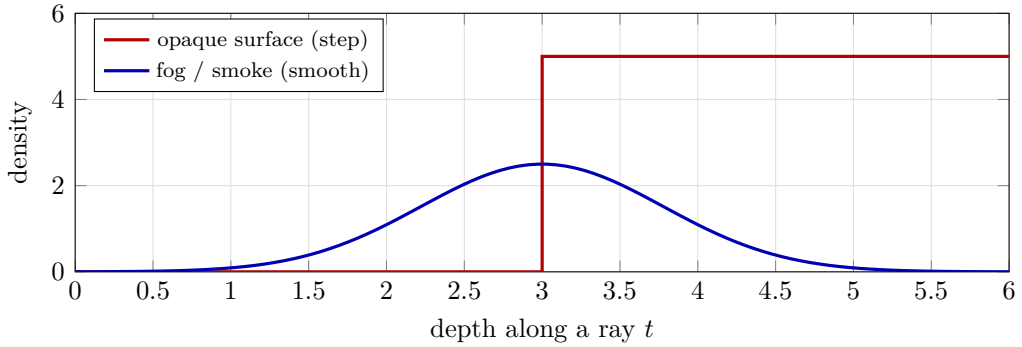


Figure 207: A surface is the limit of a density that rises from zero to "very large" in an infinitesimally thin layer.

2.6 Density Fields *(slides 10–11)*

Definition 2.1 (Density field)**.** A *density field* (volume) is a non-negative function over space, either stored on a grid or defined continuously:

$$V[x, y, z] \in \mathbb{R}^+ \quad \text{(discrete volume)}, \qquad f(\mathbf{p}) \in \mathbb{R}^+ \quad \text{(continuous field, } \mathbf{p} \in \mathbb{R}^3\text{)}.$$

The slide shows smoke (a simulation) and a CT scan (a physical density map), and states **today's goal**: *how do we render this representation?*

[PHOTO PLACEHOLDER]
Insert the image from **slide 11**:
NeRF objects (chair, Lego bulldozer, materials, drums, microphone, hotdog, ship, ficus). Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

6

Intuition

A CT scan is the perfect mental model: each voxel stores "how much X-ray gets absorbed here". A camera is like an X-ray machine for *visible light*: it needs to integrate something along each ray. Rendering = turning the 3D table of numbers into a 2D image.

3 Sensor Physics *(slides 12–16)*

Slide 12 introduces the question: *how do radiance fields interact with a camera, and how are pixel colours formed?*

3.1 Radiance *(slides 13)*

Definition 3.1 (Radiance (luminance))**.** *Radiance* $L(\boldsymbol{x}, \boldsymbol{\omega})$ is the photometric measure of the luminous intensity *per unit area* of light travelling in a given direction $\boldsymbol{\omega}$ at the point $\boldsymbol{x}$ (more precisely: per unit area perpendicular to $\boldsymbol{\omega}$ and per unit solid angle $\mathrm{d}\boldsymbol{\omega}$, see Fig. 208).

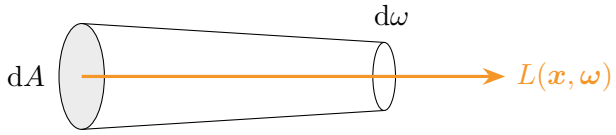


Figure 208: Radiance: power flowing through an area element $\mathrm{d}A$ into a cone $\mathrm{d}\omega$ around direction $\boldsymbol{\omega}$. Figure idea: P. Hanrahan.

Intuition

Radiance is "what one pixel of a camera would measure". Crucially it is *constant along a ray in empty space* (a wall looks equally bright from 1 m or 5 m away). So a ray through vacuum carries one number per direction—that is what makes pixel $\leftrightarrow$ ray a clean correspondence. The whole lecture is about what happens to this number when the ray is *not* in vacuum.

3.2 Surface Rendering *(slides 14)*

Let $\boldsymbol{x}^*$ be the optical centre and $\boldsymbol{\omega}$ the direction from $\boldsymbol{x}^*$ towards the sensor. For an opaque surface, the pixel value is proportional to the radiance emitted from the first surface point seen along the ray:

$$\text{pixel} \;\propto\; L(\boldsymbol{x}^*, \boldsymbol{\omega}) \;=\; L(\boldsymbol{x}^* - \lambda\boldsymbol{\omega},\; \boldsymbol{\omega}),$$

where $\lambda > 0$ is the distance to the surface along the ray.

Intuition

For surfaces the pixel is a *single-point lookup*: find λ (the depth!), evaluate the emitted radiance (shading, texture) at that point, done. This is why depth estimation was central in the previous lectures.

3.3 A Ray Through a Medium *(slides 15–16)*

If a medium (a cloud) sits between the surface and the camera, the equality breaks:

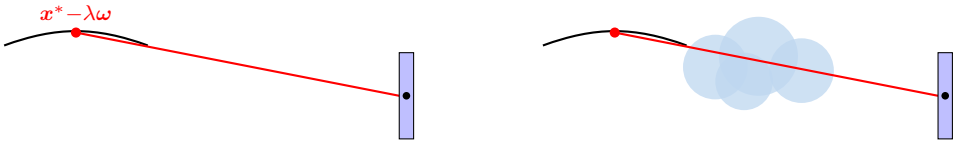
$$\text{pixel} \;\propto\; L(\boldsymbol{x}^*, \boldsymbol{\omega}) \;\neq\; L(\boldsymbol{x}^* - \lambda\boldsymbol{\omega},\; \boldsymbol{\omega}).$$

The pixel value *cannot* be reduced to the emitted radiance of a single surface point. Slide 16 phrases the new question:

How does $L(\boldsymbol{x}, \boldsymbol{\omega})$ vary along a ray?

4 Volumetric Interactions *(slides 17–24)*

Slide 17 introduces the next block: *how different media perturb the luminance.* Light interacting with a medium can be absorbed, emitted, or scattered.



surface: one point decides medium: the whole ray decides

Figure 209: Surface rendering vs. rendering through a medium: the red ray accumulates the effect of everything it crosses.

4.1 Absorption *(slides 18–19)*

Light travelling in a medium can get absorbed by it. Examples (slide 18): fog and dense black smoke. Slide 19 quotes Leonardo da Vinci's *Treatise on Painting* (c. 1480): to make an object look five times as distant, paint it five times bluer—an early description of aerial perspective, i.e. distance-dependent attenuation.

[PHOTO PLACEHOLDER]
Insert the image from **slide 18**:
Absorption: fog and a smoke plume. Figure credits: I. Gkioulekas.

4.8

[PHOTO PLACEHOLDER]
Insert the image from **slide 19**:
Absorption: da Vinci's self-portrait and Virgin of the Rocks *with the "five times bluer" quote (*Treatise on Painting*, c. 1480). Figure credits: I. Gkioulekas.*

5.5

Intuition

The longer the path through the medium, the less of the original light survives; and since different wavelengths are attenuated differently, distant things shift colour (the blue haze in the paintings). This is the first hint of the exponential law we derive in Section 6.

4.2 What else? *(slides 20)*

Light can also be *added* by the medium.

4.3 Emission *(slides 21)*

Hot gases (explosions, flames) emit their own light along the ray.

[PHOTO PLACEHOLDER]
Insert the image from **slide 21**:
Emission: fireballs. Figure credits: I. Gkioulekas.

4.8

4.4 What else? *(slides 22)*

Light shafts through a misty forest show something that is neither absorption nor emission.

[PHOTO PLACEHOLDER]
Insert the image from **slide 22**:
Light shafts through mist at sunrise ("What else?").

5

4.5 Scattering *(slides 23)*

Light along a ray can depend on light in other directions. In the atmosphere (slide figure) a sunbeam hits a molecule and is redirected towards the observer; rainbows are another example.

[PHOTO PLACEHOLDER]
Insert the image from **slide 23**:
Scattering: sun–atmosphere–observer sketch and a double rainbow. Figure credits: I. Gkioulekas.

5

4.6 Volume Rendering a Ray *(slides 24)*

How does $L(\boldsymbol{x}, \boldsymbol{\omega})$ vary along a ray? Emission, absorption and scattering.

The following figure summarises the four elementary mechanisms (they are formalised in the next section).

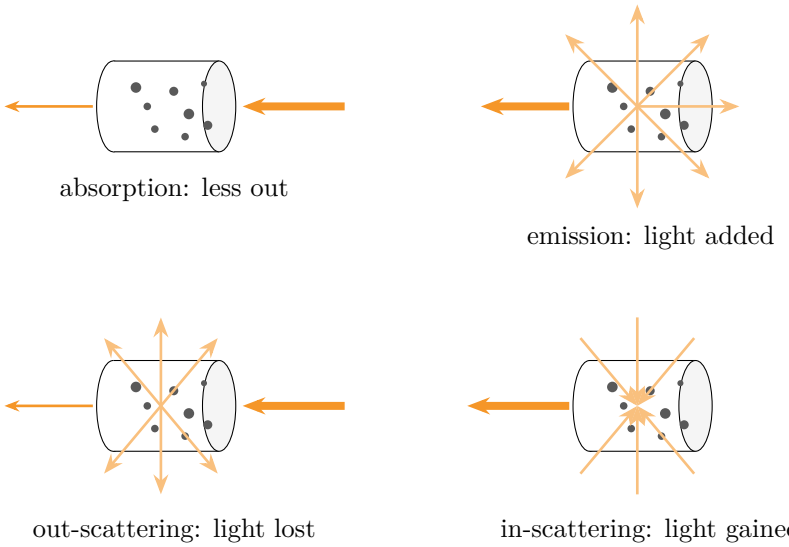


Figure 210: Interactions of light with a small cylinder of medium (thick orange = strong radiance, thin = weak).

5 Dissecting Volume Rendering *(slides 25–39)*

Today's goals (slide 25). (1) Mathematical models of emission and absorption; (2) the volume rendering equation and transmittance; (3) computational algorithms for volume rendering; (4) next lecture: combine with neural representations.

5.1 Volume Rendering a Ray *(slides 26)*

Consider a thin cylinder of medium, cross-section $\mathrm{d}A$ and length z, with the camera at one end. The slide asks: *how much light is lost or gained along the differential beam because of the interactions of light with the medium?* We study an *infinitesimal* slice of length $\mathrm{d}z$ and later integrate.

5.2 Volume Rendering a Ray: Absorption *(slides 27–28)*

Definition 5.1 (Absorption coefficient)**.** The *absorption coefficient* $\sigma_a(\boldsymbol{x}) \geq 0$ (units: length^{-1}) is the fraction of radiance absorbed per unit length at $\boldsymbol{x}$:

$$\mathrm{d}L(\boldsymbol{x}, \boldsymbol{\omega}) \;=\; -\sigma_a(\boldsymbol{x})\, L(\boldsymbol{x}, \boldsymbol{\omega})\, \mathrm{d}z.$$

Simple derivation of the absorption equation (slide 28)

Assume the slice of length $\mathrm{d}z$ contains opaque spherical particles of radius r in empty space, with ρ particles per unit volume (Fig. 211).

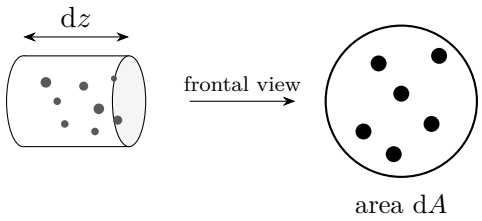


Figure 211: Opaque particles seen along the beam: each blocks a disc of area πr^2.

Lemma 5.2 (Absorption by opaque particles)**.** *Let f be the fraction of the cross-section $\mathrm{d}A$ covered by particles in the slice. Then $f = \rho\pi r^2\,\mathrm{d}z$, $L_o = (1-f)L_i$ and therefore*

$$\mathrm{d}L = L_o - L_i = -fL_i = -(\rho\pi r^2)\, L\,\mathrm{d}z = -\sigma_a L\,\mathrm{d}z, \qquad \sigma_a = \rho\pi r^2.$$

Proof. The slice contains $\rho\,\mathrm{d}A\,\mathrm{d}z$ particles, each hiding area πr^2 (assuming $\mathrm{d}z$ so small that discs do not overlap in the frontal view). The covered fraction is $f = (\rho\,\mathrm{d}A\,\mathrm{d}z)\,\pi r^2/\mathrm{d}A = \rho\pi r^2\mathrm{d}z$. Only the uncovered fraction $1-f$ of the incoming radiance L_i gets through. □

Intuition

$\sigma_a = \rho \cdot \pi r^2$ is *(how many particles per volume)* $\times$ *(how big a shadow each casts)*: denser or bigger particles $\Rightarrow$ larger σ_a. Worked number: if $\sigma_a = 0.5\,\mathrm{m}^{-1}$ then a slice of $\mathrm{d}z = 1\,\mathrm{cm}$ removes $0.5 \cdot 0.01 = 0.5\%$ of the light. Tiny per slice, but a ray crosses many slices (that is what the integral of Section 6 does).

5.3 Volume Rendering a Ray: Emission *(slides 29)*

Definition 5.3 (Emission term)**.** Let $L_e(\boldsymbol{x}, \boldsymbol{\omega})$ be the radiance emitted by the medium at $\boldsymbol{x}$ in direction $\boldsymbol{\omega}$. The gain along a slice is

$$\mathrm{d}L(\boldsymbol{x}, \boldsymbol{\omega}) = \sigma_a(\boldsymbol{x})\, L_e(\boldsymbol{x}, \boldsymbol{\omega})\, \mathrm{d}z.$$

Intuition

Note the factor σ_a: emission is weighted by the same coefficient as absorption. Where there is no matter ($\sigma_a = 0$) nothing can glow; where there is a lot, the glow is strong. It also makes the convention "L_e is the colour of the stuff, σ_a is how much stuff" very natural, which is exactly how NeRF splits its outputs (colour and density).

5.4 Emission-Absorption Model *(slides 30)*

Putting the two together:

$$\mathrm{d}L(\boldsymbol{x}, \boldsymbol{\omega}) = \underbrace{-\sigma_a(\boldsymbol{x}) L(\boldsymbol{x}, \boldsymbol{\omega}) \mathrm{d}z}_{\text{losses}} + \underbrace{\sigma_a(\boldsymbol{x}) L_e(\boldsymbol{x}, \boldsymbol{\omega}) \mathrm{d}z}_{\text{gains}}.$$

5.5 The Out-Scattering Term *(slides 31)*

Definition 5.4 (Scattering coefficient, out-scattering)**.** Light can be deflected away from the ray. With the *scattering coefficient* $\sigma_s(\boldsymbol{x})$:

$$\mathrm{d}L(\boldsymbol{x}, \boldsymbol{\omega}) = -\sigma_s(\boldsymbol{x})\, L(\boldsymbol{x}, \boldsymbol{\omega})\, \mathrm{d}z.$$

It is mathematically identical to absorption (a loss proportional to the incident radiance); the only difference is where the lost energy goes: absorption turns it into heat, out-scattering sends it into other directions.

5.6 The In-Scattering Term *(slides 32–33)*

Definition 5.5 (In-scattering and phase function)**.** Light coming from other directions $\boldsymbol{\omega}'$ can be redirected into the ray direction $\boldsymbol{\omega}$:

$$\mathrm{d}L(\boldsymbol{x}, \boldsymbol{\omega}) = \sigma_s(\boldsymbol{x})\, L_s(\boldsymbol{x}, \boldsymbol{\omega})\, \mathrm{d}z, \qquad L_s(\boldsymbol{x}_t, \boldsymbol{\omega}) = \int_{\mathbb{S}} f_p(\boldsymbol{x}_t, \boldsymbol{\omega}', \boldsymbol{\omega})\, L_i(\boldsymbol{x}_t, \boldsymbol{\omega}')\, \mathrm{d}\boldsymbol{\omega}'.$$

Here f_p is the *phase function* (the probability density that light arriving from $\boldsymbol{\omega}'$ leaves towards $\boldsymbol{\omega}$; by the usual convention it integrates to one over $\boldsymbol{\omega}$, which is not stated on the slide).

The slide's warning: *unfortunately L_s is non-trivial*, and "L_i can also depend on other scattered light!".

Intuition

To know the light along *this* ray, I need the light along *all* rays through the point $\boldsymbol{x}_t$ (a sphere of directions). But each of those rays has the same problem, and so on: the unknown L appears on both sides of the equation, inside an integral over a sphere, recursively. That is why full scattering is expensive (Monte-Carlo path tracing in graphics), and why the next section asks whether we can live without it.

5.7 The Radiative Transfer Equation *(slides 34)*

Theorem 5.6 (Radiative transfer equation (RTE))**.** *Collecting the four mechanisms,*

$$\mathrm{d}L(\boldsymbol{x},\boldsymbol{\omega}) = \underbrace{-\sigma_a(\boldsymbol{x})L(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z - \sigma_s(\boldsymbol{x})L(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z}_{\text{losses: absorption, out-scattering}} + \underbrace{\sigma_a(\boldsymbol{x})L_e(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z + \sigma_s(\boldsymbol{x})L_s(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z}_{\text{gains: emission, in-scattering}}.$$

Intuition

Four knobs, two losses and two gains, exactly the panels of Fig. 210. It is the "energy bookkeeping" of a beam crossing a small piece of fog.

5.8 The Emission-Absorption Model *(slides 35–36)*

Slide 35 is the section divider *"7.5 The Emission-Absorption Model: do we really need scattering?"*. Slide 36 "drops scattering" (the grayed terms) and keeps only

$$\mathrm{d}L(\boldsymbol{x},\boldsymbol{\omega}) = -\sigma_a(\boldsymbol{x})L(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z + \sigma_a(\boldsymbol{x})L_e(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z,$$

noting that this is **the de-facto standard in neural volume rendering methods**. The slide then asks: *isn't this too restrictive?*

5.9 Expressivity of the EA Model *(slides 37)*

The answer is: *not under an arbitrarily allowed* L_e'.

Proposition 5.7 (Scattering can be absorbed into the emission term)**.** *Define the* effective emission

$$L_e' = L_e + \frac{\sigma_s}{\sigma_a}(L_s - L).$$

Then the full RTE of Theorem 5.6 is a pure emission–absorption equation with absorption coefficient σ_a *and emission* L_e'*:* $\mathrm{d}L = -\sigma_a L\,\mathrm{d}z + \sigma_a L_e'\,\mathrm{d}z$.

Proof. Factor σ_a out of the four terms: $-\sigma_a L - \sigma_s L + \sigma_a L_e + \sigma_s L_s = -\sigma_a L + \sigma_a\big[L_e + \frac{\sigma_s}{\sigma_a}(L_s - L)\big]$. □

Intuition

A learner that is free to output *any* colour at each point and for each viewing direction can mimic scattering: whatever scattering would add or remove at a point is simply baked into the colour it outputs there. The catch: L_e' now depends on L, L_s and the viewing direction, so it is a complicated function—it needs a flexible (neural) colour model, and the learned colour will not be physically *relightable*. This is the bridge to NeRF, where the colour network takes the view direction as input.

5.10 Why Model Scattering at All *(slides 38)*

Simple scenes with a more "complete" rendering model can produce interesting effects (slide: two glasses of translucent liquid with glowing edges), whereas one would need complex, inhomogeneous scenes to reproduce them in an EA model. Scattering models also allow **relighting**, which is the focus of lesson 11.

[PHOTO PLACEHOLDER]
Insert the image from **slide 38**:
Two glasses of translucent liquid rendered with scattering. Figure credits: I. Gkioulekas.

5.11 Notation From Here On *(slides 39)*

From now on we drop the subscript: $\sigma_a \equiv \sigma$, and we work with

$$\mathrm{d}L(\boldsymbol{x},\boldsymbol{\omega}) = -\sigma(\boldsymbol{x})\,L(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z + \sigma(\boldsymbol{x})\,L_e(\boldsymbol{x},\boldsymbol{\omega})\mathrm{d}z.$$

6 From Infinitesimal to Finite Lengths *(slides 40–54)*

Slide 40 is the divider "7.6 From Infinitesimal to Finite Lengths".

6.1 From infinitesimal to finite lengths *(slides 41)*

The plan: (1) model the *absorption-only* case; (2) then allow emission.

6.2 Absorption-only Volume Rendering *(slides 42)*

For a *homogeneous* medium (uniform density σ), $\mathrm{d}L = -\sigma L\,\mathrm{d}z$.

Proposition 6.1 (Beer–Lambert law)**.** *In a homogeneous absorbing medium, light travelling a distance z is attenuated exponentially:*

$$L(\boldsymbol{x}_0 + \boldsymbol{\omega}z, \boldsymbol{\omega}) = e^{-\sigma z}\,L(\boldsymbol{x}_0, \boldsymbol{\omega}).$$

Proof. Along the ray, $\ell(z) := L(\boldsymbol{x}_0 + \boldsymbol{\omega}z, \boldsymbol{\omega})$ satisfies $\ell'(z) = -\sigma\ell(z)$, a linear ODE with solution $\ell(z) = \ell(0)e^{-\sigma z}$. □

Intuition

Per unit length a fixed *fraction* of the surviving light is removed, so the loss compounds like interest at rate σ. Numbers: with $\sigma = 0.5\,\mathrm{m}^{-1}$, after 2 m only $e^{-1} \approx 37\%$ survives, after 6 m only $e^{-3} \approx 5\%$.

6.3 Absorption Only, Heterogeneous Medium *(slides 43)*

If σ varies in space, $\mathrm{d}L = -\sigma(\boldsymbol{x})L\,\mathrm{d}z$ is still solvable.

Definition 6.2 (Transmittance)**.** For two points $\boldsymbol{x}, \boldsymbol{y}$, let $\boldsymbol{\omega}_{xy} = (\boldsymbol{y}-\boldsymbol{x})/\|\boldsymbol{y}-\boldsymbol{x}\|$. The *transmittance*

$$T(\boldsymbol{x},\boldsymbol{y}) = \exp\Big(-\int_{t=0}^{\|\boldsymbol{x}-\boldsymbol{y}\|} \sigma(\boldsymbol{x}+\boldsymbol{\omega}_{xy}t)\,\mathrm{d}t\Big)$$

is the fraction of the radiance leaving $\boldsymbol{x}$ towards $\boldsymbol{y}$ that reaches $\boldsymbol{y}$ along a straight line, under the absorption-only model. For a homogeneous medium $T(\boldsymbol{x},\boldsymbol{y}) = e^{-\sigma\|\boldsymbol{x}-\boldsymbol{y}\|}$.

Proposition 6.3 (Absorption-only solution, heterogeneous medium)**.** $L(\boldsymbol{x}_0 + \boldsymbol{\omega}z, \boldsymbol{\omega}) = T(\boldsymbol{x}_0, \boldsymbol{x}_0 + \boldsymbol{\omega}z)\,L(\boldsymbol{x}_0, \boldsymbol{\omega})$*, i.e.* $L(\boldsymbol{x}_0 + \boldsymbol{\omega}z, \boldsymbol{\omega}) = e^{-\int_0^z \sigma(\boldsymbol{x}_0+\boldsymbol{\omega}t)\mathrm{d}t}L(\boldsymbol{x}_0, \boldsymbol{\omega})$.

Proof. Let $\ell(s) = L(\boldsymbol{x}_0 + \boldsymbol{\omega}s, \boldsymbol{\omega})$ and $\tau(s) = \int_0^s \sigma(\boldsymbol{x}_0 + \boldsymbol{\omega}t)\mathrm{d}t$ (the *optical depth*). The ODE is $\ell' = -\sigma\ell$. Then $\frac{\mathrm{d}}{\mathrm{d}s}\big(e^{\tau(s)}\ell(s)\big) = e^{\tau}(\sigma\ell + \ell') = 0$, so $e^{\tau(z)}\ell(z) = \ell(0)$. □

Intuition

The integral $\tau = \int \sigma$ simply *adds up the opacity met along the way*; the exponential converts that total into a surviving fraction. It does not matter *where* the dense bits are, only their total: a thin dense slab and a long thin haze with equal $\int \sigma$ dim the light equally.

6.4 Transmittance *(slides 44–45)*

Lemma 6.4 (Multiplicativity)**.** *For $\boldsymbol{z}$ on the segment between $\boldsymbol{x}$ and $\boldsymbol{y}$: $T(\boldsymbol{x},\boldsymbol{y}) = T(\boldsymbol{x},\boldsymbol{z})\,T(\boldsymbol{z},\boldsymbol{y})$.*

Proof. The integral of σ over a segment is additive over its two sub-segments, and $e^{a+b} = e^a e^b$. □

Lemma 6.5 (Symmetry (the slide-44 question))**.** $T(\boldsymbol{x},\boldsymbol{y}) = T(\boldsymbol{y},\boldsymbol{x})$.

Proof. Both are $\exp(-\int \sigma)$ over the *same* segment; reversing the parametrisation $t \mapsto \|\boldsymbol{x}-\boldsymbol{y}\| - t$ does not change the value of the integral. □

Intuition

Transmittance only depends on how much stuff lies *between* two points, not on which way the light travels. Think of a pane of tinted glass: it dims light by the same factor in either direction. (Symmetry would fail only if the medium itself depended on the direction of travel, which is not the case here.)

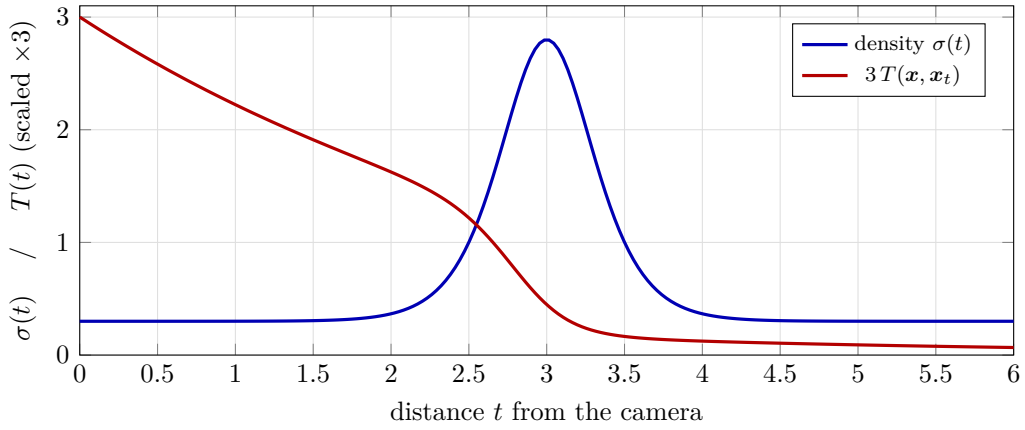


Figure 212: Transmittance along a ray through a medium with a dense layer at $t = 3$ (here $\sigma(t) = 0.3 + 2.5\,\mathrm{sech}^2\frac{t-3}{0.4}$, so $\tau(t) = 0.3t + \tanh\frac{t-3}{0.4} + 1$ exactly). T decays slowly in the haze and drops steeply across the dense layer; compare with slide 44.

6.5 Absorption-only Volume Rendering *(slides 46–48)*

Let $\boldsymbol{x}_z$ be a point behind the medium (e.g. a wall) and $\boldsymbol{x}$ the camera.

Corollary 6.6 (Absorption-only rendering)**.** $L(\boldsymbol{x},\boldsymbol{\omega}) = T(\boldsymbol{x},\boldsymbol{x}_z)\,L(\boldsymbol{x}_z,\boldsymbol{\omega})$*: the light reaching $\boldsymbol{x}$ is the light emitted at $\boldsymbol{x}_z$ modulated by the transmittance $T(\boldsymbol{x},\boldsymbol{x}_z)$.*

This is the radiance from "outside" the medium (slide 48). Slide 47 shows typical absorption-only media: whisky glasses and fog.

[**PHOTO PLACEHOLDER**]
Insert the image from **slide 47**:
Absorption-only media: tinted glass liquid and fog. Figure credits: I. Gkioulekas.

4.8

6.6 Adding Emission *(slides 49)*

Now let the medium also emit, with $\boldsymbol{x}_t = \boldsymbol{x} - t\boldsymbol{\omega}$ the point at depth t along the ray from the camera and $\boldsymbol{x}_z$ the background point at depth z.

Theorem 6.7 (Emission–absorption volume rendering equation)**.**

$$L(\boldsymbol{x},\boldsymbol{\omega}) = \underbrace{T(\boldsymbol{x},\boldsymbol{x}_z)\, L(\boldsymbol{x}_z,\boldsymbol{\omega})}_{\textit{background (outside)}} + \underbrace{\int_0^z T(\boldsymbol{x},\boldsymbol{x}_t)\,\sigma(\boldsymbol{x}_t)\, L_e(\boldsymbol{x}_t,\boldsymbol{\omega})\,\mathrm{d}t}_{\textit{accumulated emission from inside}}.$$

Proof. Let $\ell(t) = L(\boldsymbol{x}_t,\boldsymbol{\omega})$ be the radiance at depth t. Light travels towards the camera, i.e. towards *decreasing* t, so $\mathrm{d}z = -\mathrm{d}t$ in the slide's ODE and $\ell'(t) = \sigma(\boldsymbol{x}_t)\,\ell(t) - \sigma(\boldsymbol{x}_t)\, L_e(\boldsymbol{x}_t,\boldsymbol{\omega})$. With $\tau(t) = \int_0^t \sigma(\boldsymbol{x}_s)\mathrm{d}s$ and $T(\boldsymbol{x},\boldsymbol{x}_t) = e^{-\tau(t)}$:

$$\frac{\mathrm{d}}{\mathrm{d}t}\big(e^{-\tau(t)}\ell(t)\big) = e^{-\tau}(\ell' - \sigma\ell) = -\sigma(\boldsymbol{x}_t)\, e^{-\tau(t)} L_e(\boldsymbol{x}_t,\boldsymbol{\omega}).$$

Integrate from 0 to z: $e^{-\tau(z)}\ell(z) - \ell(0) = -\int_0^z \sigma\, e^{-\tau} L_e\,\mathrm{d}t$. Since $\ell(0) = L(\boldsymbol{x},\boldsymbol{\omega})$ and $\ell(z) = L(\boldsymbol{x}_z,\boldsymbol{\omega})$, rearranging gives the claim. □

Intuition

Every point along the ray "shouts" with loudness $\sigma(\boldsymbol{x}_t)L_e(\boldsymbol{x}_t,\boldsymbol{\omega})$ (how much matter × how bright it is). Its shout then travels to the camera and is attenuated by everything in between, $T(\boldsymbol{x},\boldsymbol{x}_t)$. The pixel is the sum of all shouts plus the muffled background. *Points closer to the camera are less muffled*, which is exactly why near stuff occludes far stuff without any explicit visibility test.

6.7 Emission-Absorption Volume Rendering *(slides 50)*

Slide 50 repeats the equation and asks: *can we compute this analytically? Approximately?* The next slides answer: analytically only in special cases, approximately in general.

6.8 Special Case: Homogeneous, Non-Emitting Medium *(slides 51)*

With $L_e \equiv 0$ and constant σ the integral vanishes, $T(\boldsymbol{x},\boldsymbol{x}_z) = e^{-\sigma\Delta}$ and

$$L(\boldsymbol{x},\boldsymbol{\omega}) = e^{-\sigma\Delta} L(\boldsymbol{x}_z,\boldsymbol{\omega}),$$

where Δ is the distance travelled in the medium: the inner term depends only on Δ.

6.9 Special Case: Homogeneous emitting medium *(slides 52–53)*

Lemma 6.8 (Homogeneous emitting segment)**.** *Let σ and $L_e(\boldsymbol{x},\boldsymbol{\omega}) \equiv C$ be constant on a segment of length Δ (depth δ measured from the camera side). Then*

$$L(\boldsymbol{x},\boldsymbol{\omega}) = e^{-\sigma\Delta} L(\boldsymbol{x}_z,\boldsymbol{\omega}) + \big(1 - e^{-\sigma\Delta}\big)C.$$

In particular the emission of the segment alone is $\big(1 - e^{-\sigma\Delta}\big)C$.

Proof. By Theorem 6.7, the emission integral is $\int_0^{\Delta} e^{-\sigma\delta}\sigma C \,\mathrm{d}\delta = C\big[-e^{-\sigma\delta}\big]_0^{\Delta} = C(1 - e^{-\sigma\Delta})$. □

Remark 6.9. Slide 53 writes the antiderivative as $\int \lambda e^{-\lambda x}\mathrm{d}x = -e^{\lambda x} + c$; the correct primitive is $-e^{-\lambda x} + c$, which is what the computation above uses.

Intuition

The result is a *convex blend*: weight $e^{-\sigma\Delta}$ on what is behind and weight $1 - e^{-\sigma\Delta}$ on the colour of the medium. It is precisely *alpha blending* with opacity $\alpha = 1 - e^{-\sigma\Delta}$. Numbers for $\sigma = 0.5\,\mathrm{m}^{-1}$, $\Delta = 2$ m: $\alpha = 1 - e^{-1} \approx 0.63$, so the pixel is 37% background and 63% medium colour. As $\Delta \to \infty$ the medium becomes opaque and the background disappears (Fig. 213), so a thick-enough volume behaves like a surface.

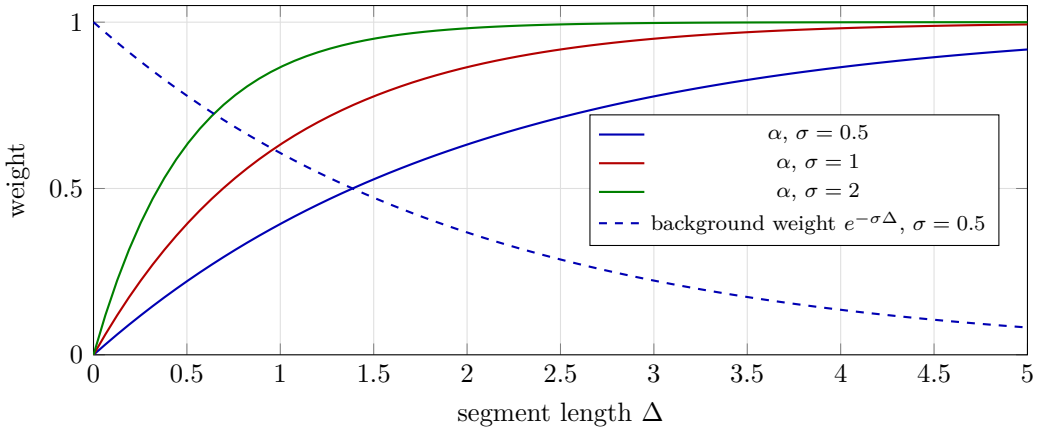


Figure 213: Opacity $\alpha = 1 - e^{-\sigma\Delta}$ of a homogeneous segment and (dashed) the fraction of background that survives, for different densities.

6.10 What Can We Render? *(slides 54)*

Slide 54 contrasts what is within reach and what is not. Within reach (✓): homogeneous EA media such as fog and mist-covered landscapes. Out of reach (×) for the homogeneous EA special case: smoke plumes with fire, light shafts, jade, rainbows, whisky.

[PHOTO PLACEHOLDER]
Insert the image from **slide 54**:
What can we render? Homogeneous EA (fog, misty scenery) vs. out of reach (smoke, light shafts, jade, rainbow, whisky). Figure credits: I. Gkioulekas.

6

Intuition

The closed forms above are lovely but only apply to uniform media. For anything with structure (a smoke plume, a learned density field) σ and L_e vary along the ray, and we need numerics.

7 Computational Volume Rendering *(slides 55–68)*

7.1 Computational Volume Rendering *(slides 56–57)*

In the general case (spatially varying coefficient and/or emission) there is **no analytical solution** to the equation of Theorem 6.7. To simplify, slide 57 first *ignores the light from outside the medium*:

$$L(\boldsymbol{x},\boldsymbol{\omega}) = \int_0^z T(\boldsymbol{x},\boldsymbol{x}_t)\,\sigma(\boldsymbol{x}_t)\,L_e(\boldsymbol{x}_t,\boldsymbol{\omega})\,\mathrm{d}t.$$

The background term is reinstated at slide 67.

7.2 Ray Marching *(slides 58–63)*

Approximate the integral with a discrete sum. Place samples $\boldsymbol{x}_{t_1},\dots,\boldsymbol{x}_{t_{N+1}}$ along the ray (with $t_1 = 0$), so there are N segments with Δ_t the distance between successive samples (slide 58: "$\boldsymbol{x}_{t_i}$ is the ith sample along the ray at depth t_i").

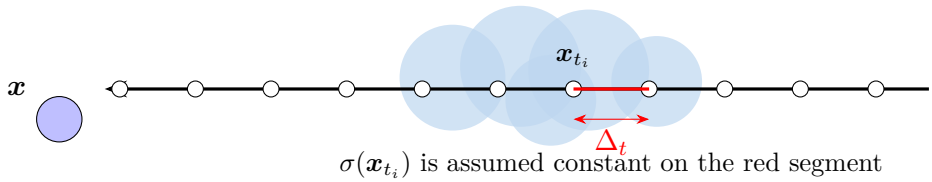


Figure 214: Ray marching: samples along the ray, each representing the segment that follows it.

Step 1: contribution of segment i

Slide 59 writes $L(\boldsymbol{x},\boldsymbol{\omega}) \approx \sum_{i=1}^{N} T(\boldsymbol{x},\boldsymbol{x}_{t_i})\times$(emission from the ith segment), and slide 60 fills in the emission using Lemma 6.8 under the assumption that the medium is homogeneous within Δ_t:

Proposition 7.1 (Ray-marching approximation)**.** *Assume $\sigma \equiv \sigma_{t_i}$ and $L_e \equiv L_e(\boldsymbol{x}_{t_i},\boldsymbol{\omega})$ on segment i. Then*

$$L(\boldsymbol{x},\boldsymbol{\omega}) \approx \sum_{i=1}^{N} T(\boldsymbol{x},\boldsymbol{x}_{t_i})\left(1 - e^{-\sigma_{t_i}\Delta_t}\right) L_e(\boldsymbol{x}_{t_i},\boldsymbol{\omega})$$

with

$$T(\boldsymbol{x},\boldsymbol{x}_{t_i}) = T(\boldsymbol{x},\boldsymbol{x}_{t_{i-1}})\,e^{-\sigma_{t_{i-1}}\Delta_t}, \quad T(\boldsymbol{x},\boldsymbol{x}_{t_1}) = 1.$$

Proof. Split the integral of Theorem 6.7 (without background) over the segments. For $t \in [t_i, t_i + \Delta_t]$, multiplicativity (Lemma 2.3) gives $T(\boldsymbol{x},\boldsymbol{x}_t) = T(\boldsymbol{x},\boldsymbol{x}_{t_i})\,e^{-\sigma_{t_i}(t-t_i)}$, so segment i contributes $T(\boldsymbol{x},\boldsymbol{x}_{t_i})\int_0^{\Delta_t} e^{-\sigma_{t_i}\delta}\sigma_{t_i}L_e\mathrm{d}\delta = T(\boldsymbol{x},\boldsymbol{x}_{t_i})(1 - e^{-\sigma_{t_i}\Delta_t})L_e$ by Lemma 6.8. The recursion for T is multiplicativity applied to the $(i-1)$-th segment, using $T(\boldsymbol{x}_{t_{i-1}},\boldsymbol{x}_{t_i}) = e^{-\sigma_{t_{i-1}}\Delta_t}$ (homogeneous within the segment); the base case is that the camera sits at the first sample. □

Remark 7.2. Slide 68 writes the recursion with $e^{+\sigma_{t_{i-1}}\Delta_t}$; the minus sign (as on slides 61 and 65) is the correct one: transmittance can only decrease along the ray.

Definition 7.3 (Opacity and weight of a sample)**.** Let $\alpha_i = 1 - e^{-\sigma_{t_i}\Delta_t}$ (opacity of segment i), $T_i = T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) = \prod_{j<i}(1-\alpha_j)$ and $w_i = T_i\alpha_i$. Then $L \approx \sum_i w_i\, L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$.

Intuition

This is front-to-back *alpha compositing*, the classic "over" operator. Walk away from the camera: each sample adds a fraction α_i of its colour, multiplied by the fraction T_i of the ray that is still "unspent". For small steps $\alpha_i \approx \sigma_{t_i}\Delta_t$, which is the plain Riemann sum of the integral; the exponential form is better because it stays in $[0, 1]$ even for huge σ or large steps.

Proposition 7.4 (Weights form a partition of unity)**.** *(Not on the slides.) With $T_{N+1} = \prod_{i=1}^{N}(1-\alpha_i)$, we have $\sum_{i=1}^{N} w_i = 1 - T_{N+1}$. So $\sum_i w_i \le 1$ and the leftover T_{N+1} is exactly the weight left for the background, as on slide 67.*

Proof. $w_i = T_i\alpha_i = T_i - T_i(1-\alpha_i) = T_i - T_{i+1}$, so the sum telescopes to $T_1 - T_{N+1} = 1 - T_{N+1}$. □

Intuition

Think of the ray as carrying one unit of "attention". Each sample takes a share w_i and hands the rest on. If the medium is dense, the shares are used up before the end and nothing is left for the background, while in thin media T_{N+1} stays close to 1 and the background shows through. This is also why the same formulas make opaque objects: a very large σ_{t_i} gives $\alpha_i \to 1$ and $w_i \to T_i$, all remaining attention goes to *that* sample, a surface.

Algorithm 7.1 (Ray marching with uniform samples (slide 62–63))**.**
Input: ray origin $\boldsymbol{x}$, direction $\boldsymbol{d}$, volume with $\sigma(\cdot)$ and colour $c(\cdot)$, n samples, max range $t_{\max}$
$T \leftarrow 1, \quad C \leftarrow 0, \quad \Delta_t \leftarrow t_{\max}/n, \quad t \leftarrow 0$
while $t < t_{\max}$ **do**
  $\boldsymbol{p} \leftarrow \boldsymbol{x} + \boldsymbol{d}\,t; \quad (\sigma, c) \leftarrow \text{volume}(\boldsymbol{p})$
  $\alpha \leftarrow 1 - \exp(-\sigma\Delta_t)$ ▷ opacity of this segment
  $C \leftarrow C + T\,\alpha\,c$
  $T \leftarrow T\exp(-\sigma\Delta_t)$ ▷ multiplicativity
  $t \leftarrow t + \Delta_t$
end while
return C

The slide's Python (verbatim in spirit):

```python
def raymarch_uniform(x, d, vol, n_samples, max_range):
    T = 1
    color = Color(0, 0, 0)
    delta_t = max_range / n_samples    # same step everywhere
    t = 0
    while t < max_range:
        p = x + d * t
        sigma, c = vol.at(p).density, vol.at(p).color
        alpha = 1 - exp(-sigma * delta_t)  # opacity of this segment
        color += T * alpha * c
        T *= exp(-sigma * delta_t)         # multiplicativity
        t += delta_t
    return color
```

Example 7.5 (A three-sample ray)**.** *(Worked numbers, not on the slides.)* Let $\Delta_t = 0.5$, $\sigma = (0.4,\ 2.0,\ 4.0)$ and scalar colours $c = (0.2,\ 0.8,\ 1.0)$, no background. Then $\alpha = (1 - e^{-0.2},\ 1 - e^{-1},\ 1 - e^{-2}) = (0.181,\ 0.632,\ 0.865)$, $T = (1,\ 0.819,\ 0.301)$, $T_4 = 0.041$, and $w = T\alpha = (0.181,\ 0.518,\ 0.260)$.

Check: $\sum w_i = 0.959 = 1 - T_4$. The pixel is $0.181 \cdot 0.2 + 0.518 \cdot 0.8 + 0.260 \cdot 1.0 \approx 0.711$. With a background of radiance 0.5 one adds $T_4 \cdot 0.5 \approx 0.020$, giving 0.731.

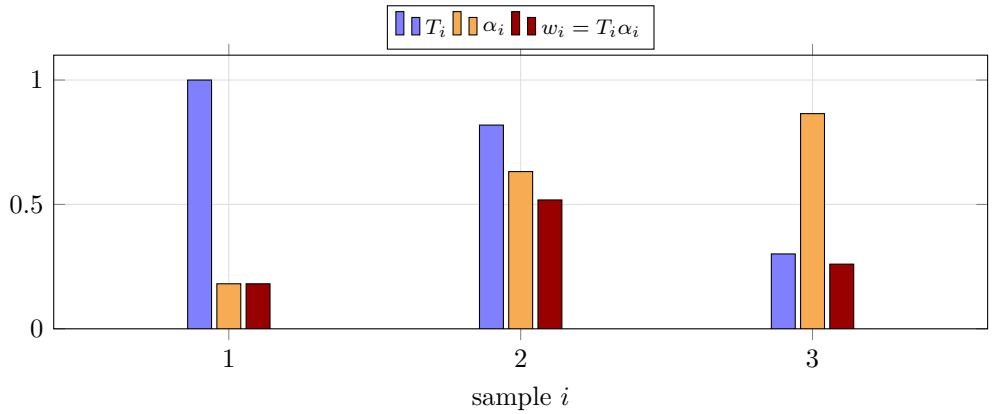


Figure 215: Transmittance, opacity and weight for Example 7.5. The densest sample (3) has the largest α but, because the ray is already partly consumed, not the largest weight.

7.3 Non-Uniform Samples *(slides 64–65)*

Slide 64 changes only the first step of the recipe: *sample non-uniform* points along the ray (N segments, $N+1$ points); the other two steps (transmittance up to each sample, aggregation) are unchanged. The only modification in the equations is that the step becomes sample dependent, $\Delta t_i \equiv t_{i+1} - t_i$:

$$L(\boldsymbol{x}, \boldsymbol{\omega}) \approx \sum_{i=1}^{N} T(\boldsymbol{x}, \boldsymbol{x}_{t_i})\big(1 - e^{-\sigma_{t_i}\Delta t_i}\big) L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega}), \qquad T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) = T(\boldsymbol{x}, \boldsymbol{x}_{t_{i-1}})\, e^{-\sigma_{t_{i-1}}\Delta t_{i-1}}.$$

Intuition

Why bother? Empty space costs samples but contributes nothing; a thin dense layer contributes a lot but can be *missed* entirely by a coarse uniform grid. Non-uniform placement spends the sample budget where the integrand lives. In neural methods the next lecture builds on exactly this idea (importance / hierarchical sampling).

7.4 Uniform vs Non-Uniform sampling *(slides 66)*

Slide 66 shows a checkerboard backdrop with a red semi-opaque wall seen face-on and tilted, rendered with 32 samples. Uniform sampling works for the wall facing the camera, but for the tilted wall it leaves *vertical stripes* (some rays step over the thin slab); non-uniform sampling with the same 32 samples reproduces the reference.

[PHOTO PLACEHOLDER]
Insert the image from **slide 66**:
Uniform vs. non-uniform sampling (reference / uniform, 32 / non-uniform, 32) for a wall facing the camera and a tilted wall.

7

To see the mechanism, I ran the algorithm of Section 6 on a 1D toy: $z = 6$, a thin slab of density $\sigma = 40$ on $[3.07, 3.22]$ (optical depth $40 \cdot 0.15 = 6$), emission $C = 1$, no background. The true radiance is $1 - e^{-6} = 0.9975$. Uniform marching with σ evaluated at the start of each segment gives:

N (uniform)	8	16	32	64	128
absolute error	0.9975	0.9975	0.0019	0.0019	0.0011

For $N \le 16$ the slab is *completely missed* (output 0 instead of 0.9975).

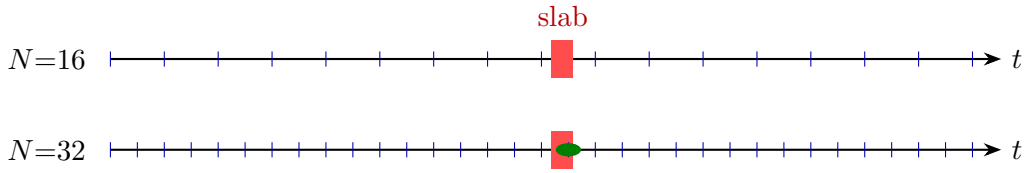


Figure 216: Uniform samples (ticks) vs. a slab of width 0.15. With $N = 16$ ($\Delta_t = 0.375$) no sample falls inside the slab; with $N = 32$ one sample (green) does and the slab is detected.

Intuition

A tilted wall means the ray's path inside the slab depends on the pixel; for some pixels the in-slab path is shorter than the step, and the samples jump over it. That is a classic *aliasing* problem, here producing a regular pattern of missing columns. In a learned volume the thin layer is a surface, so missing it means missing the object.

7.5 Background Radiance *(slides 67)*

Reinstate the background wall at $\boldsymbol{x}_z$:

$$L(\boldsymbol{x}, \boldsymbol{\omega}) \approx \sum_{i=1}^{N} T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) \big(1 - e^{-\sigma_{t_i} \Delta t_i}\big) L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega}) + T(\boldsymbol{x}, \boldsymbol{x}_{t_{N+1}})\, L(\boldsymbol{x}_z, \boldsymbol{\omega}).$$

The slide's remark: the final transmittance $T(\boldsymbol{x}, \boldsymbol{x}_{t_{N+1}})$ can be thought of as a "*background mask*" (it is 1 where the ray crossed nothing and 0 where it was fully absorbed).

Intuition

By Proposition 7.4 the pixel is a convex combination: samples take weight $\sum w_i = 1 - T_{N+1}$, the background takes T_{N+1}. Rendering the mask T_{N+1} alone gives a silhouette image, useful for training with segmentation masks, and for compositing on arbitrary backgrounds.

7.6 Summary *(slides 68)*

Slide 68 recaps: the emission–absorption model $\mathrm{d}L = -\sigma_a L\,\mathrm{d}z + \sigma_a L_e\,\mathrm{d}z$; transmittance as $\exp(-\int \sigma)$ (see Fig. above); and the computational algorithm $L \approx \sum_i T(\boldsymbol{x}, \boldsymbol{x}_{t_i})(1 - e^{-\sigma_{t_i}\Delta_t})L_e$ with the recursion for T. (Slides 69–71: preview of the next lecture, which is about NeRF-style neural fields, questions and acknowledgements. The slides are adapted from *Learning for 3D Vision*, CMU 16-825, Prof. S. Tulsiani, which credits Yannis Gkioulekas for most slides and figures, Pat Hanrahan for the radiance diagram, Wojciech Jarosz, and Studio Lernert & Sander.)

8 Summary

8.1 Comparison of the models covered

8.2 A qualitative comparison plot

Figure 217 summarises which effects each model can capture, following slides 54 and 37: green = natively captured, yellow = only through an arbitrary effective emission L'_e (in principle, with complex inhomogeneous scenes), red = not captured. This is my own qualitative reading of the slides, not a measurement.

Model	Pixel value	Needs	Limits
Surface rendering (slide 14)	$L(\boldsymbol{x}^* - \lambda\boldsymbol{\omega}, \boldsymbol{\omega})$ at the first hit	depth λ, shading	no fog, smoke, glow
Absorption only, homogeneous (42)	$e^{-\sigma z} L(\boldsymbol{x}_0, \boldsymbol{\omega})$	constant σ	no emission, uniform media
Absorption only, heterogeneous (43, 46)	$T(\boldsymbol{x}, \boldsymbol{x}_z) L(\boldsymbol{x}_z, \boldsymbol{\omega})$, $T = e^{-\int \sigma}$	$\sigma(\boldsymbol{x})$ along the ray	no emission
Emission–absorption, analytic (51–53)	$e^{-\sigma\Delta} L_{bg} + (1 - e^{-\sigma\Delta}) C$	constant σ, C	only homogeneous media
Emission–absorption, general (49)	$T L_{bg} + \int T \sigma L_e \, \mathrm{d}t$	$\sigma(\boldsymbol{x}), L_e(\boldsymbol{x}, \boldsymbol{\omega})$	no closed form; no explicit scattering
Ray marching (58–67)	$\sum_i T_i (1 - e^{-\sigma_i \Delta t_i}) L_{e,i} + T_{N+1} L_{bg}$	samples along ray	discretisation error, sampling aliasing
Full RTE (34)	EA terms plus $-\sigma_s L + \sigma_s L_s$	σ_s, phase function f_p	expensive, recursive; allows relighting

Table 14: Models of increasing generality discussed in the lecture.

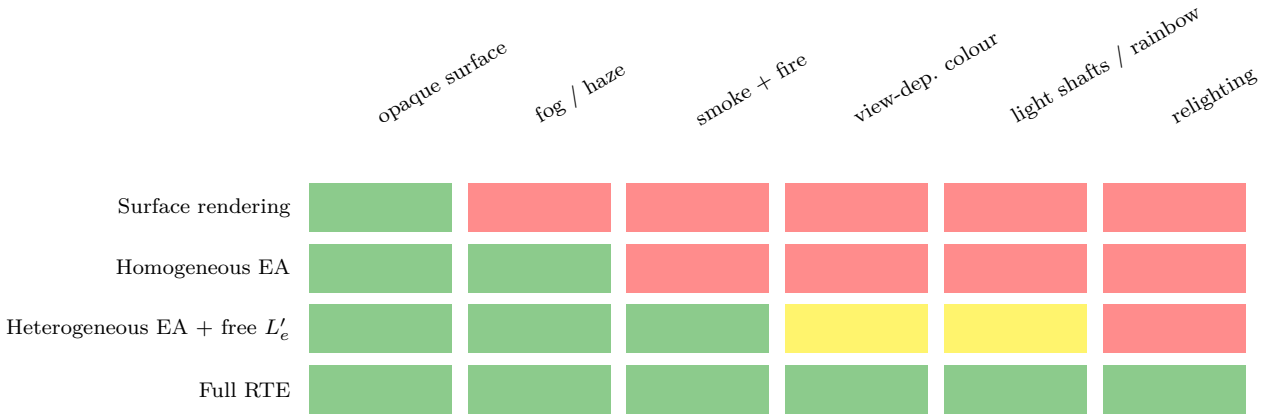


Figure 217: Qualitative capability matrix (green: yes, yellow: in principle via L'_e, red: no).

8.3 Key take-aways

- **Drop the surface assumption.** Represent the scene by a non-negative density field $\sigma(\boldsymbol{x})$ (plus emitted colour); an opaque surface is just the limit of a very sharp density jump.
- **One ODE does most of the work.** The emission–absorption model $\mathrm{d}L = -\sigma L \, \mathrm{d}z + \sigma L_e \, \mathrm{d}z$ integrates to $L = T(\boldsymbol{x}, \boldsymbol{x}_z) L_{bg} + \int_0^z T(\boldsymbol{x}, \boldsymbol{x}_t) \sigma(\boldsymbol{x}_t) L_e(\boldsymbol{x}_t, \boldsymbol{\omega}) \mathrm{d}t$, with transmittance $T = e^{-\int \sigma}$ and its multiplicativity property.
- **Scattering is expensive but can be absorbed in an effective emission** $L'_e = L_e + \frac{\sigma_s}{\sigma_a}(L_s - L)$; this is why EA is the de-facto standard in neural volume rendering (at the price of no physical relighting).
- **Ray marching is alpha compositing.** $\alpha_i = 1 - e^{-\sigma_i \Delta t_i}$, $T_{i+1} = T_i(1 - \alpha_i)$, $w_i = T_i \alpha_i$, $L \approx \sum_i w_i L_{e,i} + T_{N+1} L_{bg}$; the weights telescope to $1 - T_{N+1}$.
- **Where you sample matters.** Uniform sampling can step over thin dense layers (aliasing); non-uniform sampling spends samples where σ is large. Next lecture: combine this renderer with neural representations.

Part VII

Lecture 8

Abstract

These notes follow the flow of the lecture slides: we recall *computational volume rendering* (ray marching), then ask how a volume can be *represented* so that density and emitted light can be queried at any 3D point — a voxel grid, an MLP (the Neural Radiance Field), voxels with spherical-harmonic colours (Plenoxels) and finally feature hash-grids with a tiny MLP (Instant-NGP). Along the way we discuss positional encoding, hierarchical and inverse-depth sampling, view-dependent effects and anti-aliasing (mip-NeRF). Slide ranges are given in each section title. No companion book chapter was provided, so the formal statements (definitions, lemmas, proofs) were derived from the slides and from the cited papers (NeRF, Fourier Features, mip-NeRF, Plenoxels, Instant-NGP).

1 Neural Radiance Fields *(slides 1–2)*

The goal of the lecture is summarised in one line on slide 2:

$$\text{posed images} \longrightarrow \text{a neural network} \longrightarrow \text{novel views.}$$

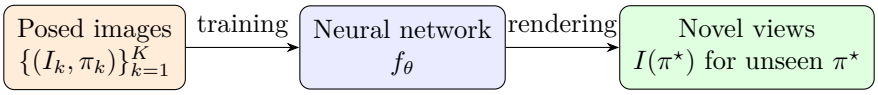


Figure 218: The NeRF problem at a glance (redrawn from slide 2): learn f_θ from images with known cameras π_k, then render images from cameras never seen during training.

[IMAGE PLACEHOLDER]
Slide **2**: Left: nine of the posed training images of the NeRF "lego" scene; right: a novel view rendered by `nerfacto` (Nerfstudio). Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

Definition 1.1 (Novel-view synthesis)**.** Given images $I_1, \dots, I_K$ of a static scene together with their camera parameters $\pi_1, \dots, \pi_K$ (intrinsics and pose), *novel-view synthesis* (NVS) is the task of producing the image $I(\pi^\star)$ that a camera with parameters $\pi^\star \notin \{\pi_k\}$ would see.

Definition 1.2 (Radiance field)**.** A *radiance field* is a function $(\boldsymbol{x}, \boldsymbol{\omega}) \mapsto (\sigma, \boldsymbol{c})$ that assigns to every 3D point $\boldsymbol{x} \in \mathbb{R}^3$ and viewing direction $\boldsymbol{\omega} \in \mathbb{S}^2$ a volume density $\sigma \in \mathbb{R}^+$ and an emitted colour (radiance) $\boldsymbol{c} \in [0,1]^3$. A *Neural* Radiance Field (NeRF) is a radiance field parameterised by a neural network f_θ.

Intuition

Think of the scene as a cloud of coloured, glowing smoke. $\sigma(\boldsymbol{x})$ says "how thick is the smoke here?" and $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega})$ says "what colour does it glow when seen from direction $\boldsymbol{\omega}$?". A photograph is what you get by looking at this cloud through a pinhole. Training means adjusting the cloud until all the photographs we have are reproduced; because the cloud is a genuine 3D object, we can then look at it from anywhere.

Roadmap of the lecture. The method has two ingredients that the slides keep separate: (i) a *rendering algorithm* that turns density and emission along a ray into a pixel colour (Sections 2–3), and (ii) a *representation* of the volume from which density and emission can be queried pointwise (grid, MLP, voxels + spherical harmonics, hash-grid; Sections 3–13).

2 Computational Volume Rendering: Ray Marching *(slides 3–8)*

2.1 Ray marching: sum over segments *(slides 3–6)*

A camera at $\boldsymbol{x}$ looks in direction $\boldsymbol{\omega}$. We discretise the ray $\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{\omega}$ into N segments delimited by $N+1$ sample points $t_1 < t_2 < \cdots < t_{N+1}$ (slide 5 draws them uniformly; slide 6 shows that they need not be). We write

$$\boldsymbol{x}_{t_i} = \boldsymbol{x} + t_i\boldsymbol{\omega}, \qquad \Delta t_i \equiv t_{i+1} - t_i, \qquad \sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i}).$$

Slide 3 states the target: the radiance reaching the camera is a sum of one *contribution per segment*.

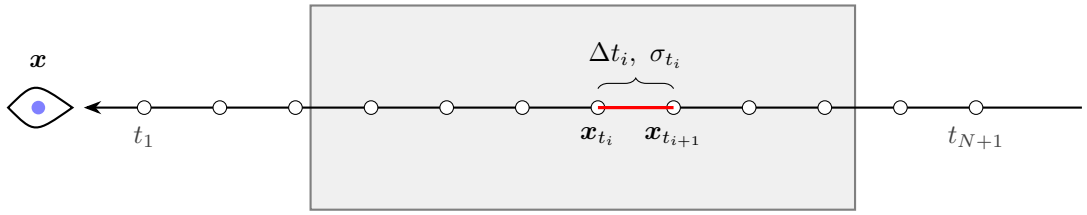


Figure 219: Ray marching through an emitting/absorbing medium (redrawn from slides 3–6; original ray-marching figures by I. Gkioulekas). Within each segment the density σ_{t_i} and the emission $L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$ are assumed constant.

Medium model. The medium is *absorbing and emitting*: every infinitesimal piece of volume emits light (colour) and absorbs part of the light passing through it. Absorption is governed by the density σ and the *transmittance*.

Definition 2.1 (Transmittance)**.** For two points $\boldsymbol{x}_a = \boldsymbol{x}_{t_a}$ and $\boldsymbol{x}_b = \boldsymbol{x}_{t_b}$ on a ray, with $t_a < t_b$, the transmittance is the fraction of light that survives the trip between them,

$$T(\boldsymbol{x}_a, \boldsymbol{x}_b) = \exp\Big(-\int_{t_a}^{t_b} \sigma(\boldsymbol{x}_s)\, \mathrm{d}s \Big) \in (0, 1].$$

Lemma 2.2 (A homogeneous emitting segment)**.** *Consider a segment of length Δt with constant density σ and constant emission L_e. The radiance it delivers to its near end, ignoring everything outside the segment, is*

$$\int_0^{\Delta t} e^{-\sigma s}\, \sigma\, L_e\, \mathrm{d}s = \big(1 - e^{-\sigma \Delta t}\big) L_e.$$

Proof. Light emitted at depth s inside the segment is attenuated by $e^{-\sigma s}$ before leaving it, and the amount emitted per unit length is σL_e. Hence $\int_0^{\Delta t} \sigma L_e e^{-\sigma s}\, \mathrm{d}s = L_e\, [-e^{-\sigma s}]_0^{\Delta t} = L_e(1 - e^{-\sigma \Delta t})$. □

Intuition

The factor $\alpha = 1 - e^{-\sigma \Delta t}$ is the segment's *opacity*: the probability that the ray is "stopped" (and thus sees the emitted colour) inside it. Thin or dilute segments ($\sigma \Delta t \ll 1$) have $\alpha \approx \sigma \Delta t$; a thick one ($\sigma \Delta t \gg 1$) has $\alpha \to 1$ and is a solid wall. Example: $\sigma \Delta t = 0.5 \Rightarrow \alpha = 0.39$, and $\sigma \Delta t = 3 \Rightarrow \alpha = 0.95$.

2.2 Transmittance as a recursion *(slides 4–6)*

The light emitted by segment i must still travel through all the segments between it and the camera; this is where $T(\boldsymbol{x}, \boldsymbol{x}_{t_i})$ enters (slide 4).

Lemma 2.3 (Multiplicativity of transmittance)**.** *With $T_i \equiv T(\boldsymbol{x}, \boldsymbol{x}_{t_i})$ we have the base case $T_1 = 1$ and the recursion*

$$T_i = T_{i-1}\, e^{-\sigma_{t_{i-1}} \Delta t_{i-1}} \quad (i \geq 2), \qquad \textit{hence} \qquad T_i = \exp\Big(-\sum_{j<i} \sigma_{t_j} \Delta t_j \Big).$$

(Slide 4 writes Δt; slide 6 shows that it may depend on i.)

Proof. $T_1 = 1$ because no medium lies between the camera and the first sample. For $i \geq 2$, split the integral in T at t_{i-1}: $\int_{t_1}^{t_i} \sigma = \int_{t_1}^{t_{i-1}} \sigma + \int_{t_{i-1}}^{t_i} \sigma$, and the exponential of a sum is a product. On the last segment σ is constant, so the second integral equals $\sigma_{t_{i-1}} \Delta t_{i-1}$. Unrolling the recursion gives the closed form. □

Theorem 2.4 (Discrete volume rendering)**.** *For a ray with segments as above, the radiance reaching the camera is*

$$L(\boldsymbol{x},\boldsymbol{\omega}) = \sum_{i=1}^{N} T(\boldsymbol{x},\boldsymbol{x}_{t_i})\left(1-e^{-\sigma_{t_i}\Delta t_i}\right) L_e(\boldsymbol{x}_{t_i},\boldsymbol{\omega}) = \sum_{i=1}^{N} w_i\, \boldsymbol{c}_i,$$

where $w_i = T_i\alpha_i$, $\alpha_i = 1-e^{-\sigma_{t_i}\Delta t_i}$ *and* $\boldsymbol{c}_i = L_e(\boldsymbol{x}_{t_i},\boldsymbol{\omega})$.

Proof. By Lemma 2.2 segment i emits $\alpha_i\boldsymbol{c}_i$ at its own near end; by Lemma 2.3 a fraction T_i of it survives the way to the camera. Light emission is additive, so we sum over segments. □

Intuition

Standing at the camera you walk along the ray. At every segment you ask two questions: "did the light from here survive everything in front of me?" (T_i) and "does this segment actually emit anything?" ($\alpha_i\boldsymbol{c}_i$). The weight w_i is "how much segment i contributes to the pixel"; this is the quantity that colours the segments red/yellow on slides 32–33 and that will drive hierarchical sampling (Section 7).

Algorithm 2.1 (Ray marching, slide 5)**.**

1. Draw samples along the ray (N segments, i.e. $N+1$ points); slide 5 uses uniform spacing.
2. Compute the transmittance T_i between the camera and each sample (recursion of Lemma 2.3).
3. Aggregate the contributions $T_i\alpha_i\boldsymbol{c}_i$ over all segments to obtain the overall radiance (the pixel colour).

Example 2.5 (Worked numbers)**.** Take $N = 4$ segments of length $\Delta t = 0.5$, densities $\sigma = (0.2, 2, 3, 0.4)$ and grey-level emissions $L_e = (0.2, 0.9, 0.5, 0.3)$. Then $\alpha = (0.095, 0.632, 0.777, 0.181)$, $T = (1, 0.905, 0.333, 0.074)$ and so $w = T\alpha = (0.095, 0.572, 0.259, 0.014)$. The pixel value is $0.095\cdot 0.2 + 0.572\cdot 0.9 + 0.259\cdot 0.5 + 0.014\cdot 0.3 \approx 0.667$. Segment 2 dominates: it is dense *and* close to the camera. Segment 4 is hidden: T_4 is only 0.074 even though $\alpha_4 = 0.18$.

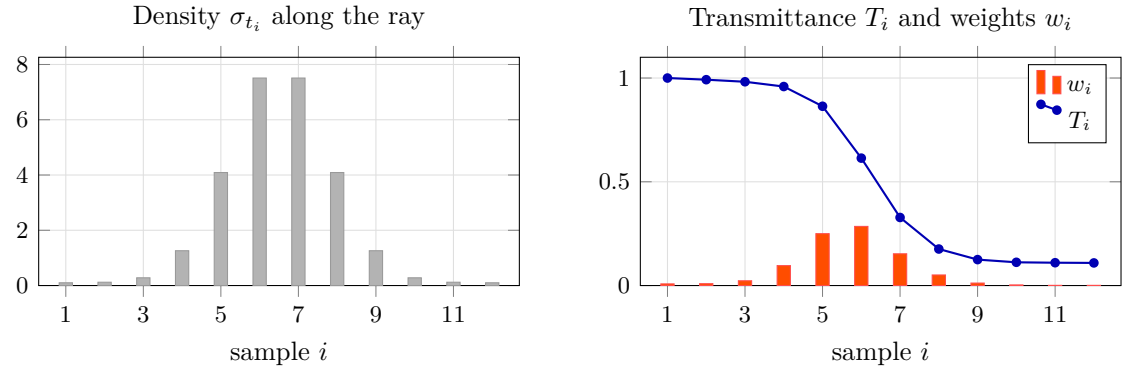


Figure 220: Original illustration (not in the slides): a ray with 12 segments of length 1/12 crossing a dense blob centred at the middle of the ray ($\sigma_{t_i} = 0.1 + 8e^{-((t_i-0.5)/0.15)^2}$ at segment midpoints). Transmittance drops while the ray crosses the blob and the weights w_i peak at the near side of the blob — the first dense samples hide the ones behind. $\sum_i w_i = 0.89$ and $T_{N+1} = 0.11$.

2.3 Enabling background radiance *(slides 7)*

If the ray does not end inside the medium but hits an opaque surface or a background (the grey wall $\boldsymbol{x}_z$ on slide 7), the light that survives the whole medium adds one more term:

$$L(\boldsymbol{x},\boldsymbol{\omega}) = \sum_{i=1}^{N} T(\boldsymbol{x},\boldsymbol{x}_{t_i})\big(1-e^{-\sigma_{t_i}\Delta t}\big)L_e(\boldsymbol{x}_{t_i},\boldsymbol{\omega}) \;+\; T(\boldsymbol{x},\boldsymbol{x}_{t_{N+1}})\,L(\boldsymbol{x}_z,\boldsymbol{\omega}).$$

Proposition 2.6 (Weights sum to one). *With $w_i = T_i\alpha_i$ we have $w_i = T_i - T_{i+1}$ and therefore*

$$\sum_{i=1}^{N} w_i + T_{N+1} = 1.$$

Consequently the rendered colour is a convex combination *of the segment colours and the background colour, and $T_{N+1} = 1 - \sum_i w_i$ is the* background mask.

Proof. By Lemma 2.3, $T_{i+1} = T_i e^{-\sigma_{t_i}\Delta t_i} = T_i(1-\alpha_i)$, so $T_i - T_{i+1} = T_i\alpha_i = w_i$. The sum telescopes: $\sum_{i=1}^{N} w_i = T_1 - T_{N+1} = 1 - T_{N+1}$. □

Intuition

This is *alpha compositing*. Every segment "paints" with opacity α_i over what is behind it; whatever the medium lets through (T_{N+1}) shows the background. Numerically, in Example 2.5 the weights sum to 0.939 and $T_5 = 0.061$; with a white background ($L = 1$) the pixel becomes $0.667 + 0.061 \cdot 1 = 0.728$. For synthetic scenes with a white background one therefore renders $\hat{C} = \sum_i w_i \boldsymbol{c}_i + (1 - \sum_i w_i) \cdot 1$. In the same way, T_{N+1} is a soft "this ray hits nothing" mask.

2.4 Summary: computational volume rendering *(slides 8)*

If we can compute (a) a per-point density $\sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i})$ and (b) a per-point, per-direction emitted light $L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$, then we can render *any* ray through the medium, hence an image from any camera viewpoint using $H \times W$ rays (one per pixel).

Proposition 2.7 (Differentiability of the renderer). *Let $\hat{C} = \sum_i w_i \boldsymbol{c}_i + T_{N+1} L_{\text{bg}}$. Then*

$$\frac{\partial \hat{C}}{\partial \boldsymbol{c}_i} = w_i, \qquad \frac{\partial \hat{C}}{\partial \sigma_{t_k}} = \Delta t_k \Big(T_{k+1}\, \boldsymbol{c}_k - \sum_{i>k} w_i \boldsymbol{c}_i - T_{N+1} L_{\text{bg}} \Big).$$

Proof. The first identity is immediate. For σ_{t_k}: α_k depends on it with $\partial\alpha_k/\partial\sigma_{t_k} = \Delta t_k e^{-\sigma_{t_k}\Delta t_k}$, so the term $w_k \boldsymbol{c}_k$ contributes $T_k \Delta t_k e^{-\sigma_{t_k}\Delta t_k} \boldsymbol{c}_k = \Delta t_k T_{k+1} \boldsymbol{c}_k$. Moreover T_i for every $i > k$ (and T_{N+1}) contains $e^{-\sigma_{t_k}\Delta t_k}$, so $\partial T_i/\partial\sigma_{t_k} = -\Delta t_k T_i$ and $w_i = T_i\alpha_i$ gives $-\Delta t_k w_i \boldsymbol{c}_i$; likewise for the background term. □

Intuition

Increasing the density at sample k has two opposite effects (slide 8's red note: the pipeline is differentiable). It makes sample k's own colour more visible (positive term) but it also hides everything behind it (negative terms). Gradient descent on a photometric loss thus "pushes density" to where it explains the pixel and removes it where it would wrongly occlude. The same holds for the camera parameters whenever density and emission are differentiable functions of position and direction.

3 Volumes: Rendering and Representation *(slides 9–10)*

Slides 9–10 split the problem into two boxes:

$$\underbrace{\begin{matrix}\sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i}) \\ L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})\end{matrix}}_{\text{representation: "How to represent volumes?"}} \longrightarrow \underbrace{L(\boldsymbol{x}, \boldsymbol{\omega})}_{\text{rendering algorithm (done)}}$$

The rendering algorithm is fixed; what remains is a *representation* from which the density and the emitted light can be computed pointwise.

[IMAGE PLACEHOLDER]
Slide **9**: A volumetric example (smoke-like plume): volumes can be rich and view-dependent, and we need a representation that can encode them. Figure credits as on the slide.

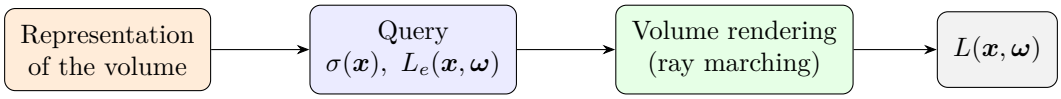


Figure 221: Division of labour in slides 9–10: the representation only has to answer pointwise queries; ray marching does the rest.

3.1 Option 1: a grid — Grid Representation *(slides 11–17)*

Definition 3.1 (Voxel-grid volume)**.** A grid stores a positive scalar in each voxel, $\sigma[x, y, z] \in \mathbb{R}^+$ (the density), and — for emission — a colour $\boldsymbol{c}[x, y, z] \in [0, 1]^3$. Since the lookup of an arbitrary real-valued point $\boldsymbol{x}$ is not a voxel index, slide 11 asks: *how to obtain $\sigma(\boldsymbol{x})$ for arbitrary $\boldsymbol{x}$?* The answer is interpolation.

Trilinear interpolation *(slides 12)*

Let $\boldsymbol{x}$ fall into a cell whose eight corners carry the values σ_{ijk}, $i, j, k \in \{0, 1\}$. Let $(u, v, w) \in [0, 1]^3$ be the local coordinates of $\boldsymbol{x}$ in the cell, and set $u_1 = u,\ u_0 = 1 - u$ (same for v, w).

Definition 3.2 (Trilinear interpolation)**.** $\sigma(\boldsymbol{x}) = \sum_{i,j,k \in \{0,1\}} u_i\, v_j\, w_k\, \sigma_{ijk}$. It is the 3D equivalent of bilinear interpolation.

Lemma 3.3 (Properties of trilinear interpolation)**.** *(i) The weights $\lambda_{ijk} = u_i v_j w_k$ are non-negative and sum to* 1*. (ii) The weight of a corner equals the volume of the box spanned by $\boldsymbol{x}$ and the* opposite *corner. (iii) σ is linear in each of u, v, w separately, reproduces the corner values, and is continuous across cell boundaries. (iv) $\partial\sigma(\boldsymbol{x})/\partial\sigma_{ijk} = \lambda_{ijk}$: the interpolation is differentiable w.r.t. the stored corner values.*

Proof. (i) $\sum \lambda_{ijk} = (u_0 + u_1)(v_0 + v_1)(w_0 + w_1) = 1$. (ii) The box between $\boldsymbol{x}$ and the corner $(0, 0, 0)$ has side lengths u, v, w, so its volume is $uvw = \lambda_{111}$; the other corners follow by symmetry (e.g. $\lambda_{000} = (1 - u)(1 - v)(1 - w)$ is the box towards $(1, 1, 1)$). (iii) Fix v, w: σ is a convex combination $(1 - u)a + ub$, linear in u; at a corner one λ equals 1 and the others vanish. On a shared face, say $u = 1$, the formula only involves the four corners of that face, so neighbouring cells agree. (iv) Linearity in the σ_{ijk}. □

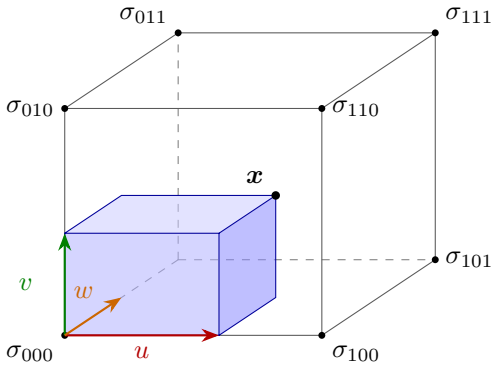


Figure 222: Trilinear interpolation (redrawn from slide 12). The shaded box between $\boldsymbol{x}$ and corner $(0,0,0)$ has volume uvw and is the weight of the *opposite* corner σ_{111}; the other seven weights are the volumes of the other seven sub-boxes, which tile the cell.

Intuition

Imagine standing in the cell and drawing the three planes through yourself parallel to the faces: they cut the cell into eight little boxes. Each corner is weighted by the box *on the other side* of you — the closer you are to a corner, the bigger the opposite box, hence the larger its weight. In 1D this is just $(1-u)a+ub$; e.g. for $u=0.25$ the nearer value gets weight 0.75. In 3D with $(u,v,w)=(0.6,0.45,0.5)$ as in the figure, $\lambda_{111}=0.6\cdot0.45\cdot0.5=0.135$ and $\lambda_{000}=0.4\cdot0.55\cdot0.5=0.11$.

Emission on the grid *(slides 13–15)*

Q: how do we model view-dependent emission $L_e(\boldsymbol{x},\boldsymbol{\omega})$? The grid answer (slide 13) is to *ignore* view dependence: $L_e(\boldsymbol{x},\boldsymbol{\omega})\equiv\boldsymbol{c}(\boldsymbol{x})$. Slide 14 therefore stores a colour $\boldsymbol{c}[x,y,z]\in[0,1]^3$ next to the density in every voxel, and trilinear interpolation is applied to both.

[IMAGE PLACEHOLDER]
Slide **14, 15**: Left: a chair stored as a coloured voxel grid (slide 14). Right: an emission-only rendering of a cloud and the corresponding density voxel grid, downsampled for visualization (slide 15).

[IMAGE PLACEHOLDER]
Slide **16**: Voxel grids are everywhere in medical imaging: a 3D brain volume with voxel (i,j,k) and a 2D MRI slice with pixel (i,j). Figure credits: www.hindawi.com/journals/cmmm/2015/450341.

Limits of the grid *(slides 17)*

[IMAGE PLACEHOLDER]
Slide **17**: Slide 17: a real captured scene (red car) — grids "cannot model view-dependent effects" such as the car's reflections, and their size grows cubically with the scene dimension. Video credits: Mildenhall et al., ECCV 2020 (NeRF).

Proposition 3.4 (Cubic memory growth)**.** *A dense grid with resolution N per axis and b bytes per voxel needs $N^3 b$ bytes. Doubling the resolution multiplies memory (and training/rendering cost per volume) by* 8*.*

Intuition

With σ and an RGB colour in 32-bit floats ($b = 16$ bytes), $N = 128$ costs 33.6 MB, $N = 512$ already 2.1 GB, and $N = 1024$ about 17.2 GB — for a scene that is mostly empty space. Together with the lack of view-dependent colour this is what motivates the MLP of the next section, and later sparse (Plenoxels) and hash-based (Instant-NGP) hybrids.

4 Neural Radiance Fields *(slides 18–20)*

Volumes: Rendering and Representation, Option 2: An MLP (slide 18). Instead of storing values on a lattice, store them in the weights of a neural network.

Definition 4.1 (Neural Radiance Field)**.** A NeRF is a multilayer perceptron (MLP)

$$f_\theta : \ (\boldsymbol{x}, \boldsymbol{\omega}) \longmapsto (\sigma, \boldsymbol{c}), \qquad \sigma \in \mathbb{R}^+, \quad \boldsymbol{c} \in [0,1]^3,$$

such that (1) given an input point it predicts the density, and (2) given a point *and* a viewing direction it predicts the radiance (colour). The scene is represented *entirely* by the weights θ.

Proposition 4.2 (Density is view-independent by design)**.** *If $\boldsymbol{\omega}$ is injected only after the layer that produces σ, then $\sigma = \sigma(\boldsymbol{x})$ does not depend on $\boldsymbol{\omega}$, while $\boldsymbol{c} = \boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega})$ may.*

Proof. σ is computed by a function composition that has $\boldsymbol{x}$ (or features of it) as its only input; $\boldsymbol{\omega}$ is concatenated afterwards, so no path from $\boldsymbol{\omega}$ to σ exists in the computation graph. □

Intuition

Physically, whether there is matter at a point cannot depend on where you look from, whereas the colour of a shiny surface can. Slide 18 asks us to ensure this "with designing the MLP that density only depends on $\boldsymbol{x}$"; this is what the architecture of Figure 223 does. This is a built-in inductive bias which makes the 3D structure consistent across views.

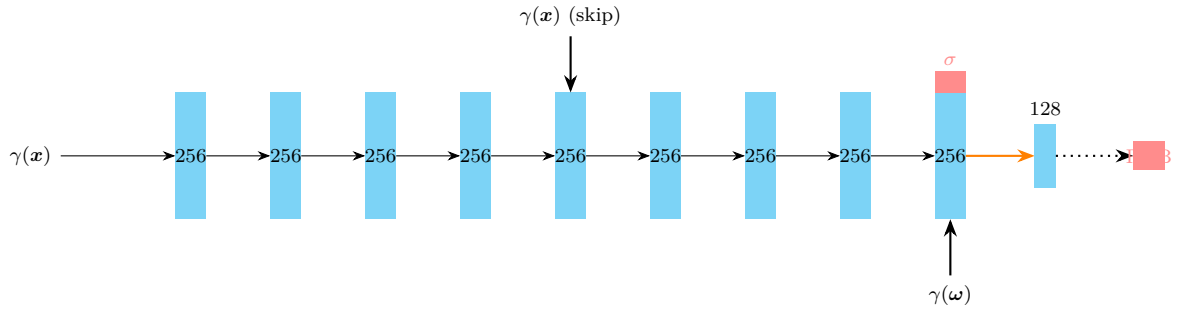


Figure 223: The NeRF MLP (redrawn from slides 18 and 24): fully connected blocks of width 256; the encoded position is fed again at the fifth block (skip connection); the density σ is read out *before* the viewing direction is concatenated; one extra 128-unit layer sees $\gamma(\boldsymbol{\omega})$ and outputs the RGB colour. Typical choices: ReLU hidden units, a non-negative activation (ReLU/softplus) for σ, sigmoid for RGB, so that $\sigma \in \mathbb{R}^+$ and $\boldsymbol{c} \in [0,1]^3$.

How to train this MLP? (slide 20). Key idea: *learn a 3D representation consistent with the observed views.* The next section turns this sentence into a loss.

5 Learning Neural Radiance Fields *(slides 21–24)*

5.1 From pixels to rays *(slides 21–22)*

Definition 5.1 (Camera ray)**.** Given a pixel $\boldsymbol{p} = (u, v)$ and camera parameters $\pi = (K, R, \boldsymbol{t}_c)$ (intrinsics K, world-from-camera rotation R and camera centre $\boldsymbol{t}_c$), the ray through $\boldsymbol{p}$ is

$$\boldsymbol{x}_t = \boldsymbol{o} + t\ \mathrm{d}, \qquad \boldsymbol{o} = \boldsymbol{t}_c, \qquad \mathrm{d} = \frac{RK^{-1}(u, v, 1)^\top}{\|RK^{-1}(u, v, 1)^\top\|}, \quad t \geq 0.$$

The slides abbreviate this as the map (pixel, camera) $\to$ ray, and the viewing direction fed to the network is $\boldsymbol{\omega} = -\mathrm{d}$ (or d, by convention).

Definition 5.2 (View-synthesis loss)**.** Let $\mathrm{render}(\boldsymbol{p}, \pi; f_\theta)$ be the volume-rendered colour (Theorem 2.4) of the ray through pixel $\boldsymbol{p}$ of the camera π when the volume is given by f_θ. The NeRF is trained by

$$\min_\theta \sum_I \sum_{\boldsymbol{p}} \left\| \mathrm{render}(\boldsymbol{p}, \pi; f_\theta) - I[\boldsymbol{p}] \right\|^2,$$

where I runs over the training images and $\boldsymbol{p}$ over their pixels.

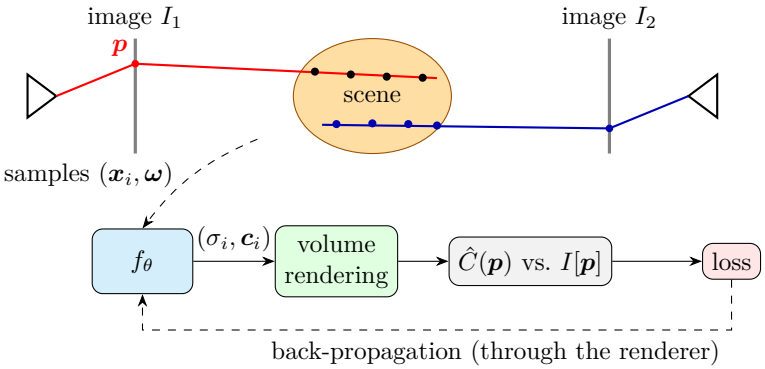


Figure 224: Learning a NeRF (redrawn and simplified from slides 21–22). Rays from two cameras are marched; the 3D points are sent to f_θ, whose outputs are composited by volume rendering and compared with the observed pixel. The gradient flows back through the (differentiable) renderer to θ.

Intuition

The loss never mentions 3D. It only says: "if I shoot the ray of pixel $\boldsymbol{p}$ through my current cloud, I should get the pixel's colour." But a 3D point on the visible surface lies on the rays of many cameras, and the only way to be right for *all* of them simultaneously is to put density at the correct place with a colour that is consistent across views. This is why slide 23 requires typically more than 50 images with *known* camera parameters (in practice obtained with structure-from-motion).

5.2 Mini-batches of rays *(slides 23)*

A single image already has $H \times W$ rays (e.g. $800 \times 800 = 640{,}000$) and each ray needs many network queries (Section 7), so we cannot back-propagate through all of them. "In practice, randomly sample a few pixels (rays) per optimisation iteration."

Proposition 5.3 (Unbiased stochastic gradients)**.** *Let $\mathcal{L}(\theta) = \frac{1}{|\mathcal{P}|}\sum_{(I,\boldsymbol{p})\in\mathcal{P}} \ell_{I,\boldsymbol{p}}(\theta)$ be the average per-ray loss over all training pixels $\mathcal{P}$, and B a batch of m pixels drawn uniformly (with replacement). Then $\mathbb{E}_B\big[\frac{1}{m}\sum_{b\in B}\nabla\ell_b\big] = \nabla\mathcal{L}(\theta)$.*

Proof. Each draw is uniform, so $\mathbb{E}[\nabla\ell_b] = \frac{1}{|\mathcal{P}|}\sum_{p\in\mathcal{P}}\nabla\ell_p = \nabla\mathcal{L}$; the average of m such expectations is the same. □

Intuition

It is the usual stochastic gradient descent, except that the "training examples" are individual rays, not whole images. Random rays from random images also decorrelate the updates, much like random mini-batches.

Algorithm 5.1 (NeRF training loop, slide 24)**.**

1. **Acquire** multiple images of a scene with associated camera viewpoints.
2. **Design** a neural network $(\sigma, \boldsymbol{c}) = f_\theta(\boldsymbol{x}, \boldsymbol{\omega})$.
3. **Train** with a view-synthesis loss using volume rendering: repeat *(a)* draw a batch of (image, pixel) pairs; *(b)* build their rays; *(c)* sample points along each ray; *(d)* query f_θ; *(e)* composite with Theorem 2.4; *(f)* compute the squared error to the pixel colours; *(g)* update θ by gradient descent.

[IMAGE PLACEHOLDER]
Slide **24**: Step 1 of the recipe: many camera viewpoints around the object (here a drum set); each camera is paired with its image. Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

6 Learning Neural Radiance Fields: An Initial Attempt *(slides 25)*

Feeding the raw coordinates $(\boldsymbol{x}, \boldsymbol{\omega})$ to the MLP and training as above gives disappointing results (slide 25): from 100 training images the network produces *blurry* novel views — **it cannot capture high-frequency details**.

[IMAGE PLACEHOLDER]
Slide **25**: Slide 25: 100 training images → optimised neural net → novel-view rendering of the lego scene (blurry result; first attempt without positional encoding). Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

7 Modeling High Frequencies: A Toy Problem *(slides 26–30)*

The slides isolate the phenomenon on a 2D toy problem: regress an image, $(x, y) \to f_\theta \to (r, g, b)$ (slide 26). A "standard" MLP returns a smooth, blurry image, because ReLU MLPs trained by gradient descent fit low frequencies first (*spectral bias*). The remedy: lift the input to a vector of sines and cosines at geometrically increasing frequencies.

[IMAGE PLACEHOLDER]
Slide **26, 27**: Image regression toy problem: input image, result with a "standard" MLP (blurry), and result using positional encoding (sharp). Figure credits: Tancik et al., NeurIPS 2020 (Fourier Features).

Definition 7.1 (Positional encoding)**.** For a scalar or vector $\boldsymbol{v}$ (applied coordinate-wise) and a number of octaves L,

$$\gamma(\boldsymbol{v}) = \big(\boldsymbol{v},\ \sin(\boldsymbol{v}), \cos(\boldsymbol{v}),\ \sin(2\boldsymbol{v}), \cos(2\boldsymbol{v}),\ \sin(4\boldsymbol{v}), \cos(4\boldsymbol{v}), \ldots,\ \sin(2^{L-1}\boldsymbol{v}), \cos(2^{L-1}\boldsymbol{v})\big).$$

(The slide lists the sin / cos terms; the original NeRF also multiplies the argument by π, i.e. $\sin(2^k \pi v)$, for coordinates normalised to $[-1, 1]$, and uses $L = 10$ for positions and $L = 4$ for directions. Whether $\boldsymbol{v}$ itself is kept is an implementation detail.) The network is now trained on $\gamma(\boldsymbol{x})$ and $\gamma(\boldsymbol{\omega})$ instead of $\boldsymbol{x}$ and $\boldsymbol{\omega}$.

Lemma 7.2 (γ induces a shift-invariant similarity)**.** *For scalars v, v',*

$$\langle \gamma(v), \gamma(v') \rangle_{\sin / \cos} = \sum_{k=0}^{L-1} \cos\big(2^k (v - v')\big)$$

which depends only on $v - v'$. For vectors, sum over the coordinates.

Proof. $\sin a \sin b + \cos a \cos b = \cos(a - b)$ applied with $a = 2^k v$, $b = 2^k v'$ for each k. □

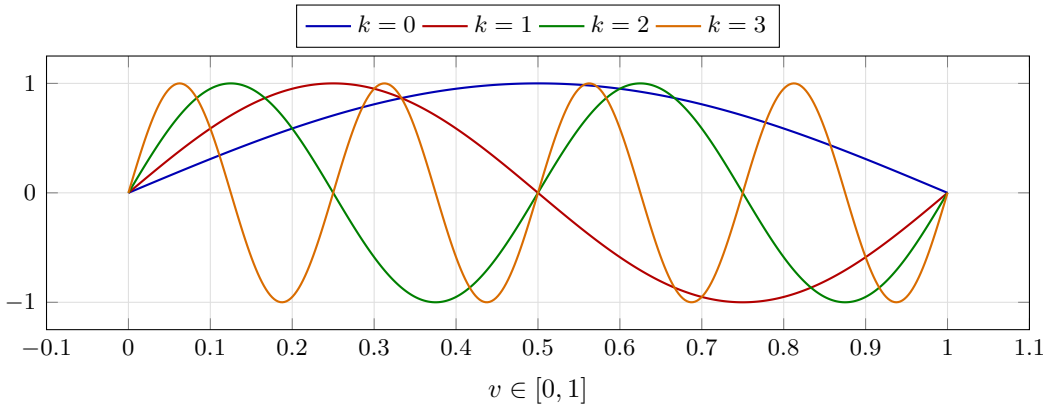


Figure 225: Original illustration: the first four components $\sin(2^k \pi v)$ of the positional encoding. Each octave doubles the frequency, so a few components span a wide range of scales; with $L = 10$ the finest component has a period of $2/2^9 = 1/256$ of the normalised coordinate range.

Intuition

Why does this help? Without encoding, two nearby points have nearly identical inputs, so the network treats them nearly identically (it is "smooth" in $\boldsymbol{x}$). After encoding, the high-frequency components $\sin(2^k \cdot)$ with large k change very fast with $\boldsymbol{x}$: two points 0.004 apart look completely different to the network, which can then give them different densities and colours. Lemma 7.2 says the network's notion of "similar points" becomes a band-limited kernel with bandwidth $\approx 2^{L-1}$ that we can tune, rather than a very smooth one. This is the Fourier-features argument of Tancik et al.: "Fourier features let networks learn high-frequency functions in low-dimensional domains".

[IMAGE PLACEHOLDER]
Slide **28**: Slide 28: with positional encodings at the input, the same pipeline (100 training images → optimised neural net → novel views) produces a sharp lego bulldozer — "a great example that execution matters". Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

[IMAGE PLACEHOLDER]
Slide **29**: Slide 29: NeRF (naive) vs. NeRF (with positional encoding) on the "fern" scene — the encoded version recovers leaf detail. Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

[IMAGE PLACEHOLDER]
Slide **30**: Slide 30 (video frame): standard input vs. Fourier-feature input for a 3D shape, a 3D MRI and a 3D NeRF. Video credits: Tancik and Srinivasan et al., NeurIPS 2020 (Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains).

8 Uniform to Hierarchical Sampling *(slides 31–33)*

The rendering sum of Theorem 2.4 is a quadrature of the volume-rendering integral along the ray, and each term costs one MLP evaluation. Uniform spacing (slide 31) is wasteful: many samples fall in empty space or behind an opaque surface, where the weight w_i is tiny. Slides 32–33 therefore reuse

$$L(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_i w_i \, \boldsymbol{c}_i, \qquad w_i = T_i\big(1 - e^{-\sigma_{t_i} \Delta t}\big),$$

where the *weights indicate how much each segment contributes to the pixel value* (red = high, yellow = low). Strategy: (1) sample points uniformly; (2) sample *more* points in the segments with large weights.

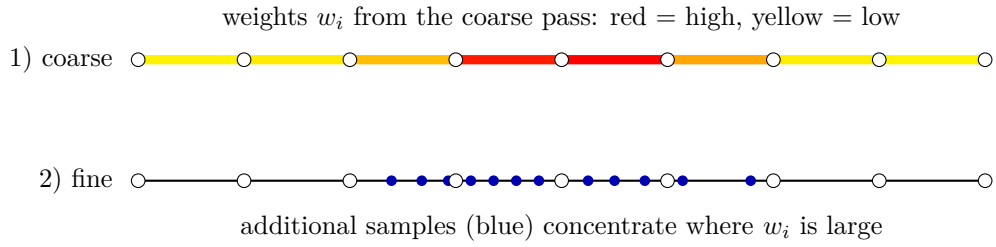


Figure 226: Hierarchical sampling (redrawn and simplified from slides 31–33). A first, uniform "coarse" pass computes the weights; a second pass draws extra samples with density proportional to them. (Sample positions are schematic.)

Definition 8.1 (Piecewise-constant PDF along the ray)**.** Normalise the coarse weights, $\hat{w}_i = w_i / \sum_j w_j$, and define the density on $[t_1, t_{N+1}]$ that equals $\hat{w}_i / \Delta t_i$ on the i-th segment and the cumulative distribution $F(t) = \int_{t_1}^{t} p(s)\,\mathrm{d}s$, which is piecewise linear.

Lemma 8.2 (Inverse-transform sampling)**.** *If $U \sim \mathcal{U}[0,1]$ and F is a strictly increasing CDF, then $t = F^{-1}(U)$ has CDF F.*

Proof. $\Pr[F^{-1}(U) \le t] = \Pr[U \le F(t)] = F(t)$, since F is monotonic and U is uniform. (For a piecewise-linear CDF with flat parts, take the left-continuous generalised inverse.) □

Algorithm 8.1 (Hierarchical sampling)**.**
1. Sample N_c points uniformly along the ray (one random point inside each of N_c equal bins in the original NeRF) and evaluate the network ("coarse" pass).
2. Compute the weights w_i and the normalised piecewise-constant PDF $\hat{w}_i$.
3. Draw N_f further points $t = F^{-1}(u_j)$ with $u_j \sim \mathcal{U}[0,1]$ (Lemma 8.2).
4. Evaluate the network on the sorted union of all $N_c + N_f$ points and render the pixel ("fine" pass). Original NeRF: $N_c = 64$, $N_f = 128$, with a coarse and a fine network, both supervised.

Example 8.3 (Inverse CDF by hand)**.** Four bins of unit length on $t \in [0, 4]$ with normalised weights $(0.1, 0.5, 0.3, 0.1)$ give the CDF values $(0, 0.1, 0.6, 0.9, 1)$ at $t = 0, 1, 2, 3, 4$. For $u = 0.5$: $F(t) = 0.5$ lies in bin 2 (between 0.1 and 0.6) at fraction $(0.5 - 0.1)/0.5 = 0.8$ of the way, so $t = 1 + 0.8 = 1.8$. Half of the random numbers $u \in [0.1, 0.6]$ fall in bin 2, so bin 2 receives half of the new samples although it is only a quarter of the ray length.

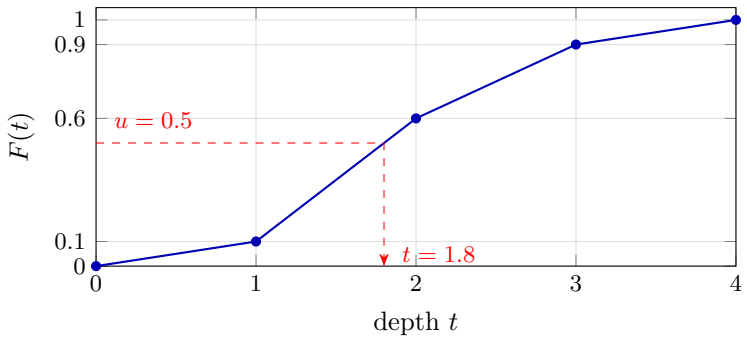


Figure 227: Original illustration: inverse-CDF sampling for the example above. The steep part of F (bin 2) is hit by many uniform u's.

Intuition

It is *importance sampling* with the weights as importance function: the weights tell us where the pixel colour is actually decided (typically near the first visible surface), and we spend our (expensive) network queries there. Empty space and occluded space, where $w_i \approx 0$, are nearly skipped. For a fixed budget this gives a sharper picture than uniformly spending all queries.

9 Depth vs Inverse-Depth Sampling *(slides 34–36)*

Uniform depth sampling (slide 34) is informative for scenes with limited depth variation, e.g. a camera orbiting an object. In a forward-facing, real scene the content can lie anywhere between a near point and infinity (slide 35: a plant very close, a wall far away); uniform spacing then places almost all samples far away, while near geometry gets very few. Alternative: sample *uniformly in* $1/d$ — a user-driven choice in NeRF: sampling in depth or in inverse-depth space.

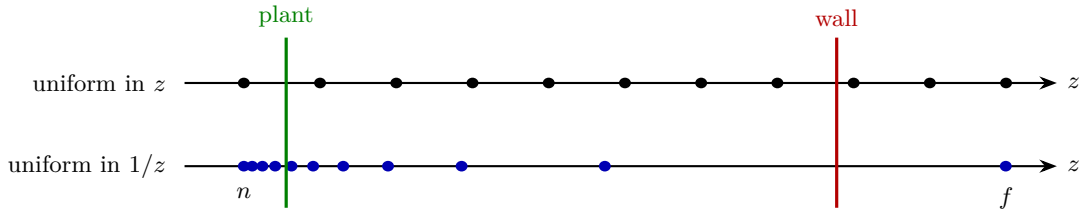


Figure 228: Original illustration, $n = 1$, $f = 10$, $N = 10$ intervals. Uniform depth puts 8 of the 11 points beyond $z = 3$; uniform inverse depth puts 7 of them within $z \leq 2.2$ and still reaches the far plane. The nearby plant is resolved by several samples in the second row.

9.1 Normalised device coordinates *(slides 36)*

The NeRF paper (Appendix C) maps the depth range into $[-1, 1]$ using normalised device coordinates (NDC) because real captures contain content "anywhere between the closest point and infinity". With near and far planes n, f and camera-space depth $z_c > 0$:

Proposition 9.1 (NDC depth is affine in inverse depth)**.**

$$z_{\text{nd}} = \frac{f+n}{f-n} - \frac{2fn}{(f-n)\,z_c} \in [-1, 1].$$

Hence sampling z_{nd} uniformly is the same as sampling $1/z_c$ uniformly. For $f \to \infty$, uniform inverse-depth samples are $z_i = \dfrac{n}{1 - i/N}$, whereas uniform depth samples are $z_i = n + \frac{i}{N}(f - n)$.

Proof. A perspective projection yields a depth of the form $z_{\text{nd}} = A + B/z_c$. The conditions $z_{\text{nd}}(n) = -1$ and $z_{\text{nd}}(f) = +1$ give $A + B/n = -1$ and $A + B/f = 1$, so $B = \frac{2}{1/f - 1/n} = -\frac{2fn}{f-n}$ and $A = 1 - B/f = \frac{f+n}{f-n}$. Since z_{nd} is affine in $1/z_c$, equal steps in z_{nd} are equal steps in $1/z_c$: $1/z_i = (1 - s_i)/n + s_i/f$ with $s_i = i/N$; letting $f \to \infty$ gives $z_i = n/(1 - s_i)$. □

Example 9.2. For $n = 1$, $f = 10$: uniform depth gives $z = 1, 1.9, 2.8, \ldots, 10$; uniform inverse depth gives $z = 1/(1 - 0.09i) = 1$, 1.10, 1.22, 1.37, 1.56, 1.82, 2.17, 2.70, 3.57, 5.26, 10. Spacing grows steadily: the first gap is 0.10, the last one 4.74.

Intuition

A camera sees a nearby thing and a far thing at very different pixel sizes: close geometry deserves a fine depth resolution, distant geometry does not need it (it moves little on the image as the camera moves — parallax is proportional to $1/z$). Sampling uniformly in $1/z$ spends samples proportionally to how much depth matters on the image, and lets the far plane go to infinity at the cost of one extra sample. A good detailed explanation of NDC is linked on slide 36 (`songho.ca/opengl/gl_projectionmatrix.html`).

10 Novel-View Synthesis and View-Dependent Effects *(slides 37–41)*

10.1 Novel-View Synthesis *(slides 37)*

[IMAGE PLACEHOLDER]
Slide **37**: Novel-view synthesis results of NeRF on real captured scenes (a triceratops skull and a mosaic water fountain): smooth camera paths through views never seen in training. Video credits: Mildenhall et al., ECCV 2020 (NeRF).

10.2 View-Dependent Effects *(slides 38–40)*

Definition 10.1 (View-dependent effect)**.** A *view-dependent* effect is an appearance change that results from the viewing direction alone, while geometry and light stay fixed — specular highlights, reflections, glossy materials. In radiance-field terms: $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_1) \neq \boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_2)$ for the *same* 3D point.

Proposition 10.2. *A representation with $L_e(\boldsymbol{x}, \boldsymbol{\omega}) \equiv \boldsymbol{c}(\boldsymbol{x})$ (the grid of Section 3, slide 13) cannot reproduce a scene in which the same 3D point has different colours in two training images.*

Proof. By construction $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_1) = \boldsymbol{c}(\boldsymbol{x}, \boldsymbol{\omega}_2)$; one value cannot match two different observations. The best it can do is average them, or distort the geometry (put density at wrong places) to explain the inconsistency. □

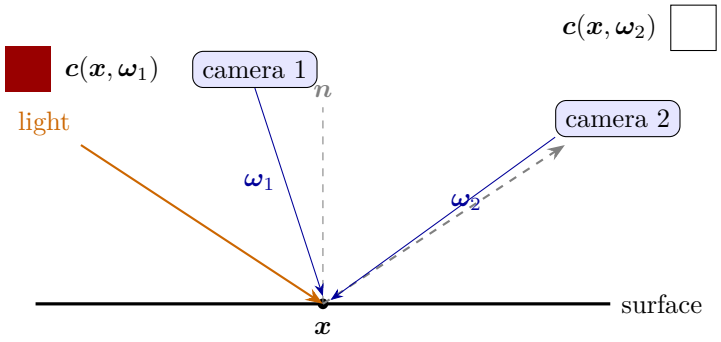


Figure 229: Original illustration: a glossy surface point seen from two directions. Camera 2 lies near the mirror direction of the light and sees a bright highlight; camera 1 sees only the dark body colour. A NeRF captures this through the dependence of $\boldsymbol{c}$ on $\boldsymbol{\omega}$.

[IMAGE PLACEHOLDER]
Slide **38**: Slide 38: same 3D point, different colour based on the viewing direction — zoomed patches from two views of the synthetic ship scene (left and right) and the colour as a function of the viewing direction. Figure credits: Mildenhall et al., ECCV 2020 (NeRF).

[IMAGE PLACEHOLDER]
Slide **39, 40**: Slides 39 and 40 (video frames): real scenes (a kitchen stove and a conference room) with highlights on metal and on the screen that move as the camera moves. Video credits: Mildenhall et al., ECCV 2020 (NeRF).

Intuition

For a perfectly matte (Lambertian) surface, colour does not depend on the viewpoint and a grid with $\boldsymbol{c}(\boldsymbol{x})$ would be enough. Real materials are not matte: the highlight on a pan slides across it when you walk around. NeRF handles it because the network gets $\boldsymbol{\omega}$ as input, but only in the *last* layers (Proposition 4.2): "geometry first, appearance later".

10.3 Learning Neural Radiance Fields: a simple recipe *(slides 41)*

Slide 41 closes the NeRF part: positional encoding at the input, a plain MLP, volume rendering and a photometric loss. It is a "very simple" representation and learning approach which **inspired many, many extensions and follow-ups**; the rest of the lecture visits some of them.

11 Is ray sampling accurate? *(slides 42)*

So far one pixel = one ray through its centre. But a real pixel integrates light over a small *footprint* on the sensor.

Definition 11.1 (Pixel value with a box filter)**.** The value of pixel $\boldsymbol{p}$ is $I[\boldsymbol{p}] = \iint_{\text{pixel}} \text{render}(\boldsymbol{q})\,\mathrm{d}\boldsymbol{q}$ (averaged over the pixel area), whereas the *point-sampled* approximation uses only $\text{render}(\boldsymbol{p}_{\text{center}})$.

[IMAGE PLACEHOLDER]
Slide **42**: Slide 42: a checkerboard plane rendered with one ray through each pixel centre (aliased, moiré pattern near the horizon) versus averaged over the whole pixel (box filter, smooth).

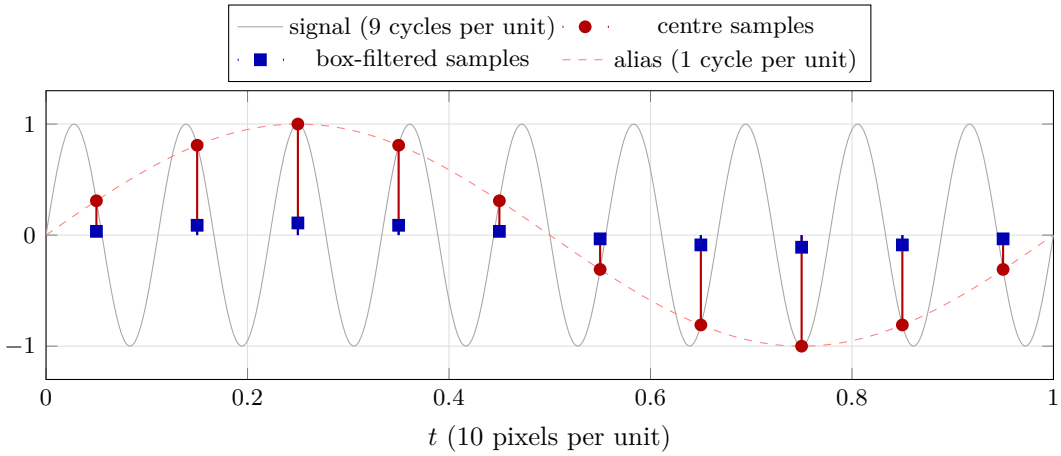


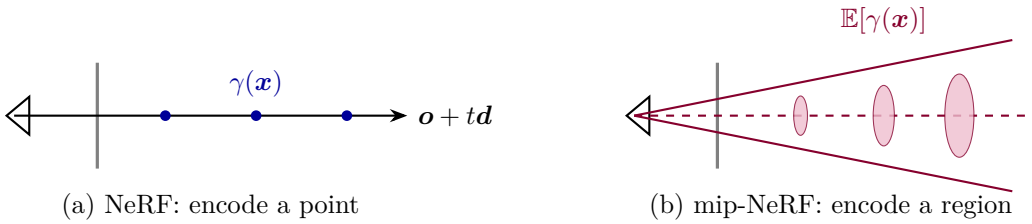
Figure 230: Original illustration of aliasing. The signal $\sin(2\pi \cdot 9t)$ is far above the Nyquist limit of 5 cycles/unit for 10 pixels. Centre sampling returns a full-amplitude *false* low-frequency pattern $\sin(2\pi t)$ (the moiré). Averaging over the pixel width 0.1 multiplies the amplitude by $\text{sinc} = \sin(0.9\pi)/(0.9\pi) \approx 0.11$, so the unresolvable detail is attenuated instead of impersonating a coarse one.

Intuition

A single ray through the pixel centre answers "what colour is the world at this exact point?" whereas the camera asked "what is the *average* colour of this little patch?". When the patch covers many fine details (distant checkerboard squares, or a scene viewed from far away) the two answers differ a lot. NeRF trained on images of different resolutions or distances then gets conflicting supervision from the same 3D point: sharp renders at close range look jagged/aliased, and blurry when it tries to compromise.

12 Pixels as Cones of Rays *(slides 43–46)*

Mip-NeRF (Barron et al., ICCV 2021) treats a pixel as a *cone* (a frustum of cone between two depths t_0, t_1) instead of a line. Slide 43's key sentence: *instead of the position encoding of a single sample, use the expected position encoding.*



(a) NeRF: encode a point (b) mip-NeRF: encode a region

Figure 231: Pixels as cones of rays (redrawn from slide 43). A conical frustum covering the pixel footprint between two depths is summarised by a Gaussian with mean $\boldsymbol{\mu}$ and covariance Σ; the network receives the expected (integrated) positional encoding of that Gaussian. Figure credits: Barron et al., ICCV 2021 (mip-NeRF).

Lemma 12.1 (Integrated positional encoding)**.** *Let $X \sim \mathcal{N}(\mu, s^2)$ be scalar. For every frequency $a > 0$,*

$$\mathbb{E}[\sin(aX)] = \sin(a\mu)\, e^{-a^2 s^2/2}, \qquad \mathbb{E}[\cos(aX)] = \cos(a\mu)\, e^{-a^2 s^2/2}.$$

Hence for $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ coordinate-wise with $\Sigma_{jj} = s_j^2$ the expected encoding of the k-th band is $\sin(2^k \mu_j)\exp(-\frac{1}{2}4^k s_j^2)$ and $\cos(2^k \mu_j)\exp(-\frac{1}{2}4^k s_j^2)$.

Proof. $\mathbb{E}[e^{iaX}] = e^{ia\mu - a^2 s^2/2}$ is the characteristic function of a Gaussian. Taking imaginary and real parts

gives $\mathbb{E}[\sin aX] = \Im$ and $\mathbb{E}[\cos aX] = \Re$ of $e^{ia\mu}e^{-a^2s^2/2}$. With $a = 2^k$ we have $a^2 = 4^k$. $\square$

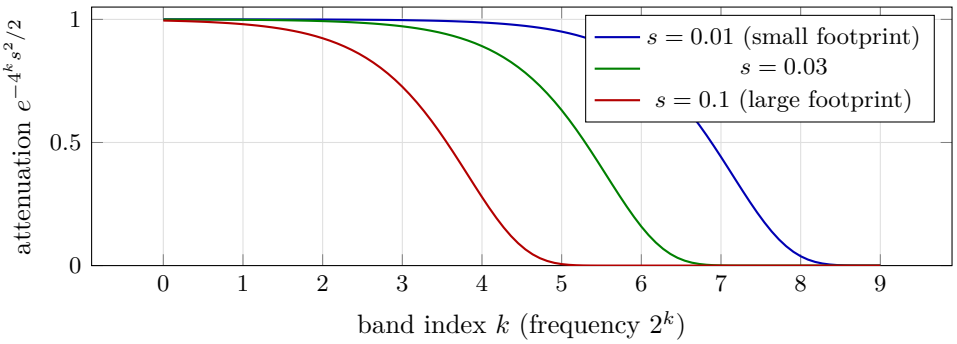


Figure 232: Original illustration of Lemma 12.1: the factor multiplying band k of the integrated encoding. A larger footprint s switches off the high-frequency bands, while a small footprint leaves them intact.

Intuition

When the pixel's footprint is large, details finer than the footprint are averaged away in the real photograph. The integrated encoding mimics this: bands whose wavelength is shorter than the footprint are smoothly *damped to zero*, while coarse bands pass through untouched; the network never sees a high frequency it could not have observed. It also now knows *how large* the region is, so one network can be trained on images at several resolutions. Slide 45 (mip-NeRF paper) shows that this improves the fine details at lower resolutions with the PSNR/SSIM-type numbers printed next to the crops.

[IMAGE PLACEHOLDER]
Slide **45**: Slide 45: ground truth, NeRF, NeRF + (area, centre, misc), and mip-NeRF crops of the drum and microphone scenes at several scales, with the error scores printed next to them. Figure credits: Barron et al., ICCV 2021 (mip-NeRF).

[IMAGE PLACEHOLDER]
Slide **46**: Slide 46 (video frames): two renderings of the microphone scene compared side by side while the scale changes. Video credits: Barron et al., ICCV 2021 (mip-NeRF).

13 Volumes: Rendering and Representation — Option 2: An MLP, revisited *(slides 47)*

Slide 47 puts the MLP in perspective: it is **a very flexible representation** (continuous, compact, able to model view-dependent effects) but **can be slow to train** — several hours for a single scene.

Proposition 13.1 (Cost of one training iteration)**.** *If a batch has m rays and each ray needs $N_c + N_f$ network evaluations (Section 7), one forward–backward pass evaluates the MLP $m(N_c + N_f)$ times.*

Intuition

With e.g. $m = 4096$ rays and $N_c + N_f = 64 + 128 = 192$ samples, one iteration requires about 786,000 evaluations of a 9-layer, 256-wide MLP — and hundreds of thousands of iterations are needed. Everything that follows tries to make the *query* cheaper: put the information in a grid, which only needs a few memory reads per point, instead of a deep network.

14 Option 3: Voxels with view-dependent radiance — Do We Need Neural Radiance? *(slides 48–51)*

Plenoxels (Fridovich-Keil, Yu et al., CVPR 2022) answer "*do we need neural radiance?*" with *no*: keep the sparse voxel grid of Option 1 but fix its two weaknesses — the missing view dependence and the cubic memory — without any neural network (slide 48).

14.1 Spherical harmonics for view-dependent colour *(slides 49)*

Definition 14.1 (Spherical harmonics)**.** The (real) spherical harmonics $Y_\ell^m : \mathbb{S}^2 \to \mathbb{R}$, $\ell \geq 0$, $-\ell \leq m \leq \ell$, are an orthonormal basis of square-integrable scalar functions on the sphere: $\int_{\mathbb{S}^2} Y_\ell^m Y_{\ell'}^{m'} \, \mathrm{d}\boldsymbol{\omega} = \delta_{\ell\ell'}\delta_{mm'}$. Degree ℓ contributes $2\ell+1$ functions, so truncating at degree $\ell_{\max}$ keeps $(\ell_{\max}+1)^2$ of them.

Definition 14.2 (View-dependent colour from basis coefficients)**.** Each voxel stores, for each colour channel, coefficients k_ℓ^m and the colour seen from direction $\boldsymbol{\omega}$ is

$$\boldsymbol{c}_{\mathrm{ch}}(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_{\ell \leq \ell_{\max}} \sum_{m=-\ell}^{\ell} k_{\ell,\mathrm{ch}}^m(\boldsymbol{x}) \, Y_\ell^m(\boldsymbol{\omega})$$

(followed in practice by a sigmoid or clamp to enforce $[0, 1]$). Plenoxels uses $\ell_{\max} = 2$: 9 coefficients per channel, 27 for RGB, plus 1 for the density, i.e. 28 numbers per voxel.

Lemma 14.3 (Linearity in the stored coefficients)**.** *The colour is a linear function of the coefficients, so trilinear interpolation of the coefficients of the eight corners and evaluation at $\boldsymbol{\omega}$ commutes: interpolating coefficients first and then evaluating $\sum kY$ gives the same as evaluating each corner's colour first and then interpolating. Moreover $\partial \boldsymbol{c}_{\mathrm{ch}} / \partial k_\ell^m = Y_\ell^m(\boldsymbol{\omega})$ (times the interpolation weight).*

Proof. Both operations are linear combinations of the corner coefficients with weights λ_{ijk} (Lemma 3.3) and $Y_\ell^m(\boldsymbol{\omega})$, which do not depend on one another. The derivative is read off the formula. □

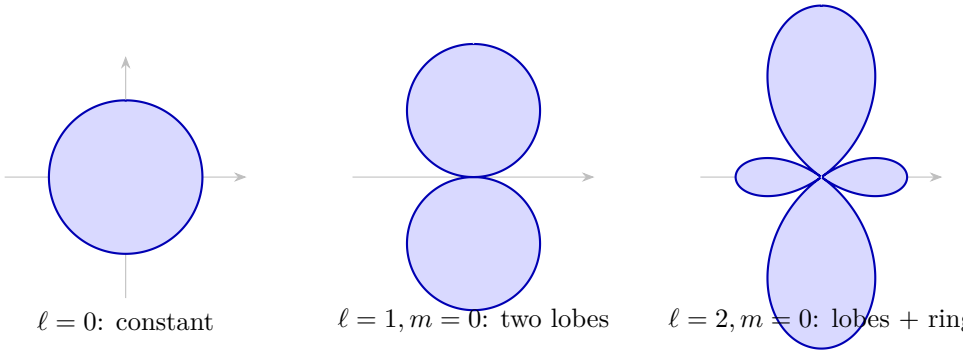


Figure 233: Original illustration (slide 49 shows the full pyramid of harmonics): polar plots of the magnitude $|Y_\ell^0(\theta)|$ of the zonal harmonics for $\ell = 0, 1, 2$ (vertical axis = polar axis; lobes alternate in sign). Higher degrees add angular detail, i.e. faster variation of colour with the viewing direction.

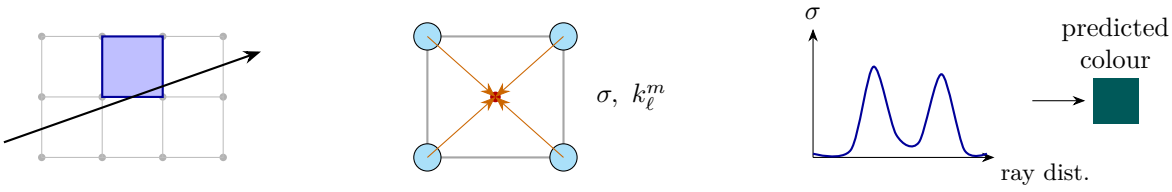
Intuition

Spherical harmonics are the Fourier series of the sphere. The $\ell = 0$ coefficient is the average, view-independent colour (the "diffuse" part); $\ell = 1$ lets the colour tilt towards one side of the sphere, and $\ell = 2$ adds richer variation like highlight shapes. Storing a handful of coefficients per voxel is therefore a compact table of *how the colour of this voxel changes around the sphere*, so the voxel no longer has the single colour of Option 1 — exactly the missing ingredient in slide 17 —, and no network is needed to get it.

14.2 The Plenoxels pipeline *(slides 50)*

Algorithm 14.1 (Plenoxels, slide 50).
1. **Sparse voxel grid.** At each voxel store the density and the spherical-harmonic coefficients for view-dependent colour.
2. **Trilinear interpolation** of density and coefficients at every sample along the ray (Lemma 3.3); evaluate $Y_\ell^m(\boldsymbol{\omega})$ to get the colour.
3. **Volumetric rendering** (Theorem 2.4) to obtain the predicted colour.
4. **Optimisation:** minimise $\mathcal{L}_{\text{recon}} + \lambda \mathcal{L}_{TV}$ over all voxel values directly by gradient descent.
5. For efficiency, **prune voxels with very low density** (they carry no visible information).

Here $\mathcal{L}_{\text{recon}}$ is the same photometric loss as for NeRF and $\mathcal{L}_{TV}$ a total-variation regulariser, which penalises large differences between neighbouring voxels, e.g. $\mathcal{L}_{TV} \propto \sum_v \sum_{d \in \{x,y,z\}} |\, f(v) - f(v + e_d) \,|$ for each stored quantity f.



(a) sparse voxel grid (b) trilinear interpolation (c) volumetric rendering

Figure 234: The Plenoxels pipeline (redrawn from slide 50; original figure by Fridovich-Keil, Yu et al., CVPR 2022): (a) sparse voxel grid; (b) trilinear interpolation of the eight corner values; (c) volumetric rendering along the ray; (d, not drawn) optimisation of the voxel values with $\mathcal{L}_{\text{recon}} + \lambda \mathcal{L}_{TV}$.

Intuition

There is nothing to "forward" through a network: a query is eight memory reads and a few multiplications, and the gradient w.r.t. a voxel value is just its interpolation weight (Lemma 3.3(iv)). That is why training takes minutes rather than hours. The price is more memory (28 numbers per voxel) and some noise when views are few, which TV smoothing fights.

14.3 NeRF vs. Plenoxels *(slides 51)*

[IMAGE PLACEHOLDER]
Slide **51**: Slide 51 (video frames): training NeRF (left) and Plenoxels (right) side by side with a `minutes:seconds` timer; Plenoxels reaches a recognisable result long before NeRF. Figure credits: Fridovich-Keil, Yu et al., CVPR 2022 (Plenoxels).

15 Towards Hybrid Representations *(slides 52–54)*

Option 4: feature hash-grid + lightweight MLP (Instant-NGP, Müller et al., SIGGRAPH 2022). The idea combines the two worlds: store *learned features* in a (hashed) grid for fast lookups, and let a *tiny* MLP map those features and the direction to density and colour (slide 53).

Definition 15.1 (Multiresolution hash encoding)**.** Use L resolution levels, with grid resolutions $N_\ell = \lfloor N_{\min} b^\ell \rfloor$ (geometric growth factor b). Level ℓ owns a table $\theta_\ell \in \mathbb{R}^{T\times F}$ of T entries with F features each. For $\boldsymbol{x} \in [0,1]^d$:

1. *Hashing of voxel vertices:* the 2^d integer vertices $\boldsymbol{c}$ of the cell containing $\boldsymbol{x}N_\ell$ are mapped to table slots by $h(\boldsymbol{c}) = \left(\bigoplus_{i=1}^{d} c_i\pi_i\right) \bmod T$, with large primes π_i ($\pi_1 = 1$, $\pi_2 = 2654435761$, $\pi_3 = 805459861$ in the paper) and $\oplus$ the bitwise XOR. If the level has at most T vertices it is indexed directly.
2. *Lookup* of the F-dimensional features of the vertices.
3. *(Multi)linear interpolation* of those features with the weights of Lemma 3.3 (the 2D version for $d = 2$ in the slide).
4. *Concatenation* of the L interpolated vectors (giving $L \cdot F$ numbers) with auxiliary inputs ξ (e.g. the encoded view direction): $\boldsymbol{y} \in \mathbb{R}^{LF+E}$.
5. *Neural network:* a tiny MLP $m(\boldsymbol{y}; \Phi)$ outputs density and colour.

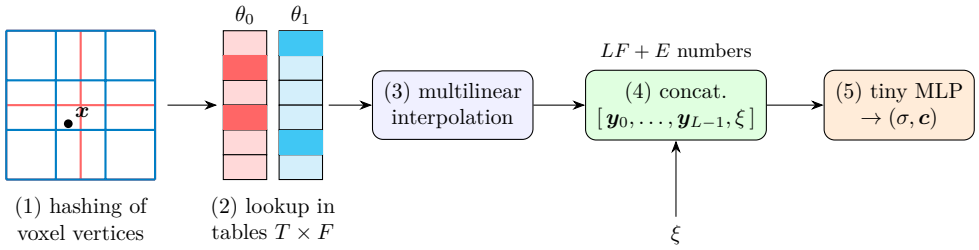


Figure 235: The hash-grid encoding (redrawn from slides 52–53; original figure by Müller et al., SIGGRAPH 2022). Two levels of a 2D toy grid are shown. Each level has its own small table; vertices from many places of space can share a table slot (hash collision).

Proposition 15.2 (Memory is fixed by the hyper-parameters, not by the scene)**.** *The encoding has $L \cdot T \cdot F$ trainable numbers regardless of the resolution N_L reached by the finest level, whereas a dense grid of the same finest resolution needs $(N_L + 1)^d$ vertices.*

Intuition

Example with the typical NeRF-style values $L = 16$, $F = 2$, $T = 2^{19}$ (as reported for Instant-NGP): $16 \cdot 524{,}288 \cdot 2 \approx 16.8$M numbers — about 34 MB in half precision — whether the finest resolution is 512 or 2048 per axis; a dense 2048^3 grid with 4 values per voxel in single precision would need ≈ 137 GB. How do we get away with this? Real scenes are mostly empty: only $\sim N^2$ vertices near surfaces matter, which is of the order of the table size, and when two vertices collide the network and the gradients (dominated by the vertex that matters more for the image) sort it out; the coarse levels have no collisions and give context. The tiny MLP (e.g. 1–2 hidden layers of 64 units in the paper) is cheap because the features already carry the spatial information.

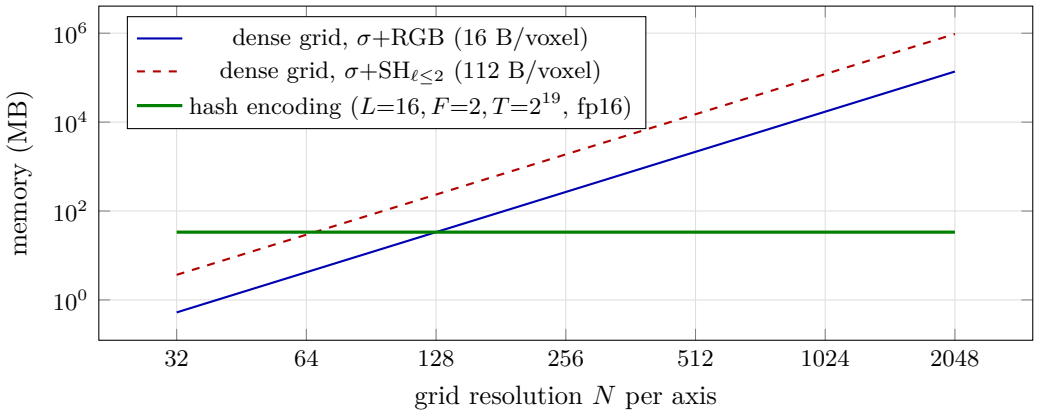


Figure 236: Original illustration: memory of dense grids grows as N^3 (a straight line of slope 3 on this log-log plot) while the hash encoding stays constant (assumed values $L = 16$, $F = 2$, $T = 2^{19}$, half precision). Plenoxels avoids most of the dense cost by storing only occupied voxels (sparse grid), so its real footprint is much smaller than the dashed line.

[IMAGE PLACEHOLDER]
Slide **54**: Slide 54 (video frames): Instant-NGP training on eight different scenes/images with an "Elapsed training time" counter — a usable result appears after seconds. Video credits: Müller et al., SIGGRAPH 2022 (Instant-NGP).

Remark. Slide 53 notes that the core idea — a hash function that efficiently stores per-voxel features, plus a tiny MLP that maps a query point's features (and direction) to colour and density — is applicable to the efficient modelling of general 2D/3D signals, not only radiance fields (images, signed distance functions, ...).

16 Summary *(slides 55)*

Slide 55: *last lecture*: volume rendering; *today*: neural representations for volumes, and training via view synthesis. The common recipe is: (representation of σ, L_e) $\rightarrow$ differentiable ray marching $\rightarrow$ photometric loss with posed images.

[IMAGE PLACEHOLDER]
Slide **55**: Slide 55: training images → MLP / sparse voxel grid with spherical harmonics → novel view of the lego scene. Figure credits: Mildenhall et al. (NeRF), Fridovich-Keil, Yu et al. (Plenoxels).

Method	**Representation**	**View-dep. colour**	**Key idea**	**Training time (slides)**	**Main limitation**
Voxel grid (Opt. 1)	$\sigma[x,y,z]$, $\boldsymbol{c}[x,y,z]$, trilinear interp.	No ($L_e \equiv \boldsymbol{c}(\boldsymbol{x})$)	Store values on a lattice; differentiable lookups	fast queries	cubic memory; no view effects
NeRF (Opt. 2)	MLP $f_\theta(\boldsymbol{x},\boldsymbol{\omega}) \to (\sigma,\boldsymbol{c})$	Yes, $\boldsymbol{\omega}$ injected late	Photometric loss through volume rendering; positional encoding; hierarchical sampling	several hours per scene	slow to train; blurry without γ
mip-NeRF	NeRF with cones and integrated encoding $\mathbb{E}[\gamma(\boldsymbol{x})]$	Yes	Pixels are footprints, not points; anti-aliasing across scales	like NeRF	still an MLP: slow
Plenoxels (Opt. 3)	sparse voxels: σ + SH coefficients	Yes (spherical harmonics)	No network; optimise voxels directly with $\mathcal{L}_{\text{recon}} + \lambda\mathcal{L}_{TV}$; prune empty voxels	minutes	large memory per voxel (28 numbers); noise with few views
Instant-NGP (Opt. 4)	multires hash-grid features + tiny MLP	Yes (direction goes into the MLP)	Fixed-size learned hash tables; tiny MLP	seconds to minutes	hash collisions; fixed table size

Table 15: Comparison of the representations seen in the lecture. All share the same rendering equation (Theorem 2.4) and photometric training signal; they differ in how $\sigma(\boldsymbol{x})$ and $L_e(\boldsymbol{x},\boldsymbol{\omega})$ are stored and queried.

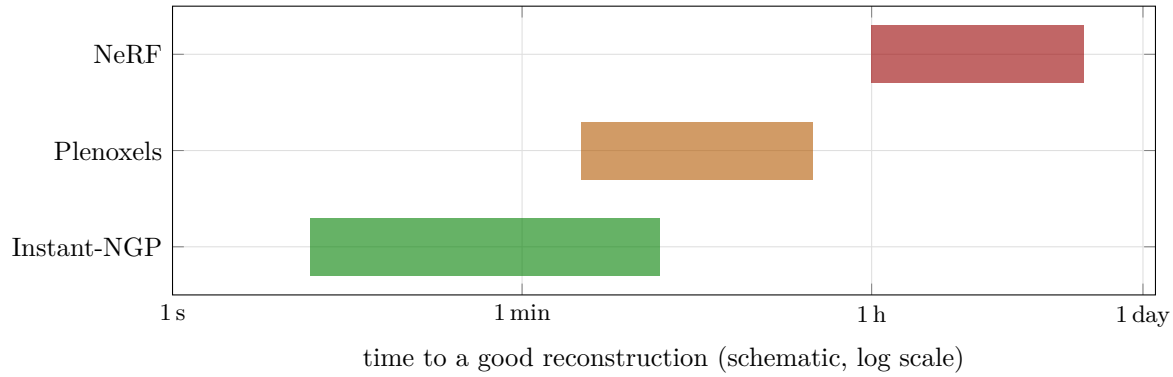


Figure 237: Qualitative comparison (schematic, orders of magnitude only; not measured here): the slides only state "several hours" for NeRF, and show timers of minutes for Plenoxels and seconds for Instant-NGP. Moving information from deep networks into explicit/hashed grids buys one to three orders of magnitude in training time.

Key take-aways

- **Volume rendering is the differentiable bridge.** Density and emitted light along a ray give a pixel via $L = \sum_i T_i(1-e^{-\sigma_i \Delta t_i})\boldsymbol{c}_i$ (+ background $T_{N+1}L_{\text{bg}}$), with weights summing to one; gradients flow to densities, colours and, if these are differentiable functions of position and direction, to the camera.
- **A radiance field is any function $(\boldsymbol{x}, \boldsymbol{\omega}) \mapsto (\sigma, \boldsymbol{c})$.** It can be a grid (fast but cubic memory, no view effects), an MLP (compact, flexible, slow), voxels with spherical harmonics, or a hash-grid with a tiny MLP; training is always "match the posed images" (view synthesis loss on randomly sampled rays).
- **Details make or break NeRF:** positional encoding γ overcomes spectral bias; density depends on $\boldsymbol{x}$ only while colour also on $\boldsymbol{\omega}$; hierarchical (importance) sampling spends queries where w_i is large; inverse-depth/NDC sampling handles unbounded scenes.
- **Pixels are areas, not points:** mip-NeRF replaces the encoding of a single point by the expected (integrated) encoding of the pixel's conical frustum, which damps frequencies finer than the footprint and removes aliasing.
- **Hybrid representations trade neural computation for memory lookups:** sparse voxels + SH (Plenoxels) and multiresolution hash features + tiny MLP (Instant-NGP) reduce training from hours to minutes or seconds.

Credits. Slides adapted from *Learning for 3D Vision*, CMU 16-825 (Prof. Shubham Tulsiani). Ray-marching figures by Ioannis (Yannis) Gkioulekas. Results and diagrams from Mildenhall et al., ECCV 2020 (NeRF); Tancik et al., NeurIPS 2020 (Fourier Features); Barron et al., ICCV 2021 (mip-NeRF); Fridovich-Keil, Yu et al., CVPR 2022 (Plenoxels); Müller et al., SIGGRAPH 2022 (Instant-NGP).

Part VIII

Lecture 9

How to read these notes. Sections follow the slide deck; the slide range is shown in each header. Formal statements are in coloured boxes, blue *Intuition* boxes explain them in plain words, and dashed *placeholder* boxes indicate which slide image to paste there (photos, renderings and videos are not redrawn). All diagrams drawn here are original TikZ/pgfplots figures. Only the slide deck was provided (no companion PDF), so results the slides only state informally are made precise inside the main flow; the Appendix contains supplementary material that is *not* on the slides.

1 Neural Implicit Rendering *(slides 1–2)*

The lecture asks how to *render* (and learn from images) a 3D scene whose geometry is stored as a neural *implicit surface* — the zero level set of a neural network — rather than as a density volume (NeRF). The running example on the slides is a signed distance field (SDF) of a teapot: white iso-contours around the object, with a ray (red arrow) shot towards the surface.

[IMAGE PLACEHOLDER]
Slide **slide 2 (also slides 11–13, 15–16, 21–22, 27, 29–30, 32, 37–38, 40–41)**: Teapot with SDF iso-contours and a query ray (Takikawa et al., NGLOD)

Intuition

Three questions organise the whole lecture (they are stated explicitly on the "Today's Goals" slide): (1) *where does a ray hit the surface, and can we differentiate that?* (2) *what colour do we assign to a surface point?* (3) *can we get the best of surfaces and volumes by turning an implicit surface into a density?*

2 Review: Volume Rendering *(slides 3–4)*

A pixel is not a single radiance sample

A pixel value is proportional to the radiance $L(\boldsymbol{x}^*, \boldsymbol{\omega})$ arriving at the sensor centre $\boldsymbol{x}^*$ from direction $\boldsymbol{\omega}$ (pointing from the scene point towards the optical centre). The slide stresses that this is *not* the radiance at one point: $L(\boldsymbol{x}^*, \boldsymbol{\omega}) \neq L(\boldsymbol{x}^* - \lambda\boldsymbol{\omega}, \boldsymbol{\omega})$, because along the ray the light is attenuated and emitted by the participating medium (the cloud in the figure). *The pixel value cannot be reduced to a radiance at a single point.*

Discrete volume rendering

Sample the ray at $\boldsymbol{x}_{t_i}$, $i = 1, \ldots, N$, with spacing Δt, density $\sigma_{t_i} = \sigma(\boldsymbol{x}_{t_i})$ and emitted radiance $L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$:

$$L(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_{i=1}^{N} T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) \left(1 - e^{-\sigma_{t_i} \Delta t}\right) L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega}), \qquad T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) = T(\boldsymbol{x}, \boldsymbol{x}_{t_{i-1}})\, e^{-\sigma_{t_{i-1}} \Delta t}. \tag{18}$$

The slide's message: we must **combine the contributions of multiple samples/segments along a ray**.

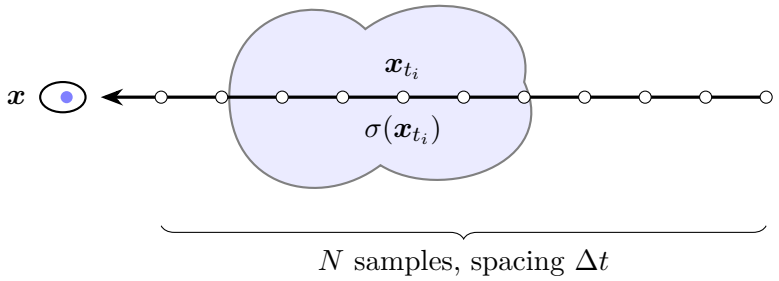


Figure 238: Redrawn from slide 4: samples $\boldsymbol{x}_{t_i}$ along the ray through a medium with density σ.

Lemma 2.1 (Rendering weights telescope)**.** *Let* $T_i := T(\boldsymbol{x}, \boldsymbol{x}_{t_i})$ *and* $w_i := T_i\,(1-e^{-\sigma_{t_i}\Delta t})$ *be the weight of sample* i *in* (18)*. Then* $T_i = \exp\!\big(-\sum_{j<i}\sigma_{t_j}\Delta t\big)$*,* $w_i = T_i - T_{i+1}$ *and hence* $\sum_{i=1}^{N} w_i = 1 - T_{N+1} \le 1$*.*

Proof. Unrolling the recursion with $T_1 = 1$ gives the exponential of the cumulative optical depth. Then $w_i = T_i - T_i e^{-\sigma_{t_i}\Delta t} = T_i - T_{i+1}$, and the sum telescopes. □

Intuition

T_i is the fraction of light that survives until sample i; w_i is the fraction that is *absorbed (or emitted) exactly in segment* i. Example with $\Delta t = 1$ and $\sigma = (0, 0.5, 2, 2)$: $T = (1, 1, 0.607, 0.082, 0.011)$ and $w = (0, 0.394, 0.524, 0.071)$, whose sum is $0.989 = 1 - T_5$. Almost all weight lies on the first dense segments: the ray "stops" where the medium becomes opaque.

3 (Neural) Volumes: A Very Expressive Representation *(slide 5)*

Paraphrasing the slide: *with great representation capacity comes greater risk of overfitting and local optima.* A neural density/radiance field can explain the training images in many ways, most of which are not physically sensible shapes. The slide shows NeRS (Zhang et al., NeurIPS 2021) reconstructions of two cars from few views, where volumes collapse to blurry, view-inconsistent blobs.

[IMAGE PLACEHOLDER]
Slide **slide 5**: Photos of a red pick-up and a blue SUV with their poor volumetric reconstructions (NeRS)

Intuition

A volume has one free number per point in space (density) plus colour; nothing forces the density to concentrate on a 2D surface. Fitting a handful of images is under-constrained, so the optimiser may pick a "cloud" that reproduces the pixels but is not the object.

4 Back to Surfaces *(slides 6–8)*

A sample result (slide 6)

NeRF-like representations do not necessarily capture geometry well: the extracted NeRF surface is noisy, while the NeuS geometry (Wang et al., NeurIPS 2021) is smooth, with a faithful rendering.

[IMAGE PLACEHOLDER]
Slide **slide 6**: NeRF geometry vs. "our geometry" and "our rendering" (NeuS)

Training in a few seconds (slide 7)

The slide shows a video of Instant-NGP (Müller et al., SIGGRAPH 2022): eight scenes being trained, with an "elapsed training time" counter (0 seconds at the start).

[IMAGE PLACEHOLDER]
Slide **slide 7**: Video frame, Instant-NGP training grid (elapsed time 0 s)

Key idea (slide 8)

The classical NeRF scenes (chair, lego, materials, drums, mic, hotdog, ship, ficus) are *all examples of scenes with well-defined surfaces (except the water).*

Key idea

Let's **enforce that the geometry corresponds to shapes with (well-defined) surfaces!**

[IMAGE PLACEHOLDER]
Slide **slide 8**: Faded NeRF synthetic scenes behind the key-idea text (Mildenhall et al.)

Intuition

If the real world is made of surfaces, a representation that can *only* express surfaces has a smaller hypothesis space, so fewer wrong explanations of the images — at the price of harder optimisation, as we will see.

5 Recap: 3D Representations *(slide 9)*

Representation	Weakness quoted on the slide
Point cloud	Difficult to extract a surface
Mesh (triangles, parametric patches)	Difficult to capture geometry with arbitrary topology and fine details (e.g. leaves, slats in chairs)
Voxel grid	Surface via marching cubes, but resolution is limited
Implicit surface	(the representation advocated next)

[IMAGE PLACEHOLDER]
Slide **slide 9**: Four representations: point cloud (cow), mesh (bunny, saddle patch), voxels, implicit surface

6 Recap: Implicit Surfaces *(slide 10)*

Definition 6.1 (Implicit surface)**.** Given a continuous $f : \mathbb{R}^3 \to \mathbb{R}$, the *implicit surface* it parametrises is its zero level set $\mathcal{S} = \{\boldsymbol{p} \mid f(\boldsymbol{p}) = 0\}$.

f "can be anything — a simple function, or a neural network" (slide code: input $\boldsymbol{p} \in \mathbb{R}^3$, output a scalar).

Intuition

Think of f as an altitude map in 3D: the surface is the "sea level" $f = 0$, the region $f < 0$ is inside and $f > 0$ is outside. Topology (holes, handles, several components) comes for free, and resolution is limited only by the function, not by a grid.

7 Neural Implicit Surfaces *(slides 11–12)*

Take f_θ to be a neural network with parameters θ: $\{\boldsymbol{p} \mid f_\theta(\boldsymbol{p}) = 0\}$ *allows representing complex geometry.* Slide question: *what additional constraint is required for a signed distance field?* Answer (slide 12): $\|\nabla f\| = 1$.

Definition 7.1 (Signed distance function (SDF))**.** For a closed surface $\mathcal{S}$ bounding a region Ω, the SDF is $d_{\mathcal{S}}(\boldsymbol{p}) = \pm \min_{\boldsymbol{y} \in \mathcal{S}} \|\boldsymbol{p} - \boldsymbol{y}\|$, with the sign convention used in the sphere-tracing slides: positive outside, negative inside.

Proposition 7.2 (Eikonal property)**.** *Wherever an SDF is differentiable,* $\|\nabla d_{\mathcal{S}}\| = 1$.

Proof. $d_{\mathcal{S}}$ is 1-Lipschitz (moving by h changes the distance to the closest surface point by at most h), so $\|\nabla d\| \leq 1$. Conversely, moving from $\boldsymbol{p}$ straight towards its closest surface point $\boldsymbol{y}$ by a step h decreases the distance by exactly h, so the directional derivative in that direction has magnitude 1 and $\|\nabla d\| \geq 1$. □

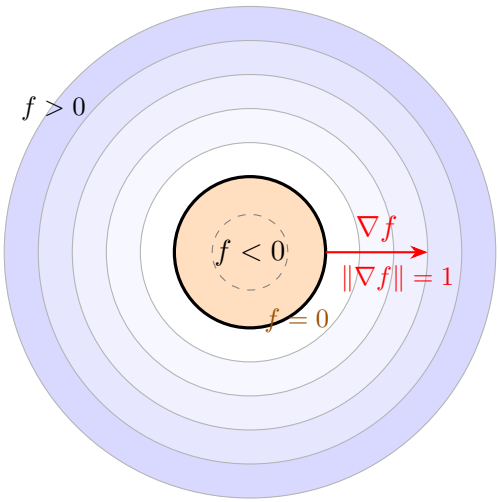


Figure 239: Original illustration: SDF of a disc. Iso-contours are equally spaced because $\|\nabla f\| = 1$.

Intuition

The iso-contours of an SDF are evenly spaced like the rings of a ripple: the value of f *is* the distance to the surface, so it grows with unit rate. A generic neural f_θ has the right zero set but arbitrary spacing; the *eikonal* penalty $\big|\|\nabla f\| - 1\big|$ (slide 33) nudges the network towards this nicer behaviour. The slide links an optional read on why distance functions satisfy an eikonal equation (Daniel Duffy's blog post).

8 Rendering Neural Implicit Surfaces *(slides 13–14)*

The appearance at each pixel is determined by a *unique surface point*. This raises two questions: **where does a ray intersect an implicit surface?** and **how to model appearance?**

Today's goals (slide 14)

1. Computational methods for (differentiable) implicit surface–ray intersection.
2. Modelling appearance (today: simple neural models; later: physically informed models).
3. Unifying surfaces and volumes: volume rendering for implicit surface representations.

9 Intersections of Rays with Implicit Surfaces *(slides 15–22)*

9.1 Problem statement *(slides 15–16)*

Input: implicit representation $f_\theta(\cdot)$ and a query ray $\boldsymbol{r} : \boldsymbol{x}_0 + t\boldsymbol{\omega}$. **Output:** the intersection $\boldsymbol{p} \equiv t^*\boldsymbol{\omega} + \boldsymbol{x}_0$ such that $f_\theta(\boldsymbol{p}) = 0$ (for now assume the ray does hit the surface; revisited later). Defining the restriction $\bar{f}_\theta(t) \equiv f_\theta(\boldsymbol{x}_0 + t\boldsymbol{\omega})$, this is **a 1-d search problem!**

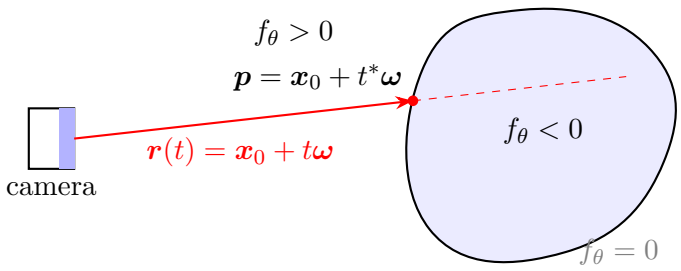


Figure 240: Original sketch for slides 13–16: the ray meets the zero level set of f_θ at t^*.

9.2 Searching for roots of a 1-d function: secant method *(slides 17–18)*

Algorithm 9.1 (Secant method)**.** Given (x_0, f_0) and (x_1, f_1) with $f_i = f(x_i)$, form the linear approximation

$$y_2 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x_2 - x_1) + f(x_1).$$

Its root ($y_2 = 0$) is

$$x_2 = x_1 - f(x_1)\frac{x_1 - x_0}{f(x_1) - f(x_0)},$$

then repeat with (x_1, x_2).

It is a general method, applicable for any f; but it is *not guaranteed to converge if far away from the root (or if f is not "nice")*. Other methods (e.g. Newton's) could be used. **Can we do better if rendering an implicit SDF and not an arbitrary function f?**

Intuition

Replace the curve by the straight line through the last two points and jump to where that line crosses zero. It is fast when f is locally almost linear, but a wiggly neural f far from the root can send the iterate anywhere.

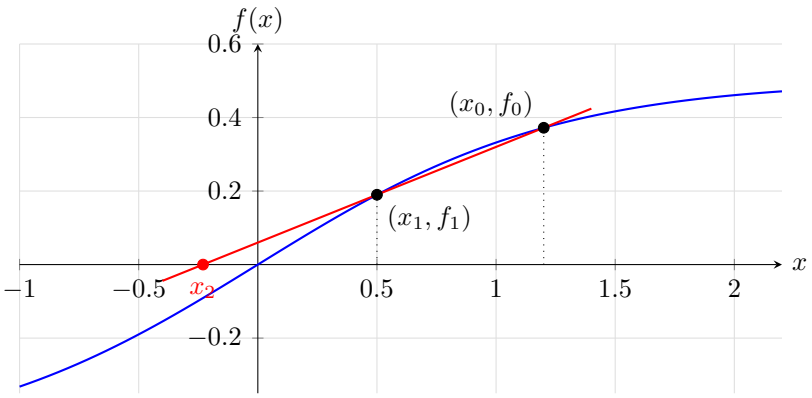


Figure 241: Original figure for slide 17: one secant step on $f(x) = 0.5\tanh(0.8x)$ with $x_0 = 1.2$, $x_1 = 0.5$ gives $x_2 \approx -0.23$ (true root 0).

9.3 Intersections of rays with SDFs: sphere tracing *(slides 19–20)*

Slide 19 is a video (Sebastian Lague, "Coding Adventure: Ray Marching") showing a 2D scene of circles and boxes and circles growing around the current ray point.

[IMAGE PLACEHOLDER]
Slide **slide 19**: Ray-marching video frame: dark background, circles/boxes, one white ray origin

Algorithm 9.2 (Sphere tracing)**.** Initialise t (e.g. 0) and $\boldsymbol{p} = \boldsymbol{x}_0 + t\boldsymbol{\omega}$. While $f(\boldsymbol{p}) > \epsilon$:

$$t \leftarrow t + f(\boldsymbol{p}), \qquad \boldsymbol{p} \leftarrow \boldsymbol{x}_0 + t\boldsymbol{\omega},$$

stopping also if a maximum number of steps is exceeded. Basically: *walk along the ray until close to a surface.*

Lemma 9.1 (Sphere tracing never crosses the surface)**.** *If f is an SDF, the open ball $B(\boldsymbol{p}, f(\boldsymbol{p}))$ contains no surface point.*

Proof. If $\boldsymbol{y} \in \mathcal{S}$ with $\|\boldsymbol{y} - \boldsymbol{p}\| < f(\boldsymbol{p}) = \min_{\boldsymbol{z}\in\mathcal{S}} \|\boldsymbol{p} - \boldsymbol{z}\|$, the minimum would be violated. □

Proposition 9.2 (Convergence to the first intersection)**.** *Let f be an SDF, the start point be outside, and t^* the first hit of the ray. The sphere-tracing iterates t_k are non-decreasing, bounded by t^*, and converge to t^* (as $\epsilon \to 0$, with unlimited steps).*

Proof. Because $\boldsymbol{p}^*$ is a surface point at distance $t^* - t_k$ from $\boldsymbol{p}_k$, $f(\boldsymbol{p}_k) \le t^* - t_k$ and so $t_{k+1} = t_k + f(\boldsymbol{p}_k) \le t^*$. The sequence is monotone and bounded, hence convergent to some $t_\infty \le t^*$; increments $f(\boldsymbol{p}_k) = t_{k+1} - t_k \to 0$, so by continuity $f(\boldsymbol{p}_\infty) = 0$, i.e. $\boldsymbol{p}_\infty \in \mathcal{S}$. Since $t_\infty \le t^*$ and t^* is the first hit, $t_\infty = t^*$. □

Intuition

The SDF tells you "nothing is closer than $f(\boldsymbol{p})$", so you can safely leap that far along *any* direction. Far from the surface the leaps are long, close to it they shrink geometrically, so you creep up on the surface from outside and never tunnel through. Slide question *Q: why will we never cross the surface?*

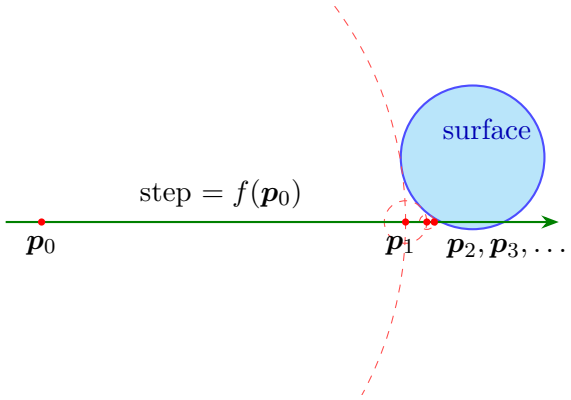


Figure 242: Original illustration of sphere tracing (slide 20) on a disc of radius 1 centred at $(6, 0.9)$: radii of the safe balls are 5.07, 0.30, 0.10, 0.05, ... and the iterates approach the first hit without crossing it.

— Lemma 9.1.

9.4 Options and differentiability *(slides 21–22)*

Two ways to get $\boldsymbol{p} \equiv t^*\boldsymbol{\omega} + \boldsymbol{x}_0$ s.t. $f_\theta(\boldsymbol{p}) = 0$: **Option 1:** a generic root-finding method (e.g. secant); **Option 2:** sphere tracing (only if rendering SDFs). Next question: *is this differentiable? Do we need to backpropagate through the iterative process?*

10 Differentiable Root Finding *(slides 23–26)*

10.1 A toy example *(slide 23)*

Let $f_\theta(t) = t^2 - \theta$ (plotted for $\theta = 4$); some root-finder returns $t^* = 2$. If θ changes, the new root is $t^*(\theta + \Delta\theta) \equiv t^* + \Delta t$. At the old root $f_{\theta+\Delta\theta}(t^*) = -\Delta\theta$, so *the root must change to compensate for the change in f*:

$$\Delta t \approx \frac{\Delta\theta}{2t^*} \qquad (\textit{why?})$$

Because $f_{\theta+\Delta\theta}$ near t^* is approximately $-\Delta\theta + 2t^*(t - t^*)$, which vanishes at $\Delta t = \Delta\theta/(2t^*)$.

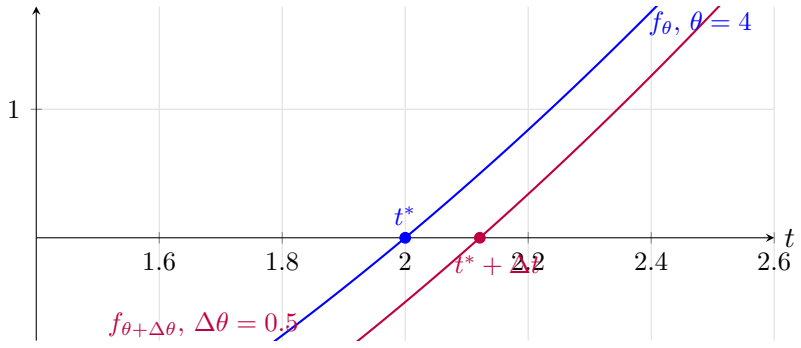


Figure 243: Original figure for slide 23: perturbing θ from 4 to 4.5 shifts the root from 2 to $\sqrt{4.5} \approx 2.121$; the linear prediction $\Delta\theta/(2t^*) = 0.125$ vs. the true shift 0.121.

10.2 The general rule *(slides 24–26)*

For a general f_θ, $f_{\theta+\Delta\theta}(t^*) \approx \frac{\partial f_\theta(t^*)}{\partial\theta}\Delta\theta$ and $\Delta t \approx -f_{\theta+\Delta\theta}(t^*)\left(\frac{\partial f_\theta(t^*)}{\partial t}\right)^{-1}$, i.e.,

$$\Delta t \approx -\left(\frac{\partial f_\theta(t^*)}{\partial t}\right)^{-1}\frac{\partial f_\theta(t^*)}{\partial\theta}\Delta\theta.$$

Theorem 10.1 (Implicit differentiation of a root)**.** *Let $F(\theta, t) = f_\theta(t)$ be differentiable and $t^*(\theta)$ a root, $F(\theta, t^*(\theta)) = 0$, with $\frac{\partial F}{\partial t}(\theta, t^*) \neq 0$. Then*

$$\frac{\partial t^*}{\partial \theta} = -\Big(\frac{\partial f_\theta(t^*)}{\partial t}\Big)^{-1} \frac{\partial f_\theta(t^*)}{\partial \theta}.$$

This holds irrespective of how t^* was found, and for any f.

Proof. By the implicit function theorem $t^*(\theta)$ is locally differentiable. Differentiate $F(\theta, t^*(\theta)) = 0$ with the chain rule: $\frac{\partial F}{\partial \theta} + \frac{\partial F}{\partial t}\frac{\partial t^*}{\partial \theta} = 0$ and solve. □

Intuition

If changing the parameters changes the value of f at the current root, the root must move to put the value back to zero. How far it moves is the "error" $\partial f/\partial\theta$ divided by how steeply f crosses zero, $\partial f/\partial t$. *Numbers:* for the toy example, $\partial f/\partial\theta = -1$ and $\partial f/\partial t = 2t^* = 4$, so $\partial t^*/\partial\theta = 1/4$, matching $\Delta t \approx \Delta\theta/(2t^*)$. Note a steep crossing (large $|\partial f/\partial t|$) makes the root insensitive to θ, while a grazing crossing ($\partial f/\partial t \to 0$) makes it explode — the first sign of the problems with silhouettes below.

11 Differentiable Ray Marching *(slides 27–29)*

Apply the theorem to $\bar{f}_\theta(t) \equiv f_\theta(\boldsymbol{x}_0 + t\boldsymbol{\omega})$ with intersection $\boldsymbol{p} \equiv t^*\boldsymbol{\omega} + \boldsymbol{x}_0$. (Camera origin $\boldsymbol{x}_0$ and direction $\boldsymbol{\omega}$ do not depend on θ.)

Corollary 11.1 (Analytic gradient of the intersection point)**.** *Differentiating $f_\theta(\boldsymbol{p}) = 0$ gives*

$$\frac{\partial f_\theta(\boldsymbol{p})}{\partial \theta} + \frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}} \cdot \frac{\partial \boldsymbol{p}}{\partial \theta} = 0, \qquad \frac{\partial \boldsymbol{p}}{\partial \theta} = \boldsymbol{\omega}\,\frac{\partial t^*}{\partial \theta},$$

hence

$$\frac{\partial t^*}{\partial \theta} = -\Big(\frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}} \cdot \boldsymbol{\omega}\Big)^{-1} \frac{\partial f_\theta(\boldsymbol{p})}{\partial \theta}, \qquad \frac{\partial \boldsymbol{p}}{\partial \theta} = -\boldsymbol{\omega}\Big(\frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}} \cdot \boldsymbol{\omega}\Big)^{-1} \frac{\partial f_\theta(\boldsymbol{p})}{\partial \theta}.$$

If we can find an intersection, we can analytically compute $\partial\boldsymbol{p}/\partial\theta$ — with no need to store (or backpropagate through) the iterations of the root finder.

Intuition

The forward pass can be as sloppy and iterative as you like (secant, sphere tracing, hundreds of steps); the backward pass only needs $\nabla_{\boldsymbol{p}} f$, $\partial f/\partial\theta$ and $\boldsymbol{\omega}$ *at the final point.* Memory does not grow with the number of marching steps. The dot product $\nabla_{\boldsymbol{p}} f \cdot \boldsymbol{\omega}$ is the cosine between ray and surface normal (times $\|\nabla f\|$): head-on hits are stable, grazing hits are not.

12 Rendering Implicit Surfaces (or SDFs) *(slides 30–32)*

Step 1: Compute the intersection (root finding or sphere tracing for SDFs; analytically differentiable).
Step 2: Render the surface point: $g_\phi(\boldsymbol{p}, \boldsymbol{\omega}) = \boldsymbol{c} \in \mathbb{R}^3$ (similar to NeRF), but slide 31 argues that *rendering should account for the surface normal in addition to the viewpoint*: two surface points with the same position and view direction but different normals reflect light differently. Hence (slide 32)

$$\boldsymbol{n} = \frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}}, \qquad g_\phi(\boldsymbol{p}, \boldsymbol{\omega}, \boldsymbol{n}) = \boldsymbol{c} \in \mathbb{R}^3.$$

(In practice $\boldsymbol{n}$ is normalised; for an SDF, $\|\nabla f\| = 1$ already.)

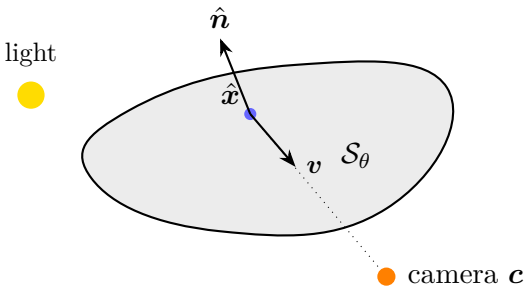


Figure 244: Original sketch of slide 31: the colour at $\hat{\boldsymbol{x}}$ depends on the normal $\hat{\boldsymbol{n}}$ and on the viewing direction $\boldsymbol{v}$ (the slide varies each in turn).

[IMAGE PLACEHOLDER]
Slide **slide 31**: Three copies of the shape $\mathcal{S}_\theta$ with changed normal / viewing direction (Yariv et al., IDR)

Intuition

The geometry of a shiny object changes colour with orientation, and the only thing that tells the colour network about orientation is $\boldsymbol{n}$. Feeding $\boldsymbol{n} = \nabla f$ also *couples* appearance and geometry: errors in colour send gradients into f_θ through $\boldsymbol{n}$ and through $\boldsymbol{p}$.

13 Learning Implicit Geometry and Appearance *(slide 33)*

Given images I_i with camera poses π_i, fit geometry f_θ and appearance g_ϕ:

$$\min_{\theta,\phi} \big\|\text{render}(f_\theta, g_\phi, \pi_i) - I_i\big\| \;+\; \lambda \,\big|\, \|\nabla f\| - 1 \,\big|, \tag{19}$$

a photometric term plus the eikonal regulariser (the slide writes it without a weight; λ is the usual trade-off coefficient).

Intuition

The first term says "rendered pixels must match photographs" and drives *both* networks through the analytic gradients of the previous section; the second keeps f_θ a well-behaved distance field, which is what makes sphere tracing (and the normals) trustworthy.

14 Rendering Neural Implicits *(slides 34–36)*

Slides 34–36 are results of IDR (Yariv et al., NeurIPS 2020): geometry/appearance pairs for statues, fruits, cans, a Smurf toy, …; a fountain reconstructed *using only 11 images*; and a training-run video with camera frusta around a skull while the SDF cross-section evolves.

[IMAGE PLACEHOLDER]
Slide **slide 34**: Grid of reconstructed shapes (grey) and their textured renderings (IDR)

[IMAGE PLACEHOLDER]
Slide **slide 35**: Fountain: geometry and rendering from only 11 images (IDR)

[IMAGE PLACEHOLDER]
Slide **slide 36**: IDR training run: camera frusta around a skull, SDF slice on a blue plane

15 Rendering Neural Implicits: Practical Considerations *(slides 37–38)*

Q1: sphere tracing with neural implicits?

There is *no guarantee* that a trained f_θ is an actual SDF. Lemma 9.1 then fails: if $|f_\theta|$ overestimates the distance the march can step through a surface; if it underestimates it is merely slow. **In practice, combine with the secant method to handle "bad" rays.** And crucially: analytic differentiation (Corollary 11.1) *still holds no matter how we compute the intersection.*

Algorithm 15.1 (A typical hybrid intersection routine)**.** (Typical implementation, not spelled out on the slides.)

1. Sphere trace with step $f_\theta(\boldsymbol{p})$ (optionally clamped) up to a maximum number of steps.
2. If f_θ changes sign between two iterates t_{k-1}, t_k (the ray stepped through the surface), keep that bracket.
3. Refine the root inside the bracket with secant (or bisection) iterations.
4. Rays with no sign change are declared *non-intersecting.*

Q2: rays that do not intersect

There is no rendering loss on those pixels, but it is **crucial to use a mask loss**. If a pixel is foreground but the ray did not intersect:

$$L(\boldsymbol{r}) \equiv \min_t f_\theta(\boldsymbol{x}_0 + t\boldsymbol{\omega}) \qquad \text{(SDF of the closest surface point should be 0)},$$

implemented in practice by uniformly sampling t.

Intuition

Without a mask, the optimiser can "cheat": move the surface so that rays miss it and hence incur no photometric loss. The mask loss says: the ray that *should* touch the object must at least come as close as possible to the surface; the gradient flows through the best sample t and pulls the surface towards the ray. (By symmetry, a background pixel whose ray *does* hit the surface must be penalised too — standard in IDR.)

16 Visibility Discontinuities *(slide 39)*

Correctly differentiating the surface rendering equation requires accounting for *visibility discontinuities*; this is the main focus of differentiable-rendering research in graphics. The slide shows the silhouette/horizon/terminator geometry of a surface seen from a camera (Gargallo et al., ICCV 2007) and a light-transport example where a *silhouette edge* lies on a ray path.

[IMAGE PLACEHOLDER]
Slide **slide 39**: Left: banana with horizon, terminator and crepuscular cone; right: sphere and Cornell-box silhouette edge

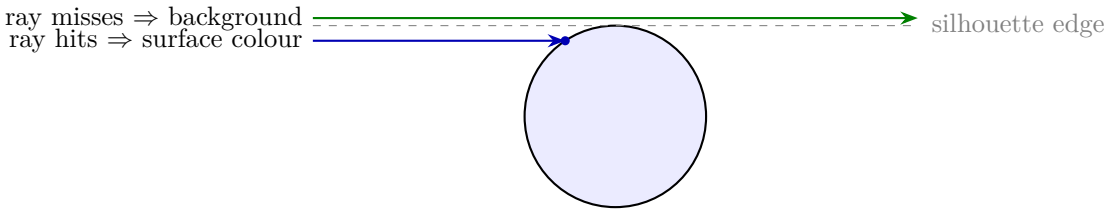


Figure 245: Original toy figure: an arbitrarily small change of θ moves a ray from just inside to just outside the silhouette, so the pixel value jumps. Implicit differentiation (Corollary 11.1) captures only the smooth part of the derivative, not this jump.

Intuition

The pixel colour is a piecewise-smooth function of the shape: smooth while the ray keeps hitting the same part of the surface, but with a *jump* when the ray crosses an object boundary. Differentiating "inside" each piece is easy (everything above); the gradient due to the boundary moving across the pixel is the hard part, and is why accurate differentiable rendering is non-trivial.

17 Rendering Implicit Surfaces *(slide 40)*

Implicit surfaces: a nice representation for geometry. **Implicit surface rendering:** render a *single* point per ray.

\+ closer to physical reality
− makes optimisation difficult (e.g. mask loss is critical)
\+ can allow physics-based rendering using surface normals (later lecture)
− accurate differentiable rendering is non-trivial due to visibility discontinuities

18 Volume Rendering for Implicit Surfaces *(slides 41–43, 49)*

Idea (slide 41): convert the implicit into a "density" and use volume rendering!

Recall (slide 42): a rendering algorithm maps $(\sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i}), L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})) \to L(\boldsymbol{x}, \boldsymbol{\omega})$ and a neural density/radiance representation $(\boldsymbol{x}, \boldsymbol{\omega}) \to f_\theta \to (\sigma, \boldsymbol{c})$, $\sigma \in \mathbb{R}^+$, $\boldsymbol{c} \in [0,1]^3$. For implicit surfaces (slides 43, 49) the network outputs a signed distance $d \in \mathbb{R}$ and we *analytically convert* it to density with a map Ψ, using volume rendering while enforcing the SDF prior $\big|\|\nabla f\| - 1\big|$.

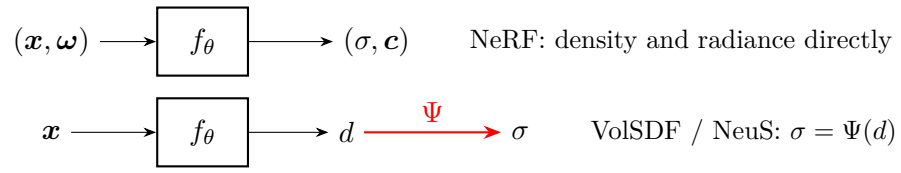


Figure 246: Redrawn from slides 42–43/49: the density is produced by a fixed analytic map Ψ of the signed distance.

Intuition

The renderer (Eq. (18)) is untouched. We only change *what the network predicts*: instead of free density we predict a distance and let a formula decide the density. Geometry is then a surface by construction, and optimisation keeps the soft, forgiving gradients of volume rendering (no mask loss, no single-point visibility problem).

19 From SDF to Density *(slides 44–48)*

19.1 A first choice: VolSDF *(slide 44)*

Definition 19.1 (VolSDF density)**.** With d_Ω the signed distance (negative inside), $\alpha, \beta > 0$:

$$\sigma(\boldsymbol{x}) = \alpha\, \Psi_\beta\big(-d_\Omega(\boldsymbol{x})\big), \qquad \Psi_\beta(s) = \begin{cases} \frac{1}{2}\exp\big(\frac{s}{\beta}\big) & s \le 0, \\ 1 - \frac{1}{2}\exp\big(-\frac{s}{\beta}\big) & s > 0. \end{cases}$$

Outside $= 0$, inside $= \alpha$, with sharpness of the drop controlled by $1/\beta$. *Just a heuristic — one among many!*

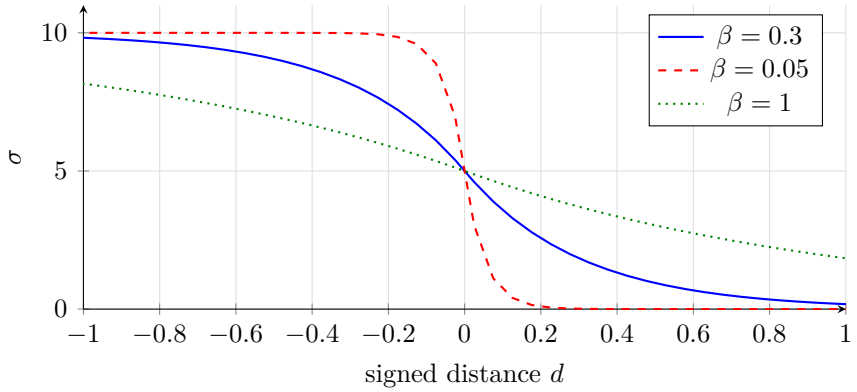


Figure 247: Original plot of the VolSDF density for $\alpha = 10$: small β approaches a sharp step at the surface, large β gives a fuzzy shell.

Proposition 19.2 (Ψ_β is a Laplace CDF)**.** *Ψ_β is the cumulative distribution function of a zero-mean Laplace variable Z with scale β, so $\sigma(\boldsymbol{x}) = \alpha \Pr[Z \ge d_\Omega(\boldsymbol{x})]$. (Proof: see Appendix A.2; supplementary.)*

Intuition

Imagine the surface is randomly "jittered" outward or inward by a Laplace-distributed amount Z; the density at $\boldsymbol{x}$ is α times the probability that $\boldsymbol{x}$ ends up inside. Points deep inside are always inside ($\sigma = \alpha$), points far outside never ($\sigma = 0$), and at the surface it is a coin flip ($\sigma = \alpha/2$).

19.2 A second heuristic: NeuS *(slides 45–46)*

Take the logistic density and sigmoid

$$\phi_s(x) = \frac{s\, e^{-sx}}{(1+e^{-sx})^2}, \qquad \Phi_s(x) = (1+e^{-sx})^{-1}, \qquad \Phi_s' = \phi_s.$$

The *naive* solution (slide 45) sets $\sigma(t) = \phi_s\big(f(\boldsymbol{p}(t))\big)$: high density around the surface and 0 otherwise, with sharpness controlled by s. *Yet another heuristic*; "the more principled one is more complicated and involves gradients through f" — slide 46 shows the problem: *high weight before the surface.*

Proposition 19.3 (Naive conversion is biased)**.** *Let $f(\boldsymbol{p}(t))$ be locally linear with a single zero at t^*, $\sigma = \phi_s(f(\boldsymbol{p}(t)))$, $T(t) = \exp(-\int_0^t \sigma)$ and $w = T\sigma$. Then $w'(t^*) < 0$: the weight peaks strictly* before *the surface.*

Proof. $T' = -\sigma T$, so $w' = T'\sigma + T\sigma' = T(\sigma' - \sigma^2)$. At t^* the density $\phi_s(f)$ is maximal ($f = 0$), so $\sigma'(t^*) = 0$ and $w'(t^*) = -T\sigma^2 < 0$. □

Definition 19.4 (NeuS opaque density)**.** Let Φ_s be as above. NeuS defines

$$\rho(t) = \max\Big(\frac{-\frac{\mathrm{d}\Phi_s}{\mathrm{d}t}\big(f(\boldsymbol{p}(t))\big)}{\Phi_s\big(f(\boldsymbol{p}(t))\big)},\, 0\Big)$$

and uses ρ in place of σ in volume rendering.

Proposition 19.5 (Unbiased weights)**.** *Assume the ray crosses a locally planar surface from outside ($f > 0$) to inside, with $\nabla f \cdot \boldsymbol{\omega} = -\kappa < 0$ constant, and start far enough outside that $\Phi_s(f(0)) \approx 1$. Then $T(t) = \Phi_s(f(\boldsymbol{p}(t)))$ and the rendering weight is*

$$w(t) = T(t)\rho(t) = \kappa\, \phi_s\big(f(\boldsymbol{p}(t))\big),$$

which is maximal exactly at the surface $f = 0$ ($t = t^*$).

Proof. Here $\frac{\mathrm{d}}{\mathrm{d}t}\Phi_s(f) = \phi_s(f)(\nabla f \cdot \boldsymbol{\omega}) = -\kappa\phi_s(f) < 0$, so $\rho = -\Phi_s(f)'/\Phi_s(f) > 0$ and $\int_0^t \rho = -\ln\Phi_s(f(\boldsymbol{p}(t))) + \ln\Phi_s(f(\boldsymbol{p}(0)))$. Hence $T = \Phi_s(f(t))/\Phi_s(f(0)) \approx \Phi_s(f(t))$ and $w = T\rho = -\frac{\mathrm{d}}{\mathrm{d}t}\Phi_s(f) = \kappa\phi_s(f)$, which peaks where ϕ_s does, at $f = 0$. □

Intuition

Naive: a bump of density *centred* on the surface, but light is already being absorbed on the way up the bump, so most of the pixel colour is taken from a slightly-in-front point: a systematic bias. NeuS instead designs ρ so that the *product* $T\rho$ is the bump we want, centred exactly at the zero crossing. Note the shape of ρ: it keeps increasing and then saturates inside ($\rho \to s\kappa$), i.e. the interior is opaque, whereas the naive σ returns to zero after the surface. *Number:* for $s = 10$ and a ray crossing at unit speed, the NeuS peak is $\phi_s(0) = s/4 = 2.5$ at t^*.

19.3 Comparison of the densities *(slides 47–48)*

Slide 47 shows a 2D SDF of a disc and a rectangle with a ray through both, and the density produced by NeRF (noisy blobs), VolSDF (smooth filled shapes), naive NeuS (dark *shells* on the outer boundary) and NeuS (density only on the first surface crossed). Slide 48 shows, for a ray, the density $\sigma(t)$, the weight $w(t) = T\sigma$ and a zoom of the weight at the first surface for each method.

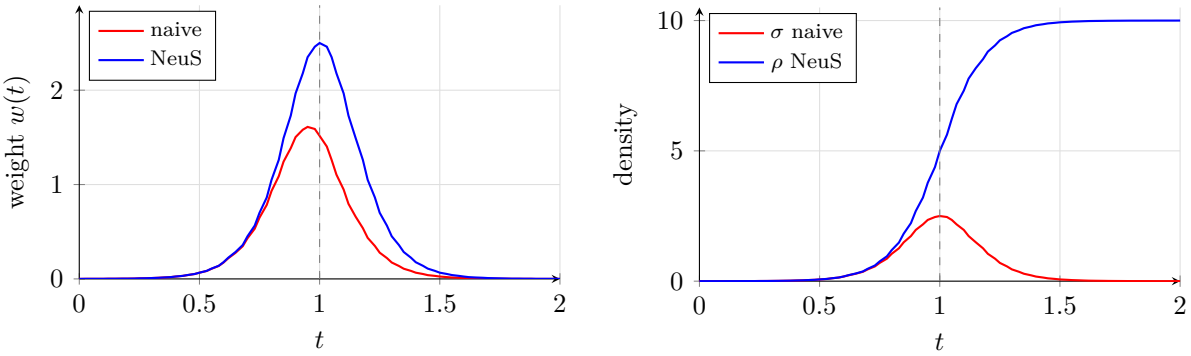


Figure 248: Original numerical illustration of slide 46 (ray crossing a plane at $t^* = 1$, $s = 10$, $f(t) = 1 - t$). Left: the naive weight peaks before the surface, the NeuS weight peaks exactly at it. Right: the densities.

[IMAGE PLACEHOLDER]
Slide **slide 47**: SDF of disc+rectangle (left); density maps of NeRF, VolSDF, naive NeuS, NeuS (right)

[IMAGE PLACEHOLDER]
Slide **slide 48**: Plots of density, weight and zoomed weight along the ray for the four methods

20 Volume Rendering for Implicit Surfaces: results *(slides 50–54)*

All results are from NeuS (Wang et al., NeurIPS 2021): slide 50 compares a reference wooden box image with IDR, NeRF and "ours" geometry (NeuS is smooth and complete where IDR is wavy and NeRF noisy); slide 51 shows scissors and tools on a stone slab; slide 52 the half-sphere scene where IDR geometry is blobby; slide 53 a basket scene *without mask supervision*; slide 54 a gallery of reconstructions (Stonehenge blocks, fruit, owl, cans, plush toy, birds, cherub statue, skull).

[IMAGE PLACEHOLDER]
Slide **slide 50**: Box: reference image, IDR, NeRF, NeuS

[IMAGE PLACEHOLDER]
Slide **slide 51**: IDR / NeRF / NeuS geometry and NeuS rendering of scissors

[IMAGE PLACEHOLDER]
Slide **slide 52**: IDR / NeRF / NeuS geometry and rendering of the half-sphere scene

[IMAGE PLACEHOLDER]
Slide **slide 53**: Input subset, NeuS geometry and rendering of a basket, without mask supervision

[IMAGE PLACEHOLDER]
Slide **slide 54**: Gallery of eight NeuS reconstructions (geometry + rendering)

Intuition

The comparison tells the whole story of the lecture: NeRF gives good images but bumpy geometry (density is unconstrained), IDR gives surfaces but struggles in optimisation (single-point rendering and masks), and NeuS combines volumetric rendering with an SDF-parametrised density to get both, even without masks.

21 Summary *(slides 55–56)*

Slide 55 recaps: *computing intersections of rays with implicit surfaces* with the analytic differentiation $\partial \boldsymbol{p}/\partial\theta = -\boldsymbol{\omega}(\nabla_{\boldsymbol{p}} f \cdot \boldsymbol{\omega})^{-1}\partial f/\partial\theta$, and *volume rendering with surface representations* via $\boldsymbol{x} \to f_\theta \to d \xrightarrow{\Psi} \sigma$. Slide 56 acknowledges the sources (adapted from CMU 16-825, Learning for 3D Vision, Prof. Shubham Tulsiani; secant slides by A. Geiger; sphere-tracing video by S. Lague; ray-marching figures by I. Gkioulekas; results from NGLOD, NeRF, Instant-NGP, NeRS, IDR, VolSDF, NeuS).

Comparison of the methods

	Implicit surface rendering (IDR)	**NeRF-like volume**	**VolSDF / NeuS**
Geometry	zero set of f_θ (SDF-like)	density σ + colour	SDF d mapped to density by Ψ / ρ
Points rendered per ray	one (the intersection)	many samples	many samples
Intersection / sampling	root finding, sphere tracing	stratified samples	stratified samples
Gradient	analytic via implicit differentiation; visibility discontinuities missed	standard backprop	standard backprop
Mask loss	critical	not needed	not needed (NeuS shown without masks)
Geometry quality	well-defined surface	does not necessarily capture geometry well	smooth, well-defined surface
Main drawback	hard optimisation, non-trivial differentiable rendering	noisy surface, overfitting risk	hand-designed conversion (heuristic; naive version biased)

Where the rendering weight goes

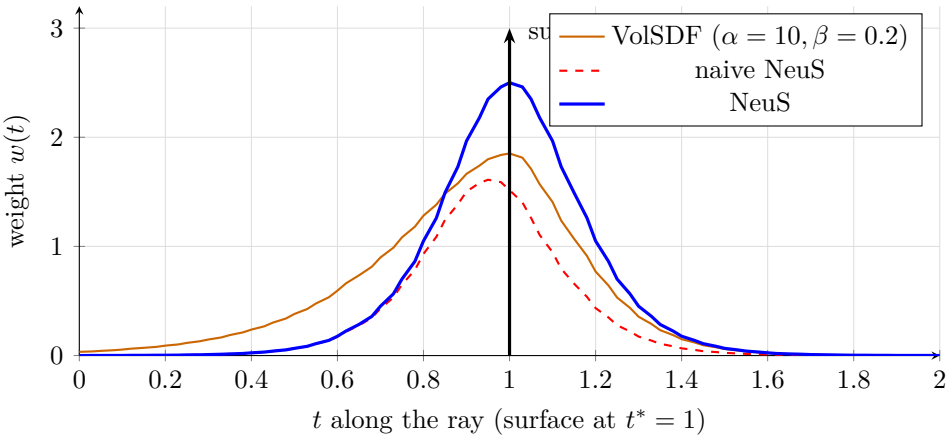


Figure 249: Original qualitative comparison for a ray crossing a single surface: surface rendering puts all weight on one point; volumetric methods spread it. NeuS is centred on the surface, naive NeuS is shifted in front of it, VolSDF depends on α, β and absorbs light slightly before the surface. (NeuS peak is $s/4 = 2.5$ for $s = 10$; curves are computed numerically.)

Key take-aways

- Volumes are expressive but under-constrained: good images do not imply good geometry. Restricting to *surfaces* (implicit zero sets, ideally SDFs with $\|\nabla f\| = 1$) injects the right prior.
- Ray–surface intersection is a 1-d root-finding problem; sphere tracing is safe for true SDFs, secant is a general fallback, and in practice they are combined.

- The intersection point is differentiable *analytically*: $\partial \boldsymbol{p}/\partial\theta = -\boldsymbol{\omega}(\nabla_{\boldsymbol{p}} f \cdot \boldsymbol{\omega})^{-1}\partial_\theta f$, independent of the solver and free of per-step memory, but it ignores visibility discontinuities and needs a mask loss.
- Volume rendering for implicit surfaces turns d into a density by a fixed map Ψ (VolSDF: Laplace CDF; NeuS: logistic, with a density that makes the weight $T\rho$ peak at the surface). Both are heuristics, and the naive logistic density is biased.
- Surface-based rendering allows physically based shading from normals (next lectures), while volumetric SDF methods are easier to optimise.

A Appendix: supplementary material (not on the slides)

The following derivations were added to make the notes self-contained; they do not correspond to specific slides.

A.1 From the continuous volume rendering integral to Eq. (18)

For a ray $\boldsymbol{x}(t)$, with density $\sigma(t)$ and emitted radiance $L_e(t)$, the continuous model is $L = \int_0^\infty T(t)\sigma(t)L_e(t)\,\mathrm{d}t$ with $T(t) = \exp(-\int_0^t \sigma(u)\,\mathrm{d}u)$. Assume $\sigma \equiv \sigma_i$ and $L_e \equiv L_{e,i}$ constant on segment $[t_i, t_i + \Delta t]$. Then $T(t) = T_i e^{-\sigma_i(t-t_i)}$ on it and

$$\int_{t_i}^{t_i+\Delta t} T(t)\sigma_i L_{e,i}\,\mathrm{d}t = T_i L_{e,i}\big(1 - e^{-\sigma_i \Delta t}\big),$$

which, summed over segments, is Eq. (18).

A.2 Ψ_β is a Laplace CDF

The Laplace density with scale β is $p(u) = \frac{1}{2\beta}e^{-|u|/\beta}$. For $s \le 0$, $\int_{-\infty}^{s} p = \frac{1}{2}e^{s/\beta}$; for $s > 0$, $\int_{-\infty}^{s} p = \frac{1}{2} + \int_0^s \frac{1}{2\beta}e^{-u/\beta}\mathrm{d}u = 1 - \frac{1}{2}e^{-s/\beta}$. These are exactly the two branches of Ψ_β, hence $\Psi_\beta(s) = \Pr[Z \le s]$, $\Psi_\beta(0) = \frac{1}{2}$, $\Psi'_\beta(s) = p(s)$, and by symmetry $\Psi_\beta(-d) = \Pr[Z \le -d] = \Pr[Z \ge d]$. As $\beta \to 0$, $\Psi_\beta(-d) \to \mathbf{1}[d < 0]$, i.e. $\sigma \to \alpha\,\mathbf{1}[\text{inside}]$.

Part IX

Lecture 10

How to read these notes. Section and subsection titles follow the slide titles, and each carries its slide range. No companion book chapter was supplied, so the formal material (definitions, lemmas, proofs, derivations) was added from the standard sources the slides cite: Ravi et al. (PyTorch3D), Liu et al. (Soft Rasterizer), Zwicker et al. (EWA splatting), Kerbl et al. (3D Gaussian Splatting) and Huang et al. (2D Gaussian Splatting). Statements that go beyond the slides are labelled as such. Dashed boxes mark photos or screenshots to be copied from the indicated slide.

Notation. The slides reuse a few symbols in different sections; the context fixes the meaning.

Symbol	Meaning
$\boldsymbol{x}_0,\ \boldsymbol{\omega},\ \lambda$	ray origin, unit direction, distance along the ray
$\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3$	triangle vertices; (α, β) barycentric coordinates (mesh sections)
$\boldsymbol{c},\ \boldsymbol{c}_k$	RGB colour of a pixel / of primitive k
$m,\ \sigma,\ \gamma$	soft mask, mask sharpness, depth-weighting temperature (soft meshes)
$\boldsymbol{\mu},\ \boldsymbol{p},\ \boldsymbol{\Sigma}$	mean (position) and covariance of a Gaussian
$\boldsymbol{R},\ \boldsymbol{S}$	rotation and diagonal scale matrix, $\boldsymbol{\Sigma} = \boldsymbol{R}\boldsymbol{S}\boldsymbol{S}^\top\boldsymbol{R}^\top$
$\alpha,\ \bar{\alpha}_k$	opacity of a Gaussian / its opacity at a given pixel (Gaussian sections)
$T_k,\ w_k$	transmittance in front of primitive k, blending weight
$\boldsymbol{W},\ \boldsymbol{t},\ \boldsymbol{K}$	world→camera rotation, translation, camera intrinsics
$\pi,\ \boldsymbol{J}$	projection function, its Jacobian at the Gaussian mean

1 Differentiable Primitive Rendering *(slides 1–7)*

The question of this lecture is how to render a scene made of *primitives* (triangles, Gaussians, voxels, ...) so that the image is a differentiable function of the primitive parameters. With such a renderer we can recover the scene by gradient descent on an image loss.

1.1 Recap: Rendering Volumes *(slide 3)*

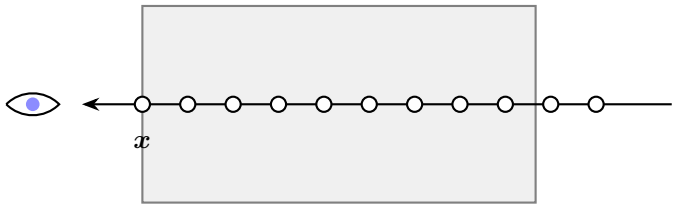
Volume rendering has two steps: (1) draw samples along the ray, (2) aggregate their contributions.

Definition 1.1 (Discrete volume rendering)**.** Let a ray $\boldsymbol{r}(t) = \boldsymbol{x}_0 + t\boldsymbol{\omega}$ be sampled at $t_1 < \cdots < t_N$ with spacings δ_i, densities σ_i and colours $\boldsymbol{c}_i$. Then

$$\boldsymbol{c} = \sum_{i=1}^{N} T_i\, \alpha_i\, \boldsymbol{c}_i, \qquad \alpha_i = 1 - e^{-\sigma_i \delta_i}, \qquad T_i = \prod_{j<i} (1 - \alpha_j).$$

Intuition

T_i is the fraction of light that survives up to sample i, and α_i is the share that sample i absorbs and returns as its own colour. Everything in this lecture is a variation of this "front-to-back" bookkeeping. The only thing that changes is *where* the contributions come from.



1. draw samples along the ray 2. aggregate their contributions

Figure 250: Volume rendering: samples are drawn along the ray through a volume and then aggregated.

1.2 Rendering Primitives *(slides 4–6)*

Gaussians (slide 4). The same two steps work for primitives such as Gaussians: sample along the ray, aggregate whatever the samples hit.

Triangles/meshes (slide 5). Here step 1 is *wasteful*: we know exactly where the primitives are, so most samples fall in empty space.

The primitive view (slide 6). Replace sampling by a direct query:

1. **Find the primitives that affect the ray/pixel;**
2. **aggregate their contributions** to render the pixel.

Definition 1.2 (Primitive-based rendering)**.** A scene is a set of primitives $\{\mathsf{p}_k\}$, each with a geometry and an appearance. For a ray $\boldsymbol{r}$ let $\mathcal{P}(\boldsymbol{r}) = \{k : \mathsf{p}_k \text{ affects } \boldsymbol{r}\}$ be the *influence set*. The pixel value is $\boldsymbol{c}(\boldsymbol{r}) = \mathrm{Agg}\big(\{(\lambda_k, \boldsymbol{c}_k, \dots)\}_{k \in \mathcal{P}(\boldsymbol{r})}\big)$, where λ_k is the depth of primitive k along the ray.

Intuition

The two design questions of the lecture are: how to compute $\mathcal{P}(\boldsymbol{r})$ *efficiently* (ray casting, then rasterization), and how to choose Agg so that the result is *differentiable*.

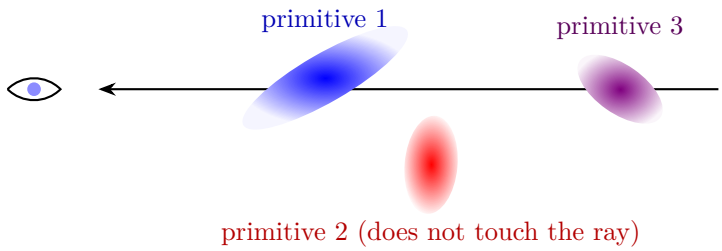


Figure 251: Only primitives 1 and 3 are in the influence set $\mathcal{P}(\boldsymbol{r})$ of this ray; primitive 2 is not.

1.3 Today's Goals *(slide 7)*

1. (Differentiable) mesh rendering via ray tracing.
2. From ray tracing to rasterization.
3. 3D Gaussian Splatting in detail: projection, tile rasterizer, optimization.

2 Mesh Rendering via Ray-casting *(slides 8–13)*

2.1 Mesh Rendering via Ray-casting *(slide 8)*

For each pixel we cast a ray and (1) identify the *closest* intersecting triangle, then (2) compute the appearance at the intersection point. Note that a *mesh* is equivalent to a *triangle set* for this algorithm: connectivity plays no role.

[**insert image here**]
taken from **slide 8**

Figure 252: Ray cast from a pixel of the image plane through the cow mesh, hitting the closest triangle (red dot on the image plane, red arrow reaching the surface).

2.2 Mesh Rendering: Ray-Triangle Intersection *(slide 9)*

Definition 2.1 (Barycentric coordinates)**.** Given a triangle $(\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3)$, every point of its plane is written $\boldsymbol{p} = \alpha\boldsymbol{v}_1 + \beta\boldsymbol{v}_2 + (1-\alpha-\beta)\boldsymbol{v}_3$. The pair (α, β) are the barycentric coordinates of $\boldsymbol{p}$. The point lies inside the triangle iff $\alpha, \beta, 1-\alpha-\beta \in (0,1)$.

For a ray $(\boldsymbol{x}_0, \boldsymbol{\omega})$ we need the point that is both on the ray and in the triangle plane:

$$\boldsymbol{p} = \boldsymbol{x}_0 + \lambda\boldsymbol{\omega} = \alpha\boldsymbol{v}_1 + \beta\boldsymbol{v}_2 + (1-\alpha-\beta)\boldsymbol{v}_3.$$

This is a linear system in (α, β, λ).

Proposition 2.2 (Closed-form ray–triangle intersection)**.** *Let* $\boldsymbol{e}_1 = \boldsymbol{v}_1 - \boldsymbol{v}_3$, $\boldsymbol{e}_2 = \boldsymbol{v}_2 - \boldsymbol{v}_3$, $\boldsymbol{s} = \boldsymbol{x}_0 - \boldsymbol{v}_3$, *and* $\boldsymbol{h} = \boldsymbol{\omega} \times \boldsymbol{e}_2$, $a = \boldsymbol{e}_1 \cdot \boldsymbol{h}$, $\boldsymbol{q} = \boldsymbol{s} \times \boldsymbol{e}_1$. *If* $a \neq 0$ *(the ray is not parallel to the triangle),*

$$\alpha = \frac{\boldsymbol{s}\cdot\boldsymbol{h}}{a}, \qquad \beta = \frac{\boldsymbol{\omega}\cdot\boldsymbol{q}}{a}, \qquad \lambda = \frac{\boldsymbol{e}_2\cdot\boldsymbol{q}}{a}.$$

Proof. Subtracting $\boldsymbol{v}_3$ from both sides gives $\alpha\boldsymbol{e}_1 + \beta\boldsymbol{e}_2 - \lambda\boldsymbol{\omega} = \boldsymbol{s}$, i.e. $[\boldsymbol{e}_1\ \boldsymbol{e}_2\ -\boldsymbol{\omega}]\,(\alpha,\beta,\lambda)^{\mathsf{T}} = \boldsymbol{s}$. By Cramer's rule with $D = \det[\boldsymbol{e}_1\ \boldsymbol{e}_2\ -\boldsymbol{\omega}] = -\boldsymbol{\omega}\cdot(\boldsymbol{e}_1 \times \boldsymbol{e}_2)$, and the cyclic invariance of the scalar triple product, one finds $D = -a$ and $\alpha = \det[\boldsymbol{s}\ \boldsymbol{e}_2\ -\boldsymbol{\omega}]/D = \boldsymbol{s}\cdot\boldsymbol{h}/a$, $\beta = \det[\boldsymbol{e}_1\ \boldsymbol{s}\ -\boldsymbol{\omega}]/D = \boldsymbol{\omega}\cdot\boldsymbol{q}/a$, $\lambda = \det[\boldsymbol{e}_1\ \boldsymbol{e}_2\ \boldsymbol{s}]/D = \boldsymbol{e}_2\cdot\boldsymbol{q}/a$. □

Intuition

Think of two questions answered at once: "where does the ray pierce the plane of the triangle?" (that is $\lambda = -\boldsymbol{s}\cdot\boldsymbol{n}/\boldsymbol{\omega}\cdot\boldsymbol{n}$ with $\boldsymbol{n} = \boldsymbol{e}_1 \times \boldsymbol{e}_2$) and "where is that point expressed in the triangle's own (α,β) frame?". Every ingredient is a smooth function of the vertices, which is why (α,β,λ) will turn out to be differentiable w.r.t. vertex positions.

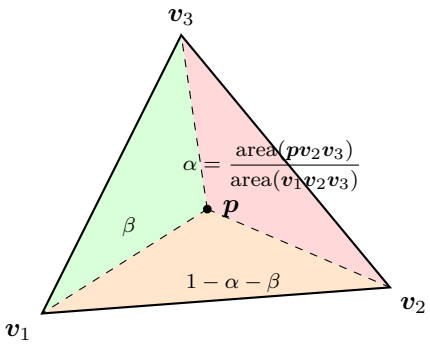


Figure 253: Barycentric coordinates as area ratios: the weight of a vertex is the relative area of the sub-triangle *opposite* to it (β is the area of $\boldsymbol{p}\boldsymbol{v}_1\boldsymbol{v}_3$, $1-\alpha-\beta$ the area of $\boldsymbol{p}\boldsymbol{v}_1\boldsymbol{v}_2$, both over the total).

2.3 Mesh Rendering: Ray-Mesh Intersection *(slide 10)*

Algorithm 2.1 (Closest intersecting triangle). `def mesh_ray_intersection(mesh, ray):`

```
    lam_min = inf
    intersection = None
    for triangle in mesh:
        (alpha, beta, lam) = intersect(ray, triangle)
        # inside test: 0 < alpha, beta, (1-alpha-beta) < 1
        if inside(alpha, beta) and 0 < lam < lam_min:
            lam_min = lam
            intersection = (triangle, (alpha, beta))
    return intersection
```

Remark 2.3. The slide's code is explicitly "rough pseudo-code". It does not update `lam_min`, which a working version must do, otherwise the *closest* hit is not guaranteed. The cost is one intersection test per triangle *per ray*.

2.4 Mesh Rendering *(slide 11)*

The two steps are two maps:

$$(1)\ \ \text{mesh}, \text{ray} \ \to\ \text{triangle}, (\alpha,\beta) \qquad (2)\ \ \text{triangle}, (\alpha,\beta) \ \to\ \boldsymbol{c} \in \mathbb{R}^3.$$

2.5 Mesh Rendering: Appearance *(slide 12)*

Definition 2.4 (Per-vertex colour and texture atlas)**. (a) Per-vertex colour:** each vertex carries a colour, $(\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3) \to (\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3)$, and $\boldsymbol{c} = \alpha\boldsymbol{c}_1 + \beta\boldsymbol{c}_2 + (1-\alpha-\beta)\boldsymbol{c}_3$. **(b) Texture atlas:** each triangle owns a $C \times C$ texture image and $\boldsymbol{c} = \text{bilinear_sample}(\text{atlas}, [\alpha, \beta])$.

Appearance can be extended to lighting-dependent shading, using the albedo as stored texture together with the surface normals.

Example 2.5. Let $\boldsymbol{c}_1 = (1,0,0)$, $\boldsymbol{c}_2 = (0,1,0)$, $\boldsymbol{c}_3 = (0,0,1)$ and $(\alpha, \beta) = (0.2, 0.5)$. Then $\boldsymbol{c} = 0.2\,\boldsymbol{c}_1 + 0.5\,\boldsymbol{c}_2 + 0.3\,\boldsymbol{c}_3 = (0.2, 0.5, 0.3)$.

Intuition

Per-vertex colour is a smooth "gradient" blended over the triangle (cheap, low resolution). A texture atlas spends memory to store detail inside each face. In both cases the colour is a *linear/bilinear* function of (α, β), hence trivially differentiable w.r.t. the barycentric coordinates.

2.6 Mesh Rendering (differentiable?) *(slide 13)*

Step 1 gives (α, β) that are differentiable w.r.t. the mesh vertex positions (Proposition 2.2); step 2 gives a colour differentiable w.r.t. the barycentric coordinates. The slide asks "Is this differentiable?" and answers: *well, sort of...* The next section shows exactly where it fails.

3 Mesh Rendering: Differentiability *(slide 14)*

The rendered colour *is* differentiable w.r.t. the vertex positions of the visible triangle. For example, we can make the colour more red by changing the position of the triangle (it changes which part of the colour gradient lands on the pixel). Two things are not differentiable:

1. rendering is discontinuous w.r.t. **occlusion/dis-occlusion**;
2. rendering is discontinuous w.r.t. **depth ordering**.

Proposition 3.1 (Hard rendering is piecewise smooth)**.** *Let $k^\star(\theta) = \arg\min_{k\,:\,\text{ray hits } k} \lambda_k(\theta)$ be the visible triangle for scene parameters θ. The pixel value is $\boldsymbol{c}(\theta) = \boldsymbol{c}_{k^\star(\theta)}(\theta)$. On every region of parameter space where $k^\star$ is constant, $\boldsymbol{c}$ is smooth. Across a boundary where the visible triangle changes, $\boldsymbol{c}$ jumps. Consequently the gradient is either undefined at the jump or carries no information about it.*

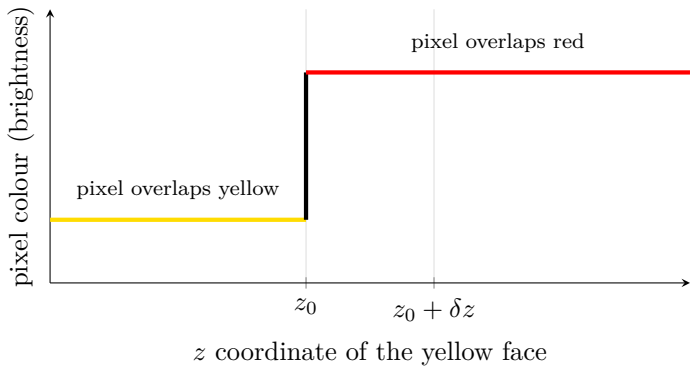


Figure 254: Depth-ordering discontinuity (redrawn from the diagram on slide 14): as the yellow face is shifted behind the red one, the pixel colour jumps from yellow to red at a single z. The derivative is 0 almost everywhere and undefined at the jump.

[**insert image here**]
taken from **slide 14**

Figure 255: Original 3D sketch: shifting the yellow triangle by $+\delta z$ changes which face the pixel overlaps (left), and the occlusion example with an orange and a purple triangle (right).

Intuition

Imagine you are blindfolded and can only feel the slope of the image at your feet. If the pixel is currently yellow, the slope of "colour vs. z" is flat: nothing you can feel says "pushing the yellow face back would make the pixel red". Likewise an *occluded* triangle has *exactly zero* influence on the image, so it never receives a gradient telling it to move into view.

4 Differentiable Mesh Rendering *(slides 15–20)*

The fix (Ravi et al., PyTorch3D, 2020; Liu et al., Soft Rasterizer, ICCV 2019) has two parts: (1) make triangles *soft*, (2) aggregate from *multiple* triangles.

4.1 Make triangles 'soft' *(slides 15–16)*

Definition 4.1 (Soft triangle mask)**.** Let Δ be the signed distance, in pixel space, from the pixel to the triangle ($\Delta < 0$ inside, > 0 outside). The soft mask is

$$m = \text{sigmoid}\Big(\frac{-\Delta}{\sigma}\Big) = \frac{1}{1 + e^{\Delta/\sigma}},$$

with a manually defined cut-off (e.g. 10σ) beyond which the mask is set to zero.



soft triangle
(fuzzy border of width r)

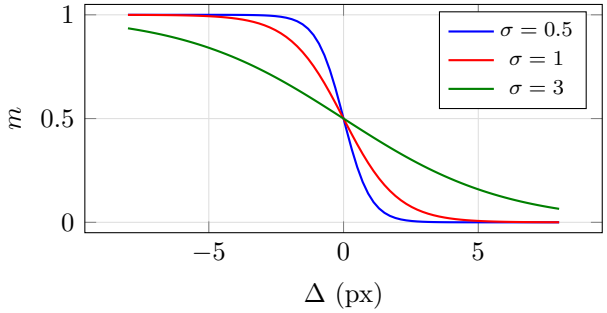


Figure 256: Left: a soft triangle has a fuzzy border of width r (slide 15–16). Right: the mask $m = \text{sigmoid}(-\Delta/\sigma)$ for several σ; $m = 0.5$ exactly on the boundary.

Proposition 4.2 (Properties of the soft mask)**.** *(i) $m \to 1$ deep inside and $m \to 0$ far outside; $m = \frac{1}{2}$ on the boundary. (ii) $\partial m/\partial\Delta = -m(1-m)/\sigma$ is non-zero for every pixel near the triangle. (iii) As $\sigma \to 0$, m converges to the hard indicator of the triangle.*

Proof. (i)–(iii) follow from the shape of the logistic function; (ii) is its derivative $\sigma'(x) = \sigma(x)(1-\sigma(x))$ with the chain rule on $x = -\Delta/\sigma$. □

Example 4.3. With $\sigma = 1$: a pixel 2 px *outside* ($\Delta = 2$) gets $m = \text{sigmoid}(-2) = 0.119$, while one 3 px *inside* ($\Delta = -3$) gets $m = \text{sigmoid}(3) = 0.953$. With $\sigma = 2$ the outside pixel gets $\text{sigmoid}(-1) = 0.269$: a larger σ blurs the silhouette.

Intuition

A hard triangle is a cookie cutter: a pixel is either in or out. A soft triangle is a cookie with a fuzzy rim. A pixel just outside the rim still "feels" the triangle a little, and therefore the loss on that pixel can pull the triangle's vertices towards it. This is the gradient signal that hard rasterization lacks.

4.2 Aggregate from multiple triangles *(slide 17)*

For each pixel:

1. find the K closest triangles that have a non-zero mask at the pixel (same code as for the single closest triangle);
2. weight each triangle by its mask and distance along the ray, $w_k \propto m_k\, e^{-\lambda_k/\gamma}$ (a small default weight w_{bg} can model the background);
3. combine the per-triangle appearance, $\boldsymbol{c} = \sum_k w_k \boldsymbol{c}_k$.

Definition 4.4 (Soft blending of K triangles)**.** With the weights normalised to sum to one,

$$w_k = \frac{m_k\, e^{-\lambda_k/\gamma}}{\sum_{j=1}^{K} m_j\, e^{-\lambda_j/\gamma} + w_{bg}}, \qquad \boldsymbol{c} = \sum_{k=1}^{K} w_k \boldsymbol{c}_k + w'_{bg}\, \boldsymbol{c}_{bg},$$

where $w'_{bg} = w_{bg}/(\sum_j m_j e^{-\lambda_j/\gamma} + w_{bg})$ is the background's share.

Proposition 4.5 (Limits and gradient of soft blending)**.** *(a) If $m_j > 0$ and $\lambda_j < \lambda_k$ then $w_k/w_j = (m_k/m_j)\, e^{-(\lambda_k - \lambda_j)/\gamma} \to 0$ as $\gamma \to 0$: the closest triangle wins and we recover the hard z-buffer. (b) For fixed masks and no background,* $\dfrac{\partial \boldsymbol{c}}{\partial \lambda_k} = -\dfrac{w_k}{\gamma}\,(\boldsymbol{c}_k - \boldsymbol{c})$.

Proof. (a) is the ratio of two weights. (b) The weights are a softmax of $-\lambda_j/\gamma + \log m_j$, so $\partial w_j/\partial \lambda_k = -\frac{1}{\gamma} w_j(\delta_{jk} - w_k)$. Hence $\partial \boldsymbol{c}/\partial \lambda_k = \sum_j \boldsymbol{c}_j\, \partial w_j/\partial \lambda_k = -\frac{1}{\gamma}(w_k \boldsymbol{c}_k - w_k \sum_j w_j \boldsymbol{c}_j)$. □

Example 4.6. Two fully visible triangles ($m = 1$) at depths $\lambda_1 = 1$, $\lambda_2 = 2$. For $\gamma = 1$ the weights are $\propto e^{-1} = 0.368$ and $e^{-2} = 0.135$, i.e. $w = (0.731, 0.269)$. For $\gamma = 0.1$ the ratio is $e^{-10} \approx 4.5 \cdot 10^{-5}$, so $w \approx (0.99995, 0.00005)$: almost a hard z-buffer.

Intuition

The mask answers "is this triangle over the pixel?", the exponential answers "how much does this one deserve to be seen, given how far it is?". γ is a temperature: large γ lets triangles behind show through (semi-transparency, good gradients), small γ is a crisp opaque rendering. By Proposition 4.5(b) even a partially hidden triangle ($w_k > 0$) gets a gradient that nudges it towards or away from the camera, which fixes the depth-ordering problem of the previous section.

4.3 Effect of γ and σ, and an example *(slides 18–19)*

4.4 Differentiable Mesh Rendering via Ray tracing *(slide 20)*

1. Identify the closest K 'softly' intersecting triangles (and the corresponding pixel-space SDF);
2. aggregate appearance using masks, depth-dependent weights, and per-triangle appearance.

[**insert image here**]
taken from **slide 18**

Figure 257: Cube rendered with standard rendering (left), with a larger γ (middle: back faces show through) and with larger γ and σ (right: blurred silhouette). From Liu et al., ICCV 2019.

[**insert image here**]
taken from **slide 19**

Figure 258: Soft Rasterizer example: a mesh is optimised from an initialization towards a target image/silhouette (the red hand is the part that has to move); the right part shows intermediate optimisation results.

Algorithm 4.1 (Ray-traced soft mesh rendering, slide 21 structure).

```
def render(mesh, camera):
    ### Iterate over all pixels
    for pixel in camera.grid:
        ray = pixel2ray(camera, pixel)
        ### Iterate over all triangles
        for triangle in mesh:
            # Find current K closest intersecting triangles
        ### Iterate over triangles influencing this pixel
        for triangle in sorted_k_closest:
            # Aggregate appearance
```

5 From Ray tracing to Rasterization *(slides 21–25)*

5.1 Redundant computation *(slide 21)*

In the code above, neighbouring pixels (two rays on the cow's nose) loop over the *same* triangles and redo much of the same work. **Idea: iterate over triangles just once, and store which pixels they affect.**

Proposition 5.1 (Cost of the two loop orders). *Let P be the number of pixels, N the number of primitives, $\bar{f}$ the average number of pixels a primitive's footprint covers and K the number of triangles kept per pixel. Pixel-major (ray tracing) costs $\Theta(PN)$ intersection tests. Primitive-major (rasterization + blending) costs $\Theta(N\bar{f})$ to fill the per-pixel structure plus $\Theta(PK)$ to blend.*

Example 5.2. $P = 10^6$ pixels, $N = 10^6$ triangles, $\bar{f} = 10$, $K = 10$: pixel-major needs 10^{12} tests; primitive-major needs about $10^7 + 10^7 = 2 \cdot 10^7$ operations (ignoring the $\log K$ factor of buffer insertion), five orders of magnitude less.

Intuition

Instead of asking every pixel "who is on top of you?", we ask every triangle "whom do you cover?". A triangle is small, so it covers few pixels, and the answer is cheap. Ray tracing is a *gather* (pixels pull), rasterization is a *scatter* (primitives push).

5.2 From Ray tracing to Rasterization *(slides 22–23)*

Ray tracing

```
for pixel in camera.grid:
  for triangle in mesh:
    # keep K closest
  for triangle in sorted_k_closest:
    # aggregate
```

Rasterization + Blending

```
for triangle in mesh:
  tri_2d = project(triangle, image)
  # update K-buffer for pixels in tri_2d
```

Rasterization

```
for pixel in camera.grid:
  for triangle in k_closest_img[pixel]:
    # aggregate
```

Blending

Figure 259: Swapping the loop order: the K closest triangles per pixel are found while iterating over triangles once, and the pixel loop only blends (slide 22).

Algorithm 5.1 (Rasterization-based soft mesh rendering). `def render(mesh, camera):`

```
    ### some structure to store K triangles for each pixel
    sorted_k_closest_img = ...
    ### Iterate over all triangles   (Rasterization)
    for triangle in mesh:
        tri_2d = project(triangle, image)
        # Update closest_K_img for pixels within tri_2d
    ### Iterate over all pixels      (Blending + Shading)
    for pixel in camera.grid:
        for triangle in sorted_k_closest_img[pixel]:
            # Aggregate appearance
```

On slide 23 the PyTorch3D renderer is drawn as the pipeline below.

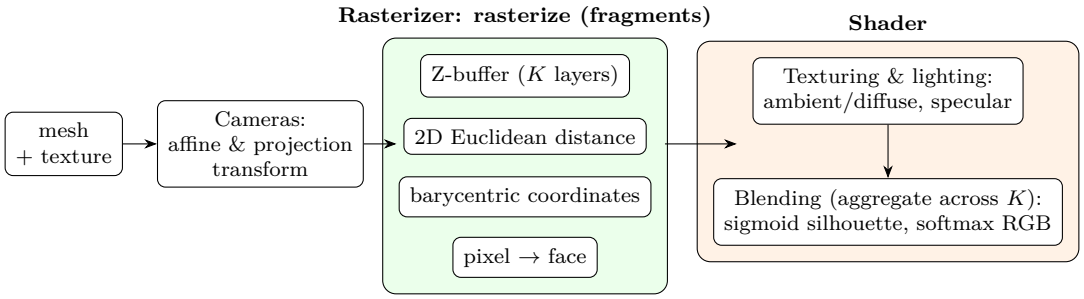


Figure 260: The PyTorch3D renderer (Ravi et al., 2020), redrawn from slide 23: a rasterizer produces per-pixel fragments (depth, distance, barycentrics, face index, K layers) and a shader textures, lights and blends them. The sigmoid silhouette is the soft mask m and the softmax RGB is the depth-weighted combination above.

5.3 Differentiable Primitive Rendering *(slides 24–25)*

The loop structure is independent of the primitive type:

Algorithm 5.2 (Generic differentiable primitive rendering).

```
def render(primitives, camera):
    # Initialize a rasterization data structure
    # (records influencing primitives for each pixel)
    for primitive in primitives:                # Rasterization
        prim2d = project(primitive, camera)
        # Update rasterization data structure
    for pixel in camera.grid:                   # Blending
        # Aggregate appearance from influencing primitives
```

For Gaussians (slide 25) the same code is instantiated with `gaussians`, projecting each one to a 2D Gaussian (`gauss2d`). To fill it in we must answer three questions: *What is the representation of a 3D Gaussian? How to project to 2D and rasterize? How to model/aggregate appearance?*

6 Differentiable Gaussian Rendering *(slides 26–38)*

6.1 What is the representation of a 3D gaussian? *(slides 26–29)*

[**insert image here**]
taken from **slide 26**

Figure 261: A real scene (forest, left) and a view of the same scene where the individual Gaussians are visible as flat shaded blobs (right).

6.1.1 Gaussian primitive *(slide 27)*

Definition 6.1 (3D Gaussian primitive). A Gaussian primitive is the tuple $(\boldsymbol{p}, \boldsymbol{\Sigma}, \alpha, \boldsymbol{c})$ with density

$$\mathcal{G}_{\boldsymbol{\Sigma}}(\boldsymbol{x}-\boldsymbol{p}) = \frac{1}{(2\pi)^{3/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\Big(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{p})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{p})\Big),$$

an opacity α, and a view-dependent colour $\boldsymbol{c}$ given by spherical-harmonics (SH) coefficients. The covariance is factorised into scale and rotation,

$$\boldsymbol{\Sigma} = \boldsymbol{R}\boldsymbol{S}\boldsymbol{S}^{\mathsf{T}}\boldsymbol{R}^{\mathsf{T}}, \qquad \boldsymbol{S} = \operatorname{diag}(s_1, s_2, s_3).$$

The level sets $(\boldsymbol{x}-\boldsymbol{p})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{p}) = \text{const}$ are ellipsoids whose axes are the columns of $\boldsymbol{R}$ with semi-axis lengths proportional to s_i.

Remark 6.2 (Normalisation, beyond the slides). The slide writes the normalised density (with the factor $1/((2\pi)^{3/2}|\boldsymbol{\Sigma}|^{1/2})$). In the reference 3DGS implementation (Kerbl et al.) the opacity α multiplies the

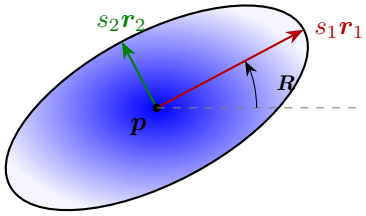


Figure 262: A 2D slice of a Gaussian: position $\boldsymbol{p}$, principal axes $\boldsymbol{r}_i$ (columns of $\boldsymbol{R}$) and scales s_i.

unnormalised exponential, so α is the peak opacity of the primitive. The code on slide 37 does exactly this.

Intuition

A 3D Gaussian is a soft ellipsoidal "cloud of paint": densest at the centre, fading smoothly outwards, stretched along the directions of $\boldsymbol{R}$. Because the fall-off is smooth and infinitely differentiable, small changes of $\boldsymbol{p}, \boldsymbol{s}, \boldsymbol{R}, \alpha$ change the image smoothly, which is what we need for gradients.

6.1.2 What Each Gaussian Stores *(slide 28)*

Parameter	Symbol	Floats
Position	$\boldsymbol{\mu}$	3
Scale	$\boldsymbol{s}$	3
Rotation (quaternion)	$\boldsymbol{q}$	4
Opacity	α	1
Colour, SH degree 3		48
Total		**59**

SH of degree 3 has $(3+1)^2 = 16$ coefficients per colour channel, i.e. $16 \times 3 = 48$ floats.

Example 6.3. About $3 \cdot 10^6$ Gaussians per scene, at $59 \times 4 = 236$ bytes each, give ≈ 708 MB, i.e. the "$\sim$700 MB" on the slide (consistent with the 734 MB of the results table later).

6.1.3 Why Not Optimize the Covariance Directly? *(slide 29)*

A gradient step on $\boldsymbol{\Sigma}$ can destroy its validity: the slide's example moves $\mathrm{diag}(1, 0.01, 1)$ by $-\eta\nabla\mathcal{L}$ to $\mathrm{diag}(1, -0.02, 1)$, a *negative variance*, which is not a Gaussian at all.

Proposition 6.4 (Factorised covariance is always PSD)**.** *For any rotation $\boldsymbol{R}$ and any real $\boldsymbol{S} = \mathrm{diag}(s_1, s_2, s_3)$, $\boldsymbol{\Sigma} = \boldsymbol{R}\boldsymbol{S}\boldsymbol{S}^\mathsf{T}\boldsymbol{R}^\mathsf{T} = (\boldsymbol{R}\boldsymbol{S})(\boldsymbol{R}\boldsymbol{S})^\mathsf{T}$ is symmetric positive semi-definite, since $\boldsymbol{x}^\mathsf{T}\boldsymbol{\Sigma}\boldsymbol{x} = \|\boldsymbol{S}^\mathsf{T}\boldsymbol{R}^\mathsf{T}\boldsymbol{x}\|^2 \geq 0$. Conversely, every symmetric positive definite $\boldsymbol{\Sigma}$ has such a factorisation (spectral theorem: $\boldsymbol{\Sigma} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\mathsf{T}$, take $\boldsymbol{R} = \boldsymbol{Q}$, $\boldsymbol{S} = \boldsymbol{\Lambda}^{1/2}$).*

Intuition

Optimising $\boldsymbol{\Sigma}$ directly is like steering a car by moving the wheels' bolts around: nothing prevents an impossible configuration. Optimising $(\boldsymbol{S}, \boldsymbol{R})$ instead is steering with the wheel: whatever you do, you still get a valid ellipsoid. In the example above the update on $s_2 = 0.1$ can make s_2 any real number, but $\Sigma_{22} = s_2^2$ is never negative.

Remark 6.5 (Beyond the slides). In the reference implementation the scale is stored in log-space (activation exp) so that $s_i > 0$, the rotation is a quaternion that is normalised before building $\boldsymbol{R}$, and

opacity is stored before a sigmoid so that $\alpha \in (0,1)$. The colour is $\boldsymbol{c}(\boldsymbol{d}) = \sum_{\ell \le 3} \sum_m k_{\ell m} Y_\ell^m(\boldsymbol{d})$ for the viewing direction $\boldsymbol{d}$.

6.2 How to project to 2D and rasterize? *(slides 30–34)*

6.2.1 World to camera *(slides 30–31)*

We use the camera extrinsics to transform each 3D Gaussian to the camera frame: $(\boldsymbol{p}, \boldsymbol{R}, \boldsymbol{S}) \to (\boldsymbol{p}', \boldsymbol{R}', \boldsymbol{S})$.

Lemma 6.6 (Affine maps keep Gaussians Gaussian)**.** *If* $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ *then* $\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b} \sim \mathcal{N}(\boldsymbol{A}\boldsymbol{\mu} + \boldsymbol{b},\ \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\mathsf{T})$.

Proof. The characteristic function of $\boldsymbol{x}$ is $\varphi(\boldsymbol{t}) = \exp(i\boldsymbol{t}^\mathsf{T}\boldsymbol{\mu} - \frac{1}{2}\boldsymbol{t}^\mathsf{T}\boldsymbol{\Sigma}\boldsymbol{t})$. For $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$, $\varphi_y(\boldsymbol{t}) = e^{i\boldsymbol{t}^\mathsf{T}\boldsymbol{b}}\varphi(\boldsymbol{A}^\mathsf{T}\boldsymbol{t}) = \exp\left(i\boldsymbol{t}^\mathsf{T}(\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b}) - \frac{1}{2}\boldsymbol{t}^\mathsf{T}\boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\mathsf{T}\boldsymbol{t}\right)$, which is the characteristic function of $\mathcal{N}(\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b}, \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\mathsf{T})$. Note that $\boldsymbol{A}$ need not be invertible. □

World→camera is affine, so

$$\boldsymbol{\mu}_c = \boldsymbol{W}\boldsymbol{\mu} + \boldsymbol{t}, \qquad \boldsymbol{\Sigma}_c = \boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^\mathsf{T} = (\boldsymbol{W}\boldsymbol{R})\boldsymbol{S}\boldsymbol{S}^\mathsf{T}(\boldsymbol{W}\boldsymbol{R})^\mathsf{T},$$

that is $\boldsymbol{p}' = \boldsymbol{W}\boldsymbol{p} + \boldsymbol{t}$, $\boldsymbol{R}' = \boldsymbol{W}\boldsymbol{R}$ and the scale is unchanged (slide 30).

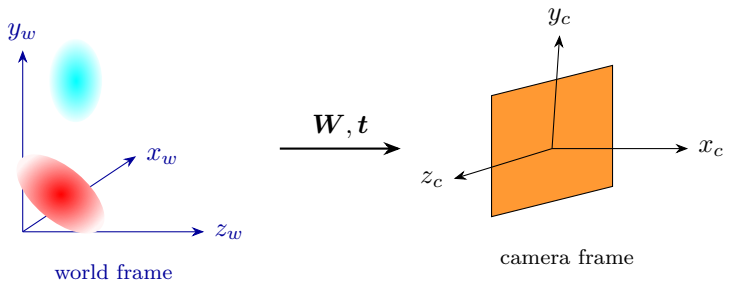


Figure 263: Gaussians are expressed in the camera frame by the affine map $(\boldsymbol{W}, \boldsymbol{t})$ (slides 30–31).

Intuition

Changing the viewpoint does not change the cloud of paint; it only changes the coordinates in which we describe it. A rigid motion rotates the ellipsoid ($\boldsymbol{R} \to \boldsymbol{W}\boldsymbol{R}$) and moves its centre, nothing else.

6.2.2 Projection to the image *(slides 32–33)*

Let π map 3D points to pixels: $\pi(\boldsymbol{x}) = \boldsymbol{u}$ with $z\,(u, v, 1)^\mathsf{T} = \boldsymbol{K}\,(X, Y, Z)^\mathsf{T}$. The 2D mean is $\boldsymbol{\mu}_{2D} = \pi(\boldsymbol{\mu}_{3D})$. *What is the image-space projection of a 3D Gaussian?* It can be approximated as a 2D Gaussian (EWA Volume Splatting, Zwicker et al., 2001), with covariance

$$\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\,\boldsymbol{\Sigma}_{3D}\,\boldsymbol{J}^\mathsf{T}, \qquad \boldsymbol{J} = \frac{\partial \pi}{\partial \boldsymbol{x}}(\boldsymbol{\mu}_{3D}).$$

The projection is *not* affine: $\pi(\boldsymbol{x}_c) = \left(f_x X/Z + c_x,\ f_y Y/Z + c_y\right)^\mathsf{T}$ with $\boldsymbol{x}_c = (X, Y, Z)^\mathsf{T}$.

Theorem 6.7 (EWA: local affine approximation of the projection)**.** *Let* $\boldsymbol{x}_c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ *with* $\boldsymbol{\Sigma}_c = \boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^\mathsf{T}$. *Linearising* π *at the mean,* $\pi(\boldsymbol{x}_c) \approx \pi(\boldsymbol{\mu}_c) + \boldsymbol{J}(\boldsymbol{x}_c - \boldsymbol{\mu}_c)$, *with*

$$\boldsymbol{J} = \left.\frac{\partial \pi}{\partial \boldsymbol{x}_c}\right|_{\boldsymbol{\mu}_c} = \begin{bmatrix} f_x/Z & 0 & -f_x X/Z^2 \\ 0 & f_y/Z & -f_y Y/Z^2 \end{bmatrix}_{\boldsymbol{\mu}_c},$$

the projected Gaussian is $\mathcal{N}(\boldsymbol{\mu}_{2D}, \boldsymbol{\Sigma}_{2D})$ *with* $\boldsymbol{\mu}_{2D} = \pi(\boldsymbol{\mu}_c)$ *and* $\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^\mathsf{T}\boldsymbol{J}^\mathsf{T} \in \mathbb{R}^{2\times 2}$.

Proof. The linearisation is an affine map of $\boldsymbol{x}_c$ with linear part $\boldsymbol{J}$. Apply Lemma 6.6. The derivatives are those of $f_x X/Z$ and $f_y Y/Z$. □

Example 6.8. Take $f_x = f_y = 500$, $(c_x, c_y) = (320, 240)$, $\boldsymbol{\mu}_c = (0.2, -0.1, 2.0)$ and $\boldsymbol{\Sigma}_c = \operatorname{diag}(0.01, 0.01, 0.04)$. Then $\boldsymbol{\mu}_{2D} = (500 \cdot 0.1 + 320,\ 500 \cdot (-0.05) + 240) = (370, 215)$ and $\boldsymbol{J} = \begin{bmatrix} 250 & 0 & -25 \\ 0 & 250 & 12.5 \end{bmatrix}$, so

$$\boldsymbol{\Sigma}_{2D} = \begin{bmatrix} 250^2 \cdot 0.01 + 25^2 \cdot 0.04 & -25 \cdot 12.5 \cdot 0.04 \\ \cdot & 250^2 \cdot 0.01 + 12.5^2 \cdot 0.04 \end{bmatrix} = \begin{bmatrix} 650 & -12.5 \\ -12.5 & 631.25 \end{bmatrix},$$

i.e. a splat with standard deviation of about 25 px. The depth uncertainty (0.04) leaks into the image through the off-axis entries of $\boldsymbol{J}$.

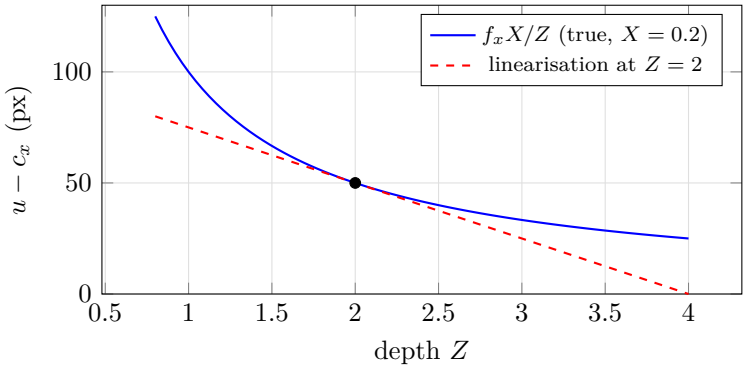


Figure 264: The linearisation is exact at the Gaussian mean and degrades away from it: the approximation is good if the Gaussian's extent is small compared with its depth.

Intuition

Seen from far away, the nonlinear perspective map looks locally like a plain linear map (a tilted ruler). If a Gaussian is small relative to its distance, replacing the true projection by its tangent line at the centre changes almost nothing, and linear maps keep Gaussians Gaussian. Large Gaussians very close to the camera are where this breaks.

6.2.3 Rasterize *(slide 34)*

1. Sort the Gaussians from closest to furthest from the camera.
2. For each pixel $\boldsymbol{u}$, compute the opacity of each Gaussian $\mathcal{G}_k$:

$$\bar{\alpha}_k = \alpha_k \frac{\exp\big(-\tfrac{1}{2}(\boldsymbol{u} - \boldsymbol{\mu}_{2D}^k)^\mathsf{T}(\boldsymbol{\Sigma}_{2D}^k)^{-1}(\boldsymbol{u} - \boldsymbol{\mu}_{2D}^k)\big)}{2\pi\,|\boldsymbol{\Sigma}_{2D}^k|^{0.5}}.$$

In practice one rasterizes *blocks* instead of the entire image, since not all Gaussians influence all blocks. (As in the remark above, implementations drop the normalising denominator so that $\bar{\alpha}_k \le \alpha_k$.)

6.3 Tile Rasterizer *(slides 35–37)*

6.3.1 Tile Rasterizer 1/3: one instance per touched tile *(slide 35)*

1. Cull Gaussians outside the frustum.
2. Split the image into 16×16-pixel tiles.
3. Make one copy (*instance*) of the Gaussian for every tile it touches.
4. Give each instance the key = (tile ID, view depth), a 64-bit integer: tile ID in the high 32 bits, depth in the low 32 bits.

Remark 6.9 (Beyond the slides). In the reference implementation a Gaussian's footprint is a disc of radius $3\sqrt{\lambda_{\max}(\boldsymbol{\Sigma}_{2D})}$ around $\boldsymbol{\mu}_{2D}$ (three standard deviations), and the tiles overlapped by that disc receive an instance.

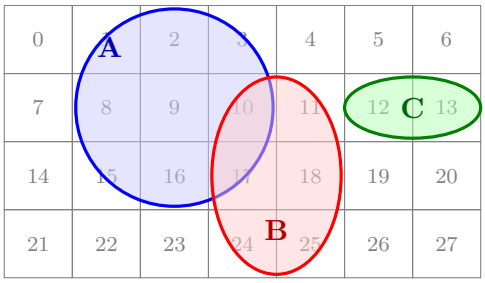


Figure 265: 28 tiles (tile ID $= 7 \cdot \text{row} + \text{col}$) and three Gaussians A, B, C with view depths 2.1, 1.4, 3.0 (reconstructed from slides 35–36). A touches 9 tiles, B 6 tiles and C 2 tiles: 17 instances in total.

6.3.2 Tile Rasterizer 2/3: one radix sort *(slide 36)*

The instances are generated per Gaussian and per overlapped tile, then sorted once with a radix sort on the 64-bit key.

Gaussian	Instances (tile: depth)
A (2.1)	t1, t2, t3, t8, t9, t10, t15, t16, t17
B (1.4)	t10, t11, t17, t18, t24, t25
C (3.0)	t12, t13
After sorting (grouped by tile, front to back inside a tile):	
t1:A t2:A t3:A t8:A t9:A t10:**B,A** t11:B t12:C t13:C t15:A t16:A t17:**B,A** t18:B t24:B t25:B	

Proposition 6.10 (One global sort gives all per-tile lists)**.** *Sorting instances lexicographically by (tile ID, depth) puts all instances of the same tile in one contiguous block, ordered by increasing depth inside the block.*

Proof. The tile ID occupies the high bits, so it is compared first and instances of different tiles are ordered by tile. Within a tile the low bits decide, i.e. the depth. (For positive IEEE-754 floats, the integer order of the bit pattern equals the numeric order, so the depth bits can be sorted as integers.) □

Intuition

It is like sorting a deck of cards by suit first and rank second with one single pass, instead of dealing the cards into four piles and sorting each pile. The GPU then hands each pile (tile) to one block of threads that already knows whom to draw, front to back. The cost is linear in the number M of instances (here $M = 17$; in a real scene millions).

Remark 6.11 (Popping). Depth is stored *per Gaussian*, not per pixel (slide 36: "depth per Gaussian, not per pixel → popping"). Two overlapping, elongated Gaussians may be at the same depth order at one pixel but swapped at another. With a single global order, a small camera motion can flip the centre depths of two Gaussians, and the image changes abruptly (pops).

6.3.3 Tile Rasterizer 3/3: Blend Each Tile *(slide 37)*

Algorithm 6.1 (Per-pixel blending, runs in parallel for every pixel).

```
    # runs in parallel for every pixel
def render_pixel(pixel, tile):
    T, C = 1.0, 0.0
    for g in sorted_gaussians[tile]:          # front to back
        d = pixel - g.mean2d
        a = min(0.99, g.opacity
                * exp(-0.5 * d @ inv(g.cov2d) @ d))
        if a < 1 / 255:
            continue                          # too faint
        if T * (1 - a) < 1e-4:
            break                             # pixel saturated
        C += g.color * a * T
        T *= 1 - a
    final_T[pixel] = T                        # kept for backward
    return C
```

The four ingredients flagged on the slide: **front to back** order; **early stop** when the pixel is saturated; **skip** $\bar{\alpha} < 1/255$; **clamp** $\bar{\alpha} \leq 0.99$.

Intuition

Front to back lets us stop as soon as the pixel is essentially opaque ($T < 10^{-4}$): everything behind cannot change the colour. Skipping very faint splats ($< 1/255$, below one 8-bit colour level) saves work, and clamping $\bar{\alpha} \leq 0.99$ guarantees $1 - \bar{\alpha} \geq 0.01$, so that the backward pass, which divides by $1 - \bar{\alpha}$, is numerically safe.

6.4 How to model/aggregate appearance? *(slide 38)*

Compute per-Gaussian weights from the opacities of the current and all previous (closer) Gaussians, take the view-dependent colour $\boldsymbol{c}_k$ from the SH coefficients and the ray direction, and aggregate:

$$w_k = \bar{\alpha}_k \prod_{j=1}^{k-1} (1 - \bar{\alpha}_j), \qquad \boldsymbol{c} = \sum_k w_k \boldsymbol{c}_k.$$

Proposition 6.12 (Weights telescope). *With $T_k = \prod_{j<k}(1 - \bar{\alpha}_j)$ we have $w_k = T_k - T_{k+1}$, hence $\sum_{k=1}^{N} w_k = 1 - T_{N+1} \leq 1$.*

Proof. $T_k \bar{\alpha}_k = T_k - T_k(1 - \bar{\alpha}_k) = T_k - T_{k+1}$; summing the telescoping series gives $T_1 - T_{N+1} = 1 - T_{N+1}$. □

Example 6.13. For $\bar{\alpha} = (0.5, 0.4, 0.8)$: $T = (1, 0.5, 0.3, 0.06)$, $w = (0.5, 0.2, 0.24)$, $\sum w = 0.94 = 1 - 0.06$. With colours red, green, blue the pixel is $(0.5, 0.2, 0.24)$, and 6% of the light is left over for whatever is behind (e.g. the background colour).

Intuition

Think of a pixel as a budget of 1 unit of light. The nearest Gaussian absorbs the fraction $\bar{\alpha}_1$; the next one absorbs $\bar{\alpha}_2$ *of what remains*, and so on. This is exactly the volume-rendering bookkeeping $T_i \alpha_i$ of the recap, with $\bar{\alpha}_k$ playing the role of α_i, but evaluated only at the primitives instead of at many empty samples.



total light budget $= 1$: each Gaussian takes its share $\bar{\alpha}_k T_k$ of what is left

Figure 266: Light budget of the example: front Gaussians take their share first; later ones only see the transmittance T_k that remains.

7 3D Gaussian Splatting: The Full Pipeline *(slides 39–41)*

7.1 The Full Pipeline *(slide 39)*

- **Forward:** project → sort → blend.
- **Backward:** gradients to every Gaussian.
- **Density control:** add and remove Gaussians.

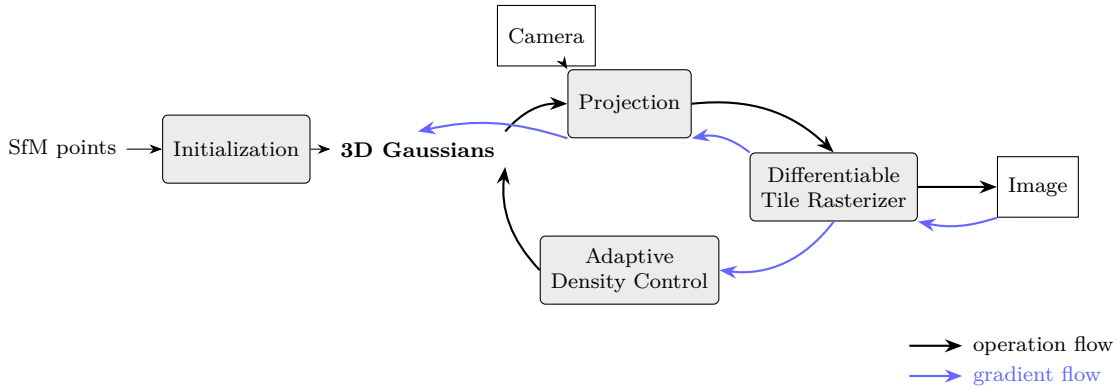


Figure 267: The 3DGS pipeline (Kerbl et al., SIGGRAPH 2023), redrawn from slide 39. Black arrows: operation flow; blue arrows: gradient flow. Gradients from the rasterizer reach the projection (and hence the Gaussian parameters) and also drive the adaptive density control.

7.2 Backward Pass: Gradients Through Blending *(slide 40)*

Let $\boldsymbol{c} = \sum_{i=1}^{N} \boldsymbol{c}_i \, \bar{\alpha}_i \, T_i$ with $T_i = \prod_{j<i}(1 - \bar{\alpha}_j)$.

Proposition 7.1 (Gradients of front-to-back blending)**.**

$$\frac{\partial \boldsymbol{c}}{\partial \boldsymbol{c}_i} = \bar{\alpha}_i T_i, \qquad \frac{\partial \boldsymbol{c}}{\partial \bar{\alpha}_i} = T_i \boldsymbol{c}_i - \frac{1}{1 - \bar{\alpha}_i} \sum_{j>i} \boldsymbol{c}_j \bar{\alpha}_j T_j.$$

Proof. $\boldsymbol{c}_i$ appears only in term i, with coefficient $\bar{\alpha}_i T_i$. The opacity $\bar{\alpha}_i$ appears (a) in term i as the factor $\bar{\alpha}_i$, giving $T_i \boldsymbol{c}_i$ (because T_i does not depend on $\bar{\alpha}_i$), and (b) inside T_j for every $j > i$ through the factor $(1 - \bar{\alpha}_i)$, so $\partial T_j / \partial \bar{\alpha}_i = -T_j/(1 - \bar{\alpha}_i)$, contributing $-\sum_{j>i} \boldsymbol{c}_j \bar{\alpha}_j T_j / (1 - \bar{\alpha}_i)$. □

Algorithm 7.1 (Backward pass: walk back to front, recover T from the final T)**.** Since $T_{i+1} = T_i(1 - \bar{\alpha}_i)$ we get $T_i = T_{i+1}/(1 - \bar{\alpha}_i)$. Only the final transmittance T_{N+1} (`final_T`) must be stored in the forward pass. With upstream gradient $\boldsymbol{g} = \partial \mathcal{L} / \partial \boldsymbol{c}$:

```
T = final_T[pixel];  S = 0                  # S = sum_{j>i} c_j a_j T_j
for i in reversed(range(N)):                # back to front
    T = T / (1 - a[i])                      # now T = T_i
    grad_c[i] = a[i] * T * g
```

```
grad_a[i] = g @ (T * c[i] - S / (1 - a[i]))
S += c[i] * a[i] * T
```

Example 7.2. Scalar colours $c = (1, 0.5, 1)$, $\bar{\alpha} = (0.5, 0.4, 0.8)$, so $T = (1, 0.5, 0.3)$. Then $\partial c/\partial \bar{\alpha}_2 = T_2 c_2 - \frac{1}{1-0.4} c_3 \bar{\alpha}_3 T_3 = 0.25 - \frac{0.24}{0.6} = -0.15$. Directly: $c = 0.9 - 0.15\,\bar{\alpha}_2$, whose derivative is -0.15. ✓

Intuition

The two terms in $\partial \boldsymbol{c}/\partial \bar{\alpha}_i$ are a tug of war. Making Gaussian i more opaque (1) adds more of its *own* colour (positive term $T_i \boldsymbol{c}_i$), but (2) hides everything behind it (negative term, the colour that would have come from behind). Whether the net effect is positive depends on whether $\boldsymbol{c}_i$ is brighter or darker (relative to the loss) than the stuff behind. Walking back to front lets us accumulate "the stuff behind" in the running sum S with no per-sample storage; the clamp $\bar{\alpha} \leq 0.99$ makes the division safe.

The upstream gradient $\partial \mathcal{L}/\partial \bar{\alpha}_i$ is then propagated through $\bar{\alpha}_i = \alpha_i \exp(\cdot)$ to α_i, $\boldsymbol{\mu}_{2D}$ and $\boldsymbol{\Sigma}_{2D}$, and through the projection ($\boldsymbol{J}$, $\boldsymbol{W}$) and the factorisation to $\boldsymbol{\mu}, \boldsymbol{s}, \boldsymbol{q}$; the SH coefficients receive $\partial \boldsymbol{c}/\partial \boldsymbol{c}_i$.

7.3 Optimization *(slide 41)*

The loss mixes an ℓ_1 term and a structural term, $\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}}$ with $\lambda = 0.2$. Training runs for 30K iterations of Adam, one full image per step, with per-parameter learning rates:

Parameter	Learning rate
position $\boldsymbol{\mu}$	$1.6 \cdot 10^{-4} \rightarrow 1.6 \cdot 10^{-6}$
SH coefficients	$2.5 \cdot 10^{-3}$
opacity	$2.5 \cdot 10^{-2}$
scale	$5 \cdot 10^{-3}$
rotation	10^{-3}

Intuition

Different parameters live on different scales, so one learning rate does not fit all: opacity (a number in $(0, 1)$) can move fast, while positions are in scene units and must settle gently, hence the decay by a factor 100 over training.

8 Initialization, Densification and Ablations *(slides 42–45)*

8.1 Initialization: SfM Points vs Random *(slide 42)*

- One Gaussian per SfM point.
- Isotropic, $\alpha = 0.1$.
- Random initialization gives a worse background.

8.2 Adaptive Density Control *(slide 43)*

Triggered by **large positional gradients**:

- **Small** Gaussian: *clone*.
- **Large** Gaussian: *split*, scale $\div 1.6$.
- **Prune** $\alpha < 0.005$.
- **Reset** α every 3000 iterations.

Algorithm 8.1 (Adaptive density control)**.** For each Gaussian whose accumulated view-space positional gradient exceeds a threshold: if it is small, clone it (copy, then let optimisation separate the

[**insert image here**]
taken from **slide 42**

Figure 268: Garden scene: random initialization (top row) vs. SfM initialization (bottom row) and a zoomed crop; the random init is visibly worse in the background. From Kerbl et al., 2023.

two); if it is large, replace it by two Gaussians with scales divided by 1.6. Remove Gaussians with $\alpha < 0.005$. Every 3000 iterations, reset all opacities to a low value.

Remark 8.1 (Beyond the slides). In Kerbl et al. the positional-gradient threshold is $\tau_{pos} = 2 \cdot 10^{-4}$, densification runs every 100 iterations until about iteration 15K, and the two Gaussians of a split are placed by sampling positions from the original Gaussian. Resetting opacity lets Gaussians that are really needed regain opacity within a few iterations while floaters stay transparent and are pruned.

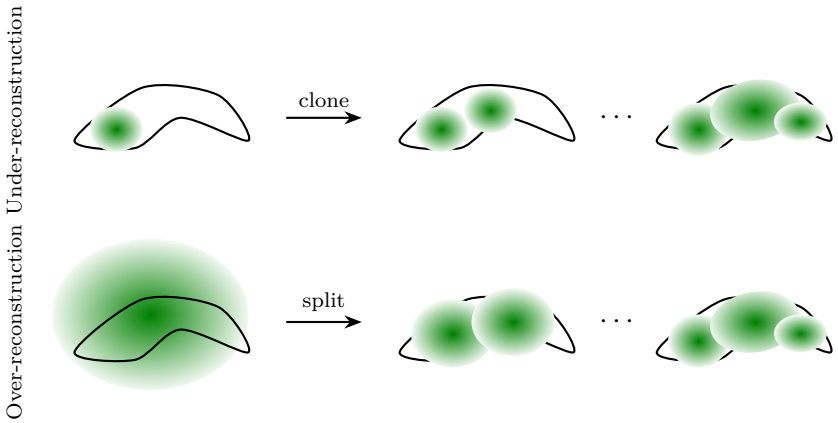


Figure 269: Adaptive density control (redrawn from slide 43): a too small Gaussian that does not cover the shape is cloned; a too large Gaussian that blurs the shape is split into smaller ones. Optimisation then continues.

Intuition

A large positional gradient means "many pixels disagree about where this Gaussian should be": it is trying to be in several places at once. If it is already small, the geometry is simply under-covered, so we add a copy (clone). If it is big, one blob is trying to explain structure that needs detail, so we replace it by smaller ones (split, $\div 1.6$ in scale). Pruning removes Gaussians that contribute almost nothing.

8.3 Ablation: Densification *(slide 44)*

Without splitting, large Gaussians cannot be refined and the background stays blurry; without cloning, thin structures such as spokes are not covered by enough Gaussians.

[**insert image here**]
taken from **slide 44**

Figure 270: Bicycle crop, three versions: **No split** (blurry background), **No clone** (broken spokes, the arrow marks the wheel), **Full**. From Kerbl et al., 2023.

8.4 Ablation: Anisotropy *(slide 45)*

[**insert image here**]
taken from **slide 45**

Figure 271: Potted plant: ground truth, full model (anisotropic Gaussians) and the isotropic ablation, which yields blobs instead of leaves. From Kerbl et al., 2023.



1 anisotropic Gaussian ≥ 6 isotropic Gaussians

Figure 272: Toy example (not on the slides): a thin leaf or branch needs a single stretched ellipsoid, but a chain of many round blobs if only isotropic Gaussians are allowed, and the result still looks dotted.

Intuition

Natural scenes are full of thin or flat things (leaves, spokes, walls). An anisotropic Gaussian is a pencil or a sheet of paint; an isotropic one is a ball. Using balls to draw lines needs many more balls and leaves visible beads, as seen in the plant ablation.

9 Results: Quality and Speed *(slides 46–48)*

Method	PSNR↑	SSIM↑	LPIPS↓	Train	FPS	Memory
Plenoxels	23.08	0.626	0.463	25m49s	6.8	2.1 GB
Instant-NGP (big)	25.59	0.699	0.331	7m30s	9.4	48 MB
Mip-NeRF360	27.69	0.792	0.237	48 h	0.06	8.6 MB
3DGS, 7K iter.	25.60	0.770	0.279	6m25s	160	523 MB
3DGS, 30K iter.	27.21	0.815	0.214	41m33s	134	734 MB

Kerbl et al., SIGGRAPH 2023, Table 1 (Mip-NeRF360 dataset).

Example 9.1 (Reading the table)**.** Against Mip-NeRF360, 3DGS (30K) loses 0.48 dB of PSNR but has better SSIM and LPIPS, trains $\approx 69\times$ faster (2880 min vs. 41.5 min) and renders $\approx 2200\times$ faster (134 vs. 0.06 FPS). The price is memory: 734 MB vs. 8.6 MB, about $85\times$. After only 7K iterations (6.5 min) 3DGS already matches Instant-NGP's PSNR with an order of magnitude higher frame rate.

Intuition

NeRF-type models store the scene in a small network and pay at render time (many network queries per ray). 3DGS stores the scene explicitly (millions of Gaussians) and pays in memory, but rendering is just "splat and blend", which GPUs do extremely well. It trades memory for speed.

[**insert image here**]
taken from **slide 47**

Figure 273: Interactive 3DGS viewer on the bicycle scene, showing 57.56 FPS (17.37 ms) with VSync on.

[**insert image here**]
taken from **slide 48**

Figure 274: Interactive 3DGS viewer on an indoor playroom scene, showing 89.37 FPS (11.19 ms).

10 Limitations *(slide 49)*

[**insert image here**]
taken from **slide 49**

Figure 275: Mip-NeRF360 (left) vs. 3DGS (right) on a train scene: 3DGS reconstructs the foreground structures that Mip-NeRF360 smears.

- **Memory:** millions of Gaussians (hundreds of MB per scene).

- **Popping:** the per-Gaussian sort (Remark in the Tile Rasterizer 2/3 subsection).
- **Aliasing under zoom** → Mip-Splatting.
- **No surfaces** → SuGaR, 2DGS.

Intuition

Each limitation is a consequence of a design choice. Explicit storage costs memory. A global depth order per Gaussian is cheap but only approximately correct. A fixed-size screen-space filter does not adapt when zooming. And a volumetric blob has no well-defined surface or normal. The next section shows how swapping the primitive addresses the last one.

11 Other Primitives *(slides 50–54)*

11.1 Other Primitives: Fuzzy Metaballs *(slide 50)*

Keselman and Hebert, *Fuzzy Metaballs: Approximate Differentiable Rendering with Algebraic Surfaces*, ECCV 2022 (Oral). The slide shows a teddy bear fitted from a target mask: the estimated mask and depth start as a small cloud at iteration 0 and are optimised towards the target.

[**insert image here**]
taken from **slide 50**

Figure 276: Fuzzy Metaballs optimisation on a teddy bear: input photo, target mask, estimated mask and estimated depth at iteration 0 (epoch 0.0).

11.2 2D Gaussian Splatting: The Primitive *(slide 51)*

Huang et al., SIGGRAPH 2024. A 3D Gaussian intersected by a plane gives different cross-sections for different viewpoints, whereas a 2D Gaussian is a flat disc whose intersection with every viewing ray is well defined and view-consistent.

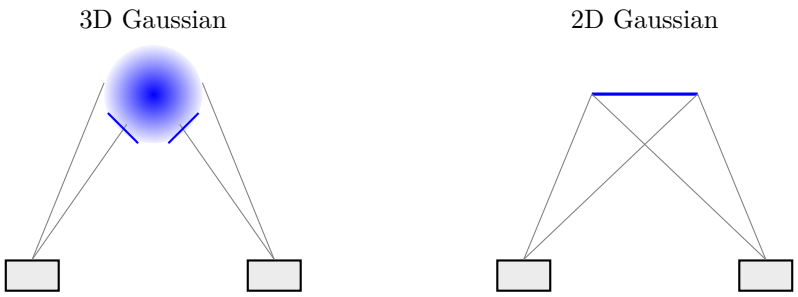


Figure 277: Left: a 3D Gaussian is intersected by planes that differ per viewpoint. Right: a 2D Gaussian is the same planar disc for every camera (schematic, after slide 51).

Definition 11.1 (2D Gaussian splat)**.** A splat has a centre $\boldsymbol{p}_k$, two tangent vectors $\boldsymbol{t}_u, \boldsymbol{t}_v$ with scales

s_u, s_v, and lives in its own tangent plane:

$$P(u,v) = \boldsymbol{p}_k + s_u \boldsymbol{t}_u\, u + s_v \boldsymbol{t}_v\, v = \boldsymbol{H}\,(u,v,1,1)^\mathsf{T}, \qquad \boldsymbol{H} = \begin{bmatrix} \boldsymbol{RS} & \boldsymbol{p}_k \\ \boldsymbol{0}^\mathsf{T} & 1 \end{bmatrix},$$

with $\boldsymbol{R} = [\boldsymbol{t}_u\ \boldsymbol{t}_v\ \boldsymbol{t}_w]$ and $\mathcal{G}(u,v) = \exp\big(-\frac{u^2+v^2}{2}\big)$. The splat is **flat**: $\boldsymbol{S} = \mathrm{diag}(s_u, s_v, 0)$; its **normal** is $\boldsymbol{t}_w = \boldsymbol{t}_u \times \boldsymbol{t}_v$.

Intuition

A 2D Gaussian is a fuzzy coin instead of a fuzzy ball. A coin has a definite orientation (its normal), and one ray hits it at one point, so a single hit gives an **exact depth**. A ball only gives a smeared volumetric depth.

11.3 2D Gaussian Splatting: Ray-Splat Intersection *(slide 52)*

1. **A pixel is two planes.** The ray through pixel (x, y) is the intersection of the x-plane $\boldsymbol{h}_x = (-1, 0, 0, x)^\mathsf{T}$ and the y-plane $\boldsymbol{h}_y = (0, -1, 0, y)^\mathsf{T}$.
2. **Move the planes to the splat frame without an inverse:** $\boldsymbol{h}_u = (\boldsymbol{WH})^\mathsf{T}\boldsymbol{h}_x$, $\boldsymbol{h}_v = (\boldsymbol{WH})^\mathsf{T}\boldsymbol{h}_y$.
3. **Solve for** (u, v)**:** $\boldsymbol{h}_u \cdot (u,v,1,1)^\mathsf{T} = \boldsymbol{h}_v \cdot (u,v,1,1)^\mathsf{T} = 0$.
4. **Edge-on case:** use a screen-space floor, $\hat{\mathcal{G}}(\boldsymbol{x}) = \max\{\mathcal{G}(\boldsymbol{u}(\boldsymbol{x})),\ \mathcal{G}(\frac{\boldsymbol{x}-\boldsymbol{c}}{\sigma})\}$.

Then proceed as in 3DGS: tiles, sort and blend.

Proposition 11.2 (Closed-form ray–splat hit)**.** *Because the third column of* $\boldsymbol{H}$ *is zero* ($s_w = 0$)*, the third component of* $\boldsymbol{h}_u, \boldsymbol{h}_v$ *vanishes and the system in step 3 reads* $h_u^1 u + h_u^2 v + h_u^4 = 0$, $h_v^1 u + h_v^2 v + h_v^4 = 0$, *with solution*

$$u = \frac{h_u^2 h_v^4 - h_u^4 h_v^2}{h_u^1 h_v^2 - h_u^2 h_v^1}, \qquad v = \frac{h_u^4 h_v^1 - h_u^1 h_v^4}{h_u^1 h_v^2 - h_u^2 h_v^1}.$$

Proof. Cramer's rule on the 2×2 system. The denominator vanishes exactly when the splat is seen edge-on (the ray is parallel to the splat's plane), which is the degenerate case handled by the floor in step 4. □

Intuition

Two planes ("everything with this x" and "everything with this y") meet in a line, which is the pixel's ray. Pushing both planes into the splat's own coordinates lets us read off the hit (u, v) by solving two linear equations: no matrix inversion, and the Gaussian weight at the hit is $\mathcal{G}(u, v)$ exactly, rather than a projected approximation. When the coin is seen edge-on it has no area on screen, so we keep a small blob of fixed screen size to avoid losing it (and its gradients).

11.4 From Cameras to LiDAR: Splat-LOAM *(slide 53)*

Giacomini, Di Giammarino, De Rebotti, Grisetti and Oswald, ICCV 2025. The pipeline: a **LiDAR scan** → **range image**; a **spherical 2DGS rasterizer**; **range-only supervision**; **map + odometry**.

Example 11.3 (Spherical projection to a range image)**.** A LiDAR point $\boldsymbol{q} = (x, y, z)$ has range $r = \|\boldsymbol{q}\|$, azimuth $\theta = \mathrm{atan2}(y, x)$ and elevation $\varphi = \arcsin(z/r)$. A range image stores $\boldsymbol{D}(u, v) = r$ at $u \propto \theta$, $v \propto \varphi$ (the standard spherical camera model, not detailed on the slide).

11.5 Beyond Gaussians: Other Primitives *(slide 54)*

- **Voxels:** SVRaster (Sun et al., CVPR 2025).
- **Triangles:** Triangle Splatting (Held et al., 2025).

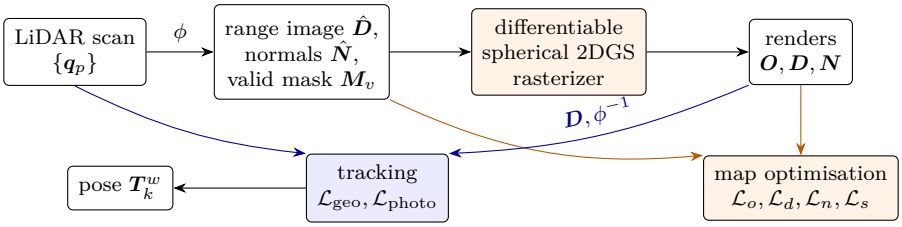


Figure 278: Splat-LOAM, schematically redrawn from slide 53 (block names from the slide; the exact loss definitions are in the paper). The map is optimised from range images, while tracking estimates the sensor pose $\boldsymbol{T}_k^w$ against the rendered depth.

[**insert image here**]
taken from **slide 53**

Figure 279: Splat-LOAM results panel: the rendered normal map of the reconstructed building (right part of the figure).

[**insert image here**]
taken from **slide 54**

Figure 280: Three alternative primitives on the garden scene: sparse voxels (SVRaster), triangles (Triangle Splatting) and Voronoi cells (Radiant Foam).

- **Voronoi cells:** Radiant Foam (Govindarajan et al., 2025).

Intuition

The recipe of this lecture stays fixed: (1) find the primitives that touch a pixel (project and bin them into tiles), (2) order them, (3) blend front to back. What changes from primitive to primitive is only the *footprint* (ellipse, coin, triangle with soft edge, box, polygonal cell) and how its opacity falls off, so that the whole pipeline remains differentiable.

12 Summary *(slide 55)*

Rasterization + blending for efficient rendering. Aggregating 'soft' primitives along a ray allows differentiable rendering. Projecting 3D Gaussians gives 2D rendering via $\boldsymbol{\mu}_{2D} = \pi(\boldsymbol{\mu}_{3D})$, $\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\boldsymbol{\Sigma}_{3D}\boldsymbol{J}^{\top}$.

Method	Primitive	How it renders / stays differentiable	Main limitation
Hard mesh ray-casting	triangles	closest hit per ray; (α, β) smooth in vertices	jumps at occlusion and depth order; no gradient to hidden faces
Soft Rasterizer / PyTorch3D	soft triangles	sigmoid mask of pixel distance + K closest, depth-softmax weights; primitive-major loop	blur controlled by σ, γ; only K layers
3D Gaussian Splatting	3D Gaussians	EWA projection $\boldsymbol{J}\boldsymbol{\Sigma}\boldsymbol{J}^{\top}$; one radix sort on (tile, depth); front-to-back α-blending with analytic backward	memory, popping, aliasing, no surface
2D Gaussian Splatting	flat Gaussians (discs)	exact ray–splat intersection in splat frame; exact depth and normal	edge-on views need a screen-space floor
Splat-LOAM	spherical 2DGS	range-image rasterizer, range-only supervision, map + odometry	LiDAR only (no appearance)
Other	voxels, triangles, Voronoi cells	same project–sort–blend skeleton with another footprint	see papers

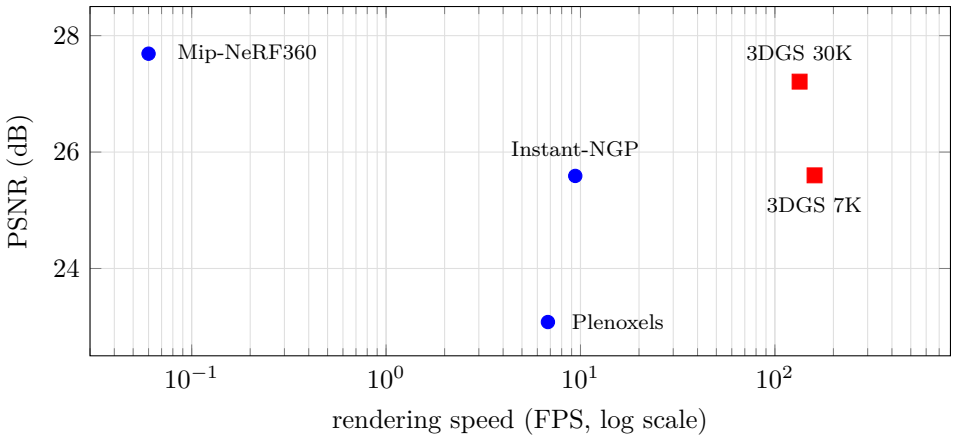


Figure 281: Quality vs. speed on Mip-NeRF360 (data from the results table): the 3DGS points (red) sit towards the top right, i.e. near-best quality at real-time rates.

Key take-aways

1. **Primitive rendering = find influencing primitives + aggregate.** Unlike volume rendering it does not waste samples in empty space.
2. **Hard rendering is not differentiable where it matters:** occlusion and depth ordering create jumps. Soft masks (sigmoid of signed distance) and depth-softmax blending turn the jumps into smooth transitions and give gradients to hidden primitives.
3. **Rasterization swaps the loop order** (primitives first, pixels second), replacing $\Theta(PN)$ tests by $\Theta(N\bar{f} + PK)$; 3DGS does it with tiles and a single radix sort on (tile ID, depth).
4. **3DGS in three formulas:** $\boldsymbol{\Sigma} = \boldsymbol{R}\boldsymbol{S}\boldsymbol{S}^\top\boldsymbol{R}^\top$ (always PSD), $\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^\top\boldsymbol{J}^\top$ (EWA), and front-to-back blending $\boldsymbol{c} = \sum_k \boldsymbol{c}_k \bar{\alpha}_k T_k$ with a back-to-front analytic gradient that only needs the final transmittance.
5. **Optimisation is more than gradient descent:** adaptive density control (clone, split, prune, opacity reset) and SfM initialisation are essential; limitations (memory, popping, aliasing, no surfaces) motivate Mip-Splatting, 2DGS, SuGaR and other primitives.

Acknowledgments (slide 56). The slides are adapted from *Learning for 3D Vision*, CMU 16-825 (Prof. Shubham Tulsiani); the Gaussian-representation slide is adapted from Vincent Sitzmann. Results and diagrams are from Ravi et al. 2020 (PyTorch3D), Liu et al. ICCV 2019 (Soft Rasterizer), Zwicker et al. 2001 (EWA Volume Splatting), Kerbl et al. SIGGRAPH 2023 (3D Gaussian Splatting), Keselman and Hebert ECCV 2022 (Fuzzy Metaballs), Huang et al. SIGGRAPH 2024 (2D Gaussian Splatting), Giacomini et al. ICCV 2025 (Splat-LOAM), Sun et al. CVPR 2025 (SVRaster), Held et al. 2025 (Triangle Splatting) and Govindarajan et al. 2025 (Radiant Foam).

Part X

Lecture 11

1 Relightable 3D Representations *(slides 1–6)*

1.1 Recap: Neural 3D Representations *(slide 3)*

Every neural 3D representation seen so far (density fields such as NeRF and 3D Gaussian Splatting, and neural surfaces such as SDFs used in IDR) can be summarised as a function that, given a 3D point and a viewing direction, returns a colour.

Definition 1.1 (Radiance-field representation)**.** A *radiance-field representation* is a parametrised function

$$g_\phi(\boldsymbol{x}, \boldsymbol{\omega}) = \boldsymbol{c} \in \mathbb{R}^3,$$

mapping a point $\boldsymbol{x} \in \mathbb{R}^3$ and a direction $\boldsymbol{\omega} \in \mathbb{S}$ to the colour $\boldsymbol{c}$ observed when looking at $\boldsymbol{x}$ from direction $\boldsymbol{\omega}$. Density fields (NeRF, 3DGS) and neural surfaces (SDFs, e.g. IDR) differ in *where* they place matter, but both output $\boldsymbol{c}$ in this way.

1.2 Radiance Fields Bake In the Lighting *(slide 4)*

In these models the colour is the light *emitted* (leaving the point) towards the camera. The question is whether the model can be rendered under a different illumination. The answer is no: the lighting under which the photos were taken is baked into g_ϕ.

[Insert image from slide 4]

Figure 282: Radiance fields bake in the lighting (NeRF ship, IDR cans). Take the four images of slide 4. Figure credits: Mildenhall et al., ECCV 2020 (NeRF); Yariv et al., NeurIPS 2020 (IDR).

Proposition 1.2 (No relighting from a radiance field)**.** *The function g_ϕ has no input or parameter that represents the illumination. Hence the only image-space family it can produce is the one under the capture light; changing the light requires changing ϕ, and nothing in ϕ says* how*.*

Intuition

Think of a photograph of a statue under a desk lamp: the bright cheek and the dark side of the nose are *painted* into the photo. A radiance field stores this painted result at every point. To move the lamp you would need to know the statue's real colour (the *albedo*), its shape and how shiny it is — exactly the ingredients the field never separated.

1.3 Goal: Relightable 3D *(slide 5)*

The goal is to start from posed images under *one unknown light* and produce a model that can render novel views *and* novel illuminations (relit images under "light 1", "light 2", . . .).

[Insert image from slide 5]

Figure 283: Goal of relightable 3D: novel views, relit under light 1, relit under light 2. Take the three images of slide 5. Figure credits: Zhang et al., CVPR 2021 (PhySG).

[Insert image from slide 2]

Figure 284: Three renderings of the same cow under different illuminations (title slide of the topic). Take the three cows of slide 2. Figure credits: S. Tulsiani.

1.4 Today's Goals *(slide 6)*

1. What a relightable model needs.
2. Why learning it from images is ambiguous.
3. Three ways to resolve the ambiguity: *constrain the model*, *capture more informative data*, *learn priors from data*.

2 What We Need: The Rendering Equation *(slides 7–9)*

Definition 2.1 (Rendering equation, reflection form)**.** Let $\boldsymbol{x}$ be a point on a surface with unit normal $\boldsymbol{n}$, Ω the hemisphere of directions around $\boldsymbol{n}$, and $\boldsymbol{\omega}_o$ the outgoing direction. The radiance leaving $\boldsymbol{x}$ towards $\boldsymbol{\omega}_o$ is

$$L_o(\boldsymbol{x}, \boldsymbol{\omega}_o) = \int_\Omega \underbrace{f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o)}_{\text{material}} \underbrace{L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)}_{\text{light}} \underbrace{(\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega}_i)}_{\text{geometry}} \, \mathrm{d}\boldsymbol{\omega}_i .$$

The three coloured factors correspond to the three unknowns of the problem: *material* (f_r), *light* (L_i) and *geometry* (the normal $\boldsymbol{n}$ in the cosine foreshortening term, together with the surface position $\boldsymbol{x}$).

Proposition 2.2 (Relighting by swapping the light)**.** *If material f_r and geometry $(\boldsymbol{x}, \boldsymbol{n})$ are known, then the image under any new light L_i^{new} is*

$$L_o^{\text{new}}(\boldsymbol{x}, \boldsymbol{\omega}_o) = \int_\Omega f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \, L_i^{\text{new}}(\boldsymbol{x}, \boldsymbol{\omega}_i) \, (\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega}_i) \, \mathrm{d}\boldsymbol{\omega}_i .$$

This is exact for direct *light only; indirect light (light bounced from other surfaces) depends on the scene and must be recomputed.*

Intuition

The equation is a recipe with three ingredients. If you know the ingredients "material" and "geometry", then "light" is just a plug-in slot: pull out the old light, plug in a new one, re-evaluate the integral. The caveat is that part of the light arriving at $\boldsymbol{x}$ comes from other objects in the scene, whose own

appearance changes with the new light.

2.1 Material: How a Surface Reflects Light *(slide 8)*

Definition 2.3 (BRDF)**.** The *bidirectional reflectance distribution function* $f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \geq 0$ tells how much of the light arriving from direction $\boldsymbol{\omega}_i$ leaves towards $\boldsymbol{\omega}_o$. It is a function on $\mathbb{S} \times \mathbb{S}$ (four degrees of freedom) at every surface point.

A *shiny* material concentrates the reflected light in a narrow lobe around the mirror direction; a *rough* material spreads it widely (Figure 285).

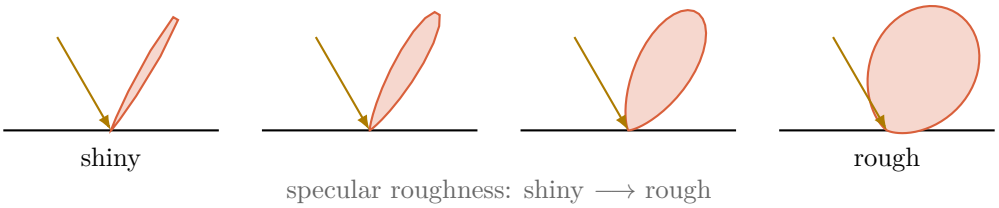


Figure 285: Schematic BRDF lobes for a fixed incoming direction (yellow arrow): the reflected energy (orange) is concentrated around the mirror direction for shiny materials and spreads out as roughness grows.

Example 2.4 (Lambertian material)**.** A *Lambertian* (perfectly diffuse) surface has $f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = a(\boldsymbol{x})/\pi$, where $a(\boldsymbol{x}) \in [0,1]^3$ is the *albedo*. Then

$$L_o(\boldsymbol{x}, \boldsymbol{\omega}_o) = \frac{a(\boldsymbol{x})}{\pi} \int_\Omega L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)\, (\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega}_i)\, \mathrm{d}\boldsymbol{\omega}_i = \frac{a(\boldsymbol{x})}{\pi} E(\boldsymbol{x}),$$

independent of $\boldsymbol{\omega}_o$, where $E(\boldsymbol{x})$ is the irradiance. Under constant light $L_i \equiv 1$ one has $E = \int_\Omega \cos\theta \, \mathrm{d}\boldsymbol{\omega}_i = \pi$, hence $L_o = a(\boldsymbol{x})$: the albedo is the image of a surface under uniform unit illumination.

2.2 Light: What Arrives at Each Point *(slide 9)*

The incident radiance $L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)$ is light arriving at $\boldsymbol{x}$ from *every* direction of the hemisphere, and it differs at every point because of occlusion, nearby objects, etc.

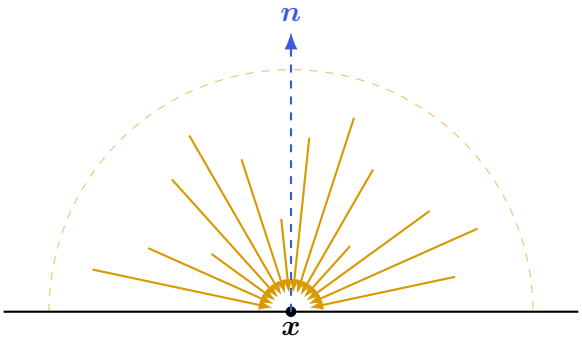


Figure 286: Incoming radiance $L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)$ at a surface point $\boldsymbol{x}$: one value per direction of the hemisphere (arrow length $\propto$ radiance). The normal $\boldsymbol{n}$ is dashed.

Intuition

Standing in a room, the light reaching your eye is not a single number: the window is bright, the wall opposite is dim, the ceiling in-between. $L_i(\boldsymbol{x}, \cdot)$ is a "panorama of brightness" seen from $\boldsymbol{x}$, and the surface multiplies it by the BRDF and sums it up. Standing under a table gives a different panorama than standing next to it.

3 Let's Learn Everything We Need *(slides 10–11)*

3.1 Learn Geometry, Material and Light with Networks *(slide 10)*

Same setup as NeRF: posed images under one unknown illumination. Each unknown is a network: an SDF network for geometry, a BRDF network for material, a light network for the light. A renderer integrates the rendering equation, and all three networks are trained jointly with a photometric loss.

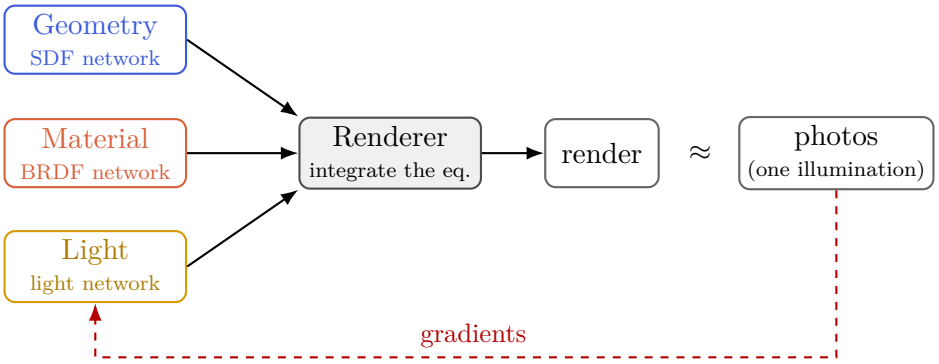


Figure 287: Inverse rendering by analysis-by-synthesis: three networks feed a differentiable renderer; the photometric error is back-propagated to all of them.

3.2 Rendering a Pixel *(slide 11)*

The integral in Definition 2.1 is evaluated by Monte Carlo sampling.

Proposition 3.1 (Monte Carlo estimator)**.** *Let* $\boldsymbol{\omega_i}^{(1)}, \dots, \boldsymbol{\omega_i}^{(N)} \sim p$ *be i.i.d. directions on* Ω *with density* $p > 0$*. Then*

$$\widehat{L}_o(\boldsymbol{x}, \boldsymbol{\omega}_o) = \frac{1}{N} \sum_{j=1}^{N} \frac{f_r(\boldsymbol{x}, \boldsymbol{\omega_i}^{(j)}, \boldsymbol{\omega}_o)\, L_i(\boldsymbol{x}, \boldsymbol{\omega_i}^{(j)})\, (\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega_i}^{(j)})}{p(\boldsymbol{\omega_i}^{(j)})}$$

is an unbiased estimator of $L_o(\boldsymbol{x}, \boldsymbol{\omega}_o)$*.*

Proof. By linearity of expectation, $\mathbb{E}[\widehat{L}_o] = \mathbb{E}_{\boldsymbol{\omega_i} \sim p}\left[\frac{h(\boldsymbol{\omega_i})}{p(\boldsymbol{\omega_i})}\right] = \int_\Omega \frac{h}{p}\, p \, \mathrm{d}\boldsymbol{\omega_i} = \int_\Omega h \, \mathrm{d}\boldsymbol{\omega_i} = L_o$, with $h = f_r L_i (\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega_i})$. □

Algorithm 3.1 (Rendering a pixel)**.**

1. Find the surface point $\boldsymbol{x}$ along the pixel ray (sphere tracing on the SDF).
2. Sample N incoming directions $\boldsymbol{\omega_i}^{(j)} \sim p$.
3. Query the BRDF network and the light network.
4. Average the weighted samples (Proposition 3.1), compare to the photo, back-propagate.

Intuition

Instead of summing over *all* directions (impossible), poll N random ones, ask "how much light comes from there and how much of it is sent to the camera?", and average. Dividing by p corrects for having sampled some directions more often than others, so the answer is right on average.

4 Does It Work? Why Not? *(slides 12–17)*

4.1 Does It Work? *(slide 12)*

On a synthetic cow, the fit (free per-pixel albedo plus a free environment map, fitted to one photo) reproduces the training view with error below 1/255. But under a *moving light* the fit is wrong: shading has been painted into the albedo.



Figure 288: Photo (one illumination), fit, and moving light: truth vs. fit. Take the four cow images of slide 12. Figure credits: S. Tulsiani.

Example 4.1 (What "error $< 1/255$" means)**.** For pixel values in $[0, 1]$, a maximum error of $1/255$ gives MSE $\leq (1/255)^2$ and PSNR $= 10 \log_{10}(1/\text{MSE}) \geq 20 \log_{10} 255 \approx 48.1$ dB: the training view is reproduced essentially perfectly, while the relit result is still wrong. Training error therefore says nothing about relighting quality.

Proposition 4.2 (Trivial explanation: shading painted on, flat light)**.** *Consider a Lambertian scene and an observed image $I(\boldsymbol{x})$ that is independent of the viewing direction. Take albedo $a'(\boldsymbol{x}) = I(\boldsymbol{x})$ and a uniform environment $L_e \equiv 1$. Then the model reproduces every training image exactly.*

Proof. By the Lambertian example, under $L_e \equiv 1$ the rendered radiance equals the albedo, $L_o = a'(\boldsymbol{x}) = I(\boldsymbol{x})$. □

4.2 Why? What One Capture Observes *(slide 13)*

Each surface point is seen under **one** light, from a *few* directions. The unknowns are a full BRDF **and** all the incoming light.

Proposition 4.3 (Counting unknowns)**.** *At a surface point with K views under one light, the data are K colour values, whereas the unknowns include a BRDF (a function of 4 angular variables) and an incident-light function (2 angular variables). The unknowns outnumber the data by an unbounded factor, so the inverse problem is under-determined.*

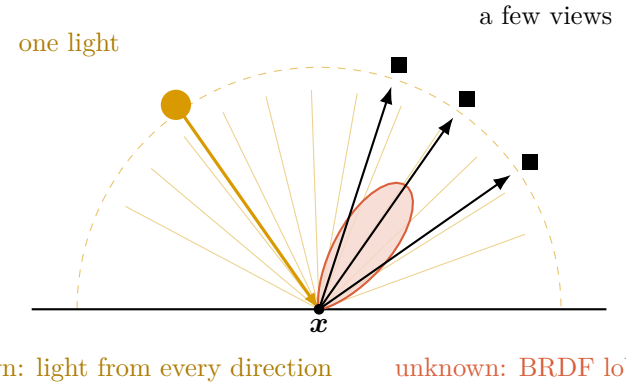


Figure 289: What a single capture observes at $\boldsymbol{x}$: one light direction (thick yellow), a handful of view directions (black squares). The thin yellow rays and the orange lobe remain unconstrained.

4.3 Many Explanations, Same Pixels *(slides 14–16)*

Proposition 4.4 (Factorisation ambiguity)**.** *For a Lambertian pixel $I = a \odot s$ (albedo a, shading/light term s, both in $\mathbb{R}^3_{>0}$) and any $c \in \mathbb{R}^3_{>0}$, the pair $(a \oslash c,\ c \odot s)$ produces the same pixel.*

Proof. $(a \oslash c) \odot (c \odot s) = a \odot s = I$. □

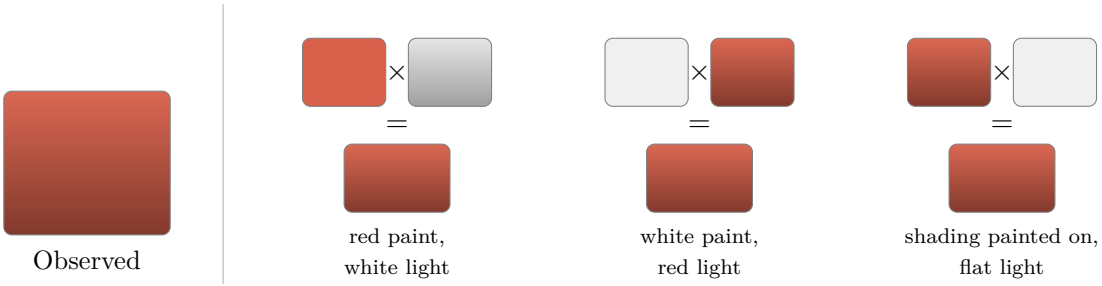


Figure 290: Same pixels, many (material, light) explanations. Left: the observed shaded red object. Right: three different factorisations (albedo × light) producing it.

Example 4.5 (Three explanations, one pixel)**.** Let the observed RGB pixel be (0.80, 0.35, 0.30). The following all give it, channel by channel:

explanation	albedo a	light term s
red paint, white light	$(0.80, 0.35, 0.30)$	$(1, 1, 1)$
white paint, red light	$(1, 1, 1)$	$(0.80, 0.35, 0.30)$
shading painted on, flat light	per-pixel $= I$	$(1, 1, 1)$ (flat)

Only the first one relights correctly if the light is turned white→blue.

Intuition

It is the same puzzle as "is the car red, or is it white under a red light?". A photo only records the *product* albedo × light. Like trying to recover the two factors of 12 from the number 12 alone: 3×4, 2×6 and 12×1 are all fine, and nothing in the product tells you which one is true.

4.4 It Happens on Real Objects *(slide 17)*

On Stanford-ORB with the *same* method (nvdiffrecmc) and similar recovered shapes, the relighting PSNR of two objects is 24.4 dB versus 7.5 dB.

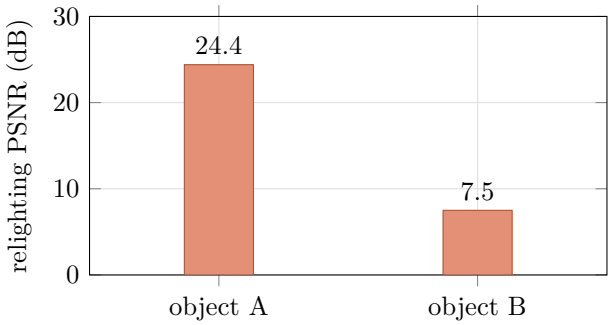


Figure 291: Same method, similar shapes, very different relighting quality (values from the slide; the two objects are those of the Stanford-ORB figure).

[Insert image from slide 17]

Figure 292: Baking and Salt objects: ground truth and prediction, novel view and relighting. Take the full 8-image grid of slide 17. Figure credits: Kuang et al., NeurIPS 2023 (Stanford-ORB).

Example 4.6 (17 dB is a lot)**.** Since $\text{PSNR} = -10 \log_{10} \text{MSE}$, a gap of $24.4 - 7.5 = 16.9$ dB means a mean-squared-error ratio of $10^{1.69} \approx 49$: the relit failure is about fifty times worse in squared error, with no sign of it in the training views.

5 How Do We Resolve the Ambiguity? *(slide 18)*

Three levers, each acting on a different part of the inverse-rendering loop:

1. **Constrain the model** (on the unknowns).
2. **Capture more (informative) data** (on the photos).
3. **Learn priors from data** (a learned prior over the unknowns).

6 Option 1: Constrain the Model *(slides 19–34)*

6.1 Constrain the Light *(slide 20)*

Definition 6.1 (Distant (environment) light)**.** Light is *far away* if it depends only on direction, not on position:

$$L_i(\boldsymbol{x}, \boldsymbol{\omega}) \approx L_e(\boldsymbol{\omega}) \qquad \forall \boldsymbol{x},$$

where $L_e : \mathbb{S} \to \mathbb{R}^3_{\geq 0}$ is an *environment map*, one per scene. Often it is additionally taken to have *no colour* (a gray environment map, as in NeRS).

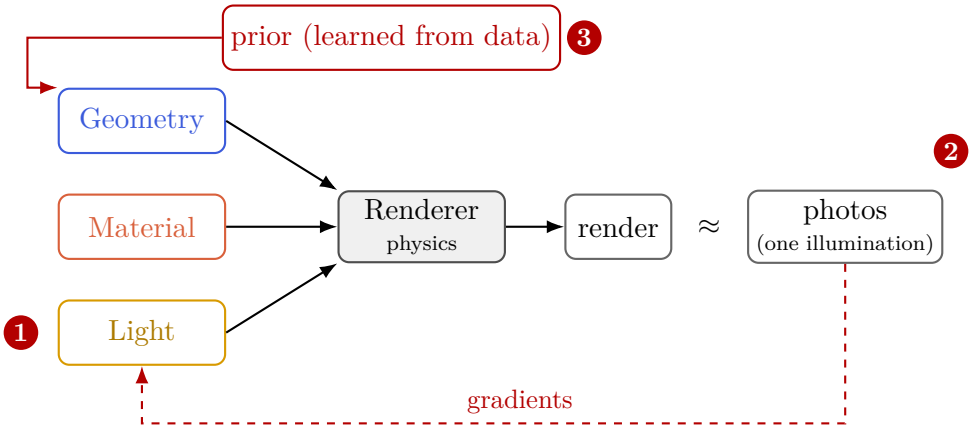


Figure 293: The three strategies: **1** constrain what the unknowns can be, **2** capture photos that carry more information, **3** inject a prior learned from data.

A gray environment map removes the colour ambiguity that the NeRS paper states as "to avoid the fundamental ambiguity (a red car vs. a white car under red light)": it rules out explanation 2 of the previous section (white paint, red light).

Proposition 6.2. *Under a colourless light* $L_e = \ell(\boldsymbol{\omega})\,(1,1,1)^\mathsf{T}$*, the factorisation of Proposition 4.4 is restricted to scalar* $c \in \mathbb{R}_{>0}$*, so the chromatic part of the pixel colour is unambiguously assigned to albedo.*

Proof. Colourless light means $s = \sigma\,(1,1,1)^\mathsf{T}$ for a scalar σ; the transformed pair $c \odot s$ stays colourless only if c has equal entries. The ratios between colour channels of $a \odot s$ are then those of a. □

[Insert image from slide 20]

Figure 294: Twelve panoramic environment maps (one per scene). Take the 4×3 grid of slide 20. Figure credits: Zhang et al., NeurIPS 2021 (NeRS); Poly Haven (env maps).

6.2 Constrain the Material *(slide 21)*

Definition 6.3 (Diffuse + specular BRDF)**.**

$$f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = \underbrace{\frac{a(\boldsymbol{x})}{\pi}}_{\text{diffuse: varies over the surface}} + \underbrace{f_s(\boldsymbol{\omega}_i, \boldsymbol{\omega}_o; \rho)}_{\text{specular: few params, often shared}},$$

with ρ the specular roughness (shiny → rough, see Figure 285).

The albedo (texture) still varies from point to point, but the specular part is often a single material for the whole object. In parameter count, a free BRDF per point has four angular variables of freedom; the constrained model has one albedo (3 numbers) per point plus one shared roughness.

Intuition

Most real objects are "a coloured paint plus one kind of gloss". By committing to this, we replace an infinite-dimensional unknown at every point by a handful of numbers, which the few available views can actually pin down.

6.3 Constraints Help Even with Sparse Views *(slides 22–23)*

A constrained BRDF generalises; free-form appearance overfits. In NeRS (cars from a few images) the constrained model gives a clean result, while a version with a free radiance field, IDR and NeRF degrade.

[Insert image from slide 22]

Figure 295: Sparse-view reconstruction: input image, NeRS, NeRS with radiance field, IDR, NeRF (two cars). Take the image grid of slide 22. Video credits: Zhang et al., NeurIPS 2021 (NeRS).

[Insert image from slide 23]

Figure 296: NeRS: photo, gray-albedo estimated light, and estimated environment map (three cars). Take the 3×3 grid of slide 23. Figure credits: Zhang et al., NeurIPS 2021 (NeRS), Fig. 10.

6.4 PhySG: The Full Pipeline *(slide 24)*

PhySG instantiates all of the above in one model:

Geometry	SDF network, sphere tracing (as in lecture 09)
Material	albedo network + **one** shared specular lobe
Light	env map = 128 spherical Gaussians
Integral	closed form (below); no shadows

6.5 Spherical Gaussians Make the Integral Closed-Form *(slide 25)*

Definition 6.4 (Spherical Gaussian)**.** A *spherical Gaussian* (SG) with axis $\boldsymbol{\xi} \in \mathbb{S}$, sharpness $\lambda > 0$ and amplitude $\boldsymbol{\mu} \in \mathbb{R}^3_{\geq 0}$ is

$$G(\boldsymbol{\omega}) = \boldsymbol{\mu}\, e^{\lambda(\boldsymbol{\xi}^\top \boldsymbol{\omega} - 1)}.$$

An environment map is approximated by a sum of SG lobes.

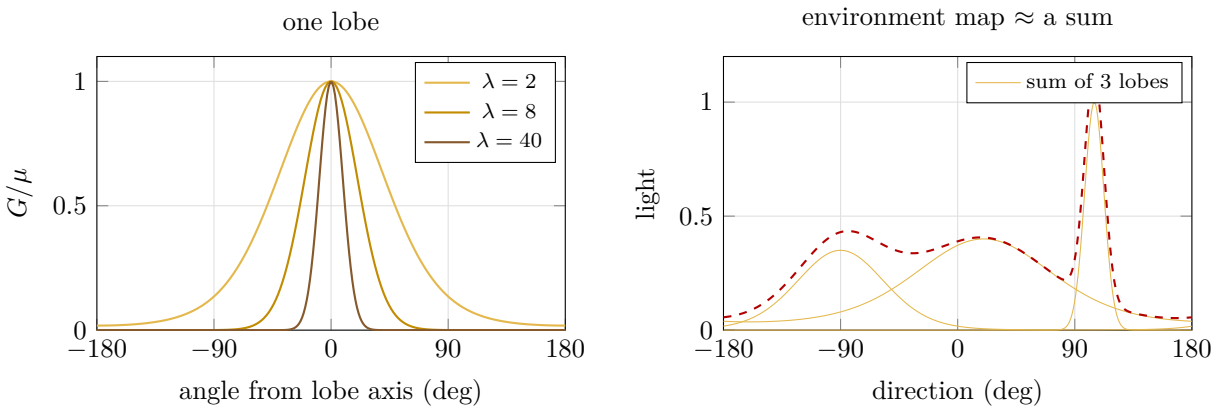


Figure 297: Left: SG lobes for increasing sharpness λ (narrower lobe). Right: three SG lobes (thin) add up to a smooth-plus-peaked environment (dashed).

Lemma 6.5 (Product of two SGs is an SG)**.** *For $G_k(\boldsymbol{\omega}) = \boldsymbol{\mu}_k e^{\lambda_k(\boldsymbol{\xi}_k^{\mathsf{T}}\boldsymbol{\omega}-1)}$, $k = 1, 2$, one has $G_1 G_2 = G_3$ with*

$$\lambda_3 = \|\lambda_1\boldsymbol{\xi}_1 + \lambda_2\boldsymbol{\xi}_2\|, \quad \boldsymbol{\xi}_3 = \frac{\lambda_1\boldsymbol{\xi}_1 + \lambda_2\boldsymbol{\xi}_2}{\lambda_3}, \quad \boldsymbol{\mu}_3 = \boldsymbol{\mu}_1 \odot \boldsymbol{\mu}_2\, e^{\lambda_3 - \lambda_1 - \lambda_2}.$$

Proof. The exponent of the product is $\lambda_1(\boldsymbol{\xi}_1^{\mathsf{T}}\boldsymbol{\omega} - 1) + \lambda_2(\boldsymbol{\xi}_2^{\mathsf{T}}\boldsymbol{\omega} - 1) = (\lambda_1\boldsymbol{\xi}_1 + \lambda_2\boldsymbol{\xi}_2)^{\mathsf{T}}\boldsymbol{\omega} - \lambda_1 - \lambda_2 = \lambda_3\boldsymbol{\xi}_3^{\mathsf{T}}\boldsymbol{\omega} - \lambda_1 - \lambda_2 = \lambda_3(\boldsymbol{\xi}_3^{\mathsf{T}}\boldsymbol{\omega} - 1) + (\lambda_3 - \lambda_1 - \lambda_2)$. □

Lemma 6.6 (Integral of an SG)**.** $\displaystyle\int_{\mathbb{S}} G(\boldsymbol{\omega})\,\mathrm{d}\boldsymbol{\omega} = \boldsymbol{\mu}\,\frac{2\pi}{\lambda}\big(1 - e^{-2\lambda}\big)$.

Proof. In spherical coordinates about $\boldsymbol{\xi}$ with $t = \cos\theta$: $\int_0^{2\pi}\int_{-1}^{1} \boldsymbol{\mu}\, e^{\lambda(t-1)}\,\mathrm{d}t\,\mathrm{d}\phi = 2\pi\boldsymbol{\mu}\, e^{-\lambda}\frac{e^{\lambda} - e^{-\lambda}}{\lambda}$. □

Theorem 6.7 (Closed-form shading with SGs)**.** *Let the light be $L_e = \sum_{k=1}^{K} G_k$, let the specular lobe f_s be an SG in $\boldsymbol{\omega}$ (at fixed $\boldsymbol{\omega}_o$ and $\boldsymbol{n}$) and let the clamped cosine be approximated by an SG G_c. Then*

$$\int_{\Omega} f_r\, L_e\, (\boldsymbol{n}^{\mathsf{T}}\boldsymbol{\omega}_i)\,\mathrm{d}\boldsymbol{\omega}_i \;\approx\; \sum_{k=1}^{K}\int_{\mathbb{S}} G_k\, G_s\, G_c\,\mathrm{d}\boldsymbol{\omega} = \sum_{k=1}^{K} \boldsymbol{\mu}'_k\,\frac{2\pi}{\lambda'_k}\big(1 - e^{-2\lambda'_k}\big),$$

where $(\boldsymbol{\mu}'_k, \lambda'_k)$ are obtained by applying Lemma 6.5 twice. The cost per surface point is $O(K)$, with no sampling.

Proof. Linearity in L_e turns the integral into a sum over lobes; each term is an integral of a product of three SGs, which is a single SG by Lemma 6.5 (applied twice), integrated by Lemma 6.6. □

Intuition

Gaussians are closed under multiplication — the reason every Gaussian-based tool (Kalman filters, Gaussian processes) is so convenient. Here, "light × shiny lobe × cosine" multiplies into just one more lobe, whose total energy we know by formula. So instead of sampling hundreds of directions, PhySG evaluates $K = 128$ short formulas.
Worked numbers: a lobe with $\lambda = 2$ integrates to $\pi(1 - e^{-4}) \approx 3.08\,\mu$, $\lambda = 8$ to $\approx 0.79\,\mu$, $\lambda = 40$ to $\approx 0.157\,\mu$. A lobe covers roughly $2\pi/\lambda$ steradians of the 4π sphere — the sharper, the smaller.

6.6 PhySG: Results *(slide 26)*

[Insert image from slide 26]

Figure 298: PhySG: novel views, diffuse-only edit, relit under light 1 and light 2 (fish figurine). Take the four images of slide 26. Video credits: Zhang et al., CVPR 2021 (PhySG).

6.7 What PhySG Ignores: Shadows & Interreflections *(slide 27)*

A shadow means less light *arriving* at $\boldsymbol{x}$. PhySG's integral has no visibility term, so the only way to explain the darkness of a shadow is a darker albedo. The same holds for interreflections. The fix is shadow rays (next).

Proposition 6.8 (Shadows are absorbed by albedo)**.** *Let the true model be $L_o = \frac{a}{\pi} V\, E$ with visibility $V \in [0,1]$ and unshadowed irradiance E. A model that omits V fits the same pixel with albedo $a' = aV$.*

Proof. $\frac{a'}{\pi} E = \frac{aV}{\pi} E = L_o$. □

Example 6.9. True albedo $a = 0.8$ and a point in the shadow of an occluder blocking 75% of the light: $V = 0.25$. The model without visibility reproduces the photo with $a' = 0.2$. Under a new light that comes from the other side, this point would be rendered much too dark.

[Insert image from slide 27]

Figure 299: Direct vs. indirect illumination, and albedo/roughness with and without modelling indirect illumination (hotdog scene). Take the full figure of slide 27. Figure credits: Zhang et al., CVPR 2022 (InvRender); Hasselgren et al., NeurIPS 2022.

6.8 Recall: Gaussians Store Baked Color *(slide 28)*

In 3D Gaussian Splatting a pixel is $\sum_i w_i\, \boldsymbol{c}_i(\boldsymbol{\omega})$. Each Gaussian stores *outgoing* light, baked in like NeRF: same problem, colour is light leaving the point.

6.9 Relightable Gaussians (IRGS): Store Material *(slide 29)*

IRGS uses 2D Gaussians (flat disks), so each disk has a well-defined normal. Each Gaussian carries a **normal**, an **albedo** and a **roughness**; the light is one environment map, as before.

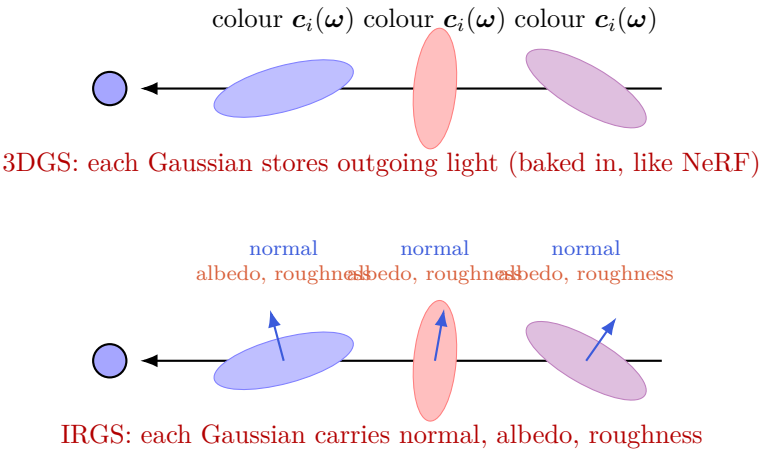


Figure 300: Top: 3DGS Gaussians store a baked colour. Bottom: IRGS Gaussians are disks with a well-defined normal and store material properties instead.

6.10 Blend Attributes, Then Shade *(slide 30)*

Definition 6.10 (Per-pixel attribute blending)**.** With front-to-back splatting weights w_i for the Gaussians covering a pixel, the pixel receives

$$\bar{\boldsymbol{n}} = \sum_i w_i \, \boldsymbol{n}_i, \qquad \bar{a} = \sum_i w_i \, a_i, \qquad \bar{r} = \sum_i w_i \, r_i, \qquad \bar{d} = \sum_i w_i \, d_i,$$

the blended normal (renormalised), albedo, roughness and depth d_i. The surface point is $\boldsymbol{x} = \boldsymbol{o} + \bar{d}\,\boldsymbol{\omega}$ for camera centre $\boldsymbol{o}$ and pixel ray direction $\boldsymbol{\omega}$, and the pixel colour is $\text{shade}(\boldsymbol{x}, \bar{\boldsymbol{n}}, \bar{a}, \bar{r}; \text{light})$.

Intuition

Shading is expensive (hundreds of rays), so we do it *once per pixel*, not once per Gaussian. First blend the "G-buffer" (normal, albedo, roughness, depth) exactly like a deferred renderer, then light the resulting surface point.

[Insert image from slide 30]

Figure 301: Blended per-pixel attributes of the cow: normal, albedo, roughness, depth $\to \boldsymbol{x}$, and the shaded result. Take the five cow images of slide 30. Figure credits: Gu et al., CVPR 2025 (IRGS).

6.11 What About Shadows? *(slide 31)*

From $\boldsymbol{x}$ (at the blended depth), trace through the Gaussians: the accumulated opacity gives the visibility $V(\boldsymbol{x}, \boldsymbol{\omega})$; and IRGS also gets **indirect light**: the colour of whatever the ray hits.

Lemma 6.11 (Visibility from opacity)**.** *Along a ray crossing Gaussians with opacities $\alpha_1, \ldots, \alpha_n$ (front to back), set $T_i = \prod_{j<i}(1-\alpha_j)$ and $w_i = \alpha_i T_i$. Then $\sum_{i=1}^{n} w_i = 1 - \prod_{i=1}^{n}(1-\alpha_i)$, so the visibility V (the fraction of light that is* not *absorbed) satisfies $V = 1 - \sum_i w_i = \prod_i (1-\alpha_i)$.*

Proof. Induction on n. For $n = 1$: $w_1 = \alpha_1 = 1 - (1 - \alpha_1)$. If it holds for n, then $\sum_{i \le n+1} w_i = 1 - T_{n+1} + \alpha_{n+1} T_{n+1} = 1 - T_{n+1}(1 - \alpha_{n+1}) = 1 - T_{n+2}$. □

Example 6.12. Two Gaussians with $\alpha = (0.5, 0.4)$: $w = (0.5,\ 0.4 \cdot 0.5 = 0.2)$, $\sum w = 0.7$, and $V = 0.5 \cdot 0.6 = 0.3 = 1 - 0.7$. Only 30% of the light gets through.

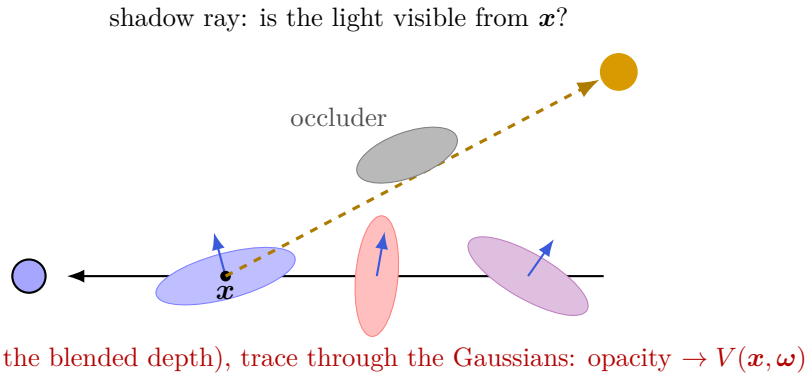


Figure 302: Shadow ray from the blended surface point $\boldsymbol{x}$. If it crosses other Gaussians, their accumulated opacity reduces the visibility V; the colour of what it hits gives indirect light.

6.12 Putting Shadows in the Rendering Equation *(slide 32)*

Proposition 6.13 (IRGS shading estimator)**.** *With N sampled directions $\boldsymbol{\omega}_j \sim p$,*

$$L_o \approx \frac{1}{N} \sum_j f_r(\boldsymbol{\omega}_j, \boldsymbol{\omega}_o) \Big[\underbrace{V(\boldsymbol{x}, \boldsymbol{\omega}_j)}_{\text{traced opacity}}\ L_e(\boldsymbol{\omega}_j) + \underbrace{L_{\text{ind}}(\boldsymbol{x}, \boldsymbol{\omega}_j)}_{\text{traced colour}} \Big] \frac{\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega}_j}{p(\boldsymbol{\omega}_j)}.$$

Opacity gives visibility V; colour gives indirect light (the Gaussians' own colour). The cost is 256 rays per pixel, on a random subset of pixels only.

Intuition

The environment light L_e is now only counted when the ray escapes (V close to 1); when the ray hits something, we use that something's colour instead — a one-bounce approximation of global illumination, implemented by the very same Gaussians. Tracing rays on just a random subset of pixels per step keeps training affordable.

Algorithm 6.1 (IRGS pixel rendering)**.**

1. Splat Gaussians: obtain blended $\bar{\boldsymbol{n}}, \bar{a}, \bar{r}, \bar{d}$ and weights w_i.
2. Compute $\boldsymbol{x} = \boldsymbol{o} + \bar{d}\boldsymbol{\omega}$.
3. Sample directions $\boldsymbol{\omega}_j$ (256 per pixel on a random pixel subset), trace each through the Gaussians to get $V(\boldsymbol{x}, \boldsymbol{\omega}_j)$ and $L_{\text{ind}}(\boldsymbol{x}, \boldsymbol{\omega}_j)$.
4. Evaluate the BRDF from $(\bar{a}, \bar{r})$ and sum as above; compare with the photo and back-propagate.

[Insert image from slide 33]

Figure 303: Global, direct and indirect illumination: the two terms of the equation rendered separately (toy scene). Take the three images of slide 33. Video credits: Gu et al., CVPR 2025 (IRGS).

[Insert image from slide 34]

Figure 304: IRGS materials and relighting: albedo, roughness, normal and five environment maps for four scenes. Take the 4×8 grid of slide 34. Figure credits: Gu et al., CVPR 2025 (IRGS).

6.13 IRGS: Results *(slides 33–34)*

7 Option 2: Capture More Informative Data *(slides 35–41)*

7.1 Capture: One Light at a Time (OLAT) *(slide 36)*

A *light stage* moves a light over the sphere of directions and takes one photo per light direction (about 2048 directions in roughly a minute). Instead of one illumination, we observe each point under thousands.

[Insert image from slide 36]

Figure 305: Light stage capture: grid of OLAT photos and the lighting environment. Take the video frame of slide 36. Video credits: Debevec et al., SIGGRAPH 2000 (Light Stage).

7.2 Light Adds Up *(slides 37–38)*

Theorem 7.1 (Superposition of light)**.** *For a fixed camera and scene, and linear (HDR) pixel values, let I_A and I_B be the images under lights A and B and I_{A+B} the image with both on. Then $I_{A+B} = I_A + I_B$, and more generally $I(c_A A + c_B B) = c_A I_A + c_B I_B$.*

Proof. The full light-transport operator (including shadows, interreflections and any number of bounces) is linear in the emitted radiance of the sources, because photons do not interact with each other: each is propagated independently and the contributions add. □

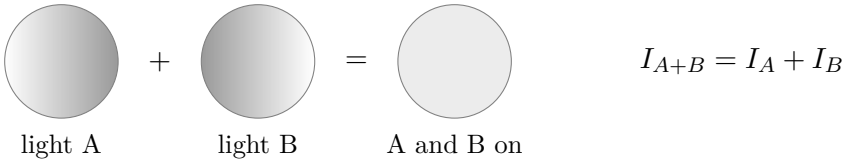


Figure 306: Light adds up: the image under two lights at once is the sum of the two single-light images (schematic sphere lit from the left, from the right, and from both).

[Insert image from slide 38]

Figure 307: Real photos of a face under two lights and their sum. Take the video frame of slide 38. Video credits: Debevec et al., SIGGRAPH 2000 (Light Stage).

7.3 Light Transport Is Linear *(slide 39)*

Definition 7.2 (Light-transport matrix)**.** Discretise the light into K single-direction lights with intensities ℓ_k and let $I_k \in \mathbb{R}^P$ be the (vectorised) image under light k alone. The *light-transport matrix* is $\boldsymbol{T} = [I_1 \,|\, \cdots \,|\, I_K] \in \mathbb{R}^{P \times K}$ and the image under lighting $\boldsymbol{\ell}$ is

$$I = \sum_k \ell_k I_k = \boldsymbol{T}\boldsymbol{\ell}.$$

$\boldsymbol{T}$ already holds material, geometry and shadows. The assumption: fixed camera and scene, linear HDR pixels.

7.4 Relighting = a Weighted Sum *(slides 40–41)*

Algorithm 7.1 (Relighting from OLAT images)**.** 1. Capture $\{I_k\}_{k=1}^K$, one per light direction $\boldsymbol{\omega}_k$.
2. For a new environment map L_e^{new}, set $\ell_k = L_e^{\text{new}}(\boldsymbol{\omega}_k)\,\Delta\boldsymbol{\omega}_k$ ($\Delta\boldsymbol{\omega}_k$ the solid angle of direction k).
3. Output $I_{\text{new}} = \sum_k \ell_k I_k$.

No BRDF anywhere, and no geometry: the result is just a sum.

Example 7.3. Take a pixel whose OLAT values at three lights are (0.80, 0.30, 0.50) and a new light with weights (0.06, 0.90, 0.03). Then $I_{\text{new}} = 0.048 + 0.270 + 0.015 = 0.333$. With $K = 2048$ images of

one megapixel in RGB, the table $\boldsymbol{T}$ holds about $2048 \cdot 3 \cdot 10^6 \approx 6.1 \cdot 10^9$ numbers.

Intuition

Imagine a dark room with 2048 tiny lamps, each lit alone for one photo. Any new lighting is a mixing board: "a bit of lamp 17, a lot of lamp 340, ..." and the final photo is the same mix of the stored photos. You never needed to understand the object; the light stage did the physics for you. The price is the rig itself, and a static subject.

[Insert image from slide 41]

Figure 308: Relighting with the light stage: weights from a captured environment map and the relit faces. Take the video frame and the strip of five faces on slide 41. Video credits: Debevec et al., SIGGRAPH 2000 (Light Stage).

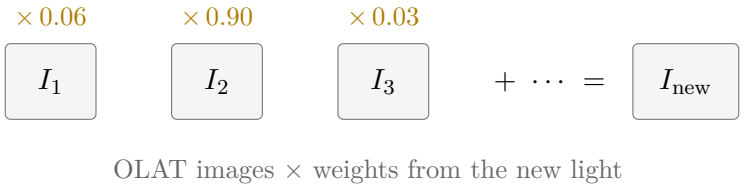


Figure 309: Relighting as a weighted sum of OLAT images (weights are those of slide 40, six out of 64 lights shown there).

8 Option 3: Learn Priors from Data *(slides 42–47)*

8.1 MaterialFusion: Prior Inside the Optimization *(slide 43)*

Keep the physical model and per-scene optimisation of Option 1, but add a learned prior on the *materials*: a material diffusion model scores how plausible the rendered albedo and roughness maps are.

Definition 8.1 (MaterialFusion objective)**.** Given posed photos under one illumination, fit mesh, materials and environment map θ by

$$\min_{\theta} \; \mathcal{L}_{\text{photo}}(\theta) + \gamma \, \mathcal{L}_{\text{prior}}\big(\text{albedo}_\theta,\ \text{roughness}_\theta\big),$$

where $\mathcal{L}_{\text{photo}}$ compares renders and photos, and $\mathcal{L}_{\text{prior}}$ is a loss derived from the material diffusion model (low when the rendered albedo and roughness look like plausible materials); $\gamma > 0$ sets the strength of the prior.

Algorithm 8.1 (MaterialFusion)**.**
1. Input: posed photos, one illumination.
2. Fit mesh, materials and environment map to the photos (photometric loss).
3. Render albedo and roughness; the material diffusion model scores their plausibility → prior loss.
4. Back-propagate the sum into θ.

Intuition

The photo loss alone accepts any explanation that matches the pixels (the previous section's puzzle); the prior is a friend who has seen thousands of real materials and says "a can of baking powder doesn't have a glossy sheet of random shading painted on its albedo". In probabilistic language the prior plays the role of $-\log p(\text{materials})$ added to the data term. Priors and per-scene optimisation are not exclusive: you get both.

[Insert image from slide 43]

Figure 310: MaterialFusion pipeline on a cow: posed photos, fitted rendering, rendered albedo and roughness. Take the images of slide 43. Figure credits: Litman et al., 3DV 2025 (MaterialFusion); S. Tulsiani.

[Insert image from slide 44]

Figure 311: Relighting a real can: reference, MaterialFusion, nvdiffrecmc (same optimisation without the prior). Take the video frame of slide 44. Video credits: Litman et al., 3DV 2025 (MaterialFusion).

8.2 Neural Gaffer: Learn to Relight Images Directly *(slide 45)*

Neural Gaffer skips the explicit BRDF and light estimate entirely: one image plus a target environment map goes in, a relit image comes out. A diffusion model learns the mapping.

[Insert image from slide 45]

Figure 312: Neural Gaffer: input images and relit results under environment maps and text-conditioned lights. Take the full grid of slide 45. Video credits: Jin et al., NeurIPS 2024 (Neural Gaffer).

8.3 How Do You Tell a Network the Light? *(slide 46)*

The environment map is converted into two images (an LDR tone-mapped copy and a log-normalised copy), both *rotated into the camera frame*, VAE-encoded and concatenated with the encoded input image; the denoising U-Net (Zero123 weights, fine-tuned) produces the relit image.

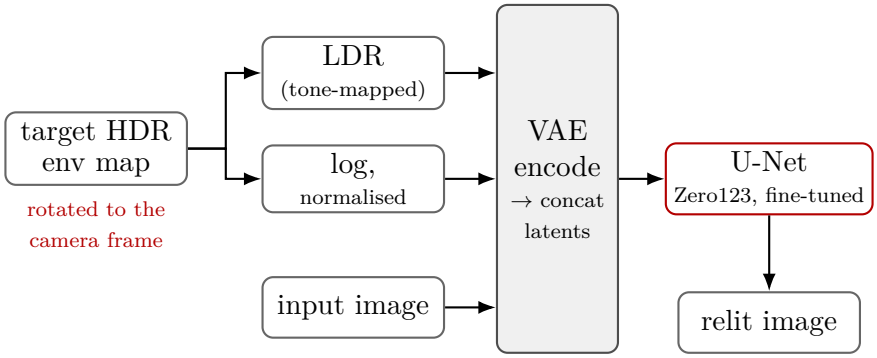


Figure 313: Neural Gaffer conditioning: each env-map pixel is made to mean a fixed direction in the camera frame.

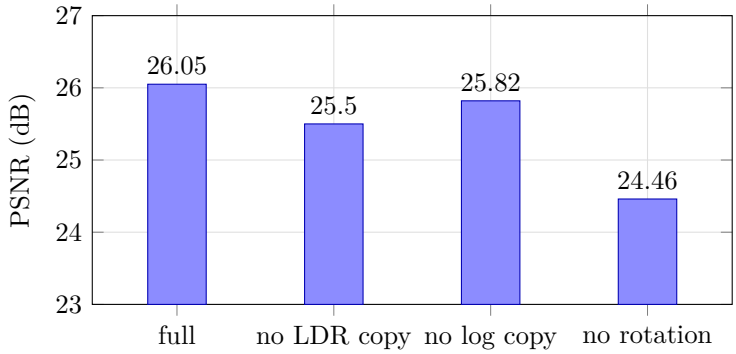


Figure 314: Neural Gaffer ablation (PSNR, values from the slide; the vertical axis starts at 23 dB). Removing the rotation into the camera frame hurts most (−1.59 dB).

Intuition

For a network, "the light" must be a fixed code. If the same env-map pixel meant "north" in one example and "to the camera's left" in another, the network could not learn a consistent rule; expressing the map in the camera frame gives every pixel a stable meaning, which is why dropping the rotation costs the most. The two copies help for a different reason: the LDR copy keeps the visible look, the log copy keeps the dynamic range of very bright lamps.

8.4 Neural Gaffer: Data, 3D and Limits *(slide 47)*

- **Data:** about 90K objects and 18.4M renders, i.e. roughly 18.4M/90K ≈ 200 renders per object (many views × many lights).
- **3D:** relight the views, then refit the NeRF (once per target light).
- **Limits:** only 256^2 images; and a yellow object or a white one under yellow light? For diffuse objects the ambiguity remains.

9 Summary *(slide 48)*

One illumination → many (material, light) explanations. The three remedies: **1** constrain the model (distant light, few-parameter BRDF, traced visibility); **2** capture more (light is linear → relight by summing OLAT images); **3** priors (learn plausible materials, or learn to relight directly).

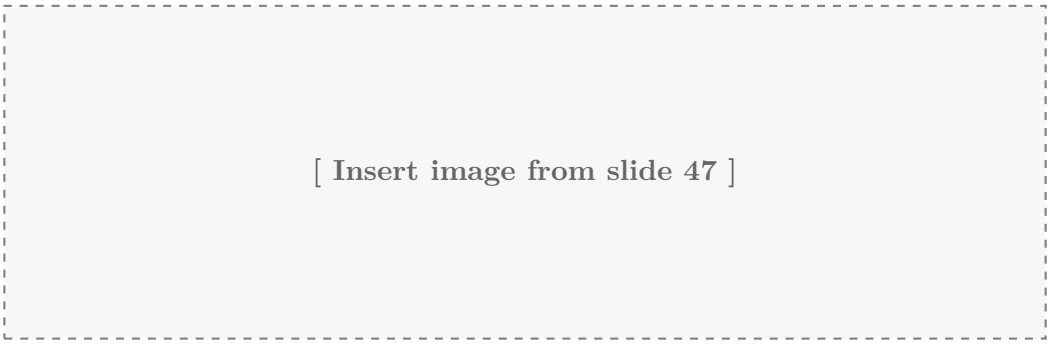


Figure 315: 3D relighting comparison: input views, nvdiffrecmc, TensoIR, Neural Gaffer (ours), ground truth and target lighting (three objects). Take the full grid of slide 47. Figure credits: Jin et al., NeurIPS 2024 (Neural Gaffer).

Method	**Option**	**Light / material model**	**Shadows, indirect**	**Needs**	**Main limitation**
NeRF, 3DGS, IDR	–	$g_\phi(\boldsymbol{x}, \boldsymbol{\omega}) = \boldsymbol{c}$, colour = emitted light	–	posed images	cannot be relit: light baked in
NeRS	1	gray env map; constrained BRDF	not covered	few images of an object	light and BRDF kept simple
PhySG	1	128-SG env map; albedo net + one shared specular lobe; SDF geometry	none (no visibility term)	posed images, one light	shadows end up in the albedo
IRGS	1	env map; per-Gaussian normal, albedo, roughness	shadow rays through Gaussians; indirect colour	posed images; 256 rays/pixel on random pixel subset	ray-tracing cost
Light stage (OLAT)	2	none: $I = \boldsymbol{T}\boldsymbol{\ell}$	already in $\boldsymbol{T}$	light-stage rig, fixed camera and scene, linear HDR	special capture setup
MaterialFusion	3	physical model + diffusion prior on albedo and roughness	via the physical renderer	posed photos, one light; per-scene optimisation	per-scene optimisation
Neural Gaffer	3	none explicit: diffusion model (image + env map $\rightarrow$ relit image)	implicit	one image + target env map; trained on $\sim$90K objects, 18.4M renders	only 256^2; diffuse colour/light ambiguity

Table 16: Comparison of the methods covered in this lecture.

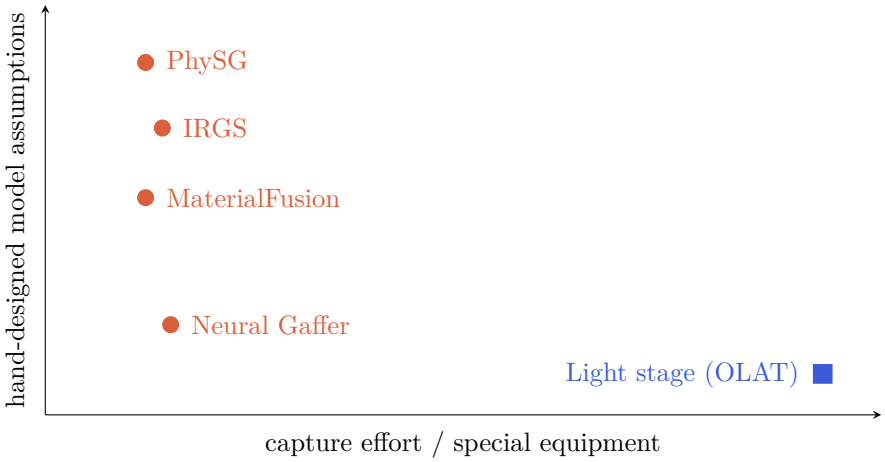


Figure 316: Schematic (qualitative, not measured) positioning of the methods: the light stage shifts the burden to the capture hardware and needs almost no model assumptions; Option 1 methods need only ordinary photos but rely on strong hand-designed assumptions; priors (Option 3) trade assumptions for learned knowledge, at the price of training data.

Key take-aways

1. Radiance fields store *emitted* colour, so the capture lighting is baked in. Relighting needs the separate ingredients of the rendering equation: geometry, material (BRDF) and light.
2. Fitting them from photos under one illumination is under-determined: a pixel only records the *product* of albedo and light, so many (material, light) explanations fit the training views perfectly yet fail under a new light.
3. **Constrain the model:** a distant, often colourless light, a few-parameter BRDF with a shared specular lobe, and (PhySG) spherical Gaussians for a closed-form integral; adding traced visibility and indirect light (IRGS) stops shadows from leaking into the albedo.
4. **Capture more:** since light transport is linear, OLAT images relight by a weighted sum with no BRDF and no geometry, but it requires a special capture setup.
5. **Learn priors:** either inside the optimisation (MaterialFusion's plausibility loss on albedo and roughness) or by learning to relight directly (Neural Gaffer), which is still limited by resolution and by the residual diffuse colour/light ambiguity.

Acknowledgments. The lecture is adapted from *Learning for 3D Vision*, CMU 16-825 (Prof. Shubham Tulsiani). Cow illustrations by S. Tulsiani; light-stage material from Debevec et al., SIGGRAPH 2000; results and diagrams from Mildenhall et al. (NeRF), Yariv et al. (IDR), Zhang et al. (PhySG, NeRS, InvRender), Kuang et al. (Stanford-ORB), Wang et al. (SIGGRAPH Asia 2009), Hasselgren et al. (nvdiffrecmc), Gu et al. (IRGS), Litman et al. (MaterialFusion), Jin et al. (Neural Gaffer); environment maps from Poly Haven.

Part XI

Lecture 12

Abstract

These lecture notes provide a rigorous and self-contained mathematical and algorithmic treatment of **Lecture 12: Gaussian-Based Active Planning** from the *Foundations of Spatial AI (2026/27)* course. We trace the transition from passive scene reconstruction to active perception and Next-Best-Pose planning, examine how **3D Gaussian Splatting (3DGS)** acts as a fast differentiable rendering engine for scoring candidate camera trajectories, analyze coverage-driven planning with **MAGICIAN**, and study a complete online semantic object-mapping architecture combining a 5 cm voxel planning state with multi-channel Gaussian signal rendering and horizon-5 beam search.

1 Active Perception and Active Mapping *(slides 1–4)*

1.1 What Is Active Perception? *(slides 1–2)*

Classical computer vision and spatial mapping pipelines often operate in a **passive** regime: a fixed sequence of sensor measurements is handed to the algorithm, which processes whatever arrives. In robotics and embodied AI, however, an agent controls its own motion and camera pose. This leads to the paradigm of **Active Perception**:

> *"What we perceive depends on what we decide to do."*

Definition 1.1 (Passive vs. Active Perception)**.** Let $\mathcal{X}$ denote the space of sensor observations (e.g., RGB-D frames), $\mathcal{T} \subset \mathrm{SE}(3)$ the space of feasible camera poses, and $\mathcal{M}$ the space of internal scene representations (maps).

- **Passive Perception:** Given an exogenous sequence of observations $\{(I_t, D_t, T_t)\}_{t=1}^{N}$, the estimation algorithm computes a map $M_N = \Phi(\{(I_t, D_t, T_t)\}_{t=1}^{N})$ without influencing the poses T_t.
- **Active Perception:** At each discrete step t, the agent maintains a map $M_t \in \mathcal{M}$ updated from past observations and applies a decision policy $\pi : \mathcal{M} \times \mathcal{T}_{\text{task}} \to \mathcal{T}$ that selects the next viewpoint $T_{t+1} = \pi(M_t, \tau)$ conditioned on the task τ, physically moves to T_{t+1}, and acquires (I_{t+1}, D_{t+1}).

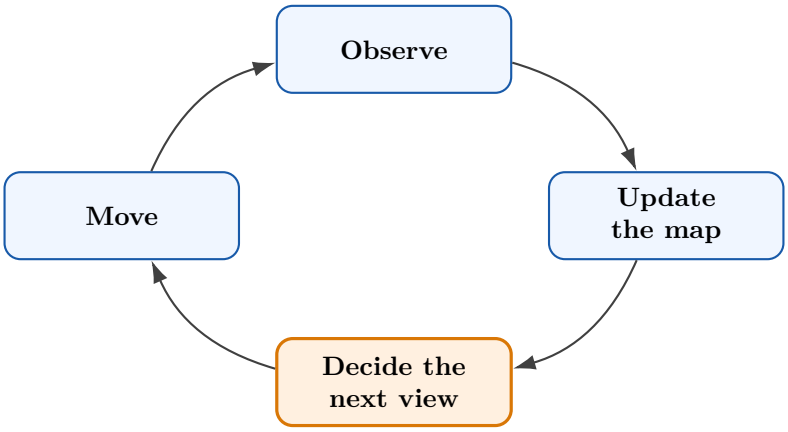


Figure 317: The closed-loop Active Perception cycle (redrawn from Slide 2). The highlighted orange block (*Decide the next view*) closes the loop between mapping and physical camera motion.

Why Passive Data Collection Fails Under a Budget

Imagine walking into a cluttered living room with a camera and only being allowed to take $N = 50$ photographs. A passive agent moving along a pre-programmed path might spend 30 frames staring at a blank white wall or the floor, leaving complex furniture occluded or blurry. An active agent inspects its current partial map after every frame and asks: *"Where is my map still incomplete or uncertain, and which camera movement will fix that fastest?"*

1.2 Active Mapping *(slide 3)*

When the specific task τ is to reconstruct a complete 3D scene within a strict budget of views B, active perception specializes to **Active Mapping** (also known as the **Next-Best-View** or **Next-Best-Pose** problem).

At each discrete time step $t \in \{1, 2, \ldots, B\}$, the system receives an input tuple consisting of:

1. **Color frame** $I_t \in \mathbb{R}^{H \times W \times 3}$ (RGB image),
2. **Depth frame** $D_t \in \mathbb{R}_+^{H \times W}$ (metric depth per pixel),
3. **Camera pose** $T_t \in \mathrm{SE}(3)$ (the 6-DoF position and orientation in world coordinates).

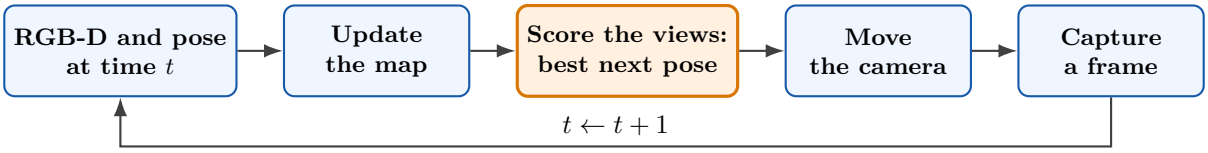


Figure 318: The sequential Active Mapping loop (redrawn from Slide 3). At each step t, the map is updated with (I_t, D_t, T_t), candidate viewpoints are scored, and the camera moves to capture frame $t+1$.

[IMAGE PLACEHOLDER: Slide 3 (top right)]

Photorealistic 3D reconstruction of an indoor living room scene (white sofas, wooden coffee table, armchairs, floor lamp, window blinds) alongside stacked historical input frames (Color, Depth, Pose T_t).

Figure: Input stream of RGB, Depth, and camera poses T_t (left) and the online reconstructed 3D scene map (right) from Slide 3.

1.3 Our Task: Focus on the Objects *(slide 4)*

Standard active mapping treats every square meter of surface area identically: a bare wall or carpet tile receives the same priority as an intricate object on a table. Slide 4 introduces the core thesis of this lecture:

> *"Not all surfaces matter the same: we care about the objects of the scene. To focus on objects, the robot must know what they are: we need semantics."*

Definition 1.2 (Semantic Object-Centric Active Mapping)**.** Let $\mathcal{C} = \{1, \ldots, K\}$ be a set of semantic categories split into **background/structural classes** $\mathcal{C}_{\text{bg}}$ (e.g., wall, floor, ceiling) and **foreground object classes** $\mathcal{C}_{\text{obj}}$ (e.g., chair, cushion, vase, table, lamp). Given a view budget B, the objective of **Semantic Object-Centric Active Mapping** is to select a trajectory of poses $\tau = (T_1, \ldots, T_B) \in \mathcal{T}^B$ that maximizes the geometric completeness, spatial resolution (proximity), multi-angle coverage, and semantic classification accuracy across all object instances $o \in \mathcal{O}$ belonging to $\mathcal{C}_{\text{obj}}$.

To achieve this, the pipeline augments the raw RGB-D input with a 2D semantic segmenter (such as **OneFormer**) that predicts pixel-wise class labels, fuses those labels into the 3D representation, and directs the planner toward object regions requiring closer or novel viewing angles.

[IMAGE PLACEHOLDER: Slide 4]

Four-panel visual summary showing: (1) Input: split RGB and depth image; (2) Classes from a segmenter (color-coded semantic segmentation mask); (3) Where the robot should look (3D grey mesh with bright orange highlighted target objects like pillows, coffee table items, side tables); (4) The map: split view showing photorealistic RGB and semantic class colors.

Figure: Overview of object-centric semantic active mapping (Slide 4), highlighting how semantic segmentation guides the planner to focus on scene objects (orange) rather than structural background.

2 The Tool: 3D Gaussian Splatting *(slides 5–8)*

2.1 3D Gaussian Splatting Representation *(slide 5)*

To both represent the scene at high visual fidelity and rapidly evaluate candidate viewpoints, the system builds upon **3D Gaussian Splatting (3DGS)** (Kerbl et al., ACM TOG 2023). Rather than using an implicit neural radiance field (NeRF) that requires costly volumetric ray-marching through a multilayer perceptron (MLP), 3DGS represents a scene explicitly as a collection of M anisotropic 3D Gaussians (often on the order of 1.0 million Gaussians for a room mapped from 50 views).

Definition 2.1 (Parameterized 3D Gaussian Primitive)**.** A 3D Gaussian primitive g_k ($k \in \{1, \ldots, M\}$) is defined by the parameter tuple

$$g_k = \big(\boldsymbol{\mu}_k,\ \boldsymbol{\Sigma}_k,\ \alpha_k,\ \mathbf{c}_k,\ \mathbf{s}_k\big),$$

where:

- **Position (Mean):** $\boldsymbol{\mu}_k \in \mathbb{R}^3$ is the 3D center in world coordinates.
- **Shape (Covariance):** $\boldsymbol{\Sigma}_k \in \mathbb{S}^3_{++}$ is a 3×3 positive-definite covariance matrix defining the 3D ellipsoid's orientation and scale, parameterized via a rotation matrix $\mathbf{R}_k \in \mathrm{SO}(3)$ (stored as a unit quaternion) and a diagonal scaling matrix $\mathbf{S}_k = \mathrm{diag}(s_{k,x}, s_{k,y}, s_{k,z})$:
$$\boldsymbol{\Sigma}_k = \mathbf{R}_k \mathbf{S}_k \mathbf{S}_k^\top \mathbf{R}_k^\top.$$
- **Opacity:** $\alpha_k \in [0, 1]$ controls the radiative absorption/blending weight of the primitive.
- **Color:** $\mathbf{c}_k \in [0, 1]^3$ (or spherical harmonics coefficients) represents the RGB appearance.
- **Semantic / Utility Payload:** $\mathbf{s}_k$ stores task-specific attributes, such as semantic class vote slots or planning signal values.

The unnormalized 3D spatial density of Gaussian k at point $\mathbf{x} \in \mathbb{R}^3$ is:

$$G_k(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right).$$

[IMAGE PLACEHOLDER: Slide 5]

Three-part figure of a reconstructed room from 50 views using 1.0 million Gaussians: (left) full 3D view of the room showing sofa, chairs, and table; (top right) close-up RGB render ('Color - 50 views'); (bottom right) close-up semantic render ('Classes - 50 views').

Figure: A living room represented by 1.0 million 3D Gaussians trained from 50 views, rendering both photorealistic color and dense semantic classes (Slide 5).

2.2 How Does 3DGS Work? *(slide 6)*

To render an image from a camera pose $T = (\mathbf{R}_c, \mathbf{t}_c) \in \mathrm{SE}(3)$, 3DGS projects each 3D Gaussian into the 2D image plane, sorts the Gaussians intersecting each pixel ray from front to back by depth, and alpha-composits their colors.

Proposition 2.2 (Differentiable Splatting and Alpha-Compositing)**.** *Given a viewing transformation* $\mathbf{W}$ *and the Jacobian* $\mathbf{J}$ *of the projective transformation at* $\boldsymbol{\mu}_k$*, the 3D covariance* $\boldsymbol{\Sigma}_k$ *projects to a 2D covariance matrix in pixel coordinates:*

$$\boldsymbol{\Sigma}'_k = \mathbf{JW}\boldsymbol{\Sigma}_k\mathbf{W}^\top\mathbf{J}^\top.$$

For a pixel $u \in \Omega$*, let* $\{1, \dots, K_u\}$ *index the Gaussians overlapping* u *sorted in ascending order of camera depth* $z_1 \le z_2 \le \cdots \le z_{K_u}$*. The effective 2D opacity at pixel* u *is* $\tilde{\alpha}_k(u) = \alpha_k G'_k(u)$*, where* $G'_k(u)$ *is the 2D Gaussian evaluated at* u*. The rendered color* $\hat{\mathbf{c}}(u)$ *is given by linear blending:*

$$\hat{\mathbf{c}}(u) = \sum_{k=1}^{K_u} w_k(u)\,\mathbf{c}_k, \qquad \text{where} \qquad w_k(u) = \tilde{\alpha}_k(u)\prod_{j=1}^{k-1}\big(1-\tilde{\alpha}_j(u)\big). \tag{20}$$

Here $T_k(u) = \prod_{j=1}^{k-1}(1-\tilde{\alpha}_j(u))$ *is the accumulated transmittance from the camera to Gaussian* k*. During online training, the Gaussian parameters are optimized via gradient descent to minimize the pixel-wise photometric* ℓ_1 *loss against the sensor image* $\mathbf{c}(u)$*:*

$$\mathcal{L} = \sum_{u\in\Omega}\big|\hat{\mathbf{c}}(u) - \mathbf{c}(u)\big|. \tag{21}$$

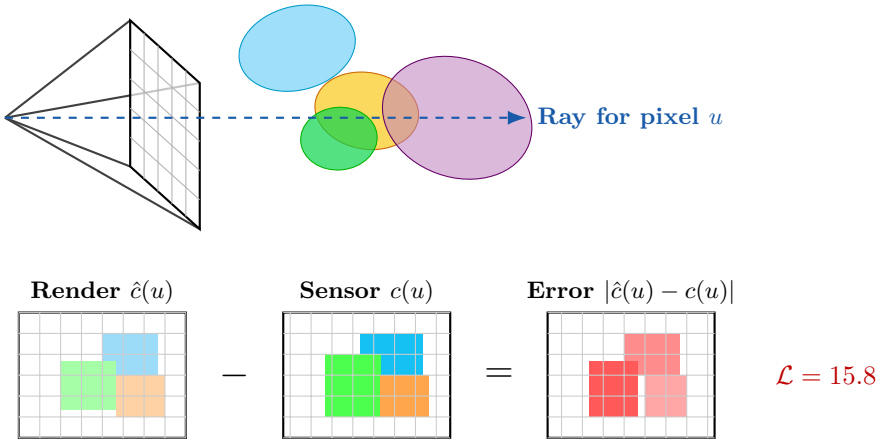


Figure 319: Illustration of 3D Gaussian Splatting (redrawn from Slide 6). Anisotropic 3D Gaussians project onto the sensor grid via $\hat{c}(u) = \sum_k w_k(u)c_k$. Comparing the rendered image against the ground-truth sensor image pixel-by-pixel yields the photometric error map $|\hat{c} - c|$ and scalar loss $\mathcal{L} = \sum_u |\hat{c}(u) - c(u)|$.

2.3 Training Views, Novel Views, and What a Planner Does with 3DGS *(slides 7–8)*

Once optimized on training views collected along the robot's trajectory, 3DGS supports **novel view synthesis** from arbitrary test viewpoints $T \in \mathrm{SE}(3)$ at hundreds of frames per second. Crucially, the exact same alpha-compositing equation (20) can render any modality attached to the Gaussians:

- **Rendered RGB:** replacing $\mathbf{c}_k$ with RGB color $\mathbf{c}_k \in [0,1]^3$,
- **Rendered Depth:** replacing $\mathbf{c}_k$ with the camera-frame z-depth $d_k(T)$ of Gaussian k,
- **Rendered Semantic Classes:** replacing $\mathbf{c}_k$ with the class label or class probability vector of Gaussian k.

Slide 8 poses the central insight linking 3DGS to active planning:

> *"3DGS is used for novel view synthesis, and rendering a new view is very fast. We use the renders to score a view for a task: $g(T, \tau) \in \mathbb{R}$."*

Definition 2.3 (Render-Based View Scoring Function)**.** Let $s_k(\tau, T) \in \mathbb{R}_{\geq 0}$ denote a task-dependent scalar (or multi-channel vector) utility assigned to Gaussian g_k under task τ when viewed from candidate pose $T \in \mathrm{SE}(3)$. The **Render-Based View Score** $g(T, \tau) \in \mathbb{R}$ is computed by rendering the utility signal across the image plane Ω and summing over all pixels:

$$g(T, \tau) = \sum_{u \in \Omega} \hat{s}(u; T, \tau) = \sum_{u \in \Omega} \sum_{k=1}^{K_u} w_k(u; T)\, s_k(\tau, T). \tag{22}$$

Why Rendering Solves Visibility and Occlusion for Free

In traditional active mapping, scoring a candidate pose T requires casting thousands of rays through a 3D voxel grid to check which surfaces are visible and which are occluded by walls or furniture in front of them. Because 3DGS already sorts Gaussians front-to-back and multiplies by the transmittance $T_k(u) = \prod_{j<k}(1 - \tilde{\alpha}_j(u))$, occluded Gaussians behind a solid wall automatically receive weight $w_k(u; T) \approx 0$! Thus, **replacing a Gaussian's color with a task utility turns the fast GPU rasterizer into an occlusion-aware information-gain calculator**.

3 MAGICIAN: Planning with Imagined Gaussians *(slides 9–11)*

3.1 MAGICIAN Architecture and Imagined Gaussians *(slides 9–10)*

Before introducing our semantic object-focused system, Slides 9–10 examine a recent state-of-the-art baseline for coverage-driven active mapping: **MAGICIAN** (*Li, S., et al., "MAGICIAN: Efficient long-term planning with imagined Gaussians for active mapping," CVPR 2026*).

A fundamental challenge in active mapping is that standard 3DGS only places Gaussians on surfaces that have *already been observed*. If the planner only renders existing Gaussians, a completely unexplored region appears empty and generates zero gain. MAGICIAN addresses this via three components:

1. **Imagined Gaussians:** A 3D **Volume Occupancy Network** takes the current reconstructed point cloud and RGB-D observations at time t and predicts occupancy probabilities even in unobserved regions, instantiating *Imagined Gaussians* over predicted surfaces.
2. **Gain as a Novelty Render (Slide 10):** Each Imagined Gaussian is assigned:

   $$\text{opacity } \alpha_k = \text{predicted occupancy} \in [0, 1], \qquad \text{color } c_k = \text{novelty} \in \{0, 1\},$$

   where novelty is 1 (visualized as bright red) if the surface has not yet been covered and 0 (grey) once seen. Rendering this novelty channel from a candidate pose T and summing the pixel intensities directly computes the expected new surface coverage.
3. **Long-Term Planning via Beam Search:** Rather than greedily selecting only the immediate next view T_{t+1}, MAGICIAN evaluates multi-step candidate trajectories $\tau_1, \ldots, \tau_{N_b}$ of length H using **beam search** to maximize cumulative coverage gain $G(\tau)$.

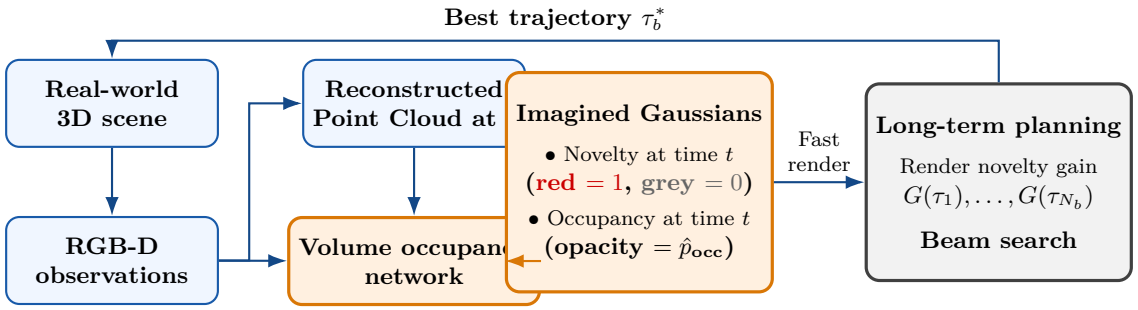


Figure 320: System architecture of MAGICIAN (redrawn from Slides 9–10). A volume occupancy network places Imagined Gaussians with opacity equal to predicted occupancy and color equal to novelty (1 = unseen, 0 = seen), enabling rapid evaluation of multi-step trajectories via beam search.

[IMAGE PLACEHOLDER: Slide 10]

3D cutaway view of a room populated with regular spherical/ellipsoidal Gaussians colored bright red (novelty = 1, unseen surfaces on walls/furniture) and grey (novelty = 0, already observed floor/furniture surfaces).

Figure: MAGICIAN's Imagined Gaussians in a room (Slide 10), where opacity encodes predicted occupancy and color encodes surface novelty (red = 1, grey = 0).

3.2 From Coverage to Objects *(slide 11)*

While MAGICIAN excels at rapid geometric exploration, Slide 11 highlights three critical limitations when the downstream robotics goal is object manipulation or semantic inspection:

1. **Single-glance saturation:** In pure coverage planning, once a surface is seen *once*—even from 6 m away at a grazing angle—its novelty drops to zero (*"Once seen, a surface counts zero, however far away"*). Yet a distant, single-view observation produces distorted, incomplete 3D reconstructions with severe holes (as shown by the coffee-table vase comparison on Slide 11).
2. **Lack of semantics:** *"A book and a wall weigh the same: no semantics."* Pure coverage wastes view budget mapping bare walls and floors at the same priority as tabletop objects.
3. **Robotics motivation:** *"Why: robots find, grasp and inspect objects; walls and floor are only where they move."*

[IMAGE PLACEHOLDER: Slide 11 (top left)]

Side-by-side close-up comparison of a decorative vase on a wooden coffee table: 'Seen once to mark as covered' (severely torn, missing geometry, black holes) vs. 'Seen multiple times to map' (clean, complete, high-resolution cylindrical vase reconstruction).

Figure: Seeing an object once from afar marks it as 'covered' in MAGICIAN but yields poor geometry (left), whereas seeing it from multiple close viewpoints yields an accurate 3D map (right) (Slide 11).

4 The Dual-Map System and Semantic Fusion *(slides 12–17)*

4.1 The System and Full Pipeline *(slides 12–13)*

To map every object accurately along with its semantic class, our system maintains **two coupled representations** updated online from each posed RGB-D frame and OneFormer semantic segmentation mask:

1. **Online Gaussian Map (the final map):** Trained online from RGB-D frames and pixel classes; stores color, geometry, and semantic class votes. It serves both as the final high-resolution output map and as the fast differentiable renderer used by the planner.
2. **Voxel Map (the planning state):** Built from depth measurements alone at 5 cm resolution (with semantic class copied from the Gaussians inside each voxel). It tracks where the robot can safely fly (free vs. occupied vs. unknown space) and records detailed observation statistics per surface voxel: closest viewing distance, viewing directions, and frontier status.

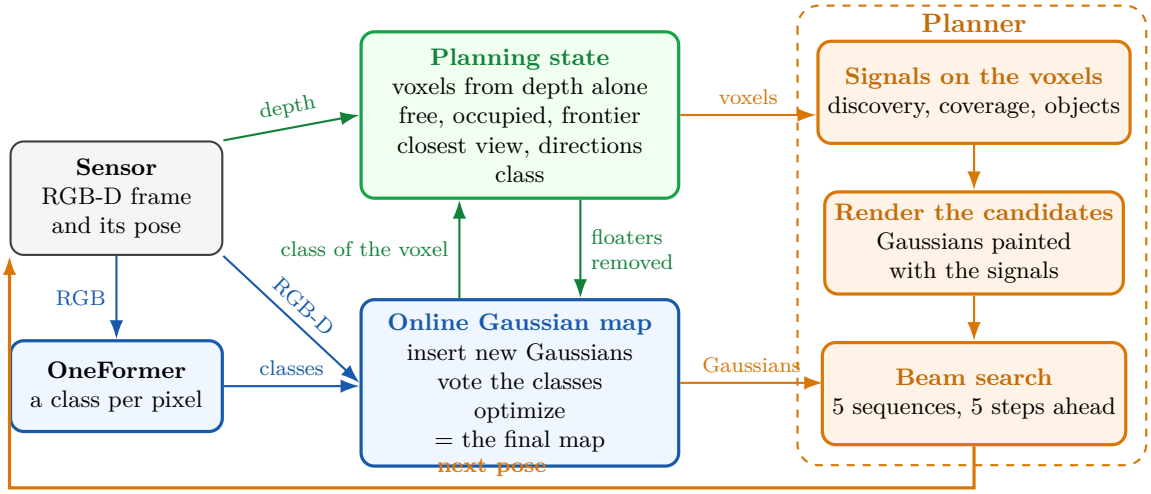


Figure 321: Complete architecture of the semantic Gaussian-based active mapping system (redrawn from Slides 13, 16, and 18). The Voxel Planning State and Online Gaussian Map exchange semantic classes and floater-pruning signals, while the Planner paints voxel utility signals onto the Gaussians to score 5-step beam search trajectories.

4.2 Semantics on a Real Room: From Pixels to Gaussians *(slides 14–15)*

2D semantic segmentation networks like OneFormer are noisy and view-dependent: the same chair might be labeled *armchair* in one frame, *sofa* from an oblique angle, or *cushion* when partially occluded. Storing a full probability distribution over $K \approx 150$ classes on 10^6 Gaussians would consume excessive GPU memory. Slides 14–15 introduce a memory-efficient, depth-consistent **4-slot surface voting mechanism**.

Algorithm 4.1 (4-Slot Depth-Gated Semantic Voting on Gaussians)**.** Each Gaussian g_k maintains at most $L = 4$ semantic vote slots $\{(c_{k,\ell}, v_{k,\ell})\}_{\ell=1}^{4}$, where $c_{k,\ell} \in \mathcal{C} \cup \{\text{empty}\}$ is a class ID and $v_{k,\ell} \in \mathbb{R}_{\geq 0}$ is its accumulated vote weight. When frame t arrives with depth map $D_t(u)$ and OneFormer segmentation mask $C_t(u)$:

1. **Visible Surface Gate (Slide 15):** For each pixel u that Gaussian g_k projects onto, let $z_k(T_t)$ be the distance of $\boldsymbol{\mu}_k$ along the camera ray and $D_t(u)$ be the sensor depth. Gaussian g_k is allowed to receive a vote **if and only if** it lies on the visible physical surface:
$$\big|z_k(T_t) - D_t(u)\big| \leq \delta_{\text{surf}}. \tag{23}$$
If $|z_k(T_t) - D_t(u)| > \delta_{\text{surf}}$ (for example, g_k is occluded behind a foreground object or not on the visible surface, as in Frame 150 on Slide 14), it casts **no vote**.
2. **4-Slot Vote Accumulation:** If (23) holds with pixel class $c = C_t(u)$ and splatting weight $w_k(u)$:
   - If c matches an existing slot $c_{k,\ell}$, increment $v_{k,\ell} \leftarrow v_{k,\ell} + w_k(u)$.
   - Else if an empty slot exists, initialize $(c_{k,\ell}, v_{k,\ell}) \leftarrow (c, w_k(u))$.
   - Else replace or decay the weakest slot if the incoming weight exceeds it.
3. **Dominant Label Assignment:** The semantic class of Gaussian g_k is the slot with the highest accumulated vote:
$$\hat{c}_k = \arg \max_{\ell \in \{1,2,3,4\}} v_{k,\ell}.$$

Why the Depth Gate Prevents Semantic Bleeding Through Walls

Suppose a Gaussian lies on a wall behind a brown armchair. When the camera looks at the armchair, the 2D projection of the wall Gaussian still falls inside the 2D pixel region labeled "armchair" by OneFormer. Without the depth check $|z_k(T_t) - D_t(u)| \leq \delta_{\text{surf}}$, the wall behind the chair would mistakenly accumulate "armchair" votes! Checking that the Gaussian's 3D depth matches the depth sensor's measurement ensures that **only Gaussians physically sitting on the visible surface receive semantic votes**.

[IMAGE PLACEHOLDER: Slide 14]

3D room map highlighting a single tracked Gaussian (circled in red/yellow near an armchair and window) at Frame 150, showing the 2D semantic frame, magnified frustum view, and the 4 semantic vote slots (all 0.00, 'not known yet = 1.00' because it is not on the visible surface).

Figure: Tracking a single Gaussian's 4 semantic vote slots at Frame 150 (Slide 14). Because the Gaussian is not on the visible surface in this frame, no vote is cast.

4.3 The Planning State: 5 cm Voxel Grid *(slides 16–17)*

While 3D Gaussians are ideal for rendering, path planning and collision checking require knowing explicitly which 3D volumes are free air, solid obstacles, or unmapped space. Furthermore, the Gaussian map can occasionally create "floater" artifacts in mid-air during optimization. Thus, Slides 16–17 define the **Planning State** on a regular 5 cm voxel grid constructed strictly from depth measurements.

Definition 4.2 (Voxel Planning State)**.** Let $\mathcal{V}$ be a uniform 3D grid of cubic voxels of side length $\Delta v = 5\,\text{cm}$. Each voxel $v \in \mathcal{V}$ stores a state tuple:

$$\mathcal{S}(v) = \big(\text{state}(v),\ \text{frontier}(v),\ z_v,\ \mathbf{b}_{\text{seen}}(v),\ \mathbf{b}_{\text{feas}}(v),\ c(v)\big),$$

where:

1. **Occupancy State** $\text{state}(v) \in \{\texttt{free}, \texttt{occupied}, \texttt{unknown}\}$: updated via depth raycasting. Determines collision-free navigable space where the robot can fly, and prunes "floater" Gaussians that drift into `free` voxels.
2. **Frontier Flag** $\text{frontier}(v) \in \{0, 1\}$: marks `occupied` voxels that are immediately adjacent to `unknown` voxels, together with the outward normal toward the open/unknown side.
3. **Closest View Distance** $z_v \in \mathbb{R}_+ \cup \{\infty\}$: the minimum Euclidean distance from which voxel v has been observed so far:
$$z_v^{(t)} = \min\big(z_v^{(t-1)},\ \|\mathbf{p}_{\text{cam}}(T_t) - \mathbf{p}_v\|_2\big).$$
4. **Directions Seen** $\mathbf{b}_{\text{seen}}(v) \in \{0,1\}^{24}$: a 24-bit bitmask partitioning the viewing sphere around v into 24 directional sectors (one bit per sector, set to 1 once v is observed from that sector).
5. **Feasible Directions** $\mathbf{b}_{\text{feas}}(v) \in \{0,1\}^{24}$: a 24-bit bitmask indicating which directional sectors around v are unoccluded (line-of-sight through free space) and could still be used to view v.
6. **Semantic Class** $c(v) \in \mathcal{C}$: aggregated by majority vote from the dominant labels $\hat{c}_k$ of the 3D Gaussians whose centers $\boldsymbol{\mu}_k$ fall inside voxel v.

[IMAGE PLACEHOLDER: Slide 17]

3D voxelized reconstruction of 'room_0, pose 149: 5 cm voxels, colored by class', showing crisp 5 cm cubes colored according to semantic categories (green floor, blue window blinds, brown armchairs, purple rug).

Figure: The 5 cm Voxel Planning State at pose 149 in room_0, built from depth geometry and colored by the semantic class transferred from the interior Gaussians (Slide 17).

5 Multi-Signal Planning and Beam Search *(slides 18–21)*

5.1 Three Signals, Three Questions *(slides 18–19)*

With the voxel planning state tracking geometry, viewing history, and semantics, how should the planner value a candidate pose T? Slide 19 decomposes the active mapping objective into **three distinct signals** corresponding to three intuitive spatial questions.

Definition 5.1 (Three Voxel Planning Signals)**.** For each occupied voxel $v \in \mathcal{V}$ and a candidate viewing pose $T \in \mathrm{SE}(3)$ at distance $d(v,T) = \|\mathbf{p}_{\mathrm{cam}}(T) - \mathbf{p}_v\|_2$ and viewing sector $m(v,T) \in \{1,\dots,24\}$:

1. **Discovery Signal** $s_{\mathrm{disc}}(v,T)$ — *"Where does the known end?"*
   - **Reads:** frontier(v) and its open side.
   - **Value:** Non-zero only on frontier voxels (frontier$(v) = 1$) when viewed from their open side; increases as the camera gets closer ($d(v,T)$ decreases), encouraging the robot to look across the boundary into unmapped space.
2. **Coverage Signal** $s_{\mathrm{cov}}(v,T)$ — *"Seen from one direction only?"*
   - **Reads:** $\mathbf{b}_{\mathrm{seen}}(v)$ (directions seen) and $\mathbf{b}_{\mathrm{feas}}(v)$ (feasible directions).
   - **Value:** Non-zero on every surface voxel seen from fewer than 2 directions ($\|\mathbf{b}_{\mathrm{seen}}(v)\|_1 < 2$), provided the candidate pose observes v from a new, feasible direction ($b_{\mathrm{seen},m}(v) = 0$ and $b_{\mathrm{feas},m}(v) = 1$).
3. **Objects Signal** $s_{\mathrm{obj}}(v,T)$ — *"Which objects need a closer or new view?"*
   - **Reads:** semantic class $c(v)$, closest view distance z_v, and directions seen $\mathbf{b}_{\mathrm{seen}}(v)$.
   - **Value:** Non-zero on **open objects** ($c(v) \in \mathcal{C}_{\mathrm{obj}}$ that have not yet met retirement criteria). Rewards each **halving of the viewing distance** down to 1 m (i.e., $d(v,T) \le \frac{1}{2} z_v$ down to $\min z_v = 1\,\mathrm{m}$) as well as observations from **new directions** ($b_{\mathrm{seen},m}(v) = 0$). Once an object has been observed closely ($\le 1\,\mathrm{m}$) from all required feasible directions, it is marked **retired** and its signal becomes 0.

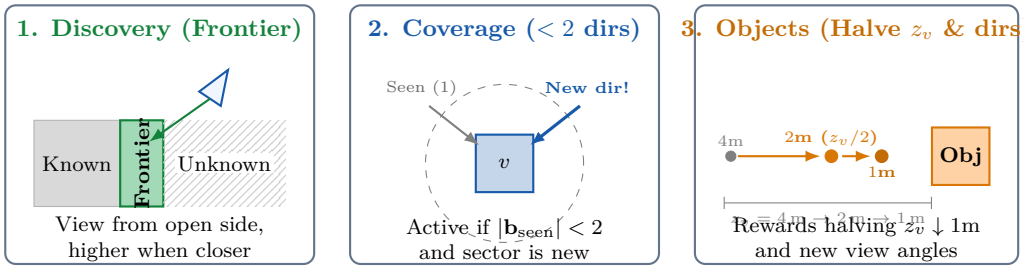


Figure 322: Geometric interpretation of the three voxel signals (Slide 19): (1) **Discovery** scores frontier voxels viewed from their open side; (2) **Coverage** scores surfaces seen from fewer than 2 directions when viewed from a new sector; (3) **Objects** scores foreground objects for each halving of viewing distance down to 1 m and for novel viewing sectors.

Channel-Modular Planning: Painting Signals onto Gaussians

Notice the architectural elegance emphasized at the bottom of Slide 19: *"Each voxel gets a value per signal; each Gaussian takes the value of its voxel. The planner sees channels: a new signal needs no change to it."* Just as an RGB image has 3 color channels (R,G,B), we can "paint" every Gaussian with a 3-channel utility vector $\mathbf{s}_k = (s_{\mathrm{disc}}, s_{\mathrm{cov}}, s_{\mathrm{obj}})$. A single rasterization pass from candidate pose T produces a 3-channel utility image whose pixel sums tell the planner how much discovery, coverage, and object improvement pose T delivers!

5.2 Looking Ahead: Beam Search and Candidate Rendering *(slides 20–21)*

A myopic (1-step greedy) planner can easily get trapped in a local corner if moving toward a distant object requires passing through a hallway that offers zero immediate reward on the first step. To plan ahead while

keeping computation tractable, Slide 20 employs **Beam Search** over a discrete action graph of candidate camera moves.

Algorithm 5.2 (Multi-Step Horizon Beam Search with Signal Rendering)**. Parameters:** Beam width $W = 5$ sequences, planning horizon $H = 5$ steps ahead, candidate neighbor action set $\mathcal{A}(T)$ (poses one move away in free voxel space).

1. **Initialization ($h = 0$):** Start with the singleton beam $\mathcal{B}_0 = \{(T_t)\}$ containing the current robot pose T_t.
2. **Iterative Expansion ($h = 1, \ldots, H$):**
   - Form the candidate set $\mathcal{C}_h = \{\tau \oplus T_{\text{next}} \mid \tau \in \mathcal{B}_{h-1},\ T_{\text{next}} \in \mathcal{A}(\text{last}(\tau))\}$.
   - For each sequence $\tau' = (T_{t+1}, \ldots, T_{t+h}) \in \mathcal{C}_h$, render the signal-painted Gaussian map along the sequence, discounting/zeroing out signals already collected by earlier poses in the same sequence τ' so that viewing the same surface twice in one trajectory is not double-counted:

$$G(\tau') = \sum_{j=1}^{h} \Delta g\big(T_{t+j} \mid T_{t+1}, \ldots, T_{t+j-1}\big).$$

   - **Pruning:** Sort $\mathcal{C}_h$ by cumulative score $G(\tau')$ and retain only the top $W = 5$ highest-scoring sequences to form $\mathcal{B}_h$.
3. **Execution (Receding Horizon):** Select the highest-scoring sequence $\tau^* = \arg\max_{\tau \in \mathcal{B}_H} G(\tau)$, execute its first move T^*_{t+1}, capture a new RGB-D frame, update the maps, and repeat.

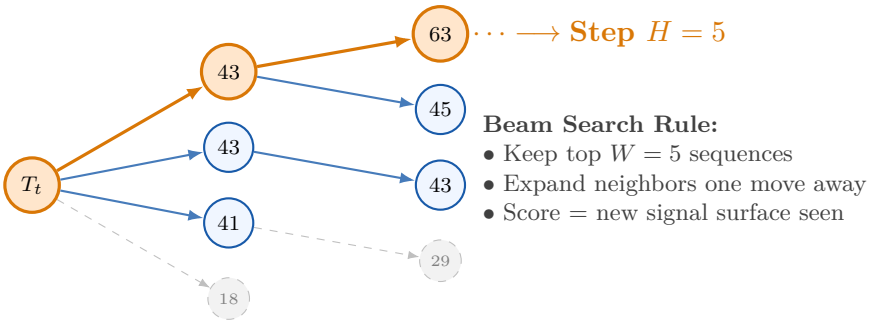


Figure 323: Conceptual illustration of Beam Search over pose sequences (adapted from Slide 20). At each depth, candidate neighbor poses are expanded and scored by rendering the signal-painted Gaussians; only the 5 best sequences (annotated with cumulative scores such as $43 \to 63$) are kept up to horizon $H = 5$.

[IMAGE PLACEHOLDER: Slide 21]

3D room view showing the current camera frustum (black wireframe) and a candidate next pose one move away (red/magenta wireframe) looking toward the coffee table and armchairs, accompanied by the multi-signal renders for Candidate 1.

Figure: Evaluating candidate poses one move away by rendering the Gaussian map once per signal channel (Slide 21).

6 Experimental Results and Generalization *(slides 22–25)*

6.1 A Real Flight: Quantitative Progression and Object Lifecycle *(slides 22–24)*

Slide 22 presents a full 600-pose autonomous flight in `room_0`, tracking both the normalized **work left** across the three signals (discovery, coverage, objects) and the lifecycle of semantic objects across three states:

- **Found:** Total number of distinct foreground objects discovered in the map so far.
- **Open:** Discovered objects that still require closer views (halving distance down to 1 m) or additional viewing directions.
- **Retired:** Objects whose distance and multi-angle viewing criteria have been completely satisfied ($s_{\text{obj}} = 0$).

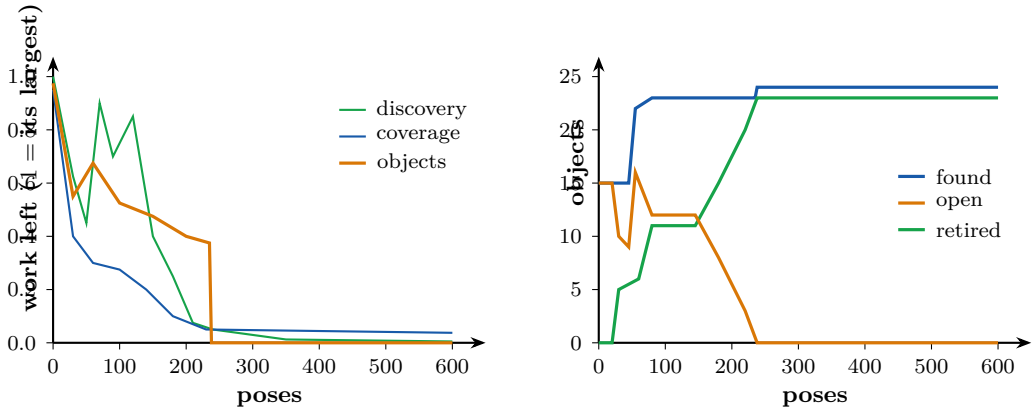


Figure 324: Quantitative dynamics during a 600-pose autonomous flight in `room_0` (redrawn from Slide 22). **Left:** Normalized work remaining for discovery, coverage, and objects drops sharply and finishes around pose 240. **Right:** Evolution of *found* (blue, reaching 24 objects), *open* (orange), and *retired* (green) objects; by pose 240, every discovered object has been thoroughly inspected and retired (`open` = 0).

Reading the Flight Curves on Slide 22

Look closely at the two plots in Figure 324 around poses 50–120 and pose 240:

1. **Temporary spikes in "work left":** Around pose 55, the discovery and object curves jump *upward*. Why? Because the robot turns a corner and sees a new part of the room! New frontier voxels appear and the number of **found** objects jumps from 15 to 23, temporarily increasing the number of **open** objects waiting for close-up inspection.
2. **Task completion around pose 240:** By pose 240, the **retired** curve (green) catches up to the **found** curve (blue), driving **open** objects (orange) and "work left" to zero. This directly motivates one of the open research questions on Slide 26: *knowing automatically when to stop!*

[IMAGE PLACEHOLDER: Slide 22 (top) and Slides 23–24]

Top of Slide 22: 3D trajectory (blue path) overlaid on RGB and semantic maps at pose 600 of 600. Slide 23: Full-bleed photorealistic online Gaussian render of room_0. Slide 24: 2x2 grid of online mapped scenes: room_2, office_0, office_2, office_3.

Figure: Online 3D Gaussian reconstructions and trajectories across Replica dataset rooms (`room_0`, `room_2`, `office_0`, `office_2`, `office_3`) from Slides 22–24.

6.2 Same System, Another Task: Object Navigation *(slide 25)*

A major strength of separating the **Planner/Renderer** from the **Signal Channels** is that changing the robot's task requires **zero changes** to the Gaussian map, differentiable renderer, or beam search planner—only the signals painted onto the Gaussians change. Slide 25 demonstrates this by contrasting **Object Mapping** with **Object Navigation** (*"Find a chair in a house never seen before"*):

- **Frontier Signal:** Rewards visible frontiers in line of sight to explore unknown rooms.
- **Relay Signal:** Guides the robot toward frontiers that are currently *out of sight* (e.g., through a doorway or corridor connecting rooms).
- **Target Signal:** A Vision-Language Model (VLM) detects bounding boxes of the target category (e.g., "chair") in each RGB frame; Gaussians falling inside the detected 2D boxes receive high target signal values.
- **Stopping Condition:** The robot stops automatically once it maneuvers close to a confirmed target object.

7 Summary and Key Takeaways *(slide 26)*

7.1 Comparison of Paradigms and Task Configurations

Table 17 synthesizes the active perception methods and task configurations covered across the lecture.

Method / Task	Scene Representation	Signals Painted on Gaussians	Key Properties & Limitations
Passive 3DGS *(Kerbl et al., 2023)*	M anisotropic 3D Gaussians $(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \alpha_k, \mathbf{c}_k)$	RGB color $\mathbf{c}_k$ (for photometric loss $\mathcal{L}$)	Fast rendering; no viewpoint planning; suffers under limited view budgets.
MAGICIAN *(Li et al., CVPR 2026)*	Reconstructed Point Cloud + Volume Occupancy Net → **Imagined Gaussians**	Opacity $= \hat{p}_{\text{occ}}$, Color = Novelty $\in \{0, 1\}$	Beam search over trajectories; maximizes raw surface coverage; ignores semantics & viewing distance (1 glance = 0 novelty).
Semantic Object Mapping *(This Lecture, Slides 12–24)*	Coupled **Online Gaussian Map** (4-slot voting) + **5 cm Voxel State**	1. **Discovery** (frontiers) 2. **Coverage** (< 2 dirs) 3. **Objects** (halve $z_v \downarrow$ 1m, new dirs)	Depth-gated semantic voting; floater removal; maps every foreground object closely from multiple angles.
Object Navigation *(This Lecture, Slide 25)*	Same Online Gaussian Map + Voxel Planning State	1. **Frontier** (in sight) 2. **Relay** (out-of-sight door) 3. **Target** (VLM 2D boxes)	Same renderer and 5-step beam search; stops when close to a confirmed target.

Table 17: Comparison of passive 3DGS, coverage-driven MAGICIAN, Semantic Object Mapping, and Object Navigation.

7.2 Key Takeaways and Open Directions *(slide 26)*

- **1. Render anything:** By replacing each Gaussian's RGB color $\mathbf{c}_k$ with a task utility signal s_k, candidate viewpoints are scored in a single fast, occlusion-aware rasterization pass: $g(T, \tau) = \sum_u \sum_k w_k(u) s_k$.

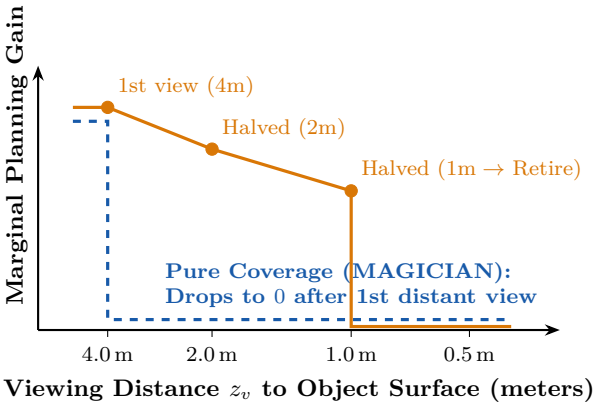


Figure 325: Qualitative comparison of marginal information gain as a function of viewing distance z_v after an object is first spotted at 4.0 m. Pure coverage planners drop to zero gain immediately after the first distant observation, whereas the Object signal continues rewarding closer views (each distance halving down to 1.0 m) until the object is retired.

- **2. Plan on what you build:** The Online Gaussian Map and the 5 cm Voxel Planning State share information bidirectionally at every frame—Gaussians vote on depth-gated pixel semantics and label the voxels, while depth-raycasted voxels remove mid-air Gaussian floaters and compute spatial signals.
- **3. Value is a design choice:** Discovering unknown frontiers, covering single-sided surfaces, and getting closer to semantic objects (or navigating to a VLM-detected chair) are simply different signal channels fed into the exact same renderer and horizon-5 beam search planner.
- **4. Open problems & project ideas:** Key open challenges include learning **when to stop** automatically once scene work is exhausted, handling **dynamic/moving scenes**, operating under **unknown/noisy camera poses** (Active SLAM), deploying on a **real flying robot**, and designing new signal channels such as open-vocabulary **text queries**.

Primary References Cited in the Lecture:

[1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). *3D Gaussian splatting for real-time radiance field rendering.* ACM Transactions on Graphics (TOG), 42(4), 1–14.

[2] Li, S., et al. (2026). *MAGICIAN: Efficient long-term planning with imagined Gaussians for active mapping.* In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2026).

Part XII
EAISUM

Lecture 2: 3D Representations

Core Concept: In 2D, pixel grids are universal. In 3D, there is *no single best representation*—each is a compromise between acquisition, memory, rendering speed, topology flexibility, and neural network compatibility.

- **Manifold:** A space that locally resembles flat Euclidean space $\mathbb{R}^n$ (e.g., an ant on a sphere sees a flat 2D plane). A 2-manifold is a **surface**. Non-manifold mesh errors occur when an edge is shared by > 2 faces or a vertex links two disconnected fans.
- **2.5D Representations (Image-like):**
  - **Depth Map ($D[x_s] \in \mathbb{R}^+$) & Normal Map ($N[x_s] \in \mathbb{S}^2$):** Store distance along the optical axis (or surface normal) per pixel. Fast and CNN-friendly, but **incomplete** (only captures the first visible hit; blind to occluded geometry).
  - **Layered Depth Images (LDI):** Store a list of multiple hits along each ray to handle occlusion, but layers jump discontinuously across pixels.
  - **Multi-Plane Images (MPI):** Stack of fronto-parallel (RGB, α) planes at fixed depths; fast to render for small view shifts, poor resolution far away.
- **Boundary / Surface Representations (Track the "skin", fix topology):**
  - **Point Clouds ($\{x_i\}_{i=1}^N \in \mathbb{R}^{N\times 3}$):** Unordered set of 3D points. Networks *must be permutation invariant* (use symmetric functions like max, mean, sum; never flatten into an FC layer). **No explicit connectivity** (cannot tell a closed loop from a gapped loop without huge point density). **Surfels** add normals n_i and radii to render as small flat disks.
  - **Triangle Meshes ($V \in \mathbb{R}^{N\times 3}, F \in \mathbb{Z}^{M\times 3}$):** Piecewise linear surfaces. Separate **geometry (V, float, differentiable)** from **topology (F, int, discrete/combinatorial)**. Gradient descent can move vertices V, but cannot easily change F or change genus (e.g., torus $\leftrightarrow$ sphere).
  - **Mesh Properties:** Winding order (a, b, c) sets the outward normal $n = (b - a) \times (c - a)$; swapping two indices flips the normal. For closed genus-0 meshes, Euler's formula $(V - E + F = 2$ with $E = 3F/2)$ yields $F = 2V - 4 \approx 2V$. Ray-triangle intersection uses **barycentric coordinates** $p = \alpha a + \beta b + \gamma c$ $(\alpha + \beta + \gamma = 1, \geq 0)$, which also interpolate UV textures.
  - **Parametric Surfaces ($f(u) = x \in \mathbb{R}^3, u \in \mathcal{M} \subset \mathbb{R}^2$):** E.g., Bézier patches. *Easy to sample/generate points* (plug in u), *hard to test inside/outside or intersect rays*, and topology is locked to the domain $\mathcal{M}$.
- **Volumetric Representations (Assign a value to every 3D point; allow arbitrary topology changes):**
  - **Voxel Grids ($V[i, j, k] \in [0, 1]$):** Fixed-size 3D arrays compatible with 3D CNNs and arbitrary topology. **Flaw:** Memory grows cubically as $\mathcal{O}(n^3)$ while the surface only occupies $\mathcal{O}(n^2)$ cells (pays for empty air). **Octrees** subdivide adaptively near boundaries, saving memory at the cost of irregular tree structures.
  - **Implicit Surfaces ($\mathcal{S} = \{x \in \mathbb{R}^3 \mid f(x) = 0\}$):** Convention: $f(x) < 0$ inside, $f(x) > 0$ outside. **Mantra: Parametric generates, Implicit tests.** For f to define a valid surface, it must change sign across $\mathcal{S}$ and have $\nabla f \neq 0$ on $\mathcal{S}$.
  - **Implicit Perks:** 1) **Normals for free:** $n(x) = \nabla f(x)/\|\nabla f(x)\|$ via autodiff. 2) **CSG Boolean operations:** Union $A \cup B \Rightarrow \min(f_A, f_B)$; Intersection $A \cap B \Rightarrow \max(f_A, f_B)$; Subtraction $A \setminus B \Rightarrow \max(f_A, -f_B)$.

- **Signed Distance Function (SDF):** An implicit field where $|f(x)|$ is the exact Euclidean distance to the closest surface point. Satisfies the **Eikonal equation** $\|\nabla f(x)\| = 1$. Enables **Sphere Tracing**: stepping along a ray by $t \leftarrow t + f(o + td)$ is guaranteed never to overshoot the surface because the ball of radius $|f(x)|$ is empty. Stored on grids (**TSDF** truncates to $[-\tau, +\tau]$ near surfaces) or in MLPs.
- **Marching Cubes:** Converts an implicit field to a triangle mesh by evaluating f on a grid, finding edges where corners have opposite signs, and linearly interpolating vertex crossings $t = \frac{f(a)}{f(a)-f(b)}$.
- **Density Fields ($\sigma(x) \geq 0$):** Represent absorption per unit length for semi-transparent media (smoke, CT scans, NeRF, 3D Gaussians) with no hard surface boundary.

2rowgraywhite

Representation	**Sample Points**	**Inside/Outside**	**Change Topology**	**Direct Render**	**Memory Scaling**
Point Cloud	Given	Hard (no surface)	Any point set	Splats	$\mathcal{O}(N)$ points
Triangle Mesh	Easy (barycentric)	Ray crossings	Hard (fixed F)	Fast GPU raster	$\mathcal{O}(V+F)$
Parametric $f(u)$	Easy (plug in u)	Hard (invert f)	Locked to $\mathcal{M}$	Convert to mesh	Control points
Voxel Grid	Marching cubes	Easy (cell lookup)	Easy (any)	Blocky / march	$\mathcal{O}(n^3)$ (wasteful)
Implicit / SDF	Hard (March. cubes)	Trivial (sign(f))	Easy (continuous)	Sphere tracing	Grid $\mathcal{O}(n^3)$ or MLP

Lecture 3: Depth Measurement, Priors & Diffusion (Marigold)

Core Concept: Pinhole projection $p^c = zK^{-1}\tilde{x}_s$ destroys distance z. A single image leaves global scale unobservable (a small near object looks identical to a large far object). Every depth method must **pay with a currency**: *Time*, *Baseline*, *Computation/Motion*, or a *Learned Prior*.

- **The Ambiguity Ladder (Free Parameters):**
  - **Metric Depth (0 free params):** True meters (from LiDAR or calibrated stereo).
  - **Up to Scale (1 free param, s):** $\hat{D} \approx sD^*$. Shape is preserved (similarity transform). Produced by monocular SfM or Eigen's log-space loss. Pinned down by 1 known length.
  - **Affine-Invariant Depth/Disparity (2 free params, s, t):** $\hat{q} \approx sq^* + t$. Pinned down by 2 known depths.
  - **Ordinal (monotone map):** Only relative front/back ordering is known.
- **Physical Measurement Principles & Error Laws:**
  - **Time of Flight (ToF / LiDAR):** Active only (needs known departure time). Measures round-trip delay $r = \frac{ct}{2}$. Timing error floor $\delta r = \frac{c\delta t}{2}$ is **independent of range** (1 ns $\approx$ 15 cm). **Flaws:** Structured sparsity (vertical scan-line gap grows linearly with range $s(r) \approx r\Delta\theta_{\text{el}}$, missing distant pedestrians), missing returns $\neq$ zero depth, and "flying points" at silhouettes where one beam straddles foreground and background.
  - **Projecting LiDAR to Depth Image:** Transform via extrinsics $p^c = R_{cl}p^l + t_{cl}$, project $\tilde{x}_s = \pi(Kp^c)$, use a z-buffer (keep smallest $p^c_z > 0$). Note: Depth map records camera z-axis $z = r\cos\theta$, *not* radial range $r = \|p^c\|$.
  - **Triangulation (Stereo):** $z = \frac{fb}{d}$ where baseline b provides metric scale. Depth error grows **quadratically with depth**: $\delta z \approx \frac{z^2}{fb}\delta d$. (LiDAR is precise at long range but sparse; stereo is dense at short range but degrades quadratically).

- **Closed-Form Monocular Cues & Network Shortcuts:**
  - **Vanishing Points:** $\lambda v = KRd$ (depends only on rotation/pitch/roll, independent of translation).
  - **Apparent Size:** $z = \frac{fH}{h}$ (requires knowing real object height H).
  - **Vertical Ground Contact Position:** $z = \frac{fY}{y_s - y_h}$ (requires flat ground, camera height Y, and horizon row y_h; error grows as z^2).
  - **What Networks Actually Use (Probes):** 1) *Pasted-Car Experiment:* Networks rely on **vertical image position** over apparent size. 2) *Pitch/Roll Crops:* Networks read raw pixel row rather than true distance to the horizon. 3) *Camera Focal Length Bias:* Networks memorize training focal length f_0, predicting $\hat{z} = \frac{f_0}{f} z$ on test camera f. 4) *Anamorphic Street Art:* Flat pavement painted like a hole fools single-image networks with zero uncertainty warning.
- **Supervised & Self-Supervised Single-View Depth:**
  - **Eigen Scale-Invariant Loss:** Log-MSE minus the per-image mean offset $\alpha = \frac{1}{n}\sum(\log y_i^* - \log y_i)$, which equals the variance $\mathrm{Var}(d_i)$ of log-differences. Invariant to global scaling $y \mapsto sy$.
  - **Affine Invariance in Disparity vs. Depth:** In disparity ($\frac{1}{z} \mapsto \frac{s}{z} + t \iff z \mapsto \frac{z}{s+tz}$), back-projecting yields a projective map $\hat{p} = \frac{p}{s+tp_z}$ that **keeps planes flat** (tilts them without bending). In depth ($\hat{z} = sz + t$), back-projecting gives $\hat{p} = (s + \frac{t}{z})p$: scale s preserves shape, but **shift t distorts/bends 3D shapes** because rays fan out.
  - **MiDaS:** Mixes heterogeneous datasets (metric, SfM, stereo movies) by predicting in **affine-invariant disparity space**, aligning each training image via closed-form least squares (s, t) before computing loss + multi-scale gradient loss.
  - **Depth Anything V2:** Trains a large teacher on exact synthetic data (sharp edges, no sensor holes), uses the teacher to pseudo-label millions of unlabeled real images, and trains a student on those pseudo-labels.
  - **Self-Supervised Stereo:** Trains without depth labels using photometric warp loss $\mathcal{L}_{\text{photo}} = \|\tilde{I}_L - I_L\|$ via **backward warping** $\tilde{I}_L(u, v) = I_R(u - d(u, v), v)$ (avoids holes/collisions of forward warping). **Blind spots:** Textureless regions (image gradient $I'_R = 0 \Rightarrow \frac{\partial \mathcal{L}}{\partial d} = 0$) and occlusions.
- **Diffusion Priors, Marigold & Marigold-DC:**
  - **Regression vs. Generative:** MSE regression outputs the conditional mean $\mathbb{E}[Y|x]$, which averages foreground and background at depth edges (creating blurry flying pixels). Diffusion samples a sharp mode from $p(D|I)$.
  - **Denoiser as a Prior:** Trained on $L_{\text{simple}} = \mathbb{E}\|\epsilon_\theta(x_0 + \sigma\epsilon, \sigma) - \epsilon\|^2$. The ideal denoiser satisfies $\sigma\epsilon^*(x, \sigma) = \nabla_x D_\sigma(x)$ (the gradient of the **smoothed distance function** to the data manifold): at small σ it points to the nearest data point; at large σ to the dataset center.
  - **DDIM Step:** $x_{t-1} = x_t + (\sigma_{t-1} - \sigma_t)\epsilon_\theta(x_t, \sigma_t)$, walking toward the clean Tweedie preview $\hat{x}_0^t = x_t - \sigma_t\epsilon_\theta(x_t, \sigma_t)$.
  - **Marigold:** Repurposes pretrained **Stable Diffusion**: freezes the VAE, encodes image $z^{(I)}$ and percentile-normalized depth $\tilde{D} \in [-1, 1]$ into latent space, concatenates $[z_\sigma^{(D)}, z^{(I)}]$ (doubling first U-Net layer channels and halving weights), and fine-tunes on 74K synthetic pairs. At test time, ensembles $N = 10$ runs (50 steps each = 500 evals) by mutually aligning scale/shift and taking the pixel median. **Output:** Affine-invariant depth (2 free params, slow).
  - **Marigold-DC (Depth Completion via Test-Time Guidance):** Upgrades sparse metric LiDAR measurements c to a dense metric depth map **with zero retraining**. Inside the 50-step DDIM loop, decodes the Tweedie preview $\hat{z}_0^t$, applies learnable scale/shift $\hat{d}_t^{(m)} = \hat{a}\mathcal{D}(\hat{z}_0^t) + \hat{b}$, computes error against sparse LiDAR pixels, and takes a gradient descent step on $(z_t, \hat{a}, \hat{b})$ while keeping U-Net/VAE weights frozen.

Lecture 4: Stereo Depth Estimation (Classical to Uncalibrated 3D)

Core Concept: Calibration and rectification turn 2D correspondence search into a 1D horizontal scanline search. Modern 3D pointmap networks (DUSt3R/MASt3R) drop calibration and rectification entirely by matching in 3D space.

- **Classical Stereo Pipeline:** 1) Calibrate → 2) Rectify → 3) Compute disparity → 4) Left-Right consistency outlier filter → 5) Triangulate $z = fb/d$ → 6) Fuse 3D model.
- **Image Rectification:** Applies a pure rotation R_{rect} about each camera center so both image planes become coplanar and parallel to the baseline b. Epipoles move to infinity along the horizontal axis, and the epipolar constraint reduces to $y_1 = y_2$ (1D search along the same row). **Fails** under forward motion (epipole lands inside the image).
- **Block Matching & Failure Modes:**
  - **Metrics: SSD** $\|w_L - w_R\|_2^2$ (sensitive to exposure); **ZNCC** $\frac{(w_L-\mu_L)^\top (w_R-\mu_R)}{\|w_L-\mu_L\|\|w_R-\mu_R\|}$ (invariant to linear gain and bias $aw + b$).
  - **Window Size Trade-off:** Small windows → noisy, ambiguous matches. Large windows → smooth, but violate the fronto-parallel assumption on slanted surfaces and cause **border bleeding** (foreground edges fatten over weakly textured backgrounds).
  - **Half-Occlusions:** A strip of background of width $d_f - d_b$ immediately to the left of a foreground object in the left image is hidden in the right image. Caught by the **Left-Right Consistency Check**: $|d_L(x, y) - d_R(x - d_L(x, y), y)| \leq \tau$.
- **Learned Stereo Evolution:**
  - **Siamese Networks (Zbontar & LeCun):** Patch-level CNN trained on triplets (anchor, positive within ±1 px, hard negative 3–6 px away) using **Hinge Loss** $\max(0, m + s_- - s_+)$. *Cosine-similarity head* (dot product of normalized features) is much faster than an MLP similarity head.
  - **DispNet:** End-to-end 2D encoder-decoder with a 1D correlation layer; pretrained on synthetic data (FlyingThings3D).
  - **GC-Net:** Builds a 4D cost volume ($H \times W \times D \times F$), regularizes with 3D convolutions, and extracts sub-pixel disparity via differentiable **Soft Argmin**: $\hat{d} = \sum_{d=0}^{d_{\max}} d \cdot \text{softmax}(-c_\theta(d))$. **Flaw:** High 3D memory cost, and bimodal cost curves at occlusion edges average out to flying pixels between modes.
- **Stereo Without Calibration: DUSt3R & MASt3R:**
  - **Pointmap ($X^{a,1} \in \mathbb{R}^{H\times W\times 3}$):** Assigns every pixel of image $a \in \{1, 2\}$ its 3D coordinate **expressed in camera 1's coordinate frame**. Two pixels correspond iff their 3D coordinates coincide.
  - **Architecture:** Shared ViT-Large Siamese encoder → twin ViT-Base decoders coupled via **cross-attention** → regression heads outputting pointmaps $\hat{X}^{a,1}$ and confidence maps C^a.
  - **Confidence-Weighted Loss:** $\mathcal{L}_{\text{conf}} = \sum C^a[i] \left\| \frac{1}{\hat{\rho}} \hat{X}^{a,1}[i] - \frac{1}{\rho} X^{a,1}[i] \right\| - \alpha \log C^a[i]$. Dividing by mean distances $\rho, \hat{\rho}$ normalizes unknown scale; the $-\alpha \log C$ penalty prevents the network from cheating by setting $C \to 0$ everywhere (optimal confidence is $C^* = \alpha/\ell_{\text{regr}}$).
  - **MASt3R Additions:** Because regressed 3D points are robust to huge viewpoint changes but locally imprecise, MASt3R adds a **local descriptor head** ($F^a \in \mathbb{R}^{H\times W\times 24}$) trained with **InfoNCE loss** (a smooth multi-class generalization of hinge loss) and extracts matches via 2D **reciprocal nearest neighbors** (mutual NN across the whole image).
  - **Trade-off vs. Classical Stereo:** Handles uncalibrated cameras, wide baselines, and forward motion, but sacrifices measured metric scale (scale comes from a learned prior), high image resolution, and low compute.

Lecture 5: Structure-from-Motion (Classical COLMAP vs. Feed-Forward VGGT)

Core Concept: Classical SfM alternates feature matching, minimal geometric solvers inside RANSAC, and sparse nonlinear Bundle Adjustment (BA). Feed-forward transformers (VGGT) predict cameras, depth, pointmaps, and tracks for dozens of views in one pass (< 1 s), providing an ideal initialization for BA.

- **Camera Calibration (Zhang's Method):** Captures a flat checkerboard ($Z = 0$) in multiple poses. Each view defines a plane-to-image homography $H = \lambda K[r_1\ r_2\ t]$. Orthonormality of rotation columns ($r_1^\top r_2 = 0, \|r_1\| = \|r_2\|$) yields 2 linear constraints per view on $B = K^{-\top}K^{-1}$ (5 DoF $\Rightarrow \geq 3$ tilted views needed), solved via SVD and refined jointly with lens distortion via nonlinear least squares (reprojection error).
- **Feature Detection & Matching:**
  - **SIFT:** Detects scale-space extrema in **Difference of Gaussians (DoG)** (approximates $\sigma^2\nabla^2 G$ for scale invariance), rotates patch to dominant gradient orientation (rotation invariance), and pools 4×4 cells of 8-bin gradient histograms into a **128D vector**. Filtered via **Lowe's Ratio Test**: $\frac{\|f - f'_{(1)}\|}{\|f - f'_{(2)}\|} < 0.8$ (rejects ambiguous repetitive patterns).
  - **SuperPoint:** Self-supervised CNN with shared encoder ($8\times$ downsampling) and two heads: keypoint head ($H/8 \times W/8 \times 65$: 64 pixel bins + 1 dustbin, softmax) and descriptor head (256D). Trained via 1) MagicPoint on synthetic shapes $\rightarrow$ 2) **Homographic Adaptation** (averages detections across $N_h = 100$ random homography warps of real images to enforce covariance) $\rightarrow$ 3) joint training on warped pairs. SIFT still wins at finest sub-pixel localization ($\varepsilon = 1$); SuperPoint wins on descriptor mAP and speed (~ 70 fps).
- **Two-Frame SfM & Epipolar Geometry:**
  - **Essential Matrix ($E = [t]_\times R$):** Operates on calibrated rays $\tilde{x}_i = K_i^{-1}\bar{x}_i$ via $\tilde{x}_2^\top E\tilde{x}_1 = 0$. Has **rank 2**, **5 DoF** (3 rot + 3 trans $-$ 1 scale), and singular values $(\sigma, \sigma, 0)$. Estimated via the **normalized 8-point algorithm** (SVD + projecting singular values to $(1, 1, 0)$) or minimal **5-point solver** inside RANSAC.
  - **4-Fold Decomposition & Cheirality:** Decomposing $E = U\,\mathrm{diag}(1, 1, 0)V^\top$ yields 4 pose candidates: ($UWV^\top$ or $UW^\top V^\top$) and $t = \pm u_3$ (baseline sign flip + 180° twisted pair rotation). Only **1 solution** puts triangulated points in front of *both* cameras ($z_1 > 0, z_2 > 0$, **cheirality check**).
  - **Fundamental Matrix ($F = K_2^{-\top}EK_1^{-1}$):** Used when intrinsics K_i are unknown ($\bar{x}_2^\top F\bar{x}_1 = 0$); rank 2, **7 DoF**.
  - **Triangulation & Parallax Trade-off:** Linear DLT ($\bar{x}_i \times P_i\tilde{x}_w = 0$) or nonlinear reprojection minimization. **Trade-off:** Small baseline/parallax angle γ makes feature matching easy, but rays are nearly parallel so depth uncertainty explodes ($\sigma_Z \propto 1/\gamma$); wide baseline gives accurate triangulation but hard matching.
- **Bundle Adjustment (BA) & Incremental SfM (COLMAP):**
  - **Formulation:** Jointly minimizes reprojection error $\sum_{i,p} w_{ip}\|x_{ip}^s - \pi(K_i(R_i x_p^w + t_i))\|_2^2$ over all camera poses $T_i \in SE(3)$ and 3D points $x_p^w \in \mathbb{R}^3$.
  - **Gauge Freedom (7 DoF):** The cost is invariant to global rigid motion (6 DoF) and global scale (1 DoF), giving the Hessian a 7D null space (fixed by anchoring $T_1 = I$ and $\|t_2\| = 1$).
  - **Lie Algebra Updates:** Rotations are not a vector space; poses are updated in the tangent space $\mathfrak{se}(3) \cong \mathbb{R}^6$ via the exponential map $T_i \leftarrow \mathrm{Exp}(\Delta\xi_i)T_i$.
  - **Schur Complement Trick:** In the Gauss-Newton normal equations $H\Delta s = -b$, points never talk directly to other points, so H_{xx} is block-diagonal (3×3 blocks). Eliminating points yields the **reduced camera system** $(H_{\xi\xi} - H_{\xi x}H_{xx}^{-1}H_{x\xi})\Delta\xi = -(b_\xi - H_{\xi x}H_{xx}^{-1}b_x)$ of size $6N$ instead of $6N + 3P$ ($\sim 10^8\times$ speedup).
  - **Incremental SfM Loop:** Seed with a wide-baseline two-view reconstruction $\rightarrow$ register next image via **PnP / P3P** (2D-to-3D matches) $\rightarrow$ triangulate new points $\rightarrow$ run local BA (and periodic global BA) + filter outliers. **Flaw:** Slow, sequential error accumulation/drift, fails on low overlap or textureless scenes. (GLOMAP is a non-learned global alternative).

- **Feed-Forward SfM: VGGT (Visual Geometry Grounded Transformer):**
  - **Input/Output:** Takes N uncalibrated images; outputs per-image camera parameters $\hat{g}_i \in \mathbb{R}^9$ (quaternion, translation, FoV), dense depth $\hat{D}_i$, world-frame pointmaps $\hat{X}_i$ (in camera 1's frame), and tracking features F_i **in one forward pass (< 0.2 s)**.
  - **Architecture:** DINO patch tokens + 1 learned **camera token** per image $\rightarrow L = 24$ layers of **Alternating Attention** (alternating *Global Attention* across all tokens of all N images with $N\times$ cheaper *Frame Attention* within each single image) $\rightarrow$ Camera head + DPT dense head + CoTracker2 tracking head.
  - **Redundancy & BA Synergy:** Predicting redundant outputs (depth + cameras + pointmaps) improves accuracy. Using VGGT predictions to initialize classical Bundle Adjustment eliminates incremental initialization and boosts pose accuracy from 85.3 $\rightarrow$ 93.5 AUC@30 in ~ 1.8 s.

Lecture 6: Visual SLAM (Filtering, Factor Graphs, ORB-SLAM & Active SLAM)

Core Concept: SLAM is joint estimation of trajectory and map in real time, with loop closure to eliminate dead-reckoning drift ($\sigma\sqrt{t}$). Modern SLAM drops dense EKF filtering in favor of sparse keyframe factor graphs.

- **Filtering (Online SLAM: $p(x_t, m \mid z_{1:t}, u_{1:t})$) vs. Graph-Based (Full SLAM: $p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$):**
  - **Bayes Filter / EKF-SLAM (MonoSLAM):** Maintains a single joint Gaussian $\mathcal{N}(\mu, \Sigma)$ over the current camera state (13D) and N 3D landmarks ($3N$). Prediction touches only camera blocks, but the **Kalman update touches the entire covariance matrix**: $\mathcal{O}(N^2)$ time and memory per frame, limiting maps to ~ 100 points at 30 Hz.
  - **Why Marginalizing Past Poses Hurts:** Marginalizing out a past pose x_k via the Schur complement connects every landmark seen by x_k to every other landmark it saw (**fill-in**, turning a sparse star into a dense clique). Also, EKF linearizes Jacobians only once, risking divergence.
  - **Factor Graphs & Pose Graphs:** Keep keyframe poses in the state! Variable nodes (poses, points) connect to factor nodes (priors, odometry, reprojection, loop closures). Maximizing the posterior product of Gaussian factors equals **nonlinear least squares**: $\min \sum_k e_k^\top \Omega_k e_k$, where $\Omega_k = \Sigma_k^{-1}$ is the information matrix.
  - **Sparsity of H:** In Gauss-Newton/Levenberg-Marquardt ($H\Delta x = -b$), block H_{ij} is non-zero **if and only if an edge links variables i and j** (graph adjacency matrix). Keeping keyframes keeps H sparse!
- **The ORB-SLAM Pipeline (3 Parallel Threads):**
  - **Features:** Uses multi-scale **ORB** (oriented FAST corners + binary descriptors) matched via fast bitwise XOR **Hamming distance** (< 33 ms, unlike SIFT).
  - **Map Initialization:** Runs RANSAC in parallel for a **Homography** H_{21} (planar/low-parallax) and **Fundamental matrix** F_{21} (general scene); picks H if $S_H/(S_H + S_F) > 0.45$, and waits to initialize until sufficient parallax exists.
  - **Thread 1 — Tracking (every frame, real-time):** Predicts pose via constant-velocity model, projects local map points from covisible keyframes, and runs **motion-only BA** (optimizing only the 6-DoF current pose with Huber loss). Inserts keyframes generously.
  - **Thread 2 — Local Mapping (per keyframe):** Updates the **Covisibility Graph** (edges between keyframes sharing ≥ 15 points), triangulates new points, runs **Local BA** on the covisible neighborhood (keeping outer keyframes fixed), and culls redundant keyframes (where $\geq 90\%$ of points are seen by ≥ 3 other keyframes).
  - **Thread 3 — Loop Closing & Relocalization:** Queries a hierarchical **Bag of Binary Words (DBoW2)** vocabulary tree (k branches, L levels $\Rightarrow k^L$ words in only kL comparisons) with an **inverted index**. Because monocular SLAM has **7-DoF drift** (rotation, translation,

and scale), loop correction optimizes the Essential Graph over **Sim**(3) **similarity transforms** ($e_{ij} = \log(Z_{ij}^{-1} S_i S_j^{-1})^\vee \in \mathbb{R}^7$) rather than $SE(3)$.

- **Advanced SLAM Directions:**
  - **Rolling-Shutter / Continuous-Time SLAM:** Rolling-shutter cameras expose each scanline at a different timestamp, skewing images under motion. Representing the trajectory as a continuous **cumulative cubic B-spline on** $SE(3)$ lets every scanline (or IMU reading) query its exact timestamp t, cutting loop error by 8×.
  - **Active SLAM & FisherRF:** Balances **exploration** (new frontiers) vs. **exploitation** (revisiting known areas to close loops and shrink covariance). Minimizes uncertainty volume via **D-optimality** $\det(\Sigma)^{1/n}$. In radiance fields (**FisherRF**), selects the next view x maximizing expected information gain $\mathrm{IG}(x) \le \frac{1}{2}\mathrm{tr}(I_x I_{\mathcal{D}}^{-1})$ using the diagonal Fisher matrix $I_x = J_x^\top J_x$ (evaluated without needing a ground-truth image).
  - **Learned Front Ends: MASt3R-SLAM** uses Sim(3) pose graphs; **VGGT-SLAM** aligns uncalibrated submaps using $SL(4)$ **projective homographies (15 DoF)**.

Lectures 7 & 8: Volume Rendering, NeRF, Plenoxels & Instant-NGP

Core Concept: Drop the hard-surface assumption and model the scene as a continuous field of density $\sigma(x)$ and view-dependent emitted radiance $c(x, \omega)$, trained via differentiable ray marching against posed 2D images.

- **Radiative Transfer vs. Emission-Absorption (EA) Model (Lecture 7):**
  - **4 Physical Interactions:** Loss from **Absorption** ($-\sigma_a L$) and **Out-scattering** ($-\sigma_s L$); Gain from **Emission** ($+\sigma_a L_e$) and **In-scattering** ($+\sigma_s L_s$, which integrates incoming light over the whole sphere of directions recursively).
  - **Why NeRF Drops Scattering:** In-scattering is recursive and expensive. By allowing an arbitrary view-dependent effective emission $L'_e(x, \omega) = L_e + \frac{\sigma_s}{\sigma_a}(L_s - L)$, the **Emission-Absorption (EA) model** $dL = -\sigma(x) L\, dz + \sigma(x) L_e(x, \omega)\, dz$ can bake scattered/reflected appearance directly into $L_e(x, \omega)$ (at the cost of not being physically relightable).
  - **Transmittance & Continuous Volume Rendering Integral:**

$$T(x, x_t) = \exp\left(-\int_0^t \sigma(x - s\omega)\, ds\right), \qquad L(x, \omega) = \int_0^z T(x, x_t)\, \sigma(x_t)\, L_e(x_t, \omega)\, dt + T(x, x_z) L_{\mathrm{bg}}$$

    Transmittance is symmetric ($T(x, y) = T(y, x)$) and multiplicative ($T(x, y) = T(x, z)T(z, y)$).
  - **Computational Ray Marching (Alpha Compositing):** Discretize the ray into N homogeneous segments of length Δt_i:

$$L(x, \omega) \approx \sum_{i=1}^{N} w_i c_i + T_{N+1} L_{\mathrm{bg}}, \qquad w_i = T_i \alpha_i, \qquad \alpha_i = 1 - e^{-\sigma_i \Delta t_i}, \qquad T_i = \prod_{j<i} (1 - \alpha_j)$$

    Because $w_i = T_i - T_{i+1}$, the weights telescope: $\sum_{i=1}^{N} w_i + T_{N+1} = 1$ (a partition of unity; T_{N+1} is the background mask).

- **Neural Radiance Fields (NeRF, Lecture 8):**
  - **Architecture Inductive Bias:** An MLP $f_\theta : (\gamma(x), \gamma(\omega)) \mapsto (\sigma, c)$. Position x enters at layer 1 to output density $\sigma(x)$; viewing direction ω is injected **only in the final head layers** so that **geometry** $\sigma(x)$ **is strictly view-independent** while color $c(x, \omega)$ captures view-dependent specularities.
  - **Positional Encoding (Overcoming Spectral Bias):** Standard ReLU MLPs are biased toward low-frequency smooth functions, producing blurry renders. Lifting inputs via $\gamma(v) = [\sin(2^k \pi v), \cos(2^k \pi v)]_{k=0}^{L-1}$ ($L = 10$ for x, $L = 4$ for ω) induces a tunable high-frequency stationary kernel $\langle \gamma(v), \gamma(v') \rangle = \sum \cos(2^k (v - v'))$ so nearby points can take sharp, distinct values.

- **Hierarchical (Coarse-to-Fine) Sampling:** Uniform sampling wastes MLP evaluations in empty air or occluded regions. A **coarse pass** evaluates $N_c = 64$ uniform samples, normalizes the rendering weights $\hat{w}_i = w_i / \sum w_j$ into a piecewise-constant PDF along the ray, and uses **inverse-CDF transform sampling** to concentrate $N_f = 128$ **fine samples** right around visible surfaces where w_i peaks.
- **Inverse-Depth / NDC Sampling:** For unbounded forward-facing scenes, sampling linearly in $1/z$ (or mapping the frustum to Normalized Device Coordinates $[-1, 1]^3$) allocates high resolution to close objects (large parallax) and coarse resolution to distant backgrounds.
- **Anti-Aliasing (mip-NeRF):** Casting a single 1D ray per pixel aliases when camera distance or resolution changes. **mip-NeRF** casts a 3D **conical frustum**, approximates each frustum slice with a 3D Gaussian $\mathcal{N}(\mu, \Sigma)$, and feeds the **Integrated Positional Encoding (IPE)** $\mathbb{E}[\sin(2^k X)] = \sin(2^k \mu) \exp(-\frac{1}{2} 4^k s^2)$, which automatically damps high frequencies whose wavelength is smaller than the pixel footprint s.

- **Trading MLP Compute for Grid Lookups (Plenoxels & Instant-NGP):**
  - **Plenoxels (No Neural Network!):** Stores density σ and **Spherical Harmonics (SH) coefficients** ($\ell_{\max} = 2 \Rightarrow 9 \times 3 = 27$ coeffs $+ 1\,\sigma = 28$ values/voxel) on a **sparse voxel grid**. Queries via trilinear interpolation, renders via ray marching, and optimizes voxel values directly with photometric loss + Total Variation ($\mathcal{L}_{\text{TV}}$) regularization, pruning empty voxels. Trains in minutes instead of hours.
  - **Instant-NGP (Multiresolution Hash Encoding):** Uses $L = 16$ grid resolutions N_ℓ. Instead of storing N_ℓ^3 cells, maps integer cell corners to a fixed-size table of T feature vectors ($F = 2$ dims) via a spatial XOR hash $h(c) = (\bigoplus c_i \pi_i) \bmod T$, interpolates across the 8 corners at each level, concatenates all L levels with $\gamma(\omega)$, and feeds a **tiny 2-layer MLP**. Memory is bounded by $L \cdot T \cdot F$ regardless of spatial resolution; hash collisions are resolved automatically by gradient descent because surface gradients dominate empty space. Trains in **seconds**.

Lecture 9: Neural Implicit Rendering (IDR, VolSDF & NeuS)

Core Concept: NeRF's unconstrained density produces noisy, bumpy surfaces. Enforcing that geometry is the zero level set of an SDF ($f_\theta(x) = 0, \|\nabla f_\theta\| = 1$) gives clean surfaces, rendered either via single-point ray intersection (IDR) or unbiased SDF-to-density volume rendering (VolSDF, NeuS).

- **Pure Surface Rendering (IDR — Yariv et al.):**
  - **Step 1 (Ray-Surface Intersection):** Find the root t^* where $\bar{f}_\theta(t^*) = f_\theta(x_0 + t^* \omega) = 0$ using a hybrid of **sphere tracing** and the **secant method** (since neural f_θ is only approximately an SDF and can overshoot).
  - **Step 2 (Analytic Implicit Differentiation):** We *do not* backpropagate through the iterative sphere-tracing loop! Applying the Implicit Function Theorem to $f_\theta(x_0 + t^*(\theta)\omega) = 0$ yields the exact analytic gradient at the final hit point $p = x_0 + t^* \omega$:

$$\frac{\partial t^*}{\partial \theta} = -\left(\nabla_p f_\theta(p) \cdot \omega\right)^{-1} \frac{\partial f_\theta(p)}{\partial \theta}, \qquad \frac{\partial p}{\partial \theta} = -\omega \left(\nabla_p f_\theta(p) \cdot \omega\right)^{-1} \frac{\partial f_\theta(p)}{\partial \theta}$$

  - **Step 3 (Normal-Aware Appearance & Loss):** Shading network $g_\phi(p, \omega, n)$ must take the surface normal $n = \nabla_p f_\theta(p)$ to model shiny reflections and pass appearance gradients into geometry. Trained with $\mathcal{L}_{\text{photo}} + \lambda_{\text{eik}} \big| \|\nabla f_\theta\| - 1 \big| + \mathcal{L}_{\text{mask}}$.
  - **Limitations of Pure Surface Rendering:** Only updates 1 point per ray; discontinuous at occlusion boundaries/silhouettes ($\nabla_p f \cdot \omega \to 0$); **requires a foreground mask loss** ($\min_t f_\theta(x_0 + t\omega)$) to pull missed rays toward the object.

- **Unifying Surfaces and Volume Rendering (VolSDF & NeuS):**
  - **Idea:** Predict an SDF $d = f_\theta(x)$, convert d into a volume density σ via a smooth analytical formula, and use standard NeRF ray marching (smooth gradients along the entire ray, **no masks required**).

- **VolSDF:** $\sigma(x) = \alpha\Psi_\beta(-d_\Omega(x))$, where Ψ_β is the CDF of a zero-mean Laplace distribution with sharpness β ($\sigma \to \alpha$ inside, 0 outside, $\alpha/2$ on the surface).
- **Why Naive Bell-Shaped SDF-to-Density Fails (Bias):** If you set density to a symmetric bell curve $\sigma(t) = \phi_s(f(p(t)))$ peaking at the surface $f = 0$, transmittance $T(t) = \exp(-\int_0^t \sigma)$ is already decaying as the ray climbs the bell curve. Thus the rendering weight $w(t) = T(t)\sigma(t)$ satisfies $w'(t^*) = -T\sigma^2 < 0$, meaning $w(t)$ **peaks strictly in front of the true surface** (systematic depth bias!).
- **NeuS Unbiased Density:** Designs the opaque density $\rho(t) = \max\left(\frac{-\frac{d\Phi_s}{dt}(f(p(t)))}{\Phi_s(f(p(t)))}, 0\right)$ where $\Phi_s(x) = (1 + e^{-sx})^{-1}$ is the sigmoid and $\phi_s = \Phi_s'$. Then along a planar surface crossing, $T(t) = \Phi_s(f(p(t)))$ and the weight $w(t) = T(t)\rho(t) = |\nabla f \cdot \omega|\, \phi_s(f(p(t)))$ **peaks at the exact zero-crossing** $f = 0$.

Lecture 10: Differentiable Primitive Rendering & 3D Gaussian Splatting (3DGS)

Core Concept: Volume ray marching wastes queries in empty space. Primitive rendering projects explicit primitives onto the screen (rasterization: scatter) and blends softened footprints front-to-back for real-time (>130 FPS) differentiable rendering.

- **Differentiable Mesh Rendering (SoftRas / PyTorch3D):**
  - **Why Hard Mesh Rendering Is Not Differentiable:** Ray-triangle barycentrics (α, β) are differentiable w.r.t. vertices of the visible triangle, but pixel color is a step function at **1) silhouette/occlusion boundaries** (moving a vertex across a pixel center causes a sudden jump) and **2) depth-ordering flips** (z-buffer arg min has zero gradient for hidden triangles).
  - **Two-Part Fix:** 1) *Make triangles soft* in 2D screen space: $m_k = \text{sigmoid}(-\Delta_k/\sigma)$ where Δ_k is signed pixel distance to triangle k's edge ($\Delta < 0$ inside). 2) *Aggregate the K closest triangles* via depth-softmax weights $w_k \propto m_k e^{-\lambda_k/\gamma}$ so partially occluded triangles still receive depth gradients.
  - **Ray Tracing vs. Rasterization Loop Order:** Ray tracing loops 'for pixel: for triangle' ($\Theta(PN)$ tests, redundant for nearby pixels). Rasterization swaps the loops to 'for triangle: project to 2D bbox and update pixel K-buffers', reducing work to $\Theta(N\bar{f} + PK)$.
- **3D Gaussian Splatting (3DGS — Kerbl et al.):**
  - **The Primitive (59 floats):** Mean $\mu \in \mathbb{R}^3$ (3), scale $s \in \mathbb{R}^3$ (3), rotation quaternion $q \in \mathbb{R}^4$ (4), opacity $\alpha \in [0, 1]$ (1), and degree-3 Spherical Harmonics color coefficients ($(3+1)^2 \times 3 = 48$).
  - **Why Factorize Covariance $\Sigma = RSS^\top R^\top$:** Taking raw gradient steps on Σ can yield negative eigenvalues (invalid Gaussian). Factorizing into rotation R and diagonal scale S guarantees $\Sigma = (RS)(RS)^\top$ is **always positive semi-definite**.
  - **EWA Splatting Projection to 2D:** World-to-camera is affine ($\Sigma_c = W\Sigma W^\top$), which keeps Gaussians exact. Perspective division $\pi(X, Y, Z) = (f_x X/Z, f_y Y/Z)$ is nonlinear, so 3DGS linearizes π around the Gaussian mean μ_c using its Jacobian $J = \frac{\partial \pi}{\partial x_c}\big|_{\mu_c}$:

$$\mu_{2D} = \pi(W\mu + t), \qquad \Sigma_{2D} = JW\Sigma W^\top J^\top \in \mathbb{R}^{2\times 2}$$

  - **Fast Tile Rasterizer:** 1) Split screen into 16×16 pixel tiles; instantiate a Gaussian copy for each overlapped tile with a 64-bit key '[tile ID (32b) | depth (32b)]'. 2) Run **one global GPU Radix Sort** to get front-to-back lists for all tiles simultaneously. 3) Blend front-to-back per pixel: $C = \sum_k c_k \bar{\alpha}_k T_k$ with $\bar{\alpha}_k = \alpha_k \exp(-\frac{1}{2} d^\top \Sigma_{2D}^{-1} d)$ (clamped ≤ 0.99, skipped if $< 1/255$, early-stopped when $T < 10^{-4}$).
  - **Backward Pass Trick:** Walks **back-to-front**, recovering $T_i = T_{i+1}/(1 - \bar{\alpha}_i)$ on the fly from the stored final transmittance T_{final} in $\mathcal{O}(1)$ memory per pixel: $\frac{\partial C}{\partial \bar{\alpha}_i} = T_i c_i - \frac{1}{1-\bar{\alpha}_i} \sum_{j>i} c_j \bar{\alpha}_j T_j$.

- **Initialization & Adaptive Density Control:** Initialized from sparse SfM points (random init degrades background). Every 100 steps, Gaussians with large view-space positional gradients (meaning they fail to fit local geometry) are either **Cloned** (if small: under-reconstruction) or **Split** with scale $\div 1.6$ (if large: over-reconstruction). Low-opacity ($\alpha < 0.005$) Gaussians are pruned, and α is periodically reset to remove floaters. Anisotropy is crucial (isotropic balls fail on thin leaves/spokes).
- **Limitations & 2DGS:** 3DGS has high memory ($\sim$ 700 MB), **popping artifacts** (sorts once per Gaussian center rather than per pixel), and lacks exact surface normals because 3D ellipsoids have view-dependent cross-sections. **2D Gaussian Splatting (2DGS)** flattens primitives into planar 2D disks ($S = \text{diag}(s_u, s_v, 0)$) with exact normals and closed-form ray-plane intersection.

Lecture 11: Relightable 3D Representations (Inverse Rendering)

Core Concept: Standard NeRF and 3DGS bake capture lighting into outgoing color $g_\phi(x, \omega)$ and **cannot be relit**. Relighting requires inverting the Rendering Equation to disentangle *Geometry*, *Material (BRDF)*, and *Light*.

- **The Rendering Equation & The Fundamental Ambiguity:**

$$L_o(x, \omega_o) = \int_\Omega \underbrace{f_r(x, \omega_i, \omega_o)}_{\text{Material (BRDF)}} \underbrace{L_i(x, \omega_i)}_{\text{Incident Light}} \underbrace{(n^\top \omega_i)}_{\text{Geometry}} \, d\omega_i$$

  Why Inverse Rendering Fails Unconstrained: A capture under one illumination only observes the product Material $\times$ Light. Many factorizations produce identical training pixels (e.g., red paint under white light = white paint under red light; or baking all shadows and shading directly into the diffuse albedo $a(x)$ under a flat white environment light) yet fail catastrophically under a new light.

- **Remedy 1 — Constrain the Physical Model (NeRS, PhySG, IRGS):**
  - **Light & Material Constraints:** Assume distant environment light $L_i(x, \omega) \approx L_e(\omega)$ (often monochrome/gray to lock chromaticity into albedo) and split BRDF into spatially varying diffuse albedo $a(x)/\pi$ plus a low-parameter shared specular roughness lobe $f_s(\omega_i, \omega_o; \rho)$.
  - **PhySG (Closed-Form Integration):** Represents the environment map, specular BRDF lobe, and cosine term as **Spherical Gaussians (SGs)** $G(\omega) = \mu e^{\lambda(\xi^\top \omega - 1)}$. Because the product of two SGs is another SG and the integral of an SG over the sphere has a closed form $\mu \frac{2\pi}{\lambda}(1 - e^{-2\lambda})$, PhySG integrates the rendering equation in closed form without Monte Carlo sampling. **Flaw:** Ignores visibility $V(x, \omega_i)$, so cast shadows get baked into dark albedo patches!
  - **IRGS (Relightable 2D Gaussians with Shadows):** Stores normal, albedo, and roughness on 2D Gaussians. First splats/blends attributes into a per-pixel G-buffer $(\bar{n}, \bar{a}, \bar{r}, \bar{d})$ (*blend first, then shade once per pixel*), then traces 256 shadow rays per pixel through the Gaussians to obtain visibility $V(x, \omega_j) = \prod(1 - \alpha_i)$ and 1-bounce indirect color $L_{\text{ind}}(x, \omega_j)$.
- **Remedy 2 — Capture More Informative Data (Light Stage / OLAT):**
  - **Linearity of Light Transport:** Photons do not interact, so $I(c_A A + c_B B) = c_A I_A + c_B I_B$ (in linear HDR space). Capturing **One Light at a Time (OLAT)** images I_k builds the light-transport matrix $T = [I_1 \mid \cdots \mid I_K]$; relighting under any new environment ℓ is a simple weighted sum $I_{\text{new}} = \sum_k \ell_k I_k$ with **zero BRDF or geometry estimation** (requires a static subject in a light stage).
- **Remedy 3 — Learned Data Priors (MaterialFusion, Neural Gaffer):**
  - **MaterialFusion:** Adds a diffusion prior loss $\gamma \mathcal{L}_{\text{prior}}(\text{albedo}_\theta, \text{roughness}_\theta)$ inside per-scene physical optimization to penalize baked shading in the albedo.
  - **Neural Gaffer:** Diffusion U-Net that relights 2D images directly without explicit BRDF/-geometry. Crucially, the target HDR environment map is **rotated into the camera coordinate frame** (giving every pixel a consistent directional meaning) and fed as both an LDR tone-mapped copy (visible look) and a log-normalized copy (high dynamic range peaks).

Lecture 12: Gaussian-Based Active Planning & Active Mapping

Core Concept: In Active Perception, the robot chooses its next camera pose T_{t+1} to maximize task information gain under a view budget. Replacing each 3D Gaussian's RGB color c_k with a task utility vector s_k turns the fast 3DGS rasterizer into an **occlusion-aware viewpoint scoring engine**: $g(T, \tau) = \sum_{u \in \Omega} \sum_k w_k(u; T)\, s_k(\tau, T)$.

- **Coverage-Driven Baseline (MAGICIAN):**
  - Predicts unobserved geometry with a 3D Volume Occupancy Network to place **Imagined Gaussians** where opacity $\alpha_k = \hat{p}_{\text{occ}}$ and color c_k = novelty $\in \{0, 1\}$ (1 = unseen, 0 = seen), then plans trajectories via Beam Search.
  - **Flaws for Robotics:** 1) *Single-glance saturation:* Seeing an object once from 6 m away sets novelty to 0, leaving torn, hole-ridden geometry. 2) *No semantics:* Wastes budget mapping blank walls instead of tabletop objects.
- **Semantic Object-Centric Dual-Map Architecture:**
  - **1) Online Gaussian Map (Final Output & Renderer):** Fuses 2D OneFormer semantic predictions using a memory-efficient **4-Slot Depth-Gated Voting** scheme. A Gaussian g_k only receives a semantic vote at pixel u if it passes the **Visible Surface Gate** $|z_k(T_t) - D_t(u)| \leq \delta_{\text{surf}}$ (prevents labels from bleeding onto occluded walls behind foreground objects).
  - **2) 5 cm Voxel Planning State (Built from Depth Alone):** Tracks 'free / occupied / unknown' space (for collision-free flight and **pruning mid-air Gaussian floaters**), frontier flags, closest viewing distance z_v, 24-bit directional bitmasks $(b_{\text{seen}}, b_{\text{feas}})$, and majority-voted Gaussian semantic class $c(v)$.
- **Multi-Signal Channel Rendering & Horizon-5 Beam Search:**
  - **Three Voxel Signals Painted onto Gaussians:** 1) **Discovery ($s_{\mathbf{disc}}$):** Active on frontier voxels viewed from their open side (higher when closer). 2) **Coverage ($s_{\mathbf{cov}}$):** Active on surface voxels seen from < 2 directions when viewed from a new feasible sector. 3) **Objects ($s_{\mathbf{obj}}$):** Active on unretired foreground objects ($c(v) \in \mathcal{C}_{\text{obj}}$); rewards each **halving of viewing distance** down to 1 m (4 m $\rightarrow$ 2 m $\rightarrow$ 1 m) plus new viewing sectors until the object is **retired** ($s_{\text{obj}} = 0$).
  - **Beam Search ($W = 5$ sequences, $H = 5$ steps ahead):** Expands neighbor poses, renders the multi-channel Gaussian utility map, discounts surfaces already seen earlier in the candidate trajectory to avoid double-counting, and executes the first step of the best sequence (receding horizon).
  - **Task Modularity (Object Navigation):** Swapping the painted channels to (*Frontier*, *Out-of-sight Doorway Relay*, and *VLM Target Bounding Box*) repurposes the exact same renderer and beam-search planner for zero-shot object navigation.